FUNDAMENTOS DE PROGRAMACIÓN

ALGORITMOS Y ESTRUCTURA DE DATOS

Luis Joyanes Aguilar

SEGUNDA EDICIÓN

Incluye guías de sintaxis: Pascal, Turbo/Borland Pascal, C y C++

McGraw Hill

MiMiQ

FUNDAMENTOS DE PROGRAMACIÓN

Algoritmos y estructuras de datos

Segunda edición

CONSULTORES EDITORIALES
ÁREA DE INFORMÁTICA Y COMPUTACIÓN

Antonio Vaquero Sánchez

Catedrático de Lenguajes y Sistemas Informáticos
Escuela Superior de Informática
Universidad Complutense de Madrid
ESPAÑA

Gerardo Quiroz Vieyra

Ingeniero en Comunicaciones y Electrónica
por la ESIME del Instituto Politécnico Nacional
Profesor de la Universidad Autónoma Metropolitana
Unidad Xochimilco
MÉXICO

Willy Vega Gálvez

Universidad Nacional de Ingeniería
PERÚ

FUNDAMENTOS DE PROGRAMACIÓN

Algoritmos y estructuras de datos

Segunda edición

Luis Joyanes Aguilar

Departamento de Lenguajes y Sistemas Informáticos
Facultad de Informática
Universidad Pontificia de Salamanca en Madrid

McGraw-Hill

MADRID • BUENOS AIRES • CARACAS • GUATEMALA • LISBOA • MÉXICO
NUEVA YORK • PANAMÁ • SAN JUAN • SANTAFÉ DE BOGOTÁ • SANTIAGO • SÃO PAULO
AUCKLAND • HAMBURGO • LONDRES • MILÁN • MONTREAL • NUEVA DELHI • PARÍS
SAN FRANCISCO • SIDNEY • SINGAPUR • ST. LOUIS • TOKIO • TORONTO

FUNDAMENTOS DE PROGRAMACIÓN
Algoritmos y estructuras de datos. Segunda edición

No está permitida la reproducción total o parcial de este libro, ni su tratamiento informático, ni la transmisión de ninguna forma o por cualquier medio, ya sea electrónico, mecánico, por fotocopia, por registro u otros métodos, sin el permiso previo y por escrito de los titulares del Copyright.

DERECHOS RESERVADOS © 1996, respecto a la segunda edición en español, por
McGRAW-HILL/INTERAMERICANA DE ESPAÑA, S. A.
Edificio Valrealty, 1.ª planta
Basauri, 17
28023 Aravaca (Madrid)

ISBN: 84-481-0603-2
Depósito legal: M. 30.502-1996

Editora: Concepción Fernández Madrid
Diseño cubierta: Juan García
Compuesto en: Folio Blanco, S. L.
Impreso en: Lavel, S. L.

IMPRESO EN COLOMBIA PRINTED IN COLOMBIA

Se imprimieron 5900 ejemplares en el mes de febrero de 1998
Impreso y encuadernado por: Lito Camargo Ltda.

CONTENIDO

Prólogo de la segunda edición XIII

Capítulo 1. Algoritmos y programas 1

1.1. Los sistemas de procesamiento de la información 2
1.2. Concepto de algoritmo 4
1.2.1. Características de los algoritmos 5
1.3. Los lenguajes de programación 6
1.3.1. Instrucciones a la computadora 7
1.3.2. Lenguajes máquina 7
1.3.3. Lenguajes de bajo nivel 8
1.3.4. Lenguajes de alto nivel 9
1.3.5. Traductores de lenguaje 10
1.3.5.1. Intérpretes 10
1.3.5.2. Compiladores 11
1.3.6. La compilación y sus fases 11
1.4. Datos, tipos de datos y operaciones primitivas 13
1.4.1. Datos numéricos 13
1.4.2. Datos lógicos (booleanos) 15
1.4.3. Datos tipo carácter y tipo cadena 15
1.5. Constantes y variables 16
1.6. Expresiones 17
1.6.1. Expresiones aritméticas 18
1.6.1.1. Reglas de prioridad 20
1.6.2. Expresiones lógicas (booleanas) 22
1.6.2.1. Operadores de relación 22
1.6.2.2. Operadores lógicos 24
1.6.2.3. Prioridad de los operadores en las expresiones lógicas 25
1.7. Funciones internas 26
1.8. La operación de asignación 27
1.8.1. Asignación aritmética 28
1.8.2. Asignación lógica 29
1.8.3. Asignación de cadenas de caracteres 29
1.8.4. Conversión de tipo 29

1.9. Entrada y salida de información 31
Actividades de programación resueltas 31
Ejercicios 37

Capítulo 2. La resolución de problemas con computadoras y las herramientas de programación 39

2.1. La resolución de problemas 40
2.2. Análisis del problema 40
2.3. Diseño del algoritmo 41
2.3.1. Escritura inicial del algoritmo 43
2.4. Resolución del problema mediante computadora 45
2.5. Representación gráfica de los algoritmos 46
2.5.1. Diagramas de flujo 46
2.6. Diagramas de Nassi-Schneiderman (N-S) 56
2.7. Pseudocódigo 58
Actividades de programación resueltas 59
Ejercicios 64

Capítulo 3. Estructura general de un programa 67

3.1. Concepto de programa 67
3.2. Partes constitutivas de un programa 68
3.3. Instrucciones y tipos de instrucciones 69
3.4. Tipos de instrucciones 70
3.4.1. Instrucciones de asignación 70
3.4.2. Instrucciones de lectura de datos (entrada) 71
3.4.3. Instrucciones de escritura de resultados (salida) 72
3.4.4. Instrucciones de bifurcación 72
3.5. Elementos básicos de un programa 73
3.5.1. Bucles 74
3.5.2. Contadores 76
3.5.3. Acumulador 79
3.5.4. Decisión o selección 80
3.5.5. Interruptores 81
3.6. Escritura de algoritmos/programas 84
3.6.1. Cabecera del programa o algoritmo 84
3.6.2. Declaración de variables 84
3.6.3. Declaración de constantes numéricas 85
3.6.4. Declaración de constantes y variables carácter 85
3.6.5. Comentarios 86
3.6.6. Estilo de escritura de algoritmos/programas 87
Actividades de programación resueltas 88
Ejercicios 93

Capítulo 4. Introducción a la programación estructurada 95

4.1. Técnicas de programación 96
4.2. Programación modular 96
4.2.1. Tamaño de los módulos 97
4.2.2. Implementación de los módulos 98
4.3. Programación estructurada 99
4.3.1. Recursos abstractos 99

4.3.2. Diseño descendente *(top-down)* 99
4.3.3. Teorema de la programación estructurada: estructuras básicas 100
4.4. Estructura secuencial 100
4.5. Estructuras selectivas 104
4.5.1. Alternativa simple (`si-entonces / if-then`) 104
4.5.2. Alternativa doble (`si-entonces-si_no / if-then-else`) 105
4.5.3. Alternativa múltiple (`segun_sea, caso_de / case`) 111
4.6. Estructuras repetitivas 117
4.6.1. Estructura `mientras («while»)` 120
4.6.2. Estructura `repetir («repeat»)` 125
4.6.3. Estructura `desde/para («for»)` 128
4.6.4. Salidas internas de los bucles 132
4.7. Estructuras de decisión anidadas 138
4.8. Estructuras repetitivas anidadas 142
4.9. La instrucción `ir_a («goto»)` 144
Actividades de programación resueltas 145
Ejercicios 160

Capítulo 5. Subprogramas (subalgoritmos): procedimientos y funciones 163

5.1. Introducción a los subalgoritmos o subprogramas 164
5.2. Funciones 166
5.2.1. Declaración de funciones 167
5.2.2. Invocación a las funciones 168
5.3. Procedimientos (subrutinas) 173
5.3.1. Sustitución de argumentos/parámetros 174
5.4. Ambito: variables locales y globales 178
5.5. Comunicación con subprogramas: paso de parámetros 181
5.5.1. Paso de parámetros 182
5.5.2. Paso por valor 183
5.5.3. Paso por referencia 184
5.5.4. Comparaciones de los métodos de paso de parámetros 185
5.5.5. Síntesis de la transmisión de parámetros 187
5.6. Funciones y procedimientos como parámetros 190
5.7. Los efectos laterales 192
5.7.1. En procedimientos 192
5.7.2. En funciones 193
5.8. Recursión (recursividad) 194
Actividades de programación resueltas 197
Ejercicios 202

Capítulo 6. Estructuras de datos (arrays) 203

6.1. Introducción a las estructuras de datos 204
6.2. Arrays unidimensionales: los vectores 205
6.3. Operaciones con vectores 208
6.3.1. Asignación 209
6.3.2. Lectura/escritura de datos 210
6.3.3. Acceso secuencial al vector (recorrido) 210
6.3.4. Actualización de un vector 212
6.4. Arrays de varias dimensiones 215
6.4.1. Arrays bidimensionales (tablas/matrices) 215
6.5. Arrays multidimensionales 218

6.6. Almacenamiento de arrays en memoria ... 221
6.6.1. Almacenamiento de un vector ... 221
6.6.2. Almacenamiento de arrays multidimensionales ... 222
Actividades de programación resueltas ... 224
Ejercicios ... 230

Capítulo 7. Las cadenas de caracteres ... 237

7.1. Introducción ... 238
7.2. El juego de caracteres ... 238
7.2.1. Código ASCII ... 238
7.2.2. Código EBCDIC ... 240
7.3. Cadena de caracteres ... 240
7.4. Datos tipo carácter ... 242
7.4.1. Constantes ... 242
7.4.2. Variables ... 242
7.4.3. Instrucciones básicas con cadenas ... 243
7.5. Operaciones con cadenas ... 244
7.5.1. Cálculo de la longitud de una cadena ... 244
7.5.2. Comparación ... 245
7.5.3. Concatenación ... 246
7.5.4. Subcadenas ... 247
7.5.5. Búsqueda ... 248
7.6. Otras funciones de cadenas ... 249
7.6.1. Insertar ... 249
7.6.2. Borrar ... 250
7.6.3. Cambiar ... 250
7.6.4. Conversión cadenas/números ... 251
Actividades de programación resueltas ... 252
Ejercicios ... 257

Capítulo 8. Archivos (ficheros) ... 259

8.1. Noción de archivo: estructura jerárquica ... 260
8.1.1. Campos ... 260
8.1.2. Registros ... 261
8.1.3. Archivos (ficheros) ... 261
8.1.4. Bases de datos ... 262
8.1.5. Estructura jerárquica ... 262
8.2. Conceptos y definiciones: terminología ... 262
8.2.1. Clave (indicativo) ... 262
8.2.2. Registro físico o bloque ... 262
8.2.3. Factor de bloqueo ... 263
8.3. Soportes secuenciales y direccionables ... 265
8.4. Organización de archivos ... 265
8.4.1. Organización secuencial ... 266
8.4.2. Organización directa ... 267
8.4.3. Organización secuencial indexada ... 268
8.5. Operaciones sobre archivos ... 270
8.5.1. Creación de un archivo ... 270
8.5.2. Consulta de un archivo ... 271
8.5.3. Actualización de un archivo ... 272
8.5.4. Clasificación de un archivo ... 273

8.5.5. Reorganización de un archivo 273
8.5.6. Destrucción de un archivo 273
8.5.7. Reunión, fusión de un archivo 274
8.5.8. Rotura/estallido de un archivo 274
8.6. Gestión de archivos 274
8.6.1. Crear un archivo 275
8.6.2. Abrir un archivo 276
8.6.3. Cerrar archivos 278
8.6.4. Borrar archivos 278
8.7. Mantenimiento de archivos 278
8.7.1. Operaciones sobre registros 280
8.8. Procesamiento de archivos secuenciales (algoritmos) 280
8.8.1. Creación 281
8.8.2. Consulta 281
8.8.3. Actualización 284
8.9. Archivos de texto 287
8.10. Procesamiento de archivos directos (algoritmos) 287
8.10.1. Operaciones con archivos de acceso directo 287
8.10.1.1. Creación 287
8.10.1.2. Altas 289
8.10.1.3. Consulta 290
8.10.1.4. Bajas 292
8.10.1.5. Modificaciones. 293
8.10.2. Clave-dirección 294
8.10.3. Tratamiento de las colisiones 294
8.10.4. Acceso a los archivos directos mediante indexación 294
8.10.4.1. Consulta 295
8.11. Procesamiento de archivos secuenciales indexados 296
Actividades de programación resueltas 297
Ejercicios 304

Capítulo 9. Ordenación, búsqueda e intercalación 305

9.1. Introducción 306
9.2. Ordenación 307
9.2.1. Método de intercambio o de burbuja 308
9.2.2. Ordenación por inserción 314
9.2.3. Ordenación por selección 317
9.2.4. Método de Shell 320
9.2.5. Método de ordenación rápida (*quicksort*) 322
9.3. Búsqueda 327
9.3.1. Búsqueda secuencial 327
9.3.2. Búsqueda binaria 332
9.3.3. Búsqueda mediante transformación de claves (*hashing*) 337
9.3.3.1. Métodos de transformación de claves 338
9.3.3.2. Colisiones 341
9.4. Intercalación 343
Actividades de programación resueltas 345
Ejercicios 358

Capítulo 10. Ordenación, búsqueda y fusión externa (archivos) 359

10.1. Introducción 360
10.2. Archivos ordenados 360

10.3. Fusión de archivos 361
10.4. Partición de archivos 364
10.4.1. Clasificación interna 365
10.4.2. Partición por contenido 365
10.4.3. Selección por sustitución 366
10.4.4. Partición por secuencias 368
10.5. Clasificación de archivos 369
10.5.1. Clasificación por mezcla directa 369
10.5.2. Clasificación por mezcla natural 372
10.5.3. Clasificación por mezcla de secuencias equilibradas 376
Actividades de programación resueltas 377
Ejercicios 383

Capítulo 11. Estructuras dinámicas lineales de datos (pilas, colas y listas enlazadas) 385

11.1. Introducción a las estructuras lineales de datos 386
11.1.1. Estructuras dinámicas de datos 387
11.2. Listas 388
11.3. Listas enlazadas 390
11.4. Procesamiento de listas enlazadas 393
11.4.1. Implementación de listas enlazadas con punteros 393
11.4.1.1. Creación de la lista 395
11.4.1.2. Inserción de un elemento 396
11.4.1.3. Eliminación de un elemento de una lista enlazada 397
11.4.1.4. Recorrido de una lista enlazada 398
11.4.1.5. Acceso a un elemento de una lista enlazada 398
11.4.2. Implementación de listas enlazadas con arrays 400
11.4.2.1. Creación de la lista 402
11.4.2.2. Inserción de un elemento 403
11.4.2.3. Eliminación de un elemento 405
11.4.2.4. Recorrido de una lista 406
11.4.2.5. Búsqueda de un determinado elemento en una lista 406
11.4.2.6. Funciones 407
11.5. Listas circulares 407
11.6. Listas doblemente enlazadas 408
11.6.1. Inserción 409
11.6.2. Eliminación 410
11.7. Pilas 411
11.7.1. Aplicaciones de las pilas 417
11.8. Colas 419
11.8.1. Representación de las colas 420
11.8.2. Aprovechamiento de la memoria 426
11.9. Doble cola 428
Actividades de programación resueltas 428
Ejercicios 436

Capítulo 12. Estructura de datos no lineales (árboles y grafos) 439

12.1. Introducción 440
12.2. Árboles 440
12.2.1. Terminología y representación de un árbol general 441
12.3. Árbol binario 442
12.3.1. Terminología de los árboles binarios 443

12.3.2. Árboles binarios completos 444
12.3.3. Conversión de un árbol general en árbol binario 445
12.3.4. Representación de los árboles binarios 449
12.3.4.1. Representación por punteros 449
12.3.4.2. Representación por listas enlazadas 450
12.3.4.3. Representación por arrays 451
12.3.5. Recorrido de un árbol binario 453
12.4. Árbol binario de búsqueda 455
12.4.1. Búsqueda de un elemento 457
12.4.2. Insertar un elemento 458
12.4.3. Eliminación de un elemento 459
12.4.4. Árbol binario mediante arrays 461
12.5. Grafos 466
12.5.1. Terminología de grafos 467
12.5.2. Representación de grafos 470
12.5.2.1. Matriz de adyacencia 470
12.5.2.2. Lista de adyacencia 472
Actividades de programación resueltas 473
Ejercicios 477

Capítulo 13. Diseño efectivo de programas y aplicaciones algorítmicas 481

13.1. Estilo de programación 482
13.1.1. Nombres de variables 483
13.1.2. Comentarios 484
13.2. Estructura del programa 484
13.3. Estructura modular y diseño descendente 484
13.3.1. Programación modular 485
13.4. Diseño de programas 486
13.4.1. Codificación 486
13.4.2. Ejecución de programas 486
13.4.3. Puesta a punto de programas 486
13.4.4. Manipulación de entrada/salida 488
13.4.5. Escritura de programas: la documentación 489
13.4.6. Documentación interna 489
13.4.7. Documentación externa 493
13.5. Aplicaciones de gestión (análisis) 494
13.5.1. Análisis de oportunidad 494
13.5.2. Análisis funcional 495
13.5.3. Análisis orgánico 495
13.5.4. Explotación 486
Ejercicios 496

Capítulo 14. Tablas de decisión 497

14.1. Introducción a las tablas de decisión 498
14.2. Definición de una Tabla de Decisión (TD) 498
14.2.1. Reglas de decisión 499
14.2.2. Modos de representación de las tablas de decisión 499
14.3. Tipos de tablas de decisión 504
14.3.1. Tablas de decisión limitadas 504
14.3.2. Tablas de decisión de entradas ampliadas o extendidas 505

14.3.3. Tablas de decisión mixtas 506
14.3.4. Comparación y transformación de los tipos de tablas 506
14.4. Tipos de reglas de decisión 507
14.4.1. Conversión de reglas <<O>> y <<SI_NO>> 508
14.5. Construcción de tablas de decisión 509
14.5.1. Estudio de condiciones, acciones y relaciones 509
14.5.2. Análisis y requisitos de una TD 511
14.5.3. Simplificación de una tabla de decisión 511
14.6. Encadenamiento de tablas de decisión 513
14.6.1. Tablas abiertas 513
14.6.2. Tablas cerradas 514
14.7. Conversión de tablas de decisión en programas 516
14.7.1. Programación directa en lenguajes de alto nivel 516
14.7.2. Transformación de la tabla en diagrama de flujo 516
Actividades de programación resueltas 522
Ejercicios 530

Apéndice A. Especificaciones del lenguaje algorítmico UPSAM 533
Apéndice B. Guía del usuario ISO/ANSI Pascal estándar 545
Apéndice C. Guía de sintaxis Turbo/Borland Pascal 7.0 583
Apéndice D. Guía de referencia lenguaje C ANSI 607
Apéndice E. Guía de sintaxis del lenguaje C++ (estándar C++ANSI) 631
Apéndice F. Codificación de algoritmos en lenguajes de programación 667
Apéndice G. Glosario de programación 687
Apéndice H. Códigos ASCII 695

Bibliografía 701

Índice 707

PRÓLOGO DE LA SEGUNDA EDICIÓN

El primer curso de programación de un *currículum* de Ciencias e Ingeniería, es probablemente uno de los más importantes. Para la mayoría de los estudiantes, el conocimiento inicial eficiente de conceptos básicos tales como algoritmos y estructura de datos, así como el modo de aprender a resolver problemas mediante computadoras, suele ser vital debido a la trascendencia que un aprendizaje gradual y correcto supondrá para su carrera y sobre todo para sus primeros cursos en la universidad (facultades, escuelas o institutos tecnológicos en Latinoamérica). Por esta causa fundamental la mayoría de los *currícula* incluyen asignaturas de carácter troncal u obligatorio tales como ***Metodología de la Programación, Introducción a la Programación o Fundamentos de Programación,*** cuyo objetivo es enseñar a los alumnos los conceptos fundamentales de la programación, especialmente la *programación estructurada*, que les permita adquirir una orientación específica, básica y fundamental y que les garantice afrontar con éxito el estudio de las restantes asignaturas del *currículum* tales como Tecnología de la Programación, Sistemas Operativos, Programación Avanzada, Estructuras de Datos, etc. o puramente científicas como Cálculo Numérico, Algoritmia Avanzada, etc.

Para tratar de conseguir los objetivos citados, decidimos escribir y publicamos en 1988 la primera edición del libro ***Fundamentos de programación. Algoritmos y estructura de datos***, que fue la continuación de uno anterior —***Metodología de la programación***— que se publicó en 1986 y que se centró fundamentalmente en el diseño básico de algoritmos. Así en esta obra nos centramos en los conceptos y diseño de algoritmos junto con los fundamentos teórico-prácticos de estructuras de datos y de programación estructurada; por esta razón nos apoyamos en herramientas clásicas de programación tales como diagramas de flujo, diagramas N-S y sobre todo pseudocódigo. En aquellos años, el uso del pseudocódigo en inglés estaba extendido, pero no así el pseudocódigo en español, por lo que pensamos que sería muy interesante el empleo del español como soporte del pseudocódigo, de modo que los lectores de nuestro idioma pudieran introducirse de un modo más fácil en el aprendizaje de la programación. Desde entonces, no sólo nuestras obras sino muchas otras han comenzado a utilizar el pseudocódigo en español y, tanto en España como en Latinoamérica, ha pasado a ser una herramienta decisiva empleada no sólo en los cursos de introducción a la programación sino en cursos avanzados y en otros similares.

Estos hechos, junto con las numerosas sugerencias, recomendaciones, consejos y críticas de colegas y alumnos españoles y latinoamericanos, nos ha llevado a escribir la segunda edición de

este libro, apoyándonos fundamentalmente en el pseudocódigo como lenguaje algorítmico por excelencia.

En estos ocho años transcurridos desde la primera edición, el libro ha sido utilizado en numerosas universidades, escuelas de ingenieros e institutos tecnológicos, a la vez que la disciplina informática —en general— y la programación —en particular— han continuado evolucionando y madurando. Nuevos conceptos y nuevos lenguajes de programación han conducido a nuevas técnicas de programación y al diseño de nuevos *currícula* en carreras de Ingeniería Informática y en Sistemas Computacionales así como en Licenciatura y Diplomatura e Ingeniería Técnica Informática. A mediados de los años ochenta tres lenguajes de programación predominaban en los ambientes informáticos: BASIC, Pascal y COBOL, junto con el clásico FORTRAN y los lenguajes emergentes C, Modula-2 y Ada. En la actualidad, Pascal y COBOL siguen siendo clásicos como primer lenguaje de programación y como lenguaje profesional, por excelencia, manteniéndose el clásico FORTRAN para aplicaciones científicas, sobre todo ya desarrolladas, y están implantándose cada vez con mayor intensidad lenguajes tales como C/C++ y Ada-95, comenzando a adquirir una notable influencia otros lenguajes como Visual BASIC o Delphi.

El uso de un lenguaje de programación, claramente, es una parte importante de cualquier curso de introducción a la programación; pero este hecho no debe oscurecer otros conceptos importantes. Así, este libro se basa en la premisa de que el estudiante/lector debe aprender a programar con la filosofía de usar la sintaxis y estructura de un lenguaje de programación pero no de *un lenguaje de programación específico*. Por esta causa y teniendo presente la experiencia con gran número de alumnos de nuestra universidad y de las muchas otras universidades donde se ha utilizado la primera edición como texto basico, hemos decidido utilizar como lenguaje de programación algorítmico el pseudocódigo en español o castellano **UPSAM** (Universidad Pontificia de Salamanca en Madrid) fruto de la experiencia acumulada en el Departamento de Lenguajes y Sistemas Informáticos e Ingeniería de Software de la Facultad y Escuela Universitaria de Informática de la Universidad Pontificia de Salamanca en Madrid, durante los diez últimos años.

EL LENGUAJE ALGORÍTMICO DE PROGRAMACIÓN UPSAM

El lenguaje algorítmico **UPSAM** o pseudocódigo español utilizado como referencia básica en el texto ha sido creado, diseñado y experimentado por todos los profesores que han impartido las asignaturas de ***Fundamentos de Programación*** en las carreras universitarias *Técnicos en Informática Fundamental (TIF), Diplomados en Informática y Licenciados en Informática*, y ***Metodología de la Programación*** en los cursos de postgrado *Master*, *Técnicos* y *Expertos en Informática* de la Universidad Pontificia de Salamanca en Madrid y en su Departamento de Lenguajes y Sistemas Informáticos e Ingeniería de Software.

La guía de sintaxis del lenguaje algorítmico **UPSAM** se describe en el Apéndice A. Ha sido redactada por los profesores Matilde Fernández, Luis Rodríguez y Luis Joyanes, y revisada y corregida por los profesores Luis Villar, Joaquín Abeger, Ángel Hermoso, Antonio Muñoz, Lucas Sánchez, Víctor Martín y Rafael Ojeda, que han aportado sugerencias y mejoras al borrador original. Asimismo también han contribuido a la redacción final con sugerencias y aportaciones y pertenecen al equipo creador del lenguaje **UPSAM** los siguientes profesores de la citada Universidad:

- Ángela Carrasco
- Francisco Oliva
- Isabel Torralvo

- M.ª del Mar García
- Mercedes Vargas
- Miguel Sánchez

Profesores de asignaturas tales como Programación I y II, Estructura de Datos, Programación de Sistemas, Metodología de la Programación, Programación de Gestión, Sistemas Operativos y otras han utilizado el pseudocódigo UPSAM en una u otra forma y han dado sugerencias también que se han incorporado a la versión final. Así y con el riesgo de olvidar algún nombre de algún colega y amigo —a quien pedimos disculpas por anticipado—, destacamos a:

- Amalia Oñate
- Andrés Castillo
- Ángel González
- Antonio Reus
- Daniel García
- David Juanes
- Enrique Torres
- Fernando Davara
- Francisco Agüera
- Francisco Mata
- Jesús Pérez
- José González
- José Luis Andrés
- Juan Ignacio Pérez
- Juan Manuel Dodero
- Juan Sanz
- Julián Margolles
- Luis Doreste
- M.ª Luisa Díez
- Manuel Redón
- Manuel Riquelme
- Paloma Centenera
- Rosa Hernández
- Virgilio Yagüe

PROPÓSITO DEL LIBRO

El propósito principal de este libro es servir como material de lectura y estudio para un curso de introducción a la programación en una carrera universitaria de Ciencias e Ingeniería, preferentemente *Ciencias de la Computación, Ingeniería Informática o en Sistemas Computacionales, Diplomatura y Licenciatura en Informática.*

El libro sigue las directrices fundamentales de asignaturas troncales de los planes de estudio españoles ***Introducción a la programación y Metodología de la Programación***, así como los temas principales de los cursos **CS1** y **CS2** de la **ACM** (Association for Computing Machinery).

Se siguen las reglas clásicas de construcción de diseño de programas estructurados, con énfasis especial en algoritmos y estructuras de datos. El libro ha conservado la organización de la primera

edición, incluyendo gran cantidad de ejemplos y ejerecicios resueltos con el objeto de que el alumno pueda comprobar su progreso. Incluye además en cada capítulo Actividades de Programación que contiene numerosos ejercicios y problemas resueltos de complejidad creciente.

La actualización y puesta al día de los nuevos lenguajes de programación nos ha hecho incluir guías de sintaxis de los lenguajes Pascal, Turbo Pascal, C y C++, así como reglas de conversión del lenguaje algorítmico a los citados lenguajes, de modo que el lector pueda utilizar a la terminación de la lectura del libro o en paralelo, alguno de los lenguajes anteriores y construir programas en los mismos sin ninguna dificultad.

Así, se han cuidado especialmente los siguientes tópicos:

1. Escribir código modular.
2. Utilizar abstracción en el proceso de desarrollo de software.
3. Escritura de documentación del código fuente.
4. Legibilidad y claridad en el código fuente.
5. Semántica consistente para todos los procesos de un programa.
6. Utilizar estrictamente las reglas de la programación estructurada.

LIBRO COMPLEMENTARIO

Todos los cursos de programación deben apoyarse en la resolución de gran número de problemas que permitan al alumno/lector adquirir práctica que le faciliten el aprendizaje. Por esta causa se han incluido gran cantidad de ejercicios, problemas y actividades de programación en el texto. Además, nuestra experiencia docente y la petición de numerosos alumnos y colegas, nos llevó a considerar escribir un libro de problemas que resolviera un buen número de ejercicios propuestos y otros nuevos, de modo que el alumno pudiese comprobar sus avances no sólo a través de libro con actividades de programación, sino con ayuda de un libro clásico de problemas pero con la aportación y experiencia metodológica de nuestra universidad. Por esta causa mis colegas Matilde Fernández, Luis Rodríguez y yo mismo, hemos escrito el libro ***Fundamentos de Programación. Libro de problemas***, también publicado en McGraw-Hill, que puede ser utilizado de modo complementario a este libro o de modo totalmente independiente ya que ha sido diseñado explícitamente para cumplir ambos requisitos y especialmente para facilitar el aprendizaje de la programación.

ORGANIZACIÓN DEL LIBRO

El libro se ha estructurado en catorce capítulos y ocho apéndices con una organización similar en todos ellos que incluyen desde conceptos teóricos a reglas y conceptos prácticos junto con actividades de programación y ejercicios propuestos al lector para que tras su realización éste pueda comprobar y medir su progreso en el mundo de la programación.

El Capítulo 1 presenta una breve descripción del concepto de sistema de procesamiento de información. Introduce al concepto de algoritmo y las primeras construcciones y elementos de un lenguaje de programación. Se describen los tipos de datos y las operaciones básicas que se realizan en cualquier programa.

El Capítulo 2 analiza los métodos para resolver problemas con computadoras y las herramientas de programación necesarias para ello. Se describen las etapas clásicas utilizadas en la resolución de

problemas así como las herramientas clásicas utilizadas para su descripción, tales como pseudocódigos, diagramas de flujo y diagramas N-S.

En el Capítulo 3 se describe el concepto de programa y sus elementos básicos, junto con las instrucciones y tipos de instrucciones más usuales. Se introduce el concepto de *flujo de control* y se incluyen los primeros problemas planteados de cierta complejidad y su resolución con alguna de las herramientas descritas en el Capítulo 2.

La programación estructurada y sus técnicas es uno de los conceptos claves que se describen y utilizan a lo largo del texto. El Capítulo 4 amplía las técnicas de programación empleadas en la programación modular y estructurada; se amplía el concepto de flujo de control desarrollado en el Capítulo 3 y se describen con una gran cantidad de ejemplos y ejercicios aclaratorios las estructuras clásicas empleadas en la programación estructurada: secuenciales, selectivas y repetitivas.

El Capítulo 5 trata de los subprogramas (subalgoritmos): procedimientos y funciones. Los temas descritos incluyen la correspondencia de argumentos y parámetros, así como la comunicación entre el programa principal y los subprogramas y entre ellos entre sí. El método de invocación de un subprograma junto con los valores devueltos en el caso de las funciones son otro de los temas considerados en el capítulo. Los problemas planteados por el uso inadecuado de variables globales conocidos como *efectos laterales* se analizan también. El concepto de *recursión* o *recursividad* se introduce en el capítulo como técnica fundamental de diseño del algunos problemas de naturaleza recursiva.

El concepto de ***array*** (*tabla, matriz o vector,* ***"arreglo"*** en Latinoamérica) es el tema central del Capítulo 6. El proceso de arrays de una dimensión (vectores) se examina en primer lugar. Se describen algunas aplicaciones típicas con vectores. A continuación se analizan los arrays multidimensionales, con especial hincapié en los arrays bidimensionales o tablas.

El procesamiento de datos tipo carácter y las cadenas de caracteres se describen en el Capítulo 7. Algunas operaciones básicas con cadenas junto a funciones intrínsecas para tratamiento de cadenas se analizan también en este capítulo.

El Capítulo 8 describe el concepto de archivo externo, su organización, almacenamiento y recuperación en dispositivos externos tales como discos y cintas. Los diferentes tipos de archivos y las operaciones típicas que se realizan con ellos son tema central del capítulo. Las organizaciones secuenciales, aleatorias o de acceso directo y las indexadas se analizan y describen junto con los algoritmos de manipulación de los archivos.

Las aplicaciones más importantes y frecuentes en cualquier algoritmo son la ordenación, búsqueda y mezcla o intercalación de información. Los algoritmos más populares y reconocidos relativos a arrays unidimensionales se analizan en el Capítulo 9. Estos algoritmos se consideran internos, dado que el procesamiento de los mismos se realiza en memoria central, para diferenciarlos de la manipulación de los archivos que se consideran externos y que se almacenan en soportes de almacenamiento externo. La ordenación, búsqueda y fusión externa se describen en el Capítulo 10 mientras que las operaciones internas correspondientes se describen en el Capítulo 9.

El Capítulo 11 ofrece una introducción al estudio de las estructuras de datos lineales. Estructuras de datos simples tales como listas enlazadas, pilas y colas se examinan a lo largo del capítulo. Se describen un gran número de operaciones importantes y sus implementaciones mediante arrays y punteros o apuntadores.

El Capítulo 12 estudia las estructuras de datos no lineales: árboles y grafos. El tipo de árbol más usual es el árbol binario y se describe en profundidad; de igual forma se describen los grafos. La terminología usual empleada en los árboles y grafos junto con algoritmos sencillos se describen en el capítulo.

En el Capítulo 13 se dan reglas para realizar un diseño eficaz y eficiente de programas. El esti-

lo de programación es sin duda tema clave en la construcción moderna de programas. El elevado tamaño de la memoria central de las computadoras junto con las grandes capacidades de almacenamiento de los discos externos de las mismas han conducido a considerar el estilo de programación como uno de los factores clave en el desarrollo de programas. Las reglas para obtener un programa de calidad se analizan a lo largo del capítulo.

Una de las herramientas clásicas empleadas en el análisis de un problema es la tabla de decisión. En el Capítulo 14 se analiza el concepto, tipos, construcción y conversión de tablas de decisión en programas. La tabla de decisión es una herramienta de gestión eficaz que se complementa excelentemente con el diseño de algoritmos para la resolución de problemas, especialmente del campo de la gestión.

El Apéndice A describe las especificaciones del lenguaje algorítmico **UPSAM** *(Universidad Pontificia de Salamanca en Madrid)* cuyo soporte fundamental es el pseudocódigo y que ha sido utilizado en todo el texto. El Apéndice B es una guía de usuario del lenguaje Pascal estándar tal como fue concebido por su inventor Niklaus Wirth; es el lenguaje clave utilizado en la mayoría de las universidades de todo el mundo para enseñar el aprendizaje de la programación a los alumnos de los primeros cursos. Hoy día no se utiliza con profusión pero su conocimiento es esencial para cualquier persona que desee aprender técnicas de programación estructurada. El Apéndice C es otra guía de usuario o de sintaxis de Turbo/Borland Pascal, la versión más popular del lenguaje implantada en computadoras personales tipo PC o Macintosh y bajo sistemas operativos tales como DOS, Windows 3.1 o Windows 95.

Los Apéndices D y E se dedican a describir los lenguajes de programación más utilizados en la actualidad en el mundo profesional: C y C++ (este último como base de la programación orientada a objetos). La estructura del lenguaje C ANSI desarrollado por Kernighan y Ritchie se describe en el Apéndice D, mientras que en el Apéndice E se describen las diferencias entre C y C++ junto con las características específicas de C++.

El Apéndice F muestra reglas para convertir algoritmos escritos en pseudocódigo o mejor en lenguaje UPSAM a lenguajes de programación clásicos tales como Pascal, FORTRAN y Modula-2. Los Apéndices A a E deben permitir al alumno la conversión de cualquier programa a los lenguajes Pascal/Turbo Pascal o C/C++.

El Apéndice G es un breve glosario de terminos de programación utilizados a lo largo del libro y que facilitarán al lector la introducción a la terminología clásica de programación. El Apéndice H incluye tablas con los códigos de caracteres estándar ASCII, empleados por la mayoría de las computadoras fabricadas en la actualidad. El libro termina con una amplia bibliografía relativa a técnicas de programación, programación estructurada y los lenguajes de programación citados en el texto, tales como Pascal, COBOL, C, C++, Ada, etc.

AGRADECIMIENTOS

Los profesores citados con anterioridad han contribuido de una u otra forma a la creación del lenguaje algorítmico UPSAM y, por consiguiente, debemos agradecerles de nuevo su contribución al mismo, así como numerosas sugerencias y consejos para mejorar la primera edición. Sin embargo, en esta segunda edición nos han prestado una especial contribución en forma de revisión y corrección de la primera edición o bien de esta segunda edición, una serie de profesores a los que deseamos expresar nuestro reconocimiento y agradecimiento especial. Estos profesores son: Antonio Muñoz, Ángel Hermoso, Luis Villar, Luis Rodríguez y Matilde Fernández, pertenecientes al Departamento

de Lenguajes y Sistemas Informáticos e Ingeniería de Software, de la Facultad y Escuela Universitaria de Informática de la Universidad Pontificia de Salamanca en Madrid.

Numerosos profesores de universidades e institutos tecnológicos de Latinoamérica nos han sugerido también ideas, aportaciones y sugerencias que hemos tenido en cuenta en esta segunda edición. Deseamos destacar de modo especial la revisión y corrección completa de la primera edición realizada por Óscar Manuel Arango Jaramillo, profesor titular de Universidad Eant del Departamento de Ingeniería de Sistemas de Medellín (Colombia).

Por último, considerar nuestro agradecimiento más especial a los numerosos alumnos de España y Latinoamérica que han estudiado con este libro y se han iniciado en la programación con él; muchas de sus críticas, sugerencias y propuestas han sido un gran acicate para que esta segunda edición pudiese ver la luz. Mi reconocimiento más sincero es para todos estos lectores y alumnos cuyo aliento y ánimo nunca me han faltado.

Majadahonda (Madrid), verano de 1996
El autor

CAPÍTULO 1

ALGORITMOS Y PROGRAMAS

CONTENIDO

1.1. Los sistemas de procesamiento de la información.
1.2. Concepto de algoritmo.
1.3. Los lenguajes de programación.
1.4. Datos, tipos de datos y operaciones primitivas.
1.5. Constantes y variables.
1.6. Expresiones.
1.7. Funciones internas.
1.8. La operación de asignación.
1.9. Entrada y salida de información.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

La principal razón para que las personas aprendan lenguajes y técnicas de programación es *utilizar la computadora como una herramienta para resolver problemas*. La resolución de un problema exige al menos los siguientes pasos:

1. Definición o análisis del problema.
2. Diseño del algoritmo.
3. Transformación del algoritmo en un programa.
4. Ejecución y validación del programa.

Uno de los objetivos fundamentales de este libro es el *aprendizaje y diseño de algoritmos*. Este capítulo introduce al lector en el concepto de algoritmo y de programa, así como las herramientas que permiten «dialogar» al usuario con la máquina: *los lenguajes de programación*.

Un algoritmo es un método para resolver un problema. Aunque la popularización del término ha llegado con el advenimiento de la era informática, **algoritmo** proviene de *Mohammed al-Khowârizmî*, matemático persa que vivió durante el siglo IX y alcanzó gran reputación por el enun-

ciado de las reglas paso a paso para sumar, restar, multiplicar y dividir números decimales; la traducción al latín del apellido en la palabra *algorismus* derivó posteriormente en algoritmo. Euclides, el gran matemático griego (del siglo IV antes de Cristo), que inventó un método para encontrar el máximo común divisor de dos números, se considera con Al-Khowârizmî el otro gran padre de la algoritmia (ciencia que trata de los algoritmos).

El resto del capítulo trata de los datos y las operaciones elementales necesarias para el diseño del algoritmo. El profesor Niklaus Wirth —inventor de Pascal, Modula-2 y Oberon— tituló uno de sus más famosos libros, *Algoritmos + Estructuras de datos = Programas*, significándonos que sólo se puede llegar a realizar un buen programa con el diseño de un algoritmo y una correcta estructura de datos. Esta ecuación será una de las hipótesis fundamentales consideradas en esta obra.

1.1. LOS SISTEMAS DE PROCESAMIENTO DE LA INFORMACIÓN

Una definición antigua de *computadora* es: «una máquina o aparato electrónico capaz de ejecutar operaciones repetitivas muy complejas a altas velocidades». Ahora bien, esta definición no describe las modernas computadoras. Éstas son más que una máquina de ejecutar operaciones aritméticas. De hecho, los términos *procesador de datos y sistemas de procesamiento (tratamiento) de la información* se utilizan con frecuencia en lugar de **computadora** (*ordenador*, en la jerga informática usual en España).

En el uso diario, datos e información son esencialmente sinónimos. Sin embargo, los informáticos suelen hacer una diferencia: *datos* se refiere a la representación de algún hecho, concepto o entidad real (los datos pueden tomar diferentes formas; por ejemplo, palabras escritas o habladas, números y dibujos); *información* implica datos procesados y organizados.

Un *sistema* en general se define como conjunto de componentes conectados e interactivos, que tienen un propósito y una unidad total. **Sistema de procesamiento de información** es un sistema que transforma datos brutos en *información organizada, significativa* y *útil*.

La Figura 1.1 muestra los tres componentes de un sistema de proceso de la información: *entrada, salida* y *procesador*. El *procesador*, que puede ser bastante complicado, se representa por una simple caja y puede aceptar datos llamados *entrada*, y esta entrada se transforma entonces para producir una información denominada *salida* o *resultados*.

Basados en este esquema, muchos dispositivos u organismos pueden ser considerados sistemas de procesamiento de la información. Un termostato que controla la temperatura de un edificio es un sistema de procesamiento de la información. La entrada es la temperatura media y la salida es una señal que controla la caldera del aire acondicionado. El corazón de un animal o un ser humano es un sistema complejo de procesamiento de la información.

El conjunto de instrucciones que especifican la secuencia de operaciones a realizar, en orden, para resolver un sistema específico o clase de problemas, se denomina algoritmo. En otras palabras, un **algoritmo** es *una fórmula para la resolución de un problema*.

Figura 1.1. Sistema de proceso de información.

Para realizar un proceso se le debe suministrar al procesador un algoritmo adecuado. Por ejemplo, al cocinero debe dársele una receta, al pianista la partitura y así sucesivamente, considerando al cocinero y al pianista como procesadores.

Cuando el procesador es una computadora, el algoritmo ha de expresarse de una forma que recibe el nombre de *programa*. Un programa se escribe en un lenguaje de programación y a la actividad de expresar un algoritmo en forma de programa se le denomina *programación*. Cada paso en el algoritmo está expresado por medio de una *instrucción* en el programa. Por consiguiente, un programa consta de una secuencia de instrucciones, cada una de las cuales especifica las operaciones que debe realizar la computadora.

Existen dos conceptos importantes a considerar en los sistemas de procesamiento de la información: hardware y software. **Hardware** es el conjunto de componentes físicos de una computadora (Figura 1.2) —*equipo físico*— y **software** es el conjunto de programas que controlan el funcionamiento de una computadora —*equipo lógico*—.

El hardware de una computadora se compone de:

1. La *Unidad Central de Proceso, UCP (Central Processing Unit, CPU)*. La UCP es el conjunto de circuitos electrónicos capaces de ejecutar algunos cálculos sencillos como suma o multiplicación de números. La potencia de una computadora depende completamente de la velocidad y fiabilidad de la UCP.
2. *Memoria central*. La información procesada por la UCP se almacena normalmente en la memoria central hasta que se terminan los cálculos. Los programas de computadora se almacenan también en la memoria central.
3. *Dispositivos de almacenamiento secundario (memoria auxiliar)*. Diferentes dispositivos, tales como discos y cintas magnéticas, se usan para almacenar grandes cantidades de información. Para ser procesados por la UCP, los datos se almacenan en dispositivos de almacenamiento auxiliar y luego tienen que llevarse a la memoria central.
4. *Periférico o dispositivos de entrada/salida (E/S)*. Estos dispositivos permiten al usuario comunicarse con la computadora. Algunos dispositivos típicos de E/S son el teclado, la impresora, monitor, etc. Un sistema de computadora puede tener diferentes dispositivos periféricos conectados a ella.

En la práctica, una instalación grande de computadora puede tener diferentes UCP, cada una con su propia memoria central compartida, una variedad de dispositivos de almacenamiento secundario

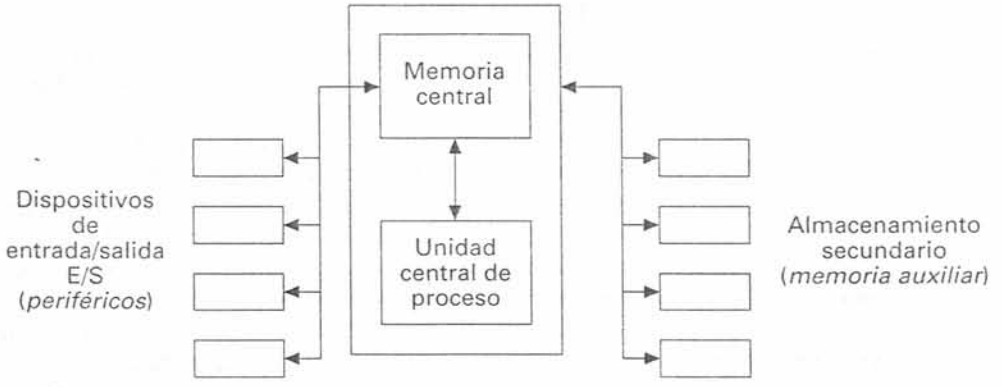


Figura 1.2. Diagrama esquemático de una computadora (hardware).

y periféricos localizados en diferentes partes de un mismo edificio o diferentes edificios e incluso diferentes ciudades o países.

1.2. CONCEPTO DE ALGORITMO

El objetivo fundamental de este texto es enseñar a resolver problemas mediante una computadora. Un programador de computadora es antes que nada una persona que resuelve problemas, por lo que para llegar a ser un programador eficaz se necesita aprender a resolver problemas de un modo riguroso y sistemático. A lo largo de todo este libro nos referiremos a la *metodología necesaria para resolver problemas mediante programas*, concepto que se denomina **metodología de la programación**. El eje central de esta metodología es el concepto, ya tratado, de algoritmo.

La resolución de un problema exige el diseño de un algoritmo que resuelva el problema propuesto.

Figura 1.3. Resolución de un problema.

Los pasos para la resolución de un problema son:

1. *Diseño del algoritmo* que describe la secuencia ordenada de pasos —sin ambigüedades— que conducen a la solución de un problema dado. (*Análisis del problema y desarrollo del algoritmo.*)
2. Expresar el algoritmo como un *programa* en un lenguaje de programación adecuado. (*Fase de codificación.*)
3. *Ejecución y validación* del programa por la computadora.

Para llegar a la realización de un programa es necesario el diseño previo de un algoritmo, de modo que sin algoritmo no puede existir un programa.

Los algoritmos son independientes tanto del lenguaje de programación en que se expresan como de la computadora que los ejecuta. En cada problema el algoritmo se puede expresar en un lenguaje diferente de programación y ejecutarse en una computadora distinta; sin embargo, el algoritmo será siempre el mismo. Así, por ejemplo, en una analogía con la vida diaria, una receta de un plato de cocina se puede expresar en español, inglés o francés, pero cualquiera que sea el lenguaje, los pasos para la elaboración del plato se realizarán sin importar el idioma del cocinero.

En la ciencia de la computación y en la programación, los algoritmos son más importantes que los lenguajes de programación o las computadoras. Un lenguaje de programación es tan sólo un medio para expresar un algoritmo y una computadora es sólo un procesador para ejecutarlo. Tanto el lenguaje de programación como la computadora son los medios para obtener un fin: conseguir que el algoritmo se ejecute y se efectúe el proceso correspondiente.

Dada la importancia del algoritmo en la ciencia de la computación, un aspecto muy importante será el *diseño de algoritmos*. A la enseñanza y práctica de esta tarea se dedica gran parte de este libro.

El diseño de la mayoría de los algoritmos requiere creatividad y conocimientos profundos de la técnica de la programación. En esencia, la *solución de un problema se puede expresar mediante un algoritmo*.

1.2.1. Características de los algoritmos

Las características fundamentales que debe cumplir todo algoritmo son:

- Un algoritmo debe ser *preciso* e indicar el orden de realización de cada paso.
- Un algoritmo debe estar *definido*. Si se sigue un algoritmo dos veces, se debe obtener el mismo resultado cada vez.
- Un algoritmo debe ser *finito*. Si se sigue un algoritmo, se debe terminar en algún momento; o sea, debe tener un número finito de pasos.

La definición de un algoritmo debe describir tres partes: *Entrada*, *Proceso* y *Salida*. En el algoritmo de receta de cocina citado anteriormente se tendrá:

Entrada: ingredientes y utensilios empleados.
Proceso: elaboración de la receta en la cocina.
Salida: terminación del plato (por ejemplo, cordero).

Ejemplo 1.1

Un cliente ejecuta un pedido a una fábrica. La fábrica examina en su banco de datos la ficha del cliente, si el cliente es solvente entonces la empresa acepta el pedido; en caso contrario, rechazará el pedido. Redactar el algoritmo correspondiente.

Los pasos del algoritmo son:

```
1. Inicio.
2. Leer el pedido.
3. Examinar la ficha del cliente.
4. Si el cliente es solvente, aceptar pedido; en caso contrario, rechazar pedido.
5. Fin.
```

Ejemplo 1.2

Se desea diseñar un algoritmo para saber si un número es primo o no.

Un número es primo si sólo puede dividirse por sí mismo y por la unidad (es decir, no tiene más divisores que él mismo y la unidad). Por ejemplo, 9, 8, 6, 4, 12, 16, 20, etc., no son primos, ya que son divisibles por números distintos a ellos mismos y a la unidad. Así, 9 es divisible por 3, 8 lo es por 2, etc.

El algoritmo de resolución del problema pasa por dividir sucesivamente el número por 2, 3, 4..., etcétera.

```
1. Inicio.
2. Poner X igual a 2 (X = 2, X, variable que representa a los divisores del numero
   que se busca N).
3. Dividir N por X (N/X).
4. Si el resultado de N/X es entero, entonces N no es un numero primo y bifurcar
   al punto 7; en caso contrario, continuar el proceso.
5. Suma 1 a X (X ← X + 1).
6. Si X es igual a N[1], entonces N es un numero primo; en caso contrario, bifurcar
   al punto 3.
7. Fin.
```

[1] Esta condición puede ser sustituida por `X = N div 2` (donde **div** es el operador división entera). Véase Tabla 1.1, página 19.

Por ejemplo, si N es 131, los pasos anteriores serían:

```
1. Inicio.
2. X = 2.
3 y 4. 131/X. Como el resultado no es entero, se continua el proceso.
5. X ← 2 + 1, luego X = 3.
6. Como X no es 131, se bifurca al punto 3.
3 y 4. 131/X resultado no es entero.
       ← 1, ←X
6. Como x no es 131 bifurca al punto 3.
3 y 4. 131/X..., etc.
7. Fin.
```

Ejemplo 1.3

Realizar la suma de todos los números pares entre 2 y 1000.

El problema consiste en sumar 2 + 4 + 6 + 8 + ... + 1000.

Utilizaremos las palabras SUMA y NUMERO (*variables*, serán denominadas más tarde) para representar las sumas sucesivas (2+4), (2+4+6), (2+4+6+8), etc.

La solución se puede escribir con el siguiente algoritmo:

```
1. Inicio.
2. Establecer SUMA a 0.
3. Establecer NUMERO a 2.
4. Sumar NUMERO a SUMA. El resultado sera el nuevo valor de la suma (SUMA).
5. Incrementar NUMERO en 2 unidades.
6. Si NUMERO =< 1000 bifurcar al paso 4; en caso contrario, escribir el ultimo
   valor de SUMA y terminar el proceso.
7. Fin.
```

1.3. LOS LENGUAJES DE PROGRAMACIÓN

Como se ha visto en el apartado anterior, para que un procesador realice un proceso se le debe suministrar en primer lugar un algoritmo adecuado. El procesador debe se capaz de *interpretar* el algoritmo, lo que significa

- comprender las instrucciones de cada paso,
- realizar las operaciones correspondientes.

Cuando el procesador es una computadora, el algoritmo se ha de expresar en un formato que se denomina *programa*. Un programa se escribe en un *lenguaje de programación* y las operaciones que conducen a expresar un algoritmo en forma de programa se llaman *programación*. Así pues, los lenguajes utilizados para escribir programas de computadoras son los *lenguajes de programación* y *programadores* son los escritores y diseñadores de programas.

Los principales tipos de lenguajes utilizados en la actualidad son tres:

- *lenguaje máquina,*
- *lenguaje de bajo nivel (ensamblador),*
- *lenguajes de alto nivel.*

1.3.1. Instrucciones a la computadora

Los diferentes pasos (*acciones*) de un algoritmo se expresan en los programas como *instrucciones*, *sentencias* o *proposiciones* (normalmente el término instrucción se suele referir a los lenguajes máquina y bajo nivel, reservando la sentencia o proposición para los lenguajes de alto nivel). Por consiguiente, un programa consta de una secuencia de instrucciones, cada una de las cuales especifica ciertas operaciones que debe ejecutar la computadora.

La elaboración de un programa requerirá conocer el juego o repertorio de instrucciones del lenguaje. Aunque en el Capítulo 3 se analizarán con más detalle las instrucciones, adelantaremos los tipos fundamentales de instrucciones que una computadora es capaz de manipular y ejecutar. Las instrucciones básicas y comunes a casi todos los lenguajes de programación se pueden condensar en cuatro grupos:

- *Instrucciones de entrada/salida*. Instrucciones de transferencia de información y datos entre dispositivos periféricos (teclado, impresora, unidad de disco, etc.) y la memoria central.
- *Instrucciones aritmético-lógicas*. Instrucciones que ejecutan operaciones aritméticas (suma, resta, multiplicación, división, potenciación), lógicas (operaciones `and`, `or`, `not`, etc.).
- *Instrucciones selectivas*. Instrucciones que permiten la selección de tareas alternativas en función de los resultados de diferentes expresiones condicionales.
- *Instrucciones repetitivas*. Instrucciones que permiten la repetición de secuencias de instrucciones un número determinado o indeterminado de veces.

1.3.2. Lenguajes máquina

Los **lenguajes máquina** son aquellos que están escritos en lenguajes directamente inteligibles por la máquina (computadora), ya que sus instrucciones son *cadenas binarias* (cadenas o series de caracteres —dígitos— 0 y 1) que especifican una operación, y las posiciones (dirección) de memoria implicadas en la operación se denominan *instrucciones de máquina* o *código máquina*. El código máquina es el conocido código binario.

Las instrucciones en lenguaje máquina dependen del *hardware* de la computadora y, por tanto, diferirán de una computadora a otra. El lenguaje máquina de un PC (computadora personal) será diferente de un sistema HP 9000 (HP, Hewlett Packard) o un sistema 6000 de IBM.

Las *ventajas* de programar en lenguaje máquina son la posibilidad de *cargar* (transferir un

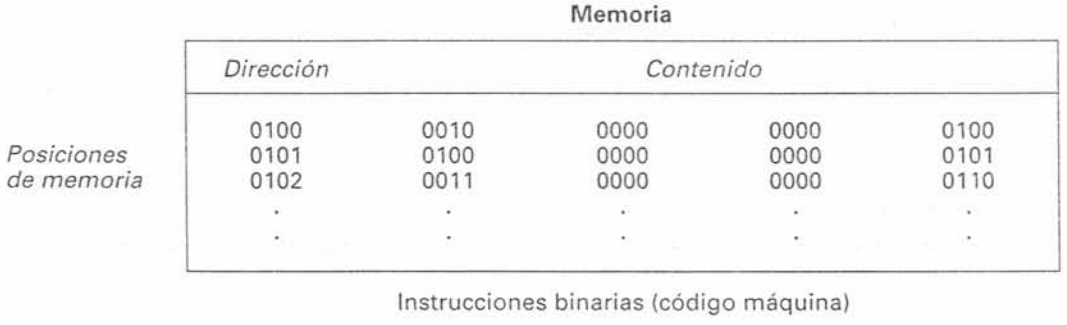


Figura 1.4. Instrucciones en lenguaje máquina.

programa a la memoria) sin necesidad de traducción posterior, lo que supone una velocidad de ejecución superior a cualquier otro lenguaje de programación.

Los *inconvenientes* —en la actualidad— superan a las ventajas, lo que hace prácticamente no recomendables los lenguajes máquina. Estos inconvenientes son:

- dificultad y lentitud en la codificación,
- poca fiabilidad,
- dificultad grande de verificar y poner a punto los programas,
- los programas sólo son ejecutables en el mismo procesador (UCP, *Unidad Central de Proceso*).

Para evitar los lenguajes máquina, desde el punto de vista del usuario, se han creado otros lenguajes que permiten escribir programas con instrucciones similares al lenguaje humano (por desgracia, casi siempre inglés, aunque existen excepciones, como es el caso de las versiones españolas del lenguaje LOGO).

Estos lenguajes son los de alto nivel y bajo nivel.

1.3.3. Lenguajes de bajo nivel

Los **lenguajes de bajo nivel** son más fáciles de utilizar que los lenguajes máquina, pero, al igual que ellos, dependen de la máquina en particular. El lenguaje de bajo nivel por excelencia es el *ensamblador (assembly language)*. Las instrucciones en lenguaje ensamblador son instrucciones conocidas como ***nemotécnicos*** (*mnemonics*). Por ejemplo, nemotécnicos típicos de operaciones aritméticas son: en ingles, `ADD`, `SUB`, `DIV`, etc.; en español, `SUM`, `RES`, `DIV`, etc.

Una instrucción típica de suma sería:

```
ADD  M, N, P
```

Esta instrucción podría significar «*sumar el número contenido en la posición de memoria M al número almacenado en la posición de memoria N y situar el resultado en la posición de memoria P*». Evidentemente, es mucho más sencillo recordar la instrucción anterior con un *nemotécnico* que su equivalente en código máquina:

```
0110     1001     1010     1011
```

Un programa escrito en lenguaje ensamblador no puede ser ejecutado directamente por la computadora —en esto se diferencia esencialmente del lenguaje máquina—, sino que requiere una fase de *traducción* al lenguaje máquina.

El programa original escrito en lenguaje ensamblador se denomina *programa fuente* y el programa traducido en lenguaje máquina se conoce como *programa objeto*, ya directamente inteligible por la computadora.

El traductor de programas fuente a objeto es un programa llamado *ensamblador (assembler)*, existente en casi todas las computadoras (Figura 1.5).

No se debe confundir —aunque en español adoptan el mismo nombre— el *programa ensamblador (assembler)*, encargado de efectuar la traducción del programa fuente escrito a lenguaje máquina, con el *lenguaje ensamblador (assembly language)*, lenguaje de programación con una estructura y gramática definidas.

Los lenguajes ensambladores presentan la *ventaja* frente a los lenguajes máquina de su mayor facilidad de codificación y, en general, su velocidad de cálculo.

Figura 1.5. Programa ensamblador.

Los *inconvenientes* más notables de los lenguajes ensambladores son:

- Dependencia total de la máquina, lo que impide la transportabilidad de los programas (posibilidad de ejecutar un programa en diferentes máquinas). El lenguaje ensamblador del PC es distinto del lenguaje ensamblador del Apple Macintosh.
- La formación de los programadores es más compleja que la correspondiente a los programadores de alto nivel, ya que exige no sólo las técnicas de programación, sino también el conocimiento del interior de la máquina.

Hoy día los lenguajes ensambladores tienen sus aplicaciones muy reducidas en la programación de aplicaciones y se centran en aplicaciones de tiempo real, control de procesos y de dispositivos electrónicos, etc.

1.3.4. Lenguajes de alto nivel

Los *lenguajes de alto nivel* son los más utilizados por los programadores. Están diseñados para que las personas escriban y entiendan los programas de un modo mucho más fácil que los lenguajes máquina y ensambladores. Otra razón es que un programa escrito en un lenguaje de alto nivel es independiente de la máquina; esto es, las instrucciones del programa de la computadora no dependen del diseño del *hardware* o de una computadora en particular. En consecuencia, los programas escritos en lenguajes de alto nivel son *portables* o *transportables*, lo que significa la posibilidad de poder ser ejecutados con poca o ninguna modificación en diferentes tipos de computadoras; al contrario que los programas en lenguaje máquina o ensamblador, que sólo se pueden ejecutar en un determinado tipo de computadora.

Los lenguajes de alto nivel presentan las siguientes *ventajas*:

- El tiempo de formación de los programadores es relativamente corto comparado con otros lenguajes.
- La escritura de programas se basa en reglas sintácticas similares a los lenguajes humanos. Nombres de las instrucciones, tales como `READ`, `WRITE`, `PRINT`, `OPEN`, etc.
- Las modificaciones y puestas a punto de los programas son más fáciles.
- Reducción del coste de los programas.
- Transportabilidad.

Los *inconvenientes* se concretan en:

- Incremento del tiempo de puesta a punto, al necesitarse diferentes traducciones del programa fuente para conseguir el programa definitivo.
- No se aprovechan los recursos internos de la máquina, que se explotan mucho mejor en lenguajes máquina y ensambladores.
- Aumento de la ocupación de memoria.
- El tiempo de ejecución de los programas es mucho mayor.

Al igual que sucede con los lenguajes ensambladores, los programas fuente tienen que ser traducidos por programas traductores, llamados en este caso *compiladores* e *intérpretes*.

Los lenguajes de programación de alto nivel existentes hoy son muy numerosos aunque la práctica demuestra que su uso mayoritario se reduce a

C C++ COBOL FORTRAN BASIC Pascal Visual BASIC

están muy extendidos:

Clipper Ada Modula-2 Prolog LISP Smalltalk

y comienzan a difundirse:

Visual Object Delphi Miranda Eiffel

aunque los dos primeros no dejan de ser entornos de programación orientados a objetos con soporte en los lenguajes Clipper y Turbo Pascal.

1.3.5. Traductores de lenguaje

Los *traductores de lenguaje* son programas que traducen a su vez los programas fuente escritos en lenguajes de alto nivel a código máquina.

Los traductores se dividen en:

- *compiladores,*
- *intérpretes.*

1.3.5.1. Intérpretes

Un *intérprete* es un traductor que toma un programa fuente, lo traduce y a continuación lo ejecuta.

Los programas intérpretes clásicos como BASIC, prácticamente ya no se utilizan, aunque las versiones QBasic y QuickBASIC se comercializan todavía con el Sistema Operativo DOS que corre en las computadoras personales. Sin embargo, está muy extendida la versión interpretada del lenguaje Smalltalk, un lenguaje orientado o objetos puro.

Figura 1.6. Intérprete.

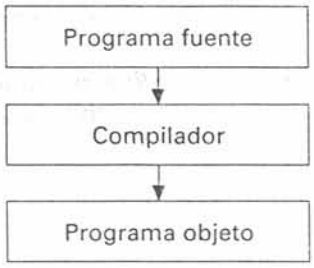


Figura 1.7. La compilación de programas.

1.3.5.2. Compiladores

Un *compilador* es un programa que traduce los programas fuente escritos en lenguajes de alto nivel —Pascal, FORTRAN, ...— a lenguaje máquina.

Los programas escritos en lenguajes de alto nivel se llaman *programas fuente* y el programa traducido *programa objeto* o *código objeto*. El compilador traduce —sentencia a sentencia— el programa fuente.

Los lenguajes compiladores típicos son: C, C++, PASCAL, FORTRAN, COBOL.

1.3.6. La compilación y sus fases

La *compilación* es el proceso de traducción de programas fuente a programas objeto. El programa objeto obtenido de la compilación ha sido traducido normalmente a código máquina.

Para conseguir el programa máquina real se debe utilizar un programa llamado *montador* o *enlazador (linker)*. El proceso de montaje conduce a un programa en lenguaje máquina directamente ejecutable (Figura 1.8).

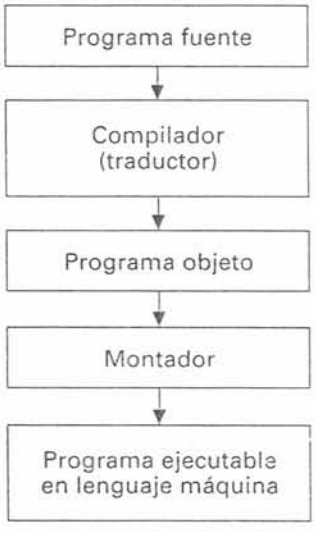


Figura 1.8. Fases de la compilación.

El proceso de ejecución de un programa Pascal, por ejemplo, tiene los siguientes pasos:

1. Escritura del *programa fuente* con un *editor* (programa que permite a una computadora actuar de modo similar a una máquina de escribir electrónica) y guardarlo en un dispositivo de almacenamiento (por ejemplo, un disco).
2. Introducir el programa fuente en memoria.
3. *Compilar* el programa con el compilador Pascal.
4. *Verificar y corregir errores de compilación* (listado de errores).
5. Obtención del *programa objeto*.
6. El montador obtiene el *programa ejecutable*.
7. Se ejecuta el programa y, si no existen errores, se tendrá la salida del programa.

El proceso de ejecución sería el mostrado en las Figuras 1.9 y 1.10.

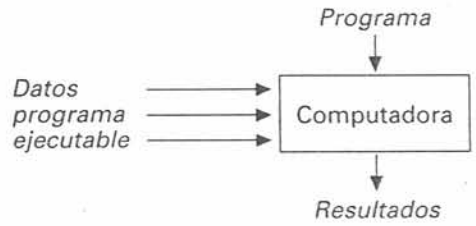


Figura 1.9. Ejecución de un programa.

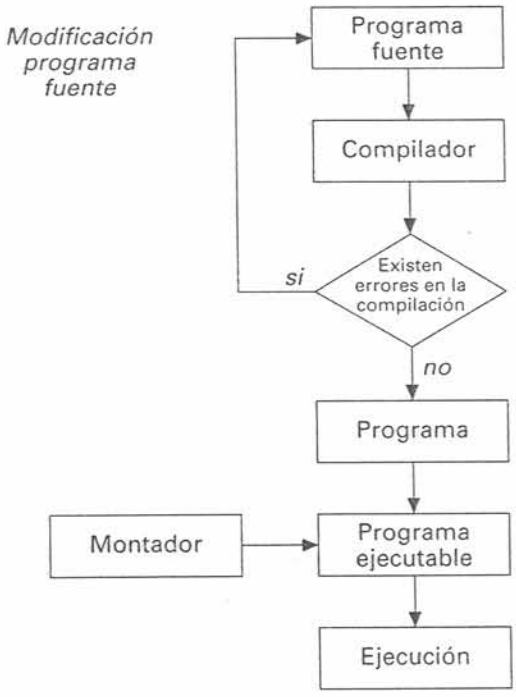


Figura 1.10. Fases de la ejecución de un programa.

1.4. DATOS, TIPOS DE DATOS Y OPERACIONES PRIMITIVAS

El primer objetivo de toda computadora es el manejo de la información o datos. Estos datos pueden ser las cifras de ventas de un supermercado o las calificaciones de una clase. Un *dato* es la expresión general que describe los objetos con los cuales opera una computadora.

La mayoría de las computadoras pueden trabajar con varios tipos (modos) de datos. Los algoritmos y los programas correspondientes operan sobre datos.

La acción de las instrucciones ejecutables de las computadores se reflejan en cambios en los valores de las partidas de datos. Los datos de entrada se transforman por el programa, después de las etapas intermedias, en datos de salida.

En el proceso de solución de problemas el diseño de la estructura de datos es tan importante como el diseño del algoritmo y del programa que se basa en el mismo.

Existen dos clases de tipos de datos: *simples* (sin estructura) y *compuestos* (estructurados). Los datos estructurados se estudian a partir del Capítulo 6 y son conjuntos de partidas de datos simples con relaciones definidas entre ellos.

Los distintos tipos de datos se representan en diferentes formas en la computadora. A nivel de máquina, un dato es un conjunto o secuencia de bits (dígitos 0 o 1). Los lenguajes de alto nivel permiten basarse en abstracciones e ignorar los detalles de la representación interna. Aparece el concepto de tipo de datos, así como su representación.

Los tipos de datos simples son los siguientes:

numéricos (*integer, real*),
lógicos (*boolean*),
carácter (*char, string*).

Existen algunos lenguajes de programación —FORTRAN esencialmente— que admiten otros tipos de datos; complejos, que permiten tratar los números **complejos**, y otros lenguajes —Pascal— que también permiten declarar y definir sus propios tipos de datos: **enumerados** (*enumerated*) y **subrango** (*subrange*).

1.4.1. Datos numéricos

El tipo *numérico* es el conjunto de los valores numéricos. Éstos pueden representarse en dos formas distintas:

- tipo numérico *entero* (*integer*),
- tipo numérico *real* (*real*).

Enteros: El tipo entero es un subconjunto finito de los números enteros. Los enteros son números completos, no tienen componentes fraccionarios o decimales y pueden ser negativos o positivos.

Ejemplos de números enteros son:

```
    5      6
  -15      4
   20     17
 1340     26
```

Los enteros se denominan en ocasiones números de punto o coma fija. Los números enteros máximos y mínimos de una computadora[2] suelen ser —32768 a +32767—. Los números enteros fuera de este rango no se suelen representar como enteros, sino como reales, aunque existen excepciones (enteros largos: FORTRAN, Quick/QBasic, C, C++, etc.).

Reales: El tipo real consiste en un subconjunto de los números reales. Los números reales siempre tienen un punto decimal y pueden ser positivos o negativos. Un número real consta de un entero y una parte decimal.

Los siguientes ejemplos son números reales:

```
 0.08          3739.41
 3.7452         -52.321
-8.12             3.0
```

En aplicaciones científicas se requiere una representación especial para manejar números muy grandes, como la masa de la Tierra, o muy pequeños, como la masa de un electrón. Una computadora sólo puede representar un número fijo de dígitos. Este número puede variar de una máquina a otra, siendo ocho dígitos un número típico. Este límite provocará problemas para representar y almacenar números muy grandes o muy pequeños como son los ya citados o los siguientes:

```
4867213432        0.00000000387
```

Existe un tipo de representación denominado *notación exponencial* o *científica* y que se utiliza para números muy grandes o muy pequeños. Así,

```
367520100000000000000
```

se representa en notación científica descomponiéndolo en grupos de tres dígitos:

```
367  520  100  000  000  000  000
```

y posteriormente en forma de potencias de 10:

3.675201×10^{20}

y de modo similar:

```
.0000000000302579
```

se representa como

3.02579×10^{-11}

La representación en coma flotante es una generalización de notación científica. Obsérvese que las siguientes expresiones son equivalentes:

$3.675201 \times 10^{19} = .3675207 \times 10^{20} = .03675201 \times 10^{21} = \ldots$
$= 36.75201 \times 10^{18} = 367.5201 \times 10^{17} = \ldots$

[2] En computadoras de 16 bits como IBM PC o compatibles.

En estas expresiones se considera la *mantisa* (parte decimal) al número real y el *exponente* (parte potencial) el de la potencia de diez.

```
36.75201    mantisa        18    exponente
```

1.4.2. Datos lógicos (booleanos)

El tipo *lógico* —también denominado *booleano*— es aquel dato que sólo puede tomar uno de dos valores:

cierto o **verdadero** (*true*) y **falso** (*false*).

Este tipo de datos se utiliza para representar las alternativas (*sí/no*) a determinadas condiciones. Por ejemplo, cuando se pide si un valor entero es par, la respuesta será verdadera o falsa, según sea par o impar.

1.4.3. Datos tipo carácter y tipo cadena

El tipo *carácter* es el conjunto finito y ordenado de caracteres que la computadora reconoce. Un dato tipo carácter contiene un solo carácter.

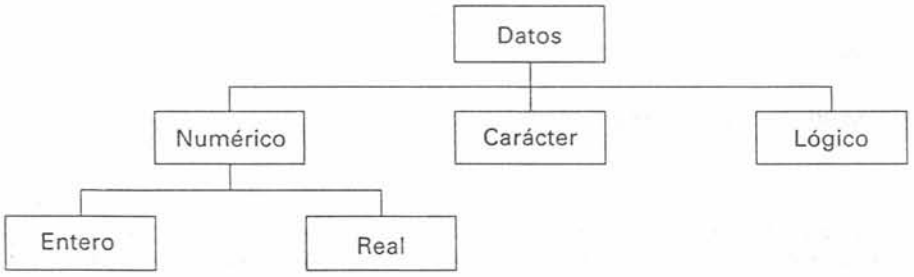
Los caracteres que reconocen las diferentes computadoras no son estándar; sin embargo, la mayoría reconoce los siguientes caracteres alfabéticos y numéricos:

- caracteres alfabéticos (A, B, C, ..., Z) (a, b, c, ..., z),
- caracteres numéricos (1, 2, ..., 9, 0),
- caracteres especiales (+, -, *, /, ^, ., ;, <, >, $, ...).

Una *cadena* (*string*) de *caracteres* es una sucesión de caracteres que se encuentran delimitados por una comilla (apóstrofo) o dobles comillas, según el tipo de lenguaje de programación. La *longitud* de una cadena de caracteres es el número de ellos comprendidos entre los separadores o limitadores. Algunos lenguajes tienen datos tipo *cadena*.

```
'Hola Mortimer'
'8 de octubre de 1946'
'Sr. McKenna'
```

RESUMEN: Los tipos de datos primitivos se clasifican en:



1.5. CONSTANTES Y VARIABLES

Los programas de computadora contienen ciertos valores que no deben cambiar durante la ejecución del programa. Tales valores se llaman *constantes*. De igual forma, existen otros valores que cambiarán durante la ejecución del programa; a estos valores se les llama *variables*.

Una ***constante*** es una partida de datos (objetos) que permanecen sin cambios durante todo el desarrollo del algoritmo o durante la ejecución del programa.

Constantes reales válidas:

```
1.234
- 0.1436
+ 54437324
```

Constantes reales no válidas:

```
1,752.63 (comas no permitidas)
82 (normalmente contienen un punto decimal, aunque existen
lenguajes que lo admiten sin punto)
```

Constantes reales en notación científica:

```
3.374562E2      equivale a      3.374562 x 10
```

Una constante *tipo carácter* o *constante de caracteres* consiste en un carácter válido encerrado dentro de apóstrofos; por ejemplo,

```
'B'     '+'     '4'     ';'
```

Si se desea incluir el apóstrofo en la cadena, entonces debe aparecer como un par de apóstrofos, encerrados dentro de simples comillas.

```
''
```

Una secuencia de caracteres se denomina normalmente una *cadena* y una *constante tipo cadena* es una cadena encerrada entre apóstrofos. Por consiguiente,

```
'Juan Minguez'
```

y

```
'Pepe Luis Garcia'
```

son constantes de cadena válidas. Nuevamente, si un apóstrofo es uno de los caracteres en una constante de cadena, debe aparecer como un par de apóstrofos:

```
'Juan''s'
```

Constantes lógicas (boolean)

Sólo existen dos constantes *lógicas* o *boolean*:

```
verdad      falso
```

La mayoría de los lenguajes de programación permiten diferentes tipos de constantes: *enteras*, *reales*, *caracteres* y *boolean* o *lógicas*, y representan datos de esos tipos.

Variables

Una *variable* es un objeto o partida de datos cuyo valor puede cambiar durante el desarrollo del algoritmo o ejecución del programa.

Dependiendo del lenguaje, hay diferentes tipos de variables, tales como *enteras*, *reales*, *carácter*, *lógicas* y *de cadena*.

Una variable que es de un cierto tipo puede tomar únicamente valores de ese tipo. Una variable de carácter, por ejemplo, puede tomar como valor sólo caracteres, mientras que una variable entera puede tomar sólo valores enteros.

Si se intenta asignar un valor de un tipo a una variable de otro tipo se producirá *un error de tipo*.

Una variable se identifica por los siguientes atributos: *nombre* que lo asigna y *tipo* que describe el uso de la variable.

Los nombres de las variables, a veces conocidos como *identificadores*, suelen constar de varios caracteres alfanuméricos, de los cuales el primero normalmente es una letra. No se deben utilizar —aunque lo permita el lenguaje, caso de FORTRAN— como nombres de identificadores palabras reservadas del lenguaje de programación.

Nombres válidos de variables son:

```
A510
NOMBRES
NOTAS
NOBRE_APELLIDOS[3]
```

Los nombres de las variables elegidas para el algoritmo o el programa deben ser significativos y tener relación con el objeto que representan, como pueden ser los casos siguientes:

```
NOMBRE    para representar nombres de personas
PRECIOS   para representar los precios de diferentes artículos
NOTAS     para representar las notas de una clase
```

Existen lenguajes —Pascal— en los que es posible darles nombre a determinadas constantes típicas utilizadas en cálculos matemáticos, financieros, etc. Por ejemplo, las constantes $\pi = 3.141592...$ y $e = 2.7182818$ (base de los logaritmos naturales) se les pueden dar los nombres `PI` y `E`.

```
PI = 3.141592
E  = 2.718282
```

1.6. EXPRESIONES

Las expresiones son combinaciones de constantes, variables, símbolos de operación, paréntesis y nombres de funciones especiales. Las mismas ideas son utilizadas en notación matemática tradicional; por ejemplo,

$$a+(b+3)+\sqrt{c} \qquad +b+(b-5)+\sqrt{c}$$

[3] Algunos lenguajes de programación admiten como válido el carácter subrayado en los identificadores.

Aquí los paréntesis indican el orden de cálculo y $\sqrt{\ }$ representa la función raíz cuadrada.

Cada expresión toma un valor que se determina tomando los valores de las variables y constantes implicadas y la ejecución de las operaciones indicadas.

Una expresión consta de *operandos* y *operadores*. Según sea el tipo de objetos que manipulan, las expresiones se clasifican en:

- *aritméticas,*
- *lógicas,*
- *carácter.*

El resultado de la expresión aritmética es de tipo numérico; el resultado de la expresión relacional y de una expresión lógica es de tipo lógico; el resultado de una expresión carácter es de tipo carácter.

1.6.1. Expresiones aritméticas

Las *expresiones aritméticas* son análogas a las fórmulas matemáticas. Las variables y constantes son numéricas (real o entera) y las operaciones son las aritméticas.

+	suma
-	resta
*	multiplicación
/	división
-, **, ^	exponenciación
div	división entera
mod	módulo (resto)

Los símbolos +, -, *, **, ^(- o **) y las palabras clave **div** y **mod** se conocen como *operadores aritméticos*. En la expresión

```
5 + 3
```

los valores 5 y 3 se denominan *operandos*. El valor de la expresión 5 + 3 se conoce como *resultado* de la expresión.

Los operadores se utilizan de igual forma que en matemáticas. Por consiguiente, A × B se escribe en un algoritmo como A * B, y 1/4 × C como C/4. Al igual que en matemáticas el signo menos juega un doble papel, como resta en A - B y como cambio de signo en -A.

No todos los operadores aritméticos existen en todos los lenguajes de programación; por ejemplo, en FORTRAN no existe **div** ni **mod**.

El operador exponenciación es diferente según sea el tipo de lenguaje de programación elegido (^, – en BASIC, ** en FORTRAN).

Los cálculos que implican tipos de datos reales y enteros suelen dar normalmente resultados del mismo tipo si los operandos lo son también. Por ejemplo, el producto de operandos reales produce un real (*véase* Tabla 1.1).

Tabla 1.1. Operadores aritméticos

Operador	Significado	Tipos de operandos	Tipo de resultado
–, ^, **	Exponenciación	Entero o real	Entero o real
+	Suma	Entero o real	Entero o real
-	Resta	Entero o real	Entero o real
*	Multiplicación	Entero o real	Entero o real
/	División	Real	Real
div	División entera	Entero	Entero
mod	Módulo (resto)	Entero	Entero

Ejemplos:

```
5 x 7        se representa por    5 * 7
6            se representa por    6/4
4
3^7          se representa por    3 ^ 7
```

Operadores DIV y MOD

El símbolo / se utiliza para la división real y el operador **div** —en algunos lenguajes, por ejemplo BASIC, se suele utilizar el símbolo \— representa la división entera.

```
A div B
```

Sólo se puede utilizar si A y B son expresiones enteras y obtiene la parte entera de A/B. Por consiguiente,

```
19 div 6
```

toma el valor 3.

Otro ejemplo puede ser la división 15/6:

```
15   |6
3     2  cociente
|
resto
```

En forma de operadores resultará la operación anterior:

```
15 div 6 = 2      15 mod 6 = 3
```

Otros ejemplos son:

```
19 div 3      equivale a 6
19 mod 6      equivale a 1
```

Ejemplo 1.4

Los siguientes ejemplos muestran resultados de expresiones aritméticas:

Expresión	Resultado	Expresión	Resultado
`10.5/3.0`	`3.5`	`10 div 3`	`3`
`1/4`	`0.25`	`18 div 2`	`9`
`2.0/4.0`	`0.5`	`30 div 30`	`1`
`6/1`	`6.0`	`6 div 8`	`0`
`30/30`	`1.0`	`10 mod 3`	`1`
`6/8`	`0.75`	`10 mod 2`	`0`

1.6.1.1. Reglas de prioridad

Las expresiones que tienen dos o más operandos requieren unas reglas matemáticas que permitan determinar el orden de las operaciones, se denominan reglas de *prioridad* o *precedencia* y son:

1. Las operaciones que están encerradas entre paréntesis se evalúan primero. Si existen diferentes paréntesis anidados (interiores unos a otros), las expresiones más internas se evalúan primero.
2. Las operaciones aritméticas dentro de una expresión suelen seguir el siguiente orden de prioridad:

   - operador exponencial (`^`, ↑ o bien `**`),
   - operadores `*`, `/`, `\`,
   - operadores **`div`** y **`mod`**,
   - operadores `+`, `-`.

En caso de coincidir varios operadores de igual prioridad en una expresión o subexpresión encerrada entre paréntesis, el orden de prioridad en este caso es de izquierda a derecha.

Ejemplo 1.5

¿Cuál es el resultado de las siguientes expresiones?

a) `3 + 6 * 14`

a) `3 + 6 * 14`

`3 + 84`

`87`

b) `8 + 7 * 3 + 4 * 6`

b) `8 + 7 * 3 + 4 * 6`

`8 + 21` `24`

`29 + 24`

`53`

Ejemplo 1.6

Obtener los resultados de las expresiones:

```
-4 * 7 + 2 ^ 3 / 4 - 5

1. -4 * 7 + 8 / 4 - 5
2. -28 + 8 / 4 - 5

3. -28 + 2 - 5
4. -26 - 5

   -31
```

Ejemplo 1.7

Convertir en expresiones aritméticas algorítmicas las siguientes expresiones algebraicas:

$5 \cdot (x + y)$

$a^2 + b^2$

$$\frac{x+y}{u+\frac{w}{a}}$$

$$\frac{x}{y} \cdot (z+w)$$

Los resultados serán:

```
5 * (x + y)
a ^ 2 + b ^ 2
(x + y) / (u + w /a)
x / y * (z + w)
```

Ejemplo 1.8

Los paréntesis tienen prioridad sobre el resto de las operaciones:

`A * (B + 3)`	*la constante* 3 *se suma primero al valor de* B, *después este resultado se multiplica por el valor de* A.
`(A * B) + 3`	A *y* B *se multiplican primero y a continuación se suma* 3.
`A + (B + C) + D`	*esta expresión equivale a* `A + B + C + D`.
`(A + B/C) + D`	*equivale a* `A + B/C + D`.
`A * B/C * D`	*equivale a* `((A * B)/C) * D` *y no a* `(A * B)/(C * D)`.

Ejemplo 1.9

Evaluar la expresión `12 + 3 * 7 + 5 * 4`.

En este ejemplo existen dos operadores de igual prioridad, * (multiplicación); por ello los pasos sucesivos son:

```
12 + 3 * 7 + 5 * 4
     21

12 + 21 + 5 * 4
          20

12 + 21 + 20 = 53
```

1.6.2. Expresiones lógicas (booleanas)

Un segundo tipo de expresiones es la *expresión lógica* o *booleana*, cuyo valor es siempre verdadero o falso. Recuerde que existen dos constantes lógicas, *verdad* (*true*) y *falso* (*false*) y que las variables lógicas pueden tomar sólo estos dos valores. En esencia, una *expresión lógica* es una expresión que sólo puede tomar estos dos valores, *verdad* y *falso*. Se denominan también *expresiones booleanas* en honor del matemático británico George Boole, que desarrollo el Álgebra lógica de Boole.

Las expresiones lógicas se forman combinando constantes lógicas, variables lógicas y otras expresiones lógicas, utilizando los *operadores lógicos* `not`, `and` y `or`, y los *operadores relacionales* (de relación o comparación) =, <, >, <=, >=, <>.

1.6.2.1. Operadores de relación

Los operadores relacionales o de relación permiten realizar comparaciones de valores de tipo numérico o carácter. Los operadores de relación sirven para expresar las condiciones en los algoritmos.

Los operadores de relación se recogen en la Tabla 1.2.

El formato general para las comparaciones es

```
expresion1    OPERADOR DE RELACION    expresion2
```

Tabla 1.2. Operadores de relación

Operador	Significado
<	menor que
>	mayor que
=	igual que
<=	menor o igual que
>=	mayor o igual que
<>	distinto de

y el resultado de la operación será verdadero o falso. Así, por ejemplo, si `A = 4` y `B = 3`, entonces

```
A > B                    es verdad
```

mientras que

```
(A - 2) < (B - 4)        es falso
```

Los operadores de relación se pueden aplicar a cualquiera de los cuatro tipos de datos estándar: *enteros, real, lógico, carácter.*

La aplicación a valores numéricos es evidente. Los ejemplos siguientes son significativos:

N1	N2	Expresión lógica	Resultado
3	6	`3 < 6`	*verdadero*
0	1	`0 > 1`	*falso*
4	2	`4 = 2`	*falso*
8	5	`8 <= 5`	*falso*
9	9	`9 >= 9`	*verdadero*
5	5	`5 <> 5`	*falso*

Para realizar comparaciones de datos tipo carácter, se requiere una secuencia de ordenación de los caracteres, similar al orden creciente o decreciente. Esta ordenación suele ser alfabética, tanto mayúsculas como minúsculas, y numérica, considerándolas de modo independiente. Pero si se consideran caracteres mixtos, se debe recurrir a un código normalizado como es el ASCII (*véase* Apéndice A). Aunque no todas las computadoras siguen el código normalizado en su juego completo de caracteres, sí son prácticamente estándar los códigos de los caracteres alfanuméricos más usuales.

Estos códigos normalizados son:

- Los caracteres especiales `#, %, $, (, ), +, -, /, . . .`, exigen la consulta del código de ordenación.
- Los valores de los caracteres que representan a los dígitos están en su orden natural. Esto es, `'0'<'1', '1'<'2', . . . , '8'<'9'`.
- Las letras mayúsculas A a Z siguen el orden alfabético (`'A'<'B', 'C'<'F'`, etc.).
- Si existen letras minúsculas, éstas siguen el mismo criterio alfabético (`'a'<'b', 'c'<'h'`, etcétera).

En general, los cuatro grupos anteriores están situados en el código ASCII en orden creciente. Así, `'1'<'A'` y `'B'<'C'`. Sin embargo, para tener completa seguridad será preciso consultar el código de caracteres de su computadora (normalmente, el **ASCII** (*American Standar Code for Information Interchange*) o bien el **EBCDIC** (*Extended Binary-Coded Decimal Interchange Code*), utilizado en computadoras IBM diferentes a los modelos PC y PS/2).

Cuando se utilizan los operadores de relación, con valores lógicos, la constante *false* (*falso*) es menor que la constante *true* (*verdad*).

```
false < true      falso < verdad
true > false      verdad > falso
```

Cuando se utilizan los operadores relacionales = y <> para comparar cantidades numéricas, es importante recordar que la mayoría de los *valores reales no pueden ser almacenados exactamente*. En consecuencia, las expresiones lógicas formales con comparación de cantidades reales con (=), a veces se evalúan como falsas, incluso aunque estas cantidades sean algebraicamente iguales. Así,

```
(1.0 / 3.0) * 3.0 = 1.0
```

teóricamente es verdadera y, sin embargo, al realizar el cálculo en una computadora se puede obtener .999999... y, en consecuencia, el resultado es falso; esto es debido a la precisión limitada de la aritmética real en las computadoras. Por consiguiente, a veces *deberá excluir las comparaciones con datos de tipo real*.

1.6.2.2. Operadores lógicos

Los *operadores lógicos* o *booleanos* básicos son **not** (*no*), **and** (*y*) y **or** (*o*). La Tabla 1.3 recoge el funcionamiento de dichos operadores.

Las definiciones de las operaciones **no**, **y**, **o** se resumen en unas tablas conocidas como *tablas de verdad*.

a	no a
verdad	falso
falso	verdad

no (6 > 10) es *verdad*
ya que (6 > 10) es *falso*

a	b	a y b
verdad	verdad	verdad
verdad	falso	falso
falso	verdad	falso
falso	falso	falso

a **y** b es *verdad* solo
si a y b son *verdad*

a	b	a o b
verdad	verdad	verdad
verdad	falso	verdad
falso	verdad	verdad
falso	falso	falso

a **o** b son *verdad*
cuando a, b o ambas
son *verdad*

Tabla 1.3. Operadores lógicos

Operador lógico	Expresión lógica	Significado
no (*not*)	**no** p (*not p*)	negación de *p*
y (*and*)	p **y** q (p *and* q)	conjunción de *p* y *q*
o (*or*)	p **o** q (p *or* q)	disyunción de *p* y *q*

En las expresiones lógicas se pueden mezclar operadores de relación y lógicos. Así, por ejemplo,

```
(1 < 5) y (5 < 10)        es verdad
(5 > 10) o ('A' < 'B')    es verdad, ya que 'A' < 'B'
```

Ejemplo 1.10

La Tabla 1.4 resume una serie de aplicaciones de expresiones lógicas.

1.6.2.3. Prioridad de los operadores en las expresiones lógicas

Los operadores aritméticos seguían un orden específico de prioridad cuando existían más de un operador en las expresiones. De modo similar, los operadores lógicos y relaciones tienen un orden de prioridad.

Tabla 1.4. Expresiones lógicas

Expresión lógica	Resultado	Observaciones
(1 > 0) y (3 = 3)	verdad	
no PRUEBA	verdad	• PRUEBA es un valor lógico *falso*.
(0 < 5) o (0 > 5)	verdad	
(5 <= 7) y (2 > 4)	falso	
no (5 <> 5)	verdad	
(numero = 1) o (7 >= 4)	verdad	• numero es una variable entera de valor 5

Tabla 1.5. Prioridad de operadores (lenguaje Pascal)

Operador	Prioridad
no (*not*)	*más alta* (primera operación ejecutada)
/, *, div, mod, y (*and*)	↓
+, -, o (*or*)	↓
<, >, =, <=, >=, <>	*más baja* (última operación ejecutada)

Tabla 1.6. Prioridad de operadores (lenguaje BASIC)

Operador	Prioridad
^	*más baja*
*, /	↓
div (\)	↓
mod	↓
+, -	↓
<, >, =, <>, <=, >=	↓
no (*not*)	↓
y (*and*)	↓
o (*or*)	*más alta*

Al igual que en las expresiones aritméticas, los paréntesis se pueden utilizar y tendrán prioridad sobre cualquier operación.

Ejemplo 1.11

```
no 4 > 6                    produce un error, ya que el operador no se aplica a 4
no (4 > 14)                 produce un valor verdadero
(1.0 < x) y (x < z + 7.0)   si x vale 7 y z vale 4, se obtiene un valor verdadero
```

1.7. FUNCIONES INTERNAS

Las operaciones que se requieren en los programas exigen en numerosas ocasiones, además de las operaciones de las operaciones aritméticas básicas, ya tratadas, un número determinado de operadores especiales que se denominan *funciones internas*, incorporadas o estándar. Por ejemplo, la función `ln` se puede utilizar para determinar el logaritmo neperiano de un número y la función `raiz2 (sqrt)` calcula la raíz cuadrada de un número positivo. Existen otras funciones que se utilizan para determinar las funciones trigonométricas.

La Tabla 1.7 recoge las funciones internas más usuales, siendo x el argumento de la función.

Tabla 1.7. Funciones internas

Función	Descripción	Tipo de argumento	Resultado
`abs(x)`	valor absoluto de x	entero o real	igual que argumento
`arctan(x)`	arco tangente de x	entero o real	real
`cos(x)`	coseno de x	entero o real	real
`exp(x)`	exponencial de x	entero o real	real
`ln(x)`	logaritmo neperiano de x	entero o real	real
`log10(x)`	logaritmo decimal de x	entero o real	real
`redondeo(x)` `(round(x))*`	redondeo de x	real	entero
`sen(x)` `(sin(x))*`	seno de x	entero o real	real
`cuadrado(x)` `(sqr(x))*`	cuadrado de x	entero o real	igual que argumento
`raiz2(x)` `(sqrt(x))*`	raíz cuadrada de x	entero o real	real
`trunc(x)`	truncamiento de x	real	entero

* Terminología en inglés.

Ejemplo 1.12

Las funciones aceptan argumentos reales o enteros y sus resultados dependen de la tarea que realice la función:

Expresión	Resultado
`raiz2 (25)`	5
`redondeo (6.6)`	7
`redondeo (3.1)`	3
`redondeo (-3.2)`	-3
`trunc (5.6)`	5
`trunc (3.1)`	3
`trunc (-3.8)`	-3
`cuadrado (4)`	16
`abs (9)`	9
`abs (-12)`	12

Ejemplo 1.13

Utilizar las funciones internas para obtener la solución de la ecuación cuadrática $ax^2 + bx + c = 0$.

Las raíces de la ecuación son:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

o lo que es igual:

$$x1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \qquad x2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

Las expresiones se escriben como

```
x1 = (-b + raiz2 (cuadrado(b) - 4 * a * c)) / (2 * a)
x2 = (-b - raiz2 (cuadrado(b) - 4 * a * c)) / (2 * a)
```

Si el valor de la expresión

```
raiz2(cuadrado(b) - 4 * a * c)
```

es negativo se producirá un error, ya que la raíz cuadrada de un número negativo no está definida.

1.8. LA OPERACIÓN DE ASIGNACIÓN

La operación de asignación es el modo de darle valores a una variable. La operación de asignación se representa con el símbolo u operador ←. La operación de asignación se conoce como *instrucción* o *sentencia* de asignación cuando se refiere a un lenguaje de programación.

El formato general de una operación de asignación es:

```
nombre de la variable ← expresion
```

La flecha (operador de asignación) se sustituye en otros lenguajes por = (BASIC, FORTRAN) o : = (Pascal) C/C++. Sin embargo, es preferible el uso de la flecha en la redacción del algoritmo para evitar ambigüedades, dejando el uso del símbolo = exclusivamente para el operador de igualdad.

La operación de asignación:

```
A ← 5
```

significa que a la variable A se le ha asignado el valor 5.

La acción de asignar es *destructiva*, ya que el valor que tuviera la variable antes de la asignación se pierde y se reemplaza por el nuevo valor. Así, en la secuencia de operaciones

```
A ← 25
A ← 134
A ← 5
```

cuando éstas se ejecutan, el valor último que toma A será 5 (los valores 25 y 134 han desaparecido).

La computadora ejecuta la sentencia de asignación en dos pasos. En el primero de ellos se calcula el valor de la expresión al lado derecho del operador, obteniéndose un valor de un tipo específico. En el segundo paso, este valor se almacena en la variable cuyo nombre aparece a la izquierda del operador de asignación, sustituyendo al valor que tenía anteriormente.

```
X ← Y + 2
```

el valor de la expresión Y + 2 se asigna a la variable X.

Es posible utilizar el mismo nombre de variable en ambos lados del operador de asignación. Por ello, acciones como

```
N ← N + 1
```

tienen sentido; se determina el valor actual de la variable N, se incrementa en 1 y a continuación el resultado se asigna a la misma variable N. Sin embargo, desde el punto de vista matemático no tiene sentido N ← N + 1.

Las acciones de asignación se clasifican según sea el tipo de expresiones en: *aritméticas*, *lógicas* y *de caracteres*.

1.8.1. Asignación aritmética

Las expresiones en las operaciones de asignación son aritméticas:

```
AMN ← 3 + 14 + 8
```

se evalúa la expresión 3 + 14 + 8 y se asigna a la variable AMN, es decir, 25 será el valor que toma AMN

```
TER1 ← 14.5 + 8
TER2 ← 0.75 * 3.4
COCIENTE ← TER1/TER2
```

Se evalúan las expresiones `14.5 + 8` y `0.75 * 3.4` y en la tercera acción se dividen los resultados de cada expresión y se asigna a la variable `COCIENTE`, es decir, las tres operaciones equivalen a `COCIENTE ← (14.5 + 8)/(0.75 * 3.4)`.

Otro ejemplo donde se pueden comprender las modificaciones de los valores almacenados en una variable es el siguiente:

```
A ← 0          la variable A toma el valor 0
N ← 0          la variable N toma el valor 0
A ← N + 1      la variable A toma el valor 0 + 1, es decir, 1
```

El ejemplo anterior se puede modificar para considerar la misma variable en ambos lados del operador de asignación:

```
N ← 2
N ← N + 1
```

En la primera acción `N` toma el valor `2` y en la segunda se evalúa la expresión `N + 1`, que tomará el valor `2 + 1 = 3` y se asignará nuevamente a `N`, que tomará el valor `3`.

1.8.2. Asignación lógica

La expresión que se evalúa en la operación de asignación es lógica. Supóngase que `M`, `N` y `P` son variables de tipo lógico.

```
M ← 8 < 5
N ← M o (7 <= 12)
P ← 7 > 6
```

Tras evaluar las operaciones anteriores, las variables `M`, `N` y `P` tomarán los valores *falso*, *verdad*, *verdad*.

1.8.3. Asignación de cadenas de caracteres

La expresión que se evalúa es de tipo cadena:

```
x ← '12 de octubre de 1942'
```

La acción de asignación anterior asigna la cadena de caracteres `'12 de octubre de 1942'` a la variable tipo cadena `x`.

1.8.4. Conversión de tipo

En las asignaciones no se pueden asignar valores a una variable de un tipo diferente del suyo. Se presentará un error si se trata de asignar valores de tipo carácter a una variable numérica o un valor numérico a una variable tipo carácter.

Ejemplo 1.14

¿Cuáles son los valores de A, B *y* C *después de la ejecución de las siguientes operaciones?*

```
A ← 3
B ← 4
C ← A + 2 * B
C ← C + B
B ← C - A
A ← B * C
```

En las dos primeras acciones A y B toman los valores 3 y 4.

```
C ← A + 2 * B      la expresión A + 2 * B tomará el valor 3 + 2 * 4 = 3 + 8 = 11
C ← 11
```

La siguiente acción

```
C ← C + B
```

producirá un valor de `11 + 4 = 15`

```
C ← 15
```

En la acción `B ← C - A` se obtiene para B el valor `15 - 3 = 12` y por último:

```
A ← B * C
```

A tomará el valor `B * C`, es decir, `12 * 15 = 180`; por consiguiente, el último valor que toma A será `180`.

Ejemplo 1.15

¿Cuál es el valor de x después de las siguientes operaciones?

```
x ← 2
x ← cuadrado(x + x)
x ← raiz2(x + raiz2(x) + 5)
```

Los resultados de cada expresión son:

```
x ← 2                    x toma el valor 2
x ← cuadrado(2 + 2)      x toma el valor 4 al cuadrado; es decir, 16

x ← raiz2(16 + raiz2(16) + 5)
```

en esta expresión se evalúa primero **`raiz2(16)`**, que produce 4 y, por último, **`raiz2(16+4+5)`** proporciona **`raiz2(25)`**, es decir, 5.

Los resultados de las expresiones sucesivas anteriores son:

```
x ← 2
x ← 16
x ← 5
```

1.9. ENTRADA Y SALIDA DE INFORMACIÓN

Los cálculos que realizan las computadoras requieren para ser útiles la *entrada* de los datos necesarios para ejecutar las operaciones que posteriormente se convertirán en resultados, es decir, *salida*.

Las operaciones de entrada permiten leer determinados valores y asignarlos a determinadas variables. Esta entrada se conoce como operación de **lectura** (*read*). Los datos de entrada se introducen al procesador mediante dispositivos de entrada (teclado, tarjetas perforadas, unidades de disco, etc.). La salida puede aparecer en un dispositivo de salida (pantalla, impresora, etc.). La operación de salida se denomina **escritura** (*write*).

En la escritura de algoritmos las acciones de lectura y escritura se representan por los formatos siguientes:

```
leer (lista de variables de entrada)
escribir (lista de expresiones de salida)
```

Así, por ejemplo:

```
leer (A, B, C)
```

representa la lectura de tres valores de entrada que se asignan a las variables A, B y C.

```
escribir ('hola Vargas')
```

visualiza en la pantalla —o escribe en el dispositivo de salida— el mensaje 'hola Vargas'.

NOTA 1: Si se utilizaran las palabras reservadas en inglés, como suele ocurrir en los lenguajes de programación, se deberá sustituir

```
leer        escribir
```

por

```
read        write o bien print
```

NOTA 2: Si no se especifica el tipo de dispositivo del cual se leen o escriben datos, los dispositivos de E/S por defecto son el teclado y la pantalla.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

1.1. Diseñar un algoritmo para cambiar una rueda de un coche.

Algoritmo pinchazo

1. Inicio.
2. Si gato del coche está averiado llamar a la estación de servicio y bifurcar al punto 11; en caso contrario, continuar el proceso.
3. Levantar el coche con el gato.
4. Aflojar y sacar los tornillos de las ruedas.
5. Si NO todos los tornillos estan flojos y quitados, bifurcar al paso 4; en caso contrario, continuar el proceso.
6. Quitar la rueda.

7. Poner rueda de repuesto.
8. Poner los tornillos y apretarlos.
9. Si NO están puestos todos los tornillos volver al paso 8, en caso contrario, continuar el proceso.
10. Bajar el gato.
11. Fin.

1.2. Encontrar el valor de la variable VALOR después de la ejecución de las siguientes operaciones:

```
(a) VALOR ← 4.0 * 5
(b) X ← 3.0
    Y ← 2.0
    VALOR ← X ^ Y - Y
(c) VALOR ← 5
    X ← 3
    VALOR ← VALOR * X
```

```
(a) VALOR = 20.0
(b) X = 3.0
    Y = 2.0
    VALOR = 3 ^ 2 - 2 = 9 - 2 = 7   VALOR = 7
(c) VALOR = 5
    X = 3
    VALOR = VALOR * X = 5 * 3 = 15  VALOR = 15
```

1.3. Deducir el resultado que se produce con las siguientes instrucciones:

```
var entero: X, Y
X ← 1
Y ← 5
escribir(X, Y)
```

X e Y toman los valores 1 y 5.

La instrucción de salida **(escribir)** presentará en el dispositivo de salida 1 y 5, con los formatos específicos del lenguaje de programación; por ejemplo,

```
1    5
```

1.4. Deducir el valor de las expresiones siguientes:

```
X ← A + B + C
X ← A + B * C
X ← A + B / C
X ← A + B \ C
X ← A + B mod C
X ← (A + B) \ C
X ← A + (B / C)
Siendo A = 5   B = 25   C = 10
```

Expresión	X
A + B + C = 5 + 25 + 10	40
A + B * C = 5 + 25 * 10	255
A + B / C = 5 + 25 / 10	7.5
A + B \ C = 5 + 25 \ 10 = 5 + 2	7
A + B mod C = 5 + 25 mod 10 = 5 + 5	10
(A + B) / C = (5 + 25) / 10 = 30 / 10	3
A + (B / C) = 5 + (25 / 10) = 5 + 2.5	7.5

1.5. Escribir las siguientes expresiones en forma de expresiones algorítmicas:

$a)\ \frac{M}{N}+P$ $\quad b)\ M+\frac{N}{P-Q}$ $\quad c)\ \frac{\text{sen } o(x)+\cos eno(x)}{tan(x)}$

$d)\ \frac{m+n}{p-q}$ $\quad e)\ \frac{m+\frac{n}{p}}{q-\frac{r}{5}}$ $\quad f)\ \frac{-b+\sqrt{b^2}-4ac}{2a}$

a) $M / N + P$
b) $M + N / (P - Q)$
c) $(SEN(X) + COS(X)) / TAN(X)$
d) $(M + N) / (P - Q)$
e) $(M + N / P) / (Q - R / 5)$
f) $(-B + \textbf{raiz2}\ (B \wedge 2 - 4 * A * C)) / (2 * A)$

1.6. Calcular el valor de las siguientes expresiones:

a) `8 + 7 * 3 + 4 * 6`
b) `- 2 ^ 3`
c) `(33 + 3 * 4) / 5`
d) `2 ^ 2 * 3`
e) `3 + 2 * (18 - 4 ^ 2)`
f) `16 * 6 - 3 * 2`

```
a) 8 + 7 * 3 + 4 * 6
     8 + 21      24
       29   +   24
            53

b) -2 ^ 3
     -8

c) (33 + 3 * 4) / 5
     33 +   12   / 5
          45     / 5
                 9

d) 2 ^ 2 * 3
     4 * 3
       12

e) 3 + 2 * (18 - 4 ^ 2)
   3 + 2 * (18 -  16)
   3 + 2 *      2
   3 +      4
            7
```

```
f) 16 * 6 - 3 * 2
     96   -   6
          90
```

1.7. Cómo se intercambian los valores de dos variables, A y B.

El procedimiento para conseguir intercambiar los valores de dos variables entre sí debe recurrir a una variable AUX y a las instrucciones asignación siguientes:

```
AUX ← A
  A ← B
  B ← AUX
```

Veámoslo con el ejemplo del ejercicio, donde

```
A ← 10
```

Instrucción	A	B	AUX	Observaciones
A ← 10	10	—	—	
B ← 5	10	5	—	
AUX ← A	10	5	10	La variable AUX toma el valor de A
A ← B	5	5	10	A toma el valor de B, 5
B ← AUX	5	10	10	B toma el valor *inicial* de A, 10

Ahora A = 5 y B = 10.

1.8. Se tienen tres variables A, B y C. Escribir las instrucciones necesarias para intercambiar entre sí sus valores del modo siguiente:

- B toma el valor de A
- C toma el valor de B
- A toma el valor de C

NOTA: Sólo se debe utilizar una variable auxiliar.

Utilizaremos una variable auxiliar AUX.
Las instrucciones que resuelven el problema de intercambio son:

```
AUX ← A
  A ← C
  C ← B
  B ← AUX
```

Comprobémoslo con los valores de A, B y C: 5, 10 y 15.

	Instrucción	A	B	C	AUX	Observaciones
(1)	A ← 5	5	—	—	—	
(2)	B ← 10	—	10	—	—	
(3)	C ← 15	—	—	15	—	
	AUX ← A	5	10	15	5	
	A ← C	15	10	15	5	
	C ← B	15	10	10	5	
	B ← AUX	15	5.	10	5	

Obsérvese que al igual que en el ejercicio de intercambio de valores entre dos variables, la variable AUX no modifica su valor.

1.9. Deducir el valor que toma la variable tras la ejecución de las instrucciones:

```
A ← 4
B ← A
B ← A + 3
```

Mediante una tabla se da un método eficaz para obtener los sucesivos valores:

```
                        A   B
(1)  A ← A              4   –
(2)  B ← A              4   4
(3)  B ← A + 3          4   7
```

Después de la instrucción (1) la variable A contiene el valor 4.
La variable B no ha tomado todavía ningún valor y se representa esa situación con un guión.
La instrucción (2) asigna el valor actual de A (4) a la variable B.
La instrucción (3) efectúa el cálculo de la expresión A + 3, lo que produce un resultado de 7 (4 + 3) y este valor se asigna a la variable B, cuyo último valor (4) se destruye.
Por consiguiente, los valores finales que tienen las variables A y B son:

```
A = 4        B = 7
```

1.10. ¿Qué se obtiene en las variables A y B, después de la ejecución de las siguientes instrucciones?

```
A ← 5
B ← A + 6
A ← A + 1
B ← A - 5
```

Siguiendo las directrices del ejercicio anterior:

	Instrucción			Observaciones
(1)	A ← 5	5	–	B no toma ningún valor
(2)	B ← A + 6	5	11	Se evalúa A + 6(5 + 6) y se asigna a B
(3)	A ← A + 1	6	11	Se evalúa A + 1(5 + 1) y se asigna a A, borrándose el valor que tenía (5) y tomando el nuevo valor (6)
(4)	B ← A - 5	6	1	Se evalúa A - 5(6 - 1) y se asigna a B

Los valores últimos de A y B son: A = 6, B = 1.

1.11. ¿Qué se obtiene en las variables A, B y C después de ejecutar las instrucciones siguientes?

```
A ← 3
B ← 20
C ← A + B
B ← A + B
A ← B - C
```

	Instrucción	A	B	C	Observaciones
(1)	A ← 3	3	–	–	B y C no toman ningún valor
(2)	B ← 20	3	20	–	C sigue sin valor
(3)	C ← A + B	3	20	23	Se evalúa A + B (20 + 3) y se asigna a C
(4)	B ← A + B	3	23	23	Se evalúa A + B (20 + 3) y se asigna a B; destruye el valor antiguo (20)
(5)	A ← B - C	0	23	23	Se evalúa B - C(23 - 23) y se asigna a A

Los valores finales de las variables son:

```
A = 0      B = 23      C = 23
```

1.12. ¿Qué se obtiene en A y B tras la ejecución de

```
A ← 10
B ← 5
A ← B
B ← A
```

	Instrucción	A	B	Observaciones
(1)	A ← 10	10	–	B no toma valor
(2)	B ← 5	10	5	B recibe el valor inicial 5
(3)	A ← B	5	5	A toma el valor de B(5)
(4)	B ← A	5	5	B toma el valor actual de A(5)

Los valores finales de A y B son 5.
En este caso se podría decir que la instrucción (4) B ← A es *redundante* respecto a las anteriores, ya que su ejecución no afecta al valor de las variables.

1.13. Determinar el mayor de tres números enteros.

Los pasos a seguir son:

1. Comparar el primero y el segundo entero, deduciendo cuál es el mayor.
2. Comparar el mayor anterior con el tercero y deducir cuál es el mayor. Éste será el resultado.

Los pasos anteriores se pueden descomponer en otros pasos más simples en lo que se denomina *refinamiento del algoritmo*.

1. Obtener el primer número (entrada), denominarlo NUM1.
2. Obtener el segundo número (entrada), denominarlo NUM2.
3. Comparar NUM1 con NUM2 y seleccionar el mayor; si los dos enteros son iguales, seleccionar NUM1. Llamar a este número MAYOR.
4. Obtener el tercer número (entrada) y denominarlo NUM3.
5. Comparar MAYOR con NUM3 y seleccionar el mayor; si los dos enteros son iguales, seleccionar el MAYOR. Denominar a este número MAYOR.
6. Presentar el valor de MAYOR (salida).
7. Fin.

EJERCICIOS

1.1. Diseñar los algoritmos que resuelvan los siguientes problemas:

a) Ir al cine.
b) Comprar una entrada para los toros.
c) Colocar la mesa para comer.
d) Cocer un huevo.
e) Hacer una taza de té.
f) Fregar los platos del almuerzo.
g) Buscar el número de teléfono de un alumno.
h) Reparar un pinchazo de una bicicleta.
i) Pagar una multa de tráfico.
j) Cambiar un neumático pinchado (se dispone de herramientas y gato).
k) Hacer palomitas de maíz en una olla puesta al fuego con aceite, sal y maíz.
l) Cambiar el cristal roto de una ventana.
m) Hacer una llamada telefónica. Considerar los casos: *a)* manual, con operadora; *b)* automático; *c)* cobro revertido.
n) Quitar una bombilla quemada de un techo.
o) Encontrar la media de una lista indeterminada de números positivos terminada con un número negativo.

1.2. ¿Cuáles de los siguientes identificadores no son válidos?

a) `XRayo`	*b)* `X_Rayo`
c) `R2D2`	*d)* `X`
e) `45`	*f)* `N14`
g) `ZZZZ`	*h)* `3µ`

1.3. ¿Cuáles de las siguientes constantes no son válidas?

a) `234`	*b)* `-8.975`
c) `12E - 5`	*d)* `0`
e) `32,767`	*f)* `1/2`
g) `3.6E + 7`	*h)* `-7E12`
i) `3.5 x 10`	*j)* `0,456`
k) `0.000001`	*l)* `224E1`

1.4. Evaluar la siguiente expresión para `A = 2` y `B = 5`:

```
3 * A - 4 * B / A ^ 2
```

1.5. Evaluar la expresión

```
4 / 2 * 3 / 6 + 6 / 2 / 1 / 5 ^ 2 / 4 * 2
```

1.6. Escribir las siguientes expresiones algebraicas como expresiones algorítmicas:

a) $\sqrt{b^2} - 4ac$	*b)* $\frac{x^2 + y^2}{z^2}$	*c)* $\frac{3x + 2y}{2z}$	*d)* $\frac{a+b}{c-d}$
e) $4x^2 - 2x + 7$	*f)* $\frac{x+y}{x} - \frac{3x}{5}$	*g)* $\frac{a}{bc}$	*h)* xyz
i) $\frac{y_2 - y_1}{x_2 - x_1}$	*j)* $2\pi r$	*k)* $\frac{4}{3}\pi r^3$	*l)* $(x_2 - x_1)^2 + (y_2 - y_1)^2$

1.7. Escribir las siguientes expresiones algorítmicas como expresiones algebraicas:

a) `b ^ 2 - 4 * a * c`
b) `3 * X ^ 4 - 5 * X ^ 3 + X * 12 - 17`
c) `(b + d) / (c + 4)`
d) `(x ^ 2 + y ^ 2) ^ (1 / 2)`

1.8. Si el valor de `A` es `4`, el valor de `B` es `5` y el valor de `C` es `1`, evaluar las siguientes expresiones:

a) `B * A - B ^ 2 / 4 * C`
b) `(A * B) / 3 ^ 2`
c) `(((B + C) / 2 * A + 10) * 3 * B) - 6`

1.9. Si el valor de `A` es `2`, `B` es `3` y `C` es `2`, evaluar la expresión:

```
a ^ b ^ c
```

1.10. Obtener el valor de cada una de las siguientes expresiones aritméticas:

a) 7 **div** 2
b) 7 **mod** 2
c) 12 **div** 3
d) 12 **mod** 3
e) 0 **mod** 5
f) 15 **mod** 5
g) 7 * 10 – 50 **mod** 3 * 4 + 9
h) (7 * (10 – 5) **mod** 3) * 4 + 9

NOTA: Considérese la prioridad de Pascal: *más alta*: *****, **/**, **div**, **mod**; *más baja*: **+**, **-**.

1.11. Encontrar el valor de cada una de las siguientes expresiones o decir si no es una expresión válida:

a) 9 - 5 - 3
b) 2 **div** 3 + 3 / 5
c) 9 **div** 2 / 5
d) 7 **mod** 5 **mod** 3
e) 7 **mod** (5 **mod** 3)
f) (7 **mod** 5) **mod** 3
g) (7 **mod** 5 **mod** 3)
h) ((12 + 3) **div** 2) / (8 - (5 + 1))
i) 12 / 2 * 3
j) **raiz2** (**cuadrado**(4))
k) **cuadrado** (**raiz2**(4))
l) **trunc**(81.5) + **redondeo**(81.5)

Considérese la prioridad del E jercicio 1.10.

CAPÍTULO 2

LA RESOLUCIÓN DE PROBLEMAS CON COMPUTADORAS Y LAS HERRAMIENTAS DE PROGRAMACIÓN

CONTENIDO

2.1. La resolución de problemas.
2.2. Análisis del problema.
2.3. Diseño del algoritmo.
2.4. Resolución del problema mediante computadora.
2.5. Representación gráfica de los algoritmos.
2.6. Diagramas Nassi-Schneiderman (N-S).
2.7. Pseudocódigo.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

La resolución de problemas con computadoras se puede dividir en tres fases:

- *análisis del problema,*
- *diseño del algoritmo,*
- *resolución del algoritmo en la computadora.*

En este capítulo se analizan las tres fases anteriores. El análisis y el diseño del algoritmo requiere la descripción del problema en subproblemas a base de «refinamientos sucesivos» y una herramienta de programación —diagrama de flujo, diagrama N-S o pseudocódigo—; *los conceptos fundamentales del análisis, diseño y herramientas de programación (diagramas de flujo, diagramas N-S y pseudocódigos)* se describen como conocimientos indispensables para el aprendizaje de la programación de computadoras.

2.1. LA RESOLUCIÓN DE PROBLEMAS

La principal razón para que las personas aprendan a programar en general y los lenguajes de programación en particular es utilizar la computadora como una herramienta para la resolución de problemas. Ayudado por una computadora, la resolución de un problema se puede dividir en tres fases importantes:

1. *análisis del problema,*
2. *diseño o desarrollo del algoritmo,*
3. *resolución del algoritmo en la computadora.*

El primer paso —análisis del problema— requiere que el problema sea definido y comprendido claramente para que pueda ser analizado con todo detalle. Una vez analizado el problema, se debe desarrollar el algoritmo —procedimiento paso a paso para solucionar el problema dado—. Por último, para resolver el algoritmo mediante una computadora se necesita codificar el algoritmo en un lenguaje de programación Pascal, C/C++, COBOL, FORTRAN, etc., es decir, convertir el algoritmo en programa, ejecutarlo y comprobar que el programa soluciona verdaderamente el problema. Las fases del proceso de resolución de un problema mediante computadora se indican en la Figura 2.1.

2.2. ANÁLISIS DEL PROBLEMA

El propósito del análisis de un problema es ayudar al programador para llegar a una cierta comprensión de la naturaleza del problema. El problema debe estar bien definido si se desea llegar a una solución satisfactoria.

Para poder definir con precisión el problema se requiere que las especificaciones de entrada y salida sean descritas con detalle. Una buena definición del problema, junto con una descripción detallada de las especificaciones de entrada y salida, son los requisitos más importantes para llegar a una solución eficaz.

El análisis del problema exige una lectura previa del problema a fin de obtener una idea general de lo que se solicita. La segunda lectura deberá servir para responder a las preguntas:

- ¿Qué información debe proporcionar la resolución del problema?
- ¿Qué datos se necesitan para resolver el problema?

La respuesta a la primera pregunta indicará los resultados deseados o las *salidas del problema*. La respuesta a la segunda pregunta indicará qué datos se proporcionan o las *entradas del problema*.

Figura 2.1. La resolución de un problema.

Figura 2.2. Análisis del problema.

Ejemplo 2.1

Leer el radio de un círculo y calcular e imprimir su superficie y la longitud de la circunferencia.

Análisis

Las entradas de datos en este problema se concretan en el radio del círculo. Dado que el radio puede tomar cualquier valor dentro del rango de los números reales, el tipo de datos radio debe ser real.

Las salidas serán dos variables: superficie y circunferencia, que también serán de tipo real.

Entradas:	Radio del círculo (variable `RADIO`).
Salidas:	Superficie del círculo (variable `AREA`).
	Circunferencia del círculo (variable `CIRCUNFERENCIA`).
Variables:	`RADIO`, `AREA`, `CIRCUNFERENCIA` (tipo real).

2.3. DISEÑO DEL ALGORITMO

Una computadora no tiene capacidad para solucionar problemas más que cuando se le proporcionan los sucesivos pasos a realizar. Estos pasos sucesivos que indican las instrucciones a ejecutar por la máquina constituyen, como ya conocemos, el *algoritmo*.

La información proporcionada al algoritmo constituye su *entrada* y la información producida por el algoritmo constituye su *salida*.

Los problemas complejos se pueden resolver más eficazmente con la computadora cuando se rompen en subproblemas que sean más fáciles de solucionar que el original. Este método se suele denominar *divide y vencerás* (*divide and conquer*) y consiste en dividir un problema complejo en otros más simples. Así, el problema de encontrar la superficie y la longitud de un círculo se puede dividir en tres problemas más simples o *subproblemas* (Figura 2.3).

La descomposición del problema original en subproblemas más simples y a continuación dividir estos subproblemas en otros más simples que pueden ser implementados para su solución en la computadora se denomina *diseño descendente* (*top-down design*). Normalmente los pasos diseñados en el primer esbozo del algoritmo son incompletos e indicarán sólo unos pocos pasos (un máximo de doce aproximadamente). Tras esta primera descripción, éstos se amplían en una descripción más detallada con más pasos específicos. Este proceso se denomina *refinamiento del algoritmo* (*stepwise refinement*). Para problemas complejos se necesitan con frecuencia diferentes *niveles de refinamiento* antes de que se pueda obtener un algoritmo claro, preciso y completo.

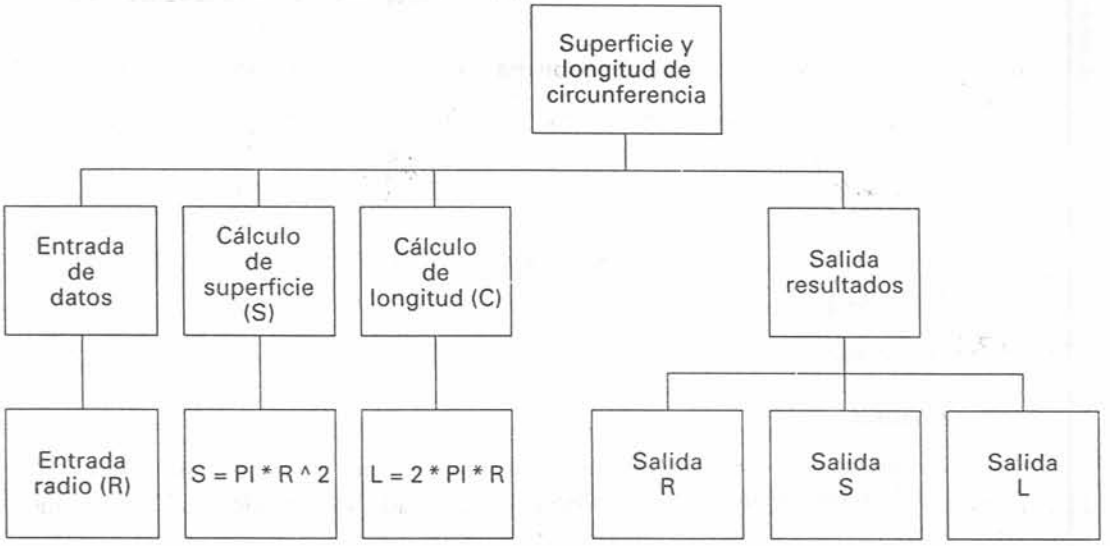


Figura 2.3. Refinamiento de un algoritmo.

El problema de cálculo de la circunferencia y superficie de un círculo se puede descomponer en subproblemas más simples: (1) leer datos de entrada, (2) calcular superficie y longitud de circunferencia, y (3) escribir resultados (datos de salida).

Subproblema	Refinamiento
leer radio	leer radio
calcular superficie	superficie = 3.141592 * radio ^ 2
calcular circunferencia	circunferencia = 2 * 3.141592 * radio
escribir resultados	*escribir* radio, circunferencia, superficie

Las *ventajas* más importantes del diseño descendente son:

- El problema se comprende más fácilmente al dividirse en partes más simples denominadas *módulos*.
- Las modificaciones en los módulos son más fáciles.
- La comprobación del problema se puede verificar fácilmente.

Tras los pasos anteriores (*diseño descendente y refinamiento por pasos*) es preciso representar el algoritmo mediante una determinada herramienta de programación: *diagrama de flujo*, *pseudocódigo* o *diagrama N-S*.

Así pues, el diseño del algoritmo se descompone en las fases recogidas en la Figura 2.4.

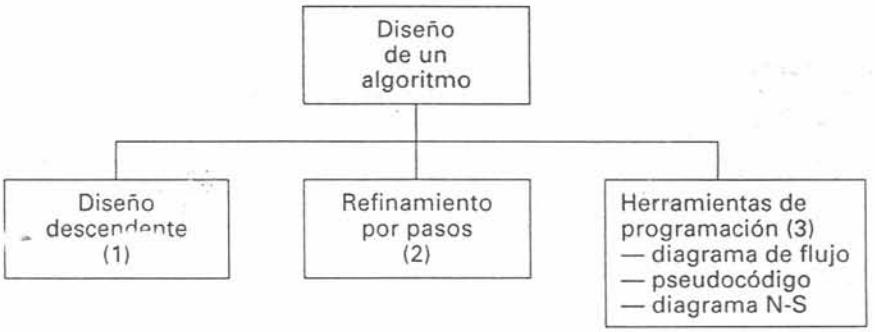


Figura 2.4. Fases del diseño de un algoritmo.

2.3.1. Escritura inicial del algoritmo

Como ya se ha comentado anteriormente, el sistema para describir («escribir») un algoritmo consiste en realizar una descripción paso a paso con un lenguaje natural del citado algoritmo. Recordemos que un algoritmo es un método o conjunto de reglas para solucionar un problema. En cálculos elementales estas reglas tienen las siguientes propiedades:

- deben estar seguidas de alguna secuencia definida de pasos hasta que se obtenga un resultado coherente,
- sólo puede ejecutarse una operación a la vez.

El flujo de control usual de un algoritmo es secuencial; consideremos el algoritmo que responde a la pregunta

¿Qué hacer para ver la película Tiburón*?*

La respuesta es muy sencilla y puede ser descrita en forma de algoritmo general de modo similar a:

```
ir al cine
comprar una entrada (billete o ticket)
ver la pelicula
regresar a casa
```

El algoritmo consta de cuatro acciones básicas, cada una de las cuales debe ser ejecutada antes de realizar la siguiente. En términos de computadora, cada acción se codificará en una o varias sentencias que ejecutan una tarea particular.

El algoritmo descrito es muy sencillo; sin embargo, como ya se ha indicado en párrafos anteriores, el algoritmo general se descompondrá en pasos más simples en un procedimiento denominado *refinamiento sucesivo*, ya que cada acción puede descomponerse a su vez en otras acciones simples. Así, por ejemplo, un primer refinamiento del algoritmo `ir al cine` se puede describir de la forma siguiente:

```
1. inicio
2. ver la cartelera de cines en el periodico
```

```
3. si no proyectan «Tiburon» entonces
   3.1. decidir otra actividad
   3.2. bifurcar al paso 7
   si_no
   3.3. ir al cine
   fin_si
4. si hay cola entonces
   4.1. ponerse en ella
   4.2. mientras haya personas delante hacer
        4.2.1. avanzar en la cola
        fin_mientras
   fin_si
5. si hay localidades entonces
   5.1. comprar una entrada
   5.2. pasar a la sala
   5.3. localizar la(s) butaca(s)
   5.4. mientras proyectan la pelicula hacer
        5.4.1. ver la pelicula
        fin_mientras
   5.5. abandonar el cine
   si_no
   5.6. refunfuñar
   fin_si
6. volver a casa
7. fin
```

En el algoritmo anterior existen diferentes aspectos a considerar. En primer lugar, ciertas palabras reservadas se han escrito deliberadamente en negrita (**mientras, si no**, etc.). Estas palabras describen las estructuras de control fundamentales y procesos de toma de decisión en el algoritmo. Estas incluyen los conceptos importantes de *selección* (expresadas por **si-entonces-si_no** *if-then-else*) y de *repetición* (expresadas con **mientras-hacer** o a veces **repetir-hasta** e **iterar-fin-iterar**, en inglés, ***while-do*** y ***repeat-until***) que se encuentran en casi todos los algoritmos, especialmente los de proceso de datos. La capacidad de decisión permite seleccionar alternativas de acciones a seguir o bien la repetición una y otra vez de operaciones básicas.

```
si proyectan la pelicula seleccionada ir al cine
  si_no ver la television, ir al futbol o leer el periodico

mientras haya personas en la cola, ir avanzando repetidamente
  hasta llegar a la taquilla
```

Otro aspecto a considerar es el método elegido para describir los algoritmos: empleo de *indentación* (sangrado o justificación) en escritura de algoritmos. En la actualidad es tan importante la escritura de programa como su posterior lectura. Ello se facilita con la *indentación* de las acciones interiores a las estructuras fundamentales citadas: selectivas y repetitivas. A lo largo de todo el libro la indentación o sangrado de los algoritmos será norma constante.

Para terminar estas consideraciones iniciales sobre algoritmos, describiremos las acciones necesarias para refinar el algoritmo objeto de nuestro estudio; para ello analicemos la acción

```
Localizar las(s) butaca(s)
```

Si los números de los asientos están impresos en la entrada, la acción compuesta se resuelve con el siguiente algoritmo:

```
1. inicio //algoritmo para encontrar la butaca del espectador
2. caminar hasta llegar a la primera fila de butacas
3. repetir
     compara numero de fila con numero impreso en billete
     si no son iguales, entonces pasar a la siguiente fila
   hasta_que se localice la fila correcta
4. mientras numero de butaca no coincida con numero de billete
     hacer avanzar a traves de la fila a la siguiente butaca
   fin-mientras
5. sente
6. fin
```

En este algoritmo la repetición se ha mostrado de dos modos, utilizando ambas notaciones, **repetir... hasta_que** y **mientras... fin_mientras**. Se ha considerado también, como ocurre normalmente, que el número del asiento y fila coincide con el número y fila rotulado en el billete.

2.4. RESOLUCIÓN DEL PROBLEMA MEDIANTE COMPUTADORA

Una vez que el algoritmo está diseñado y representado gráficamente mediante una herramienta de programación (diagrama de flujo, pseudocódigo o diagrama N-S) se debe pasar a la fase de resolución práctica del problema con la computadora.

Esta fase se descompone a su vez en las siguientes subfases:

1. *codificación del algoritmo en un programa,*
2. *ejecución del programa,*
3. *comprobación del programa.*

En el diseño del algoritmo éste se describe en una herramienta de programación tal como un diagrama de flujo, diagrama N-S o pseudocódigo. Sin embargo, el programa que implementa el algoritmo debe ser escrito en un lenguaje de programación y siguiendo las reglas gramaticales o sintaxis del mismo. La fase de conversión del algoritmo en un lenguaje de programación se denomina *codificación*, ya que el algoritmo escrito en un lenguaje específico de programación se denomina *código*.

Tras la codificación del programa, deberá ejecutarse en una computadora y a continuación de comprobar los resultados pasar a la fase final de documentación.

Figura 2.5. Resolución del problema mediante computadora.

2.5. REPRESENTACIÓN GRÁFICA DE LOS ALGORITMOS

Para representar un algoritmo se debe utilizar algún método que permita independizar dicho algoritmo del lenguaje de programación elegido. Ello permitirá que un algoritmo pueda ser codificado indistintamente en cualquier lenguaje. Para conseguir este objetivo se precisa que el algoritmo sea representado gráfica o numéricamente, de modo que las sucesivas acciones no dependan de la sintaxis de ningún lenguaje de programación, sino que la descripción pueda servir fácilmente para su transformación en un programa, es decir, su *codificación*.

Los métodos usuales para representar un algoritmo son:

1. *diagrama de flujo,*
2. *diagrama N-S* (Nassi-Schneiderman),
3. *lenguaje de especificación de algoritmos:* ***pseudocódigo,***
4. *lenguaje español,*
5. *fórmulas.*

Los métodos 4 y 5 no suelen ser fáciles de transformar en programas. Una descripción en *español narrativo* no es satisfactoria, ya que es demasiado prolija y generalmente ambigua. Una *fórmula*, sin embargo, es buen sistema de representación. Por ejemplo, la fórmula para la solución de una ecuación cuadrática es un medio sucinto de expresar el procedimiento algorítmico que se debe ejecutar para obtener las raíces

$$x1 = (-b + \sqrt{b^2 - 4ac}) / 2a \qquad x2 = (-b - \sqrt{b^2 - 4ac}) / 2a$$

significa

1. *Eleve al cuadrado b.*
2. *Toma a; multiplicar por c; multiplicar por 4.*
3. *Restar el resultado de 2 del resultado de 1, etc.*

Sin embargo, no es frecuente que un algoritmo pueda ser expresado por medio de una simple fórmula.

2.5.1. Diagramas de flujo

Un **diagrama de flujo** (*flowchart*) es una de las técnicas de representación de algoritmos más antigua y a la vez más utilizada, aunque su empleo ha disminuido considerablemente, sobre todo desde la aparición de lenguajes de programación estructurados. Un diagrama de flujo es un diagrama que utiliza los símbolos (cajas) estándar mostrados en la Figura 2.6 y que tiene los pasos del algoritmo escritos en esas cajas unidas por flechas, denominadas *líneas de flujo*, que indican la secuencia en que se deben ejecutar.

La Figura 2.7 es un diagrama de flujo básico.

El diagrama citado (Figura 2.7) representa la resolución de un programa que deduce el salario neto de un trabajador a partir de la lectura del nombre, horas trabajadas, precio de la hora, y sabiendo que los impuestos aplicados son el 25 por 100 sobre el salario bruto.

Los símbolos estándar normalizados por **ANSI** (abreviatura de *American National Standards Institute*) son muy variados. En la Figura 2.8 se representa una plantilla de dibujo típica donde se

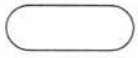
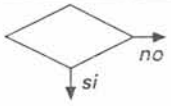
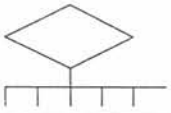
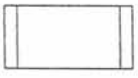
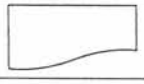
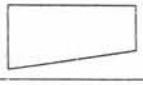
Símbolos principales	Función
	Terminal (representa el comienzo, «inicio», y el final, «fin», de un programa. Puede representar también una parada o interrupción programada que sea necesario realizar en un programa).
	Entrada/Salida (cualquier tipo de introducción de datos en la memoria desde los periféricos, «entrada», o registro de la información procesada en un periférico, «salida»).
	Proceso (cualquier tipo de operación que pueda originar cambio de valor, formato o posición de la información almacenada en memoria, operaciones aritméticas, de transferencia, etc.).
	Decisión (indica operaciones lógicas o de comparación entre datos —normalmente dos— y en función del resultado de la misma determina cuál de los distintos caminos alternativos del programa se debe seguir; normalmente tiene dos salidas —respuestas SI o NO—, pero puede tener tres o más, según los casos).
	Decisión múltiple (en función del resultado de la comparación se seguirá uno de los diferentes caminos de acuerdo con dicho resultado).
	Conector (sirve para enlazar dos partes cualesquiera de un ordinograma a través de un conector en la salida y otro conector en la entrada. Se refiere a la conexión en la misma página del diagrama).
	Indicador de dirección o línea de flujo (indica el sentido de ejecución de las operaciones).
	Línea conectora (sirve de unión entre dos símbolos).
	Conector (conexión entre dos puntos del organigrama situado en páginas diferentes).
	Llamada a subrutina o a un proceso predeterminado (una subrutina es un módulo independiente del programa principal, que recibe una entrada procedente de dicho programa, realiza una tarea determinada y regresa, al terminar, al programa principal).
Símbolos secundarios	**Función**
	Pantalla (se utiliza en ocasiones en lugar del símbolo de E/S).
	Impresora (se utiliza en ocasiones en lugar del símbolo de E/S).
	Teclado (se utiliza en ocasiones en lugar del símbolo de E/S).
	Comentarios (se utiliza para añadir comentarios clasificadores a otros símbolos del diagrama de flujo. Se pueden dibujar a cualquier lado del símbolo).

Figura 2.6. Símbolos de diagrama de flujo.

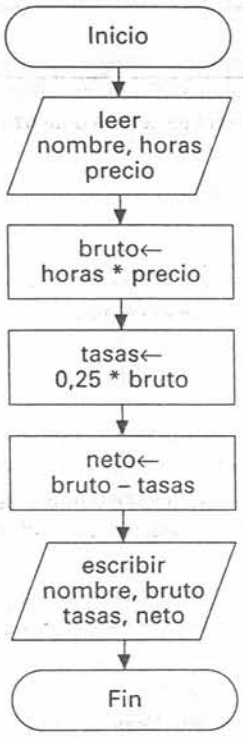


Figura 2.7. Diagrama de flujo.

contemplan la mayoría de los símbolos utilizados en el diagrama; sin embargo, los símbolos más utilizados representan:

- proceso,
- decisión,
- conectores,
- fin,
- entrada/salida,
- dirección del flujo,

y se resumen en la Figura 2.9.

En un diagrama de flujo:

- existe una caja etiquetada `"inicio"`, que es de tipo elíptico,
- existe otra caja etiquetada `"fin"` de igual forma que la anterior,
- si existen otras cajas, normalmente son rectangulares, tipo rombo o paralelogramo (el resto de las figuras se utilizan sólo en diagramas de flujo generales o de detalle y no siempre son imprescindibles).

Se puede escribir más de un paso del algoritmo en una sola caja rectangular. El uso de flechas significa que la caja no necesita ser escrita debajo de su predecesora. Sin embargo, abusar demasiado de esta flexibilidad conduce a diagramas de flujo complicados e ininteligibles.

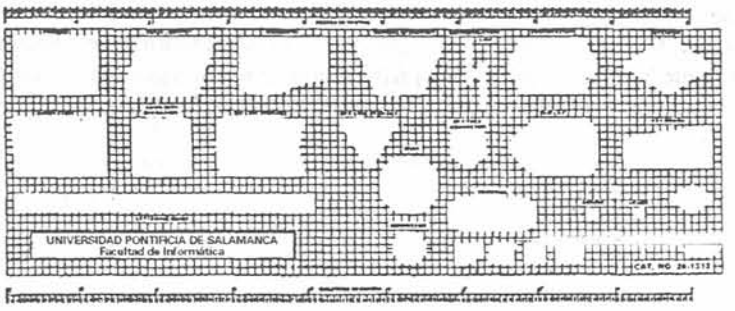


Figura 2.8. Plantilla para dibujo de diagramas de flujo.

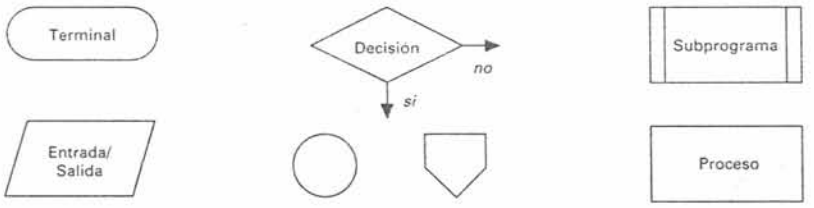


Figura 2.9. Símbolos más utilizados en los diagramas de flujo.

Símbolos de diagramas de flujo

Cada símbolo visto anteriormente indica el *tipo de operación a ejecutar* y el diagrama de flujo ilustra gráficamente la *secuencia en la que se ejecutan las operaciones*.

Las *líneas de flujo* (→) representan el flujo secuencial de la lógica del programa.

Un rectángulo (▭) significa algún tipo de *proceso* en la computadora, es decir, acciones a realizar (sumar dos números, calcular la raíz cuadrada de un número, etc.).

El paralelogramo (▱) es un símbolo de entrada/salida que representa cualquier tipo de *entrada* o *salida* desde el programa o sistema; por ejemplo, entrada de teclado, salida en impresora o pantalla, etc.

El símbolo rombo (◇) es una caja de decisión que representa respuestas sí/no o bien diferentes alternativas 1, 2, 3, 4, ..., etc.

Cada diagrama de flujo comienza y termina con un símbolo terminal (⬭).

Un pequeño círculo (○) es un *conector* y se utiliza para conectar caminos, tras roturas previas del flujo del algoritmo.

Otros símbolos de diagramas de flujo menos utilizados de mayor detalle que los anteriores son:

Un trapezoide (⏢) indica que un *proceso manual* se va a ejecutar en contraste con el rectángulo, que indica proceso automático.

El símbolo general de entrada/salida se puede subdividir en otros símbolos: *teclado* (⌨), *pantalla* (▭), *impresora* (🖷), *disco magnético* (⛁), *disquete* o *disco flexible* (💾), *casete* (📼).

Ejemplo 2.2

Calcular la media de una serie de números positivos, suponiendo que los datos se leen desde un terminal. Un valor de cero —como entrada— indicará que se ha alcanzado el final de la serie de números positivos.

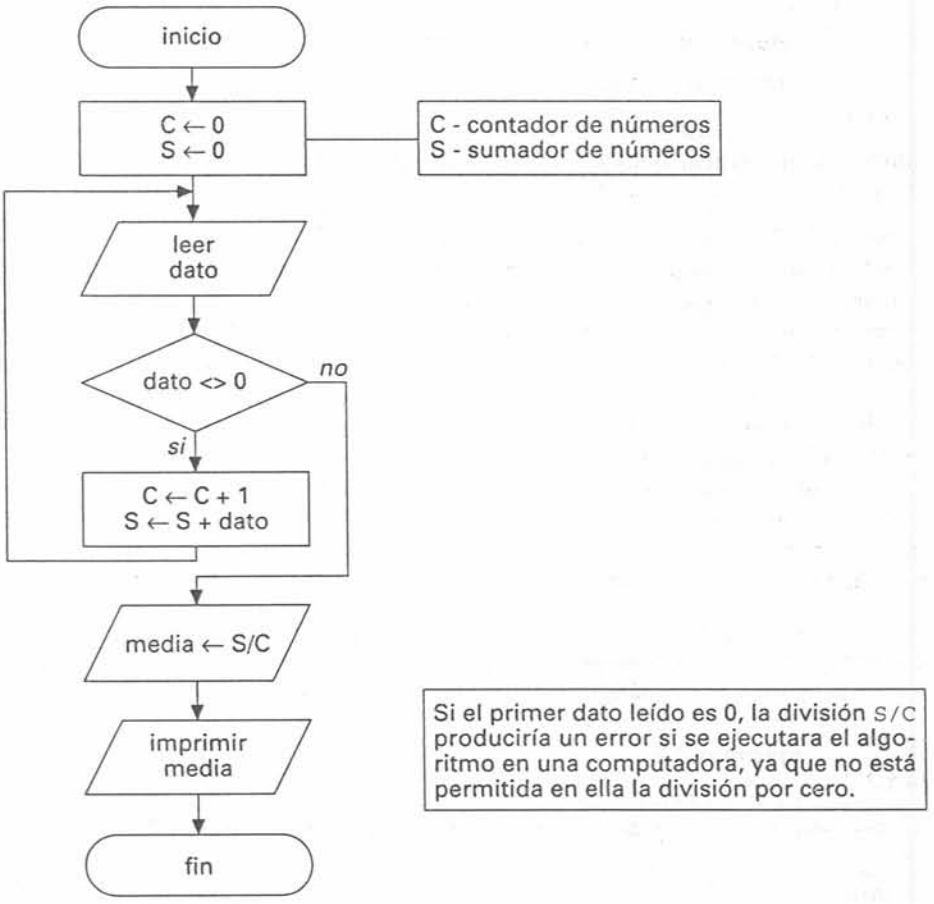
El primer paso a dar en el desarrollo del algoritmo es descomponer el problema en una serie de pasos secuenciales. Para calcular una media se necesita sumar y contar los valores.

Por consiguiente, nuestro algoritmo en forma descriptiva sería:

```
1. Inicializar contador de numeros C y variable suma S.
2. Leer un numero.
3. Si el numero leido es cero:
   • calcular la media;
   • imprimir la media;
   • fin del proceso.
   Si el numero leido no es cero:
   • calcular la suma;
   • incrementar en uno el contador de numeros;
   • ir al paso 2.
4. Fin.
```

El refinamiento del algoritmo conduce a los pasos sucesivos necesarios para realizar las operaciones de lectura de datos, verificación del último dato, suma y media de los datos.

Diagrama de flujo 2.2

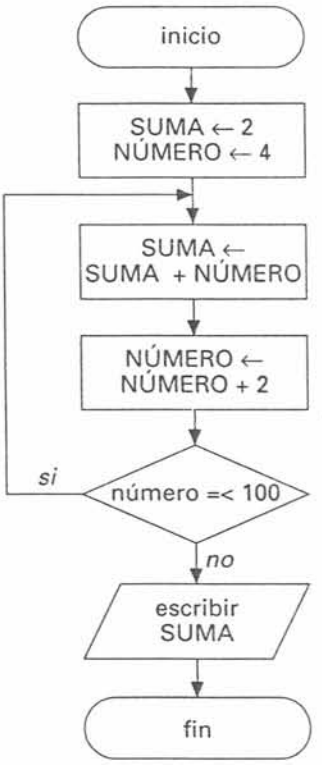


Si el primer dato leído es `0`, la división `S/C` produciría un error si se ejecutara el algoritmo en una computadora, ya que no está permitida en ella la división por cero.

Ejemplo 2.3

Suma de los números pares comprendidos entre `2` *y* `100`.

Diagrama de flujo 2.3



Ejemplo 2.4

Se desea realizar el algoritmo que resuelva el siguiente problema:

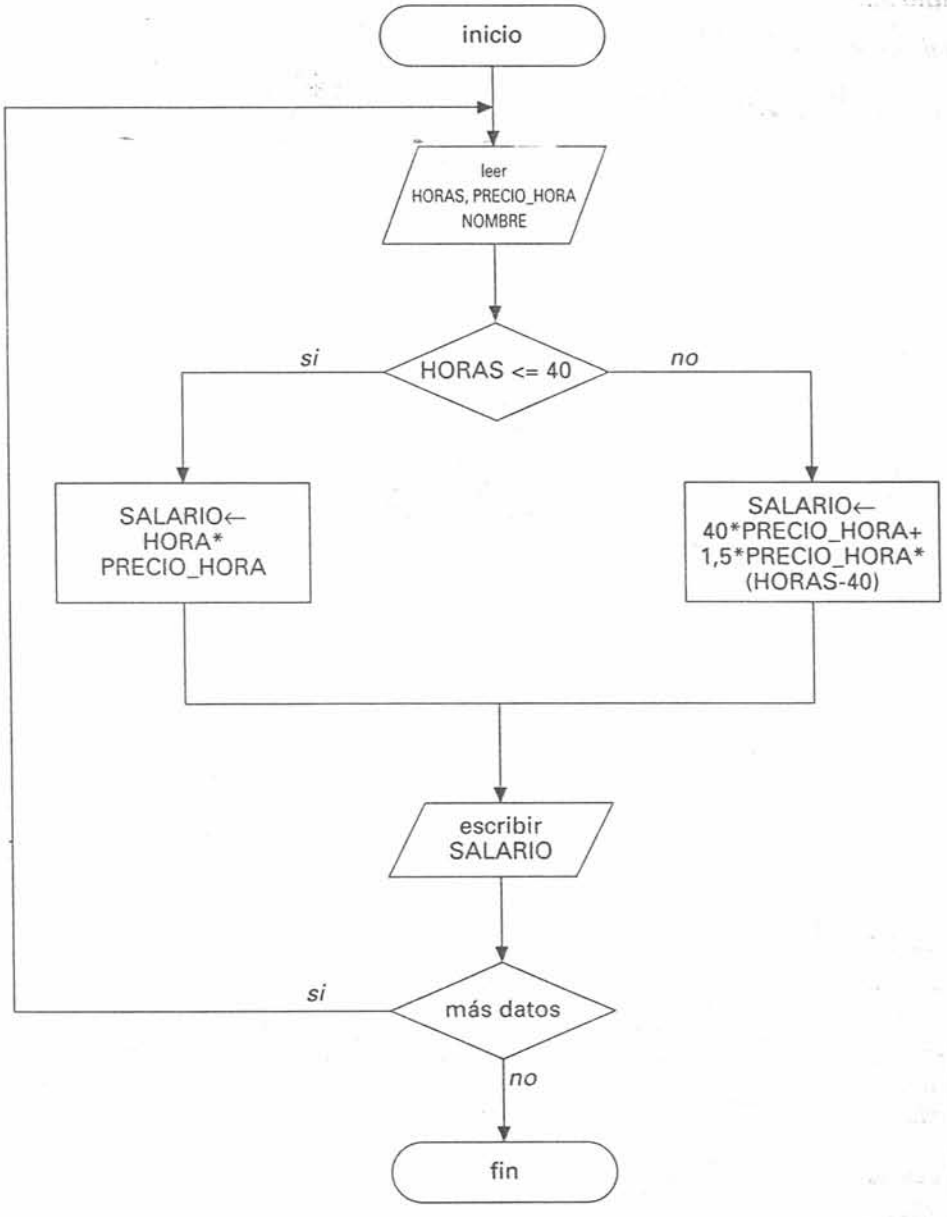
Cálculo de los salarios mensuales de los empleados de una empresa, sabiendo que éstos se calculan en base a las horas semanales trabajadas y de acuerdo a un precio especificado por hora. Si se pasan de cuarenta horas semanales, las horas extraordinarias se pagarán a razón de 1,5 veces la hora ordinaria.

Los cálculos son sencillos son:

```
1. Leer datos del archivo de la empresa, hasta que se encuentre la ficha final del
   archivo (HORAS, PRECIO_HORA, NOMBRE).
2. Si HORAS <= 40, entonces SALARIO es el producto de HORAS por
   PRECIO_HORA.
3. Si HORAS > 40, entonces SALARIO es la suma de 40 veces PRECIO_HORA más 1,5
   veces PRECIO_HORA por (HORAS-40).
```

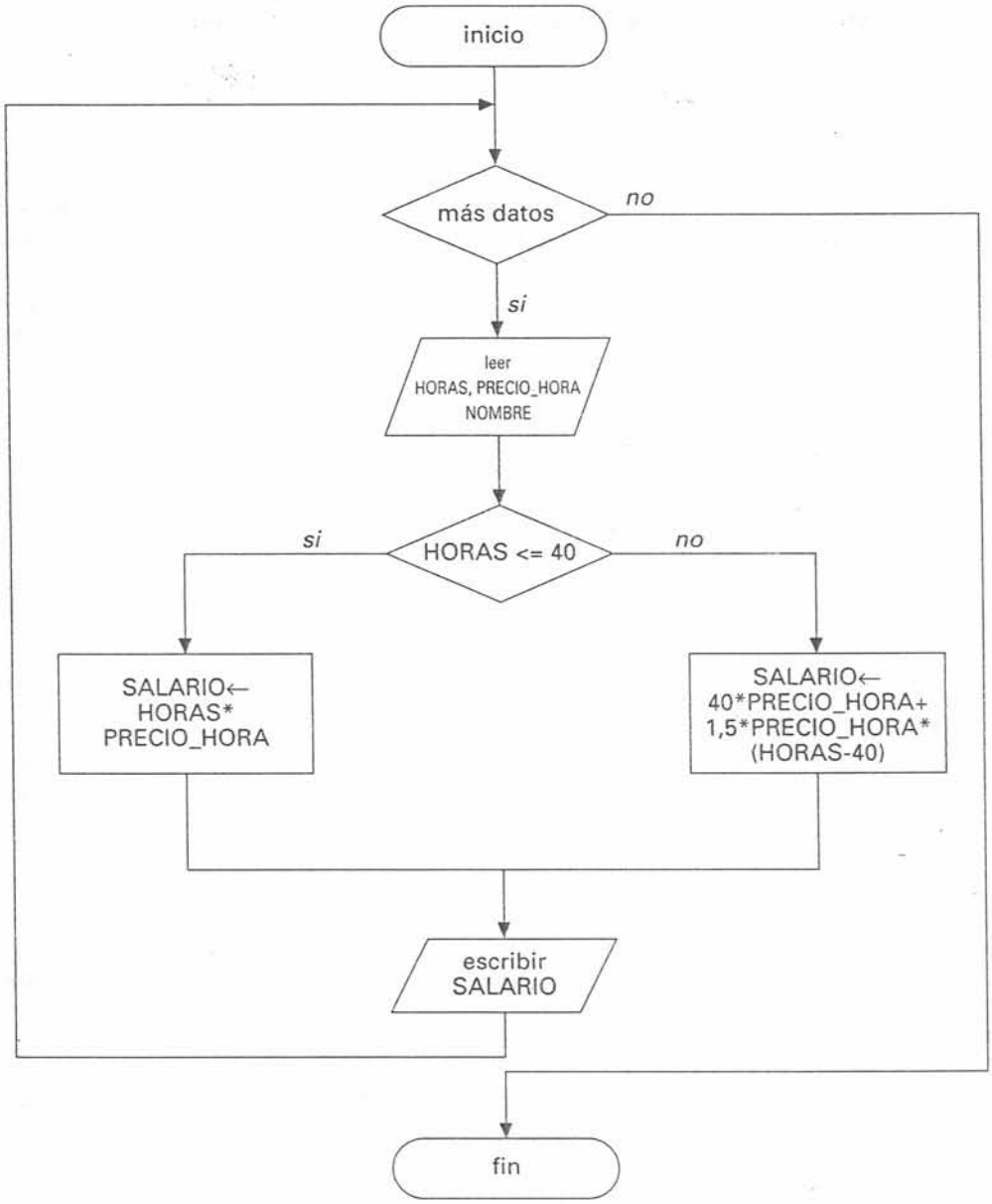
El diagrama de flujo completo del algoritmo se indica a continuación:

Diagrama de flujo 2.4.a



o bien:

Diagrama de flujo 2.4. b



Ejemplo 2.5

La escritura de algoritmos para realizar operaciones sencillas de conteo es una de las primeras cosas que un ordenador puede aprender.

Supongamos que se proporciona una secuencia de números, tales como

```
5  3  0  2  4  4  0  0  2  3  6  0  2
```

y desea contar e imprimir el número de ceros de la secuencia.

El algoritmo es muy sencillo, ya que sólo basta leer los números de izquierda a derecha, mientras se cuentan los ceros.

Dicho algoritmo, por consiguiente, utiliza como variable la palabra NUMERO para los números que se examinan y TOTAL para el número de ceros encontrados. Los pasos a seguir en el algoritmo escrito en un lenguaje natural son:

```
1. Establecer TOTAL a cero.
2. ¿Quedan más numeros a examinar?
3. Si no quedan numeros, imprimir el valor de TOTAL y fin.
4. Si existen mas numeros, ejecutar los pasos 5 a 8.
5. Leer el siguiente numero y dar su valor a la variable NUMERO.
6. Si NUMERO = 0, incrementar TOTAL en uno.
7. Si NUMERO <> 0, no modificar TOTAL.
8. Retornar al paso 2.
```

El diagrama de flujo se muestra en el DF 2.5.

Ejemplo 2.6

Dados tres números, determinar si la suma de cualquier pareja de ellos es igual al tercer número. Si se cumple esta condición, escribir «Iguales» y, en caso contrario, escribir «Distintas».

Por ejemplo, si los números son:

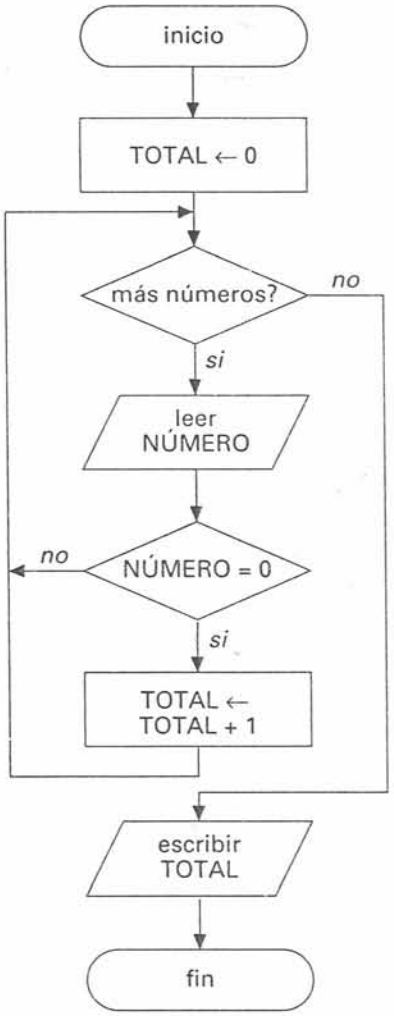
```
3  9  6
```

la respuesta es `"Iguales"`, ya que `3 + 6 = 9`. Sin embargo, si los números son:

```
2  3  4
```

el resultado será `"Distintas"`.

Diagrama de flujo 2.5 (DF 2.5)



Para solucionar este problema, se puede comparar la suma de cada pareja con el tercer número. Con tres números solamente existen tres parejas distintas y el algoritmo de resolución del problema será fácil.

```
1. Leer los tres valores, A, B y C.
2. Si A + B = C escribir "Iguales" y parar.
3. Si A + C = B escribir "Iguales" y parar.
4. Si B + C = A escribir "Iguales" y parar.
5. Escribir "Distintas" y parar.
```

El diagrama de flujo correspondiente al algoritmo es el 2.6.

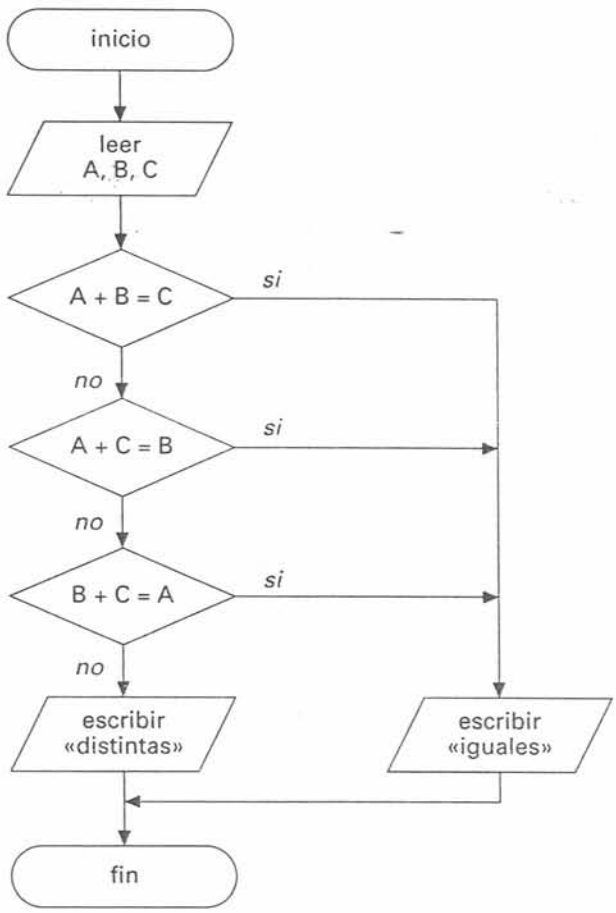


Figura 2.10. Diagrama de flujo 2.6 (Ejemplo 2.6).

2.6. DIAGRAMAS DE NASSI-SCHNEIDERMAN (N-S)

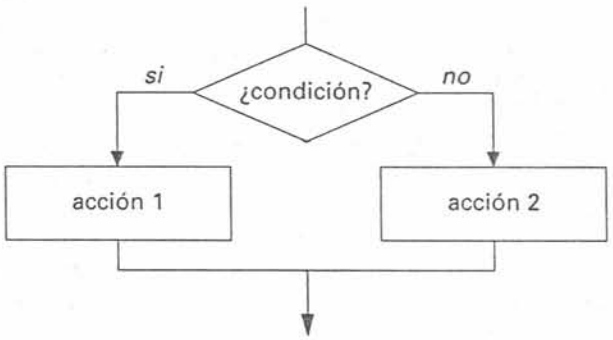
El diagrama N-S de Nassi-Schneiderman —también conocido como diagrama de Chapin— es como un diagrama de flujo en el que se omiten las flechas de unión y las cajas son contiguas. Las acciones sucesivas se escriben en cajas sucesivas y, como en los diagramas de flujo, se pueden escribir diferentes acciones en una caja.

Un algoritmo se representa de la forma siguiente:

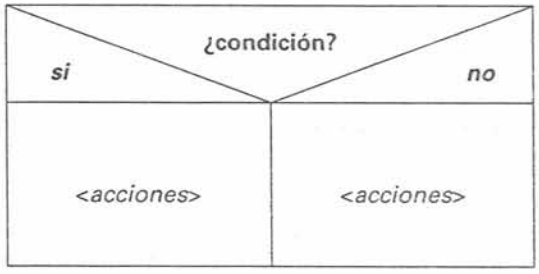
```
nombre del algoritmo
<accion 1>
<accion 2>
<accion 3>
...
fin
```

```
   leer
nombre, horas, precio
   calcular
salario ← horas * precio
   calcular
impuestos ← 0.25 * salario
   calcular
neto ← salario - impuestos
   escribir
nombre, salario, impuestos, neto
```

La estructura condicional (el rombo)



se representa por



Ejemplo 2.7

Se desea calcular el salario neto semanal de un trabajador en función del número de horas trabajadas y la tasa de impuestos:

- las primeras 35 horas se pagan a tarifa normal,
- las horas que pasen de 35 se pagan a 1,5 veces la tarifa normal,
- las tasas de impuestos son:
  a) las primeras 60.000 pesetas son libres de impuestos,
  b) las siguientes 40.000 pesetas tienen un 25 por 100 de impuesto,
  c) las restantes, un 45 por 100 de impuestos,
- la tarifa horaria es 800 pesetas.

Se desea también escribir el nombre, salario bruto, tasas y salario neto (*este ejemplo se deja como ejercicio al alumno*).

2.7. PSEUDOCÓDIGO

El pseudocódigo es *un lenguaje de especificación (descripción) de algoritmos*. El uso de tal lenguaje hace el paso de codificación final (esto es, la traducción a un lenguaje de programación) relativamente fácil. Los lenguajes APL Pascal y Ada se utilizan a veces como lenguajes de especificación de algoritmos.

El pseudocódigo nació como un lenguaje similar al inglés y era un medio de representar básicamente las estructuras de control de programación estructurada que se verán en capítulos posteriores. Se considera un *primer borrador*, dado que el pseudocódigo tiene que traducirse posteriormente a un lenguaje de programación. El pseudocódigo no puede ser ejecutado por una computadora. La *ventaja del pseudocódigo* es que en su uso, en la planificación de un programa, el programador se puede concentrar en la lógica y en las estructuras de control y no preocuparse de las reglas de un lenguaje específico. Es también fácil modificar el pseudocódigo si se descubren errores o anomalías en la lógica del programa, mientras que en muchas ocasiones suele ser difícil el cambio en la lógica, una vez que está codificado en un lenguaje de programación. Otra ventaja del pseudocódigo es que puede ser traducido fácilmente a lenguajes estructurados como Pascal, C, FORTRAN 77/90, Ada 83/95, C++. etc.

El pseudocódigo original utiliza para representar las acciones sucesivas palabras reservadas en inglés —similares a sus homónimas en los lenguajes de programación—, tales como **start**, **end**, **stop**, **if-then-else**, **while-end**, **repeat-until**, etc. La escritura de pseudocódigo exige normalmente la *indentación* (sangría en el margen izquierdo) de diferentes líneas.

La representación en pseudocódigo del diagrama de flujo de la Figura 2.7 es la siguiente:

```
start
  //calculo de impuesto y salarios
  read nombre, horas, precio_hora
  salario_bruto ← horas * precio_hora
  tasas ← 0,25 * salario_bruto
  salario_neto ← salario_bruto - tasas
  write nombre, salario_bruto, tasas, salario_neto
end
```

El algoritmo comienza con la palabra `start` y finaliza con la palabra `end`, en inglés (en español, `inicio`, `fin`). Entre estas palabras, sólo se escribe una instrucción o acción por línea.

La línea precedida por // se denomina *comentario*. Es una información al lector del programa y no realiza ninguna instrucción ejecutable, sólo tiene efecto de documentación interna del programa. Algunos autores suelen utilizar corchetes o llaves.

No es recomendable el uso de apóstrofos o simples comillas como representan en BASIC de Microsoft los comentarios, ya que este carácter es representativo de apertura o cierre de cadenas de caracteres en lenguajes como Pascal o FORTRAN, y daría lugar a confusión.

Otro ejemplo aclaratorio en el uso del pseudocódigo podría ser un sencillo algoritmo del arranque matinal de un coche.

```
inicio
  //arranque matinal de un coche
  introducir la llave de contacto
  tirar del estrangulador del aire
  girar la llave de contacto
  pisar el acelerador
  oir el ruido del motor
  pisar de nuevo el acelerador
  esperar unos instantes a que se caliente el motor
  llevar el estrangulador de aire a su posicion
fin
```

Por fortuna, aunque el pseudocódigo nació como un sustituto del lenguaje de programación y, por consiguiente, sus palabras reservadas se conservaron o fueron muy similares a las de dichos lenguajes, prácticamente el inglés, el uso del pseudocódigo se ha extendido en la comunidad hispana con términos en español como **inicio**, **fin**, **parada**, **leer**, **escribir**, **si-entonces-si_no**, **mientras**, **fin_mientras**, **repetir**, **hasta_que**, etc. Sin duda, el uso de la terminología del pseudocódigo en español ha facilitado y facilitará considerablemente el aprendizaje y uso diario de la programación. En esta obra, al igual que en otras nuestras [1] utilizaremos el pseudocódigo en español y daremos en su momento las estructuras equivalentes en inglés, al objeto de facilitar la traducción del pseudocódigo al lenguaje de programación seleccionado.

Así pues, en los pseudocódigos citados anteriormente deberían ser sustituidas las palabras **start**, **end**, **read**, **write**, por **inicio**, **fin**, **leer**, **escribir**, respectivamente.

```
inicio      start      leer        read
.
.
.
.
.
fin         end        escribir    write
```

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

2.1. Se desea obtener el salario neto de un trabajador conociendo el número de horas trabajadas, el salario hora y la tasa de impuestos que se le debe deducir.
El algoritmo general es:

```
1. Obtener valores de horas trabajadas, salario_hora y tasas.
2. Calcular salario_bruto, total de impuestos y salario_neto.
3. Visualizar salario_bruto, total de impuestos y salario_neto.
```

[1] *Metodología de la Programación*. McGraw-Hill, 1986.

Las *entradas* del algoritmo son:

```
horas trabajadas, salario_hora, tasas
```

Las *salidas* del algoritmo son:

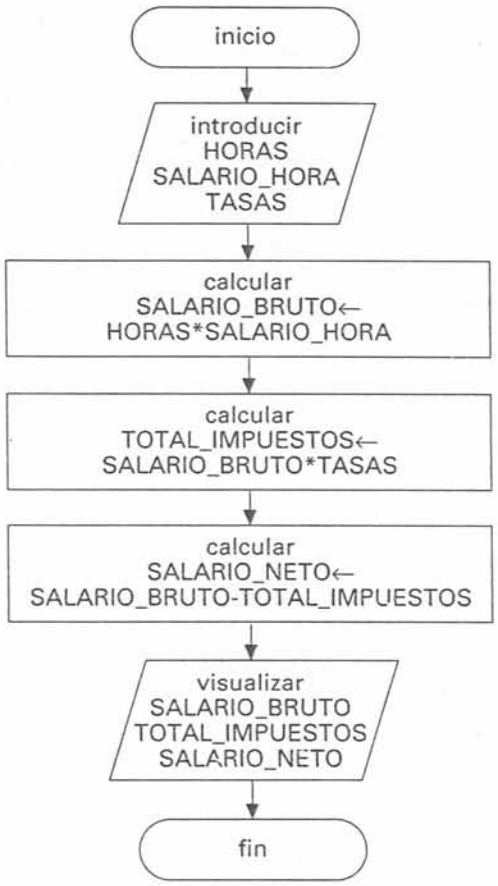
```
paga bruta, total de impuestos y paga neta
```

El refinamiento del algoritmo en pasos de nivel inferior es:

```
1. Obtener valores de horas trabajadas, salario bruto y tasas.
2. Calcular salario bruto, total de impuestos y paga neta.
   2.1. Calcular salario bruto multiplicando las horas trabajadas por el salario
        hora.
   2.2. Calcular el total de impuestos multiplicando salario bruto por tasas
        (tanto por ciento de impuestos).
   2.3. Calcular el salario neto restando el total de impuestos de la paga bruta.
3. Visualizar salario bruto, total de impuestos, salario neto.
```

El diagrama de flujo 2.7 representa este algoritmo.

Diagrama de flujo 2.7



2.2. Calcular el valor de la suma 1 + 2 + 3 + 4 + ... + 100.

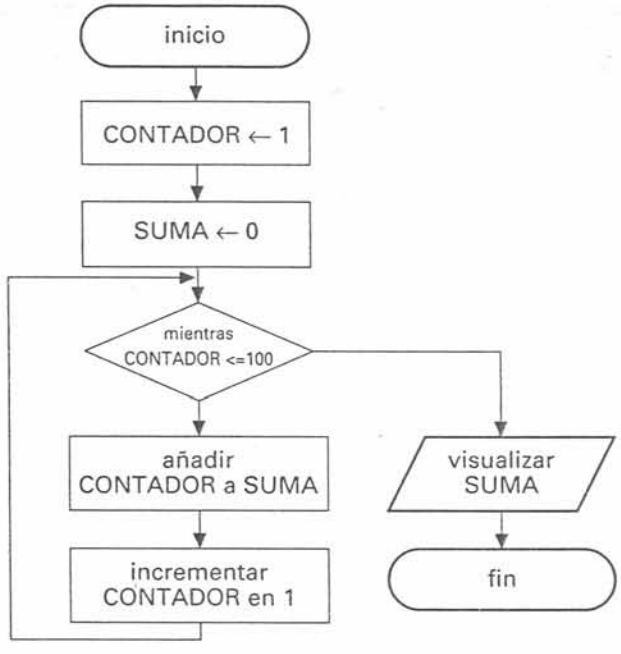
Variables CONTADOR (números sucesivos a sumar: 1, 2, 3, . . .).
SUMA (totalizador de sumas).

Pseudocódigo

```
1. inicio.
2. Establecer CONTADOR a 1.
3. Establecer SUMA a 0.
4. mientras CONTADOR <= 100 hacer lo siguiente:
   4.1. Sumar CONTADOR a SUMA.
   4.2. Incrementar CONTADOR en 1.
   fin_mientras
5. Visualizar SUMA.
6. fin.
```

Este algoritmo se representa por el diagrama de flujo 2.8.

Diagrama de flujo 2.8



2.3. Un corredor de maratón (distancia = 42,195 km) ha recorrido la carrera en 2 horas 25 minutos. Se desea un algoritmo que calcule el tiempo medio en minutos por kilómetro.

El análisis del problema es el siguiente:

Entrada: Cantidad total de tiempo empleado en la carrera; se establece el número total de kilómetros en 42.195, ya que es igual para todos.
Salida: Número medio de minutos por kilómetro, transformando previamente el tiempo total a minutos.
Proceso: Dividir el tiempo total en minutos por el número de kilómetros.

El pseudocódigo es:

```
inicio
  introducir tiempo total
  verificar entrada correcta
  establecer distancia = 42.195 km
  pasar tiempo total a minutos
  calcular media tiempo/kilometro
  escribir resultado
fin
```

NOTA: El tiempo `T` se ha de convertir en minutos, bien tras introducir su valor o en la instrucción 4 (pasar tiempo total a minutos). La fórmula de conversión es:

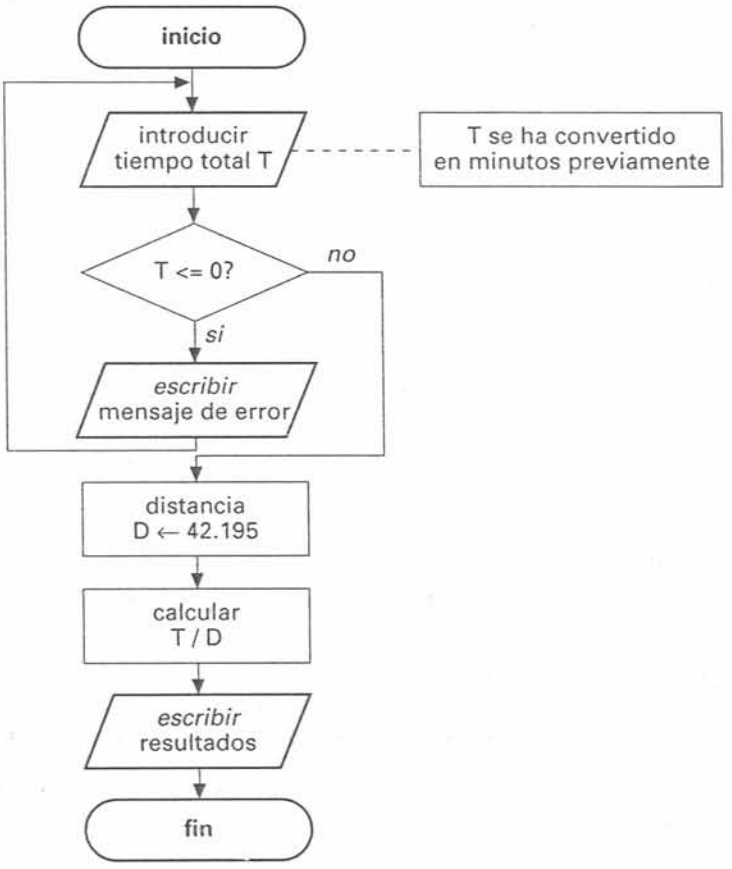
```
T = 2 * 60 + 25
```

o con carácter general, si el tiempo es *h* horas *m* minutos

```
T = h * 60 + m
```

El diagrama de flujo de este algoritmo es el representado a continuación.

Diagrama de flujo 2.9



2.4. Definir el algoritmo necesario para intercambiar los valores de dos variables numéricas.

Análisis

Para realizar este análisis se utiliza una variable denominada auxiliar que de modo temporal toma uno de los valores dados.

Variables: A B AUX

El método consiste en asignar una de las variables a la variable auxiliar:

```
AUX ← A
```

A continuación se asigna el valor de la otra variable B a la primera:

```
A ← B
```

Por último, se asigna el valor de la variable auxiliar a la segunda variable A:

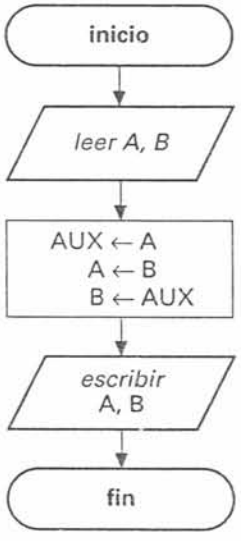
```
B ← AUX
```

Variables:		
	A	primer valor,
	B	segundo valor,
	AUX	variable auxiliar.

Pseudocódigo

```
inicio
  leer(A, B)
  AUX ← A
  A ← B
  B ← AUX
  escribir(A, B)
fin
```

Diagrama de flujo 2.10



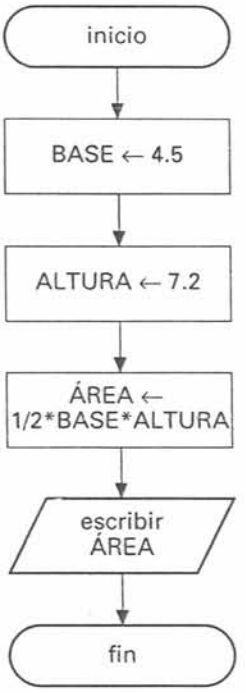
2.5. Escribir un algoritmo para calcular el área de un triángulo dada la base y la altura.

Análisis

La fórmula geométrica del área o superficie de un triángulo es:

$S = (1/2)\, B \times H$ $\quad B = base$ $\quad H = altura$

Se necesita asignar los valores de la base y la altura a variables —por ejemplo, BASE y ALTURA, respectivamente—; a continuación, calcular el área utilizando la fórmula y asignar los resultados del cálculo a una variable llamada AREA. Supongamos B = 4.5 y H = 7.2.



EJERCICIOS

2.1. Deducir los resultados que se obtienen del siguiente algoritmo:

```
var entero: x, y, z
inicio
  x ← 5
  y ← 20
  z ← x + y
  escribir (x, y)
  escribir (z)
fin
```

2.2. ¿Qué resultados producirá este algoritmo?

```
var entero: nx, doble
inicio
  NX ← 25
  DOBLE ← NX * 2
  escribir(NX)
  escribir(DOBLE)
fin
```

2.3. Escribir un algoritmo que calcule y escriba el cuadrado de 243.
2.4. Escribir un algoritmo que lea un número y escriba su cuadrado.
2.5. Determinar el área y volumen de un cilindro cuyas dimensiones radio y altura se leen desde el teclado.
2.6. Calcular el perímetro y la superficie de un cuadrado dada la longitud de su lado.
2.7. Realizar el algoritmo que sume dos números.
2.8. Calcular la superficie de un círculo.
2.9. Calcular el perímetro y la superficie de un rectángulo dadas la base y la altura del mismo.
2.10. Escribir un algoritmo que lea un nombre de una marca de automóviles seguida del nombre de su modelo e informe del modelo seguido del nombre.
2.11. Determinar la hipotenusa de un triángulo rectángulo conocidas las longitudes de los catetos.
2.12. Diseñar un algoritmo que realice la siguiente conversión: una temperatura dada en grados Celsius a grados Fahrenheit.
NOTA: La fórmula de conversión es: F = (9/5) C + 32.
2.13. Diseñar un algoritmo que calcule el área de un triángulo en función de las longitudes de sus lados:

$$\text{Área} = \sqrt{p(p-a)(p-b)(p-c)}$$

donde $p = (a + b + c)/2$ (semiperímetro).
2.14. Se desea un algoritmo para convertir metros a pies y pulgadas (1 metro = 39.37 pulgadas, 1 pie = 12 pulgadas).
2.15. El cambio de divisas en la Bolsa de Madrid el día 25 de agosto de 1987 fue el siguiente:

100 chelines austriacos	= 956,871 pesetas
1 dólar EE.UU.	= 122,499 pesetas
100 dracmas griegas	= 88,607 pesetas
100 francos belgas	= 323,728 pesetas
1 franco francés	= 20,110 pesetas
1 libra esterlina	= 178,938 pesetas
100 liras italianas	= 9,289 pesetas

2.16. Desarrollar algoritmos que realicen las siguientes conversiones:

a) Leer una cantidad en chelines austriacos e imprimir el equivalente en pesetas.
b) Leer una cantidad en dracmas griegos e imprimir el equivalente en francos franceses.
c) Leer una cantidad en pesetas e imprimir el equivalente en dólares y en liras italianas.

CAPÍTULO 3

ESTRUCTURA GENERAL DE UN PROGRAMA

CONTENIDO

3.1. Concepto de programa.
3.2. Partes constitutivas de un programa.
3.3. Instrucciones y tipos de instrucciones.
3.4. Tipos de instrucciones.
3.5. Elementos básicos de un programa.
3.6. Escritura de algoritmos/programas.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

En los capítulos anteriores se ha visto la forma de diseñar algoritmos para resolver problemas con computadora. En este capítulo se introduce al lector en el proceso de la programación, que se manifiesta esencialmente en los programas.

El *concepto de programa* como un conjunto de instrucciones y sus tipos constituye la parte fundamental del capítulo. La *descripción de los elementos básicos* de programación, que se encontrarán en casi todos los programas: interruptores, contadores, totalizadores, etc., junto con las normas elementales para la escritura de algoritmos y programas, conforman el resto del capítulo.

3.1. CONCEPTO DE PROGRAMA

Un *programa de computadora* es un conjunto de instrucciones —órdenes dadas a la máquina— que producirán la ejecución de una determinada tarea. En esencia, *un programa es un medio para conseguir un fin.*

Figura 3.1. El proceso de la programación.

El *proceso de programación* es, por consiguiente, un proceso de solución de problemas —como ya se vio en el Capítulo 2— y el desarrollo de un programa requiere las siguientes fases:

1. *definición y análisis del problema;*
2. *diseño de algoritmos:*
   - diagrama de flujo,
   - diagrama N-S,
   - pseudocódigo;
3. *codificación del programa;*
4. *depuración y verificación del programa;*
5. *documentación;*
6. *mantenimiento.*

Las fases 1 y 2 ya han sido analizadas en los capítulos anteriores y son el objetivo fundamental de este libro; sin embargo, dedicaremos atención, a lo largo del libro (véase Capítulo 13) y en los apéndices, a las fases 3, 4, 5 y 6, aunque éstas son propias de libros específicos sobre lenguajes de programación.

3.2. PARTES CONSTITUTIVAS DE UN PROGRAMA

Tras la decisión de desarrollar un programa, el programador debe establecer el conjunto de especificaciones que debe contener el programa: *entrada*, *salida* y *algoritmos de resolución*, que incluirán las técnicas para obtener las salidas a partir de las entradas.

Conceptualmente, un programa puede ser considerado como una caja negra, como se muestra en la Figura 3.2. La caja negra o el algoritmo de resolución, en realidad, es el conjunto de códigos que transforman las entradas del programa (*datos*) en salidas (*resultados*).

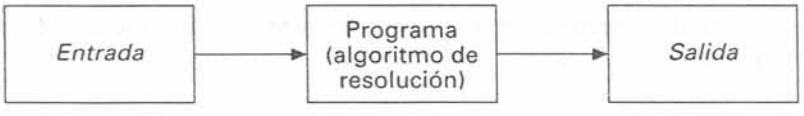


Figura 3.2. Bloques de un programa.

El programador debe establecer de dónde provienen las entradas al programa. Las entradas, en cualquier caso, procederán de un dispositivo de entrada —teclado, disco,...— El proceso de introducir la información de entrada —datos— en la memoria de la computadora se denomina *entrada de datos*, operación de *lectura* o acción de **leer**.

Las salidas de datos se deben presentar en dispositivos periféricos de salida: *pantalla*, *impresoras*, *discos*, etc. La operación de *salida de datos* se conoce también como *escritura* o acción de **escribir**.

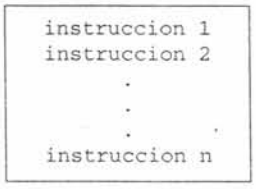
3.3. INSTRUCCIONES Y TIPOS DE INSTRUCCIONES

El proceso de diseño del algoritmo o posteriormente de codificación del programa consiste en definir las acciones o instrucciones que resolverán el problema.

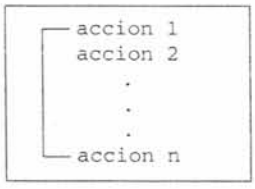
Las *acciones* o *instrucciones* se deben escribir y posteriormente almacenar en memoria en el mismo orden en que han de ejecutarse, es decir, *en secuencia*.

Un programa puede ser lineal o no lineal.

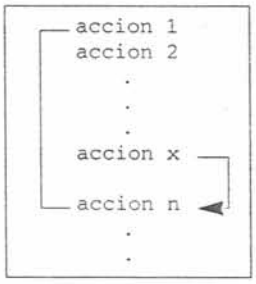
Un programa es *lineal* si las instrucciones se ejecutan secuencialmente, sin bifurcaciones, decisión ni comparaciones.



En el caso del algoritmo las instrucciones se suelen conocer como *acciones*, y se tendría:



Un programa es *no lineal* cuando se interrumpe la secuencia mediante instrucciones de bifurcación:



3.4. TIPOS DE INSTRUCCIONES

Las instrucciones disponibles en un lenguaje de programación dependen del tipo de lenguaje. Por ello, en este apartado estudiaremos las instrucciones —acciones— básicas que se pueden implementar de modo general en un algoritmo y que esencialmente soportan todos los lenguajes. Dicho de otro modo, las instrucciones básicas son independientes del lenguaje.

La clasificación más usual, desde el punto de vista anterior, es:

1. instrucciones de inicio/fin,
2. instrucciones de asignación,
3. instrucciones de lectura,
4. instrucciones de escritura,
5. instrucciones de bifurcación.

Las instrucciones 1, 2, 3 y 4 ya han sido descritas en capítulos anteriores y se recogen en la Tabla 3.1.

3.4.1. Instrucciones de asignación

Como ya son conocidas del lector, repasaremos su funcionamiento con ejemplos:

a) `A ← 80` *la variable A toma el valor de 80.*

b) ¿Cuál será el valor que tomará la variable `C` tras la ejecución de las siguientes instrucciones?

```
A ← 12
B ← A
C ← B
```

`A` contiene `12`, `B` contiene `12` y `C` contiene `12`.

NOTA: Antes de la ejecución de las tres instrucciones, el valor de `A`, `B` y `C` es indeterminado. Si se desea darles un valor inicial, habrá que hacerlo explícitamente, incluso cuando este valor sea `0`. Es decir, habrá que definir e inicializar las instrucciones.

```
A ← 0
B ← 0
C ← 0
```

Tabla 3.1. Instrucciones/acciones básicas

Tipo de instrucción	Pseudocódigo inglés	Pseudocódigo español
comienzo de proceso	`begin`	`inicio`
fin de proceso	`end`	`fin`
entrada (lectura)	`read`	`leer`
salida (escritura)	`write`	`escribir`
asignación	`A ← 5`	`B ← 7`

c) ¿Cuál es el valor de la variable AUX al ejecutarse la instrucción 5?

```
1. A   ← 10
2. B   ← 20
3. AUX ← A
4. A   ← B
5. B   ← AUX
```

- en la instrucción 1, A toma el valor 10
- en la instrucción 2, B toma el valor 20
- en la instrucción 3, AUX toma el valor anterior de A, o sea 10
- en la instrucción 4, A toma el valor anterior de B, o sea 20
- en la instrucción 5, B toma el valor anterior de AUX, o sea 10
- tras la instrucción 5, AUX sigue valiendo 10

d) ¿Cuál es el significado de N ← N + 5 si N tiene el valor actual de 2?

```
N ← N + 5
```

Se realiza el cálculo de la expresión N + 5 y su resultado 2 + 5 = 7 se asigna a la variable situada a la izquierda, es decir, N tomará un nuevo valor 7.

Se debe pensar en la variable como en una posición de memoria, cuyo contenido puede variar mediante instrucciones de asignación (un símil suele ser un buzón de correos, donde el número de cartas depositadas en él variará según el movimiento diario del cartero de introducción de cartas o del dueño del buzón de extracción de dichas cartas).

3.4.2. Instrucciones de lectura de datos (entrada)

Esta instrucción lee datos de un dispositivo de entrada.

¿Cuál será el significado de las instrucciones siguientes?

a) ***leer***(NUMERO, HORAS, TASA)

Leer del terminal los valores NUMERO, HORAS y TASAS, archivándolos en la memoria; si los tres números se teclean en respuesta a la instrucción son 12325, 32, 1200, significaría que se han asignado a las variables esos valores y equivaldría a la ejecución de las instrucciones.

```
NUMERO ← 12325
HORAS  ← 32
TASA   ← 1200
```

b) ***leer*** (A, B, C)

Si se leen del terminal 100, 200, 300, se asignarían a las variables los siguientes valores:

```
A = 100
B = 200
C = 300
```

3.4.3. Instrucciones de escritura de resultados (salida)

Estas instrucciones se escriben en un dispositivo de salida.

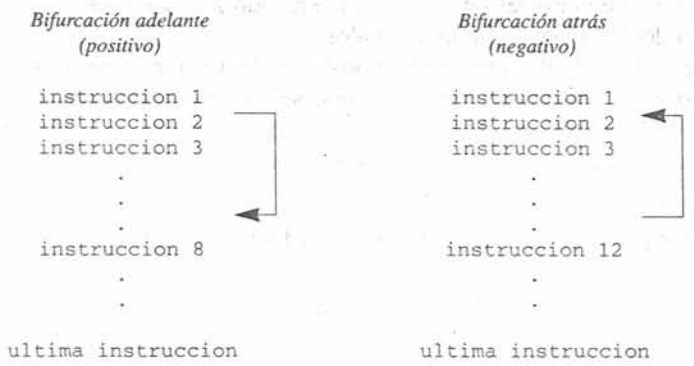
Explicar el resultado de la ejecución de las siguientes instrucciones:

```
A ← 100
B ← 200
C ← 300
escribir (A, B, C)
```

Se visualizarían en la pantalla o imprimirían en la impresora los valores 100, 200 y 300 que contienen las variables A, B y C.

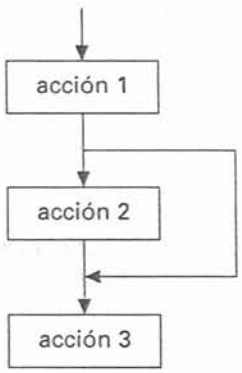
3.4.4. Instrucciones de bifurcación

El desarrollo lineal de un programa se interrumpe cuando se ejecuta una bifurcación. Las bifurcaciones pueden ser, según el punto del programa a donde se bifurca, hacia *adelante* o hacia *atrás*.

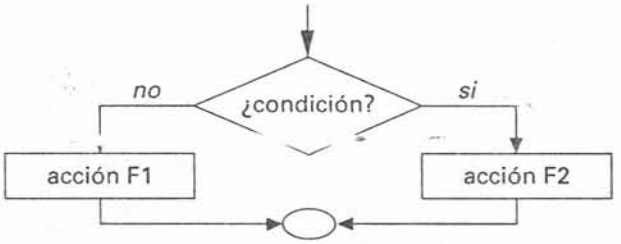


Las bifurcaciones en el flujo de un programa pueden realizarse de un modo incondicional o condicional.

Bifurcación incondicional: la bifurcación se realiza siempre que el flujo del programa pase por la instrucción sin necesidad del cumplimiento de ninguna condición.



Bifurcación condicional: la bifurcación depende del cumplimiento de una determinada condición. Si se cumple la condición, el flujo sigue ejecutando la acción F2. Si no se cumple, se ejecuta la acción F1.



3.5. ELEMENTOS BÁSICOS DE UN PROGRAMA

En programación se debe separar la diferencia entre el diseño del algoritmo y su implementación en un lenguaje específico. Por ello, se debe distinguir claramente entre los conceptos de programación y el medio en que ellos se implementan en un lenguaje específico. Sin embargo, una vez que se comprendan los conceptos de programación, cómo utilizarlos, la enseñanza de un nuevo lenguaje es relativamente fácil.

Los lenguajes de programación —como los restantes lenguajes— tienen elementos básicos que se utilizan como bloques constructivos, así como reglas para las que esos elementos se combinan. Estas reglas se denominan *sintaxis* del lenguaje. Solamente las instrucciones sintácticamente correctas pueden ser interpretadas por la computadora y los programa que contengan errores de sintaxis son rechazados por la máquina.

Los elementos básicos constitutivos de un programa o algoritmo son:

- *palabras reservadas (*`inicio`, `fin`, `si-entonces...`, *etc.),*
- *identificadores* (nombres de variables esencialmente),
- *caracteres especiales* (coma, apóstrofo, etc.),
- *constantes,*
- *variables,*
- *expresiones,*
- *instrucciones.*

Además de estos elementos básicos, existen otros elementos que forman parte de los programas, cuya comprensión y funcionamiento será vital para el correcto diseño de un algoritmo y naturalmente la codificación del programa.

Estos elementos son:

- *bucles,*
- *contadores,*
- *acumuladores,*
- *interruptores,*
- *estructuras:*
  - — *secuenciales,*
  - — *selectivas,*
  - — *repetitivas.*

El amplio conocimiento de todos los elementos de programación y el modo de su integración en los programas constituyen las técnicas de programación que todo buen programador debe conocer.

3.5.1. Bucles

Un *bucle* o *lazo* (*loop*)[1] es un segmento de un algoritmo o programa, cuyas instrucciones se repiten un número determinado de veces mientras se cumple una determinada *condición* (existe o es verdadera la condición). Se debe establecer un mecanismo para determinar las tareas repetitivas. Este mecanismo es una condición que puede ser verdadera o falsa y que se comprueba una vez a cada *paso* o *iteración* del bucle (total de instrucciones que se repiten en el bucle).

Un bucle consta de tres partes:

- decisión,
- cuerpo del bucle,
- salida del bucle.

El bucle de la Figura 3.3 es infinito, ya que las instrucciones (1), (2) y (3) se ejecutan indefinidamente, pues no existe salida del bucle, al no cumplirse una determinada condición.

Si tras la lectura de la variable N se coloca una condición, el bucle dejará de ser infinito y tendrá fin cuando la condición sea verdadera.

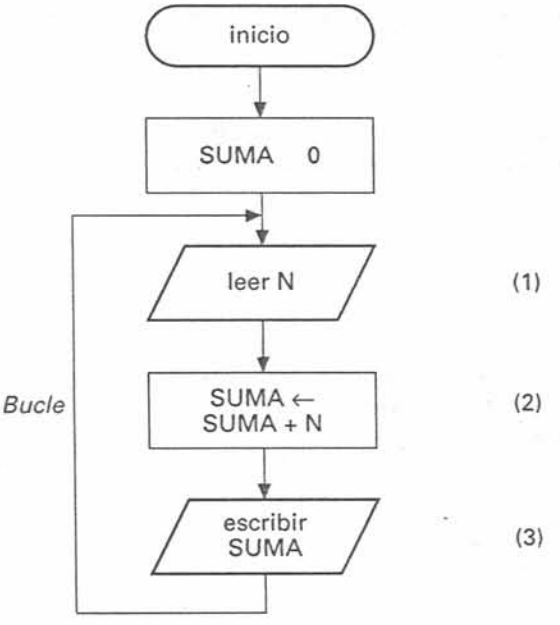


Figura 3.3. Bucle infinito.

[1] Este término se traduce también, a veces, por *lazo* o *ciclo*.

El diagrama de flujo escrito en *pseudocódigo* es:

```
inicio
  SUMA ← 0
  1: leer (N)
  si N = 0 entonces
    escribir(SUMA)
    ir_a fin
  si_no
    SUMA ← SUMA + N
  fin_si
  ir_a 1
fin
```

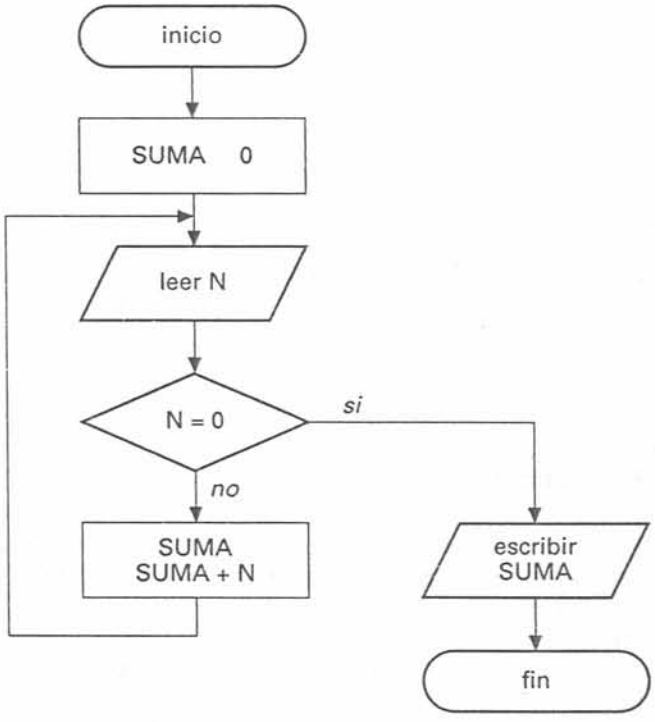


Figura 3.4. Bucle con fin.

Bucles anidados

En un algoritmo pueden existir varios bucles. Los bucles pueden ser *anidados* o *independientes*.

Los bucles son *anidados* cuando están dispuestos de tal modo que unos son interiores a otros; los bucles son *independientes* cuando son externos unos a otros (Figura 3.5).

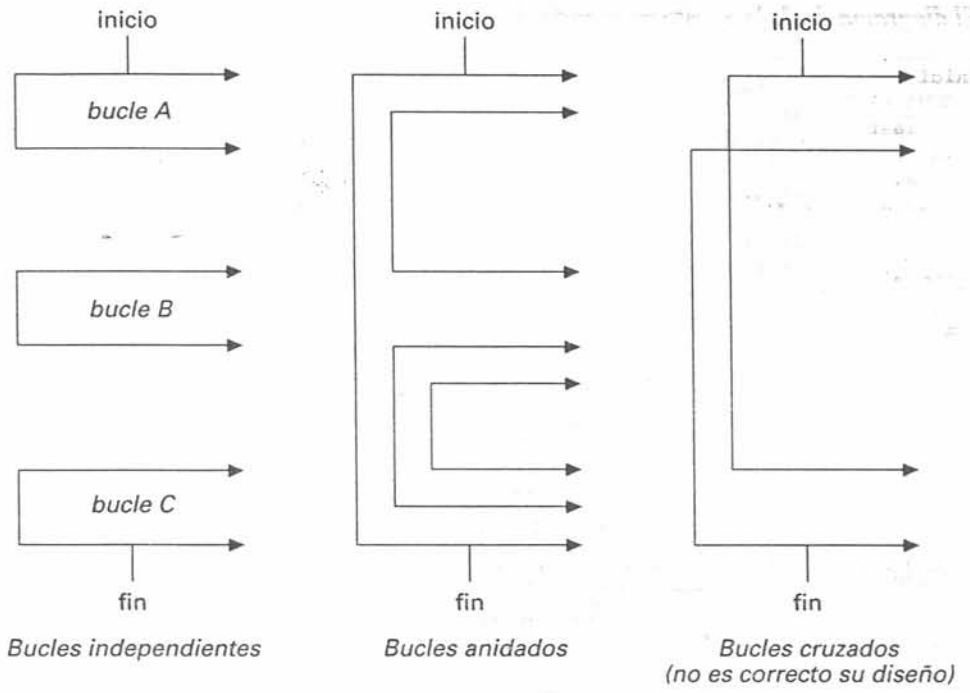


Figura 3.5. Tipos de bucles.

3.5.2. Contadores

Los procesos repetitivos son la base del uso de las computadoras. En estos procesos se necesitan normalmente contar los sucesos o acciones internas del bucle, como pueden ser los elementos de un fichero, el número de iteraciones a realizar por el bucle, etc. Una forma de controlar un bucle es mediante un contador.

Un *contador* es una variable cuyo valor se incrementa o decrementa en una cantidad constante en cada iteración.

La Figura 3.6 presenta un diagrama de flujo para un algoritmo que se desea repetir 50 veces; el contador se representa en este ejemplo con la variable CONT. La instrucción que representa a un contador es la asignación `CONT ← CONT + 1`.

La Figura 3.7 es otro ejemplo de un diagrama de flujo con contador; en este caso, negativo. Se dice también *descontar*.

El contador puede ser positivo (incrementos, uno en uno) o negativo (decrementos, uno en uno). En la Figura 3.6 el contador cuenta desde 1 a 50 y deja de contar cuando la variable CONT toma el valor 51 y se termina el bucle. En la Figura 3.7 el contador cuenta negativamente, o lo que es lo mismo, descuenta o decrementa; comienza a contar en n y se va decrementando hasta llegar a cero, en cuyo caso se termina el bucle y se realiza la acción escribir.

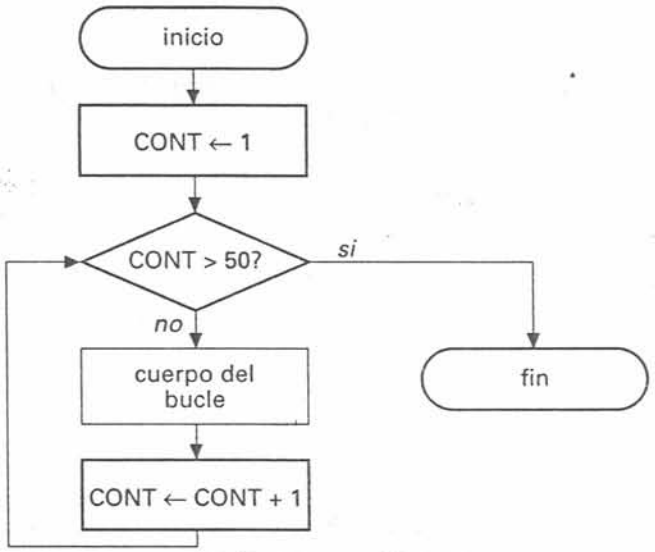
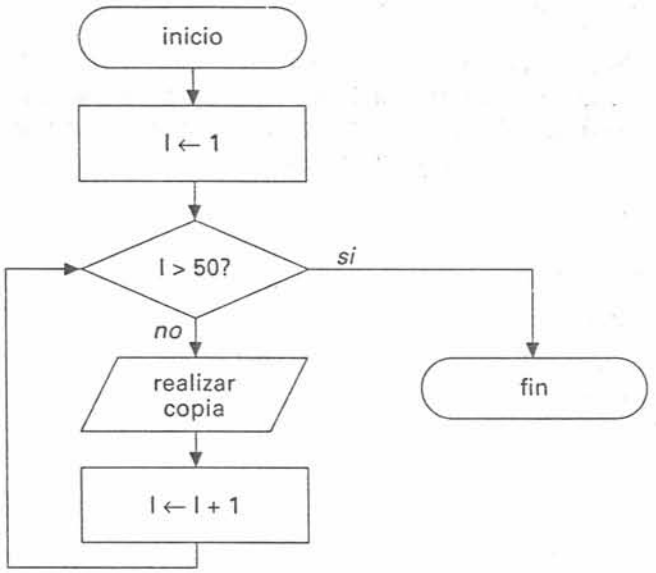


Figura 3.6. Ejemplo de contador positivo.

Ejemplo 3.1

Si desea hacer cincuenta copias de un informe de ventas, ¿cuál sería el diagrama de flujo que resolviese el problema?

La tarea implica realizar cincuenta veces la impresión de una copia del informe. Dado que el contador `I` se inicializa a `1`, el valor de este contador coincidirá con el número de copias que se hayan realizado. Cuando `I` tome el valor `51`, significará que se habrán realizado cincuenta copias.



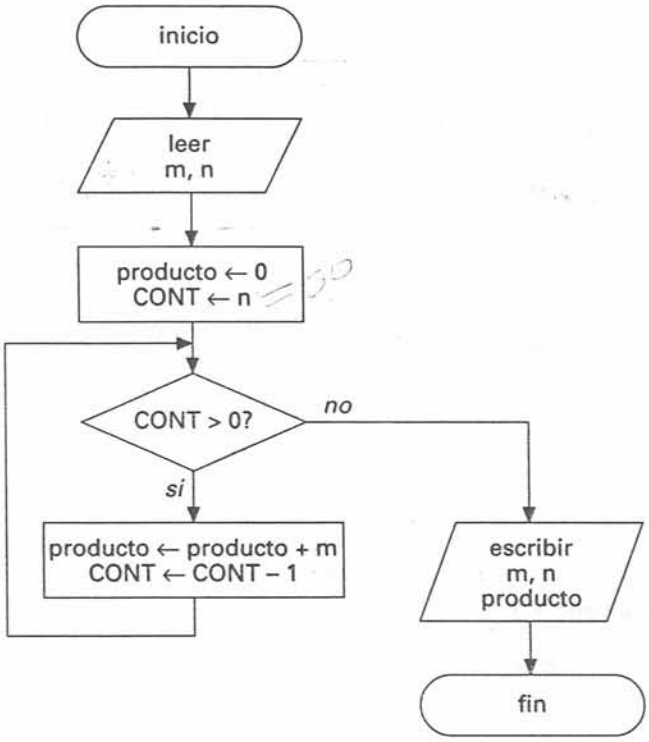


Figura 3.7. Bucle contador negativo.

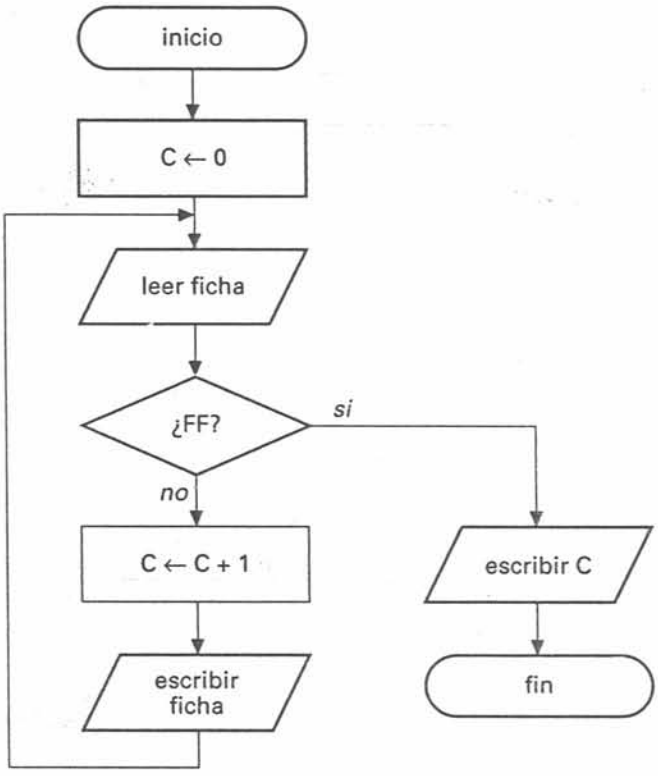
Ejemplo 3.2

Leer todas las fichas de un archivo e imprimir el número total de ellas.

Para realizar estas operaciones es usual poner en todos los archivos una ficha denominada fin de fichero o de archivo (FF), que se sitúa al final del archivo, con lo cual cuando se lee el archivo y se encuentra esta ficha (FF) se sabe que se ha alcanzado el final del archivo.

El *algoritmo* en lenguaje natural sería:

```
1. inicio
2. C ← 0
3. leer(ficha)
4. si (FF) ir_a (8)
5. C ← C + 1
6. escribir(ficha)
7. ir_a (3)
8. escribir(C)
9. fin
```



3.5.3. Acumulador

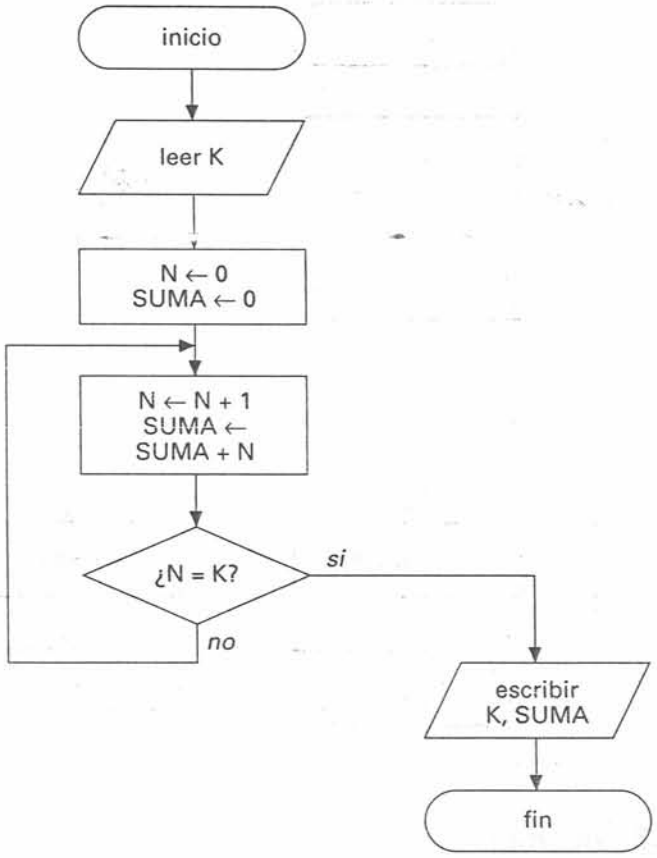
Un *acumulador* o *totalizador* es una variable cuya misión es almacenar cantidades variables resultantes de sumas sucesivas. Realiza la misma función que un contador, con la diferencia de que el incremento o decremento de cada suma es variable en lugar de constante, como en el caso del contador.

Se representa por la instrucción `S ← S + N`, donde `N` es una variable y no una constante.

Ejemplo 3.3

Sumar los primeros K números enteros. El algoritmo correspondiente es:

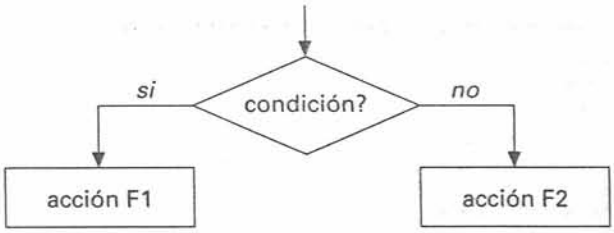
```
 1. inicio
 2. leer (K)
 3. N ← 0
 4. SUMA ← 0
 5. N ← N + 1
 6. SUMA ← SUMA + N
 7. si N = K, ir_a (9)
 8. ir_a (5)
 9. escribir(K, SUMA)
10. fin
```



3.5.4. Decisión o selección

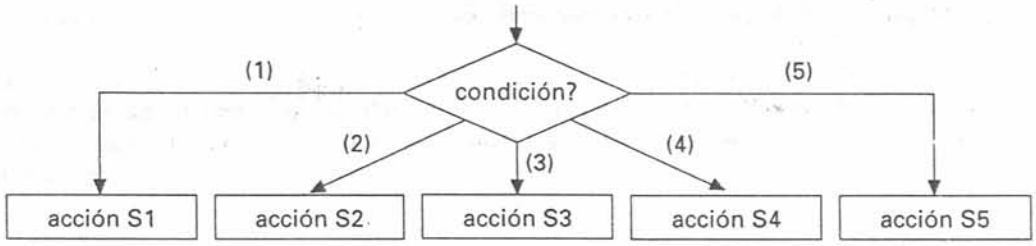
Cuando el programador desea especificar dos o más caminos alternativos en un algoritmo (o un programa), se deben utilizar estructuras de decisión o selección.

Una instrucción de decisión o selección evalúa una condición y, en función del resultado de esa condición, se bifurcará a un determinado punto.



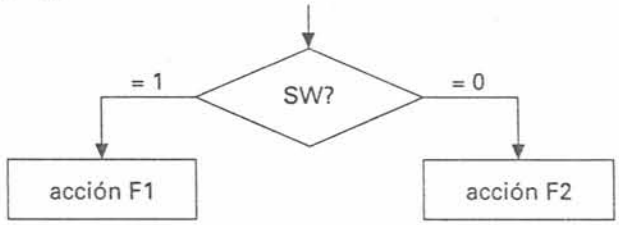
En el caso anterior, se tiene una instrucción alternativa con dos posibles caminos: si la condición es verdadera, se ejecuta la acción S1, y si la condición es falsa, se ejecuta la acción S2. Es posible

también realizar diferentes acciones para el caso de alternativas múltiples. En el caso de que la condición ofreciese cinco posibles soluciones, se representaría como

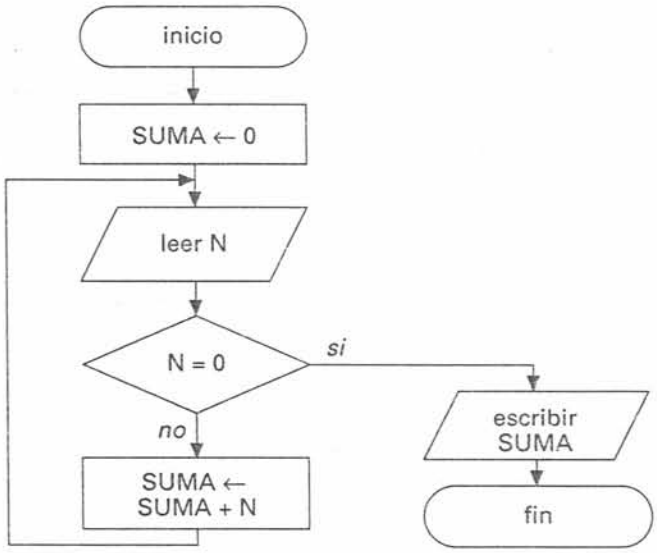


3.5.5. Interruptores

Un *interruptor* o *conmutador* (*switch*) —a veces se les denomina *indicador*, *bandera* (*flag*)— es una variable que puede tomar diversos valores a lo largo de la ejecución del programa y que permite comunicar información de una parte a otra del mismo. Los interruptores pueden tomar dos valores diferentes, 1 y 0. (De ahí su nombre de interruptor, `"encendido" / "apagado"`, `"abierto" / "cerrado"`.)



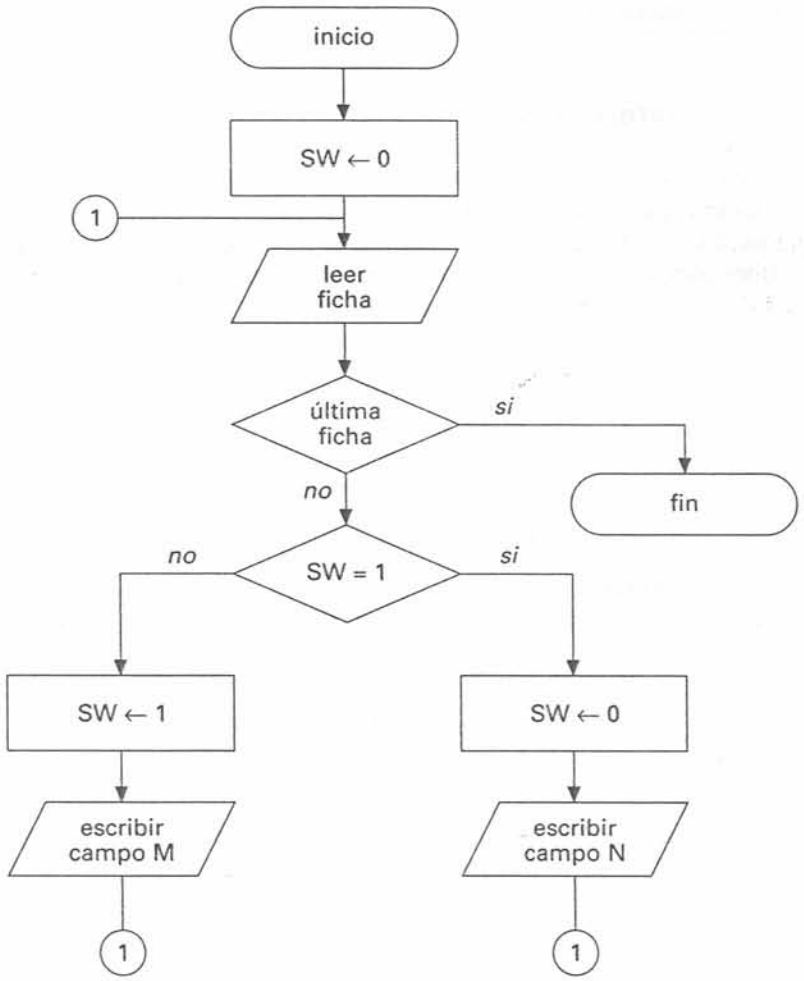
Si `SW` es igual a `1`, se ejecuta la acción `S1`, y si `SW` es igual a `0`, se ejecuta la acción `S2`.



Ejemplo 3.4

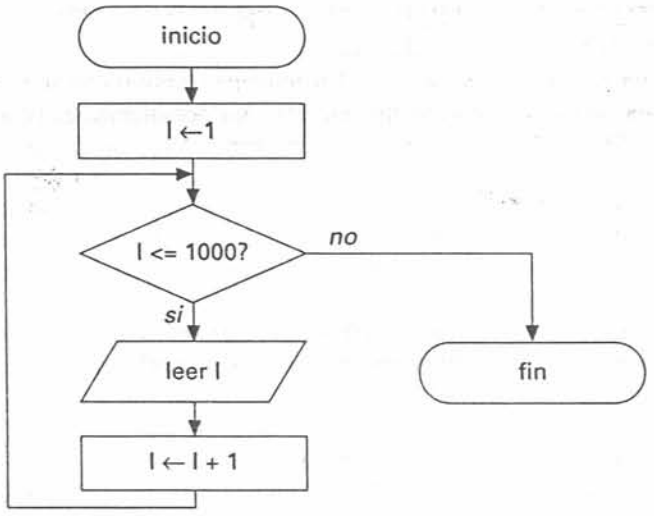
Se tiene un archivo con un conjunto de fichas con dos campos, `M` y `N`. *Se desea listar el campo* `M` *de las fichas pares y el campo* `N` *de las fichas impares.*

Los interruptores `-SW-` se suelen inicializar a un valor determinado (`0` o `1`) y luego se ha de modificar su valor alternativamente a medida que se leen las fichas impares y pares. De este modo, cuando `SW = 0`, se leerán las fichas impares y cuando `SW = 1` se leerán las fichas pares.



Ejemplo 3.5

Diseñar un diagrama de flujo que escriba los números de 1 a 1000.

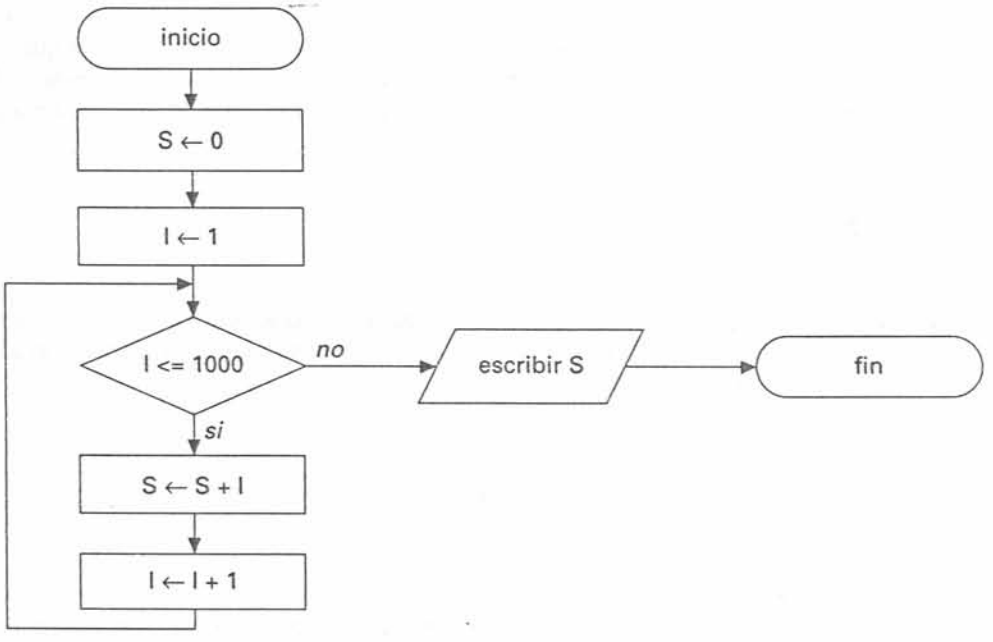


Ejemplo 3.6

Calcular la suma de los números de `1` *a* `1000`.

```
S = 1 + 2 + 3 + 4 + ... + 999 + 1000
```

La suma se realiza con totalizador o acumulador `S`, que irá realizando las sucesivas sumas parciales, `1`, `1 + 2`, `1 + 2 + 3`, `1 + 2 + 3 + 4`, etc. Para ello se inicializa la variable del sumador `S` a cero y el contador `I` representará los números sucesivos `1` a `1000`.



3.6. ESCRITURA DE ALGORITMOS/PROGRAMAS

La escritura de un algoritmo mediante una herramienta de programación debe ser lo más clara posible y estructurada, de modo que su lectura facilite considerablemente el entendimiento del algoritmo y su posterior codificación en un lenguaje de programación.

Los algoritmos deben ser escritos en lenguajes similares a los programas. En nuestro libro utilizaremos esencialmente el lenguaje algorítmico, basado en pseudocódigo, y la estructura del algoritmo requerirá la lógica de los programas escritos en el lenguaje de programación estructurado, por ejemplo, Pascal.

Un algoritmo constará de dos componentes: *una cabecera de programa* y *un bloque algoritmo*.

La *cabecera de programa* es una acción simple que comienza con la palabra **algoritmo**. Esta palabra estará seguida por el nombre asignado al programa completo.

El *bloque algoritmo* es el resto del programa y consta de dos componentes o secciones: *las acciones de declaración* y *las acciones ejecutables*.

Las *declaraciones* definen o declaran las variables y constantes que tengan nombres. Las *acciones ejecutables* son las acciones que posteriormente deberá realizar la computación cuando el algoritmo convertido en programa se ejecute.

```
Algoritmo
cabecera del programa
seccion de declaracion
seccion de acciones
```

3.6.1. Cabecera del programa o algoritmo

Todos los algoritmos y programas deben comenzar con una cabecera en la que se exprese el identificador o nombre correspondiente con la palabra reservada que señale el lenguaje. En los lenguajes de programación, la palabra reservada suele ser **program**. En algoritmia se denomina **algoritmo**.

```
algoritmo DEMO1
```

3.6.2. Declaración de variables

En esta sección se declaran o describen todas las variables utilizadas en el algoritmo, listándose sus nombres y especificando sus tipos. Esta sección comienza con la palabra reservada **var** (abreviatura de *variable*) y tiene el formato

```
var
   tipo-1: lista de variables-1
   tipo-2: lista de variables-2
    .
    .
   tipo-n: lista de variables-n
```

donde cada *lista de variables* es una variable simple o una lista de variables separadas por comas y cada *tipo* es uno de los tipos de datos básicos (*entero*, *real*, *carácter* o *lógico*).

Por ejemplo, la sección de declaración de variables

```
var
  entera: Numero-Empleado
  real:   Horas
  real:   Impuesto
  real:   Salario
```

o de modo equivalente

```
var
  entera: Numero-Empleado
  real: Horas, Impuesto, Salario
```

declara que sólo las tres variables `Hora`, `Impuesto` y `Salario` son de tipo real.

Es una buena práctica de programación utilizar nombres de variables significativos que sugieran lo que ellas representan, ya que eso hará más fácil y legible el programa.

También es buena práctica incluir breves comentarios que indiquen cómo se utiliza la variable.

```
var
  entera: Numero_empleado      //numero de empleado
    real: Horas                //horas trabajadas
          Impuesto             //impuesto a pagar
          Salario              //cantidad ganada
```

3.6.3. Declaración de constantes numéricas

En esta sección se declaran todas las constantes que tengan nombre. Su formato es:

```
const
  pi = 3.141592
  tamaño = 43
  horas = 6.50
```

Los valores de estas constantes ya no pueden variar en el transcurso del algoritmo.

3.6.4. Declaración de constantes y variables carácter

Las constantes de carácter simple y cadenas de caracteres pueden ser declaradas en la sección del programa **`const`**, al igual que las constantes numéricas.

```
const
  estrella = '*'
  frase = '12 de octubre'
  mensaje = 'Hola mi nene'
```

Las variables de caracteres se declaran de dos modos:

1. Almacenar un solo carácter.

   ```
   var caracter: nombre, inicial, nota, letra
   ```

   Se declaran `nombre`, `inicial`, `nota` y `letra`, que almacenarán sólo un carácter.

2. Almacenar múltiples caracteres (*cadenas*). El almacenamiento de caracteres múltiples dependerá del lenguaje de programación. Así, en

   BASIC *nombre variable cadena = cadena de caracteres*

   ```
   NOMBRE$ = «Pepe Luis Garcia Rodriguez»
   ```

   Pascal formato tipo array (véase Capítulo 6).

   Existen algunas versiones de Pascal, como es el caso de Turbo Pascal, que tienen implementados un tipo de datos denominados *string* (cadena) que permite declarar variables de caracteres o de cadena que almacenan palabras compuestas de diferentes caracteres.

```
var nombre: string[20];      en Turbo Pascal
var cadena[20]: nombre       en pseudocódigo
```

3.6.5. Comentarios

La documentación de un programa es el conjunto de información interna y externa al programa, que facilitará su posterior mantenimiento y puesta a punto.

La documentación puede ser *interna* y *externa*. La *documentación interna* es la que se acompaña en el código o programa fuente y se realiza a base de comentarios significativos. Estos comentarios se representan con diferentes notaciones, según el tipo de lenguaje de programación. La *documentación externa* se acompañará en información ajena al programa y proporcionada por el programador.

BASIC

1. Los comentarios se incluyen dentro de líneas que comienzan con la palabra reservada REM.

```
100 REM *** autor J.R. Mackoy ***
200 REM subrutina ordenacion
.............
```

2. En las versiones BASIC de Microsoft (BASICA, GWBASIC y MBASIC) los comentarios se incluyen en líneas que comienzan por el carácter apóstrofo (').

```
100 '*** autor J.R. Mackoy ***
200 'subrutina ordenacion
```

3. Existen algunas versiones de BASIC como HP BASIC, VAX BASIC, etc., que utilizan el carácter admiración (!) en lugar del carácter apóstrofo.

```
100 !*** autor J.R. Mackoy ***
200 !subrutina ordenacion
```

FORTRAN

Las líneas que contiene comentarios comienzan en su columna 1 con el carácter asterisco (*) o la letra C y se pueden escribir a partir de la columna 2:

```
columna 1   7

  C           *** autor J.R. Mackoy ***
  *               subrutina ordenacion
```

PASCAL

Los comentarios se encierran entre los símbolos

```
(*                *)
```

o bien:

```
{                 }

(* autor J.R. Mackoy *)
{subrutina ordenacion}
```

MODULA-2

Los comentarios se encierran entre los símbolos

```
(*                *)
```

C

Los comentarios se encierran entre caracteres llaves ({})

```
{Esto es un comentario en C}
```

C++

Admite, además de los caracteres llave, la doble barra inclinada

```
//Esto es un comentario en C++
```

NOTA: A lo largo del libro utilizaremos preferentemente para representar nuestros comentarios los símbolos `//`. Sin embargo, algunos autores de algoritmos, a fin de independizar la simbología del lenguaje, suelen representar los comentarios entre corchetes ([ ]).

3.6.6. Estilo de escritura de algoritmos/programas

El método que seguiremos normalmente a lo largo del libro para escribir algoritmos será el descrito al comienzo del Apartado 3.6.

```
algoritmo Identificador            //cabecera
//seccion de declaraciones
var tipo de datos: lista de identificadores
const lista de identificadores = valor
inicio                             //cuerpo del algoritmo
  <sentencia S1>
  <sentencia S2>
    .
    .
    .
  <sentencia Sn>
fin
```

NOTAS:

1. En ocasiones, la declaración de constantes y variables las omitiremos o se describirán en una tabla de variables que hace sus mismas funciones.
2. Las cadenas de caracteres se encerrarán entre comillas simples.
3. Utilizar siempre sangrías en los bucles o en aquellas instrucciones que proporcionen legibilidad al programa, como **inicio** y **fin**.

Ejemplo 3.7

```
algoritmo raices
//resuelve una ecuacion de 2° grado
var real: a,b,c,d

inicio
  leer(a, b, c)
  D ← b ^ 2 - 4 * a * c
  si D < 0 entonces
       escribir ('raices complejas')
    si-no
      si D = 0 entonces
      escribir (-b/2*a)
      si-no
      escribir((-b-raiz2(D))/(2*a))
      escribir((-b+raiz2(D))/(2*a))
      fin-si
  fin-si
fin
```

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

3.1. Determinar la cantidad total a pagar por una llamada telefónica, teniendo en cuenta lo siguiente:

- toda llamada que dure menos de tres minutos (cinco pasos) tiene un coste de 10 pesetas,
- cada minuto adicional a partir de los tres primeros es un paso de contador y cuesta 5 pesetas.

Análisis

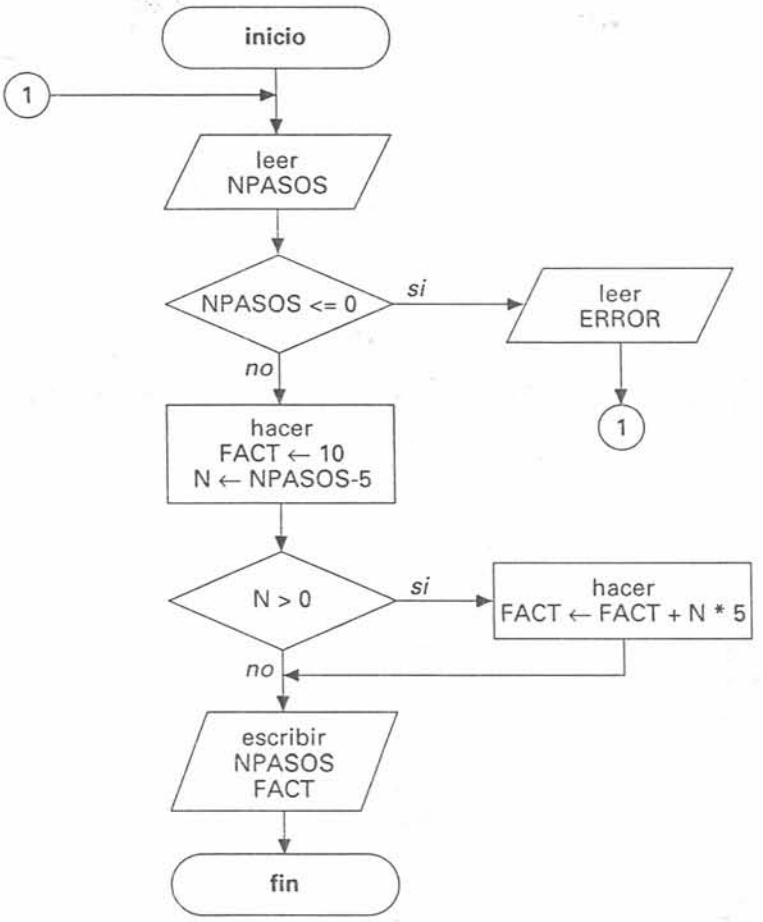
El algoritmo de resolución del problema entraña los siguientes pasos:

1. Inicio.
2. Leer el número de pasos (npasos) hablados por telefono.
3. Comprobar que el numero de pasos es mayor que cero, ya que realmente se ha realizado la llamada si el numero de pasos es distinto de cero (positivo). Si el numero de pasos es menor a cero, se producira un error.
4. Calcular el precio de la conferencia de acuerdo con los siguientes conceptos:

   - si el numero de pasos es menor que 5, el precio es de 10 pesetas,
   - si el numero de pasos es mayor que 5, es preciso calcular los pasos que exceden de 5, ya que estos importan 5 pesetas cada uno; al producto de los pasos sobrantes por cinco pesetas se le suman 10 pesetas y se obtendra el precio total.

Variables

NPASOS	Número de pasos de la llamada
N	Número de pasos que exceden a 5
FACT	Importe o precio de la llamada



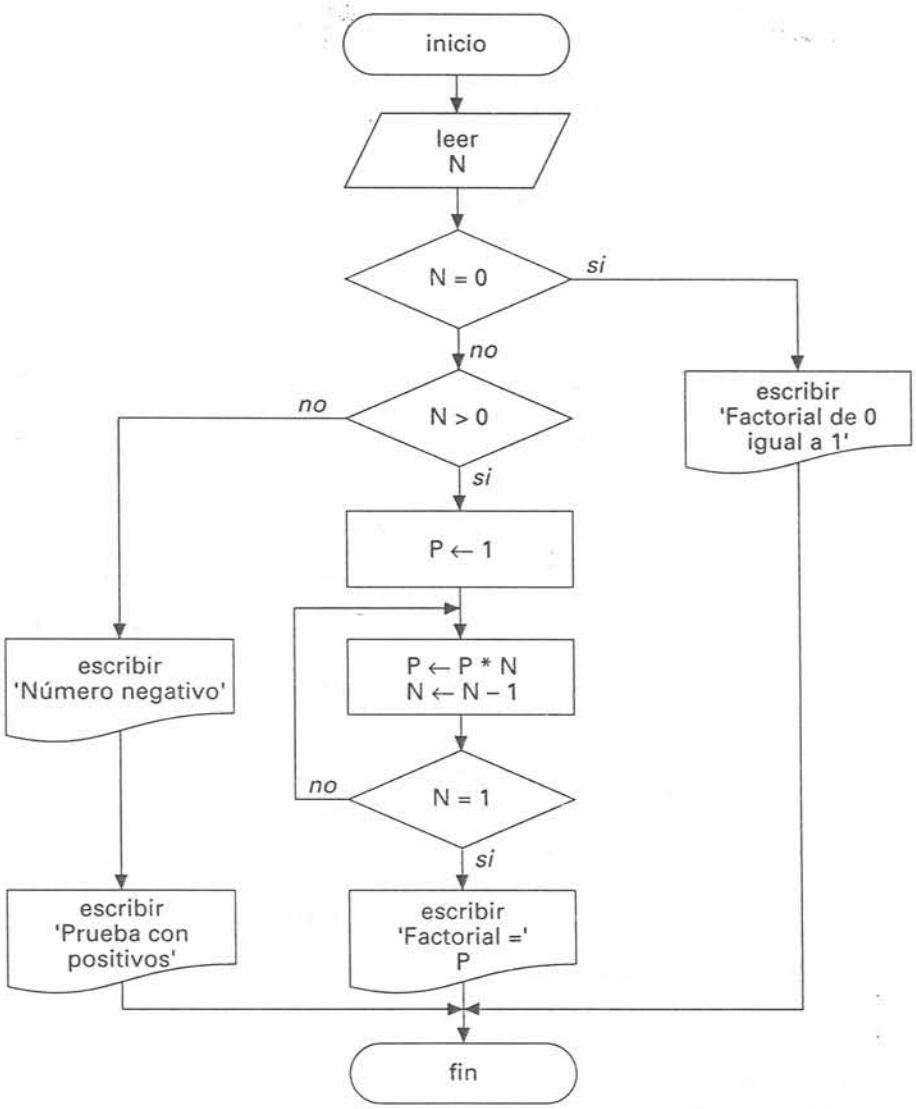
3.2. Escribir un algoritmo que calcule el producto de los *n* primeros números naturales.

Análisis

El problema puede calcular el producto `N * (N - 1) * (N - 2) * ... * 3 * 2 * 1`, que en términos matemáticos se le conoce con el nombre de `FACTORIAL de N`.
El algoritmo que resuelve el problema será el siguiente:

```
1. Leer N.
2. Caso de que N = 0, visualizar "Factorial de 0 igual 1".
3. Comprobar que N > 0 (los numeros negativos no se consideran).
4. Hacer la variable P que va a contener el productor igual a 1.
```

```
5. Realizar el producto P = P * N.
   Disminuir en una unidad sucesivamente hasta llegar a N = 1, y de modo
   simultaneo los productos P * N.
6. Visualizar P.
7. Fin.
```



Pseudocódigo

```
algoritmo Factorial
var
  entero: N
  real: P
```

```
inicio
  leer(N)
  si N = 0 entonces
    escribir('Factorial de 0 igual a 1')
  si_no
    si N > 0 entonces
          P ← 1
      1: P ← P * N
          N ← N - 1
      si N = 1 entonces
        escribir('Factorial  =', P)
      si_no
        ir_a 1
      fin_si
    si_no
      escribir('Numero negativo')
      escribir('Pruebe con positivos')
    fin_si
  fin_si
fin
```

3.3. Diseñar el diagrama de flujo para resolver una ecuación de segundo grado $Ax^2 + Bx + C = 0$.

Análisis

La ecuación de segundo grado es $Ax^2 + Bx + C = 0$ y las soluciones o raíces de la ecuación son:

$$X1 = \frac{-B + \sqrt{B^2 - 4AC}}{2A} \qquad X2 = \frac{-B + \sqrt{B^2 - 4AC}}{2A}$$

Para que la ecuación de segundo grado tenga solución es preciso que el discriminante sea mayor o igual que 0.
El discriminante de una ecuación de segundo grado es:

$D = B^2 - 4AC$

Por consiguiente, si

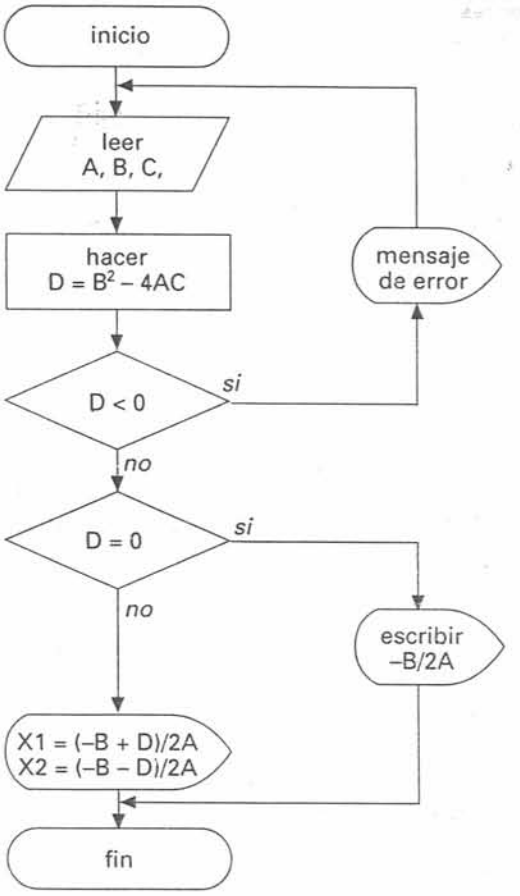
$D = 0$ $\quad X1 = -B/2A \quad X2 = -B/2A$
$D < 0$ $\quad X1$ y $X2$

no tienen solución real.

En consecuencia, el algoritmo que resolverá el problema es el siguiente:

1. Inicio.
2. Introducir los coeficientes A, B y C.
3. Calculo del discriminante $D = B^2 - 4AC$.
4. Comprobar el valor de D:
   - si D es menor que 0, visualizar un mensaje de error,
   - si D es igual a 0, se obtienen dos raices iguales X1 = X2 = -B / 2A
   - si D es mayor que 0, se calculan las dos raices X1 y X2.
5. Fin del algoritmo.

Diagrama de flujo



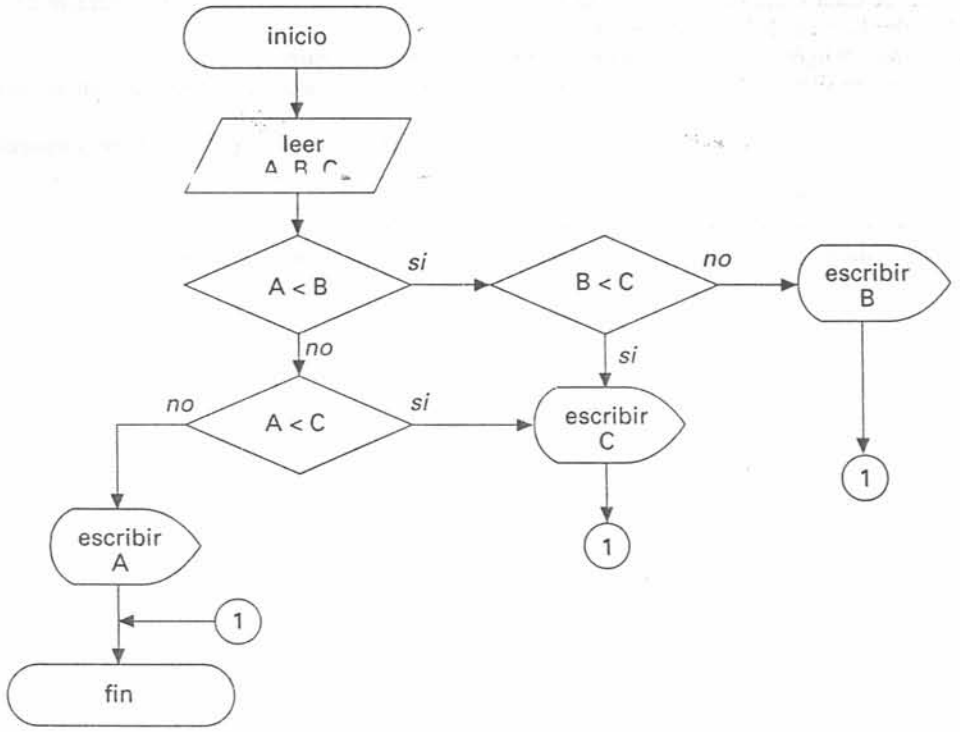
3.4. Escribir un algoritmo que acepte tres números enteros e imprima el mayor de ellos.

Análisis

El diseño del algoritmo requiere de una serie de comparaciones sucesivas. Las operaciones sucesivas son las siguientes:

```
1. Inicio.
2. Introducir los tres numeros A, B, C.
3. Comparar A y B:
   • si A es menor que B:
     – comparar B y C:
       • si B es mayor que C, el mayor es B,
       • si B es menor que C, el mayor es C.
   • si A es mayor que B:
     – comparar A y C:
       • si A es menor que C, el mayor es C,
       • si A es mayor que C, el mayor es A.
```

Diagrama de flujo



EJERCICIOS

3.1. Se desea calcular independiente la suma de los números pares e impares comprendidos entre 1 y 200.
3.2. Leer una serie de números distintos de cero (el último número de la serie es –99) y obtener el número mayor. Como resultado se debe visualizar el número mayor y un mensaje de indicación de número negativo, caso de que se haya leído un número negativo.
3.3. Calcular y visualizar la suma y el producto de los números pares comprendidos entre 20 y 400, ambos inclusive.
3.4. Leer 500 números enteros y obtener cuántos son positivos.
3.5. Se trata de escribir el algoritmo que permita emitir la factura correspondiente a una compra de un artículo determinado, del que se adquieren una o varias unidades. El IVA a aplicar es del 15 por 100 y si el precio bruto (precio venta más IVA) es mayor de 50.000 pesetas, se debe realizar un descuento del 5 por 100.
3.6. Calcular la suma de los cuadrados de los cien primeros números naturales.
3.7. Sumar los números pares del 2 al 100 e imprimir su valor.
3.8. Sumar diez números introducidos por teclado.
3.9. Calcular la media de cincuenta números e imprimir su resultado.
3.10. Calcular los *N* primeros múltiplos de 4 (4 inclusive), donde *N* es un valor e introducido por teclado.
3.11. Diseñar un diagrama que permita realizar un contador e imprimir los cien primeros números enteros.
3.12. Dados diez números enteros, visualizar la suma de los números pares de la lista, cuántos números pares existen y cuál es la media aritmética de los números impares.

3.13. Calcular la nota media de los alumnos de una clase considerando n-número de alumnos y c-número de notas de cada alumno.
3.14. Escribir la suma de los diez primeros números pares.
3.15. Escribir un algoritmo que lea los datos de entrada de un archivo que sólo contiene números y sume los números positivos.
3.16. Desarrollar un algoritmo que determine en un conjunto de cien números naturales:
- ¿Cuántos son menores de 15?
- ¿Cuántos son mayores de 50?
- ¿Cuántos están comprendidos entre 25 y 45?

3.17. Sumar una secuencia de enteros detectando si están ordenados.
3.18. ¿*N* es primo?

CAPÍTULO 4

INTRODUCCIÓN A LA PROGRAMACIÓN ESTRUCTURADA

CONTENIDO

4.1. Técnicas de programación.
4.2. Programación modular.
4.3. Programación estructurada.
4.4. Estructura secuencial.
4.5. Estructuras selectivas.
4.6. Estructuras repetitivas.
4.7. Estructuras de decisión anidadas.
4.8. Estructuras repetitivas anidadas.
4.9. La instrucción `ir_a (goto)`.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

En la actualidad, dado el tamaño considerable de las memorias centrales y las altas velocidades de los procesadores —citemos, por ejemplo, los microprocesadores como el 80486, Pentium y Pentium Pro de Intel, 68030 de Motorola, PowerPC de Apple e IBM, de 16 y 32 bits—, *el estilo de escritura de los programas se vuelve una de las características más sobresalientes en las técnicas de programación. La legibilidad de los algoritmos y posteriormente de los programas* exige que su diseño sea fácil de comprender y su flujo lógico fácil de seguir.

La *programación modular* enseñará la descomposición de un programa en módulos más simples de programar, y la programación estructurada permitirá la escritura de programas fáciles de leer y modificar. En *un programa estructurado* el flujo lógico se gobierna por las estructuras de control básicas:

1. *secuenciales,*
2. *repetitivas,*
3. *selección.*

En este capítulo se introducen los tres conceptos anteriores: *secuencial* —ejecución instrucción a instrucción—, *repetitivo* —capacidad de repetir una serie de operaciones— y *selectivo* —posibilidad de seleccionar una acción entre un conjunto de alternativas específicas.

El estudio de las estructuras de control se realiza basado en las herramientas de programación ya estudiadas: diagramas de flujo, diagramas N-S y pseudocódigos.

4.1. TÉCNICAS DE PROGRAMACIÓN

Las herramientas de programación vistas en el Capítulo 2, fundamentalmente los diagramas de flujo, fueron la primera representación gráfica utilizada para el diseño de algoritmos y programas. Este tipo de herramienta constituye el fundamento de programación convencional clásica y es de utilidad para la solución de problemas de pequeña y mediana complejidad. Sin embargo, para problemas complejos el diseño de algoritmos implica un aumento considerable de la representación gráfica, una reducción de la legibilidad del algoritmo y un aumento de la memoria física de la computadora ocupada por el programa. Para evitar estos inconvenientes, las nuevas teorías de programación se centran en las técnicas de *programación modular* y *programación estructurada*, de modo que se pueda conseguir un programa eficaz. El diseño de un programa entraña la descomposición del problema en módulos (partes independientes) —*programación modular*—, la programación de cada módulo mediante métodos estructurados —*programación estructurada*— y su unión posterior mediante procedimientos ascendentes o descendentes.

4.2. PROGRAMACIÓN MODULAR

La *programación modular* es uno de los métodos de diseño más flexibles y potentes para mejorar la productividad de un programa. En programación modular, el programa se divide en *módulos* (partes independientes), cada una de las cuales ejecuta una única actividad o tarea y se codifican independientemente de otros módulos. Cada uno de estos módulos se analizan, codifican y ponen a punto por separado.

Cada programa contiene un módulo denominado *programa principal* que controla todo lo que sucede; se transfiere el control a *submódulos* (posteriormente se denominarán *subprogramas*), de modo que ellos puedan ejecutar sus funciones; sin embargo, cada submódulo devuelve el control al módulo principal cuando se haya completado su tarea. Si la tarea asignada a cada submódulo es demasiado compleja, éste deberá romperse en otros módulos más pequeños. El proceso sucesivo de subdivisión de módulos continúa hasta que cada módulo tenga solamente una tarea específica que ejecutar. Esta tarea puede ser *entrada*, *salida*, *manipulación de datos*, *control de otros módulos* o *alguna combinación de éstos*. Un módulo puede transferir temporalmente (*bifurcación*) el control a otro módulo; sin embargo, cada módulo debe devolver el control al módulo del cual se recibe originalmente el control.

Los módulos son independientes en el sentido en el que ningún módulo puede tener acceso directo a cualquier otro módulo, con excepción del módulo al que llama y sus propios submódulos. Sin embargo, los resultados producidos por un módulo pueden ser utilizados por cualquier otro módulo cuando se transfiera a ellos el control.

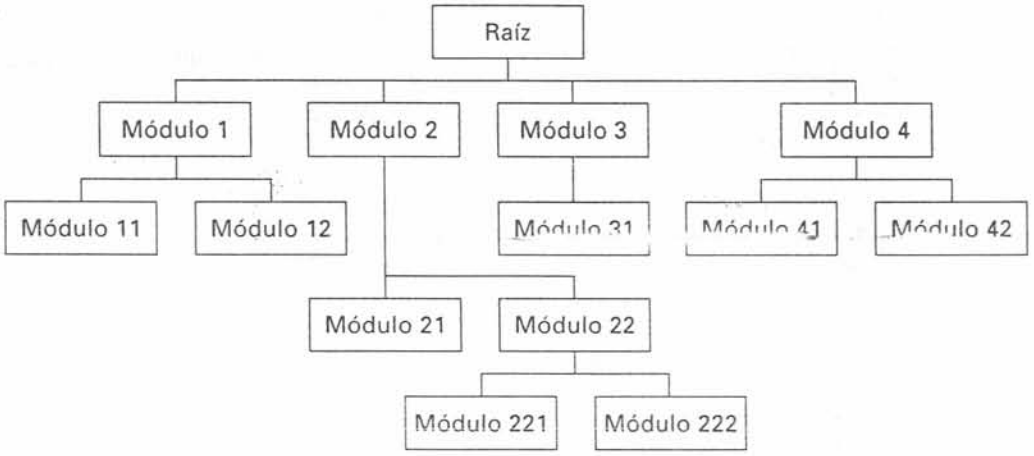


Figura 4.1. Programación modular.

Dado que los módulos son independientes, diferentes programadores pueden trabajar simultáneamente en diferentes partes del mismo programa. Esto reducirá el tiempo del diseño del algoritmo y posterior codificación del programa. Además, un módulo se puede modificar radicalmente sin afectar a otros módulos, incluso sin alterar su función principal.

La descomposición de un programa en módulos independientes más simples se conoce también como el método de «divide y vencerás» (*divide and conquer*). Se diseña cada módulo con independencia de los demás y siguiendo un método ascendente o descendente se llegará hasta la descomposición final del problema en módulos en forma jerárquica.

4.2.1. Tamaño de los módulos

Una dificultad en la programación modular reside en la determinación del tamaño de un módulo. Aunque no exista una regla fija que permita definir el tamaño estándar del módulo, una buena aproximación es considerar módulos del tamaño físico de una página (30 o 40 líneas de instrucciones). Evidentemente no siempre se conseguirá descomponer un problema en módulos de este tamaño, pero el programador deberá realizar esfuerzos para su diseño.

Puede no ser claro lo que debe incluirse dentro de un módulo. Desgraciadamente, no hay reglas, pero tal como se ha dicho antes, un módulo de una o dos líneas probablemente será demasiado corto y será mejor incorporarlo en otro módulo; los módulos grandes de 30 o más líneas probablemente ejecutarán tareas múltiples y se deberán romper en unidades más pequeñas.

Ejemplo 4.1

Se desea diseñar un algoritmo que realice las siguientes tareas:

- impresión de cabeceras de un informe,
- lectura de datos,
- ejecutar cálculos,
- imprimir líneas detalladas de información,
- imprimir totales.

Este programa se puede descomponer en módulos, de modo que exista un módulo principal o de control y diferentes submódulos.

Módulo principal:

- Llamada a submódulo «impresión de cabeceras».
- Llamada a submódulo «proceso de datos».
- Llamada a submódulo «impresión de totales».
- Fin de proceso.

Submódulo `impresión de cabeceras`*:*

- Instrucción para impresión de cabeceras.
- Retorno al módulo principal.

Submódulo `proceso de datos`*:*

- Lectura de datos.
- Ejecución de cálculos
- Impresión detallada de líneas.
- Retorno al módulo principal.

Submódulo `impresión de totales`*:*

- Instrucciones de impresión de totales.
- Retorno al módulo principal.

La representación gráfica se muestra en la Figura 4.2, de modo que existirá un módulo principal y tres submódulos.

4.2.2. Implementación de los módulos

La programación modular se implementa utilizando módulos que toman diferentes nombres, según el lenguaje de programación: *funciones* en C/C++, *subrutinas* en BASIC, *procedimientos* en Pascal, *subrutinas* en FORTRAN, *secciones* en COBOL y *funciones* como módulos comunes en todos los lenguajes (C, C++, Ada, etc.).

En el Capítulo 5 se analizarán con detenimiento la programación modular, el diseño de los módulos y la comunicación entre ellos y el módulo principal.

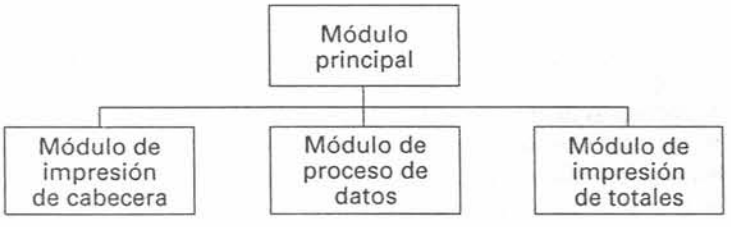


Figura 4.2. Estructura modular simple.

4.3. PROGRAMACIÓN ESTRUCTURADA

El término *programación estructurada* se refiere a un conjunto de técnicas que han ido evolucionando desde los primeros trabajos de Edsgar Dijkstra. Estas técnicas aumentan considerablemente la productividad del programa reduciendo el elevado tiempo requerido para escribir, verificar, depurar y mantener los programas. La programación estructurada utiliza un número limitado de estructuras de control que minimizan la complejidad de los problemas y, por consiguiente, reducen los errores.

La programación estructurada hace los programas más fáciles de escribir, verificar, leer y mantener. Los programas deben estar dotados de una estructura.

La ***programación estructurada*** es el conjunto de técnicas que incorporan:

- *diseño descendente (top-down),*
- *recursos abstractos,*
- *estructuras básicas.*

4.3.1. Recursos abstractos

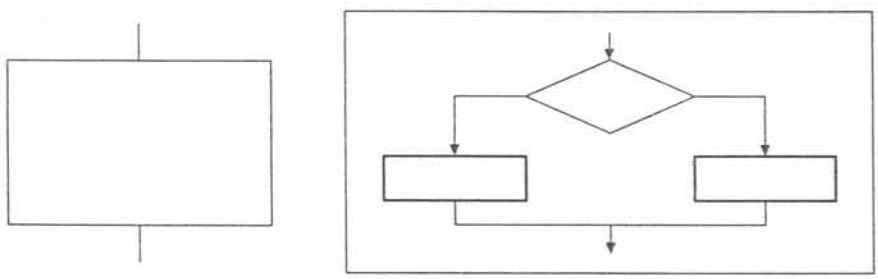
La programación estructurada se auxilia de los *recursos abstractos* en lugar de los recursos concretos de que se dispone (un determinado lenguaje de programación).

Descomponer un programa en términos de recursos abstractos —según Dijkstra— consiste en descomponer una determinada acción compleja en función de un número de acciones más simples, capaces de ser ejecutadas por una computadora y que constituirán sus instrucciones.

4.3.2. Diseño descendente (*top-down*)

El ***diseño descendente*** (*top-down*) es el proceso mediante el cual un problema se descompone en una serie de niveles o pasos sucesivos de refinamiento (*stepwise*). La metodología descendente consiste en efectuar una relación entre las sucesivas etapas de estructuración, de modo que se relacionen unas con otras mediante entradas y salidas de información. Es decir, se descompone el problema en etapas o estructuras jerárquicas, de modo que se puede considerar cada estructura desde dos puntos de vista: *qué hace* y *cómo lo hace*.

Si se considera un nivel *n* de refinamiento, las estructuras se consideran de la siguiente forma:
El diseño descendente se puede ver en la Figura 4.4.



Nivel *n*: vista desde el exterior
"¿qué hace?"

Nivel *n* + 1: vista desde el interior
"¿cómo lo hace?"

Figura 4.3. Vistas interior y exterior de un módulo.

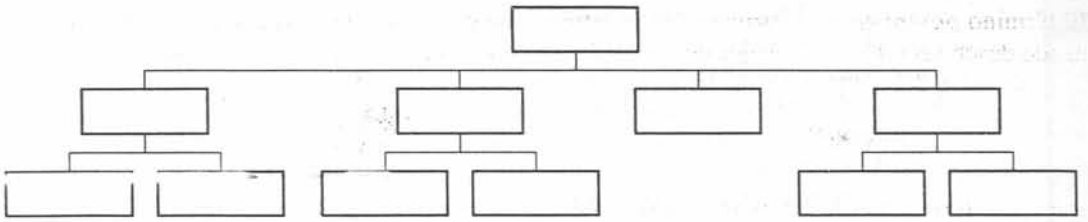


Figura 4.4. Diseño descendente.

4.3.3. Teorema de la programación estructurada: estructuras básicas

En mayo de 1966 Böhm y Jacopini demostraron que un *programa propio* puede ser escrito utilizando solamente tres tipos de estructuras de control:

- *secuenciales,*
- *selectivas,*
- *repetitivas.*

Un *programa se define como propio* si cumple las siguientes características:

- posee un solo punto de entrada y uno de salida o fin para control del programa,
- existen caminos desde la entrada hasta la salida que se pueden seguir y que pasan por todas partes del programa,
- todas las instrucciones son ejecutables y no existen lazos o bucles infinitos (sin fin).

4.4. ESTRUCTURA SECUENCIAL

La estructura secuencial es aquélla en la que una acción (instrucción) sigue a otra en secuencia. Las tareas se suceden de tal modo que la salida de una es la entrada de la siguiente y así sucesivamente hasta el final del proceso. La Figura 4.5 representa una estructura secuencial. La estructura secuencial tiene una entrada y una salida. Su representación gráfica se muestra en las Figuras 4.5, 4.6 y 4.7.

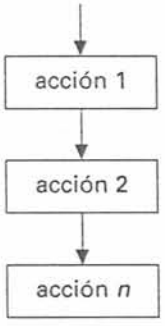


Figura 4.5. Estructura secuencial.

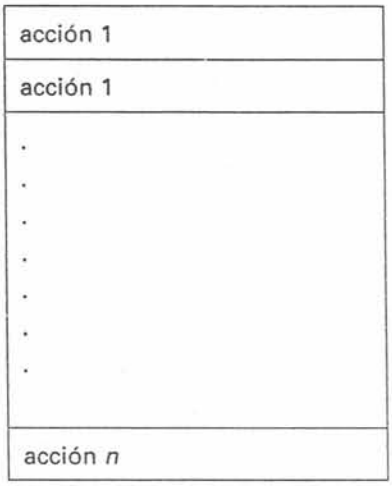


Figura 4.6. Diagrama N-S de una estructura secuencial.

```
inicio
  <accion1>
  <accion2>
  .
  .
  .
fin
```

Figura 4.7. Pseudocódigo de una estructura secuencial.

Ejemplo 4.2

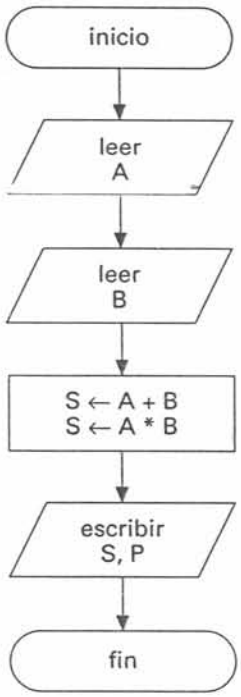
Cálculo de la suma y producto de dos números.

La suma S de dos números es S = A + B. El pseudocódigo y el diagrama de flujo correspondientes se muestran a continuación:

Pseudocódigo

```
inicio
  leer(A)
  leer(B)
  S ← A + B
  P ← A * B
  escribir(S, P)
fin
```

Diagrama de flujo



Ejemplo 4.3

Se trata de calcular el salario neto de un trabajador en función del número de horas trabajadas, precio de la hora de trabajo y, considerando unos descuentos fijos, el sueldo bruto en concepto de impuestos (20 por 100).

Pseudocódigo

```
inicio
  //calculo salario neto
  leer(nombre, horas, precio_hora)
  salario_bruto ← horas * precio_hora
  impuestos ← 0.20 * salario_bruto
  salario_neto ← salario_bruto - impuestos
  escribir(nombre, salario_bruto, salario_neto)
fin
```

Diagrama de flujo

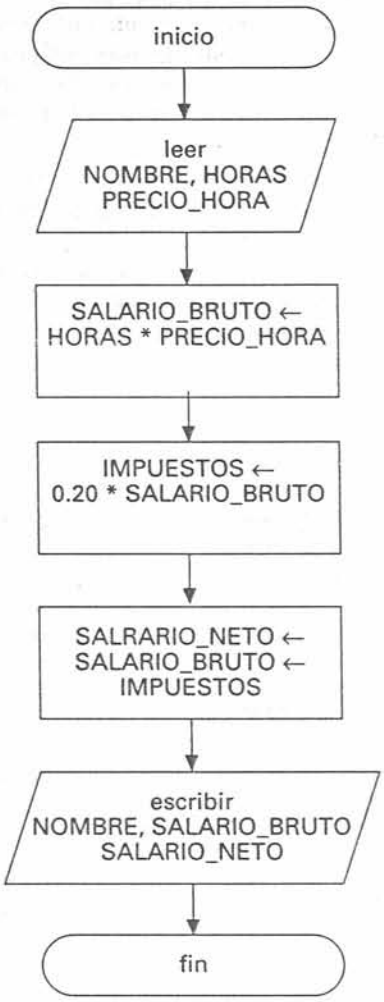


Diagrama N-S

leer nombre, horas, precio
salario_bruto ← horas * precio
impuestos ← 0.20 * salario_bruto
salario_neto ← salario_bruto - impuestos
escribir nombre, salario_bruto, salario_neto

4.5. ESTRUCTURAS SELECTIVAS

La especificación formal de algoritmos tiene realmente utilidad cuando el algoritmo requiere una descripción más complicada que una lista sencilla de instrucciones. Éste es el caso cuando existen un número de posibles alternativas resultantes de la evaluación de una determinada condición.

Las estructuras selectivas se utilizan para tomar decisiones lógicas; de ahí que se suelan denominar también *estructuras de decisión* o *alternativas*.

En las estructuras selectivas se evalúa una condición y en función del resultado de la misma se realiza una opción u otra. Las condiciones se especifican usando expresiones lógicas. La representación de una estructura selectiva se hace con palabras en pseudocódigo (`if`, `then`, `else` o bien, en español, `si`, `entonces`, `si_no`), con una figura geométrica en forma de rombo o bien con un triángulo en el interior de una caja rectangular.

Las estructuras selectivas o alternativas pueden ser:

- simples,
- dobles,
- múltiples.

4.5.1. Alternativa simple (`si-entonces / if-then`)

La estructura alternativa simple `si-entonces` (en inglés, `if-then` o bien `IF-THEN`) ejecuta una determinada acción cuando se cumple una determinada condición. La selección `si-entonces` evalúa la condición y

- si la condición es verdadera, entonces ejecuta la acción S1 (o acciones caso de ser S1 una acción compuesta y constar de varias acciones),
- si la condición es falsa, entonces no hacer nada.

Las representaciones gráficas de la estructura condicional simple se muestran en la Figura 4.8.

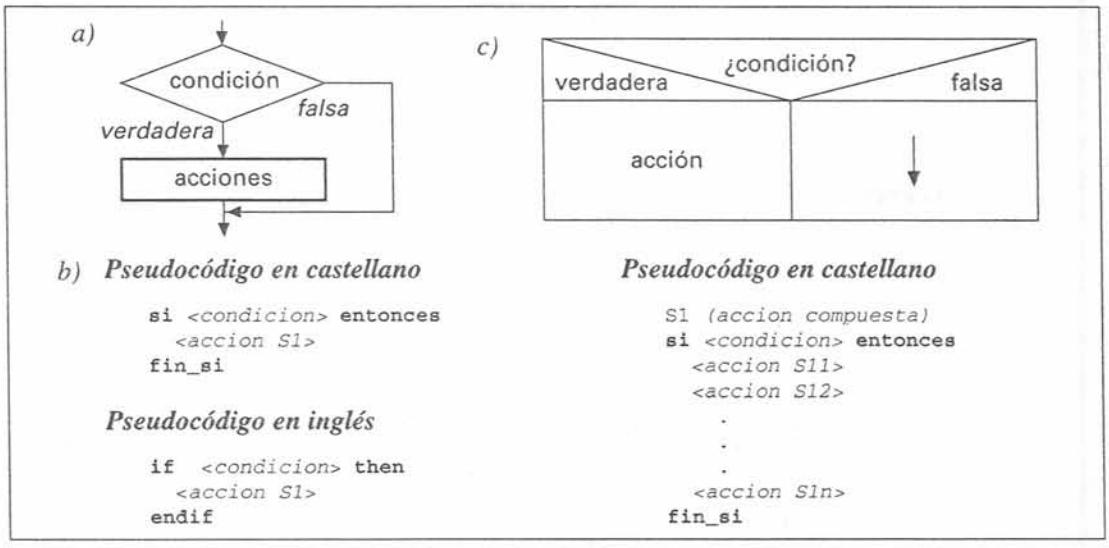


Figura 4.8. Estructuras alternativas simples: a) diagramas de flujo; b) pseudocódigo; c) diagrama N-S.

Obsérvese que las palabras del pseudocódigo **si** y **fin_si** se alinean verticalmente *indentando* (sangrando) la <*accion*> o bloque de acciones.

4.5.2. Alternativa doble (si-entonces-si_no / if-then-else)

La estructura anterior es muy limitada y normalmente se necesitará una estructura que permita elegir entre dos opciones o alternativas posibles, en función del cumplimiento o no de una determinada condición.

Si la condición `C` es verdadera, se ejecuta la acción `S1` y, si es falsa, se ejecuta la acción `S2`.

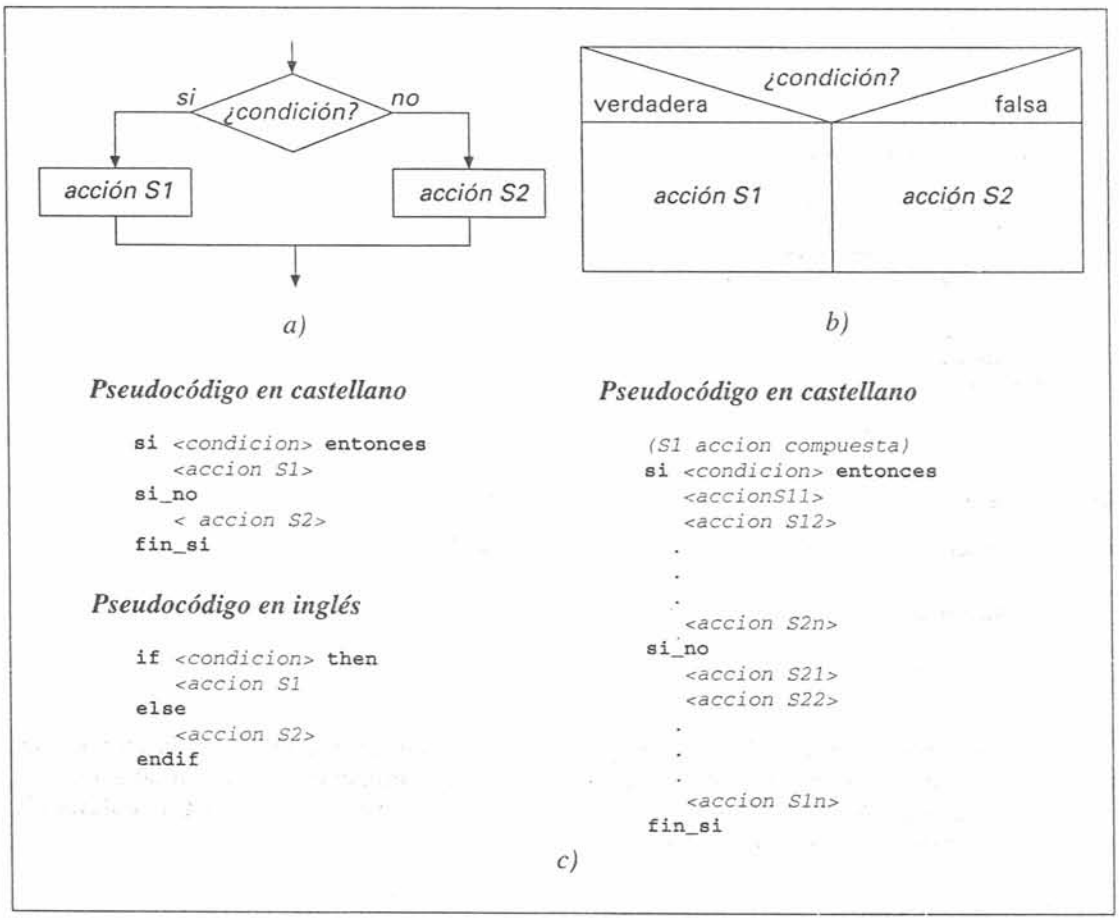


Figura 4.9. Estructura alternativa doble: a) diagrama de flujo; b) diagrama N-S; c) pseudocódigo.

Obsérvese que en el pseudocódigo las acciones que dependen de **entonces** y **si_no** están *indentadas* (*sangradas*) en relación con las palabras **si** y **fin_si**; este procedimiento aumenta la legibilidad de la estructura y es el medio más idóneo para representar algoritmos.

Ejemplo 4.4

Resolución de una ecuación de primer grado.

Si la ecuación es ax + b = 0, a y b son los datos, y las posibles soluciones son:

- $a <> 0$ $x = -b/a$
- $a = 0 \quad b <> 0$ **entonces** *«solución imposible»*
- $a = 0 \quad b = 0$ **entonces** *«solución indeterminada»*

El algoritmo correspondiente será:

```
algoritmo RESOL1
var
   real: a, b, x
inicio
  leer(a, b)
  si a <> 0 entonces
    x ← -b/a
    escribir(x)
  si_no
    si b <> 0 entonces
      escribir ('solucion imposible')
    si_no
      escribir ('solucion indeterminada')
    fin_si
  fin_si
fin
```

Ejemplo 4.5

Calcular la media aritmética de una serie de números positivos.

La media aritmética de *n* números es:

$$\frac{x1 + x2 + x3 + \ldots + xn}{n}$$

En el problema se supondrá la entrada de datos por el teclado hasta que se introduzca el primer número negativo, en nuestro caso –99. Para calcular la media aritmética se necesita saber cuántos números se han introducido hasta llegar a –99; para ello se utilizará un contador C que llevará la cuenta del número de datos introducidos.

```
VARIABLES:
s (suma): real
n (contador de numeros): entera
m (media): real

algoritmo Media
inicio
  s ← 0   //inicializacion de s y n
  n ← 0
```

```
datos:
  leer(x)  //calculo     el primer numero ha de ser mayor que cero
  si x < 0 entonces
    ir_a media
  si_no
    n ← n + 1
    s ← s + x
    ir_a datos
  fin_si
media:
  m ← s/n  //media de los numeros positivos
  escribir(m)
fin
```

En este ejemplo se observa una bifurcación hacia un punto referenciado por una etiqueta alfanumérica denominada (`media`) y otro punto referenciado por (`datos`).

Trate el alumno de simplificar este algoritmo de modo que sólo contenga un punto de bifurcación.

Ejemplo 4.6

Se desea obtener la nómina semanal —salario neto— de los empleados de una empresa cuyo trabajo se paga por horas y del modo siguiente:

- las horas inferiores o iguales a 35 horas (normales) se pagan a una tarifa determinada que se debe introducir por teclado al igual que el número de horas y el nombre del trabajador,
- las horas superiores a 35 se pagarán como extras al precio de 1.5 horas normales,
- los impuestos a deducir a los trabajadores varían en función de su sueldo mensual:
  - — sueldo <= 20.000, libre de impuestos,
  - — las siguientes 15.000 pesetas al 20 por 100,
  - — el resto, al 30 por 100.

Análisis

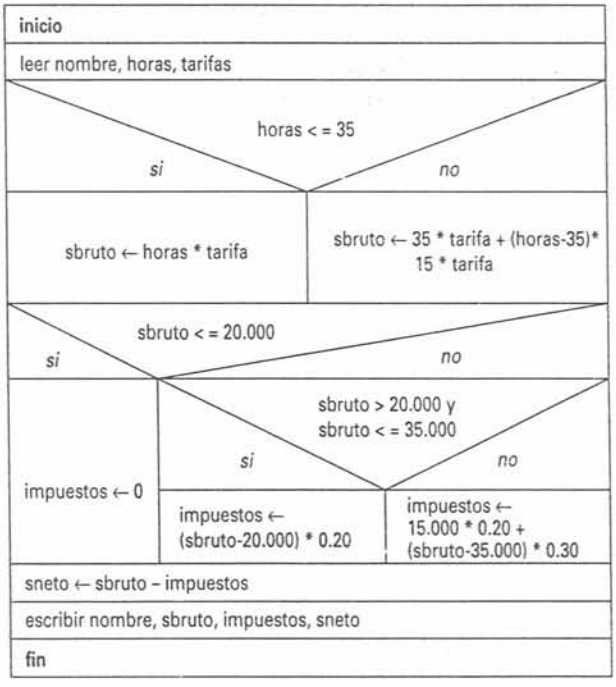
Las operaciones a realizar serán:

```
1. Inicio.
2. Leer nombre, horas trabajadas, tarifa.
3. Verificar si horas trabajadas <= 35, en cuyo caso:
     salario_bruto = horas * tarifa; en caso contrario,
     salario_bruto = 35 * tarifa + (horas-35) * tarifa * 1.5.
4. Calculo de impuestos
     si salario_bruto <= 20.000, entonces impuestos = 0
     si salario_bruto <= 35.000 entonces
     impuestos = (salario_bruto - 20.000) * 0.20
     si salario_bruto > 35.000 entonces
     impuestos = (salario_bruto - 35.000) * 0.30 + (15.000 * 0.20)
5. Calculo del salario_neto:
     salario_neto = salario_bruto - impuestos.
6. Fin.
```

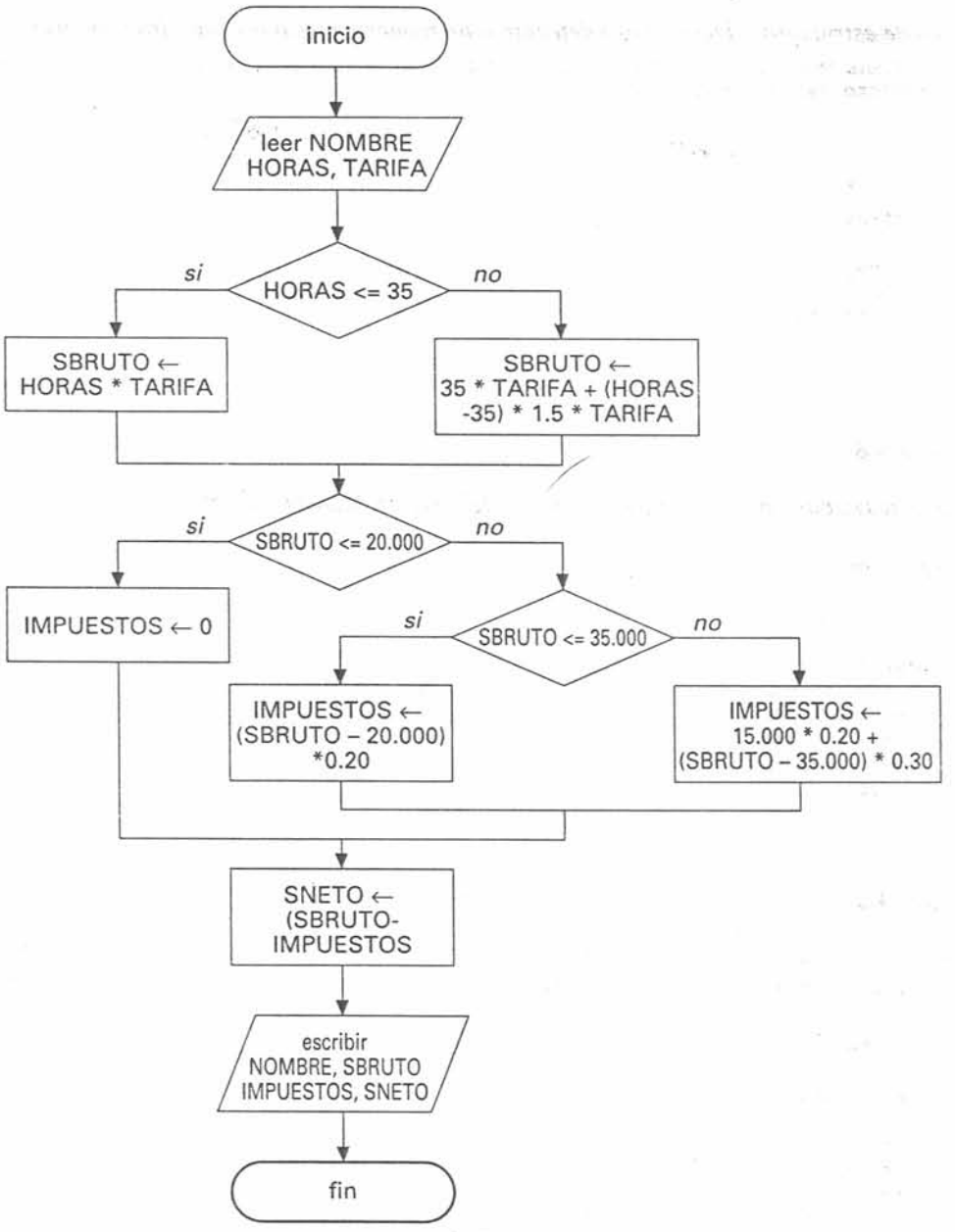
Representación del algoritmo en pseudocódigo

```
algoritmo Nomina
var
  cadena: nombre
  real: horas, impuestos, sbruto, sneto
inicio
  leer(nombre, horas, tarifa)

  si horas <= 35 entonces
    sbruto ← horas * tarifa
  si_no
    sbruto ← 35 * tarifa + (horas - 35) * 1.5 * tarifa
  fin_si

  si sbruto <= 20.000 entonces
    impuestos ← 0
  si_no
    si (sbruto > 20.000) y (sbruto <= 35.000) entonces
      impuestos ← (sbruto - 20.000) * 0.20
    si_no
      impuestos ← (15.000 * 0.20) + (sbruto - 35.000)* 0.30
    fin_si
  fin_si

  sneto ← sbruto - impuestos
  escribir(nombre, sbruto, impuestos, neto)
fin
```

Representación del algoritmo en diagrama N-S



Representación del algoritmo en diagrama de flujo



Ejemplo 4.7

Empleo de estructura selectiva para detectar si un número tiene o no parte fraccionaria.

```
algoritmo Parte_fraccionaria
var
  real : n
inicio
  escribir('Deme numero ')
  leer(n)
  si n = trunc(n) entonces
      escribir('El numero no tiene parte fraccionaria')
  si_no
      escribir('Numero con parte fraccionaria')
  fin_si
fin
```

Ejemplo 4.8

Estructura selectiva para averiguar si un año leído de teclado es o no bisiesto.

```
algoritmo Bisiesto
var
  entero : año
inicio
  leer(año)
  si (año MOD 4 = 0) y (año MOD 100 <> 0) o (año MOD 400 = 0) entonces
    escribir('El año ', año, ' es bisiesto')
  si_no
    escribir('El año ', año, ' no es bisiesto')
  fin_si
fin
```

Ejemplo 4.9

Algoritmo que nos calcule el área de un triángulo conociendo sus lados. La estructura selectiva se utiliza para el control de la entrada de datos en el programa.

```
algoritmo Area_triangulo
var
  real : a,b,c,p,area

inicio
  escribir('Deme los lados ')
  leer(a,b,c)
  p ← (a + b + c) / 2
  si (p > a) y (p > b) y (p > c) entonces
     area ← raiz2(p * (p - a) * (p - b) * (p - c))
     escribir( area)
  si_no
    escribir('No es un triangulo')
  fin_si
fin
```

4.5.3. Alternativa múltiple (`segun_sea, caso de / case`)

Con frecuencia —en la práctica—, es necesario que existan más de dos elecciones posibles (por ejemplo, en la resolución de la ecuación de segundo grado existen tres posibles alternativas o caminos a seguir, según que el discriminante sea negativo, nulo o positivo). Este problema, como se verá más adelante, se podría resolver por estructuras alternativas simples o dobles, *anidadas* o *en cascada*; sin embargo, este método si el número de alternativas es grande puede plantear serios problemas de escritura del algoritmo y naturalmente de legibilidad.

La estructura de decisión múltiple evaluará una expresión que podrá tomar *n* valores distintos, 1, 2, 3, 4, ..., *n*. Según que elija uno de estos valores en la condición, se realizará una de las *n* acciones, o lo que es igual, el flujo del algoritmo seguirá un determinado camino entre los *n* posibles.

La representación gráfica de la estructura de decisión múltiple se representa en la Figura 4.10.

```
Modelo 1:

segun_sea expresion (E) hacer
     e1: accion S11
         accion S12
         .
         .
         accion S1a
     e2: accion S21
         accion S22
         .
         .
         accion S2b
         .
         .
     en: accion S31
         accion S32
         .
         .
         accion S3p
     si-no: accion Sx
fin_segun


Modelo 2 (simplificado):

segun E hacer
       .
       .
       .
fin_segun
```

```
Modelo 3 (simplificado):

opcion E de
  .
  .
fin_opcion


Modelo 4 (simplificado):

caso_de E hacer
  .
  .
  .
fin_hacer


Modelo 5 (simplificado):

si E es n hacer
  .
  .
  .
fin_si
```

Figura 4.10. Estructuras de decisión múltiple.

Diagrama de flujo

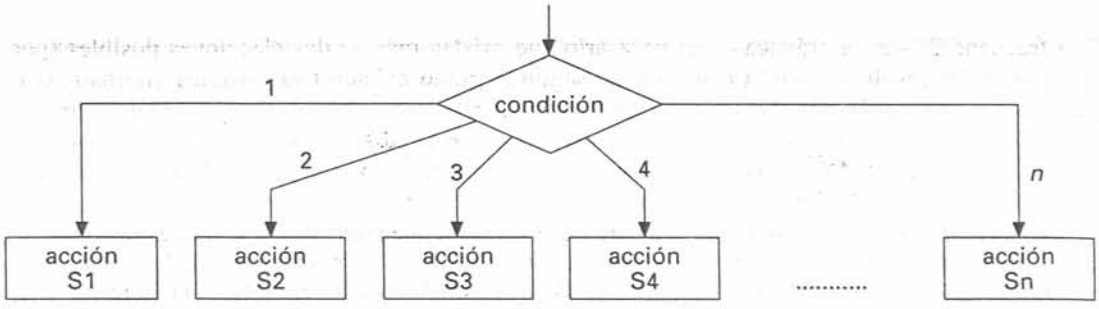
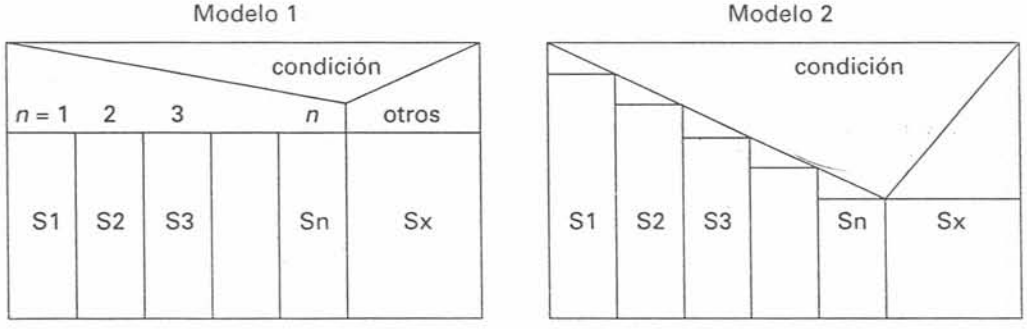


Diagrama N-S



Pseudocódigo

En inglés la estructura de decisión múltiple se representa:

```
case expresion of
  [e1]: accion S1
  [e2]: accion S2
  .
  .
o bien:
  [en]: accion Sn
  otherwise
       accion Sx
end_case
```

```
case expresion of
  [e1]: accion S1
  [e2]: accion S2


  [en]: accion Sn
  else
       accion Sx
end_case
```

La estructura de decisión múltiple en pseudocódigo se puede representar de diversas formas, pudiendo ser las acciones `S1`, `S2`, etc., *simples* como en el caso anterior o *compuestas* como en los siguientes modelos de formatos.

NOTAS:

1. Obsérvese que para cada valor de la expresión (e) se pueden ejecutar una o varias acciones. Algunos lenguajes, como Pascal, denominan a estas instrucciones *compuestas* y las delimitan con las palabras reservadas **`begin-end`** (**`inicio-fin`**); es decir, en pseudocódigo.

```
segun_sea E hacer
  e1: accion S1
  e2: accion S2
    .
    .
  en: accion Sn
  si_no: accion Sx
fin_segun
```

o bien en el caso de instrucciones compuestas:

```
segun_sea E hacer
  e1: inicio
        accion S11
        accion S12
        .
        .
        accion S1a
      fin
  e1: inicio
        accion S21
        .
        .
        .
      fin
  en: inicio
        .
        .
        .
      fin
  si_no: accion Sx
fin_segun
```

2. Los valores que toman las expresiones (E) no tienen por qué ser consecutivos ni únicos; se pueden considerar rangos de constantes numéricas o de caracteres como valores de la expresión E.

```
caso_de E hacer
  2, 4, 6, 8, 10: escribir 'numeros pares'
  1, 3, 5, 7, 9:  escribir 'numeros impares'
fin_caso
```

¿Cuál de los anteriores modelos se puede considerar representativo? En realidad, como el pseudocódigo es un lenguaje algorítmico universal, cualquiera de los modelos se podría ajustar a su presentación; sin embargo, nosotros consideramos como más estándar los modelos 1, 2 y 4. En esta otra seguiremos normalmente el modelo 1, aunque en ocasiones, y para familiarizar al lector en su uso, podremos utilizar los modelos citados 2 y 4.

Ejemplo 4.10

Se desea diseñar un algoritmo que escriba los nombres de los días de la semana en función del valor de una variable DIA introducida por teclado, que representa su posición dentro de la semana.

Los días de la semana son 7; por consiguiente, el rango de valores de DIA será 1 . . . 7, y caso de que DIA tome un valor fuera de este rango se deberá producir un mensaje de error advirtiendo la situación anómala.

```
algoritmo Nombre_dias
var
  entero: DIA
  inicio
    leer (DIA)
  segun_sea DIA hacer
    1: escribir('LUNES')
    2: escribir('MARTES')
    3: escribir('MIERCOLES')
    4: escribir('JUEVES')
    5: escribir('VIERNES')
    6: escribir('SABADO')
    7: escribir('DOMINGO')
    si_no
      escribir ('ERROR')
  fin_segun
fin
```

Ejemplo 4.11

Leída una fecha, decir el día de la semana, suponiendo que el día 1 de dicho mes fue lunes.

```
algoritmo Dia_semana
var
  entero : dia

inicio
  escribir('Diga el día ')
  leer( dia)
  segun_sea dia MOD 7 hacer
     1:
       escribir('Lunes')
     2:
       escribir('Martes')
     3:
       escribir('Miercoles')
     4:
       escribir('Jueves')
     5:
       escribir('Viernes')
     6:
       escribir('Sabado')
     0:
       escribir('Domingo')
  fin_segun
fin
```

Ejemplo 4.12

Preguntar qué día de la semana fue el día 1 del mes actual y calcular que día de la semana es hoy.

```
algoritmo Dia_semana_modificado
var
    entero   : dia,d1
    caracter : dia1
```

```
inicio
  escribir('El dia 1 fue (L,M,X,J,V,S,D) ')
  leer( dia1)
  segun_sea dia1 hacer
      'L':
        d1← 0
      'M':
        d1← 1
      'X':
        d1← 2
      'J':
        d1← 3
      'V':
        d1← 4
      'S':
        d1← 5
      'D':
        d1← 6
      si_no
        d1← -40
  fin_segun

  escribir('Diga el dia ')
  leer( dia)
  dia ← dia + d1

  segun_sea dia MOD 7 hacer
    1:
      escribir('Lunes')
    2:
      escribir('Martes')
    3:
      escribir('Miercoles')
    4:
      escribir('Jueves')
    5:
      escribir('Viernes')
    6:
      escribir('Sabado')
    0:
      escribir('Domingo')
  fin_segun
fin
```

Ejemplo 4.13

Se desea convertir las calificaciones alfabéticas A, B, C, D *y* E *a calificaciones numéricas* 4, 5, 6, 7 *y* 8, *respectivamente.*

Los valores de A, B, C, M, D y E se representarán por la variable LETRA, el algoritmo de resolución del problema es:

```
algoritmo Calificaciones
var
  caracter: LETRA
  entero: CALIFICACIONES
```

```
inicio
  leer(LETRA)
  segun_sea LETRA hacer
    'A': calificacion ← 4
    'B': calificacion ← 5
    'C': calificacion ← 6
    'D': calificacion ← 7
    'E': calificacion ← 8
    si-no
      escribir('ERROR')
  fin_segun
fin
```

Como se ve, en el pseudocódigo no se contemplan otras posibles calificaciones —por ejemplo, 0, resto notas numéricas—; si así fuese, habría que modificarlo en el siguiente sentido:

```
segun_sea LETRA hacer
  'A': calificacion ← 4
  'B': calificacion ← 5
  'C': calificacion ← 6
  'D': calificacion ← 7
  'E': calificacion ← 8

  si_no: calificacion ← 0
fin_segun
```

Ejercicio 4.14

Algoritmo que nos indique si un número entero, leído de teclado, tiene 1, 2, 3 o más de 3 dígitos. Considerar los negativos.

Se puede observar que, la estructura **segun_sea** *<expresion>* **hacer**, son varios `si` *<expr. logica>* **entonces** . . . . . anidados en la rama **si_no**. Si se cumple el primero ya no pasa por los demás.

```
algoritmo Digitos
var
  entero : n

inicio
  leer(n)
  segun_sea n hacer
    -9 .. 9:
      escribir('Tiene 1 digito')
    -99 .. 99:
      escribir('Tiene 2')
    -999 .. 999:
      escribir('Tiene tres')
    si_no
      escribir('Tiene mas de tres')
  fin_segun
fin
```

Ejemplo 4.15

Se desea leer por teclado un número comprendido entre 1 y 10 (inclusive) y se desea visualizar si el número es par o impar.

En primer lugar, se deberá detectar si el número está comprendido en el rango válido (1 a 10) y a continuación si el número es `1, 3, 5, 7, 9`, escribir un mensaje de `'impar'`; si es `2, 4, 6, 8, 10`, escribir un mensaje de `'par'`.

```
algoritmo PAR_IMPAR
var entero: numero
inicio
  leer(numero)
  si (numero >= 1) y (numero <= 10) entonces
    segun_sea numero hacer
      1, 3, 5, 7, 9 : escribir('impar')
      2, 4, 6, 8, 10: escribir('par')
    fin_segun
  fin_si
fin
```

4.6. ESTRUCTURAS REPETITIVAS

Las computadoras están especialmente diseñadas para todas aquellas aplicaciones en las cuales una operación o conjunto de ellas deben repetirse muchas veces. Un tipo muy importante de estructura es el algoritmo necesario para repetir una o varias acciones un número determinado de veces. Un programa que lee una lista de números puede repetir la misma secuencia de mensajes al usuario e instrucciones de lectura hasta que se lean todos los números de un fichero.

Las estructuras que repiten una secuencia de instrucciones un número determinado de veces se denominan *bucles*, y se llama *iteración* al hecho de repetir la ejecución de una secuencia de acciones. Un ejemplo aclarará la cuestión.

Supongamos que se desea sumar una lista de números escritos desde teclado —por ejemplo, calificaciones de los alumnos de una clase—. El medio conocido hasta ahora es leer los números y añadir sus valores a una variable SUMA que contenga las sucesivas sumas parciales. La variable SUMA se hace igual a cero y a continuación se incrementa en el valor del número cada vez que uno de ellos se lea. El algoritmo que resuelve este problema es:

```
algoritmo Suma
var
  entero: suma, numero
inicio
  SUMA ← 0
  leer(numero)
  SUMA ← SUMA + numero
  leer(numero)
  SUMA ← SUMA + numero
  leer(numero)
fin
```

y así sucesivamente para cada número de la lista. En otras palabras, el algoritmo repite muchas veces las acciones.

```
leer(numero)
SUMA ← SUMA + numero
```

Tales opciones repetidas se denominan *bucles* o *lazos*. La acción (o acciones) que se repite en un bucle se denomina *iteración*. Las dos principales preguntas a realizarse en el diseño de un bucle son: ¿qué contiene el bucle? y ¿cuántas veces se debe repetir?

Cuando se utiliza un bucle para sumar una lista de números, se necesita saber cuántos números se han de sumar. Para ello necesitaremos conocer algún medio para *detener* el bucle. En el ejemplo siguiente usaremos la técnica de solicitar al usuario el número que desea, por ejemplo, N. Existen dos procedimientos para contar el número de iteraciones, usar una variable TOTAL que se inicializa a la cantidad de números que se desea y a continuación se decrementa en uno cada vez que el bucle se repite (este procedimiento añade una acción más al cuerpo del bucle: TOTAL ← TOTAL - 1), o bien inicializar la variable TOTAL en 0 o en 1, e ir incrementando en uno a cada iteración hasta llegar al número deseado.

```
algoritmo Suma_numero
var
  entero: N, TOTAL
  real: NUMERO, SUMA
inicio
  leer(N)
  TOTAL ← N
  SUMA ← 0
  mientras TOTAL > 0 hacer
    leer(NUMERO)
    SUMA ← SUMA + NUMERO
    TOTAL ← TOTAL - 1
  fin_mientras
  escribir('La suma de los', N, 'numeros es', SUMA)
fin
```

El bucle podrá también haberse terminado poniendo cualquiera de estas condiciones:

- *hasta_que* TOTAL sea cero
- *desde* 1 *hasta* N

Para detener la ejecución de los bucles se utiliza una condición de parada. El pseudocódigo de una estructura repetitiva tendrá siempre este formato:

```
inicio
//inicializacion de variables
repetir
  acciones S1, S2, ...
  salir segun condicion
  acciones Sn, Sn + 1, ...
fin_repetir
```

Aunque la condición de salida se indica en el formato anterior en el interior del bucle —y existen lenguajes que así la contienen expresamente [1]—, lo normal es que *la condición se indique al final o al principio del bucle*, y así se consideran tres tipos de instrucciones o estructuras repetitivas o iterativas generales y una particular que denominaremos **iterar**, que contiene la salida en el interior del bucle.

iterar	*(loop)*
mientras	*(while)*
repetir	*(repeat)*
desde | para	*(for)*

[1] Modula-2 y QuickBASIC/QBasic, entre otros.

El algoritmo de suma anterior podría expresarse en pseudocódigo estándar:

```
algoritmo SUMA_numeros
var
  entero: N, TOTAL
  real: NUMERO, SUMA
inicio
  leer(N)
  TOTAL ← N
  SUMA ← 0
  repetir
    leer(NUMERO)
    SUMA ← SUMA + NUMERO
    TOTAL ← TOTAL - 1
  hasta_que TOTAL = 0
  escribir('La suma es', SUMA)
fin
```

Los tres casos generales de estructuras repetitivas dependen de la situación y modo de la condición. La condición se evalúa tan pronto se encuentra en el algoritmo y su resultado producirá los tres tipos de estructuras citadas.

1. La condición de salida del bucle se realiza al principio del bucle (estructura **mientras**).

   ```
   algoritmo SUMA1
   inicio
     //Inicializar K, S a cero
       K ← 0
       S ← 0
     leer(n)
     mientras K < n hacer
       K ← K + 1
       S ← S + K
     fin_mientras
     escribir (S)
   fin
   ```

   Se ejecuta el bucle mientras se verifica una condición `(K < n)`.

2. La condición de salida se origina al final del bucle; el bucle se ejecuta *hasta que* se verifica una cierta condición.

   ```
   repetir
     K ← K + 1
     S ← S + K
   hasta_que K > n
   ```

3. La condición de salida se realiza con un contador que cuenta el número de iteraciones.

   ```
   desde i = vi hasta vf hacer
     S ← S + i
   fin_desde
   ```

   `i` es un contador que cuenta desde el valor inicial `(vi)` hasta el valor final `(vf)` con los incrementos que se consideren.

4.6.1. Estructura `mientras` (`"while"`)

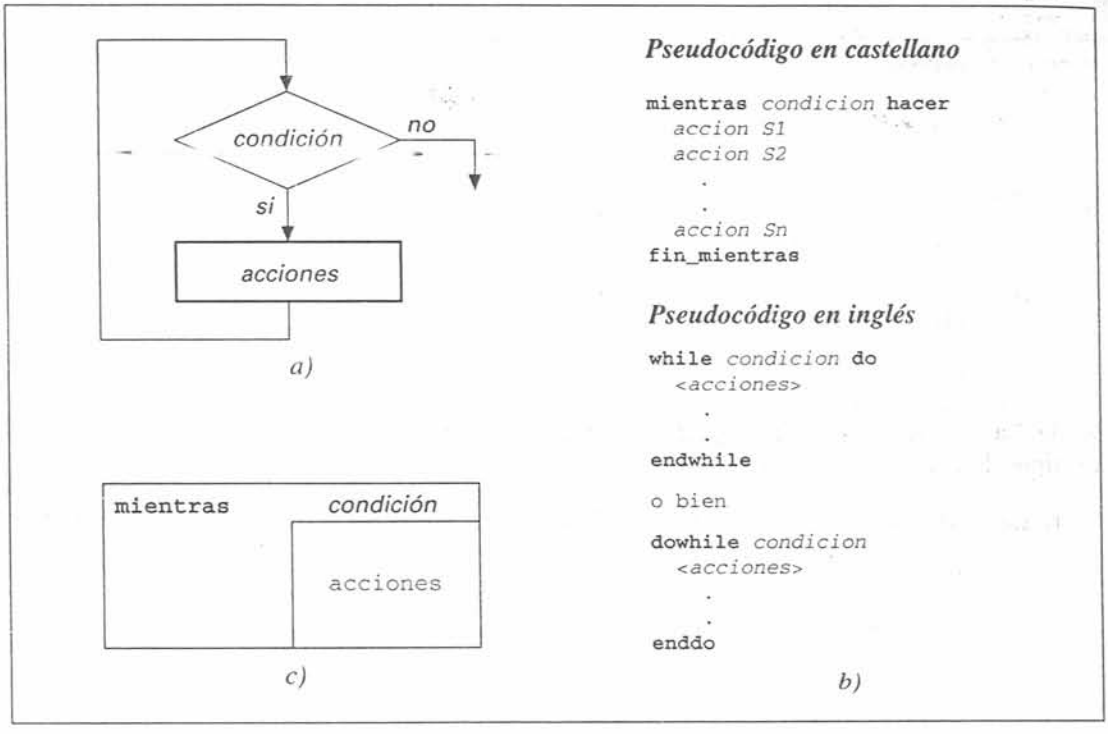


Figura 4.11. Estructura `mientras`: a) diagrama de flujo; b) pseudocódigo; c) diagrama N-S.

La estructura repetitiva **`mientras`** (en inglés **`while`** o **`dowhile`**: *hacer mientras*) es aquélla en que el cuerpo del bucle se repite mientras se cumple una determinada condición. Las representaciones gráficas son las anteriores.

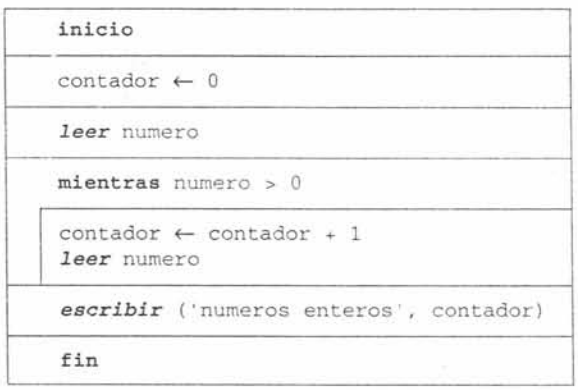
Cuando se ejecuta la instrucción **mientras**, la primera cosa que sucede es que se evalúa la condición (una expresión booleana). Si se evalúa *falsa*, ninguna acción se toma y el programa prosigue en la siguiente instrucción del bucle. Si la expresión booleana es *verdadera*, entonces se ejecuta el cuerpo del bucle, después de lo cual se evalúa de nuevo la expresión booleana. Este proceso se repite una y otra vez *mientras* la expresión booleana (condición) sea verdadera.

Ejemplo 4.16

Contar los números enteros positivos introducidos por teclado. Se consideran dos variables enteras NUMERO *y* CONTADOR *(contará el número de enteros positivos). Se supone que se leen números positivos y se detiene el bucle cuando se lee un número negativo o cero.*

```
algoritmo Cuenta_enteros
var
  entero: numero, contador
```

```
inicio
  contador ← 0
  leer(numero)
  mientras numero > 0 hacer
    contador ← contador + 1
    leer (numero)
  fin_mientras
  escribir('El numero de enteros positivos es', contador)
fin
```



La secuencia de las acciones de este algoritmo se puede reflejar en el siguiente pseudocódigo:

Paso	Pseudocódigo	Significado
1	contador ← 0	*inicializar contador a 0*
2	**leer**(numero)	*leer primer número*
3	**mientras** numero > 0 **hacer**	*comprobar si número > 0* *si es así, continuar con el paso 4, si no, continuar con el paso 7*
4	contador ← contador + 1	*incrementar contador*
5	**leer**(numero)	*leer siguiente número*
6	**fin_mientras**	*seguir con el paso 3*
7	**escribir**(contador)	*visualizar resultados*

Obsérvese que los pasos 3 a 6 se ejecutarán mientras los números de entrada sean positivos. Cuando se lea –15 (después de 4 pasos), la expresión *numero > 0* produce un resultado falso y se transfiere el control a la acción **escribir** y el valor del contador será 4.

Ejecución de un bucle cero veces

Obsérvese que en una estructura **mientras** la primera cosa que sucede es la evaluación de la expresión booleana; si se evalúa *falsa* en ese punto, entonces el cuerpo del bucle nunca se ejecuta.

Puede parecer *inútil* ejecutar el cuerpo del bucle *cero veces*, ya que no tendrá efecto en ningún valor o salida. Sin embargo, a veces es la acción deseada.

```
inicio
  n ← 5
  s ← 0
  mientras n <= 4 hacer
    leer(x)
    s ← s + x
  fin_mientras
fin
```

En el ejemplo anterior se puede apreciar que nunca se cumplirá la condición (expresión booleana $n <= 4$), por lo cual se ejecutará la acción **fin** y no se ejecutará ninguna acción del bucle.

Ejemplo 4.17

El siguiente bucle no se ejecutará si el primer número leído es negativo o cero.

```
C ← 0
leer(numero)
mientras numero > 0 hacer
  C ← C + 1
  leer(numero)
fin_mientras
```

Bucles infinitos

Algunos bucles no exigen fin y otros no encuentran el fin por error en su diseño. Por ejemplo, un sistema de reservas de líneas aéreas puede repetir un bucle que permita al usuario añadir o borrar reservas. El programa y el bucle corren siempre, o al menos hasta que la computadora se apaga. En otras ocasiones un bucle no se termina nunca porque nunca se cumple la condición.

Un bucle que nunca se termina se denomina *bucle infinito* o *sin fin*. Los bucles sin fin no intencionados son perjudiciales para la programación y se deben evitar siempre.

Consideremos el siguiente bucle que visualiza el interés producido por un capital a las tasas de interés comprendidas en el rango desde 10 a 20 por 100.

```
leer(capital)
tasa ← 10
mientras tasa <> 20 hacer
  interes ← tasa * 0.01 * capital     //tasa * capital/100 = tasa * 0.01 * capital
  escribir('interes producido', interes)
  tasa ← tasa + 2
fin_mientras
escribir('continuacion')
```

Los sucesivos valores de la tasa serán 10, 12, 14, 16, 18, 20, de modo que al tomar *tasa* el valor 20 se detendrá el bucle y se escribirá el mensaje 'continuación'.

Supongamos que se cambia la línea última del bucle por

```
tasa ← tasa + 3
```

El problema es que el valor de la tasa salta ahora de 19 a 22 y nunca será igual a 20 (10, 13, 16, 19, 22, ...). El bucle sería infinito, la expresión booleana que permite terminar al bucle será:

```
tasa < 20          o bien:          tasa >= 20
```

Regla práctica: Las pruebas o tests en las expresiones booleanas es conveniente que sean *mayor* o *menor* que en lugar de pruebas de *igualdad* o *desigualdad*. En el caso de la codificación en un lenguaje de programación, esta regla debe seguirse rígidamente en el caso de comparación de *números reales*, ya que como esos valores se almacenan en cantidades aproximadas las comparaciones de igualdad de valores reales normalmente plantean problemas. Siempre que realice comparaciones de números reales, use las relaciones <, <=, > o >=.

Terminación de bucles con datos de entrada

Si su algoritmo o programa está leyendo una lista de valores con un bucle **mientras**, se debe incluir algún tipo de mecanismo para terminar el bucle. Existen cuatro métodos típicos para terminar un bucle de entrada:

1. preguntar antes de la iteración,
2. encabezar la lista de datos con su tamaño,
3. finalizar la lista con su valor de entrada,
4. agotar los datos de entrada.

Examinémoslos por orden.

El primer método simplemente solicita con un mensaje al usuario si existen más entradas.

```
Suma ← 0
escribir('Existen mas numeros en la lista s/n')
leer(Resp)  //variable Resp, tipo caracter
mientras(Resp = 'S') o (Resp = 's') hacer
  escribir('numero')
  leer(N)
  Suma ← Suma + N
  escribir('Existen mas numeros (s/n)')
  leer(Resp)
fin_mientras
```

Este método a veces es aceptable y es muy útil en ciertas ocasiones, pero suele ser tedioso para listas grandes; en este caso, es preferible incluir una señal de parada.

El método de conocer en la cabecera del bucle el tamaño o el número de iteraciones ya ha sido visto en ejemplos anteriores.

Tal vez el método más correcto para terminar un bucle que lee una lista de valores es con un *centinela*. Un valor *centinela* es un valor especial usado para indicar el final de una lista de datos. Por ejemplo, supongamos que se tienen unas calificaciones de unos tests (cada calificación comprendida entre 0 y 100); un valor centinela en esta lista puede ser –999, ya que nunca será una calificación válida y cuando aparezca este valor se terminará el bucle. Si la lista de datos son números positivos, un valor centinela puede ser un número negativo que indique el final de la lista. El siguiente ejemplo realiza la suma de todos los números positivos introducidos desde el terminal.

```
suma ← 0
leer(numero)
mientras numero >= 0 hacer
  suma ← suma + numero
  leer(numero)
fin_mientras
```

Obsérvese que el último número leído de la lista no se añade a la suma si es negativo, ya que se sale fuera del bucle. Si se desea sumar los números 1, 2, 3, 4 y 5 con el bucle anterior, el usuario debe introducir, por ejemplo:

```
1    2    3    4    5    -1
```

el valor final –1 se lee, pero no se añade a la suma. Nótese también que cuando se usa un valor centinela se invierte el orden de las instrucciones de lectura y suma con un valor centinela, éste debe leerse al final del bucle, por lo que se debe poner la instrucción leer al final del mismo.

El último método de agotamiento de datos de entrada es comprobar simplemente que no existen más datos de entrada. Este sistema suele depender del tipo de lenguaje; por ejemplo, Pascal puede detectar el final de una línea; en los archivos secuenciales se puede detectar el fin físico de archivo (*eof*, `end of file`).

Ejemplo 4.18

Considerar los siguientes algoritmos. ¿Qué visualizará y cuántas veces se ejecuta el bucle?

1.
```
i ← 0
mientras i < 6 hacer
  escribir(i)
  i ← i + 1
fin_mientras
```

La salida es el valor de la variable de control *i* al principio de cada ejecución del cuerpo del bucle: 0, 1, 2, 3, 4 y 5. El bucle se ejecuta seis veces.

2.
```
i ← 0
mientras i < 6 hacer
  i ← i + 1
  escribir(i)
fin_mientras
```

La salida será entonces 1, 2, 3, 4, 5 y 6. El cuerpo del bucle se ejecuta también seis veces. Obsérvese que cuando `i = 5`, la expresión booleana es verdadera y el cuerpo del bucle se ejecuta; con `i = 6` la sentencia **escribir** se ejecuta, pero a continuación se evalúa la expresión booleana y se termina el bucle.

Ejemplo 4.19

Calcular la media de un conjunto de notas de alumnos. Pondremos un valor centinela de –99 que detecte el fin del bucle.

```
inicio
  total ← 0
  n ← 0        //numero de alumnos
  leer(nota)  //la primera nota debe ser distinta de -99
  mientras nota <> -99 hacer
    total ← total + nota
    n ← n + 1
    leer (nota)
  fin_mientras
  media ← total/n
  escribir('La media es', media)
fin
```

Obsérvese que `total`, `n` y `media` se inicializan a cero antes de la instrucción **`mientras`**. Cuando el bucle termina, la variable `total` contiene la suma de todas las notas y, por consiguiente `total/n`, siendo `n` el número de alumnos, será la media de la clase.

4.6.2. Estructura `repetir` (`"repeat"`)

Existen muchas situaciones en las que se desea que un bucle se ejecute al menos una vez *antes* de comprobar la condición de repetición. En la estructura **`mientras`** si el valor de la expresión booleana es inicialmente falso, el cuerpo del bucle no se ejecutará; por ello, se necesitan otros tipos de estructuras repetitivas.

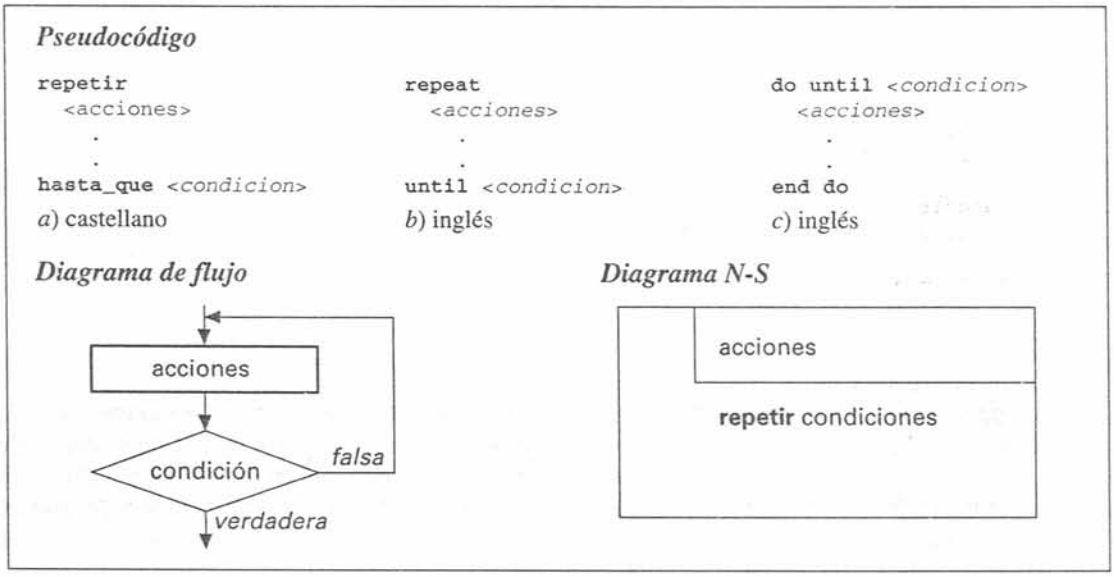


Figura 4.12. Estructura `repetir`: pseudocódigo, diagrama de flujo, diagrama N-S.

La estructura **`repetir (repeat)`** se ejecuta hasta que se cumpla una condición determinada que se comprueba al final del bucle.

El bucle **`repetir-hasta_que`** se repite mientras el valor de la expresión booleana de la condición sea *falsa*, justo la opuesta de la sentencia **`mientras`**.

```
algoritmo Repetir
var
  real: numero
  entero: contador
inicio
  contador ← 1
  repetir
    leer(numero)
    contador ← contador + 1
  hasta_que contador > 30
  escribir('Numeros leidos 30')
fin
```

En el ejemplo anterior, el bucle se repite hasta que el valor de la variable *contador* exceda a 30, lo que sucederá después de 30 ejecuciones del cuerpo del bucle.

Ejemplo 4.20

Desarrollar el algoritmo necesario para calcular el factorial de un número N que responda a la fórmula:

$$N! = N \cdot (N-1) \cdot (N-2), \ldots, 3 \cdot 2 \cdot 1$$

El `algoritmo` correspondiente es:

```
algoritmo Factorial
var
  entero: I, N
  real: Factorial
inicio
//N >= 0
  leer(N)
  Factorial ← 1
  I ← 1
    repetir
      Factorial ← Factorial * I
      I ← I + 1
    hasta_que I = N + 1
  escribir('El factorial del numero', N, 'es', Factorial)
fin
```

Con una estructura **repetir** el cuerpo del bucle *se ejecuta siempre al menos una vez*. Cuando una instrucción **repetir** se ejecuta, lo primero que sucede es la ejecución del bucle y, a continuación, se evalúa la expresión booleana resultante de la condición. Si se evalúa como falsa, el cuerpo del bucle se repite y la expresión booleana se evalúa una vez. Después de cada iteración del cuerpo del bucle, la expresión booleana se evalúa; si es *verdadera*, el bucle termina y el programa sigue en la siguiente instrucción a **hasta_que**.

Diferencias de las estructuras `mientras` y `repetir`

- La estructura **mientras** termina cuando la condición es falsa, mientras que **repetir** termina cuando la condición es verdadera.
- En la estructura **repetir** el cuerpo del bucle se ejecuta siempre al menos una vez; por el contrario, **mientras** es más general y permite la posibilidad de que el bucle pueda no ser ejecutado. Para usar la estructura **repetir** debe estar seguro de que el cuerpo del bucle —bajo cualquier circunstancia— se repetirá al menos una vez.

Ejemplo 4.21

Encontrar el entero positivo mayor (num) para el cual la suma 1 + 2 + 3 + ... + num es menor o igual que límite.

```
1. Introducir limite.
2. Inicializar num y suma a 0.
3. Repetir las acciones siguientes hasta que suma > limite:
   • incrementar num en 1,
   • añadir num a suma.
```

```
4. Restar num a suma y decrementar num en 1.
5. Visualizar num y suma.
```

El *pseudocódigo* de este algoritmo es:

```
algoritmo Mas_pequeño
var
  entero: num, limite, suma
inicio
  leer(limite)
  num ← 0
  suma ← 0
  repetir
    num ← num + 1
    suma ← suma + num
  hasta_que suma > limite
  suma ← suma - num
  num  ← num - 1
  escribir(num, suma)
fin
```

Ejemplo 4.22

Escribir los números 1 a 100.

```
algoritmo Uno_cien
var
  num: entero
inicio
  num ← 1
  repetir
    escribir(num)
    num ← num + 1
  hasta_que num = 101
fin
```

Ejemplo 4.23

Es muy frecuente tener que realizar validación de entrada de datos en la mayoría de las aplicaciones. Este ejemplo detecta cualquier entrada comprendida entre 1 y 12, rechazando las restantes, ya que se trata de leer los números correspondientes a los meses del año.

```
algoritmo Validar_mes
var
  entero: mes
inicio
  escribir('Introducir numero de mes')
  repetir
    leer(mes)
    si (mes < 1) o (mes > 12) entonces
      escribir('Valor entre 1 y 12')
    fin_si
  hasta_que (mes >= 1) y (mes <= 12)
fin
```

Este sistema es conocido como *interactivo* por entablar un «*diálogo imaginario*».

4.6.3. Estructura `desde/para ("for")`

En muchas ocasiones se conoce de antemano el número de veces que se desean ejecutar las acciones de un bucle. En estos casos en los que el número de iteraciones es fijo, se debe usar la estructura **`desde`** o **`para`** (**`for`**, en inglés).

La estructura **`desde`** ejecuta las acciones del cuerpo del bucle un número especificado de veces y de modo automático controla el número de iteraciones o pasos a través del cuerpo del bucle.

Las herramientas de programación de la estructura **`desde`** o **`para`** se muestran a continuación:

```
desde v ← vi hasta vf [incremento|
          decremento < valor> ] hacer
    <acciones>
      .
      .
      .
fin_desde
v: variable indice
vi, vf: valores inicial y final de la variable
```

a) modelo 1

```
para v ← vi hasta vf [incremento|
         decremento<valor>] hacer
    <acciones>
      .
      .
      .
fin_para
```

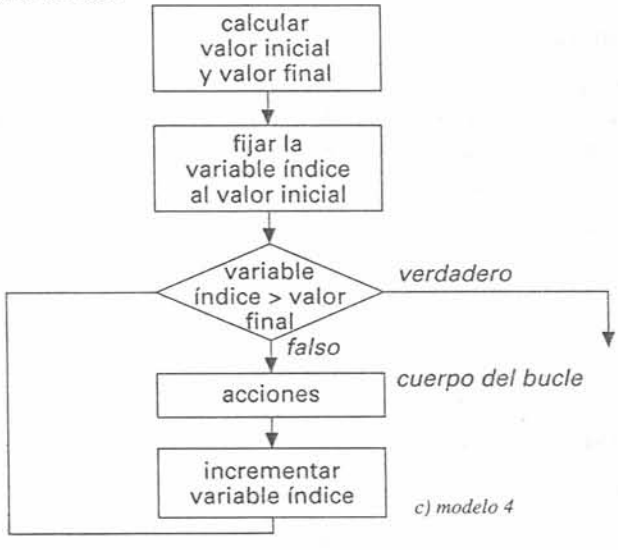
a) modelo 2

Diagrama N-S, estructura **`desde`**

```
desde v ← vi hasta vf [incremento|decremento <valor> hacer
        <acciones>
fin_desde
```

b) modelo 3

Diagrama de flujo estructura **`desde`**

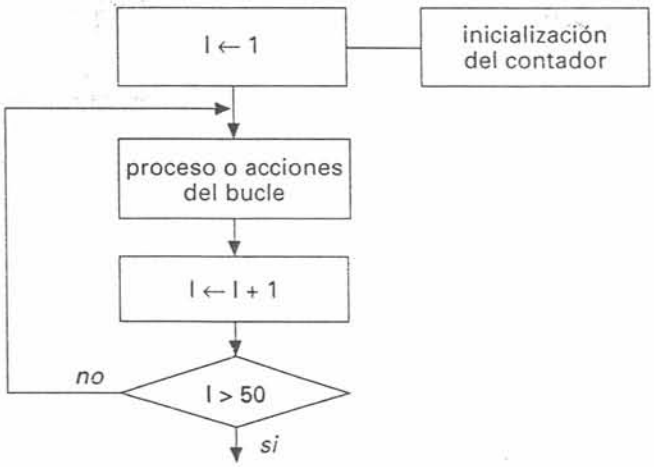


c) modelo 4

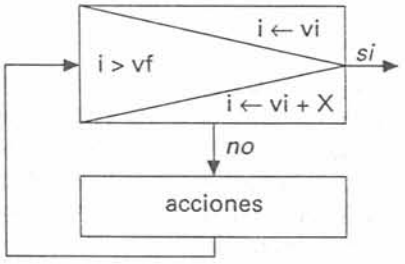
Figura 4.13. Estructura `desde (for)`: a) pseudocódigo; b) diagrama N-S; c) diagrama de flujo.

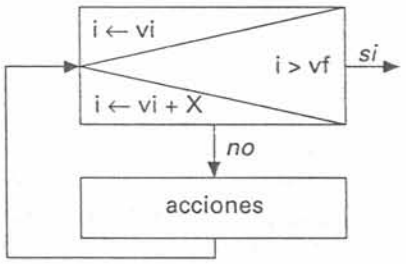
Otras representaciones de estructuras repetitivas `desde/para (for)`

Un bucle **`desde (for)`** se representa con los símbolos de proceso y de decisión mediante un contador. Así, por ejemplo, en el caso de un bucle de lectura de cincuenta números para tratar de calcular su suma:

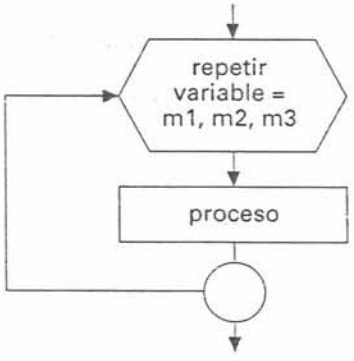


Es posible representar el bucle con símbolos propios:



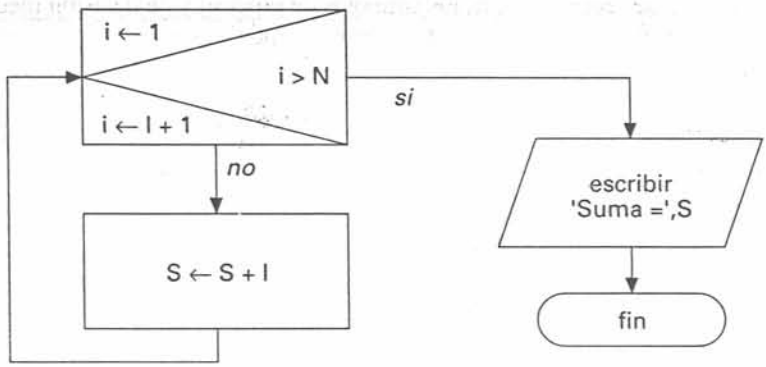


o bien mediante este otro símbolo:



`m1` = *contador inicial*
`m2` = *contador final*
`m3` = *incremento o paso*

Como aplicación, *calcular la suma de los* N *primeros enteros.*



equivale a

```
algoritmo Suma
var
  entero: T, N, S
inicio
  S ← 0
  desde I ← 1 hasta N hacer
    S ← S + I
  fin_desde
  escribir('Suma =', S)
fin
```

La estructura **desde** comienza con un valor inicial de la variable índice y las acciones especificadas se ejecutan a menos que el valor inicial sea mayor que el valor final. La variable índice se incrementa en uno y si este nuevo valor no excede al final, se ejecutan de nuevo las acciones. Por consiguiente, las acciones específicas en el bucle se ejecutan para cada valor de la variable índice desde el valor inicial hasta el valor final con el incremento de uno en uno.

El incremento de la variable índice siempre es 1 si no se indica expresamente lo contrario. Dependiendo del tipo de lenguaje, es posible que el incremento sea distinto de uno, positivo o negativo. Así, por ejemplo, BASIC y FORTRAN admiten diferentes valores positivos o negativos del incremento, y Pascal sólo admite incrementos cuyo tamaño es la unidad: bien positivos, bien negativos.

La variable índice o de control normalmente será de tipo entero y es normal emplear como nombres las letras I, J, K.

El formato de la estructura **desde** varía si se desea un incremento distinto a 1, bien positivo, bien negativo (decremento).

```
desde v ← vi hasta vf inc | dec paso hacer    //inc, incremento
                                              //dec, decremento
  <acciones>
    .
    .
    .
fin_desde
```

Si el valor inicial de la variable índice es menor que el valor final, los incrementos deben ser positivos, ya que en caso contrario la secuencia de acciones no se ejecutaría. De igual modo, si el valor inicial es mayor que el valor final, el incremento debe ser en este caso negativo, es decir, *decremento*. Al incremento se le suele denominar también *paso* ("`step`", en inglés).

Es decir,

```
desde i ← 20 hasta 10 hacer
  <acciones>
fin_desde
```

no se ejecutaría, ya que el valor inicial es `20` y el valor final `10`, y como se supone un incremento positivo, de valor 1, se produciría un error.

El ***pseudocódigo*** correcto debería ser:

```
desde i ← 20 hasta 10 decremento 1 hacer
  <acciones>
fin_desde
```

Realización de una estructura `desde` con estructura `mientras`

Es posible, como ya se ha mencionado en apartados anteriores, sustituir una estructura **`desde`** por una **`mientras`**; en las líneas siguientes se indican dos formas para ello:

1. *Estructura* **`desde`** *con incrementos de la variable índice positivos.*

```
v ← vi
mientras v <= vf hacer
  <acciones>
  v ← v + incremento
fin_mientras
```

2. *Estructura* **`desde`** *con incrementos de la variable índice negativos.*

```
v ← vi
mientras v >= vf hacer
  <acciones>
  v ← v - decremento
fin_mientras
```

La estructura **`desde`** puede realizarse con algoritmos basados en estructura **`mientras`** y **`repetir`**, por lo que pueden ser intercambiables cuando así lo desee. Las estructuras equivalentes a **`desde`** son las siguientes:

a)
```
inicio
  i ← n
  mientras i > 0 hacer
    <acciones>
    i ← i - 1
  fin_mientras
fin
```

b)
```
inicio
  i ← 1
  mientras i <= n hacer
    <acciones>
    i ← i + 1
  fin_mientras
fin
```

c)
```
inicio
   i ← 0
   repetir
     <acciones>
     i ← i + 1
   hasta_que i = n
fin
```

d)
```
inicio
   i ← 1
   repetir
     <acciones>
     i ← i + 1
   hasta_que i > n
fin
```

e)
```
inicio
   i ← n + 1
   repetir
     <acciones>
     i ← i - 1
   hasta_que i = 1
fin
```

f)
```
inicio
   i ← n
   repetir
     <acciones>
     i ← i - 1
   hasta_que i < 1
fin
```

4.6.4. Salidas internas de los bucles

Aunque no se incluye dentro de las estructuras básicas de la programación estructurada, en ocasiones es necesario disponer de una estructura repetitiva que permita la salida en un punto intermedio del bucle cuando se cumpla una condición. Esta nueva estructura sólo está disponible en algunos lenguajes de programación específicos; la denominaremos **salir** o **iterar** para diferenciarla de **repetir_hasta** ya conocida. Las salidas de bucles suelen ser válidas en estructuras **mientras**, **repetir** y **desde**.

El formato de la estructura es:

```
repetir
  <acciones>
  si <condicion> entonces
    salir_bucle
  fin_si
  <acciones>
fin_repetir
```

En general, la instrucción **salir** no produce un programa legible y comprensible como lo hacen **mientras** y **repetir**. La razón para esta ausencia de claridad es que la salida de un bucle ocurre en el medio del bucle, mientras que normalmente la salida del bucle es al principio o al final del mismo. Le recomendamos que no recurra a esta opción —aunque la tenga su lenguaje— más que cuando no exista otra alternativa o disponga de la estructura **iterar** (*loop*).

Ejemplo 4.24

Una aplicación de un posible uso de la instrucción salir se puede dar cuando se incluyen mensajes de petición en el algoritmo para la introducción sucesiva de informaciones.

Algoritmo 1
```
leer(informacion)
repetir
  procesar informacion
  leer(informacion)
hasta_que fin de lectura
```

Algoritmo 2
```
leer(informacion)
mientras_no fin de lectura
  procesar informacion
  leer(informacion)
fin_mientras
```

En los algoritmos anteriores cada entrada (lectura) de información va acompañada de su correspondiente proceso, pero la primera lectura está fuera del bucle. Se pueden incluir en el interior del

bucle todas las lecturas de información si se posee una estructura **salir (exit)**. Un ejemplo de ello es la estructura siguiente:

```
repetir
  leer(informacion)
  si fin de lectura entonces
    salir bucle
  fin_si
  procesar informacion
fin_repetir
```

Ejemplo 4.25 (Bucle controlado por centinela)

Algoritmo para calcular el salario de una serie de trabajadores, cuyos datos serán introducidos por teclado. Sabiendo que las horas normales se pagan a 2.000 pts. y las extraordinarias a 2.500 mientras no excedan de 10. Si las horas extraordinarias sobrepasaran las 10, el exceso se pagaría a 2.350.

La introducción de datos terminará cuando se escriba '' como nombre para un trabajador.*

```
algoritmo Calcular_salario
var
    cadena : nombre
    real   : hnormales, hextraordinarias, salario

inicio
  escribir('Deme nombre (* para fin)')
  leer(nombre)

  mientras nombre <> '*' hacer
    escribir('Deme nº de horas normales ')
    leer( hnormales)
    escribir('Deme nº de horas extraordinarias ')
    leer( hextraordinarias)
    salario ← hnormales * 2000
    si hextraordinarias <= 10 entonces
        salario ← salario + hextraordinarias * 2500
      si_no
        salario ← salario + 10 * 2500+(hextraordinarias - 10) * 2350
    fin si
    escribir( nombre, salario)
    escribir('Deme nombre (* para fin)          ')
    leer(nombre)
  fin_mientras
fin
```

Ejemplo 4.26 (Bucle controlado por bandera)

Algoritmo que nos determine si un número entero leído de teclado es o no primo. Un número es compuesto en cuanto que tiene algún divisor distinto de sí mismo o la unidad.

Vamos probando con todos los divisores posibles del número (2 a *n* div 2) y en cuanto que detectemos un divisor ya podremos decir que no es primo.

```
algoritmo Es_o_no_primo
var
  entero : n,divisor
  logico : primo
```

```
inicio
  escribir('Deme numero ')
  leer(n)
  primo ← verdad
  divisor ← 2

// la variable primo controla el bucle
mientras (divisor <= n div 2) y primo hacer
  si n MOD divisor = 0 entonces
    primo ← falso
  si_no
    divisor ← divisor + 1
  fin si
fin_mientras
  si primo entonces  // La variable primo pasa informacion a
                     // otra parte del programa
    escribir( n, ' es primo')
  si_no
    escribir( n, ' es divisible por ', divisor)
  fin_si
fin
```

Ejemplo 4.27 (Bucle controlado por contador)

Algoritmo que nos pide la introducción por teclado de las notas de una clase de 40 alumnos y nos calcula el tanto por ciento de suspensos.

```
algoritmo Tanto_por_ciento
var
    entero : contalumnos, contsuspensos
    real   : tantoporciento, n
inicio
  contalumnos ← 1
  contsuspensos ← 0

  mientras contalumnos <= 40 hacer
    escribir( 'Alumno', contalumnos,'- Deme nota ')
    leer(n)
    si n < 5 entonces
      contsuspensos ← contsuspensos + 1
    fin_si
    contalumnos ← contalumnos + 1
  fin_mientras

  tantoporciento ← contsuspensos * 100 / 40
  escribir('El tanto por ciento de suspensos es ', tantoporciento)
fin
```

Ejemplo 4.28 (Bucle `mientras`: se puede ejecutar de 0 a N veces)

Algoritmo para descomponer un número entero, leído de teclado, en sus factores primos.

```
algoritmo Descomponer
var
  entero : n, divisor
```

```
inicio
  escribir('Deme un numero ')
  leer(n)
  escribir('Su descomposicion en factores primos es : 1')
  divisor ← 2
  mientras n > 1 hacer
    si n mod divisor = 0 entonces
      escribir(divisor)
      n ← n div divisor
    si_no
      divisor ← divisor + 1
    fin_si
  fin_mientras
  escribir()
fin
```

Ejemplo 4.29

Pseudocódigo que lea un número entero positivo y lo escriba invertido.

```
algoritmo Invertir
var
  entero : n, resto

inicio
  escribir('Deme numero')
  leer(n)

  mientras n > 0 hacer
    resto ← n mod 10
    escribir(resto)
    n ← n div 10
  fin_mientras

fin
```

Ejemplo 4.30

Dado un capital C, *que se coloca a un interés anual* R, *averiguar al cabo de cuántos años se doblará.*

```
algoritmo Calcular_capital
var
  real   : cinicial,c,i
  entero : años
inicio
  escribir('Deme capital')
  leer(cinicial)
  c ← cinicial
  años ← 0
  escribir('Deme interes anual en %')
  leer(i)
  repetir
    c ← c + c * i / 100
    años ← años + 1
  hasta_que c >= 2 * cinicial
  escribir('Para duplicar el capital se necesitan ', años,' años')
fin
```

Ejemplo 4.31

Dado que el valor de e ^ x (e^x) se puede aproximar por la suma

$$1 + x + x \wedge 2 / 2! + x \wedge 3 / 3! + \ldots\ldots\ldots + x \wedge n / n!$$

escribir un algoritmo que tome un valor de x y calcule e ^ x (e^x), mediante el método expuesto y empleando los términos necesarios hasta que x ^ n / n! sea menor que 0.01.

```
algoritmo E_elevado_a_x
var
  entero : x,i
  real   : e,num,den

inicio
  escribir('Dame el valor de x ')
  leer( x)
  e ← 1
  num ← 1
  den ← 1
  i ← 1
  repetir
    num ← x * num
    den ← den * i
    i ← i + 1
    e ← e + num / den
  hasta_que (num / den) < 0.01
  escribir( 'e elevado a ', x, ' es ', e)
fin
```

Ejemplo 4.32

Algoritmo que lea día, mes y año, así como el día de la semana que fue el 1 de enero de dicho año y nos saque por pantalla el día de la semana que corresponde a la fecha que le hemos dado.

Deben considerarse los años bisiestos y depurar mes y día.

```
algoritmo Dia_semana
var
  entero   : dd, mm, aa, contm, dtotal, d1
  caracter : dia
  logico   : correcta, bisiesto

inicio
  repetir              //comienzo de repetir
    bisiesto ← falso
    correcta ← verdad
    escribir('Deme fecha (dd mm aaaa)')
    leer(dd, mm, aa)
    si (aa MOD 4 = 0) Y (aa MOD 100 <> 0) O (aa MOD 400 = 0) entonces
      bisiesto ← verdad
    fin_si

    segun_sea mm hacer
      1, 3, 5, 6, 8, 10, 12:
        si dd > 31 entonces
          correcta ← falso
        fin_si
```

```
    4, 7, 9, 11:
      si dd > 30 entonces
        correcta ← falso
      fin_si
    2:
      si bisiesto entonces
        si dd > 29 entonces
          correcta ← falso
        fin_si
        si_no
          si dd > 28 entonces
            correcta ← falso
          fin_si
        fin_si
  si_no
    correcta ← falso
  fin_segun
hasta_que correcta              //fin de repetir

dtotal ← 0
desde contm ← 1 hasta mm - 1 hacer      //comienzo desde
  segun_sea contm hacer
    1, 3, 5, 6, 8, 10, 12:
      dtotal ← dtotal + 31
    4, 7, 9, 11:
      dtotal ← dtotal + 30
    2:
      si bisiesto entonces
          dtotal ← dtotal + 29
      si_no
          dtotal ← dtotal + 28
      fin_si
  fin_segun
fin_desde

dtotal ← dtotal + dd
escribir('Deme dia de la semana que fue el 1 de ENERO (l/m/x/j/v/s/d) ')
leer(dia)
segun_sea dia hacer
  'l':
    d1 ← 0
  'm':
    d1 ← 1
  'x':
    d1 ← 2
  'j':
    d1 ← 3
  'v':
    d1 ← 4
  's':
    d1 ← 5
  'd':
    d1 ← 6
fin_segun
```

```
    dtotal ← dtotal + d1
    segun_sea dtotal mod 7 hacer
      1:
        escribir('Lunes')
      2:
        escribir('Martes')
      3:
        escribir('Miercoles')
      4:
        escribir('Jueves')
      5:
        escribir('Viernes')
      6:
        escribir('Sabado')
      0:
        escribir('Domingo')
    fin_segun

fin
```

4.7. ESTRUCTURAS DE DECISIÓN ANIDADAS

Las estructuras de selección **si-entonces** y **si-entonces-si_no** implican la selección de una de dos alternativas. Es posible también utilizar la instrucción **si** para diseñar estructuras de selección que contengan más de dos alternativas. Por ejemplo, una estructura **si-entonces** puede contener otra estructura **si-entonces**, y esta estructura **si-entonces** puede contener otra, y así sucesivamente cualquier número de veces; a su vez, dentro de cada estructura pueden existir diferentes acciones.

```
si condicion1 entonces
    si condicion2 entonces
        escribir 'hola Mortimer'
        ...
```

Las estructuras **si** interiores a otras estructuras **si** se denominan *anidadas* o *encajadas*:

```
si condicion1 entonces
  si condicion2 entonces
    .
    .
    .
      <acciones>
  fin_si
fin_si
```

Una estructura de selección de *n* alternativas o de decisión múltiple puede ser construida utilizando una estructura **si** con este formato:

```
si condicion1 entonces
  <acciones>
```

```
si_no
  si condicion2 entonces
    <acciones>
  si_no
    si condicion3 entonces
      <acciones>
    si_no
      .
      .
      .
    fin_si
  fin_si
fin_si
```

Una estructura selectiva múltiple constará de una serie de estructuras **si**, unas interiores a otras. Como las estructuras **si** pueden volverse bastante complejas para que el algoritmo sea claro, será preciso utilizar *indentación* (sangría o sangrado), de modo que exista una correspondencia entre las palabras reservadas **si** y **fin_si**, por un lado, y **entonces** y **si_no**, por otro.

La escritura de las estructuras puede variar de unos lenguajes a otros, por ejemplo, una estructura **si** admite también los siguientes formatos:

```
si expresion booleana1 entonces
  <acciones>
si_no
  si expresion booleana2 entonces
    <acciones>
  si_no
    si expresion booleana3 entonces
      <acciones>
    si_no
      <acciones>
    fin_si
  fin_si
fin_si
```

o bien:

```
si expresion booleana1 entonces
  <acciones>
si_no si expresion booleana2 entonces
         <acciones>
         fin_si
    .
    .
    .
fin_si
```

Ejemplo 4.33

Diseñar un algoritmo que lea tres números A, B, C *y visualice en pantalla el valor del más grande. Se supone que los tres valores son diferentes.*

Los tres números son A, B y C; para calcular el más grande se realizarán comparaciones sucesivas por parejas.

```
algoritmo Mayor
var
  real: A, B, C, Mayor
inicio
  leer(A, B, C)
  si A > B entonces
    si A > C entonces
      Mayor ← A          //A > B, A > C
    si_no
      Mayor ← C          //C >= A > B
    fin_si
  si_no
    si B > C entonces
      Mayor ← B          //B >= A, B > C
    si_no
      Mayor ← C          //C >= B >= A
    fin_si
  fin_si
  escribir('Mayor:', Mayor)
fin
```

Ejemplo 4.34

El siguiente algoritmo lee tres números diferentes, A, B *y* C, *e imprime los valores máximo y mínimo. El procedimiento consistirá en comparaciones sucesivas de parejas de números.*

```
algoritmo Ordenar
var
  real : a,b,c
inicio
  escribir('Deme 3 numeros')
  leer(a, b, c)
  si a > b entonces          // consideramos los dos primeros (a, b)
                             // y los ordenamos
     si b > c entonces       // tomo el 3º (c) y lo comparo con el menor (a o b)
        escribir(a, b, c)
      si_no
      si c > a entonces      // si el 3º es mayor que el menor averiguo si va
                             // delante o detras del mayor
        escribir(c, a, b)
      si_no
        escribir(a, c, b)
      fin_si
    fin_si
  si_no
    si a > c entonces
      escribir(b, a, c)
    si_no
      si c > b entonces
        escribir(c, b, a)
      si_no
        escribir(b, c, a)
      fin_si
    fin_si
  fin_si
fin
```

Ejemplo 4.35

Pseudocódigo que nos permita calcular las soluciones de una ecuación de segundo grado, incluyendo los valores imaginarios.

```
algoritmo Soluciones_ecuacion
var
  real : a,b,c,d,x1,x2,r,i
inicio
  escribir('Deme los coeficientes')
  leer(a, b, c)
  si a = 0 entonces
    escribir('No es ecuacion de segundo grado')
  si_no
    d ← b * b - 4 * a * c
    si d = 0 entonces
      x1 ← -b / (2 * a)
      x2 ← x1
      escribir(x1,x2)
    si_no
      si d > 0 entonces
        x1 ← (-b + raiz2(d)) / (2 * a)
        x2 ← (-b - raiz2(d)) / (2 * a)
        escribir(x1, x2)
      si_no
        r ← (-b) / (2 * a)
        i ← raiz2(abs(d)) / (2 * a)
        escribir(r, '+', i, 'i')
        escribir(r, '-', i, 'i')
      fin_si
    fin_si
  fin_si
fin
```

Ejemplo 4.36

Algoritmo al que le demos la hora `HH`, `MM`, `SS` *y nos calcule la hora dentro de un segundo. Leeremos las horas minutos y segundos como números enteros.*

```
algoritmo Hora_segundo_siguiente
var
  entero : hh,mm,ss
inicio
escribir('Deme hh,mm,ss')
leer(hh, mm, ss)
si (hh < 24) Y (mm < 60) Y (ss < 60) entonces
  ss ← ss + 1
  si ss = 60 entonces
    ss ← 0
    mm ← mm + 1
    si mm = 60 entonces
      mm ← 0
      hh ← hh + 1
      si hh = 24 entonces
        hh ← 0
      fin_si
    fin_si
  fin_si
 escribir(hh, ':', mm, ':', ss)
 fin_si
fin
```

4.8. ESTRUCTURAS REPETITIVAS ANIDADAS

De igual forma que se pueden anidar o encajar estructuras de selección, es posible insertar un bucle dentro de otro. Las reglas para construir estructuras repetitivas anidadas son iguales en ambos casos: la estructura interna debe estar incluida totalmente dentro de la externa y no puede existir solapamiento. La representación gráfica se indica en la Figura 4.14.

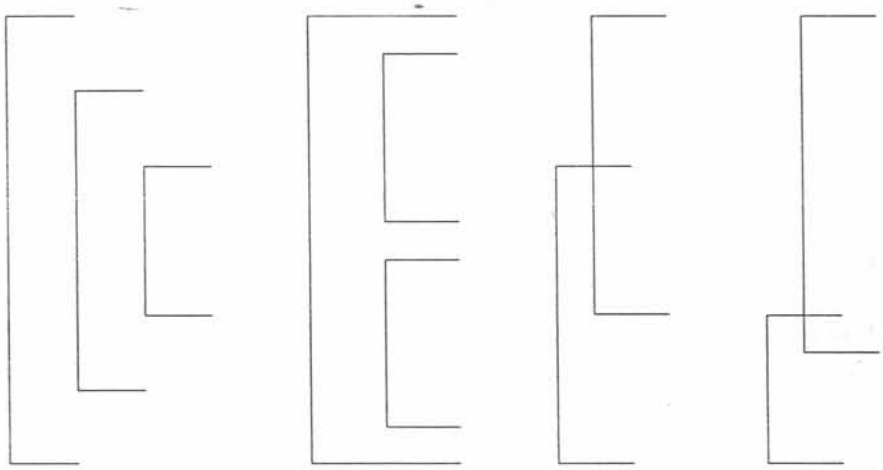


Figura 4.14. Bucles anidados: a) y b), correctos; c) y d), incorrectos.

Las variables índices o de control de los bucles toman valores de modo tal que por cada valor de la variable índice del ciclo externo se debe ejecutar totalmente el bucle interno.

Es posible anidar cualquier tipo de estructura repetitiva con tal que cumpla las condiciones de la Figura 4.14.

Ejemplo 4.37

Se conoce la población de cada una de las veinticinco ciudades más grandes de las ocho provincias de Andalucía y se desea identificar y visualizar la población de la ciudad más grande de cada provincia.

El problema consistirá, en primer lugar, en la obtención de la población mayor de cada provincia y realizar esta operación ocho veces, una para cada provincia.

```
1. Encontrar y visualizar la ciudad mayor de una provincia.
2. Repetir el paso 1 para cada una de las ocho provincias andaluzas.
```

El procedimiento para deducir la ciudad más grande de entre las veinticinco de una provincia se consigue creando una variable auxiliar `MAYOR` —inicialmente de valor 0— que se va comparando sucesivamente con los veinticinco valores de cada ciudad, de modo tal que, según el resultado de comparación, se intercambian valores de la ciudad por el de la variable `MAYOR`.

El algoritmo correspondiente sería:

```
algoritmo CIUDADMAYOR
var
  entero: i          //contador de provincias
  entero: j          //contador de ciudades
  entero: MAYOR      //mayor poblacion
  entero: CIUDAD     //poblacion de una ciudad
inicio
  i ← 1
  mientras i <= 8 hacer
    MAYOR ← 0
    j ← 1
    mientras j <= 25 hacer
      leer(CIUDAD)
      si CIUDAD > MAYOR entonces
        MAYOR ← CIUDAD
        j ← j + 1
      fin_si
    fin_mientras
    escribir('La mayor poblacion tiene', MAYOR)
    i ← i + 1
  fin_mientras
fin
```

Ejemplo 4.38

Calcular el factorial de n *números leídos del terminal.*

El problema consistirá en realizar una estructura repetitiva de n iteraciones del algoritmo del problema ya conocido del cálculo del factorial de un entero.

```
algoritmo Factorial2
var
  entero: i, NUMERO, n
  real: FACTORIAL
inicio
  // lectura de la cantidad de numeros
  leer(n)
  desde i ← 1 hasta n hacer
    leer(NUMERO)
    FACTORIAL ← 1
    desde j ← 1 hasta NUMERO hacer
      FACTORIAL ← FACTORIAL * j
    fin_desde
    escribir('El factorial del numero', NUMERO, 'es',FACTORIAL)
  fin_desde
fin
```

Ejemplo 4.39

Imprimir todos los números primos entre 2 *y* 100 *inclusive.*

```
algoritmo Primos
var entero : i, divisor
    logico : primo
```

```
inicio
  desde i ← 2 hasta 100 hacer
    primo ← verdad
    divisor ← 2
    mientras (divisor <= raiz2(i)) y primo hacer
      si i mod divisor = 0 entonces
        primo ← falso
      si_no
        divisor ← divisor + 1
      fin_si
    fin_mientras
    si primo entonces
      escribir( i,' ')
    fin_si
  fin_desde
fin
```

4.9. LA INSTRUCCIÓN `ir_a («goto»)`

El flujo de control de un algoritmo es siempre secuencial, excepto cuando las estructuras de control estudiadas anteriormente realizan transferencias de control no secuenciales.

La programación estructurada permite realizar programas fáciles y legibles utilizando las tres estructuras ya conocidas: *secuenciales*, *selectivas* y *repetitivas*. Sin embargo, en ocasiones es necesario realizar bifurcaciones incondicionales; para ello se recurre a la instrucción **`ir_a (goto)`**. Esta instrucción siempre ha sido problemática y prestigiosos informáticos, como Dijkstra, han tachado la instrucción **`goto`** como nefasta y perjudicial para los programadores y recomiendan no utilizarla en sus algoritmos y programas. Por ello, lenguajes como Pascal *huyen* de esta instrucción y prácticamente no la utilizan nunca.

Aunque la instrucción **`ir_a (goto)`** la tienen todos los lenguajes de programación en su juego de instrucciones, existen algunos que dependen más de ella que otros, como BASIC y FORTRAN. En general, no existe ninguna necesidad de utilizar instrucciones **`ir_a`**. Cualquier algoritmo o programa que se escriba con instrucciones **`ir_a`** se puede reescribir para hacer lo mismo y no incluir ninguna instrucción **`ir_a`**. Un programa que utiliza muchas instrucciones **`ir_a`** es más difícil de leer que un programa bien escrito que utiliza pocas o ninguna instrucción **`ir_a`**. En muy pocas situaciones las instrucciones **`ir_a`** son útiles; tal vez, las únicas razonables son diferentes tipos de situaciones de salida de bucles. Cuando se encuentra un error u otra condición de terminación, una instrucción **`ir_a`** puede ser utilizada para saltar directamente al final de un bucle, subprograma o un procedimiento completo.

Las bifurcaciones o *saltos* producidos por una instrucción **`ir_a`** deben realizarse a instrucciones que estén numeradas o posean una etiqueta que sirva de punto de referencia para el salto.

Por ejemplo, un programa puede ser diseñado para terminar con una detección de un error.

```
algoritmo Error
.
.
.
  si <condicion error> entonces
    ir_a(100)
  fin_si
100:fin
```

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

4.1. Calcular el factorial de un número N utilizando la estructura desde.

Recordemos que factorial de N responde a la fórmula

$$N! = N \cdot (N-1) \cdot (N-2) \cdot (N-3) \cdot \ldots \cdot 3 \cdot 2 \cdot 1$$

El algoritmo **desde** supone conocer el número de iteraciones:

```
inicio
var
  entero: I, N
  real: FACTORIAL
inicio
  // N >= 0
  leer(N)
  FACTORIAL ← 1
  desde I ← 1 hasta N hacer
    FACTORIAL ← FACTORIAL * I
  fin_desde
  escribir('El factorial de', N, 'es', FACTORIAL)
fin
```

4.2. Imprimir las treinta primeras potencias de 4, es decir, 4 elevado a 1, 4 elevado a 2, etc.

```
algoritmo Potencias4
var
  entero: n
inicio
  desde n ← 1 hasta 30 hacer
    escribir(4 ^ n)
  fin_desde
fin
```

4.3. Calcular la suma de los n primeros números enteros utilizando la estructura desde.

$$S = 1 + 2 + 3 + \ldots + n$$

El *pseudocódigo* correspondiente es:

```
algoritmo SumaNenteros
var
  entero: i, n
  real: suma
inicio
  leer(n)
  suma ← 0
  desde i ← 1 hasta n hacer
    suma ← suma + i
  fin_desde
  //escribir el resultado de suma
  escribir(suma)
fin
```

4.4. Diseñar el algoritmo para imprimir la suma de los números impares menores o iguales que *n*.

Los números impares son 1, 3, 5, 7, ..., *n*, un pseudocódigo será:

```
algoritmo Sumaimparesmenores
var
  entero: i, n
  real: s
inicio
  S ← 0
  leer(n)
  desde i ← 1 hasta n incremento 2 hacer
    S ← S + i
  fin_desde
  escribir(S)
fin
```

4.5. Dados dos números enteros, realizar el algoritmo que calcule su cociente y su resto.

Sean los números M y N. El método para obtener el cociente y el resto es por restas sucesivas; el método sería restar sucesivamente el divisor del dividendo hasta obtener un resultado menor que el divisor, que será el resto de la división; el número de restas efectuadas será el cociente:

M	N		
50	13	50 – 13 = 37	C = 1
11	3	37 – 13 = 24	C = 2
		24 – 13 = 11	C = 3

Como 11 es menor que el divisor, 13, se terminarán las restas sucesivas y entonces 11 será el resto, y 3 (número de restas) el cociente.
Por consiguiente, el algoritmo será el siguiente:

```
algoritmo Cociente_y_resto
var
  entero : a,b,cociente,resto
inicio
  escribir('Deme dividendo')
  leer(M)
  escribir('Deme divisor')
  leer(N)
  resto ← M
  cociente ← 0
  mientras resto >= N hacer
    resto ← resto - N
    cociente ← cociente + 1
  fin_mientras
  escribir('Cociente: ',cociente,'  Resto: ', resto)
fin
```

4.6. Realizar el algoritmo para obtener la suma de los números pares hasta 1000 inclusive.

Método 1:

S = 2 + 4 + 6 + 8 + ... + 1000

```
algoritmo Sumapares
var
  real: NUMERO, SUMA
inicio
  SUMA ← 2
  NUMERO ← 4
  mientras NUMERO <= 1000 hacer
    SUMA ← SUMA + NUMERO
    NUMERO ← NUMERO + 2
  fin_mientras
fin
```

Método 2:

```
//identica cabecera y declaraciones

inicio
  SUMA ← 2
  NUMERO ← 4
  repetir
    SUMA ← SUMA + NUMERO
    NUMERO ← NUMERO + 2
  hasta_que NUMERO > 1000
fin
```

4.7. Buscar y escribir la primera vocal leída del teclado. (Se supone que se leen, uno a uno, caracteres desde el teclado.)

```
algoritmo Buscar_vocal
var
  caracter: p
inicio
  repetir
    leer(p)
  hasta_que (p = 'a')o(p = 'e')o(p = 'i')o(p = 'o')o(p = 'u')
  escribir('Primero', p)
fin
```

4.8. Leer dos números y deducir si están en orden creciente.

Dos números, *a* y *b*, están en orden creciente si *a* <= *b*.

```
algoritmo Comparacion1
var
  real: a,b
inicio
  escribir('dar dos numeros')
  leer(a, b)
  si a <= b entonces
    escribir('orden creciente')
  si_no
    escribir('orden decreciente')
  fin_si
fin
```

4.9. Se desea leer de una consola a una serie de números hasta obtener un número inferior a 100.

```
algoritmo Menor_100
var
  real: numero
inicio
  repetir
    escribir('Teclear un numero')
    leer(numero)
  hasta_que numero < 100
  escribir('El numero es', numero)
fin
```

4.10. Escribir un algoritmo que permita escribir en una pantalla la frase '¿Desea continuar? S/N' hasta que la respuesta sea 'S' o 'N'.

```
algoritmo SN
var
  caracter: respuesta
inicio
  repetir
    escribir('Desea continuar S/N')
    leer(respuesta)
  hasta_que(respuesta = 'S') o (respuesta = 'N')
fin
```

4.11. Leer sucesivamente números del teclado, hasta que aparezca un número comprendido entre 1 y 5.

```
algoritmo Numero1_5
var
  entero: numero
inicio
  repetir
    escribir('Numero comprendido entre 1 y 5')
    leer(numero)
  hasta_que(numero >= 1) y (numero <= 5)
  escribir('Numero encontrado', numero)
fin
```

4.12. Calcular el factorial de un número *n* con métodos diferentes al Ejercicio 4.1.

$n! = n \times (n-1) \times (n-2) \times \ldots \times 3 \times 2 \times 1$

es decir:

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
$4! = 4 \times 3 \times 2 \times 1 = 24$
$3! = 3 \times 2 \times 1 = 6$
$2! = 2 \times 1 = 2$
$1! = 1 = 1$

Para codificar estas operaciones, basta pensar que

$$(n+1)! = (n+1) \times n \times \underbrace{(n-1) \times (n-2) \times \ldots \times 3 \times 2 \times 1}_{n!}$$

$(n+1)! = (n+1) \times n!$

Por consiguiente, para calcular el factorial FACTORIAL de un número, necesitaremos un contador *i* que cuente de uno en uno y aplicar la fórmula

```
FACTORIAL = FACTORIAL * I
```

inicializando los valores de FACTORIAL e I a 1 y realizando un bucle en el que I se incremente en 1 a cada iteración, es decir,

***Algoritmo** 1 de Factorial de n:*

```
FACTORIAL ← 1
i ← 1
repetir
  FACTORIAL ← FACTORIAL * i
  i ← i + 1
hasta_que i = n + 1
```

***Algoritmo** 2 de Factorial de n:*

```
FACTORIAL ← 1
i ← 0
repetir
  FACTORIAL ← FACTORIAL * (i + 1)
  i ← i + 1
hasta_que i = n
```

***Algoritmo** 3 de Factorial de n:*

```
FACTORIAL ← 1
i ← 0
repetir
  FACTORIAL ← FACTORIAL * (i + 1)
  i ← i + 1
hasta_que i > n - 1
```

***Algoritmo** 4 de factorial de n:*

```
FACTORIAL ← 1
i ← 1
desde i ← 1 hasta n - 1 hacer
  FACTORIAL ← FACTORIAL * (i + 1)
fin_desde
```

Un algoritmo completo con lectura del número n por teclado podría ser el siguiente:

```
algoritmo Factorial
var
  entero: i, n
  real: f
inicio
  f ← 1
  i ← 1
  leer(n)      //n > 0
  repetir
    f ← f * i
    i ← i + 1
  hasta_que i = n + 1
  escribir('Factorial de', n, 'es', f)
fin
```

4.13. Calcular el valor máximo de una serie de 100 números.

Para resolver este problema, necesitaremos un contador que cuente de 1 a 100, para contabilizar los sucesivos números.
El algoritmo que calcula el valor máximo será repetido y partiremos considerando que el primer número leído es el valor máximo, por lo cual se realizará una primera asignación del número 1 a la variable *máximo*.

```
leer(numero)
n = 1
maximo = numero
```

La siguiente acción del algoritmo será realizar comparaciones sucesivas:

- leer un nuevo número;
- compararlo con el valor máximo:
  - — si es inferior o igual, implica que el valor máximo es el antiguo;
  - — si es superior, implica que el valor máximo es el recientemente leído, por lo que éste se convertirá en máximo mediante una asignación;
  - — repetir las acciones anteriores hasta que `n = 100`.

```
algoritmo Maximo
var
  entero: n, numero, maximo
inicio
  leer(numero)
  n ← 1
  maximo ← numero
  repetir
    n ← n + 1
    leer(numero)
    si numero > maximo entonces
      maximo ← numero
    fin_si
  hasta_que n = 100
  escribir('Numero mayor o maximo', maximo)
fin
```

Otras soluciones

```
1. algoritmo Otromaximo
   var
     entero: n, numero, maximo
   inicio
     leer(numero)
     maximo ← numero
     n ← 2
     repetir
       n ← n + 1
       leer(numero)
       si numero > maximo entonces
             maximo ← numero
       fin_si
     hasta_que n > 100
     escribir('Numero mayor o maximo', maximo)
   fin
```

```
2. algoritmo Otromaximo
   var
     entero: n, numero, maximo
   inicio
     leer(numero)
     maximo ← numero
     repetir_para n = 2 a 100       //pseudocodigo sustituto de desde
       leer(numero)
       si numero > maximo entonces
          maximo ← numero
       fin_si
     fin_repetir
     escribir('Maximo,', maximo)
   fin
```

NOTA: Los programas anteriores suponen que los números pueden ser positivos o negativos; si se desea comparar sólo números positivos, los programas correspondientes serían:

```
1. algoritmo Otromaximo
   var
     entero: n, numero, maximo
   inicio
     n ← 0
     maximo ← 0
     repetir
       leer(numero)
       n ← n + 1
       si numero > maximo entonces
          maximo ← numero
       fin_si
     hasta_que n = 100
     escribir('Maximo numero', maximo)
   fin
```

```
2. algoritmo Otromaximo
   var
     entero: n, numero, maximo
   inicio
     n ← 0
     maximo ← 0
     desde N ← 1 hasta 100 hacer
       leer(numero)
       si numero > maximo entonces
          maximo ← numero
       fin_si
     fin_desde
     escribir('Maximo numero =', maximo)
   fin
```

4.14. Bucles anidados.

Las estructuras de control tipo bucles pueden anidarse internamente, es decir, se puede situar un bucle en el interior de otro bucle.
La anidación puede ser:

- bucles **repetir** dentro de bucles **repetir**,
- bucles **para (desde)** dentro de bucles **repetir**, etc.

Ejemplo 1. Bucle para en el interior de un bucle `repetir-hasta_que`:

```
repetir
  leer(n)
  desde i ← 1 hasta 5 hacer
    escribir(n * n)
  fin_desde
hasta_que n = 0
escribir('Fin')
```

Si ejecutamos estas instrucciones, se obtendrá para:

```
n = 5        resultados      25
                             25
                             25
                             25
                             25

n = 2        resultados      4
                             4
                             4
                             4
                             4

n = 0        resultados      0
                             0
                             0
                             0
                             0
             fin
```

Ejemplo 2. Bucles anidados para (desde):

```
desde i ← 1 hasta 3 hacer
  desde j ← 1 hasta 10 hacer
    escribir(i, 'por', j, '=', i*j)
  fin_desde
fin_desde
```

Los valores sucesivos de i, j, i * j, serán:

```
i = 1  j = 1     i * j = 1 x 1 = 1
       j = 2     i * j = 1 x 2 = 2
       j = 3     i * j = 1 x 3 = 3
       j = 4     i * j = 1 x 4 = 4
       ..........................
       j = 10    i * j = 1 x 10 = 10

i = 2  j = 1     i * j = 2 x 1 = 2
       j = 2     i * j = 2 x 2 = 4
       j = 3     i * j = 2 x 3 = 6
       j = 4     i * j = 2 x 4 = 8
       ..........................
       j = 10    i * j = 2 x 10 = 20

i = 3  j = 1     i * j = 3 x 1 = 3
       j = 2     i * j = 3 x 2 = 6
       j = 3     i * j = 3 x 3 = 9
       j = 4     i * j = 3 x 4 = 12
       ..........................
       j = 10    i * j = 3 x 10 = 30
```

Es decir, se obtendrá la tabla de multiplicar de 1, 2 y 3.

Ejemplo 3. La codificación completa para obtener la popular tabla de multiplicar de los escolares sería la siguiente:

```
algoritmo Tabla_de_multiplicar
var
  entero: i, j, producto
inicio
  para i ← 1 hasta 3 hacer
    escribir('Tabla del', i)
    para j ← 1 hasta 10 hacer
      producto ← i * j
      escribir(i, 'por', j, '=', producto)
    fin_para
  fin_para
fin
```

4.15. Se dispone de una lista de N números. Se desea calcular el valor del número mayor.

```
algoritmo
var
  entero: I
  real: NUM, MAX
  entero: N
inicio
  leer(N)      // N > 0
  leer(NUM)
  MAX ← NUM
  desde I ← 2 hasta 100 hacer
    leer(NUM)
    si NUM > MAX entonces
      MAX ← NUM
    fin_si
  fin_desde
fin
```

4.16. Determinar simultáneamente los valores máximo y mínimo de una lista de 100 números.

```
algoritmo Max_min
var
  I: entero
  MAX, MIN, NUMERO: real
inicio
  leer(NUMERO)
  MAX ← NUMERO
  MIN ← NUMERO
  desde I ← 2 hasta 100 hacer
    leer(NUMERO)
    si NUMERO > MAX entonces
      MAX ← NUMERO
    si_no
      si NUMERO < MIN entonces
        MIN ← NUMERO
      fin_si
    fin_si
  fin_desde
  escribir('Maximo', MAX, 'Minimo', MIN)
fin
```

4.17. Se dispone de un cierto número de valores de los cuales el último es el 999 (no se procesa: es final de lista) y se desea determinar el valor máximo de las medias correspondientes a parejas de valores sucesivos.

```
algoritmo Media_parejas
var
  entero: N1, N2
  real: M,MAX
inicio
  leer(N1, N2)
  MAX ← (N1 + N2)/2
  mientras (N2 <> 999) y (N1 <> 999) hacer
    M ← (N1 + N2)/2
    si M > MAX entonces
      MAX ← M
    fin_si
    leer (N1, N2)
  fin_mientras
  escribir('Media maxima =', MAX)
fin
```

4.18. Detección de entradas numéricas —enteros— erróneas.

Análisis

Esta subrutina es una aplicación sencilla de «interruptor». Se sitúa el valor inicial del interruptor (SW = 0) antes de recibir la entrada de datos.
La detección de números no enteros se realizará con una estructura repetitiva **mientras**, que se realizará si SW = 0. La instrucción que detecta si un número leído desde el dispositivo de entradas es entero:

```
leer(N)
```

realizará la comparación de N y parte entera de N:

- si son iguales, N es entero,
- si son diferentes, N no es entero.

Un método para calcular la parte entera es utilizar la función estándar **ent (int)** existente en muchos lenguajes de programación.

Pseudocódigo

```
algoritmo Error
var
  entero:SW
  real:N
inicio
  SW ← 0
  mientras SW = 0 hacer
    leer('dato de entrada', N)
    si N <> ent(N) entonces
      escribir('Dato no valido')
      escribir('Ejecute nuevamente')
    si_no
      escribir('Correcto', N, 'es entero')
      SW ← 1
    fin_si
  fin_mientras
fin
```

4.19. Calcular el factorial de un número dado (otro nuevo método).

Análisis

El factorial de un número N (N!) es el conjunto de productos sucesivos siguientes:

$N! = N * (N-1) * (N-2) * (N-3) * ... * 3 * 2 * 1$

Los factoriales de los primeros números son:

1! = 1
2! = 2 * 1 = 2 * 1!
3! = 3 * 2 * 1 = 3 * 2!
4! = 4 * 3 * 2 * 1 = 4 * 3!
.
.
.
$N! = N * (N-1) * (N-2) * ... * 2 * 1 = N * (N-1)!$

Los cálculos anteriores significan que el factorial de un número se obtiene con el producto del número N por el factorial de (N – 1)!
Como comienzan los productos en 1, un sistema de cálculo puede ser asignar a la variable *factorial* el valor 1. Se necesita otra variable I que tome los valores sucesivos de 1 a N para poder ir efectuando los productos sucesivos.
Dado que en los números negativos no se puede definir el factorial, se deberá incluir en el algoritmo una condición para verificación de error, caso de que se introduzcan números negativos desde el terminal de entrada (N < 0).
La solución del problema se realiza por dos métodos:

1. Con la estructura **repetir (repeat)**.
2. Con la estructura **desde (for)**.

Pseudocódigo

Método 1 (estructura **repetir**):

```
algoritmo FACTORIAL
var
  entero: I, N
  real: factorial
inicio
  repetir
    leer(N)
  hasta_que N > 0
  factorial ← 1
  I ← 1
  repetir
    factorial ← factorial * I
    I ← I + 1
  hasta_que I = N + 1
  escribir(factorial)
fin
```

Pseudocódigo

```
algoritmo Jornal
var
  cadena: Dia, Turno
  real: HT, Jornal
inicio
  leer(HT, Dia, Turno)
 -si Dia    'Domingo' entonces
    si Turno = 'diurno' entonces
      Jornal ← 500 * HT
    si_no
      Jornal ← 800 * HT
    fin_si
  si_no
    si Turno = 'diurno' entonces
      Jornal ← 700 * HT
    si_no
      Jornal ← 1100 * HT
    fin_si
  fin_si
  escribir(Jornal)
fin
```

4.23. Construir un algoritmo que escriba los nombres de los días de la semana, en función de la entrada correspondiente a la variable DIA.

Análisis

El método a seguir consistirá en clasificar cada día de la semana con un número de orden:

```
1. LUNES
2. MARTES
3. MIERCOLES
4. JUEVES
5. VIERNES
6. SABADO
7. DOMINGO
```

Si Dia > 7 o < 1 Error de entrada: Rango (1 a 7).
Si el lenguaje de programación soporta sólo la estructura **si-entonces-si_no (if-then-else)**, se codifica con el método 1; caso de soportar la estructura **segun_sea (case)**, la codificación será el método 2.

Pseudocódigo

Método 1:

```
algoritmo Dias_semana1
var
  Dia: entero
```

```
inicio
  leer(Dia)
  si Dia = 1 entonces
    escribir('LUNES')
  si_no
    si Dia = 2 entonces
      escribir('MARTES')
    si_no
      si Dia = 3 entonces
        escribir('MIERCOLES')
      si_no
        si Dia = 4 entonces
          escribir('JUEVES')
        si_no
          si Dia = 5 entonces
            escribir('VIERNES')
          si_no
            si Dia = 6 entonces
              escribir('SABADO')
            si_no
              si Dia = 7 entonces
                escribir('DOMINGO')
              si_no
                escribir('error')
                escribir('rango 1-7')
              fin_si
            fin_si
          fin_si
        fin_si
      fin_si
    fin_si
  fin_si
fin
```

Método 2:

```
algoritmo Dias_semana2
var
  Dia: entero
inicio
  leer(Dia)
  segun_sea Dia hacer
    1: escribir('LUNES')
    2: escribir('MARTES')
    3: escribir('MIERCOLES')
    4: escribir('JUEVES')
    5: escribir('VIERNES')
    6: escribir('SABADO')
    7: escribir('DOMINGO')
    si_no escribir('error de entrada, rango 1-7')
  fin_segun
fin
```

4.24. Realizar un algoritmo que escriba los *N* primeros números de la serie de Fibonacci.

NOTA: La serie de Fibonacci es 1, 1, 2, 3, 4, 8, 13, ..., y se genera de acuerdo a la ley siguiente:

Fibonacci(1) = 1
Fibonacci(2) = 1
Fibonacci(3) = 2 = Fibonacci(2) + Fibonacci(1)
Fibonacci(4) = 3 = Fibonacci(2) + Fibonacci(3)
.
.
Fibonacci(N para N > 1) = Fibonacci(N – 1) + Fibonacci(N – 2)

NOTA: La realización del pseudocódigo se deja como tarea al alumno.

EJERCICIOS

4.1. Determinar la media de una lista indefinida de números positivos, terminados con un número negativo.
4.2. Dado el nombre o número de un mes y si el año es o no bisiesto, deducir el número de días del mes.
4.3. Sumar los números enteros de 1 a 100 mediante: a) estructura `repetir`; b) estructura `mientras`; c) estructura `desde`.
4.4. Determinar la media de una lista de números positivos terminada con un número no positivo después del último número válido.
4.5. Imprimir todos los números primos entre 2 y 1.000 inclusive.
4.6. Se desea leer las calificaciones de una clase de informática y contar el número total de aprobados (5 o mayor que 5).
4.7. Leer las notas de una clase de Informática y deducir todas aquellas que sean `NOTABLES (>= 7 y < 9)`.
4.8. Leer 100 números. Determinar la media de los números positivos y la media de los números negativos.
4.9. Un comercio dispone de dos tipos de artículos en fichas correspondientes a diversas sucursales con los siguientes campos:

- código del artículo `A` o `B`,
- precio unitario del artículo,
- número de artículos.

La última ficha del archivo de artículos tiene un código de artículo, una letra `X`. Se pide:

- el número de artículos existentes de cada categoría,
- el importe total de los artículos de cada categoría.

4.10. Una estación climática proporciona un par de temperaturas diarias (máxima, mínima) (no es posible que alguna o ambas temperaturas sea 9 grados). La pareja fin de temperaturas es 0,0. Se pide determinar el número de días, cuyas temperaturas se han proporcionado, las medias máxima y mínima, el número de errores —temperaturas de 0°— y el porcentaje que representaban.
4.11. Calcular:

$$E(x) = 1 + x + \frac{x^2}{2!} + \ldots + \frac{n^n}{n!}$$

a) Para *N* que es un entero leído por teclado.
b) Hasta que *N* sea tal que $x^n/n < E$ (por ejemplo, $E = 10^{-4}$).

4.12. Calcular el enésimo término de la serie de Fibonacci definida por:

$$A1 = 1 \quad A2 = 1 \quad A3 = 1 + 2 = A1 + A2 \quad A_n = A_{n-1} + A_{n-2} \; (n >= 3)$$

4.13. Se pretende leer todos los empleados de una empresa —situados en un archivo `EMPRESA`— y a la terminación de la lectura del archivo se debe visualizar un mensaje «existen trabajadores mayores de 65 años en un número de ...» y el número de `t` trabajadores mayores de 65 años.

4.14. Un capital `C` está situado a un tipo de interés `R`, ¿al término de cuántos años se doblará?

4.15. Los empleados de una fábrica trabajan en dos turnos, diurno y nocturno. Se desea calcular el jornal diario de acuerdo con los siguientes puntos:

- la tarifa de las horas diurnas es de 500 pesetas,
- la tarifa de las horas nocturnas es de 800 pesetas,
- caso de ser domingo, la tarifa se incrementará en 200 pesetas el turno diurno y 300 pesetas el turno nocturno.

4.16. Averiguar si dados dos números leídos del teclado, uno es divisor de otro.

4.17. Se introduce la hora del día en horas, minutos y segundos. Se desea escribir la hora correspondiente al siguiente segundo.

4.18. Se desea conocer una serie de datos de una empresa con 50 empleados: a) ¿Cuántos empleados ganan más de 300.000 pesetas al mes (salarios altos); b) entre 100.000 y 300.000 pesetas (salarios medios), y c) menos de 100.000 pesetas (salarios bajos y empleados a tiempo parcial)?

4.19. Imprimir una tabla de multiplicar como

```
      1   2   3   4   ...  15
**    **  **  **  **  ...  **
1*    1   2   3   4   ...  15
2*    2   4   6   8   ...  30
3*    3   6   9   12  ...  45
4*    4   8   12  16  ...  60
.
.
.
15*   15  30  45  60  ...  225
```

4.20. Dado un entero positivo `n` (> 1), comprobar si es primo o compuesto.

REFERENCIAS BIBLIOGRÁFICAS

(1) DIJKSTRA, E.W.: «Goto Statement Considered Harmful», *Communications of the ACM*, vol. 11, núm. 3, marzo 1968, págs. 147-148, 538, 541.

(2) KNUTH, D.E.: «Structured Programming with goto Statements», *Computing Surveys*, vol. 6, núm. 4, diciembre 1974, págs. 261-298.

CAPÍTULO 5

SUBPROGRAMAS (SUBALGORITMOS): PROCEDIMIENTOS Y FUNCIONES

CONTENIDO

5.1. Introducción a los subalgoritmos o subprogramas.
5.2. Funciones.
5.3. Procedimientos (subrutinas).
5.4. Ámbito: variables locales y globales.
5.5. Comunicación con subprogramas: paso de parámetros.
5.6. Funciones y procedimientos como parámetros.
5.7. Los efectos laterales.
5.8. Recursión (recursividad).

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

La resolución de problemas complejos se facilita considerablemente si se dividen en problemas más pequeños (subproblemas). *La solución de estos subproblemas se realiza con subalgoritmos.* El uso de subalgoritmos permite al programador desarrollar soluciones de problemas complejos utilizando el método descendente introducido en los capítulos anteriores. *Los subalgoritmos (subprogramas) pueden ser de dos tipos: funciones y procedimientos o subrutinas.* Los subalgoritmos son unidades de programa o módulos que están diseñados para ejecutar alguna tarea específica. Estas funciones y procedimientos se escriben solamente una vez, pero pueden ser referenciados en diferentes puntos de un programa, de modo que se puede evitar la duplicación innecesaria del código.

El uso de unidades de programas en el estilo de programación modular son independientes; el programador puede escribir cada módulo y verificarlo sin preocuparse de los detalles de otros módulos. Esto facilita considerablemente la localización de un error cuando se produce. Los programas desarrollados de este modo son normalmente también más fáciles de comprender, ya que la estructura de cada unidad de programa

puede ser estudiada independientemente de las otras unidades de programa.

En este capítulo se describen las *funciones y procedimientos,* junto con los conceptos de *variables locales y globales,* así como parámetros. Se introduce también el concepto de *recursividad* como una nueva herramienta de resolución de problemas.

5.1. INTRODUCCIÓN A LOS SUBALGORITMOS O SUBPROGRAMAS

Un método ya citado para solucionar un problema complejo es dividirlo en subproblemas —problemas más sencillos— y a continuación dividir estos subproblemas en otros más simples, hasta que los problemas más pequeños sean fáciles de resolver. Esta técnica de dividir el problema principal en subproblemas se suele denominar «*divide y vencerás*» (*divide and conquer*). Este método de diseñar la solución de un problema principal obteniendo las soluciones de sus subproblemas se conoce como *diseño descendente* (*top-dow design*). Se denomina descendente, ya que se inicia en la parte superior con un problema general y el diseño específico de las soluciones de los subproblemas. Normalmente las partes en que se divide un programa deben poder desarrollarse independientemente entre sí.

Las soluciones de un diseño descendente pueden implementarse fácilmente en lenguajes de programas de alto nivel, como C/C++, Pascal o FORTRAN. Estas partes independientes se denominan *subprogramas* o *subalgoritmos* si se emplean desde el concepto algorítmico.

La correspondencia entre el diseño descendente y la solución por computadora en términos de programa principal y sus subprogramas se analizará a lo largo de este capítulo.

Consideremos el problema del cálculo de la superficie (área) de un rectángulo. Este problema se puede dividir en tres subproblemas:

Subproblema 1: entrada de datos de altura y base.
Subproblema 2: cálculo de la superficie.
Subproblema 3: salida de resultados.

El algoritmo correspondiente, que resuelve los tres subproblemas es:

```
leer(datos (altura, base))        //entrada de datos
area ← base * altura              //calculo de la superficie
escribir(base, altura, area)      //salida de resultados
```

El método descendente se muestra en la Figura 5.1.

El problema principal se soluciona por el correspondiente **programa** o **algoritmo principal** —también denominado *controlador* o *conductor* (*driver*)— y la solución de los subproblemas mediante **subprogramas**, conocidos como **procedimientos** (**subrutinas**) o **funciones**. Los subprogramas, cuando se tratan en lenguaje algorítmico, se denominan también *subalgoritmos*.

Un subprograma puede realizar las mismas acciones que un programa: 1) aceptar datos, 2) realizar algunos cálculos, y 3) devolver resultados. Un subprograma, sin embargo, se utiliza por el programa para un propósito específico. El subprograma recibe datos desde el programa y le devuelve resultados. Haciendo un símil con una oficina, el problema es como el jefe que da instrucciones a sus subordinados —subprogramas—; cuando la tarea se termina, el subordinado devuelve sus

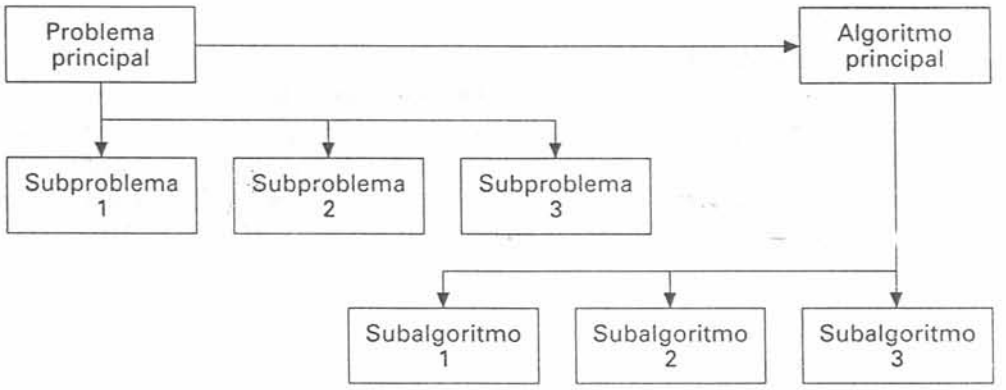


Figura 5.1. Diseño descendente.

resultados al jefe. Se dice que el programa principal *llama* o *invoca* al subprograma. El subprograma ejecuta una tarea, a continuación *devuelve* el control al programa. Esto puede suceder en diferentes lugares del programa. Cada vez que el subprograma es llamado, el control retorna al lugar desde donde fue hecha la llamada (Figura 5.2). Un subprograma puede llamar a su vez a sus propios subprogramas (Figura 5.3). Existen —como ya se ha comentado— dos tipos importantes de subprogramas: *funciones* y *procedimientos* o *subrutinas*.

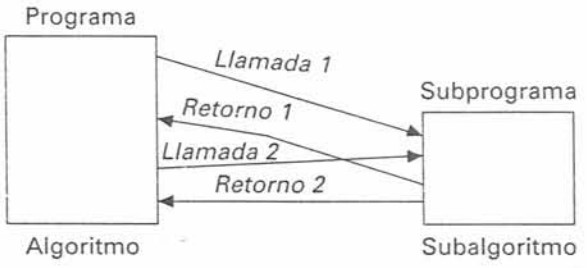


Figura 5.2. Un programa con un subprograma: función y procedimiento o subrutina, según la terminología específica del lenguaje: subrutina en BASIC y FORTRAN, función en C, procedimiento función en Pascal.

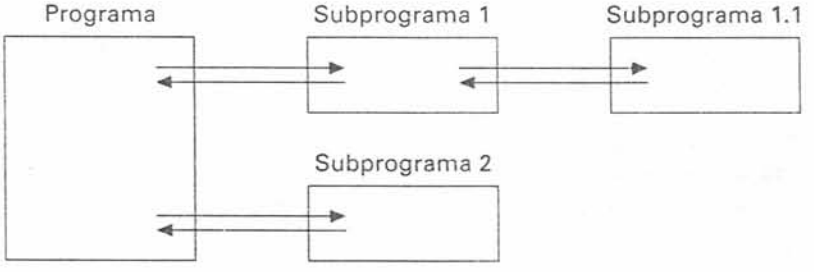


Figura 5.3. Un programa con diferentes niveles de subprogramas.

5.2. FUNCIONES

Matemáticamente una función es una operación que toma uno o más valores llamados *argumentos* y produce un valor denominado *resultado* —valor de la función para los argumentos dados—. Todos los lenguajes de programación tienen funciones incorporadas o intrínsecas —en el Capítulo 2 se vieron algunos ejemplos— y funciones definidas por el usuario. Así, por ejemplo:

$$f(x) = \frac{x}{1+x^2}$$

donde f es el nombre de la función y x es el argumento. Obsérvese que ningún valor específico se asocia con x; es un *parámetro formal* utilizado en la definición de la función. Para evaluar f debemos darle un *valor real* o *actual* a x; con este valor se puede calcular el resultado. Con $x = 3$ se obtiene el valor 0.3 que se expresa escribiendo

$$f(3) = 0.3$$

$$f(3) = \frac{3}{1+9} = \frac{3}{10} = 0.3$$

Una función puede tener varios argumentos. Por consiguiente,

$$f(x, y) = \frac{x - y}{\sqrt{x} - \sqrt{y}}$$

es una función con dos argumentos. Sin embargo, solamente un único valor se asocia con la función para cualquier par de valores dados a los argumentos.

Cada lenguaje de programación tiene sus propias funciones incorporadas, que se utilizan escribiendo sus nombres con los argumentos adecuados en expresiones tales como

```
raiz2(A + cos(x))
```

Cuando la expresión se evalúa, el valor de x se da primero al subprograma (función) coseno y se calcula cos(x). El valor de A + cos(x) se utiliza entonces como argumento de la función `raiz2` (raíz cuadrada), que evalúa el resultado final.

Cada función se evoca utilizando su nombre en una expresión con los argumentos actuales o reales encerrados entre paréntesis.

Las funciones incorporadas al sistema se denominan *funciones internas* o *intrínsecas* y las funciones definidas por el usuario, *funciones externas*. Cuando las funciones estándares o internas no permiten realizar el tipo de cálculo deseado es necesario recurrir a las funciones externas, que pueden ser definidas por el usuario mediante una *declaración de función*.

A una función no se le llama explícitamente, sino que se le invoca o referencia mediante un nombre y una lista de parámetros actuales.

El algoritmo o programa llama o invoca a la función con el nombre de esta última en una

expresión seguida de una lista de argumentos que deben coincidir en cantidad, tipo y orden con los de la función que fue definida. La función devuelve un único valor.

Las funciones son diseñadas para realizar tareas específicas: tomar una lista de valores —llamados *argumentos* (parámetros de entrada: **E**)— y devolver un único valor.

5.2.1. Declaración de funciones

La declaración de una función requiere una serie de pasos que la definen. Una función como tal subalgoritmo o subprograma tiene una constitución similar a los algoritmos, por consiguiente, constará de una cabecera que comenzará con el tipo del valor devuelto por la función, seguido de la palabra **funcion** y del nombre y argumentos de dicha función. A continuación irá el *cuerpo* de la función, que será una serie de acciones o instrucciones cuya ejecución hará que se asigne un valor al nombre de la función. Esto determina el valor particular del resultado que ha de devolverse al programa llamador.

La declaración de la función será:

```
<tipo_de_resultado> funcion <nombre_fun> (lista de parametros)
[declaraciones locales]
inicio
  <acciones>      //cuerpo de la funcion
  devolver (<expresion>)
fin_funcion
```

lista de parametros: lista de *parámetros formales* o *argumentos*, con uno o más argumentos de la siguiente forma:

```
({E|S|E/S} tipo_de_datoA: parametro 1[, parametro 2]...
 {E|S|E/S} tipo_de_datoB: parametro x[, parametro y]...)
  E: Entrada; S: Salida; E/S: Entrada/Salida
```

nombre_func: nombre asociado con la función, que será un nombre de identificador válido

<acciones> instrucciones que constituyen la definición de la función y que deben contener una única instrucción: **devolver** (<expresion>)

tipo_de_resultado tipo del resultado que devuelve la función

Por ejemplo, la función:

$$f(x,y)=\frac{x}{1+x^2}$$

se definirá como:

```
real funcion F(x)
inicio
  devolver (x/(1 + x * x))
fin_funcion
```

Otro ejemplo puede ser la definición de la función trigonométrica, cuyo valor es:

$$\tan(x) = \frac{\text{sen}(x)}{\cos(x)}$$

donde sen(x) y cos(x) son las funciones seno y coseno —normalmente funciones internas—. La declaración de la función es:

```
real funcion tan (E real:x)        // x, parametro de Entrada
//funcion tan igual a sen (x)/cos(x), angulo x en radianes
inicio
  devolver (sen(x)/cos(x))
fin_funcion
```

Observe que se incluye un comentario para describir la función. Es buena práctica incluir documentación que describa brevemente lo que hace la función, lo que representan sus parámetros o cualquier otra información que explique la definición de la función. *En aquellos lenguajes de programación* —como Pascal— *que exigen sección de declaraciones, éstas se situarán al principio del programa.*

Para que las acciones descritas en un subprograma función sean ejecutadas, se necesita que éste sea invocado desde un programa principal o desde otros subprogramas a fin de proporcionarle los argumentos de entrada necesarios para realizar esas acciones.

Los argumentos de la declaración de la función se denominan *parámetros formales, ficticios* o *mudos* (*dummy*); son nombres de variables, de otras funciones o procedimientos y sólo se utilizan dentro del cuerpo de la función. Los argumentos utilizados en llamada a la función se denominan *parámetros actuales*, que a su vez pueden ser constantes, variables, expresiones, valores de funciones o nombres de funciones o procedimientos.

5.2.2. Invocación a las funciones

Una función puede ser llamada de la forma siguiente:

```
nombre_funcion (lista de parametros actuales)
```

`nombre_funcion`	función que llama
`lista de parametros actuales`	constantes, variables, expresiones, valores de funciones, nombres de funciones o procedimientos

Cada vez que se llama a una función desde el algoritmo principal se establece automáticamente una correspondencia entre los parámetros formales y los parámetros actuales. Debe haber exactamente el mismo número de parámetros actuales que de parámetros formales en la declaración de la función y se presupone una correspondencia uno a uno de izquierda a derecha entre los parámetros formales y los actuales.

Una llamada a la función implica los siguientes pasos:

1. A cada parámetro formal se le asigna el valor real de su correspondiente parámetro actual.
2. Se ejecuta el cuerpo de acciones de la función.
3. Se devuelve el valor de la función al nombre de la función y se retorna al punto de llamada.

Ejemplo 5.1

Definición de la función: y = x^n (potencia n de x).

```
real funcion potencia(E real:x;E entero:n)
var
  entero: i, y
inicio
  y ← 1
  desde i ← 1 hasta abs(n) hacer
    y ← y * x
  fin_desde
  si n < 0 entonces
    y ← 1/y
  fin_si
  devolver (y)
fin_funcion
```

abs(n) es la función valor absoluto de n a fin de considerar exponentes positivos o negativos.

Invocación de la función:

```
z ← potencia (2.5, -3)
```

parámetros actuales

Transferencia de información:

```
x = 2.5     n = -3

z = 0.064
```

Ejemplo 5.2

Función potencia para el cálculo de N *elevado a* A. *El número* N *deberá ser positivo, aunque podrá tener parte fraccionaria,* A *es un real.*

```
algoritmo Elevar_a_potencia
var
  real : a,n

inicio
escribir('Deme numero positivo ')
leer(n)
escribir('Deme exponente ')
leer(a)
escribir('N elevado a =', potencia(n, a))
fin

real funcion potencia (E real: n, a)
inicio
  devolver(exp(a * ln(n)))
fin_funcion
```

Ejemplo 5.3

Diseñar un algoritmo que contenga un subprograma de cálculo del factorial de un número y una llamada al mismo.

Como ya es conocido por el lector el algoritmo factorial, lo indicaremos expresamente:

```
entero funcion FACTORIAL(E entero:n)
var
  entero: i,f
  //advertencia, segun el resultado, f puede ser real
inicio
  f ← 1
  desde i ← 1 hasta n hacer
    f ← f * i
  fin_desde
  devolver (f)
fin_funcion
```

y el algoritmo que contiene un subprograma de cálculo del factorial de un número y una llamada al mismo:

```
algoritmo Funcion_factorial
var entero: x, y, numero

inicio
  escribir ('Deme un numero entero y positivo')
  leer(numero)
  x ← factorial(numero)
  y ← factorial(5)
  escribir(x, y)
fin

entero funcion factorial (E entero: n)
var entero: i, f
inicio
  f ← 1
  desde i ← 1 hasta n hacer
    f ← f * i
  fin_desde
  devolver(f)
fin_funcion
```

En este caso los parámetros actuales son: una variable (`numero`) y una constante (`5`).

Ejemplo 5.4

Realizar el diseño de la función $y = x^3$ *(cálculo del cubo de un número).*

```
algoritmo Prueba
var
  entero: N
inicio //Programa principal
   N ← cubo(2)
   escribir ('2 al cubo es', N)
   escribir ('3 al cubo es', cubo(3))
fin

entero funcion cubo(E entero: x)
inicio
  devolver(x * x * x)
fin_funcion
```

La salida del algoritmo sería:

```
2 al cubo es 8
3 al cubo es 27
```

Las funciones pueden tener muchos argumentos, pero solamente un resultado: *el valor de la función*. Esto limita su uso, aunque se encuentran con frecuencia en cálculos científicos. Un concepto más potente es el proporcionado por el subprograma procedimiento que se examina en el siguiente apartado.

Ejercicio 5.5

Algoritmo que contiene y utiliza unas funciones (seno y coseno) a las que les podemos pasar el ángulo en grados.

```
algoritmo Sen_cos_en_grados
var real : g

inicio
  escribir('Deme angulo en grados')
  leer(g)
  escribir(seno(g))
  escribir(coseno(g))
fin

real funcion coseno (E real : g)
inicio
  devolver(cos(g * 2 * 3.141592/360))
fin_funcion

real funcion seno (E real g)
inicio
  devolver( sen(g * 2 * 3.141592/360))
fin_funcion
```

Ejercicio 5.6

Algoritmo que simplifique un quebrado, dividiendo numerador y denominador por su máximo común divisor.

```
algoritmo Simplificar_quebrado
var
  entero : n, d

inicio
  escribir('Deme numerador')
  leer(n)
  escribir('Deme denominador')
  leer(d)
  escribir(n, '/', d, '=', n div mcd(n, d),'/', d div mcd(n, d))
  fin
  entero funcion mcd (E entero: n, d)
  var
    entero : r
  inicio
    r ← n mod d
```

```
    mientras r <> 0 hacer
      n ← d
      d ← r
      r ← n MOD d
    fin_mientras
  devolver(d)
fin_funcion
```

Ejercicio 5.7

Supuesto que nuestro compilador no tiene la función seno. Podríamos calcular el seno de x mediante la siguiente serie:

$$\text{sen}(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots \text{ (hasta 17 término}$$

x (ángulo en radianes).

El programa nos tiene que permitir el cálculo del seno de ángulos en grados, mediante el diseño de una función `seno(x)`, que utilizará, a su vez, las funciones `potencia(x,n)` y `factorial(n)`, que también deberán ser implementadas en el algoritmo.

Se terminará cuando respondamos `N` (**no**) a la petición de otro ángulo.

```
algoritmo Calcular_seno
var real      : gr
    caracter : resp

inicio
  repetir
    escribir('Deme angulo en grados')
    leer(gr)
    escribir('Seno(', gr, ')=', seno(gr))
    escribir('¿Otro angulo?')
    leer(resp)
  hasta_que resp = 'N'
fin

real funcion factorial (E entero:n)
var
  real   : f
  entero : i
inicio
    f ← 1
    desde i ← 1 hasta n hacer
      f ← f * i
    fin_desde
    devolver(f)
fin_funcion

real funcion potencia (E real: x;  E entero:n)
var real : pot
  entero : i
inicio
    pot ← 1
    desde i ← 1 hasta n hacer
      pot ← pot * x
    fin_desde
    devolver(pot)
fin_funcion
```

```
real funcion seno (E real:gr)
var real : x, s
  entero : i, n
inicio
  x ← gr * 3.141592 / 180
  s ← x
  desde i ← 2 hasta 17 hacer
    n ← 2 * i - 1
    si i mod 2 <> 0 entonces
      s ← s - potencia(x, n) / factorial(n)
    si_no
      s ← s + potencia(x, n) / factorial(n)
    fin_si
  fin_desde
  devolver(s)
fin_funcion
```

5.3. PROCEDIMIENTOS (subrutinas)

Aunque las funciones son herramientas de programación muy útiles para la resolución de problemas, su alcance está muy limitado. Con frecuencia, se requieren subprogramas que calculen varios resultados en vez de uno solo, o que realicen la ordenación de una serie de números, etc. En estas situaciones la *función* no es apropiada y se necesita disponer del otro tipo de subprograma: el *procedimiento* o *subrutina*.

Un *procedimiento* o *subrutina*[1] es un subprograma que ejecuta un proceso específico. Ningún valor está asociado con el nombre del procedimiento; por consiguiente, no puede ocurrir en una expresión. Un procedimiento se llama escribiendo su nombre, por ejemplo, SORT, para indicar que un procedimiento denominado SORT se va a usar. Cuando se invoca el procedimiento, los pasos que lo definen se ejecutan y a continuación se devuelve el control al programa que le llamó.

Procedimiento *versus* función

Los procedimientos y funciones son subprogramas cuyo diseño y misión son similares, sin embargo, existen unas diferencias esenciales entre ellos:

1. Un procedimiento es llamado desde el algoritmo o programa principal mediante su nombre y una lista de parámetros actuales, o bien con la instrucción `llamar_a (call)`. Al llamar al procedimiento se detiene momentáneamente el programa que se estuviera realizando y el control pasa al procedimiento llamado. Después de que las acciones del procedimiento se ejecutan, se regresa a la acción inmediatamente siguiente a la que se llamó.
2. Las funciones devuelven un valor, los procedimientos pueden devolver 0,1 o n valores y en forma de lista de parámetros.
3. El procedimiento se declara igual que la función, pero su nombre no está asociado a ninguno de los resultados que obtiene.

[1] En FORTRAN, la subrutina representa el mismo concepto que procedimiento. Sin embargo, en BASIC las subrutinas no podrán pasar parámetros. En el resto de los lenguajes el término general para definir un subprograma es procedimiento o simplemente subprograma.

La *declaración de un procedimiento* es similar a la de funciones.

```
procedimiento nombre [(lista de parametros formales)]
...
  <acciones>
fin_procedimiento
```

Los parámetros formales tienen el mismo significado que en las funciones; los parámetros variables —en aquellos lenguajes que los soportan, por ejemplo, Pascal— están precedidos cada uno de ellos por la palabra **var** para designar que ellos obtendrán resultados del procedimiento en lugar de los valores actuales asociados a ellos.

El procedimiento se llama mediante la instrucción

```
[llamar_a] nombre [(lista de parametros actuales)]
```

La palabra **llamar_a (call)** es opcional y su existencia depende del lenguaje de programación. El ejemplo siguiente ilustra la definición y uso de un procedimiento para realizar la división de dos números y obtener el cociente y el resto.

Variables enteras: DIVIDENDO
DIVISOR
COCIENTE

Procedimiento

```
procedimiento division (E entero:Dividendo,Divisor; S entero: Cociente, Resto)

inicio
  Cociente ← Dividendo div Divisor
  Resto ← Dividendo - Cociente * Divisor
fin_procedimiento
```

Algoritmo principal

```
algoritmo Aritmetica
var
  entero: M, N, P, Q, S, T
inicio
  leer(M, N)
  llamar_a division (m, N, P, Q)
  escribir(P, Q)
  llamar_a division (M * N - 4, N + 1,S, T)
  escribir(S, T)
fin
```

5.3.1. Sustitución de argumentos/parámetros

La lista de parámetros, bien *formales* en el procedimiento o *actuales* (reales) en la llamada, se conoce como *lista de parámetros*, pueden ser de Entrada (**E**), Salida (**S**) o Entrada/Salida (**E/S**).

```
procedimiento demo
.
.
.
fin_procedimiento
```

o bien:

```
procedimiento demo (lista de parametros formales)
```

y la instrucción llamadora

```
llamar_a demo (lista de parametros actuales)
```

Cuando se llama al procedimiento, cada parámetro formal toma como valor inicial el valor del correspondiente parámetro actual. En el ejemplo siguiente se indican la sustitución de parámetros y el orden correcto.

```
algoritmo Demo
  //definicion del procedimiento
  entero: años
  real: numeros, tasa
inicio
  ...
  llamar_a calculo(numero, años, tasa)
  ...
fin

procedimiento calculo(S real: p1; E entero: p2; E real: p3)
inicio
  p3 ... p1 ... p2 ... p2
fin_procedimiento
```

Las acciones sucesivas a realizar son las siguientes:

1. Los parámetros reales sustituyen a los parámetros formales.
2. El cuerpo de la declaración del procedimiento se sustituye por la llamada del procedimiento.
3. Por último, se ejecutan las acciones escritas por el código resultante.

Ejemplo 5.8

Algoritmo que transforma un número introducido por teclado en notación decimal a romana. El número será entero y positivo y no excederá de 3.000.

Sin utilizar programación modular:

```
algoritmo Romanos
var entero : n,digito,r,j

inicio
    repetir
      escribir('Deme numero')
      leer(n)
```

```
hasta_que (n >= 0) Y (n <= 3000)
r ← n
digito ← r div 1000
r ← r mod 1000
desde j ← 1 hasta digito hacer
  escribir('M')
fin_desde
digito ← r div 100
r ← r mod 100
si digito = 9 entonces
  escribir('C', 'M')
si_no
si digito > 4 entonces
  escribir('D')
  desde j ← 1 hasta digito - 5 hacer
    escribir('C')
  fin_desde
si_no
  si digito = 4 entonces
    escribir('C','D')
  si_no
    desde j ← 1 hasta digito hacer
      escribir('C')
    fin_desde
  fin_si
fin_si
fin_si
digito ← r div 10
r ← r mod 10
si digito = 9 entonces
  escribir('X', 'C')
si_no
  si digito > 4 entonces
    escribir('L')
    desde j ← 1 hasta digito - 5 hacer
      escribir('X')
      fin_desde
  si_no
    si digito = 4 entonces
      escribir('X','L')
    si_no
      desde j ← 1 hasta digito hacer
        escribir('X')
      fin_desde
    fin_si
  fin_si
fin_si
digito ← r
si digito = 9 entonces
  escribir('I', 'X')
si_no
  si digito > 4 entonces
    escribir('V')
  desde j ← 1 hasta digito - 5 hacer
    escribir('I')
  fin_desde
```

```
    si_no
      si digito = 4 entonces
        escribir('I','V')
      si_no
        desde j ← 1 hasta digito hacer
          escribir('I')
        fin_desde
        fin_si
      fin_si
    fin_si
fin
```

Mediante programación modular:

```
algoritmo Romanos
var n,r,digito: entero

inicio
  repetir
  escribir('Deme numero')
  leer(n)
  hasta_que (n >= 0) Y (n <= 3000)
  r ← n
  digito ← r div 1000
  r ← r mod 1000
  calccifrarom(digito, 'M', ' ',' ')
  digito ← r div 100
  r ← r mod 100
  calccifrarom(digito, 'C', 'D', 'M')
  digito ← r div 10
  r ← r mod 10
  calccifrarom(digito, 'X', 'L', 'C')
  digito ← r
  calccifrarom(digito, 'I', 'V', 'X')
fin

procedimiento calccifrarom(E entero:digito; E caracter:v1,v2,v3)
var entero: j
inicio
  si digito = 9 entonces
    escribir( v1, v3)
  si_no
    si digito > 4 entonces
      escribir(v2)
      desde j ← 1 hasta digito - 5 hacer
        escribir(v1)
      fin_desde
    si_no
    si digito = 4 entonces
      escribir(v1, v2)
    si_no
      desde j ← 1 hasta digito hacer
        escribir(v1)
      fin_desde
    fin_si
    fin_si
  fin_si
fin_procedimiento
```

5.4. ÁMBITO: VARIABLES LOCALES Y GLOBALES

Las variables utilizadas en los programas principales y subprogramas se clasifican en dos tipos:

- *variables locales,*
- *variables globales.*

Una *variable local* es aquella que está declarada y definida dentro de un subprograma, en el sentido de que está dentro de ese subprograma y es distinta de las variables con el mismo nombre declaradas en cualquier parte del programa principal. *El significado de una variable se confina a! procedimiento en el que está declarada.* Cuando otro subprograma utiliza el mismo nombre se refiere a una posición diferente en memoria. Se dice que tales variables son locales al subprograma en el que están declaradas.

Una *variable global* es aquella que está declarada para el programa o algoritmo principal, del que dependen todos los subprogramas.

La parte del programa/algoritmo en que una variable se define se conoce como *ámbito* (*scope*, en inglés).

El uso de variables locales tiene muchas ventajas. En particular, hace a los subprogramas independientes, con la comunicación entre el programa principal y los subprogramas manipulados estructuralmente a través de la lista de parámetros. Para utilizar un procedimiento sólo necesitamos conocer lo que hace y no tenemos que estar preocupados por su diseño, es decir, cómo están programados.

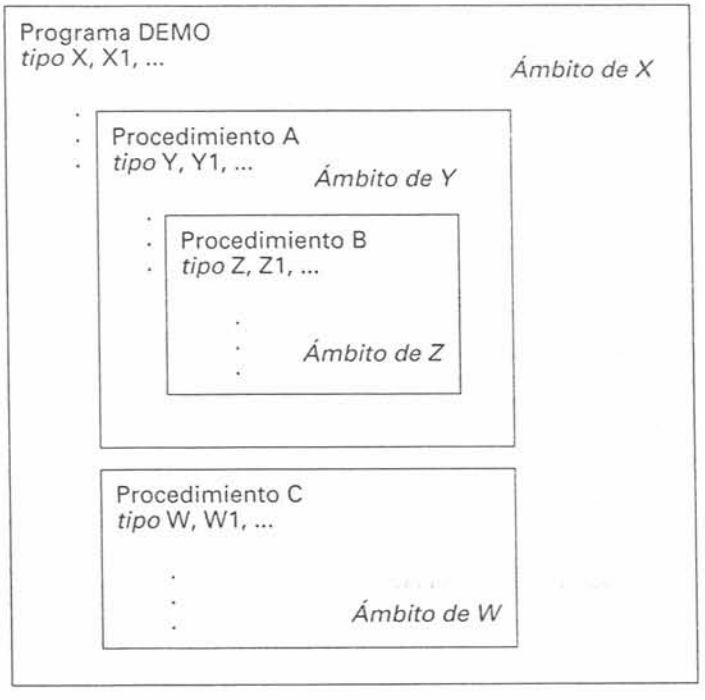


Figura 5.4. Ámbito de identificadores.

Esta característica hace posible dividir grandes proyectos en piezas más pequeñas independientes. Cuando diferentes programadores están implicados, ellos pueden trabajar independientemente.

A pesar del hecho importante de los subprogramas independientes y las variables locales, la mayoría de los lenguajes proporcionan algún método para tratar ambos tipos de variables.

Una variable local a un subprograma no tiene ningún significado en otros subprogramas. Si un subprograma asigna un valor a una de sus variables locales, este valor no es accesible a otros programas, es decir, no pueden utilizar este valor. A veces, también es necesario que una variable tenga el mismo nombre en diferentes subprogramas.

Por el contrario, las variables globales tienen la ventaja de compartir información de diferentes subprogramas sin una correspondiente entrada en la lista de parámetros.

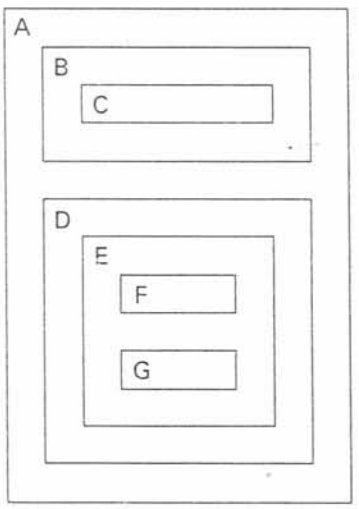
En un programa sencillo con un subprograma, cada variable u otro identificador es o bien local al procedimiento o global al programa completo. Sin embargo, si el programa incluye procedimientos que engloban a otros procedimientos —*procedimientos anidados*—, entonces la noción de global/local es algo más complicado de entender.

El *ámbito* de un identificador (variables, constantes, procedimientos) es la parte del programa donde se conoce el identificador. Si un procedimiento está definido localmente a otro procedimiento, tendrá significado sólo dentro del ámbito de ese procedimiento. A las variables les sucede lo mismo; si están definidas localmente dentro de un procedimiento, su significado o uso se confina a cualquier función o procedimiento que pertenezca a esa definición.

La Figura 5.5 muestra un esquema de un programa con diferentes procedimientos, algunas variables son locales y otras globales. En la citada figura se muestra el ámbito de cada definición.

Los lenguajes que admiten variables locales y globales suelen tener la posibilidad explícita de definir dichas variables como tales en el cuerpo del programa o, lo que es lo mismo, definir su ámbito de actuación, para ello se utilizan las cabeceras de programas y subprogramas, con lo que se definen los ámbitos.

Las variables definidas en un ámbito son accesibles en el mismo, es decir, en todos los procedimientos interiores.



Variables definidas en	*Accesibles desde*
A	A,B,C,D,E,F,G
B	B,C
C	C
D	D,E,F,G
E	E,F,G
F	F
G	G

Figura 5.5. Ámbito de definición de variables.

Ejemplo 5.9

La función (signo) realiza la siguiente tarea: dado un número real x, *si* x *es* 0, *entonces se devuelve un* 0; *si* x *es positivo, se devuelve* 1, *y si* x *es negativo, se devuelve un valor* –1.

La declaración de la función es:

```
entero funcion signo(E real: x)
var entero:s
inicio
  //valores de signo: +1,0,-1
  si x = 0 entonces s ← 0
  si x > 0 entonces s ← 1
  si x < 0 entonces s ← -1
  devolver (s)
fin_funcion
```

Antes de llamar a la función, la variable (S), como se declara dentro del subprograma, es local al subprograma y sólo se conoce dentro del mismo. Veamos ahora un pequeño algoritmo donde se invoque la función.

```
algoritmo SIGNOS
var
  entero: a, b, c
  real: x, y, z
inicio
  x ← 5.4
  a ← signo(x)
  y ← 0
  b ← signo(y)
  z ← 7.8975
  c ← signo(z-9)
  escribir('Las respuestas son', a, ' ', b, ' ', c)
fin
```

Si se ejecuta este algoritmo, se obtienen los siguientes valores:

```
x = 5.4                 x es el parámetro actual de la primera llamada a signo(x)
a = signo(5.4)          a toma el valor 1
y = 0
b = signo(0)            b toma el valor de 0
z = 7.8975
c = signo(7.8975-9)     c toma el valor –1
```

La línea escrita al final será:

```
Las respuestas son 1  0  -1
```

Ejemplo 5.10

```
algoritmo DEMOX
var entero: A, X, Y
inicio
  x ← 5
  A ← 10
  y ← F(x)
  escribir (x, A, y)
fin
```

```
entero funcion F(E entero: N)
var
  entero: X
inicio
  A ← 5
  X ← 12
  devolver(N+A)
fin_funcion
```

A la variable global A se puede acceder desde el algoritmo y desde la función. Sin embargo, X identifica a dos variables distintas: una local al algoritmo y sólo se puede acceder desde él y otra local a la función.

Al ejecutar el algoritmo se obtendrían los siguientes resultados:

```
X = 5
A = 10
Y = F(5)          invocación a la función F(N) se realiza un paso del parámetro actual X al parámetro formal N
  A = 5           se modifica el valor de A en el algoritmo principal por ser A global
  X = 12          no se modifica el valor de X en el algoritmo principal porque X es local
  F = 5 + 5 = 10  se pasa el valor del argumento X(5) a través del parámetro N
Y = 10
```

se escribirá la línea

```
5     5     10
```

ya que X es el valor de la variable local X en el algoritmo; A, el valor de A en la función, ya que se pasa este valor al algoritmo; Y es el valor de la función F(X).

5.5. COMUNICACIÓN CON SUBPROGRAMAS: PASO DE PARÁMETROS

Cuando un programa llama a un subprograma, la información se comunica a través de la lista de parámetros y se establece una correspondencia automática entre los parámetros formales y actuales. *Los parámetros actuales son «sustituidos» o «utilizados» en lugar de los parámetros formales.*

La declaración del subprograma se hace con

```
procedimiento nombre (clase tipo_de_dato: F1
                      clase tipo_de_dato: F2
                      ......................
                      clase tipo_de_dato :Fn)
  .
  .
  .
fin_procedimiento
```

y la llamada al subprograma con

```
llamar_a nombre (A1, A2, ... , An)
```

donde F1, F2, . . . , Fn son los parámetros formales y A1, A2, . . . , An los parámetros actuales o reales.

Las clases de parámetros podrían ser:

```
(E) Entrada
(S) Salida
(E/S) Entrada/Salida
```

Existen dos métodos para establecer la correspondencia de parámetros:

1. *Correspondencia posicional.* La correspondencia se establece aparejando los parámetros reales y formales según su posición en las listas: así, `Fi` se corresponde con `Ai`, donde `i = 1, 2, . . . , n`. Este método tiene algunas desventajas de legibilidad cuando el número de parámetros es grande.
2. *Correspondencia por el nombre explícito*, también llamado *método de paso de parámetros por nombre*. En este método, en las llamadas se indica explícitamente la correspondencia entre los parámetros reales y formales. Este método se utiliza en Ada. Un ejemplo sería:

   ```
   SUB(Y => B, X => 30);
   ```

   que hace corresponder el parámetro actual `B` con el formal `Y`, y el parámetro actual `30` con el formal `X` durante la llamada de `SUB`.

Por lo general, la mayoría de los lenguajes usan exclusivamente la correspondencia posicional y ése será el método empleado en este libro.

Las cantidades de información que pueden pasarse como parámetros son *datos*, *tipos* —en los lenguajes que admiten su declaración— y *subprogramas*.

5.5.1. Paso de parámetros

Existen diferentes métodos para la *transmisión* o *el paso de parámetros a subprogramas*. Es preciso conocer el método adoptado por cada lenguaje, ya que la elección puede afectar a la semántica del lenguaje. Dicho de otro modo, un mismo programa puede producir diferentes resultados bajo diferentes sistemas de paso de parámetros.

Los parámetros pueden ser clasificados como:

entradas: (**E**)	las entradas proporcionan valores desde el programa que llama y que se utilizan dentro de un procedimiento. En los subprogramas función, las entradas son los argumentos en el sentido tradicional;
salidas: (**S**)	las salidas producen los resultados del subprograma; de nuevo si se utiliza el caso una función, éste devuelve un valor calculado por dicha función, mientras que con procedimientos pueden calcularse cero, una o varias salidas;
entradas/salidas: (**E/S**)	un solo parámetro se utiliza para mandar argumentos a un programa y para devolver resultados.

Desgraciadamente, el conocimiento del tipo de parámetros no es suficiente para caracterizar su funcionamiento; por ello, examinaremos los diferentes métodos que se utilizan para pasar o transmitir parámetros.

Los métodos más empleados para realizar el paso de parámetros son:

- *paso por valor* (también conocido por *parámetro valor*),
- *paso por referencia o dirección* (también conocido por *parámetro variable*),
- *paso por nombre*,
- *paso por resultado*.

5.5.2. Paso por valor

El paso por valor se utiliza en muchos lenguajes de programación; por ejemplo, C, Modula-2, Pascal, Algol y Snobol. La razón de su popularidad es la analogía con los argumentos de una función, donde los valores se proporcionan en el orden de cálculo de resultados. Los parámetros se tratan como variables locales y los valores iniciales se proporcionan copiando los valores de los correspondientes argumentos.

Los parámetros formales —locales a la función— reciben como valores iniciales los valores de los parámetros actuales y con ello se ejecutan las acciones descritas en el subprograma.

No se hace diferencia entre un argumento que es variable, constante o expresión, ya que sólo importa el valor del argumento. La Figura 5.6 muestra el mecanismo de paso por valor de un procedimiento con tres parámetros.

El mecanismo de paso se resume así:

Valor primer parámetro: `A = 5`.
Valor segundo parámetro: constante = `18`.
Valor tercer parámetro: expresión `B * 3 + 4 = 25`.

Los valores `5`, `18` y `25` se transforman en los parámetros `X`, `Y`, `Z`, respectivamente, cuando se ejecuta el procedimiento.

Aunque el *paso por valor* es sencillo, tiene una limitación acusada: *no existe ninguna otra conexión con los parámetros actuales*, y entonces los cambios que se produzcan por efecto del subprograma no producen cambios en los argumentos originales y, por consiguiente, no se pueden pasar valores de retorno al punto de llamada: es decir, todos los *parámetros* son sólo de *entrada*. El parámetro actual no puede modificarse por el subprograma. Cualquier cambio realizado en los valores de los parámetros formales durante la ejecución del subprograma se destruye cuando se termina el subprograma.

La llamada por valor no devuelve información al programa que llama.

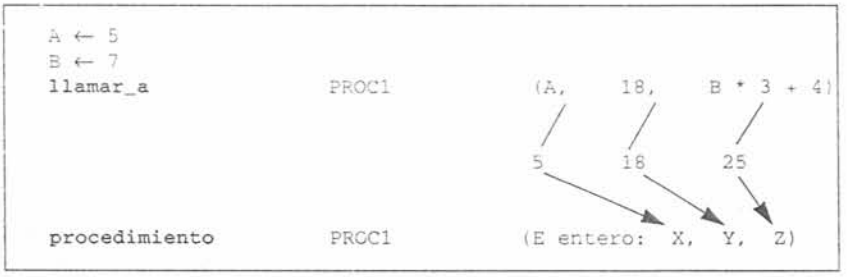


Figura 5.6. Paso por valor.

Existe una variante de la llamada por valor y es la llamada por *valor resultado*. Las variables indicadas por los parámetros formales se inicializan en la llamada al subprograma por valor tras la ejecución del subprograma; los resultados (valores de los parámetros formales) se transfieren a los actuales. Este método se utiliza en algunas versiones de FORTRAN.

5.5.3. Paso por referencia

En numerosas ocasiones se requiere que ciertos parámetros sirvan como parámetros de salida, es decir, se devuelvan los resultados a la unidad o programas que llama. Este método se denomina *paso por referencia* o también de *llamada por dirección o variable*. La unidad que llama pasa a la unidad llamada la dirección del parámetro actual (que está en el ámbito de la unidad llamante). Una referencia al correspondiente parámetro formal se trata como una referencia a la posición de memoria, cuya dirección se ha pasado. Entonces una variable pasada como parámetro real es compartida, es decir, se puede modificar directamente por el subprograma.

Este método existe en FORTRAN, COBOL, Modula-2, Pascal, PL/1 y Algol 68. La característica de este método se debe a su simplicidad y su analogía directa con la idea de que las variables tienen una posición de memoria asignada desde la cual se pueden obtener o actualizar sus valores.

El área de almacenamiento (direcciones de memoria) se utiliza para pasar información de entrada y/o salida; en ambas direcciones.

En este método los parámetros son de entrada/salida y los parámetros se denominan *parámetros variables*.

Los parámetros valor y parámetros variable se suelen definir en la cabecera del subprograma. En el caso de lenguajes como Pascal, los parámetros variables deben ir precedidos por la palabra clave **var;**

```
program muestra;
//parametros actuales a y b, c y d paso por referencia
  procedure prueba(var x,y:integer);
  begin //procedimiento
//proceso de los valores de x e y
  end;

begin
    .
    .
    .
1. prueba(a, c);
    .
    .
    .
2. prueba(b, d);
    .
    .
    .
end.
```

La primera llamada en (1) produce que los parámetros a y c sean sustituidos por x e y si los valores de x e y se modificaran dentro de a y c en el algoritmo principal. De igual modo, b y d son sustituidos por x e y, y cualquier modificación de x o y en el procedimiento afectará también al programa principal.

La llamada por referencia es muy útil para programas donde se necesita la comunicación del valor en ambas direcciones.

Notas:

Ambos métodos de paso de parámetros se aplican tanto a la llamada de funciones como a las de procedimientos:

1. Una función tiene la posibilidad de devolver los valores al programa principal de dos formas: a) como valor de la función, b) por medio de argumentos gobernados por la llamada de referencia en la correspondencia parámetro actual-parámetro formal.
2. Un procedimiento sólo puede devolver valores por el método de devolución de resultados.

El lenguaje Pascal permite que el programador especifique el tipo de paso de parámetros y, en un mismo subprograma, unos parámetros se pueden especificar por valor y otros por referencia.

```
procedure Q(i:integer; var j:integer);
begin
  i := i + 10;
  j := j + 10;
  write(i, j)
end;
```

Los parámetros formales son `i`, `j`, donde `i` se pasa por valor y `j` por referencia.

5.5.4. Comparaciones de los métodos de paso de parámetros

Para examinar de modo práctico los diferentes métodos, consideremos un ejemplo único y veamos los diferentes valores que toman los parámetros. El algoritmo correspondiente con un procedimiento SUBR:

```
algoritmo DEMO
var
  entero: A,B,C
inicio   //DEMO
  A ← 3
  B ← 5
  C ← 17
  llamar_a SUBR(A, A, A + B, C)
  escribir(C)
fin      //DEMO

procedimiento SUBR (<Modo> entero: x, y; E entero:z; <Modo> entero: v)

inicio
  x ← x + 1
  v ← y + z
fin_procedimiento
```

Modo por valor:

a) sólo por valor:

no se transmite ningún resultado, por consiguiente,
C *no varía* C = 17

b) valor resultado:

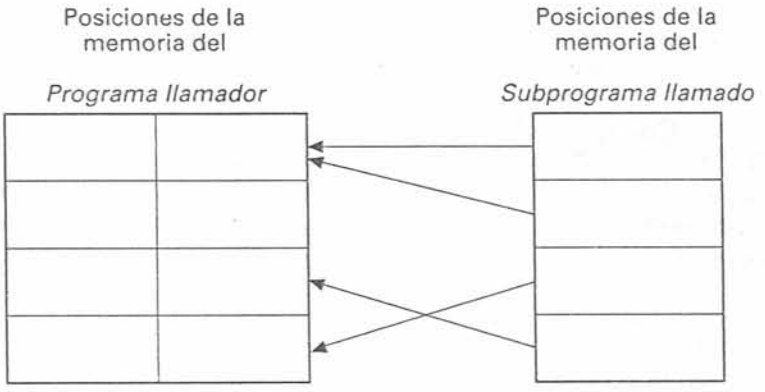
```
A = 3                          x = A = 3
B = 5          pasa al procedimiento   y = A = 3
C = 17                         z = A + B = 8
                               v = C = 17
```

al ejecutar el procedimiento quedará:

```
x = x + 1 = 3 + 1 = 4
v = y + z = 3 + 8 = 11
```

el parámetro llamado v pasa el valor del resultado v a su parámetro actual correspondiente, C.
Por tanto, C = 11.

Modo por referencia:



C recibirá el valor 12.

Utilizando variables globales:

```
algoritmo DEMO
var entero: A,B,C
inicio
  A ← 3
  B ← 5
  c ← 17
  llamar_a SUBR
  escribir (c)
fin
```

```
procedimiento SUBR
inicio
  a ← a + 1
  c ← a + a + b
fin_procedimiento
```

Es decir, el valor de C será 13.

La llamada por referencia es el sistema estándar utilizado por FORTRAN para pasar parámetros. La llamada por nombre es estándar en Algol 60. Simula 67 proporciona llamadas por valor, referencia y nombre.

Pascal permite pasar bien por valor bien por referencia

```
procedure demo(y:integer; var z:real);
```

especifica que y se pasa por valor mientras que z se pasa por referencia —indicado por la palabra reservada **var**—. La elección entre un sistema u otro puede venir determinado por diversas consideraciones, como evitar efectos laterales no deseados provocados por modificaciones inadvertidas de parámetros formales (*véase* 5.7).

5.5.5. Síntesis de la transmisión de parámetros

Los métodos de transmisión de parámetros más utilizados son *por valor* y *por referencia*.

El paso de un parámetro por valor significa que el valor del argumento —*parámetro actual o real*— se asigna al parámetro formal. En otras palabras, antes de que el subprograma comience a ejecutarse, el argumento se evalúa a un valor específico (por ejemplo, 8 o 12). Este valor se copia entonces en el correspondiente parámetro formal dentro del subprograma.

Una vez que el procedimiento arranca, cualquier cambio del valor de tal parámetro formal no se refleja en un cambio en el correspondiente argumento. Esto es, cuando el subprograma se termine, el argumento actual tendrá exactamente el mismo valor que cuando el subprograma comenzó, con independencia de lo que haya sucedido al parámetro formal. Este método es el método por defecto en Pascal si no se indica explícitamente otro. Estos parámetros de entrada se denominan *parámetros valor*. En los algoritmos indicaremos como < modo > E (entrada). El paso de un *parámetro por referencia o dirección* se llama *parámetro variable*, en oposición al parámetro por valor. En este caso, la

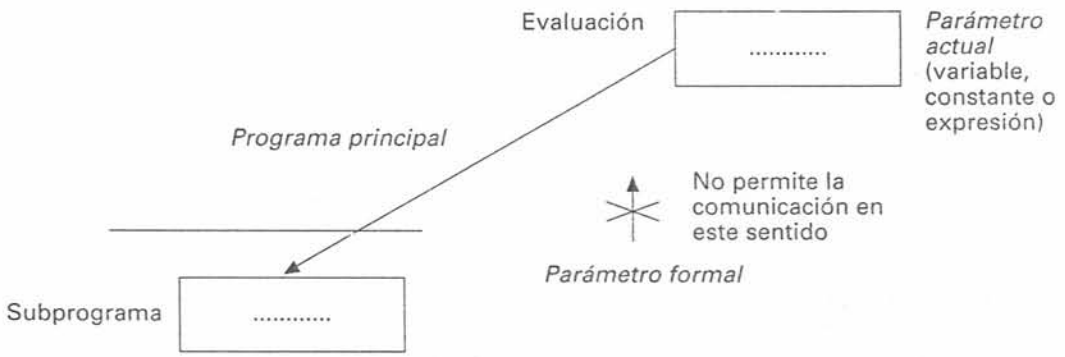


Figura 5.7. Paso de un parámetro por valor.

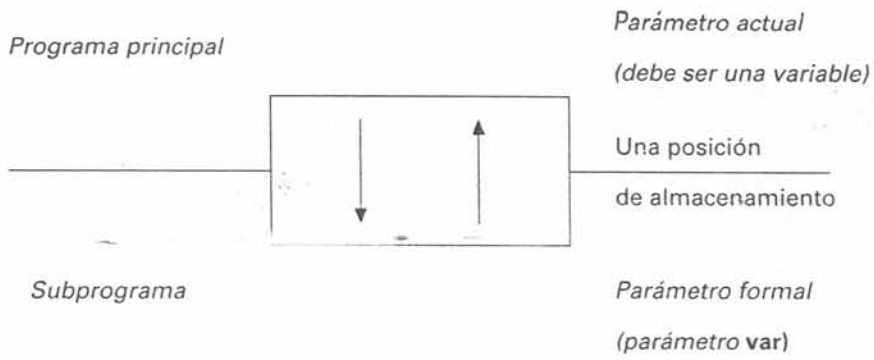


Figura 5.8. Paso de un parámetro por referencia.

posición o dirección (no el valor) del argumento o parámetro actual se envía al subprograma. Si a un parámetro formal se le da el atributo de parámetro variable —en Pascal con la palabra reservada **var**— y si el parámetro actual es una variable, entonces un cambio en el parámetro formal se refleja en un cambio en el correspondiente parámetro actual, ya que ambos tienen la misma posición de memoria.

Para indicar que deseamos transmitir un parámetro por dirección, lo indicaremos con la palabra *parámetro variable* —en Pascal se indica con la palabra reservada **var**— y especificaremos como *<modo> E/S* (*entrada/salida*) o *S* (*salida*).

Ejemplo 5.11

Se trata de realizar el cálculo del área de un círculo y la longitud de la circunferencia en función del valor del radio leído desde el teclado.

Recordemos las fórmulas del área del círculo y de la longitud de la circunferencia:

$A = pi \times r^2 = pi \times r \times r$
$C = 2pi \times r = 2 \times pi \times r$ *donde* pi = 3.141592

Los parámetros de entrada: `radio`.
Los parámetros de salida: `area, longitud`.
El procedimiento `circulo` calcula los valores pedidos.

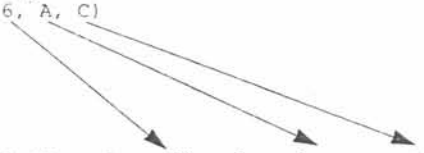
```
procedimiento circulo(E real: radio; S real: area, longitud)
  //parametros valor: radio
  //parametros variable: area, longitud
var
  real: pi
inicio
  pi ← 3.141592
  area ← pi * radio * radio
  longitud ← 2 * pi * radio
fin_procedimiento
```

Los parámetros formales son: radio, área, longitud, de los cuales son valor `(radio)` y variable `(area, longitud)`.

Invoquemos el procedimiento `circulo` utilizando la instrucción

```
llamar_a circulo(6, A, C)
```

```
  //{programa principal
inicio
//llamada al procedimiento
  llamar_a circulo(6, A, C)
     .
     .
     .
fin

procedimiento circulo(E real: radio; S real: area, longitud)
//parametros valor:radio
//parametros variable: area, longitud

inicio
  pi ← 3.141592
  area ← pi * radio * radio
  longitud ← 2 * pi * radio
fin_procedimiento
```



Ejemplo 5.12

Consideremos un subprograma `N` *con dos parámetros formales:* `i`, *transmitido por valor, y* `j`, *por variable.*

```
algoritmo M
//variables A, B enteras
var
  entero: A, B

inicio
  A ← 2
  B ← 3
  llamar_a N(A,B)
  escribir(A, B)
fin     //algoritmo M

procedimiento N(E entero: I; E/S entero: j)
  //parametros valor i
  //parametros variable j
inicio
  i ← i + 10
  j ← j + 10
  escribir(i, j)
fin_procedimiento
```

Si se ejecuta el procedimiento `N`, veamos qué resultados se escribirán:

`A` y `B` son parámetros actuales,
`i` y `j` son parámetros formales.

Como `i` es por valor, se transmite el valor de `A` a `i`, es decir, `i = A = 2`. Cuando `i` se modifica por defecto de `i ← i + 10` a `12`, `A` no cambia y, por consiguiente, a la terminación de `N`, `A` sigue valiendo `2`.

El parámetro B se transmite por referencia, es decir, j es un parámetro variable. Al comenzar la ejecución de N, B se almacena como el valor j y cuando se suma 10 al valor de j, i en sí mismo no cambia. El valor del parámetro B se cambia a 13. Cuando los valores i, j se escriben en N, los resultados son:

```
12  y  13
```

pero cuando retornan a M y al imprimir los valores de A y B, sólo ha cambiado el valor B. El valor de i = 12 se pierde en N cuando éste ya termina. El valor de j también se pierde, pero éste es la dirección, no el valor 13.

Se escribirá como resultado final de la instrucción ***escribir***(A, B):

```
2   13
```

5.6. FUNCIONES Y PROCEDIMIENTOS COMO PARÁMETROS

Hasta ahora los subprogramas que hemos considerado implicaban dos tipos de parámetros formales: *parámetros valor* y *parámetros variable*. Sin embargo, en ocasiones se requiere que un procedimiento o función dado invoque a otro procedimiento o función que ha sido definido fuera del ámbito de ese procedimiento o función. Por ejemplo, se puede necesitar que un procedimiento P invoque la función F que puede estar o no definida en el procedimiento P; esto puede conseguirse transfiriendo como parámetro el procedimiento o función externa (F) o procedimiento o función dado (por ejemplo, el P). En resumen, algunos lenguajes de programación —entre ellos Pascal— admiten *parámetros procedimiento* y *parámetros función*.

Ejemplos:

```
procedimiento P(E func: F1; E real: x,y)
real funcion F(E func: F1,F2; E entero:x,y)
```

Los parámetros formales del procedimiento P son la función F1 y las variables X e Y, y los parámetros formales de la función F son las funciones F1 y F2, y las variables x e y.

Procedimientos función

Para ilustrar el uso de los parámetros función, consideremos la función *integral* para calcular el área bajo una curva f(x) para un intervalo a <= x <= b.

La técnica conocida para el cálculo del área es subdividir la región en rectángulos, como se muestra en la Figura 5.9 y sumar las áreas de los rectángulos. Estos rectángulos se construyen subdividiendo el intervalo [a, b] en m subintervalos iguales y formando rectángulos con estos subintervalos como bases y alturas dadas por los valores de f en los puntos medios de los subintervalos.

La función *integral* debe tener los parámetros formales a, b y n, que son parámetros valor ordinarios actuales de tipo real; se asocian con los parámetros formales a y b; un parámetro actual de tipo entero —las subdivisiones— se asocia con el parámetro formal n y una función actual se asocia con el parámetro formal f.

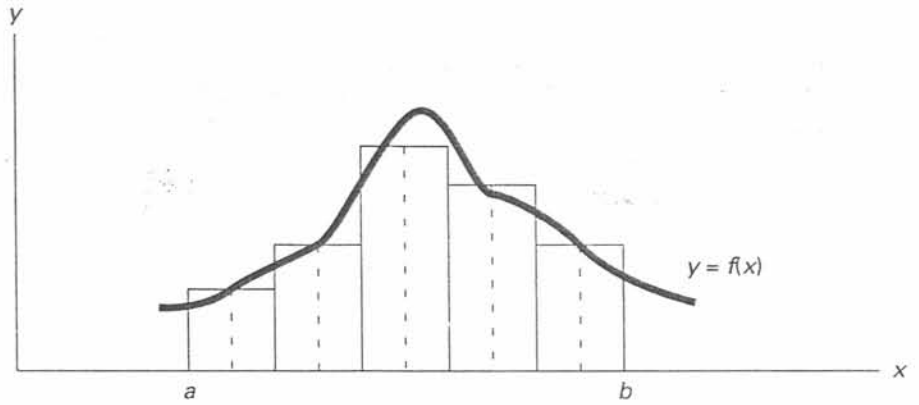


Figura 5.9. Cálculo del área bajo la curva `f(x)`.

Los parámetros función se designan como tales con una cabecera de función dentro de la lista de parámetros formales. La función integral podrá definirse por

```
real funcion integral(E func: f; E real: a,b; E entero: n)
el tipo func-tipo real funcion (E real:x) : func-
```

aquí la función `func: F` especifica que `f` es una función parámetro que denota una función cuyo parámetro formal y valor son de tipo real. El correspondiente parámetro función actual debe ser una función que tiene un parámetro formal real y un valor real. Por ejemplo, si integrado es una función de valor real con un parámetro de tipo real `funcion`, es decir, de tipo `: func`

```
Area ← Integral(Integrado, 0, 1.5, 20)
```

es una referencia válida a función.

Diseñar un algoritmo que utilice la función Integral para calcular el área bajo el gráfico de las función es `f1 (x)` = $x^3 - 6x^2 + 10x$ *e Integrado por Integrado* `(x)` = $x^2 + 3x + 2$ *para* $0 <= x <= 4$.

```
algoritmo Area_bajo_curvas

tipo
  real funcion(E real : x) : func
var
  real   : a,b
  entero : n

inicio
escribir('¿Entre que limites?')
leer(a, b)
escribir('¿Subintervalos?')
leer(n)
escribir(integral(f1, a, b, n))
escribir(integral(integrado, a, b, n))
fin
```

```
real funcion f1 (E real : x)
inicio
  devolver( x * x * x - 6 * x * x + 10 * x)
fin_funcion

real funcion integrado (E real : x)
inicio
  devolver( x * x + 3 * x + 2 )
fin_funcion

real funcion integral (E func : f; E real : a, b;  E entero : n)
var
  real    : baserectangulo,altura,x,s
  entero : i

inicio
  baserectangulo ← (b - a) / n
  x ← a + baserectangulo/2
  s ← 0
  desde i ← 1 hasta n hacer
    altura ← f(x)
    s ← s + baserectangulo * altura
    x ← x + baserectangulo
  fin_desde
  devolver(s)
fin_funcion
```

5.7. LOS EFECTOS LATERALES

Las modificaciones que se produzcan mediante una función o procedimiento en los elementos situados fuera del subprograma (función o procedimiento) se denominan *efectos laterales*. Aunque en algunos casos los efectos laterales pueden ser beneficiosos en la programación, es conveniente no recurrir a ellos de modo general. Consideramos a continuación los efectos laterales en funciones y en procedimientos.

5.7.1. En procedimientos

La *comunicación* del procedimiento con el resto del programa se debe realizar normalmente a través de parámetros. Cualquier otra *comunicación* entre el procedimiento y el resto del programa se conoce como *efectos laterales*. Como ya se ha comentado, los efectos laterales son perjudiciales en la mayoría de los casos, como se indica en la Figura 5.10.

Si un procedimiento modifica una variable global (distinta de un parámetro actual), éste es un *efecto lateral*. Por ello, excepto en contadas ocasiones, no debe aparecer en la declaración del procedimiento. Si se necesita una variable temporal en un procedimiento, utilice una variable local, no una variable global. Si se desea que el programa modifique el valor de una variable global, utilice un parámetro formal variable en la declaración del procedimiento y a continuación utilice la variable global como el parámetro actual en una llamada al procedimiento.

En general, se debe seguir la regla de *«ninguna variable global en procedimientos»*, aunque esta prohibición no significa que los procedimientos no puedan manipular variables globales. De hecho, el cambio de variables globales se debe pasar al procedimiento como parámetros actuales. Las variables globales no se deben utilizar directamente en las instrucciones en el cuerpo de un procedimiento; en su lugar, utilice un parámetro formal o variable local.

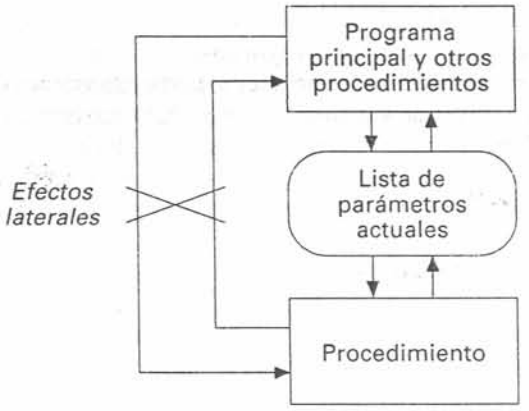


Figura 5.10. Efectos laterales en procedimientos.

En aquellos lenguajes en que es posible declarar constantes —como Pascal— se pueden utilizar constantes globales en una declaración de procedimiento; la razón reside en el hecho de que las constantes no pueden ser modificadas por el procedimiento y, por consiguiente, no existe peligro de que se puedan modificar inadvertidamente.

5.7.2. En funciones

Una función toma los valores de los argumentos y devuelve *un único valor*. Sin embargo, al igual que los procedimientos, una función —en algunos lenguajes de programación— puede hacer cosas similares a un procedimiento o subrutina. Una función puede tener parámetros variables además de parámetros valor en la lista de parámetros formales. Una función puede cambiar el contenido de una variable global y ejecutar instrucciones de entrada/salida (escribir un mensaje en la pantalla, leer un valor del teclado, etc.). Estas operaciones se conocen como *parámetros laterales* y se deben evitar.

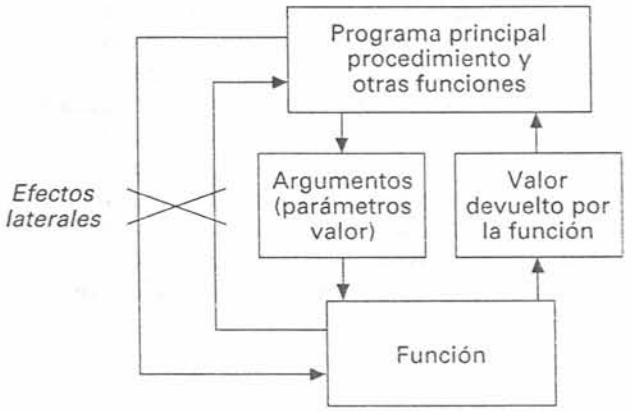


Figura 5.11. Efectos laterales en una función.

Los efectos laterales están considerados —normalmente— como una mala técnica de programación, pues hacen más difícil de entender los programas.

Toda la información que se transfiere entre procedimientos y funciones debe realizarse a través de la lista de parámetros y no a través de variables globales. Esto convertirá el procedimiento o función en módulos independientes que pueden ser comprobados y depurados por sí solos, lo que evitará no preocuparnos por el resto de las partes del programa.

5.8. RECURSIÓN (RECURSIVIDAD)

Como ya se conoce, un subprograma puede llamar a cualquier otro subprograma y éste a otro, y así sucesivamente; dicho de otro modo, los subprogramas se pueden anidar. Se puede tener

```
A llamar_a B, B llamar_a C, C llamar_a D
```

Cuando se produce el retorno de los subprogramas a la terminación de cada uno de ellos el proceso resultante será:

```
D retornar_a C, C retornar_a B, B retornar_a A
```

¿Qué sucedería si dos subprogramas de una secuencia son los mismos?

```
A llamar_a A
```

o bien:

```
A llamar_a B, B llamar_a A
```

En primera instancia, parece incorrecta. Sin embargo, existen lenguajes de programación —Pascal, C, entre otros— en que un subprograma puede llamarse a sí mismo.

Una función o procedimiento que se puede llamar a sí mismo se llama *recursivo*. La *recursión* (**recursividad**) es una herramienta muy potente en algunas aplicaciones, sobre todo de cálculo. La recursión puede ser utilizada como una alternativa a la repetición o estructura repetitiva.

El uso de la recursión es particularmente idóneo para la solución de aquellos problemas que pueden definirse de modo natural en términos recursivos.

La escritura de un procedimiento o función recursivo es similar a sus homónimos no recursivos; sin embargo, para evitar que la recursión continúe indefinidamente es preciso incluir una condición de terminación.

La razón de que existan lenguajes que admiten la recursividad se debe a la existencia de estructuras específicas tipo *pilas* (*stack*, en inglés) para este tipo de procesos y memorias dinámicas. Las direcciones de retorno y el estado de cada subprograma se guardan en estructuras tipo pilas (*véase* Capítulo 11). En el Capítulo 11 se profundizará en el tema de las pilas; ahora nos centraremos sólo en el concepto de recursividad y en su comprensión con ejemplos básicos.

Ejemplo 5.13

Muchas funciones matemáticas se definen recursivamente. Un ejemplo de ello es el factorial de un número entero n.

La función factorial se define como

$$n! = \begin{cases} 1 \text{ si } n = 0 \quad 0! = 1 \\ (n) \times (n-1) \times (n-2) \times \ldots \times 3 \times 2 \times 1 \text{ si } n > 0 \; n.(n-1).(n-2) \ldots .3.2.1 \end{cases}$$

Si se observa la fórmula anterior cuando `n > 0`, es fácil definir `n!` en función de `(n - 1)!` Por ejemplo, `5!`

```
5! = 5×4×3×2×1    = 120
4! = 4×3×2×1      = 24
3! = 3×2×1        = 6
2! = 2×1          = 2
1! = 1×1          = 1
0! = 1            = 1
```

Se pueden transformar las expresiones anteriores en

```
5! = 5×4!
4! = 4×3!
3! = 3×2!
2! = 2×1!
1! = 1×0!
```

En términos generales sería:

$$n! = \begin{cases} 1 & si \quad n = 0 \\ n(n-1)! & si \quad n > 0 \end{cases}$$

La función `FACTORIAL` de `N` expresada en términos recursivos sería:

```
FACTORIAL ← N * FACTORIAL(N - 1)
```

La definición de la función sería:

```
entero funcion factorial(E entero: n)
//calculo recursivo del factorial
inicio
  si n = 0 entonces
    devolver (1)
  si_no devolver (n * factorial(n - 1))
  fin_si
fin_funcion
```

Para demostrar cómo esta versión recursiva de `FACTORIAL` calcula el valor de `n!`, consideremos el caso de `n = 3`. Un proceso gráfico se representa en la Figura 5.12.

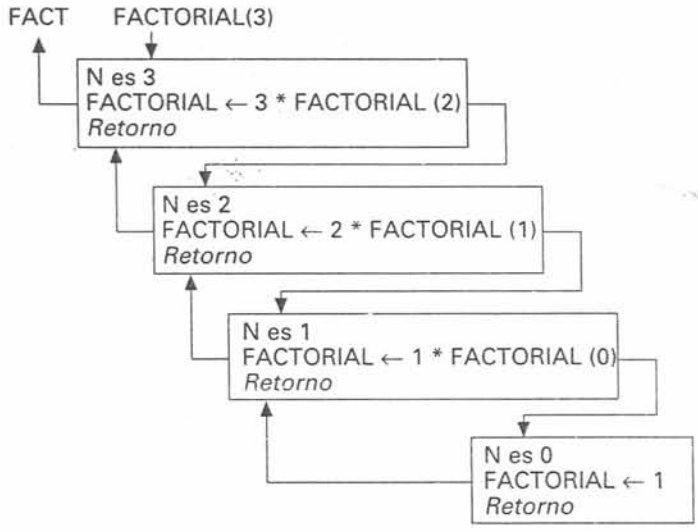
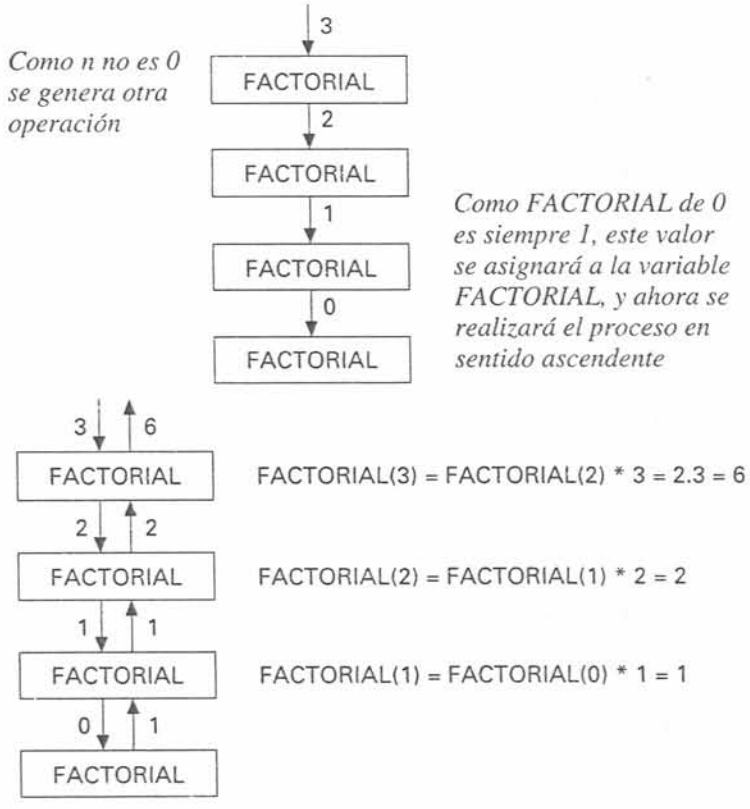


Figura 5.12. Cálculo recursivo de `FACTORIAL` de 3.

Los pasos sucesivos extraídos de la Figura 5.12 son:



Ejemplo 5.14

Otro ejemplo típico de una función recursiva es la serie Fibonacci. Esta serie fue concebida originalmente como modelo para el crecimiento de una granja de conejos (multiplicación de conejos) por el matemático italiano del siglo XVI, Fibonacci.

La serie es la siguiente:

1, 1, 2, 3, 5, 8, 13, 21, 34 ...
Esta serie crece muy rápidamente; como ejemplo, el término 15 es 610.
La serie de Fibonacci (*fib*) se expresa así:

$fib(1) = 1$
$fib(2) = 1$
$fib(n) = fib(n - 1) + fib(n - 2)$ para $n > 2$

Una función recursiva que calcula el elemento enésimo de la serie de Fibonacci es:

```
entero funcion fibonacci(E entero: n)
//calculo del elemento n-esimo
inicio
  si (n = 1) o (n = 2) entonces
    devolver (1)
  si_no
    devolver (fibonacci(n - 2) + fibonacci(n - 1)
  fin_si
fin_funcion
```

Aunque es fácil de escribir la función de Fibonacci, no es muy eficaz definida de esta forma, ya que cada paso recursivo genera otras dos llamadas a la misma función.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

5.1. Diseñar el algoritmo que calcule el factorial de un número entero.

```
entero funcion factorial(E entero: n)
var entero: f, i
inicio
  si n = 0 entonces
    devolver (1)
  si_no
    desde i ← 1 hasta n hacer
      f ← f * i
    fin_desde
    devolver (f)
  fin_si
fin_funcion
```

5.2. Diseñar un algoritmo que calcule el máximo común divisor de dos números mediante el algoritmo de Euclides.

Sean los dos números A y B. El método para hallar el máximo común divisor (mcd) de dos números A y B por el método de Euclides es:

1. Dividir el número mayor (A) por el menor (B). Si el resto de la división es cero, el número B es el máximo común divisor.
2. Si la división no es exacta, se divide el número menor (B) por el resto de la división anterior.
3. Se siguen los pasos anteriores hasta obtener un resto cero. El último divisor es el mcd buscado.

Algoritmo

```
entero funcion mcd(E entero: a, b)
inicio
  mientras a <> b hacer
    si a > b entonces
      a ← a - b
    si_no
      b ← b - a
    fin_si
  fin_mientras
  devolver(a)
fin_funcion
```

5.3. Para calcular el máximo común divisor (mcd) de dos números se recurre a una función específica definida con un subprograma. Se desea calcular la salida del programa principal con dos números A y B, cuyos valores son 10 y 25, es decir, el mcd(A, B) y comprobar el método de paso de parámetros por valor.

```
algoritmo Maxcomdiv
var
  entero: N, X, Y
inicio //programa principal
  x ← 10
  y ← 25
  n ← mcd(x, y)
  escribir(x, y, n)
fin
entero funcion mcd(E entero: a,b)
inicio
  mientras a <> b hacer
    si a > b entonces
      a ← a - b
    si_no
      b ← b - a
    fin_si
  fin_mientras
  devolver (a)
fin
fin_funcion
```

Los parámetros formales son a y b y recibirán los valores de X e Y.

```
a = 10
b = 25
```

Las variables locales a la función son A y B y no modificarán los valores de las variables X e Y del algoritmo principal.

Variables del programa principal			*Variables de la función*		
X	Y	N	a	b	mcd(a, b)
10	25		10	25	

Las operaciones del algoritmo son:

```
a = 10          b = 25
```

1. b > a realizará la operación b ← b - a
   y, por consiguiente, b tomará el valor 25 - 10 = 15
   y a sigue valiendo 10.

2. a = 10 b = 15
   se realiza la misma operación anterior
   b ← b - a, es decir, b = 5
   a permanece inalterable

3. a = 10 b = 5
   como a > b entonces se realiza a ← a - b, es decir, a = 5.

Por consiguiente, los valores finales serían:

```
a = 5     b = 5     mcd(a, b) = 5
```

Como los valores A y B no se pasan al algoritmo principal, el resultado de su ejecución será:

```
10     25     5
```

5.4. Escribir un algoritmo que permita ordenar tres números mediante un procedimiento de intercambio en dos variables (paso de parámetros por referencia).

El algoritmo que permite realizar el intercambio de los valores de variables numéricas es el siguiente:

```
AUXI ← A
  A  ← B
  B  ← AUXI
```

y la definición del procedimiento será:

```
PROCEDIMIENTO intercambio (E/S real: a, b)
var real : auxi

inicio
  auxi ← a
  a ← b
  b ← auxi
fin_procedimiento
```

El algoritmo de ordenación se realizará mediante llamadas al procedimiento `intercambio`.

```
algoritmo Ordenar_3_numeros
var real : x,y,z

inicio
  escribir('Deme 3 numeros reales')
  leer(x, y, z)
  si x > y entonces
    intercambio (x, y)
  fin_si
  si y > z entonces
    intercambio (y, z)
  fin_si
  si x > y entonces
    intercambio (x, y)
  fin_si
  escribir( x, y, z)
fin
```

Paso de parámetros por referencia

Los tres números X, Y, Z que se van a ordenar son:

```
132     45     15
```

Los pasos sucesivos al ejecutarse el algoritmo o programa principal son:

1. Lectura X, Y, Z parámetros actuales

   ```
   X = 132
   Y = 45
   Z = 15
   ```

2. Primera llamada al procedimiento `intercambio(a, b) x > y`.
   La correspondencia entre parámetros será la siguiente:

Parámetros actuales	*Parámetros formales*
X	A
Y	B

   Al ejecutarse el procedimiento se intercambiarán los valores de A y B que se devolverán a las variables X e Y; luego valdrán:

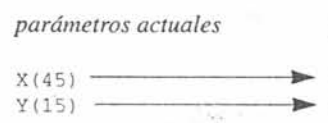
   ```
   X = 45
   Y = 132
   ```

3. Segunda llamada al procedimiento *intercambio* con `Y > Z` (ya que `Y = 132` y `Z = 15`)

Parámetros actuales		*Parámetros formales*
Y	⟶	A
Z	⟶	B

   Antes llamada al procedimiento `Y = 132, Z = 15`.
   Después terminación del procedimiento `Z < 0 132, Y = 15`. ya que A y B han intercambiado los valores recibidos, 132 y 15.

4. Los valores actuales de X, Y, Z son 45, 15, 132; por consiguiente, X > Y, y habrá que hacer otra nueva llamada al procedimiento intercambio.



Después de la ejecución del procedimiento A y B intercambiarán sus valores y valdrán A = 15, B = 45, por lo que se pasan al algoritmo principal X = 15, Y = 45. Por consiguiente, el valor final de las tres variables será:

```
X = 15          Y = 45          Z = 132
```

ya ordenados de modo creciente.

5.5. Diseñar un algoritmo que llame a la función **signo(X)** y calcule: a) el signo de un número, b) el signo de la función coseno.

Variables de entrada: P (real)
Variables de salida: Y –signo del valor P-(entero) Z –signo del coseno de P-(entero);

Pseudocódigo

```
algoritmo Signos
var entero: y, z
    real: P

inicio
  leer(P)
  Y ← signo(p)
  Z ← signo(cos (p))
  escribir(Y, Z)
fin

entero funcion signo(E real: x)
inicio
  si x > 0 entonces
    devolver (1)
  si_no
    si x < 0 entonces
      devolver (-1)
    si_no
      devolver (0)
    fin_si
  fin_si
fin_funcion
```

Notas de ejecución

Parámetro actual	*Parámetro formal*
P	X

El parámetro formal X se constituye por el parámetro actual. Así, por ejemplo, si el parámetro P vale -1.45. Los valores devueltos por la función signo que se asignarán a las variables Y, Z son:

```
Y ← Signo(-1.45)
Z ← Signo(cos (-1.45))
```

resultando

```
Y = -1
Z = 1
```

EJERCICIOS

5.1. Diseñar una función que calcule la media de tres números leídos del teclado y poner un ejemplo de su aplicación.
5.2. Diseñar la función FACTORIAL que calcule el factorial de un número entero en el rango 100 a 1.000.000
5.3. Diseñar un algoritmo para calcular el máximo común divisor de cuatro números basado en un subalgoritmo función mcd (máximo común divisor de dos números).
5.4. Diseñar un procedimiento que realice el intercambio de valores de dos variables A y B.
5.5. Diseñar una función que encuentre el mayor de dos números enteros.
5.6. Diseñar una función que calcule x^n para x, variable real y n variable entera.
5.7. Diseñar un procedimiento que acepte un número de mes, un número de día y un número de año y los visualice en el formato

```
dd/mm/aa
```

Por ejemplo, los valores 19,09,1987 se visualizarían como

```
19/9/87
```

y para los valores 3, 9 y 1905

```
3/9/05
```

5.8. Realizar un procedimiento que realice la conversión de coordenadas polares (r, θ) a coordenadas cartesianas (x, y)

```
x = r.cos (J)
y = r.sen(J)
```

5.9. Escribir una función Salario que calcule los salarios de un trabajador para un número dado de horas trabajadas y un salario hora. Las horas que superen las 40 horas semanales se pagarán como extras con un salario hora 1,5 veces el salario ordinario.
5.10. Escribir una función booleana Digito que determine si un carácter es uno de los dígitos 0 al 9.

CAPÍTULO 6

ESTRUCTURAS DE DATOS (ARRAYS)[1]

CONTENIDO

6.1. Introducción a las estructuras de datos.
6.2. Arrays unidimensionales: los vectores.
6.3. Operaciones con vectores.
6.4. Arrays de varias dimensiones.
6.5. Arrays multidimensionales.
6.6. Almacenamiento de arrays en memoria.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

En los capítulos anteriores se ha introducido el concepto de datos de tipo simple que representan valores de tipo simple, como un número entero, real o un carácter. En muchas situaciones se necesita, sin embargo, procesar una colección de valores que están relacionados entre sí por algún método, por ejemplo, una lista de calificaciones, una serie de temperaturas medidas a lo largo de un mes, etc. El procesamiento de tales conjuntos de datos, utilizando datos simples, puede ser extremadamente difícil y por ello la mayoría de los lenguajes de programación incluyen características de estructuras de datos. *Las estructuras de datos básicas* que soportan la mayoría de los lenguajes de programación son los *arrays* —concepto matemático de «vector» y «matriz».

Un **array** (*matriz, tabla, arreglo*) es una secuencia de posiciones de la memoria central a las que se puede acceder directamente, que contiene datos del mismo tipo y pueden ser seleccionados individualmente mediante el uso de subíndices. Este capítulo estudia el concepto de *arrays unidimensionales* y *multidimensionales*, así como el procesamiento de los mismos.

[1] En Latinoamérica, el término *array* se suele traducir por la palabra **arreglo**.

6.1. INTRODUCCIÓN A LAS ESTRUCTURAS DE DATOS

Una *estructura de datos* es una colección de datos que pueden ser caracterizados por su organización y las operaciones que se definen en ella.

Las estructuras de datos son muy importantes en los sistemas de computadora. Los tipos de datos más frecuentes utilizados en los diferentes lenguajes de programación son:

- **datos simples**
  - *estándar*
    - entero (*integer*)
    - real (*real*)
    - carácter (*char*)
    - lógico (*boolean*)
  - *definido por el programador (no estándar)*
    - subrango (*subrange*)
    - enumerativo (*enumerated*)
- **datos estructurados**
  - *estáticos*
    - *array* (vector/matriz)
    - registro
    - archivo (fichero)
    - conjunto
    - cadena (*string*)
  - *dinámicos*
    - lista (pila/cola)
    - lista enlazada
    - árbol
    - grafo

Los tipos de datos *simples* o *primitivos* significan que no están compuestos de otras estructuras de datos; los más frecuentes y utilizados por casi todos los lenguajes son: *enteros*, *reales* y *carácter* (*char*), siendo los tipos *lógicos*, *subrango* y *enumerativos* propios de lenguajes estructurados como Pascal. Los tipos de datos compuestos están construidos basados en tipos de datos primitivos; el ejemplo más representativo es la *cadena* (*string*) de caracteres.

Los tipos de datos simples pueden ser organizados en diferentes estructuras de datos: *estáticas* y *dinámicas*. Las **estructuras de datos estáticas** son aquéllas en las que el tamaño ocupado en memoria se define antes de que el programa se ejecute y no puede modificarse dicho tamaño durante la ejecución del programa. Estas estructuras están implementadas en casi todos los lenguajes: *array* (vectores/tablas-matrices), *registros*, *ficheros* (los *conjuntos* son específicos del lenguaje Pascal).

Las **estructuras de datos dinámicas** no tienen las limitaciones o restricciones en el tamaño de memoria ocupada que son propias de las estructuras estáticas. Mediante el uso de un tipo de datos específico, denominado *puntero*, es posible construir estructuras de datos dinámicas que son soportadas por la mayoría de los lenguajes, y en aquellos que sí tienen estas características ofrecen soluciones eficaces y efectivas en la solución de problemas complejos —Pascal es el lenguaje tipo por excelencia con posibilidad de estructuras de datos dinámicos—. Las estructuras dinámicas por excelencia son las *listas* —enlazadas, pilas, colas—, *árboles* —binarios, árbol-b, de búsqueda binaria— y *grafos*.

La elección del tipo de estructura de datos idónea a cada aplicación dependerá esencialmente del tipo de aplicación y, en menor medida, del lenguaje, ya que en aquellos en que no está implementada una estructura —por ejemplo, las listas y árboles no los soporta BASIC— deberá ser simulada con el algoritmo adecuado, dependiendo del propio algoritmo y las características del lenguaje su fácil o difícil solución.

Una característica importante que diferencia a los tipos de datos es la siguiente: los tipos de datos simples tienen como característica común que cada variable representa a un elemento; los tipos de datos estructurados tienen como característica común que un *identificador* (nombre) puede representar múltiples datos individuales, pudiendo cada uno de éstos ser referenciado independientemente.

6.2. ARRAYS UNIDIMENSIONALES: LOS VECTORES

Un *array* (*matriz* o *vector*) es un conjunto finito y ordenado de elementos homogéneos. La propiedad «*ordenado*» significa que el elemento primero, segundo, tercero, ..., enésimo de un array puede ser identificado. Los elementos de un *array* son homogéneos, es decir, del mismo tipo de datos. Un *array* puede estar compuesto de todos sus elementos de tipo cadena, otro puede tener todos sus elementos de tipo entero, etc. Los *arrays* se conocen también como *matrices* —en matemáticas— y *tablas* —en cálculos financieros.

El tipo más simple de *array* es el *array unidimensional* o *vector* (matriz de una dimensión). Un vector de una dimensión denominado NOTAS que consta de n elementos se puede representar por la Figura 6.1.

El *subíndice* o *índice* de un elemento `[1, 2, ..., i, n]` designa su posición en la ordenación del vector. Otras posibles notaciones del vector son:

$a_1, a_2, \ldots, a_i, \ldots, a_n$

`A(1), A(2), ..., A(i), ..., A(n)`

Obsérvese que sólo el vector global tiene nombre `(NOTAS)`. Los elementos del vector se referencian por su *subíndice* o *índice* (*subscript*), es decir, su posición relativa en el vector.

En algunos libros y tratados de programación, además de las notaciones anteriores se suele utilizar esta otra:

```
A[L:U] = {A[I]}
```

para `I = L, L + 1, ..., U - 1, U` *donde cada elemento* `A[I]` *es de tipo de datos* `T`

que significa: `A`, vector unidimensional con elementos de datos tipo `T`, cuyos subíndices varían en el rango de `L` a `U`, lo cual significa que el índice no tiene por qué comenzar necesariamente en 0 o en 1.

Como ejemplo de un vector o *array* unidimensional, se puede considerar el vector `TEMPERATURA` que contiene las temperaturas horarias registradas en una ciudad durante las veinticuatro horas del día. Este vector constará de veinticuatro elementos de tipo real, ya que las temperaturas normalmente no serán enteras siempre.

NOTAS[1]	NOTAS[2]	.....	NOTAS[I]	.....	NOTAS[N]

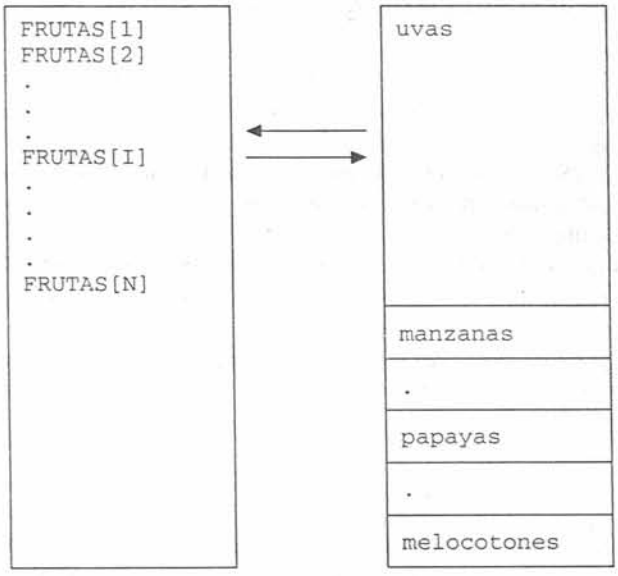
Figura 6.1. Vector.

El valor mínimo permitido de un vector se denomina *límite inferior* del vector (`L`) y el valor máximo permitido se denomina *límite superior* (`U`). En el ejemplo del vector `TEMPERATURAS` el límite inferior es `1` y el superior `24`.

```
TEMPERATURAS[I]    donde 1 <= I <= 24
```

El número de elementos de un vector se denomina *rango del vector*. El rango del vector A[L:U] es U - L + 1. El rango del vector `B[1:n]` es n.

Los vectores, como ya se ha comentado, pueden contener datos no numéricos, es decir, tipo «carácter». Por ejemplo, un vector que representa las frutas que se venden en un supermercado:

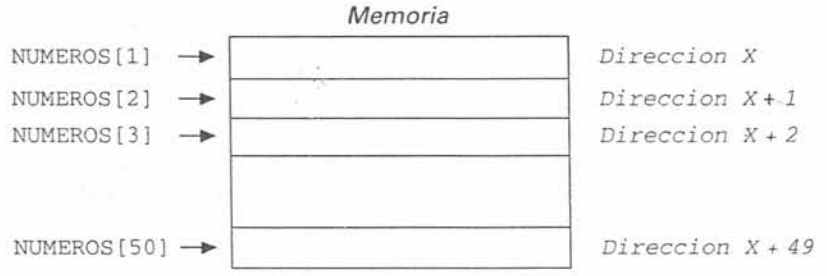


Otro ejemplo de un vector pueden ser los nombres de los alumnos de una clase. El vector se denomina `ALUMNOS` y tiene treinta elementos de rango:

ALUMNOS

1	Luis Francisco
2	Jose
3	Victoria
	.
i	Martin
	.
30	Graciela

Los vectores se almacenan en memoria central de la computadora en un orden adyacente. Así, un vector de cincuenta números denominado NUMEROS se representa gráficamente por cincuenta posiciones de memoria sucesivas:



Cada elemento de un vector se puede procesar como si fuese una variable simple al ocupar una posición de memoria. Así,

```
NUMEROS[25] ← 72
```

almacena el valor entero o real 72 en la posición 25ª del vector NUMEROS y la instrucción de salida

```
escribir (NUMEROS[25])
```

visualiza el valor almacenado en la posición 25ª, en este caso 72.

Esta propiedad significa que cada elemento de un vector —y posteriormente una tabla o matriz— es accesible directamente. Ésta será una de las *ventajas* más importantes de usar un vector: *almacenar un conjunto de datos.*

Consideremos un vector X de ocho elementos:

X[1]	X[2]	X[3]	X[4]	X[5]	X[6]	X[7]	X[8]
14.0	12.0	8.0	7.0	6.41	5.23	6.15	7.25
Elemento 1.°	Elemento 2.°						Elemento 8.°

Algunas instrucciones que manipulan este vector se representan en la Tabla 6.1.

Tabla 6.1. Operaciones básicas con vectores

Acciones	Resultados
escribir(X[1])	Visualiza el valor de X [1] o 14.0
X[4] ← 45	Almacena el valor 45 en X[4]
SUMA ← X[1] + X[3]	Almacena la suma de X[1] y X[3] o bien 22.0 en la variable SUMA
SUMA ← SUMA + X[4]	Añade en la variable SUMA el valor de X[4], es decir, SUMA = 67.0
X(5) ← X[5] + 3.5	Suma 3.5 a X[5]; el nuevo valor de X[5] será 9.91
X(6) ← X[1] + X[2]	Almacena la suma de X[1] y X[2] en X[6]; el nuevo valor de X[6] será 26

Antes de pasar a tratar las diversas operaciones que se pueden efectuar con vectores, consideremos la notación de los diferentes elementos.

Supongamos un vector V de ocho elementos.

V[1]	V[2]	V[3]	V[4]	V[5]	V[6]	V[7]	V[8]
12	5	-7	14.5	20	1.5	2.5	-10

Los subíndices de un vector pueden ser enteros, variables o expresiones enteras. Así, por ejemplo, si

```
I ← 4
V[I + 1]    representa el elemento V[5] de valor 20
V[I + 2]    representa el elemento V[6] de valor 1.5
V[I - 2]    representa el elemento V[2] de valor 5
V[I + 3]    representa el elemento V[7] de valor 2.5
```

Los *arrays* unidimensionales, al igual que posteriormente se verán los *arrays* multidimensionales, necesitan ser dimensionados previamente a su uso dentro de un programa.

6.3. OPERACIONES CON VECTORES

Un vector, como ya se ha mencionado, es una secuencia ordenada de elementos como

```
X[1], X[2], ..., X[n]
```

El límite inferior no tiene por qué empezar en uno. El vector L

```
L[0], L[1], L[2], L[3], L[4]), L[5]
```

contiene seis elementos, en el que el primer elemento comienza en cero. El vector P, cuyo rango es 7 y sus límites inferior y superior son −3 y 3, es:

```
P[-3], P[-2], P[-1], P[0], P[1], P[2], P[3]
```

Las operaciones que se pueden realizar con vectores durante el proceso de resolución de un problema son:

- *asignación*,
- *lectura/escritura*,
- *recorrido* (acceso secuencial),
- *actualizar* (añadir, borrar, insertar),
- *ordenación*,
- *búsqueda*.

En general, las operaciones con vectores implican el procesamiento o tratamiento de los elementos individuales del vector.

Las notaciones algorítmicas que utilizaremos en este libro son:

```
tipo
  array [dimensiones] de <tipo de dato>:<nombre_del_tipo_array>
```

```
tipo
  array[1..10] de caracter: nombres
var
  nombres: n
```

significa que `nombres` es un array (vector) unidimensional de diez elementos (1 a 10) de tipo carácter.

```
tipo
  array ['A'..'Z'] de real: lista
var
  lista: l
```

representa un vector cuyos subíndices son `A`, `B`, `...` y cuyos elementos son de tipo real.

```
tipo
  array[0..100] de entero: numero
var
  numero: nu
```

`numero` es un vector cuyos subíndices van de `0` a `100` y de tipo entero.

Las operaciones que analizaremos en esta sección serán: *asignación*, *lectura/escritura*, *recorrido* y *actualización*, dejando por su especial relevancia como tema exclusivo de un capítulo la *ordenación* o *clasificación* y *búsqueda*.

6.3.1. Asignación

La asignación de valores a un elemento del vector se realizará con la instrucción de asignación:

`A[29] ← 5` *asigna el valor* `5` *al elemento* `20` *del vector* `A`

Si se desea asignar valores a todos los elementos de un vector, se debe recurrir a estructuras repetitivas (**`desde`**, **`mientras`** o **`repetir`**) e incluso selectivas (**`si-entonces`**, **`segun`**).

```
leer(A[i])
```

Si se introducen los valores `5`, `7`, `8`, `14` y `12` mediante asignaciones

```
A[1] ← 5
A[2] ← 7
A[3] ← 8
A[4] ← 14
A[5] ← 12
```

El ejemplo anterior ha asignado diferentes valores a cada elemento del vector A; si se desea dar el mismo valor a todos los elementos, la notación algorítmica se simplifica con el formato:

```
desde i = 1 hasta 5 hacer
  A[i] ← 8
fin_desde
```

donde A[i] tomará los valores numéricos A[1] = 8, A[2] = 8, . . . , A[5] = 8. Se puede utilizar también la notación

```
A ← 8
```

para indicar la asignación de un mismo valor a cada elemento de un vector A. Esta notación se considerará con mucho cuidado para evitar confusión con posibles variables simples numéricas de igual nombre (A).

6.3.2. Lectura/escritura de datos

La lectura/escritura de datos en *arrays* u operaciones de entrada/salida normalmente se realizan con estructuras repetitivas, aunque puede también hacerse con estructuras selectivas. Las instrucciones simples de lectura/escritura se representarán como

leer(A)	lectura del vector A
escribir(A)	escritura del vector A
leer(V[5])	leer el elemento V[5] del vector V

6.3.3. Acceso secuencial al vector (recorrido)

Se puede acceder a los elementos de un vector para introducir datos (*leer*) en él o bien para visualizar su contenido (*escribir*). A la operación de efectuar una acción general sobre todos los elementos de un vector se la denomina *recorrido* del vector. Estas operaciones se realizan utilizando estructuras repetitivas, cuyas variables de control (por ejemplo, I) se utilizan como subíndices del vector (por ejemplo, S[I]. El incremento del contador del bucle producirá el tratamiento sucesivo de los elementos del vector.

Ejemplo 6.1

Lectura de veinte valores enteros de un vector denominado F.

Procedimiento 1:

```
algoritmo Leer_vector

tipo
  array[1..20] de entero: final
var
  final: f

inicio
  desde i ← 1 hasta 20 hacer
    leer(F[i])
  fin_desde
fin
```

La lectura de veinte valores sucesivos desde el teclado rellenará de valores el vector F, comenzando con el elemento F(1) y terminando en F(20). Si se cambian los límites inferior y superior (por ejemplo, 5 y 10), el bucle de lectura sería:

```
desde i ← 5 hasta 10 hacer
  leer(F[i])
fin_desde
```

Procedimiento 2:

Los elementos del vector se pueden leer también con bucles **mientras** o **repetir**.

```
i ← 1
mientras i <= 20 hacer
  leer(F[i])
  i ← i + 1
fin_mientras
```

o bien:

```
i ← 1
repetir
  leer(F[i])
  i ← i + 1
hasta_que i > 20
```

La salida o escritura de vectores se representa de un modo similar. La estructura

```
desde i ← 1 hasta 20 hacer
  escribir(F[i])
fin_desde
```

visualiza todo el vector completo (un elemento en cada línea independiente).

Ejemplo 6.2

Este ejemplo procesa un array PUNTOS, *realizando las siguientes operaciones; a) lectura del array, b) cálculo de la suma de los valores del array, c) cálculo de la media de los valores.*

El *array* lo denominaremos PUNTOS; el límite superior del rango se *fijará* mediante la constante LIMITE, a la que se asigne el valor 40, y el límite inferior lo consideraremos 1.

```
algoritmo Media_puntos

const
  LIMITE = 40
tipo
  array[1..LIMITE] de real: PUNTUACION
var
  PUNTUACION: PUNTOS
  real: suma, media
  entero: i
```

```
inicio
  suma ← 0
  escribir('Datos del array')
  desde i ← 1 hasta LIMITE hacer
    leer(PUNTOS[i])
    suma ← suma + PUNTOS[i]
  fin_desde
  media ← suma/LIMITE
  escribir('La media es', media)
fin
```

Se podría ampliar el ejemplo, en el sentido de visualizar los elementos del *array*, cuyo valor es superior a la media. Mediante una estructura **desde** se podría realizar la operación, añadiéndole al algoritmo anterior.

```
escribir('Elementos del array superior a la media')
desde i ← 1 hasta LIMITE hacer
  si PUNTOS[i] > media entonces
    escribir(PUNTOS[i])
  fin_si
fin_desde
```

Ejemplo 6.3

Calcular la media de las estaturas de una clase. Deducir cuántos son más altos que la media y cuántos son más bajos que dicha media (Figura 6.2).

Tabla de variables

n	número de estudiantes de la clase	: entera
H[1]...H[n]	estatura de los n alumnos	: real
i	contador de alumnos	: entera
MEDIA	media de estaturas	: real
ALTOS	alumnos de estatura mayor que la media	: entera
BAJOS	alumnos de estatura menor que la media	: entera
SUMA	totalizador de estaturas	: real

6.3.4. Actualización de un vector

La operación de actualizar un vector puede constar a su vez de tres operaciones elementales:

añadir elementos,
insertar elementos,
borrar elementos,

Se denomina *añadir datos* a un vector la operación de añadir un nuevo elemento al final del vector. La única condición necesaria para esta operación consistirá en la comprobación de espacio de memoria suficiente para el nuevo elemento; dicho de otro modo, que el vector no contenga todos los elementos con que fue definido al principio del programa.

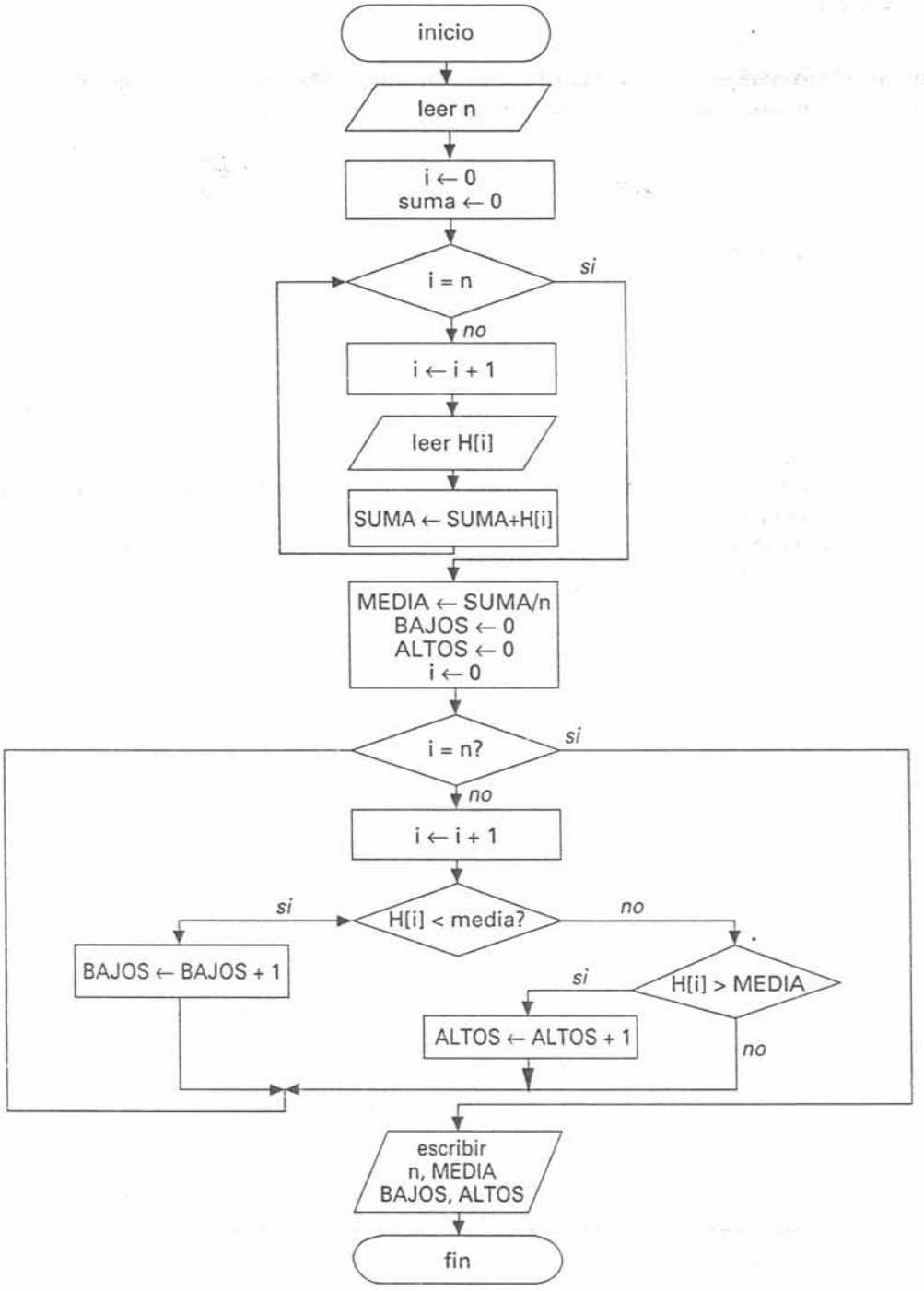


Figura 6.2. Diagrama de flujo para el cálculo de la estatura media de una clase.

Ejemplo 6.4

Un array TOTAL *se ha dimensionado a seis elementos, pero sólo se le han asignado cuatro valores a los elementos* TOTAL[1], TOTAL[2], TOTAL[3] *y* TOTAL[4]. *Se podrán añadir dos elementos más con una simple acción de asignación.*

```
TOTAL[5] ← 14
TOTAL[6] ← 12
```

La *operación de insertar un elemento* consiste en introducir dicho elemento en el interior del vector. En este caso se necesita un desplazamiento previo hacia abajo para colocar el elemento nuevo en su posición relativa.

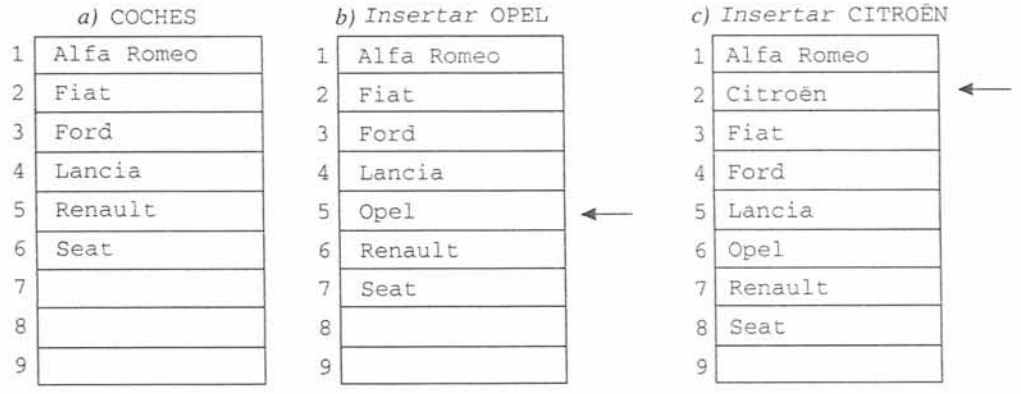
Ejemplo 6.5

Se tiene un array COCHES[2] *de nueve elementos que contiene siete marcas de automóviles en orden alfabético y se desea insertar dos nuevas marcas: OPEL y CITROËN.*

Como *Opel* está comprendido entre *Lancia* y *Renault*, se deberán desplazar hacia abajo los elementos 5 y 6, que pasarán a ocupar la posición relativa 6 y 7. Posteriormente debe realizarse la operación con *Citroën*, que ocupará la posición 2.

El algoritmo que realiza esta operación para un vector de *n* elementos es el siguiente, suponiendo que haya espacio suficiente en el vector.

```
1. //Calcular la posicion del elemento a insertar (por ejemplo, P)
2. //Inicializar contador de inserciones i ← n
3. mientras i >= P hacer
     {transferir el elemento actual i-esimo hacia abajo, a la posicion i + 1}
     COCHES[i + 1] ← COCHES[i]
     //decrementar contador
     i ← i - 1
     fin_mientras
4. //insertar el elemento en la posicion P
     COCHES[P] ← 'nuevo elemento'
5. //actualizar el contador de elementos del vector
6. n ← n + 1
7. fin
```



[2] En Latinoamérica, el término español COCHE no se suele utilizar para determinar un automóvil, en su lugar se suele emplear el término CARRO.

Si se deseara realizar más inserciones, habría que incluir una estructura de decisión **si-entonces** para preguntar si se van a realizar más inserciones.

La operación de borrar un elemento al final del vector no presenta ningún problema; el borrado de un elemento del interior del vector provoca el movimiento hacia arriba de los elementos inferiores a él para reorganizar el vector.

El algoritmo de borrado del elemento *j*-ésimo del vector COCHES es el siguiente:

```
algoritmo Borrado
inicio
  //se utilizara una variable auxiliar -AUX- que contendra el valor del elemento
  que se desea borrar
  AUX ← COCHES[j]
  desde i ← j hasta N - 1 hacer
    //llevar elemento j + 1 hacia arriba
    COCHES[i] ← COCHES[i + 1]
  fin_desde
  //actualizar contador de elementos
  //ahora tendra un elemento menos, N - 1
  N ← N - 1
fin
```

6.4. ARRAYS DE VARIAS DIMENSIONES

Los vectores examinados hasta ahora se denominan arrays unidimensionales y en ellos cada elemento se define o referencia por un índice o subíndice. Estos vectores son elementos de datos escritos en una secuencia. Sin embargo, existen grupos de datos que son representados mejor en forma de tabla o matriz con dos o más subíndices. Ejemplos típicos de tablas o matrices son: tablas de distancias kilométricas entre ciudades, cuadros horarios de trenes o aviones, informes de ventas periódicas (mes/unidades vendidas o bien mes/ventas totales), etc. Se pueden definir *tablas* o *matrices* como *arrays multidimensionales*, cuyos elementos se pueden referenciar por dos, tres o más subíndices. Los *arrays* no unidimensionales los dividiremos en dos grandes grupos:

arrays bidimensionales	(2 dimensiones)
arrays multidimensionales	(3 o más dimensiones)

6.4.1. Arrays bidimensionales (tablas/matrices)

El *array bidimensional* se puede considerar como un vector de vectores. Es, por consiguiente, un conjunto de elementos, todos del mismo tipo, en el cual el orden de los componentes es significativo y en el que se necesita especificar dos subíndices para poder identificar cada elemento del *array*.

Si se visualiza un *array* unidimensional, se puede considerar como una columna de datos; un *array* bidimensional es un grupo de columnas, como se ilustra en la Figura 6.3.

El diagrama representa una tabla o matriz de treinta elementos (5 × 6) con 5 filas y 6 columnas. Como en un vector de treinta elementos, cada uno de ellos tiene el mismo nombre. Sin embargo, un subíndice no es suficiente para especificar un elemento de un *array* bidimensional; por ejemplo, si el nombre del *array* es M, no se puede indicar M[3], ya que no sabemos si es el tercer elemento de la primera fila o de la primera columna. Para evitar la ambigüedad, los elementos de un *array* bidi-

su valor será un código representativo de cada figura del juego

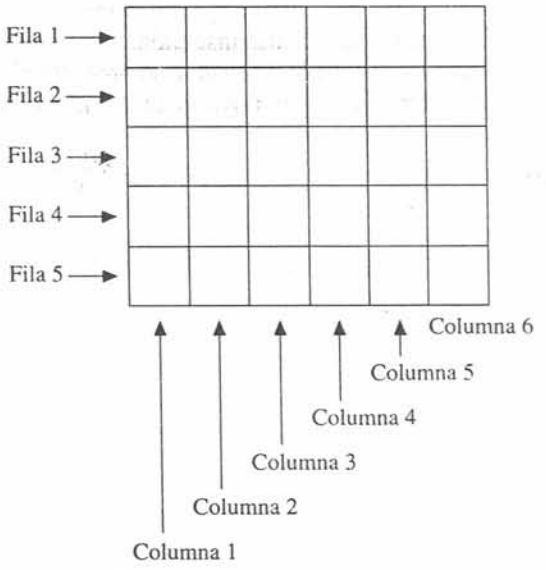


Figura 6.3. Array bidimensional.

mensional se referencian con dos subíndices: el primer subíndice se refiere a la *fila* y el segundo subíndice se refiere a la columna. Por consiguiente, `M[2, 3]` se refiere al elemento de la segunda fila, tercera columna. En nuestra tabla ejemplo `M[2, 3]` contiene el valor 18.

Un *array bidimensional* `M`, también denominado *matriz* (términos matemáticos) o *tabla* (términos financieros), se considera que tiene dos dimensiones (una dimensión por cada subíndice) y necesita un valor para cada subíndice y poder identificar un elemento individual. En notación estándar, normalmente el primer subíndice se refiere a la fila del *array*, mientras que el segundo subíndice se refiere a la columna del *array*. Es decir, `B[I, J]` es el elemento de `B` que ocupa la I^a fila y la J^a columna, como se indica en la Figura 6.4.

El elemento `B[I,J]` también se puede representar por $B_{I,J}$. Más formalmente en notación algorítmica, el *array* `B` con elementos del tipo `T` (numéricos, alfanuméricos, etc.) con *subíndices fila* que varían en el rango de `1` a `M` y *subíndices columna* en el rango de `1` a `N` es:

```
B[1:M, 1:N] = {B[I, J]}
```

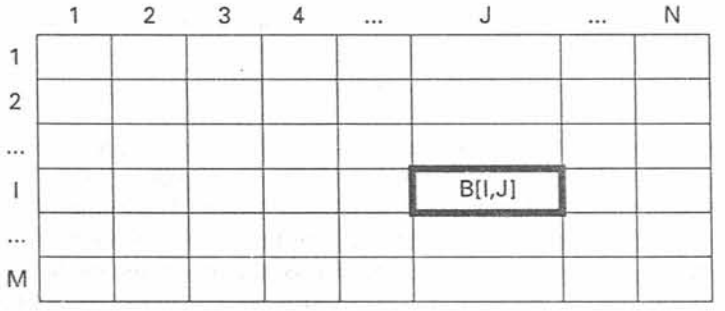


Figura 6.4. Elemento `B[I, J]` del array `B`.

donde `I = 1, ..., M` *o bien:* `1 <= I <= M`
`J = 1, ..., N` `1 <= J <= N`

cada elemento `B[I, J]` *es de tipo* `T`.

El *array* `B` se dice que tiene `M` por `N` elementos. Existen `N` elementos en cada fila y `M` elementos en cada columna `(M * N)`.

Los *arrays de dos dimensiones* son muy frecuentes: las calificaciones de los estudiantes de una clase se almacenan en una tabla `NOTAS` de dimensiones `NOTAS[20, 5]`, donde 20 es el número de alumnos y 5 el número de asignaturas. El valor del subíndice `I` debe estar entre 1 y 20, y el de `J` entre 1 y 5. Los subíndices pueden ser variables o expresiones numéricas, `NOTAS(M, 4)` y en ellos el subíndice fila irá de 1 a `M` y el de columnas de 1 a `N`.

En general, se considera que un *array* bidimensional comienza sus subíndices en 0 o en 1 (según el lenguaje de programación, 0 en BASIC, 1 en FORTRAN), pero pueden tener límites seleccionados por el usuario durante la codificación del algoritmo. En general, el *array* bidimensional `B` con su primer subíndice, variando desde un límite inferior `L` (inferior, *low*) a un límite superior `U` (superior, *up*). En notación algorítmica:

```
B[L1:U1, L2:U2] = {B[I, J]}
```

donde `L1 <= I <= U1`
`L2 <= J <= U2`

cada elemento `B[I, J]` *es del tipo* `T`.

El número de elementos de una fila de `B` es `U1 - L1 + 1` y el número de elementos en una columna de `B` es `U2 - L2 + 1`. Por consiguiente, el número total de elementos del *array* `B` es `(U2 - L2 + 1) * (U1 - L1 + 1)`.

Ejemplo 6.6

La matriz `T` *representa una tabla de notaciones de saltos de altura (primer salto), donde las filas representan el nombre del atleta y las columnas las diferentes alturas saltadas por el atleta. Los símbolos almacenados en la tabla son:* `x`, *salto válido;* `0`, *salto nulo o no intentado.*

Columna / Fila	T					
	2.00	2.10	2.20	2.30	2.35	2.40
Garcia	x	0	x	x	x	0
Perez	0	x	x	0	x	0
Gil	0	0	0	0	0	0
Mortimer	0	0	0	x	x	x

Ejemplo 6.7

Un ejemplo típico de un array bidimensional es un tablero de ajedrez. Se puede representar cada posición o casilla del tablero mediante un array, en el que cada elemento es una casilla y en el que su valor será un código representativo de cada figura del juego.

Figura 6.5. Array típico, «tablero de ajedrez».

Los diferentes elementos serán:

```
elemento[i, j] = 0      si no hay nada en la casilla [i, j]
elemento[i, j] = 1      si el cuadro (casilla) contiene un peón blanco
elemento[i, j] = 2      un caballo blanco
elemento[i, j] = 3      un alfil blanco
elemento[i, j] = 4      una torre blanca
elemento[i, j] = 5      una reina blanca
elemento[i, j] = 6      un rey blanco
```

y los correspondientes números negativos para las piezas negras.

Ejemplo 6.8

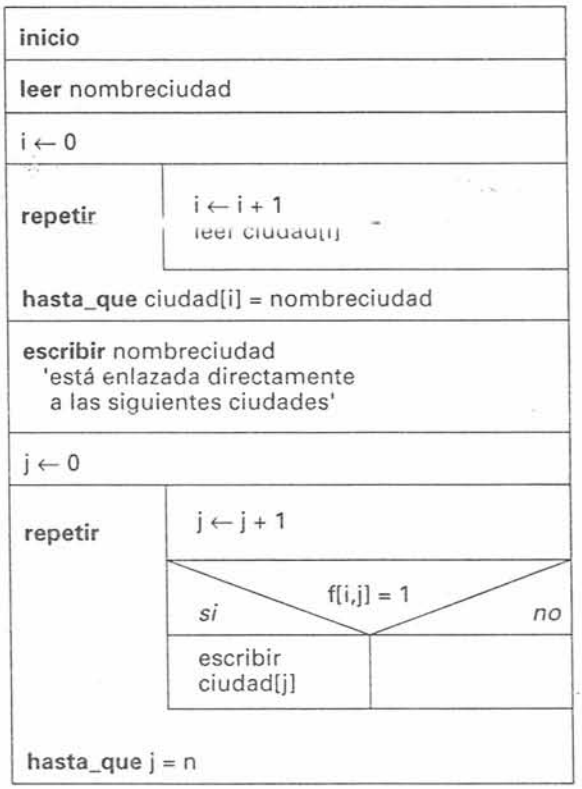
Supongamos que se dispone de un mapa de ferrocarriles y los nombres de las estaciones (ciudades) están en un vector denominado `'ciudad'`. *El diagrama N-S siguiente lee un nombre de una ciudad e imprime los nombres de las ciudades con las que está enlazada directamente. El array* `f` *puede tener los siguientes valores:*

```
f[i, j] = 1     si existe enlace entre las ciudades i y j
f[i, j] = 0     no existe enlace
```

NOTA: El *array* `f` resume la información de la estructura de la red de enlaces.

6.5. ARRAYS MULTIDIMENSIONALES

Un *array* puede ser definido de tres dimensiones, cuatro dimensiones, hasta de *n-dimensiones*. Los conceptos de rango de subíndices y número de elementos se pueden ampliar directamente desde *arrays* de una y dos dimensiones a estos *arrays* de orden más alto. En general, un *array* de *n-dimensiones* requiere que los valores de los *n* subíndices puedan ser especificados a fin de identificar un elemento individual del *array*. Si cada componente de un *array* tiene *n* subíndices, el *array* se dice que es sólo de *n-dimensiones*.



El *array* A de *n-dimensiones* se puede identificar como

```
A[L1:U1, L2:U2, ... Ln:Un]
```

y un elemento individual del *array* se puede especificar por

```
A[I1, I2, ..., In]
```

donde cada subíndice I_k está dentro de los límites adecuados

```
Lk <= Ik <= Uk
```

donde

```
k = 1, 2, ..., n
```

El número total de elementos de un *array* A es:

```
Π (Uk - Lk - 1)        Π (simbolo del producto)
```

que se puede escribir alternativamente como

```
(U1 - L1 + 1)*(U2 - L2 + 1)*...*(UN - LN + 1)
```

Si los límites inferiores comenzasen en 1, el *array* se representaría por

```
A[ S1, S2, ..., Sn ]  y un elemento individual  A(k1,k2,...,kn)
```

donde

```
1 <= k1 <= S1
1 <= k2 <= S2

.
1 <= kn <= Sn
```

Ejemplo 6.9

Un array de tres dimensiones puede ser uno que contenga los datos relativos al número de estudiantes de la universidad ALFA *de acuerdo a los siguientes criterios:*

- *cursos* (primero a quinto),
- *sexo* (varón/hembra),
- *diez facultades*.

El *array* ALFA puede ser de dimensiones 5 por 2 por 10 (alternativamente $10 \times 5 \times 2$ o $10 \times 2 \times 5$, $2 \times 5 \times 10$, etc.). La Figura 6.6 representa el *array* ALFA.

El valor de elemento ALFA[I, J, K] es el número de estudiantes del curso I de sexo J de la facultad K. Para ser válido I, debe ser 1, 2, 3, 4 o 5; J debe ser 1 o 2; K debe estar comprendida entre 1 y 10 inclusive.

Ejemplo 6.10

Otro array de tres dimensiones puede ser PASAJE *que representa el estado actual del sistema de reserva de una línea aérea, donde*

i = 1, 2, ..., 10	representa el número de vuelo,
j = 1, 2, ..., 60	representa la fila del avión,
k = 1, 2, ..., 4	representa el asiento dentro de la fila.

Entonces

```
pasaje[i, j, k] = 0    asiento libre
pasaje[i, j, k] = 1    asiento ocupado
```

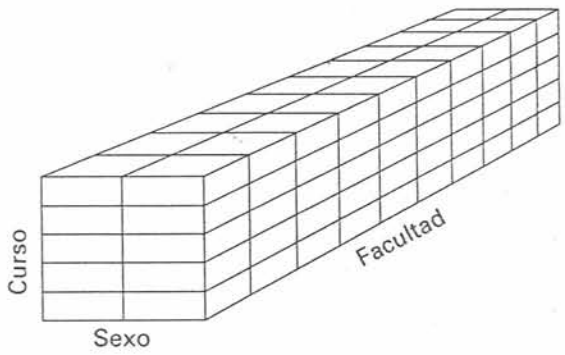


Figura 6.6. Array de tres dimensiones.

6.6. ALMACENAMIENTO DE ARRAYS EN MEMORIA

Las representaciones gráficas de los diferentes *arrays* de una o dos dimensiones se recogen en la Figura 6.7

Debido a la importancia de los *arrays*, casi todos los lenguajes de programación de alto nivel proporcionan medios eficaces para almacenar y acceder a los elementos de los *arrays*, de modo que el programador no tenga que preocuparse sobre los detalles específicos de almacenamiento. Sin embargo, el almacenamiento en la computadora está dispuesto fundamentalmente en secuencia contigua, de modo que cada acceso a una matriz o tabla la máquina debe realizar la tarea de convertir la posición dentro del *array* en una posición perteneciente a una línea.

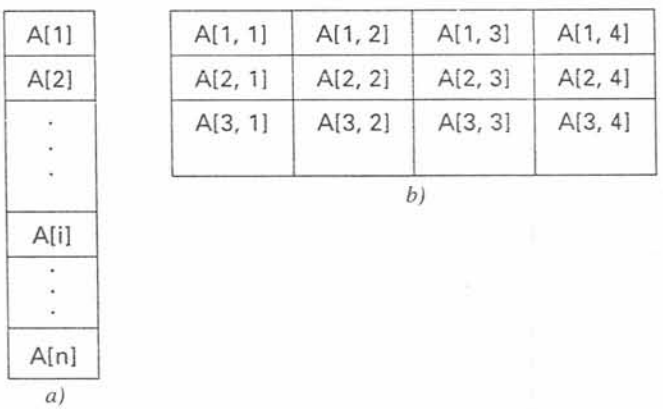
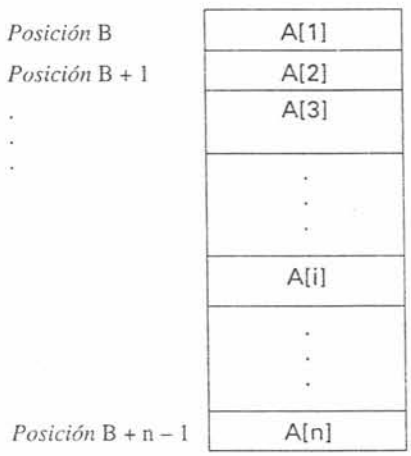


Figura 6.7. Arrays de una y dos dimensiones.

6.6.1. Almacenamiento de un vector

El almacenamiento de un vector en memoria se realiza en celdas o posiciones secuenciales. Así, en el caso de un vector `A` con un subíndice de rango `1` a `n`,



Si cada elemento del *array* ocupa `S` bytes (1 *byte* = 8 bits), y `B` es la dirección inicial de la memoria central de la computadora —*posición o dirección base*—, la dirección inicial del elemento `I`-ésimo sería:

```
B + (I - 1)* S
```

NOTA. Si el límite inferior no es igual a 1, considérese el *array* declarado como `N[4:10]`; la dirección inicial de `N[6]` es:

```
B + (6 - 4) * S
```

En general, el elemento `N[I]` de un *array* definido como `N(L:U)` tiene la dirección inicial

```
B + (I - L) * S
```

6.6.2. Almacenamiento de arrays multidimensionales

Debido a que la memoria de la computadora es lineal, un *array* multidimensional debe estar linealizado para su disposición en el almacenamiento.

Los lenguajes de programación pueden almacenar los *arrays* en memoria de dos formas: *orden de fila mayor* y *orden de columna mayor*.

El medio más natural en que se leen y almacenan los *arrays* en la mayoría de los compiladores es el denominado *orden de fila mayor* (véase Figura 6.8). Por ejemplo, si un *array* es `B[2, 3]`, el orden de los elementos en la memoria es:

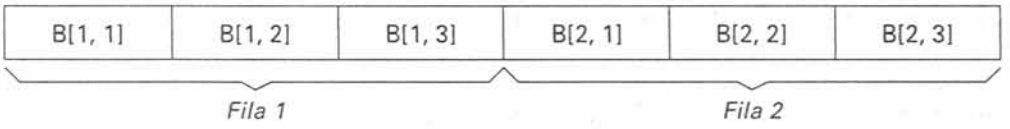


Figura 6.8. Orden de fila mayor.

C, BASIC, COBOL, PL/1 y Pascal *almacenan los elementos por filas*.

FORTRAN emplea el *orden de columna mayor,* en el que las entradas de la primera fila vienen primero.

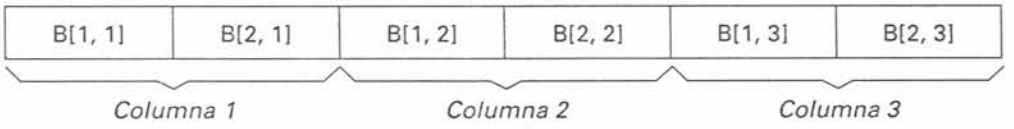


Figura 6.9. Orden de columna mayor.

De modo general, el compilador del lenguaje de alto nivel debe ser capaz de calcular con un índice `[i, j]` la posición del elemento correspondiente.

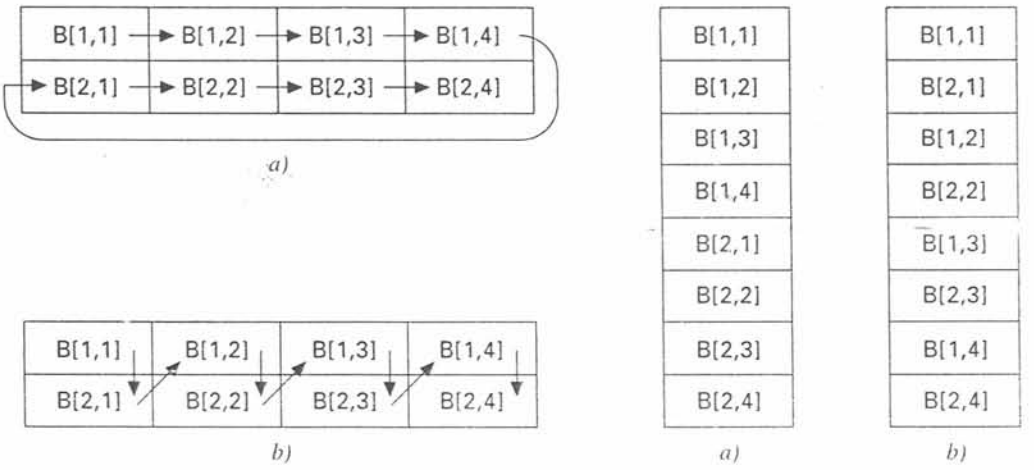


Figura 6.10. Almacenamiento de una matriz: a) por filas, b) por columnas.

En un *array* en orden de fila mayor, cuyos subíndices máximos sean `m` y `n` (`m`, filas; `n`, columnas), la posición `p` del elemento `[i, j]` con relación al primer elemento es:

```
p = n(i - 1) + j
```

Para calcular la dirección real del elemento `[i, j]` se añade `p` a la posición del primer elemento y se resta 1. La representación gráfica del almacenamiento de una tabla o matriz `B[2, 4]` y `C[2, 4]`.

En el caso de un *array* de tres dimensiones, supongamos un *array tridimensional* `A[2, 4, 3]`. La Figura 6.11 representa el *array* `A` y su almacenamiento en memoria.

En orden a determinar si es más ventajoso almacenar un *array* en orden de columna mayor o en orden de fila mayor, es necesario conocer en qué orden se referencian los elementos del *array*. De hecho, los lenguajes de programación no le dan opción al programador para que elija una técnica de almacenamiento.

Consideremos un ejemplo del cálculo del valor medio de los elementos de un *array* `A` de `50` por `300` elementos, `A[50, 300]`. Los algoritmos de almacenamiento respectivos serán:

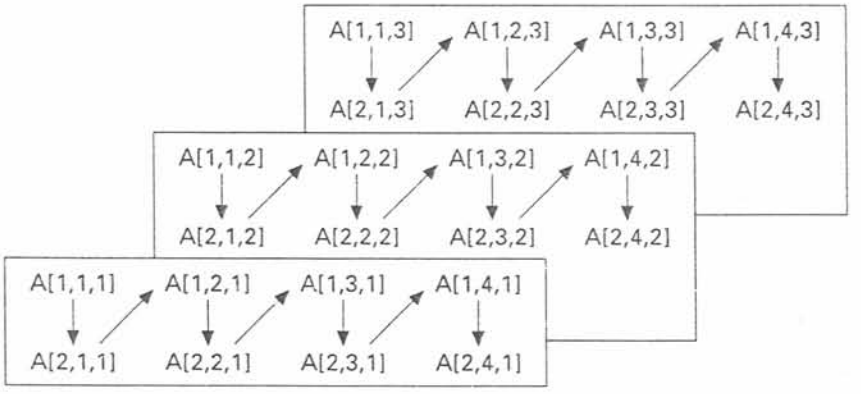


Figura 6.11. Almacenamiento de una matriz `A[2,4,3]` por columnas.

Almacenamiento por columna mayor:

```
total ← 0
desde j ← 1 hasta 300 hacer
  desde i ← 1 hasta 50 hacer
    total ← total + a[i, j]
  fin_desde
fin_desde
media ← total/(300 * 50)
```

Almacenamiento por fila mayor:

```
total ← 0
desde i ← 1 hasta 50 hacer
  desde j ← 1 hasta 300 hacer
    total ← total + a[i, j]
  fin_desde
fin_desde
media ← total/(300 * 50)
```

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

6.1. Escribir un algoritmo que permita calcular el cuadrado de los cien primeros números enteros y a continuación escribir una tabla que contenga dichos cien números cuadrados.

El problema consta de dos partes:

1. Cálculo de los cuadrados y almacenamiento en la tabla.
2. Escritura de la tabla T, T[1], T[2], ..., T[100] que contiene los siguientes valores:

```
T[1] = 1 * 1 = 1
T[2] = 2 * 2 = 4
T[3] = 3 * 3 = 9
...
```

El algoritmo se puede construir con estructuras de decisión o alternativas, o bien con estructuras repetitivas. En nuestro caso utilizaremos una estructura repetitiva **desde**.

```
algoritmo Cuadrados

tipo
  array[1..100] de entero: tabla
var
  tabla: T
  entero: I, C

inicio
  desde I ← 1 hasta 100 hacer
    C ← I * I
    T[I] ← C
  fin_desde
  desde I ← 1 hasta 100 hacer
    escribir(T[I])
  fin_desde
fin
```

6.2. Se tienen N temperaturas. Se desea calcular su media y determinar entre todas ellas cuáles son superiores o iguales a esa media.

Análisis

En un primer momento se leen los datos y se almacenan en un vector (array unidimensional) TEMP[1:N].
A continuación se van realizando las sumas sucesivas a fin de obtener la media.
Por último, con un bucle de lectura de la tabla se va comparando cada elemento de la misma con la media y luego, mediante un contador, se calcula el número de temperaturas igual o superior a la media.

Tabla de variables

N	Número de elementos del vector o tabla
TEMP	Vector o tabla de temperatura
SUMA	Sumas sucesivas de las temperaturas
MEDIA	Media de la tabla
C	Contador de temperaturas >= MEDIA

Pseudocódigo

```
algoritmo Temperaturas

const
  N = 100
tipo
  array[1..N] de real: temperatura
var
  temperatura: Temp
  entero: I, C
  real: suma, media

inicio
  suma ← 0
  media ← 0
  C ← 0

  desde I ← 1 hasta N hacer
    leer(Temp[I])
    suma ← suma + Temp[I]
  fin_desde
  media ← suma/N

  desde I ← 1 hasta N hacer
    si Temp[I] >= media entonces
      C ← C + 1
      escribir(Temp[I])
    fin_si
  fin_desde
  escribir('La media es:', media)
  escribir('El total de temperaturas >=', media, 'es:', C)
fin
```

6.3. Escribir el algoritmo que permita sumar el número de elementos positivos y el de negativos de una tabla T.

Sea una tabla de dimensiones M, N leídas desde el teclado.

Tabla de variables

```
I, J, M, N: entero
SP: real
SN: real
```

Pseudocódigo

```
algoritmo Suma_resta

const
  M = 50
  N = 20
tipo
  array[1..M, 1..N] de real: tabla
var
  tabla: T
  entero: I, J
  real: SP, SN
inicio
  SP ← 0
  SN ← 0
  desde I ← 1 hasta M hacer
    desde J ← 1 hasta N hacer
      si T[I, J] >= 0 entonces
        SP ← SP + T[I, J]
      si_no
        SN ← SN + T[I, J]
      fin_si
    fin_desde
  fin_desde
  escribir('Suma de positivos', SP, 'de negativos', SN)
fin
```

6.4. Inicializar una matriz de dos dimensiones con un valor constante dado K.

Análisis

El algoritmo debe tratar de asignar la constante K a todos los elementos de la matriz A[M, N].

```
A[1, 1] = K        A[1, 2] = K ... A[1, N] = K
.
.
A[M, 1] = K        A[M, 2] = K ... A[M, N] = K
```

Dado que es una matriz de dos dimensiones, se necesitan dos bucles anidados para recorrer todos sus elementos.

Pseudocódigo

```
algoritmo Inicializa_matriz
inicio
  k ← 0
  desde I ← 1 hasta M hacer
    desde J ← 1 hasta N hacer
      A[I, J] ← K
    fin_desde
  fin_desde
fin
```

6.5. Realizar la suma de dos matrices bidimensionales.

Análisis

Las matrices `A[I, J]`, `B[I, J]` para que se puedan sumar deben tener las mismas dimensiones. La matriz suma `S[I, J]` tendrá iguales dimensiones y cada elemento será la suma de las correspondientes matrices `A` y `B`. Es decir,

```
S[I, J] = A[I, J] + B[I, J]
```

Dado que se trata de matrices de dos dimensiones, el proceso se realizará con dos bucles anidados.

Pseudocódigo

```
algoritmo Suma_matrices
inicio
  desde I ← 1 hasta N hacer
    desde J ← 1 hasta M hacer
      S[I, J] ← A[I, J] + B[I, J]
    fin_desde
  fin_desde
fin
```

6.6. Se tiene una tabla `T` de dos dimensiones. Calcular la suma de sus elementos.

Supongamos sean `M` y `N` las dimensiones de `T`, y que sus componentes son números reales.

Tabla de variables

`I`	Contador de filas
`J`	Contador de columnas
`M`	Número de filas de la tabla `T`
`N`	Número de columnas de la tabla `T`
`T`	Tabla
`S`	Suma de los elementos de la tabla
`I, J, M, N`	Enteros
`T, S`	Reales

Pseudocódigo

```
algoritmo Suma_elementos

const
  M = 50
  N = 20
tipo
  array[1..M, 1..N] de real: tabla
var
  entero: I, J
  tabla: T
  real: S

inicio
  desde I ← 1 hasta M hacer
    desde J ← 1 hasta N hacer
      leer(T[I, J])
    fin_desde
  fin_desde
```

```
  S ← 0    //inicializacion de la suma S
  desde I ← 1 a M hacer
    desde J ← 1 a N hacer
      S ← S + T[I, J]
    fin_desde
  fin_desde
  escribir('La suma de los elementos de la matriz =', S)
fin
```

6.7. Realizar la búsqueda de un determinado nombre en una lista de nombres, de modo que el algoritmo imprima los siguientes mensajes según el resultado:

`'Nombre encontrado'`	*si el nombre está en la lista*
`'Nombre no existe'`	*si el nombre no está en la lista*

Se recurrirá en este ejercicio a utilizar un interruptor `SW`, de modo que si `SW` = falso, el nombre no existe en la lista, y si `SW` = cierto, el nombre existe en la lista (o bien, caso de no existir la posibilidad de variables lógicas, definir `SW` como `SW = 0`, si es falso, y `SW = 1`, si es verdadero o cierto).

Método 1:

```
algoritmo Busqueda
const
  N = 50
tipo
  array[1..N] de cadena: listas
var
  listas: l
  logico: SW
  cadena: nombre
  entero: y
inicio
  SW ← falso
  leer(nombre)
  desde I ← 1 hasta N hacer
    si l[I] = nombre entonces
      SW ← verdadero
    fin_si
  fin_desde
  si SW entonces
    escribir('Encontrado')
  si_no
    escribir('No existe', nombre)
  fin_si
fin
```

Método 2:

```
algoritmo Busqueda
const
  N = 50
tipo
  array[1..N] de cadena: listas
var
  listas: l
  entero: SW
  cadena: nombre
  entero: y
```

```
inicio
  SW ← 0
  leer(nombre)
  desde I ← 1 hasta N hacer
    si 1[I] = nombre entonces
      SW ← 1
    fin_si
  fin_desde
  si SW = 1 entonces
    escribir('Encontrado')
  si_no
    escribir('No existe', nombre)
  fin_si
fin
```

6.8. Se desea permutar las filas I y J de una matriz (array) de dos dimensiones (M * N) : M filas, N columnas.

Análisis

La tabla T(M * N) se puede representar por:

```
T[1, 1]   T[1, 2]   T[1, 3]   ...      T[1, N]
T[2, 1]   T[2, 2]   T[2, 3]   ...      T[2, N]
.
.
T[M, 1]   T[M, 2]   T[M, 3]   ...      T[M, N]
```

El sistema para permutar globalmente toda la fila I con la fila J se debe realizar permutando uno a uno el contenido de los elementos T[I, K] y T[J, K].
Para intercambiar entre sí los valores de dos variables, recordemos que se necesitaba una variable auxiliar AUXI.
Así, para el caso de las variables A y B:

```
AUXI ← A
  A  ← B
  B  ← AUXI
```

En el caso de nuestro ejercicio, para intercambiar los valores T[I, K] y T[J, K] se debe utilizar el algoritmo:

```
AUXI    ← T[I, K]
T[I, K] ← T[J, K]
T[J, K] ← AUX
```

Tabla de variables

```
I, J, K, M, N     entero
AUXI              entero
T                 tabla
```

Pseudocódigo

```
algoritmo Intercambio

const
  M = 50
  N = 30
tipo
  array[1..M, 1.. N] de entero: tabla
```

```
var
  tabla: T
  entero: Auxi, I, J, K

inicio

//En este ejercicio y dado que ya se han realizado muchos ejemplos de lectura de
//arrays con dos bucles desde, la operacion de lectura completa del array se
//representara con la instruccion de leertabla (T)

  leertabla(T)
  //Deducir I, J a intercambiar
  leer(I, J)
  desde K ← 1 hasta N hacer
    Auxi ← T[I, K]
    T[I, K] ← T[J, K]
    T[J, K] ← Auxi
  fin_desde
  //Escritura del nuevo array
  escribirtabla(T)
fin
```

EJERCICIOS

6.1. Algoritmo que nos permita calcular la desviación estándar (SIGMA) de una lista de N números (N < = 15). Sabiendo que

$$\text{Desviación} = \sqrt{\frac{\sum_{i=1}^{n}(x_i - m)^2}{n-1}}$$

```
algoritmo Calcular_desviacion
  tipo
    array[1..15] de real : arr
  var
    arr    : x
    entero : n

inicio
  llamar_a leer_array(x, n)
  escribir('La desviacion estandar es ',desviacion(x, n))
fin

procedimiento leer_array(S arr:x; S entero:n)
  var
    entero : i
  inicio
    repetir
      escribir('Diga numero de elementos de la lista ')
      leer(n)
    hasta_que n <= 15
    escribir('Deme los elementos:')
    desde i ← 1 hasta n hacer
      leer(x[i])
    fin_desde
  fin_procedimiento
```

```
real funcion desviacion(E arr : x;  E entero : n)
  var
    real    : suma, xm, sigma
    entero : i
inicio
  suma ← 0
  desde i ← 1 hasta n hacer
    suma ← suma + x[i]
    fin_desde
    xm ← suma / n
    sigma ← 0
    desde i ← 1 hasta n hacer
        sigma ← sigma + cuadrado (x[i] - xm)
    fin_desde
    devolver(raiz2 (sigma / (n - 1)))
fin_funcion
```

6.2. Utilizando arrays, escribir un algoritmo que visualice un cuadrado mágico de orden impar n, comprendido entre 3 y 11. El usuario debe elegir el valor de n.

Un cuadrado mágico se compone de números enteros comprendidos entre 1 y n. La suma de los números que figuran en cada fila, columna y diagonal son iguales.

Ejemplo:

8	1	6
3	5	7
4	9	2

Un método de generación consiste en situar el número 1 en el centro de la primera fila, el número siguiente en la casilla situada por encima y a la derecha y así sucesivamente. El cuadrado es cíclico, la línea encima de la primera es de hecho la última y la columna a la derecha de la última es la primera. En el caso de que el número generado caiga en una casilla ocupada, se elige la casilla que se encuentre debajo del número que acaba de ser situado.

```
algoritmo Cuadrado_magico
  var  entero : n
inicio
  repetir
    escribir('Deme las dimensiones del cuadrado (3 a 11) ')
    leer(n)
  hasta_que (n mod 2 <>0) y (n <= 11) y (n >= 3)
  dibujarcuadrado(n)
fin

procedimiento dibujarcuadrado(E entero:n)
  var  array[1..11,1..11] de entero : a
       entero                       : i,j,c
inicio
  i ← 2
  j ← n div 2

  desde c ← 1 hasta n * n hacer
    i ← i - 1
    j ← j + 1
    si j > n entonces
       j ← 1
    fin_si
```

```
    si i < 1 entonces
       i ← n
    fin_si
    a[i,j] ← c
    si c mod n = 0 entonces
      j ← j - 1
      i ← i + 2
    fin_si
  fin_desde

  desde i ← 1 hasta n hacer
    desde j ← 1 hasta n hacer
      escribir(a[i,j])
      //al codificar esta instruccion en un lenguaje sera conveniente utilizar el
      //parametro correspondiente de "no avance de linea" en la salida en
      //pantalla o impresora
    fin_desde
    escribir(NL)
    //NL, representa Nueva Linea; es decir, avance de linea
  fin_desde

fin_procedimiento
```

6.3. Obtener un algoritmo que efectúe la multiplicación de dos matrices A, B:

```
A ← m * p elementos
B ← p * n elementos
```

Matriz producto: C ← m * n elementos, tal que

$$C_{i,j} = \sum_{k=1}^{p} a_{i,k} * b_{k,j}$$

```
algoritmo Multiplicar_matrices

  tipo array[1..10,1..10] de real : arr
  var  entero : m, n, p
       arr    : a ,b, c

inicio
  repetir
    escribir('Dimensiones de la 1ª matriz (nº filas nº columnas)')
    leer (m, p)
    escribir('Columnas de la 2ª matriz ')
    leer(n)
  hasta_que (n < = 10) y (m < = 10) y (p < = 10)
  escribir ('Deme elementos de la 1ª matriz')
  llamar_a leer_matriz(a,m,p)
  escribir ('Deme elementos de la 2ª matriz')
  llamar_a leer_matriz(b,p,n)
  llamar_a calcescrproducto(a, b, c, m, p, n)
fin

procedimiento leer_matriz(S arr:matriz;E/entero:filas,columnas)
var entero : i, j
```

```
inicio
  desde i ← 1 hasta filas hacer
    escribir('Fila ',i,':')
    desde j ← 1 hasta columnas hacer
      leer(matriz[i,j])
    fin_desde
    fin_desde
fin_procedimiento

procedimiento calcescrproducto(E arr: a, b, c; E entero: m,p,n)
  var entero : i, j, k

inicio
  desde i ← 1 hasta m hacer
    desde j ← 1 hasta n hacer
      c[i,j] ← 0
      desde k ← 1 hasta p hacer
        c[i,j] ← c[i,j] + a[i,k] * b[k,j]
      fin_desde
        escribir (c[i,j])     //no avanzar linea
      fin_desde
    escribir  (NL)            //avanzar linea, nueva linea
  fin_desde
fin_procedimiento
```

6.4. Algoritmo que triangule una matriz cuadrada y halle su determinante. En las matrices cuadradas el valor del determinante coincide con el producto de los elementos de la diagonal de la matriz triangulada, multiplicado por –1 tantas veces como hayamos intercambiado filas al triangular la matriz.

Proceso de triangulación de una matriz

para i **desde** 1 **a** n - 1 **hacer:**

a) Si el elemento de lugar (i,i) es nulo, intercambiar filas hasta que dicho elemento sea no nulo o agotar los posibles intercambios.

b) A continuación se busca el primer elemento no nulo de la fila i-ésima y, en el caso de existir, se usa para hacer ceros en la columna de abajo.
Sea dicho elemento matriz[i,r]
Multiplicar fila i por matriz[i + 1,r]/matriz[i,r] y restarlo a la i + 1
Multiplicar fila i por matriz[i + 2,r]/matriz[i,r] y restarlo a la i + 2
.....
Multiplicar fila i por matriz[m,r]/matriz[i,r] y restarlo a la m

```
algoritmo Triangulacion_matriz
const m = <expresion>
      n = <expresion>
tipo  array[1..m, 1..n] de real : arr
var arr  : matriz
    real : dt

inicio
  llamar_a leer_matriz(matriz)
  llamar_a triangula(matriz, dt)
  escribir('Determinante= ', dt)
fin
```

```
procedimiento leer_matriz (S arr : matriz)
  var entero: i,j

  inicio
    escribir('Deme los valores para la matriz')
    desde i ← 1 hasta m hacer
      desde j ← 1 hasta n hacer
        leer( matriz[i, j])
      fin_desde
    fin_desde
  fin_procedimiento


procedimiento escribir_matriz (E arr : matriz)
var entero   : i,j
      caracter : c
inicio
  escribir('Matriz ')
  desde i ← 1 hasta m hacer
    desde j ← 1 hasta n hacer
      escribir( matriz[i, j])'// Al codificar, usar el parametro de 'no avance de
                              //linea''
    fin_desde
    Escribir(NL)
  fin_desde
  escribir('Pulse tecla para continuar')
  leer(c)
fin_procedimiento


procedimiento interc(E/S real: a,b)
var real : auxi
inicio
  auxi ← a
  a ← b
  b ← auxi
fin_procedimiento


procedimiento triangula (E arr : matriz; S real:dt)
var entero: signo
    entero: t,r,i,j
    real  : cs
inicio
    signo ← 1
    desde i ← 1 hasta m - 1 hacer
      t ← 1
      si matriz[i, i] = 0 entonces
       repetir
        si matriz[i + t, i] <> 0 entonces
         signo ← signo * (-1)
          desde j ← 1 hasta n hacer
           llamar_a interc(matriz[i,j],matriz[i + t,j])
            fin_desde
             llamar_a escribir_matriz(matriz)
              fin_si
               t ← t + 1
             hasta_que (matriz[i, i] <> 0) o (t = m - i + 1)
           fin_si
```

```
            r ← i - 1
            repetir
            r ← r + 1
          hasta_que (matriz[i, r] <> 0) o (r = n)
          si matriz[i, r] <> 0 entonces
            desde t ← i + 1 hasta m hacer
              si matriz[t, r] <> 0 entonces
                cs ← matriz[t, r]
              desde j ← r hasta n hacer
                matriz[t, j] ← matriz[t, j] - matriz[i, j] * (cs / matriz[i, r])
              fin_desde
            llamar_a escribir_matriz(matriz)
          fin_si
        fin_desde
      fin_si
    fin_desde
    dt ← signo
    desde i ← 1 hasta m hacer
      dt ← dt * matriz[i, i]
    fin_desde
    fin_procedimiento
```

6.5. Determinar los valores de I, J, después de la ejecución de las instrucciones siguientes:

```
var
  I, J: entero
  A: array[1..10] de entero
inicio
  I ← 1
  J ← 2
  A[I] ← J
  A[J] ← I
  A[J + I] ← I + J
  I ← A[I] + A[J]
  A[3] ← 5
  J ← A[I] - A[J]
fin
```

6.6. Escribir el algoritmo que permita obtener el número de elementos positivos de una tabla.

6.7. Rellenar una matriz identidad de 4 por 4.

6.8. Leer una matriz de 3 por 3 elementos y calcular la suma de cada una de sus filas y columnas, dejando dichos resultados en dos vectores, uno de la suma de las filas y otro de las columnas.

6.9. Cálculo de la suma de todos los elementos de un vector, así como la media aritmética.

6.10. Calcular el número de elementos negativos, cero y positivos de un vector dado de sesenta elementos.

6.11. Calcular la suma de los elementos de la diagonal principal de una matriz cuatro por cuatro (4×4).

6.12. Se dispone de una tabla T de cincuenta números reales distintos de cero. Crear una nueva tabla en la que todos sus elementos resulten de dividir los elementos de la tabla T por el elemento T[K], siendo K un valor dado.

6.13. Se dispone de una lista (vector) de N elementos. Se desea diseñar un algoritmo que permita insertar el valor x en el lugar k-ésimo de la mencionada lista.

6.14. Se desea realizar un algoritmo que permita controlar las reservas de plazas de un vuelo MADRID-CARACAS, de acuerdo con las siguientes normas de la compañía aérea:

Número de plazas del avión: 300.
Plazas numeradas de 1 a 100: fumadores.
Plazas numeradas de 101 a 300: no fumadores.

Se debe realizar la reserva a petición del pasajero y cerrar la reserva cuando no haya plazas libres o el avión esté próximo a despegar. Como ampliación de este algoritmo, considere la opción de anulaciones imprevistas de reservas.

6.15. Cada alumno de una clase de licenciatura en Ciencias de la Computación tiene notas correspondientes a ocho asignaturas diferentes, pudiendo no tener calificación en alguna asignatura. A cada asignatura le corresponde un determinado coeficiente.

1. Escribir un algoritmo que permita calcular la media de cada alumno.
2. Modificar el algoritmo para obtener las siguientes medias:
   - general de la clase,
   - de la case en cada asignatura,
   - porcentaje de faltas (no presentado a examen).

6.16. Se dispone de las notas de cuarenta alumnos. Cada uno de ellos puede tener una o varias notas. Escribir un algoritmo que permita obtener la media de cada alumno y la media de la clase a partir de la entrada de las notas desde el terminal.

6.17. Una empresa tiene diez almacenes y necesita crear un algoritmo que lea las ventas mensuales de los diez almacenes, calcule la media de ventas y obtenga un listado de los almacenes cuyas ventas mensuales son superiores a la media.

6.18. Se dispone de una lista de cien números enteros. Calcular su valor máximo y el orden que ocupa en la tabla.

6.19. Un avión dispone de ciento ochenta plazas, de las cuales sesenta son de 'no fumador' y numeradas de 1 a 60 y ciento ochenta plazas numeradas de 61 a 180. Diseñar un algoritmo que permita hacer la reserva de plazas del avión y se detenga media hora antes de la salida del avión, en cuyo momento se abrirá la lista de espera.

6.20. Calcular las medias de las estaturas de una clase. Deducir cuántos son más altos que la media y cuántos más bajos que dicha media.

6.21. Las notas de un colegio se tienen en una matriz de 30×5 elementos (30, número de alumnos; 5, número de asignaturas). Se desea listar las notas de cada alumno y su media. Cada alumno tiene como mínimo dos asignaturas y máximo cinco, aunque los alumnos no necesariamente todos tienen que tener cinco materias.

6.22. Dado el nombre de una serie de estudiantes y las calificaciones obtenidas en un examen, calcular e imprimir la calificación media, así como cada calificación y la diferencia con la media.

6.23. Se introducen una serie de valores numéricos desde el teclado, siendo el valor final de entrada de datos o centinela –99. Se desea calcular e imprimir el número de valores leídos, la suma y media de los valores y una tabla que muestre cada valor leído y sus desviaciones de la media.

6.24. Se tiene una lista de N nombres de alumnos. Escribir un algoritmo que solicite el nombre de un alumno, busque en la lista (array) si el nombre está en la lista.

CAPÍTULO 7

LAS CADENAS DE CARACTERES

CONTENIDO

7.1. Introducción.
7.2. El juego de caracteres.
7.3. Cadena de caracteres.
7.4. Datos tipo carácter.
7.5. Operaciones con cadenas.
7.6. Otras funciones de cadenas.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

Las computadoras normalmente sugieren operaciones aritméticas ejecutadas sobre datos numéricos. Sin embargo, ese concepto no es estadísticamente cierto, sino que, al contrario, hoy día es cada vez más frecuente el uso de las computadoras para procesar problemas de tipo esencialmente alfanuméricos o de tipo texto.

En el Capítulo 1 se estudió el concepto de tipo de datos carácter (*char*) y se definió un carácter como un símbolo del juego de caracteres de la computadora. Una **constante carácter** se definió como cualquier carácter encerrado entre separadores (apóstrofo o dobles comillas). Una secuencia finita de caracteres se denomina normalmente una **cadena**, y una **constante tipo cadena** consiste en una cadena encerrada entre apóstrofos o dobles comillas. El procesamiento de cadenas es el objetivo fundamental de este capítulo.

7.1. INTRODUCCIÓN

Las computadoras nacieron para resolver problemas numéricos en cálculos científicos y matemáticos. Sin embargo, el paso de los años ha cambiado las aplicaciones y hoy día las computadoras no sólo se utilizan en cálculos numéricos, sino también para procesar datos de caracteres. En aplicaciones de gestión, la generación y actualización de listas de dirección, inventarios, etc., la información alfabética es fundamental. La edición de textos, traductores de lenguajes, base de datos son otras aplicaciones donde las cadenas de caracteres tienen gran utilidad.

En este capítulo se tratará el concepto de cadena de caracteres y su procesamiento, utilizando para ello una notación algorítmica similar a la utilizada hasta ahora.

Una *cadena de caracteres* es una secuencia de cero o más símbolos, que incluyen letras del alfabeto, dígitos y caracteres especiales.

7.2. EL JUEGO DE CARACTERES

Los lenguajes de programación utilizan *juegos de caracteres* «alfabeto» para comunicarse con las computadoras. Las primeras computadoras sólo utilizaban informaciones numéricas digitales mediante el código o alfabeto digital, y los primeros programas se escribieron en ese tipo de código, denominado *código máquina* —basado en dos dígitos, 0 y 1—, por ser inteligible directamente por la máquina (computadora). La enojosa tarea de programar en código máquina hizo que el alfabeto evolucionase y los lenguajes de programación comenzaran a utilizar códigos o juegos de caracteres similares a los utilizados en los lenguajes humanos. Así, hoy día la mayoría de las computadoras trabajan con diferentes tipos de juegos de caracteres, de los que se destacan el código ASCII y el EBCDIC.

De este modo, una computadora a través de los diferentes lenguajes de programación utiliza un juego o código de caracteres que serán fácilmente interpretados por la computadora y que pueden ser programados por el usuario.

Dos son los códigos más utilizados actualmente en computadoras, **ASCII** (**A**merican **S**tandard **C**ode for **I**nformation **I**nterchange) y **EBCDIC** (**E**xtended **B**inary **C**oded **D**ecimal **I**nterchange **C**ode).

El *código ASCII básico* utiliza 7 *bits* (dígitos binarios, 0, 1) para cada carácter a representar, lo que supone un total de 2^7 (128) caracteres distintos. El código ASCII ampliado utiliza 8 bits y, en ese caso, consta de 256 caracteres. Este código ASCII ha adquirido una gran popularidad, ya que es el estándar en todas las familias de computadoras personales.

El *código EBCDIC* utiliza 8 *bits* por carácter y, por consiguiente, consta de 256 caracteres distintos. Su notoriedad reside en ser el utilizado por la firma IBM (sin embargo, en sus computadoras personales IBM ha seguido el código ASCII).

En general, un carácter ocupará un byte de almacenamiento de memoria.

7.2.1. Código ASCII

El código ASCII se compone de los siguientes tipos de caracteres:

- *alfabéticos* (a, b, ..., z/A, B, ..., Z),
- *numéricos* (0, 1, 2, 3, ..., 8, 9),
- *especiales* (+, –, *, /, {, }, <, >, etc.),

Valor ASCII	Carácter	Valor ASCII	Carácter	Valor ASCII	Carácter	Valor ASCII	Carácter
000	NUL	032	espacio	064	@	096	'
001	SOH	033	!	065	A	097	a
002	STX	034	"	066	B	098	b
003	ETX	035	#	067	C	099	c
004	EOT	036	$	068	D	100	d
005	ENQ	037	%	069	E	101	e
006	ACK	038	&	070	F	102	f
007	BEL	039	'	071	G	103	g
008	BS	040	(	072	H	104	h
009	HT	041	)	073	I	105	i
010	LF	042	*	074	J	106	j
011	VT	043	+	075	K	107	k
012	FF	044	,	076	L	108	l
013	CR	045	-	077	M	109	m
014	SO	046	.	078	N	110	n
015	SI	047	/	079	O	111	o
016	DLE	048	0	080	P	112	p
017	DC1	049	1	081	Q	113	q
018	DC2	050	2	082	R	114	r
019	DC3	051	3	083	S	115	s
020	DC4	052	4	084	T	116	t
021	NAK	053	5	085	U	117	u
022	SYN	054	6	086	V	118	v
023	ETB	055	7	087	W	119	w
024	CAN	056	8	088	X	120	x
025	EM	057	9	089	Y	121	y
026	SUB	058	:	090	Z	122	z
027	ESC	059	;	091	[	123	{
028	FS	060	<	092	\	124	|
029	GS	061	=	093	]	125	}
030	RS	062	>	094	↑	126	~
031	US	063	?	095	_	127	DEL

NOTA: Los 32 primeros caracteres y el último son caracteres de control: no son imprimibles.

Figura 7.1. Código ASCII básico.

- *de control:* son caracteres no imprimibles y que realizan una serie de funciones relacionadas con la escritura, transmisión de datos, separador de archivos, etc., en realidad con los dispositivos de entrada/salida. Destacamos entre ellos:

  DEL *eliminar o borrar*
  STX *inicio de texto*
  LF *avance de línca*
  FF *avance de página*
  CR *retorno de carro*

Los caracteres del 128 al 255, pertenecientes en exclusiva al código ASCII ampliado, no suelen ser estándar y normalmente cada fabricante los utiliza para situar en ellos caracteres específicos de su máquina o de otros alfabetos, caracteres gráficos, etc. En la Figura 7.2 se muestra el código ASCII de la familia de computadoras IBM PC y compatibles, donde se puede apreciar tanto el ASCII básico estándar como el ampliado.

D	P	D	P	D	P	D	P	D	P	D	P	D	P	D	P
0		32		64	@	96	`	128	Ç	160	á	192	└	224	α
1	☺	33	!	65	A	97	a	129	ü	161	í	193	┴	225	β
2	☻	34	"	66	B	98	b	130	é	162	ó	194	┬	226	Γ
3	♥	35	#	67	C	99	c	131	â	163	ú	195	├	227	π
4	♦	36	$	68	D	100	d	132	ä	164	ñ	196	─	228	Σ
5	♣	37	%	69	E	101	e	133	à	165	Ñ	197	┼	229	σ
6	♠	38	&	70	F	102	f	134	å	166	ª	198	╞	230	μ
7	•	39	'	71	G	103	g	135	ç	167	º	199	╟	231	γ
8	◘	40	(	72	H	104	h	136	ê	168	¿	200	╚	232	φ
9	○	41	)	73	I	105	i	137	ë	169	⌐	201	╔	233	θ
10	◙	42	*	74	J	106	j	138	è	170	¬	202	╩	234	Ω
11	♂	43	+	75	K	107	k	139	ï	171	½	203	╦	235	δ
12	♀	44	,	76	L	108	l	140	î	172	¼	204	╠	236	∞
13	♪	45	-	77	M	109	m	141	ì	173	¡	205	═	237	Ø
14	♫	46	.	78	N	110	n	142	Ä	174	«	206	╬	238	∈
15	☼	47	/	79	O	111	o	143	Å	175	»	207	╧	239	∩
16	►	48	0	80	P	112	p	144	É	176	▒	208	╨	240	≡
17	◄	49	1	81	Q	113	q	145	æ	177	▒	209	╤	241	±
18	↕	50	2	82	R	114	r	146	Æ	178	▓	210	╥	242	≥
19	!!	51	3	83	S	115	s	147	ô	179	│	211	╙	243	≤
20	¶	52	4	84	T	116	t	148	ö	180	┤	212	╘	244	⌠
21	§	53	5	85	U	117	u	149	ò	181	╡	213	╒	245	⌡
22	▬	54	6	86	V	118	v	150	û	182	╢	214	╓	246	÷
23	↨	55	7	87	W	119	w	151	ù	183	╖	215	╫	247	≈
24	↑	56	8	88	X	120	x	152	ÿ	184	╕	216	╪	248	°
25	↓	57	9	89	Y	121	y	153	Ö	185	╣	217	┘	249	∙
26	→	58	:	90	Z	122	z	154	Ü	186	║	218	┌	250	·
27	←	59	;	91	[	123	{	155	¢	187	╗	219	▒	251	√
28	⌐	60	<	92	\	124	¦	156	£	188	╝	220	▄	252	n
29	↔	61	=	93	]	125	}	157	¥	189	╜	221	▌	253	2
30	▲	62	>	94	^	126	~	158	Pt	190	╛	222	▐	254	■
31	▼	63	?	95	_	127	⌂	159	ƒ	191	┐	223	▀	255	

D: Código decimal.
P: Escritura del carácter correspondiente al código en la pantalla.

Figura 7.2. Código ASCII de la computadora IBM PC.

7.2.2. Código EBCDIC

Este código es muy similar al ASCII, incluyendo también, además de los caracteres alfanuméricos y especiales, caracteres de control. Es propio de computadoras de IBM, con la excepción de los modelos PC, XT, AT y PS/2.

7.3. CADENA DE CARACTERES

Una *cadena (string)* de caracteres es un conjunto de caracteres —incluido el blanco— que se almacenan en un área contigua de la memoria. Pueden ser entradas o salidas a/desde un terminal.

La *longitud* de una cadena es el número de caracteres que contiene. La cadena que no contiene ningún carácter se denomina *cadena vacía* o *nula*, y su longitud es cero; no se debe confundir con

D	C	D	C	D	C	D	C	D	C	D	C	D	C	D	C
0	NUL	21	NL	43	CU2	79	,	124	@	150	o	195	C	227	T
1	SOH	22	BS	45	ENQ	80	&	125	'	151	p	196	D	228	U
2	STX	23	IL	46	ACK	90	!	126	=	152	q	197	E	229	V
3	ETX	24	CAN	47	BEL	91	$	127	"	153	r	198	F	230	W
4	PF	25	EM	50	SYN	92	*	129	a	155	}	199	G	231	X
5	HT	26	CC	52	PN	93	)	130	b	161	~	200	H	232	Y
6	LC	27	CU1	53	RS	94	;	131	c	162	s	201	I	233	Z
7	DEL	28	IFS	54	UC	95	¬	132	d	163	t	208	}	240	0
10	SMM	29	IGS	55	EOT	96	-	133	e	164	u	209	J	241	1
11	VT	30	IRS	59	CU3	97	/	134	f	165	v	210	K	242	2
12	FF	31	IUS	60	DC4	106	¦	135	g	166	w	211	L	243	3
13	CR	32	DS	61	NAK	107	,	136	h	167	x	212	M	244	4
14	SO	33	SOS	63	SUB	108	%	137	i	168	y	213	N	245	5
15	SI	34	FS	64	SP	109	_	139	{	169	z	214	O	246	6
16	DLE	36	BYP	74	¢	110	>	145	j	173	[	215	P	247	7
17	DC1	37	LF	75	.	111	?	146	k	189	]	216	Q	248	8
18	DC2	38	ETB	76	<	121	`	147	l	192	{	217	R	249	9
19	DC3	39	ESC	77	(	122	:	148	m	193	A	224	\	250	I
20	RES	40	SM	78	+	123	#	149	n	194	B	226	S		

D: Código decimal.
C: Código EBCDIC.

Figura 7.3. Código EBCDIC.

una cadena compuesta sólo de blancos —espacios en blanco—, ya que ésta tendrá como longitud el número de blancos de la misma.

La representación de las cadenas suele ser con comillas simples o dobles. En nuestro libro utilizaremos las comillas simples por ser esa notación la más utilizada en diversos lenguajes, como Pascal, FORTRAN, etc.

Las comillas actúan como separadores:

```
'12 de octubre de 1492'
'Por fin llegaste'
'  '
'AMERICA ES GRANDE'
```

Las cadenas pueden contener entre sus separadores cualquier carácter válido del código aceptado por el lenguaje y la computadora; el blanco es uno de los caracteres más utilizado; si se le quiere representar de modo especial en la escritura en papel, se emplea alguno de los siguientes símbolos:

_ ␣ ⌴ □

Por nuestra parte utilizaremos –, ⌴ y □ indistintamente, dejando libertad al lector para usar el que mejor convenga a su estilo de programación.

Las cadenas anteriores tienen longitudes respectivas de 21, 16, 3 y 17.

Una *subcadena* es una cadena de caracteres que ha sido extraída de otra de mayor longitud.

```
'12 de'      es una subcadena de    '12 de octubre'
'BASIC'      es una subcadena de    'lenguaje BASIC'
'CHE'        es una subcadena de    'CARCHELEJO'
```

7.4. DATOS TIPO CARÁCTER

En el Capítulo 1 se analizaron los diferentes tipos de datos y, entre ellos, existía el dato tipo *carácter (char)* que se incorpora en diferentes lenguajes de programación, bien con este nombre o bien como datos tipo cadena. Así pues, en esta sección trataremos las constantes y las variables tipo carácter o cadena.

7.4.1. Constantes

Una constante tipo carácter es un carácter encerrado entre comillas, y una constante de tipo cadena es un conjunto de caracteres válidos encerrados entre comillas —apóstrofos— para evitar confundirlos con nombres de variables, operadores, enteros, etc. Si se desea escribir un carácter comilla, se debe escribir duplicado. Como se ha comentado anteriormente, existen lenguajes —BASIC, por ejemplo— que encierran las cadenas entre dobles comillas. Nuestros algoritmos sólo tendrán una, por ser más usual esta práctica en la mayoría de los lenguajes.

```
'Carchelejo es un pueblo de Jaen'
```

es una constante de tipo cadena, de una longitud fija igual a 31.

```
'¿'
```

es una constante de tipo carácter.

7.4.2. Variables

Una *variable de cadena* o *tipo carácter* es una variable cuyo valor es una cadena de caracteres.

Las variables de tipo carácter o cadena se deben declarar en el algoritmo y, según el lenguaje, tendrán una notación u otra. Nosotros, al igual que muchos lenguajes, las declararemos en la tabla o bloque de declaración de variables.

```
var
  caracter: A, B
  cadena: NOMBRE, DIRECCION
```

Atendiendo a la declaración de la longitud, las variables se dividen en *estáticas*, *semiestáticas* y *dinámicas*.

Variables estáticas son aquéllas en las que su longitud se define antes de ejecutar el programa y ésta no puede cambiarse a lo largo de éste.

FORTRAN:
```
CHARACTER A1 * 10, A2 * 15
```
las variables A1 y A2 se declaran con longitudes 10 y 15, respectivamente.

Pascal:
```
var NOMBRE: array[1..30] of char        o bien
var NOMBRE:STRING[N]
```

En Pascal, una variable de tipo carácter —*char*— sólo puede almacenar un carácter y, por consiguiente, una cadena de caracteres debe representarse mediante un *array* de caracteres. En el ejemplo NOMBRE se declara como una cadena de 30 caracteres (en este caso NOMBRE[1] será el primer carácter de la cadena, NOMBRE[2] será el segundo carácter de la cadena, etc.).

Pascal admite también tratamiento de cadenas (STRING) como dato.

Variables semiestáticas son aquéllas cuya longitud puede variar durante la ejecución del programa, pero sin sobrepasar un límite máximo declarado al principio.

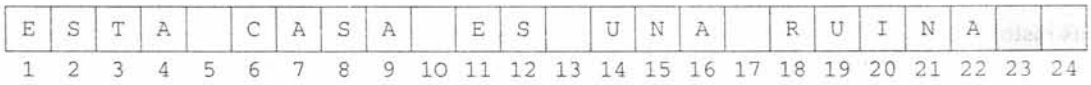
Variables dinámicas son aquéllas cuya longitud puede variar sin limitación dentro del programa.

El lenguaje SNOBOL es típico de variables dinámicas.

La representación de las diferentes variables de cadena en memoria utiliza un método de almacenamiento diferente.

Cadenas de longitud fija

Se consideran vectores de la longitud declarada, con blancos a izquierda o derecha, si la cadena no tiene la longitud declarada. Así, la cadena siguiente:



se declaró con una dimensión de 24 caracteres y los dos últimos se rellenan con blancos.

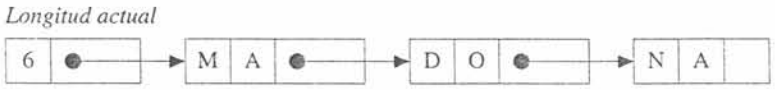
Cadenas de longitud variable con un máximo

Se considera un puntero (en el Capítulo 11 ampliaremos este concepto) con dos campos que contienen la longitud máxima y la longitud actual:

Cadenas de longitud indefinida

Se representan mediante listas enlazadas, que son listas que se unen mediante puntero.

Estas listas contienen elementos con caracteres empaquetados —2/elemento— y enlazados cada uno con el siguiente por un puntero (la cadena de caracteres es 'MADONA').



7.4.3. Instrucciones básicas con cadenas

Las instrucciones básicas: *asignar* y *entrada/salida* (*leer/escribir*) se realizan de un modo similar al tratamiento de dichas instrucciones con datos numéricos.

Asignación

Si la variable NOMBRE se ha declarado como tipo cadena

```
var cadena: NOMBRE
```

la instrucción de asignación debe contener en el lado derecho de la asignación una constante tipo cadena o bien otra variable del mismo tipo. Así,

```
NOMBRE ← 'Luis Hermenegildo'
```

significa que la variable NOMBRE toma por valor la cadena `'Luis Hermenegildo'`.

Entrada/Salida

La entrada/salida desde un terminal se puede realizar en modo carácter; para ello bastará asignar —a través del correspondiente dispositivo— una cadena de caracteres a una variable tipo cadena.

Así, por ejemplo, si A, B, C y D se han declarado como variables tipo cadena:

```
var cadena: A, B, C, D.
```

las instrucciones

```
leer(A, B)
escribir(C, D)
```

asignarán a A y B las cadenas introducidas por el teclado y visualizarán o imprimirán en el dispositivo de salida las cadenas que representan las variables C y D.

7.5. OPERACIONES CON CADENAS

El tratamiento de cadenas es un tema importante, debido esencialmente a la gran cantidad de información que se almacena en ellas. Según el tipo de lenguaje de programación elegido se tendrá mayor o menor facilidad para la realización de operaciones. Así, por ejemplo, BASIC tiene grandes posibilidades, FORTRAN sólo operaciones elementales, y Pascal, dependiendo del compilador, soporta procedimientos y funciones predefinidas, o es preciso definirlos por el usuario con la natural complejidad que suponga el diseño del algoritmo correspondiente. En cualquier caso, las operaciones con cadena más usuales son:

- cálculo de la longitud,
- comparación,
- concatenación,
- extracción de subcadenas,
- búsqueda de información.

7.5.1. Cálculo de la longitud de una cadena

La longitud de una cadena, como ya se ha comentado, es el número de caracteres de la cadena. Así,

```
'Don Quijote de la Mancha'
```

tiene veinticuatro caracteres.

La operación de determinación de la longitud de una cadena se representará por la función **longitud**, cuyo formato es:

```
longitud (cadena)
```

La función **longitud** tiene como argumento una cadena, pero su resultado es un valor numérico entero:

```
longitud('Don Quijote de la Mancha')   proporciona 24
longitud('□□□')                        cadena de tres blancos proporciona 3
longitud('□□□Mortadelo')               cadena 'Mortadelo' rellenada de blancos a la izquierda para
                                       tener longitud 12
```

En consecuencia, la función **longitud** se puede considerar un dato tipo entero y, por consiguiente, puede ser un operando dentro de expresiones aritméticas.

```
4 + 5 + longitud('DEMO□') = 4 + 5 + 5 = 13
```

7.5.2. Comparación

La *comparación* de cadenas (igualdad y desigualdad) es una operación muy importante, sobre todo en la clasificación de datos tipo carácter, que se utiliza con mucha frecuencia en aplicaciones de proceso de datos (clasificaciones de listas, tratamiento de textos, etc.).

Los criterios de comparación se basan en el orden numérico del código o juego de caracteres que admite la computadora o el propio lenguaje de programación. En nuestro lenguaje algorítmico utilizaremos el código ASCII como código numérico de referencia. Así,

- el carácter 'A' será < el carácter 'C'
  (código 65) *(código 67)*
- el carácter '8' será < el carácter 'i'
  (código 56) *(código 105)*

En la comparación de cadenas se pueden considerar dos operaciones más elementales: *igualdad* y *desigualdad*.

Igualdad

Dos cadenas *a* y *b* de longitudes *m* y *n* son iguales si:

- el número de caracteres de *a* y *b* son los mismos ($m = n$)
- cada carácter de *a* es igual a su correspondiente de *b*; si $a = a_1a_2...a_n$ y $b = b_1b_2...b_n$, se debe verificar que $a_i = b_i$ para todo *i* en el rango $1 <= i <= n$.

```
Así:   'EMILIO' = 'EMILIO'   es una expresión verdadera
       'EMILIO' = 'EMILIA'   es una expresión falsa
       'EMILIO' = 'EMILIO '  es una expresión falsa: contiene un blanco final y, por consiguiente, las
                             longitudes no son iguales
```

Desigualdad

Los criterios para comprobar la desigualdad de cadena utilizan normalmente los operadores de relación <, <=, >=, < > o ><, y se ajustan a una comparación sucesiva de caracteres correspondien-

tes en ambas cadenas hasta conseguir dos caracteres diferentes. De este modo, se pueden conseguir clasificaciones alfanuméricas

```
'GARCIA' < 'GOMEZ'
```

ya que las comparaciones sucesivas de caracteres son:

```
G-A-R-C-I-A  G = G, A < O, ...
G-O-M-E-Z
```

una vez que se encuentra una desigualdad, no es preciso continuar; como se observa, las cadenas no tienen por qué tener la misma longitud para ser comparadas.

Ejemplo 7.1

En las sucesivas comparaciones se puede apreciar una amplia gama de posibles casos.

```
'LUIS'      <     'LUISITO'      verdad
'ANA'       <     'MARTA'        verdad
'TOMAS'     <     'LUIS'         falsa
'BARTOLO'   <=    'BARTOLOME'    verdad
'CARMONA'   >     'MADRID'       falsa
'LUIS '     >     'LUIS'         verdad
```

Se puede observar de los casos anteriores que la presencia de cualquier carácter —incluso el blanco—, se considera mayor siempre que la ausencia. Por eso, `'LUIS '` es mayor que `'LUIS'`.

7.5.3. Concatenación

La concatenación es la operación de reunir varias cadenas de caracteres en una sola, pero conservando el orden de los caracteres de cada una de ellas.

El símbolo que representa la concatenación varía de unos lenguajes a otros. Los más utilizados son:

```
+     //     &     o
```

En nuestro libro utilizaremos + y, en ocasiones, &.

Las cadenas para concatenarse pueden ser constantes o variables.

```
'MIGUEL' + 'DE' + 'CERVANTES' = 'MIGUELDECERVANTES'
```

Puede comprobar que las cadenas, en realidad, se *«pegan»* unas al lado de las otras; por ello, si al concatenar frases desea dejar blancos entre ellas, deberá indicarlos expresamente en alguna de las cadenas. Así, las operaciones

```
'MIGUEL ' + 'DE' + ' CERVANTES'
'MIGUEL' + ' DE ' + 'CERVANTES'
```

producen el mismo resultado

```
'MIGUEL DE CERVANTES'
```

Ejemplo 7.2

Es posible concatenar variables de cadena.

```
var cadena:     A, B, C
A & B & C       equivale a A & (B & C)
```

La asignación de constantes tipo cadena a variables tipo cadena puede también realizarse con expresiones concatenadas.

Ejemplo 7.3

Las variables A, B son de tipo *cadena*.

```
var cadena: A, B

A ← 'FUNDAMENTOS'
B ← 'DE PROGRAMACION'
```

La variable C puede recibir como valor

```
C ← A+' '+B
```

que produce un resultado de

```
C = 'FUNDAMENTOS DE PROGRAMACION'
```

7.5.4. Subcadenas

Otra operación —función— importante de las cadenas es aquella que permite la extracción de una parte específica de una cadena: *subcadena*.

La operación *subcadena* se representa en dos formatos por:

```
subcadena (cadena_fuente,p,longsub)
```

- *cadena_fuente* es la cadena de la que debe extraerse una subcadena.
- *p* es un número o expresión numérica entero que corresponde a la posición inicial de la subcadena.
- *longsub* es la longitud de la subcadena.

```
subcadena (cadena_fuente, p)
```

En este caso, la subcadena comienza en p y termina en el final de la cadena.

Ejemplos:

```
subcadena ('abcdef', 2, 4)      equivale a      'bcde'
subcadena ('abcdef', 6, 1)      equivale a      'f'
subcadena ('abcdef', 3)         equivale a      'cdef'
subcadena ('abcdef', 3, 4)      equivale a      'cdef'
       longitud = 5 caracteres
subcadena ('12 DE OCTUBRE', 4, 5) = DE OC
                ↑
          posición 4
```

Es posible realizar operaciones de concatenación con subcadenas.

```
subcadena ('PATO DONALD',1,4)& subcadena ('ESTA TIERRA',5,4)
```

equivale a la cadena 'PATO TIE'.

La aplicación de la función a una subcadena,

```
subcadena (cadena_fuente, p, longsub)
```

puede producir los siguientes resultados:

1. Si *longsub* no existe, entonces la subcadena comienza en el mismo carácter posición y termina con el último carácter.
2. Si *longsub* <= 0, el resultado es una cadena vacía.
3. Si *p* > longitud (cadena_fuente), la subcadena resultante será vacía.
   subcadena ('MORTIMER', 9, 2)
   produce una cadena vacía.
4. Si *p* <= 0, el resultado es también una cadena vacía.
   subcadena ('valdez', 0, 4) *y subcadena* ('valdez', 8)
   proporcionan cadenas nulas.

7.5.5. Búsqueda

Una operación frecuente para realizar con cadenas es localizar si una determinada cadena forma parte de otra cadena más grande o buscar la posición en que aparece un determinado carácter o secuencia de caracteres de un texto.

Estos problemas pueden resolverse con las funciones de cadena estudiadas hasta ahora, pero será necesario diseñar los algoritmos correspondientes. Esta función suele ser interna en algunos lenguajes y la definiremos por **indice** o **posicion**, y su formato es:

```
indice (cadena1, cadena2)
```

o bien:

```
posicion (cadena1, cadena2)
```

donde cadena2 es el texto que se trata de localizar.

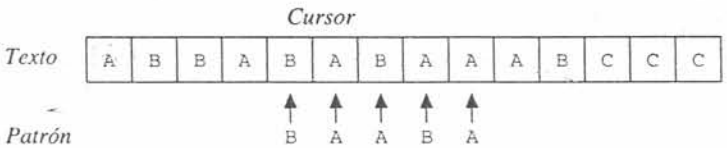
El resultado de la función es un valor entero:

- igual a P >= 1, donde P indica la posición del primer carácter de la primera coincidencia de subcadena en cadena,
- igual a cero, si subcadena es una cadena vacía o no aparece en la cadena.

Así, suponiendo la cadena C = 'LA CAPITAL ES MADRID'

```
posicion (C, 'CAP')      toma un valor  4
posicion (C, ' ES ')     toma un valor  11
posicion (C, 'PADRID')   toma un valor  0
```

La función posición en su forma más general realiza la operación que se denomina *coincidencia* o *reconocimiento de patrones* (*pattern-matching*). Esta operación busca una cadena patrón o modelo dentro de una cadena de texto.



Esta operación utiliza un cursor o puntero en la cadena de texto original y va comprobando los sucesivos valores de ambas cadenas: si no son todos iguales, produce un 0, y si no proporciona la posición del primer carácter coincidente.

```
posicion ('ABCDE', 'F')          produce   0
posicion ('ABXYZCDEF', 'XYZ')    produce   3
```

La función **indice (posicion)** al tomar también un valor numérico entero se puede utilizar en expresiones aritméticas o en instrucciones de asignación a variables numéricas.

```
P ← indice (C, 'F')
```

7.6. OTRAS FUNCIONES DE CADENAS

Existen otras funciones de cadena internas al lenguaje o definidas por el usuario, que suelen ser de utilidad en programación y cuyo conocimiento es importante que lo conozca el lector:

- *insertar* cadenas,
- *borrar* cadenas,
- *cambiar* cadenas,
- *convertir* cadenas en números y viceversa.

7.6.1. Insertar

Si se desea insertar una cadena C dentro de un texto o cadena más grande, se debe indicar la posición. El formato de la función **insertar** es:

```
insertar (t, p, s)
```

- `t` texto o cadena *donde* se va a insertar,
- `p` posición *a partir de la cual* se va a insertar,
- `s` *subcadena* que se va a *insertar*.

```
insertar ('ABCDEFGHI', 4, 'XXX')     produce   'ABCXXXDEFGHI'
insertar ('MARIA O', 7, 'DE LA ')    produce   'MARIA DE LA O'
```

Algoritmo *de inserción*

Si su lenguaje no posee definida esta función, se puede implementar con el siguiente algoritmo:

```
inicio
  devolver(subcadena(t,1,p - 1) + S + subcadena(t,p,longitud(t)- p + 1))
fin
```

Veámoslo con un ejemplo: **insertar** ('ABCDEFGHI' 4, 'XXX')

donde t = 'ABCDEFGHI', y , S = 'XXX' p = 4

```
subcadena (t,1,p - 1) = subcadena (t,1,3) = 'ABC'
subcadena (t,p,longitud(t)- p + 1) = subcadena (t,4,9 - 4 + 1) =
subcadena (t,4,6,) = 'DEFGHI'
```

por consiguiente,

```
insertar ('ABCDEFGHI',4'XXX') = 'ABC'+'XXX'+'DEFGHI' =
                                'ABCXXXDEFGHI'
```

7.6.2. Borrar

Si se desea eliminar una subcadena que comienza en la posición *p* y tiene una longitud *n* se tiene la función **borrar**.

```
borrar (t, p, n)
```

- *t* texto o cadena de donde se va a eliminar una subcadena,
- *p* posición a partir de la cual se va a borrar (eliminar),
- *n* longitud de la subcadena a eliminar,

```
borrar ('supercalifragilistico', 6, 4) = 'superfragilistico'
borrar ('supercalifragilistico', 3, 10) = 'sugilistico'
```

Algoritmo borrar

Si no se posee la función estándar **borrar**, será preciso definirla. Ello se consigue con el algoritmo:

```
inicio
  devolver (subcadena (t,1,p - 1) + subcadena (t,p + n,longitud(t) - p - n + 1))
fin
```

7.6.3. Cambiar

La operación insertar trata de sustituir en un texto t la primera ocurrencia de una subcadena S1 por otra S2. Éste es el caso frecuente en los programas de tratamiento de textos, donde a veces es nece-

sario sustituir una palabra cualquiera por otra (... en el archivo `"DEMO"` sustituir la palabra 'ordenador' por 'computadora'), acomodando las posibles longitudes diferentes. La función que realiza la operación de insertar tiene el formato

```
cambiar (t, S1, S2)
```

- *t* texto donde se realizarán los cambios,
- *S1* subcadena a sustituir,
- *S2* subcadena nueva.

```
cambiar ('ABCDEFGHIJ', 'DE', 'XXX') = 'ABCXXXFGHIJ'
```

Si la subcadena `S1` no coincide exactamente con una subcadena de `t`, no se produce ningún cambio y el texto o cadena original no se modifica

```
cambiar ('ABCDEFGHIJK', 'ZY', 'XXX') = 'ABCDEFGHIJK'
```

Algoritmo cambio

Si no se dispone de esta función como estándar, es posible definir un algoritmo haciendo uso de las funciones analizadas.

```
cambiar (t, S1, S2)
```

El algoritmo se realiza llamando a las funciones **`indice`**, **`borrar`** e **`insertar`**.

```
cadena funcion cambiar (E cadena: t, S1,S2)
var entero : j
inicio
  j ← indice(t, S1)
  t ← borrar(t, j, longitud(S1))
  devolver (insertar(t, j, S2))
fin
```

La primera instrucción, `j ← indice(t, S1)`, calcula la posición donde se debe comenzar la inserción, que es, a su vez, el primer elemento de la subcadena `S1`.

La segunda instrucción

```
t ← borrar(t, j, longitud(S1))
```

borra la subcadena `S1` y la nueva cadena se asigna a la variable de cadena `t`.

La tercera instrucción inserta en la nueva cadena `t` —original sin la cadena `S1`— la subcadena `S2` a partir del carácter de posición `j`, como se había previsto.

7.6.4. Conversión cadenas/números

Existen funciones o procedimientos en los lenguajes de programación (**`val`** y **`str`** en BASIC, **`val`** y **`string`** en Turbo PASCAL) que permiten convertir un número en una cadena y viceversa.

En nuestros algoritmos los denotaremos por **valor** y **cad**.

`valor(una-cadena)`	*convierte la cadena en un número; siempre que la cadena fuese de dígitos numéricos*
`cad (un-numero)`	*convierte un valor numérico en una cadena*
`codigo(un_caracter)`	*devuelve el código ASCII de un carácter*
`car (un_codigo)`	*devuelve el carácter asociado a un código ASCII*

Ejemplos

```
valor ('12345') = 12345
cad    (12345) = '12345'
```

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

7.1. Se desea eliminar los blancos de una frase dada terminada en un punto. Se supone que es posible leer los caracteres de la frase de uno en uno.

Análisis

Para poder efectuar la lectura de la frase, almacena ésta en un array de caracteres (F) —esto es posible en lenguajes como Pascal; en BASIC sería preciso recurrir a enojosas tareas de operaciones con funciones de cadenas MID$, LEFT$ o RIGHT$—, de modo que F[1] contiene el primer carácter *i*-ésimo de la frase dada.

Construiremos una nueva frase sin blancos en otro array G.

Algoritmo

Los pasos a dar para la realización del algoritmo son:

- Inicializar contador de letras de la nueva frase G.
- Leer el primer carácter.
- Repetir.
  Si el primer carácter no es en blanco, entonces
  escribir en el lugar siguiente del array G, leer
  carácter siguiente de la frase dada.
  Hasta que el último carácter se encuentre.
- Escribir la nueva frase -G- ya sin blancos.

Tabla de variables:

F	array de caracteres de la frase dada,
G	array de caracteres de la nueva frase,
I	contador del array F,
J	contador del array G.

Pseudocódigo

```
algoritmo Blanco
inicio
  I ← 1
  J ← 0
  F[i] ← leercar    // leercar es una funcion que permite la lectura de un caracter
  repetir
```

```
    si F[I] <> ' ' entonces
      J ← J + 1
      G[J] ← F[I]
    fin_si
    I ← I + 1
    F[I] ← leercar
  hasta_que F[i] = '.'
  //escritura de la nueva frase G
  desde I ← 1 hasta J hacer
    escribir(G[I])      // no avanzar linea
  fin_desde
fin
```

7.2. Leer un carácter y deducir si está situado antes o después de la letra M en orden alfabético.

Análisis

La comparación de datos de tipo carácter se realiza mediante los códigos numéricos ASCII, de modo que una letra estará situada antes o después de ésta si su código ASCII es menor o mayor. La propia computadora se encarga de realizar la comparación de datos tipo carácter de acuerdo al código ASCII, siempre que los datos a comparar sean de tipo carácter. Por ello se deben declarar de tipo carácter las variables que representan las comparaciones.

```
Variables C: caracter
```

Pseudocódigo

```
algoritmo Caracter
var
  caracter: C
inicio
  leer(C)
  si C < 'M' entonces
    escribir(C, 'esta antes que M en orden alfabetico')
  si_no
    escribir(C, 'esta despues que M en orden alfabetico')
  fin_si
fin
```

7.3. Leer los caracteres y deducir si están en orden alfabético.

Tabla de variables:

```
CAR1, CAR2: caracter
```

Pseudocódigo

```
algoritmo Comparacion
var
  caracter: CAR1, CAR2
inicio
  leer(CAR1, CAR2)
  si CAR1 <= CAR2 entonces
    escribir('en orden')
  si_no
    escribir('desordenados')
  fin_si
fin
```

7.4. Leer una letra de un texto. Deducir si está o no comprendida entre las letras mayúsculas `I-M` inclusive.

Variables:

```
LETRA: caracter.
```

Pseudocódigo

```
algoritmo
var
  caracter: LETRA
inicio
  leer(LETRA)
  si LETRA > = 'I' y LETRA =< 'M' entonces
    escribir('esta comprendida')
  si_no
    escribir('no esta comprendida')
  fin_si
fin
```

7.5. Contar el número de letras `'i'` de una frase terminada en un punto. Se supone que las letras pueden leerse independientemente.

En este algoritmo el contador de letras sólo se incrementa cuando se encuentran las letras `'i'` buscadas.

Pseudocódigo

```
algoritmo letras_i
var
  entero: N
  caracter: LETRA
inicio
  N ← 0
  repetir
    LETRA ← leercar   //la funcion leercar permite la lectura independiente
    si LETRA = 'i' entonces
      N ← N + 1
    fin_si
  hasta_que LETRA = '.'
  escribir('La frase tiene', N, 'letras i')
fin
```

7.6. Contar el número de vocales de una frase terminada en un punto.

Pseudocódigo

```
algoritmo Vocales
var
  entero: NUMVOCALES
  caracter: C
inicio
NUMVOCALES ← 0
  repetir
    C ← leercar
    si C = 'a' o C = 'e' o C = 'i' o C = 'o' o C = 'u' entonces
      NUMVOCALES ← NUMVOCALES + 1
    fin_si
  hasta_que C = '.'
  escribir('El numero de vocales es =', NUMVOCALES)
fin
```

7.7. Se desea contar el número de letras 'a' y el número de letras 'b' de una frase terminada en un punto. Se supone que es posible leer los caracteres independientemente.

Método 1:

```
algoritmo Letras_a_b
var
  entero: NA, NB
  caracter: C
inicio
  NA ← 0
  NB ← 0
  repetir
    C ← leercar
    si C ='a' entonces
      NA ← NA + 1
    fin_si
    si C = 'b' entonces
      NB ← NB + 1
    fin_si
  hasta_que C = '.'
  escribir('Letras a =', NA, 'Letras b=', NB=)
fin
```

Método 2:

```
algoritmo Letras_a_b
var
  entero: NA, NB
  caracter: C
inicio
  NA ← 0
  NB ← 0
  repetir
    C ← leercar
    si C = 'a' entonces
      NA ← NA + 1
    si_no
      si C = 'b' entonces
        NB ← NB + 1
      fin_si
    fin_si
  hasta_que C = '.'
fin
```

Método 3:

```
algoritmo Letras_a_b
var
  entero: NA, NB
  caracter: C
inicio
  NA ← 0
  NB ← 0
  repetir
    C ← leercar
    segun_sea C hacer
      'a': NA ← NA + 1
      'b': NB ← NB + 1
    fin_segun
  hasta_que C = '.
fin
```

7.8. Leer cien caracteres de un texto y contar el número de letras 'b'.

Tabla de variables:

```
I, NE: entero
C: caracter
```

Pseudocódigo

```
algoritmo Letras_b
var
  entero: I, NE
  caracter: C
inicio
  NE ← 0
  desde I ← 1 hasta 100
    C ← leercar
    si C = 'b' entonces
      NE ← NE + 1
    fin_si
  fin_desde
  escribir('Existen', NE, 'letras b')
fin
```

7.9. Escribir una función convertir (num, b) que nos permita transformar un número entero y positivo en base 10 a la base que le indiquemos como parámetro. Compruebe el algoritmo para las bases 2 y 16.

```
algoritmo Cambio_de_base
  var entero: num, b
  inicio
    escribir('Deme numero')
    leer(num)
    escribir('Indique base')
    leer(b)
    escribir(convertir(num,b),'es el numero', num, 'en base',b)
  fin

cadena funcion convertir(E entero: num,b)
  var entero: r
      caracter: c
      cadena: cade
  inicio
    cade ← ''
    si num > 0 entonces
      mientras num > 0 hacer
        r ← num MOD b
        si r > 9 entonces
          c ← Car(r + 55)
          si_no
          c ← Car(r + codigo('0'))
        fin_si
        cade ← c + cade
        num ← num div b
      fin_mientras
    si_no
      cade ← '0'
  fin_si
  devolver(cade)
fin_funcion
```

EJERCICIOS

7.1. Escribir un algoritmo para determinar si una cadena especificada ocurre en una cadena dada, y si es así, suprimir un asterisco (*) en la primera posición de cada ocurrencia.

7.2. Escribir un algoritmo para contar el número de ocurrencias de cada una de las palabras `'a'`, `'an'` y `'and'` en las diferentes líneas de texto.

7.3. Contar el número de ocurrencias de una cadena especificada en diferentes líneas de texto.

7.4. Escribir un algoritmo que permita la entrada de un nombre, consistente en un nombre, un primer apellido y un segundo apellido, en ese orden, y que imprima a continuación el último apellido, seguido del primer apellido y el nombre. Por ejemplo: `Luis Garcia Garcia` producirá: `Garcia Garcia Luis`.

7.5. Escribir un algoritmo que elimine todos los espacios finales en una cadena determinada. Por ejemplo: `'J. R. GARCIA '` se deberá transformar en `'J. R. GARCIA'`.

7.6. Diseñar un algoritmo cuya entrada sea una cadena `S` y un factor de multiplicación `N`, cuya función sea generar la cadena dada `N` veces. Por ejemplo:

```
'¡Hey!', 3
```

se convertirá en

```
'¡Hey! ¡Hey! ¡Hey!'
```

7.7. Diseñar un algoritmo que elimine todas las ocurrencias de cada carácter en una cadena dada a partir de otra cadena dada. Las dos cadenas son:

- `CADENA1` es la cadena donde deben eliminarse caracteres,
- `LISTA` es la cadena que proporciona los caracteres que deben eliminarse.

```
CADENA1 = 'EL EZNZZXTX'
LISTA = 'YZ'
```

la cadena pedida es `'EL ENT'`.

7.8. Escribir un algoritmo que convierta los números arábigos en romanos y viceversa (`I = 1, V = 5, X = 10, L = 50, C = 100, D = 500` y `M = 1000`).

CAPÍTULO 8

ARCHIVOS (FICHEROS)

CONTENIDO

8.1. Noción de archivo: estructura jerárquica.
8.2. Conceptos y definiciones = terminología.
8.3. Soportes secuenciales y direccionables.
8.4. Organización de archivos.
8.5. Operaciones sobre archivos.
8.6. Gestión de archivos.
8.7. Mantenimiento de archivos.
8.8. Procesamiento de archivos secuenciales (algoritmos).
8.9. Archivos de texto.
8.10. Procesamiento de archivos de acceso directo (algoritmos).
8.11. Procesamiento de archivos secuenciales indexados.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

Los datos que se han tratado hasta este capítulo y procesados por un programa pueden residir simultáneamente en la memoria principal de la computadora. Sin embargo, las grandes cantidades de datos se almacenan normalmente en dispositivos de memoria auxiliar. Las diferentes técnicas que han sido diseñadas para la estructuración de estas colecciones de datos complejas se alojaban en *arrays*; en este capítulo se realiza una *introducción a la organización y gestión de datos estructurados sobre dispositivos de almacenamiento secundario*, tales como cintas y discos magnéticos. Estas colecciones de datos se conocen como **archivos (ficheros)**. Las técnicas requeridas para gestionar archivos son diferentes de las técnicas de organización de datos que son efectivas en memoria principal, aunque se construyen sobre la base de esas técnicas. Este capítulo introductorio está concebido para la iniciación a los archivos, lo que son y sus misiones en los sistemas de información y los problemas básicos en su organización y gestión.

8.1. NOCIÓN DE ARCHIVO: ESTRUCTURA JERÁRQUICA

Las estructuras de datos enunciadas en los capítulos anteriores se encuentran almacenadas en la memoria central o principal. Este tipo de almacenamiento, conocido por *almacenamiento principal* o *primario*, tiene la ventaja de su pequeño tiempo de acceso y, además, que este tiempo necesario para acceder a los datos almacenados en una posición es el mismo que el tiempo necesario para acceder a los datos almacenados en otra posición del dispositivo —memoria principal—. Sin embargo, no siempre es posible almacenar los datos en la memoria central o principal de la computadora, debido a las limitaciones que su uso plantea:

- La cantidad de datos que puede manipular un programa no puede ser muy grande debido a la limitación de la memoria central de la computadora (aunque hoy día esto ya casi no se cumple, debido a las memorias centrales de 8 a 16 MB).
- La existencia de los datos en la memoria principal está supeditada al tiempo que la computadora está encendida y el programa ejecutándose (tiempo de vida efímero). Esto supone que los datos desaparecen de la memoria principal cuando la computadora se apaga o se deja de ejecutar el programa.

Estas limitaciones dificultan:

- La manipulación de gran número de datos, ya que —en ocasiones— pueden no caber en la memoria principal[1] (por ejemplo, si la información a almacenar es mayor de 640 K-bytes en el caso de computadoras IBM PC y compatibles).
- Que la transmisión de salida de resultados de un programa pueda ser tratada como entrada a otro programa.

Para superar estas dificultades se necesitan dispositivos de almacenamiento secundario (memorias externas o auxiliares), como cintas, discos magnéticos, tarjetas perforadas, etc., donde se almacenará la información o datos que podrán ser recuperados para su tratamiento posterior. Las estructuras de datos aplicadas a colección de datos en almacenamientos secundarios se llaman *organización de archivos*. La noción de *archivo* o *fichero* está relacionada con los conceptos de:

- almacenamiento permanente de datos,
- fraccionamiento o partición de grandes volúmenes de información en unidades más pequeñas que puedan ser almacenadas en memoria central y procesadas por un programa.

Un *archivo* o *fichero* es un conjunto de datos estructurados en una colección de entidades elementales o básicas denominadas *registros* o *artículos*, que son de igual tipo y constan a su vez de diferentes entidades de nivel más bajo denominadas *campos*.

8.1.1. Campos

Los caracteres se agrupan en *campos* de datos. Un *campo* es un *item* o *elemento de datos elementales*, tales como un nombre, número de empleados, ciudad, número de identificación, etc.

[1] Hoy día, cada vez más, el límite de la memoria comienza a no ser problema, dado que 8MB comienza a ser cifra usual para memoria central, incluso en computadoras personales domésticas, y 512MB o 1GM para memoria externa o permanente en disco rígido o duro (1MB = 1 MegaByte = 1.024.000 bytes; 1GB = 1 GigaByte = 10^9 bytes).

Un campo está caracterizado por su tamaño o longitud y su tipo de datos (cadena de caracteres, entero, lógico, etc.). Los campos pueden incluso variar en longitud. La mayoría de los lenguajes de programación no manejan los campos de longitud variable y se suponen de longitud fija.

Campos

Nombre	Dirección	Fecha de nacimiento	Estudios	Salario	Trienios

Figura 8.1. Campos de un registro.

Un campo es la unidad mínima de información de un registro.

Los datos contenidos en un campo se dividen con frecuencia en *subcampos*; por ejemplo, el campo fecha se divide en los subcampos día, mes, año.

Campo	0	7	0	7	1	9	9	5
Subcampo	Día		Mes		Año			

Los rangos numéricos de variación de los subcampos anteriores son:

```
1 ≤ dia ≤ 31
1 ≤ mes ≤ 12
1 ≤ año ≤ 1995
```

8.1.2. Registros

Un *registro* es una colección de información, normalmente relativa a una entidad particular. Un registro es una colección de campos lógicamente relacionados, que pueden ser tratados como una unidad por algún programa. Un ejemplo de un registro puede ser la información de un determinado empleado, que contiene los campos de nombre, dirección, fecha de nacimiento, estudios, salario, trienios, etc.

Los registros pueden ser todos de *longitud fija*; por ejemplo, los registros de empleados pueden contener el mismo número de campos, cada uno de la misma longitud, para nombre, dirección, fecha, etc. También pueden ser de *longitud variable*.

Los registros organizados en campos se denominan *registros lógicos*.

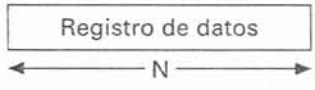


Figura 8.2. Registro.

8.1.3. Archivos (ficheros)

Un ***fichero*** (*archivo*) de datos —o simplemente un **archivo**— es una colección de registros relacionados entre sí con aspectos en común y organizados para un propósito específico. Por ejemplo, un fichero de una clase escolar contiene un conjunto de registros de los estudiantes de esa clase. Otros ejemplos pueden ser el fichero de nóminas de una empresa, inventarios, stocks, etc.

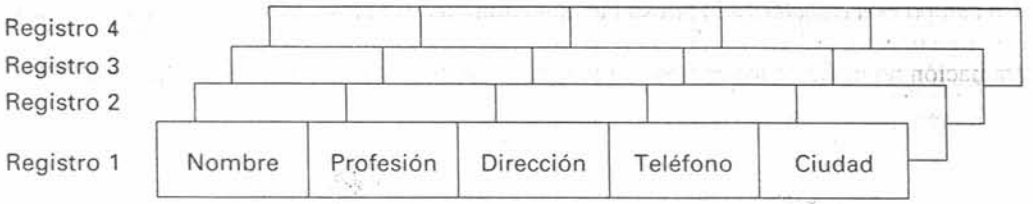


Figura 8.3. Estructura de un archivo «suscriptores».

La Figura 8.3 recoge la estructura de un archivo correspondiente a los suscriptores de una revista de Informática.

Un archivo en una computadora es una estructura diseñada para contener datos. Los datos están organizados de tal modo que puedan ser recuperados fácilmente, actualizados o borrados y almacenados de nuevo en el archivo con todos los cambios realizados.

8.1.4. Bases de datos

Una colección de archivos a los que puede accederse por un conjunto de programas y que contienen todos ellos datos relacionados constituye una base de datos. Así, una base de datos de una universidad puede contener archivos de estudiantes, archivos de nóminas, inventarios de equipos, etc.

8.1.5. Estructura jerárquica

Los conceptos carácter, campos, registro, archivo y base de datos son *conceptos lógicos* que se refieren al medio en que el usuario de computadoras ve los datos y se organizan. Las estructuras de datos se organizan de un modo jerárquico, de modo que el nivel más alto lo constituye la base de datos y el nivel más bajo el carácter.

8.2. CONCEPTOS Y DEFINICIONES: TERMINOLOGÍA

Aunque en el apartado anterior ya se han comentado algunos términos relativos a la teoría de archivos, en este apartado se enunciarán todos los términos más utilizados en la gestión y diseño de archivos.

8.2.1. Clave (indicativo)

Una *clave* (*key*) o *indicativo* es un campo de datos que identifica el registro y lo diferencia de otros registros. Esta clave debe ser diferente para cada registro. Claves típicas son nombres o números de identificación.

8.2.2. Registro físico o bloque

Un *registro físico* o *bloque* es la cantidad más pequeña de datos que pueden transferirse en una operación de entrada/salida entre la memoria central y los dispositivos periféricos o viceversa. Ejemplos

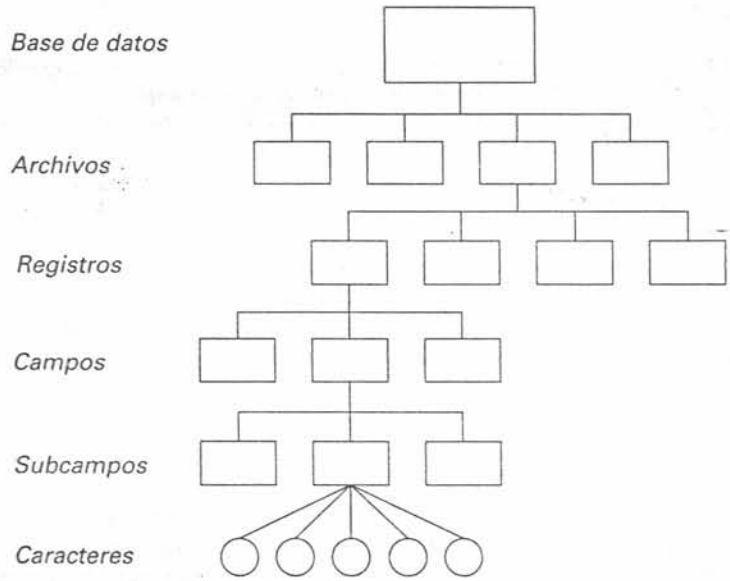


Figura 8.4. Estructuras jerárquicas de datos.

de registros físicos son: una tarjeta perforada, una línea de impresión, un sector de un disco magnético, etc.

Un bloque puede contener uno o más registros lógicos. Un registro lógico puede ocupar menos de un registro físico, un registro físico o más de un registro físico.

8.2.3. Factor de bloqueo

Otra característica que es importante en relación con los archivos es el concepto de *factor de bloqueo* o *blocaje*. El número de registros lógicos que puede contener un registro físico se denomina factor de bloqueo.

a) Un registro por bloque (factor = 1).

b) N registros por bloque (factor = N).

Figura 8.5. Factor de bloqueo.

Se pueden dar las siguientes situaciones:

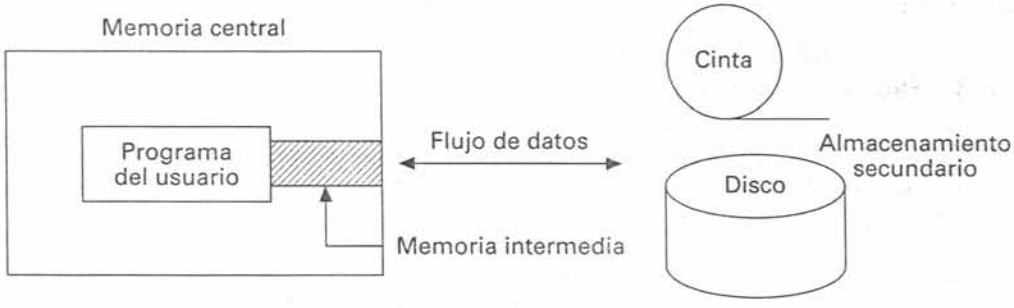
- *Registro lógico > Registro físico*. En un bloque se contienen varios registros físicos por bloque; se denominan *registros expandidos*.
- *Registro lógico = Registro físico*. El factor de bloqueo es 1 y se dice que los registros *no están bloqueados*.
- *Registro lógico < Registro físico*. El factor de bloqueo es mayor que 1 y los registros *están bloqueados*.

La importancia del factor de bloqueo se puede apreciar mejor con un ejemplo. Supongamos que se tienen dos archivos. Uno de ellos tiene un factor de bloqueo de 1 (un registro en cada bloque). El otro archivo tiene un factor de bloqueo de 10 (10 registros/bloque). Si cada archivo contiene un millón de registros, el segundo archivo requerirá 900.000 operaciones de entrada/salida menos para leer todos los registros. En el caso de las computadoras personales con un tiempo medio de acceso de 10 milisegundos, el primer archivo emplearía alrededor de 3 horas más para leer todos los registros del archivo.

Un factor de bloqueo mayor que 1 siempre mejora el rendimiento; entonces, ¿por qué no incluir todos los registros en un solo bloque? La razón reside en que las operaciones de entrada/salida que se realizan por bloques se hacen a través de un área de la memoria central denominada *memoria intermedia* (*buffer*) y entonces el aumento del bloque implicará aumento de la memoria intermedia y, por consiguiente, se reducirá el tamaño de la memoria central.

El tamaño de una memoria intermedia de un archivo es el mismo que el del tamaño de un bloque. Como la memoria central es más cara que la memoria secundaria, no conviene aumentar el tamaño del bloque alegremente, sino más bien conseguir un equilibrio entre ambos criterios.

En el caso de las computadoras personales, el registro físico suele ser un sector del disco (512 bytes).



La Tabla 8.1 resume los conceptos lógicos y físicos de un registro.

Resumen de archivos

- *Un archivo está siempre almacenado en un soporte externo a la memoria central.*
- *Existe independencia de las informaciones respecto de los programas.*
- *Todo programa de tratamiento intercambia información con el archivo y la unidad básica de entrada/salida es el registro.*
- *La información almacenada es permanente.*
- *En un momento dado, los datos extraídos por el archivo son los de un registro y no los del archivo completo.*
- *Los archivos, en memoria auxiliar, permiten una gran capacidad de almacenamiento.*

Tabla 8.1. Unidades de datos lógicos y físicos

Organización lógica	Organización física	Descripción
	Bit	Un dígito binario
Carácter	Byte (octeto, 8 bits)	En la mayoría de los códigos un carácter se representa por un byte
Campo	Palabra	Un campo es un conjunto relacionado de caracteres. Una palabra de computadora es un número fijo de bytes
Registro	Bloque (1 página = bloques de longitud)	Los registros pueden estar bloqueados
Archivo	Área	Varios archivos se pueden almacenar en un área de almacenamiento
Base de datos	Áreas	Colección de archivos de datos relacionados se pueden organizar en una base de datos

8.3. SOPORTES SECUENCIALES Y DIRECCIONABLES

El soporte es el medio físico donde se almacenan los datos. Los tipos de soporte utilizados en la gestión de archivos son:

- *soportes secuenciales,*
- *soportes direccionables.*

Los *soportes secuenciales* son aquéllos en los que los registros —informaciones— están escritos unos a continuación de otros y para acceder a un determinado registro n se necesita pasar por los $n - 1$ registros anteriores. La secuencia puede corresponder al orden físico de los registros en el archivo o bien al orden de claves (ascendente o descendente) de los registros.

Los *soportes direccionables* se estructuran de modo que las informaciones registradas se pueden localizar directamente por su dirección y no se requiere pasar por los registros precedentes. En estos soportes los registros deben poseer un campo clave que los diferencie del resto de los registros del archivo. Una dirección en un soporte direccionable puede ser número de pista y número de sector en un disco.

Los soportes direccionables son los discos magnéticos, aunque pueden actuar como soporte secuencial.

8.4. ORGANIZACIÓN DE ARCHIVOS

Según las características del soporte empleado y el modo en que se han organizado los registros, se consideran dos tipos de acceso a los registros de un archivo:

- *acceso secuencial,*
- *acceso directo.*

El *acceso secuencial* implica el acceso a un archivo según el orden de almacenamiento de sus registros, uno tras otro.

El *acceso directo* implica el acceso a un registro determinado, sin que ello implique la consulta de los registros precedentes. Este tipo de acceso sólo es posible con soportes direccionables.

La *organización* de un archivo define la forma en la que los registros se disponen sobre el soporte de almacenamiento, o también se define la organización como la forma en que se estructuran los datos en un archivo. En general, se consideran tres organizaciones fundamentales:

- *organización secuencial,*
- *organización directa o aleatoria («random»),*
- *organización secuencial indexada («indexed»).*

8.4.1. Organización secuencial

Un archivo con organización secuencial es una sucesión de registros almacenados consecutivamente sobre el soporte externo, de tal modo que para acceder a un registro *n* dado es obligatorio pasar por todos los *n* – 1 artículos que le preceden.

Los registros se graban consecutivamente cuando el archivo se crea y se debe acceder consecutivamente cuando se leen dichos registros.

Figura 8.6. Organización secuencial.

- El orden físico en que fueron grabados (escritos) los registros es el orden de lectura de los mismos.
- Todos los tipos de dispositivos de memoria auxiliar soportan la organización secuencial.

Los ficheros organizados secuencialmente contienen un registro particular —el último— que contiene una marca fin de archivo (detectable mediante las funciones (`EOF` o bien `FDA/FDF`)[2]. Esta marca fin de archivo suele ser un carácter especial, como `'*'`.

8.4.2. Organización directa

Un archivo está organizado en modo directo cuando el orden físico no se corresponde con el orden lógico. Los datos se sitúan en el archivo y se accede a ellos directa —aleatoriamente— mediante su posición, es decir, el lugar relativo que ocupan.

Esta organización tiene la *ventaja* de que se pueden leer y escribir registros en cualquier orden y posición. Son muy rápidos de acceso a la información que contienen.

La organización directa tiene el *inconveniente* de que necesita programar la relación existente entre el contenido de un registro y la posición que ocupa. El acceso a los registros en modo directo implica la posible existencia de huecos libres dentro del soporte y, por consecuencia, pueden existir huecos libres entre registros.

La correspondencia entre clave y dirección debe poder ser programada y la determinación de la relación entre el registro y su posición física se obtiene mediante una fórmula.

Las condiciones para que un archivo sea de organización directa son:

- almacenado en un soporte direccionable,
- los registros deben contener un campo específico denominado *clave* que identifica cada registro de modo único, es decir, dos registros distintos no pueden tener un mismo valor de clave,
- existencia de una correspondencia entre los posibles valores de la clave y las direcciones disponibles sobre el soporte.

En la práctica el programador no gestiona directamente direcciones absolutas, sino *direcciones relativas* respecto al principio del archivo. La manipulación de dirección relativa permite diseñar el programa con independencia de la posición absoluta del archivo en el soporte.

El programador crea una relación perfectamente definida entre la clave indicativa de cada registro y su posición física dentro del dispositivo de almacenamiento.

Consideremos a continuación el fenómeno de las *colisiones* mediante un ejemplo.

La clave de los registros de estudiantes de una Facultad de Ciencias es el número de expediente escolar que se le asigna en el momento de la matriculación y que consta de ocho dígitos. Si el número de estudiantes es un número decimal de ocho dígitos, existen 10^8 posibles números de estudiantes (0 a 99999999), aunque lógicamente *nunca* existirán tantos estudiantes (incluso incluyendo alumnos ya graduados). El archivo de estudiantes constará a lo sumo de decenas o centenas de miles de estudiantes. Se desea almacenar este archivo en un disco sin utilizar mucho espacio. Si se desea obtener el algoritmo de direccionamiento, se necesita una *función de conversión de claves* o *función «hash»*. Suponiendo que N es el número de posiciones disponibles para el archivo, el algoritmo de direccionamiento convierte cada valor de la clave en una dirección relativa d, comprendida entre 1 y N. Como la clave puede ser numérica o alfanumérica, el algoritmo de conversión debe prever esta posibilidad y asignar a cada registro correspondiente a una clave una posición física en el soporte de almacenamiento. Así mismo, el algoritmo o función de conversión de claves debe eliminar o reducir al máximo las colisiones.

Se dice que en un algoritmo de conversión de claves se produce una *colisión* cuando dos regis-

[2] **EOF** (*End Of File*), **FDA** (Final De Archivo), **FDF** (Final De Fichero), **FF** (Fin Fichero), **FA** (Fin Archivo).

tros de claves distintas producen la misma dirección física en el soporte. El *inconveniente de una colisión* radica en el hecho de tener que situar el registro en una posición diferente de la indicada por el algoritmo de conversión y, por consiguiente, el acceso a este registro será más lento. Las colisiones son inevitables en las organizaciones directas. Sin embargo, un tratamiento adecuado en las operaciones de lectura/escritura disminuirá su efecto perjudicial en el archivo.

Para representar la función de transformación o conversión de claves (*hash*), se puede utilizar una notación matemática. Así, si K es una clave, $f(K)$ es la correspondiente dirección; f es la función llamada *función de conversión*.

Ejemplo 8.1

Una compañía de empleados tiene un número determinado de vendedores y un archivo en el que cada registro corresponde a un vendedor. Existen 200 vendedores, cada uno referenciado por un número de cinco dígitos. Si tuviésemos que asignar un archivo de 100.000 registros, cada registro se corresponderá con una posición del disco.

Para el diseño del archivo crearemos 250 registros (un 25 por 100 más que el número de registros necesarios —25 por 100 suele ser un porcentaje habitual—) que se distribuirán de la siguiente forma:

1. Posiciones 0-199 constituyen el área principal del archivo y en ella se almacenarán todos los vendedores.
2. Posiciones 200-249 constituyen el área de desbordamiento, si $K(1) <> K(2)$, pero $f(K(1)) = f(K(2))$, y el registro con clave $K(1)$ ya está almacenado en el área principal, entonces el registro con $K(2)$ se almacena en el área de desbordamiento.

La función f se puede definir como:

$f(k)$ = *resto* cuando K se divide por 199, esto es, el módulo de 199; 199 ha sido elegido por ser el número primo mayor y que es menor que el tamaño del área principal.

Para establecer el archivo se borran primero 250 posiciones. A continuación, para cada registro de vendedor se calcula $p = f(K)$. Si la posición p está vacía, se almacena el registro en ella. En caso contrario, se busca secuencialmente a través de las posiciones 200, 201, ..., para el registro con la clave deseada.

8.4.3. Organización secuencial indexada[3]

Un diccionario es un archivo secuencial, cuyos registros son las entradas y cuyas claves son las palabras definidas por las entradas. Para buscar una palabra (una clave) no se busca secuencialmente desde la 'a' hasta la 'z', sino que se abre el diccionario por la letra inicial de la palabra. Si se desea buscar 'índice', se abre el índice por la letra *I* y en su primera página se busca la cabecera de página hasta encontrar la página más próxima a la palabra, buscando a continuación palabra a palabra hasta encontrar 'índice'. El diccionario es un ejemplo típico de archivo secuencial indexado con dos niveles de índices, el nivel superior para las letras iniciales y el nivel menor para las cabeceras de página. En una organización de computadora las letras y las cabeceras de páginas se guardarán en un archivo de índice independiente de las entradas del diccionario (archivo de datos). Por consiguiente, cada archivo secuencial indexado consta de un archivo índice y un archivo de datos.

[3] El término correcto aceptado por el Diccionario de la Real Academia de la Lengua Española sería *indizada*.

Un archivo está organizado en forma secuencial indexada si:

- el tipo de sus registros contiene un campo clave identificador,
- los registros están situados en un soporte direccionable por el orden de los valores indicados por la clave,
- existe un índice con cada una de las posiciones direccionables, que almacena la dirección de la posición y el valor de la clave; en esencia, el índice contiene la clave del último registro y la dirección de acceso al primer registro del bloque.

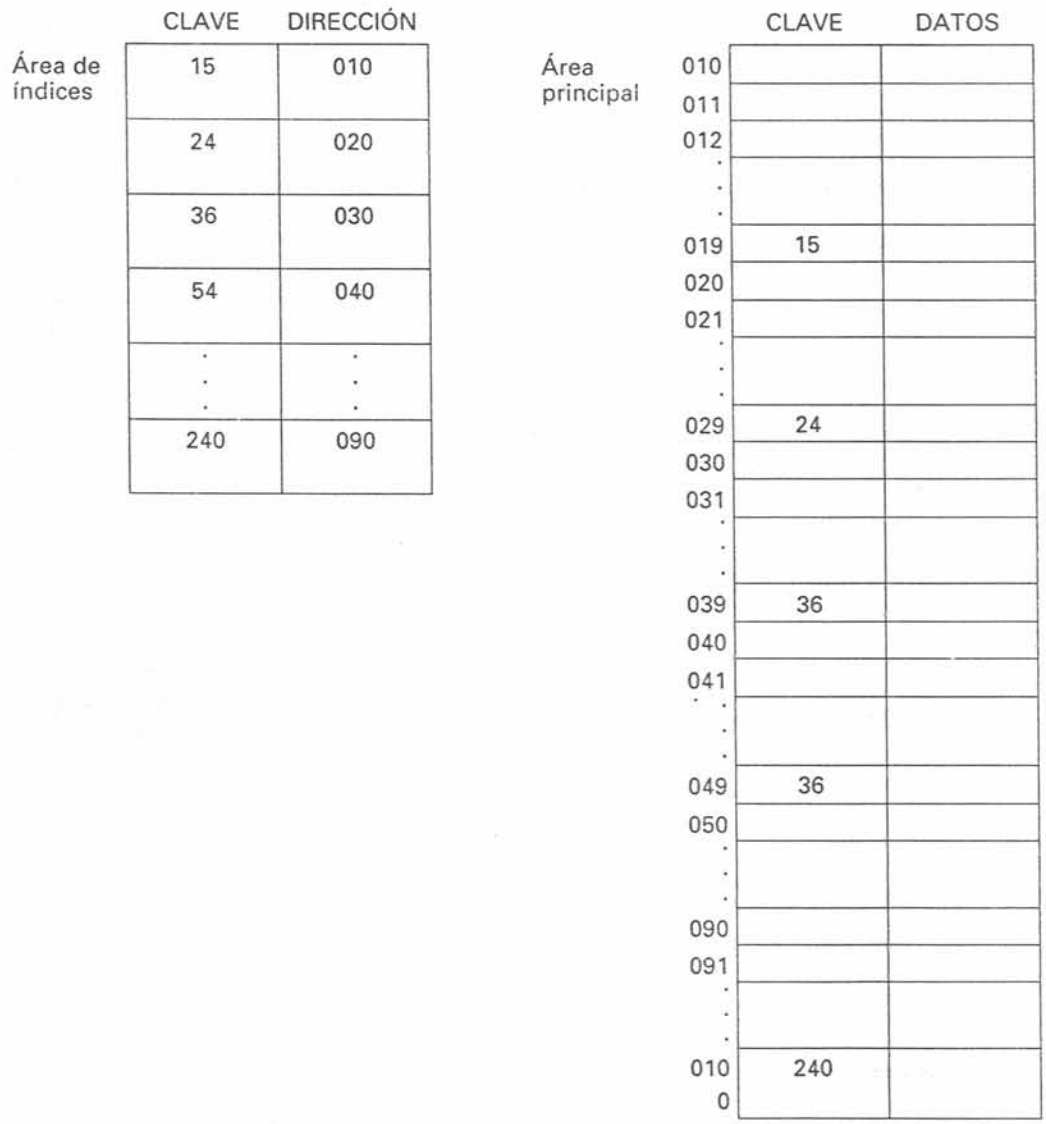


Figura 8.7. Organización secuencial indexada.

Un archivo en organización secuencial indexada consta de las siguientes partes:

- *área de datos* o *primaria*: contiene los registros en forma secuencial y está organizada en secuencia de claves sin dejar huecos intercalados,
- *área de índices*: es una tabla que contiene los niveles de índice; la existencia de varios índices enlazados se denomina *nivel de indexación*,
- *área de desbordamiento* o *excedentes*: utilizada, si fuese necesario, para las actualizaciones.

El área de índices es equivalente, en su función, al índice de un libro. En ella se refleja el valor de la clave identificativa más alta de cada grupo de registros del archivo y la dirección de almacenamiento del grupo.

Los archivos secuenciales indexados presentan las siguientes *ventajas*:

- rápido acceso,
- el sistema de gestión de archivos se encarga de relacionar la posición de cada registro con su contenido mediante la tabla de índices.

Y los siguientes *inconvenientes*:

- desaprovechamiento del espacio, por quedar huecos intermedios cada vez que se eliminan registros del archivo,
- se necesita espacio adicional para el área de índices.

Los soportes que se utilizan para esta organización son los que permiten el acceso directo —los discos magnéticos—. Los soportes de acceso secuencial no pueden utilizarse, ya que no disponen de direcciones para sus posiciones de almacenamiento.

8.5. OPERACIONES SOBRE ARCHIVOS

Tras la decisión del tipo de organización que ha de tener el archivo y los métodos de acceso que se van a aplicar para su manipulación, es preciso considerar todas las posibles operaciones que conciernen a los registros de un archivo. Las distintas operaciones que se pueden realizar son:

- *creación,*
- *consulta,*
- *actualización* (altas, bajas, modificación, consulta),
- *clasificación,*
- *reorganización,*
- *destrucción* (borrado),
- *reunión, fusión,*
- *rotura, estallido.*

8.5.1. Creación de un archivo

Es la primera operación que sufrirá el archivo de datos. Implica la elección de un entorno descriptivo que permita un ágil, rápido y eficaz tratamiento del archivo.

Figura 8.8. Creación de un archivo ordenado de empleados.

Para utilizar un archivo, éste tiene que existir, es decir, las informaciones de este archivo tienen que haber sido almacenadas sobre un soporte y ser utilizables. La *creación* exige organización, estructura, localización o reserva de espacio en el soporte de almacenamiento, transferencia del archivo del soporte antiguo al nuevo.

Un archivo puede ser creado por primera vez en un soporte, proceder de otro previamente existente en el mismo o en diferente soporte, ser el resultado de un cálculo o ambas cosas a la vez.

La Figura 8.8 muestra un organigrama de la creación de un archivo ordenado de empleados de una empresa por el campo clave (número o código de empleado).

8.5.2. Consulta de un archivo

Es la operación que permite al usuario acceder al archivo de datos para conocer el contenido de uno, varios o todos los registros.

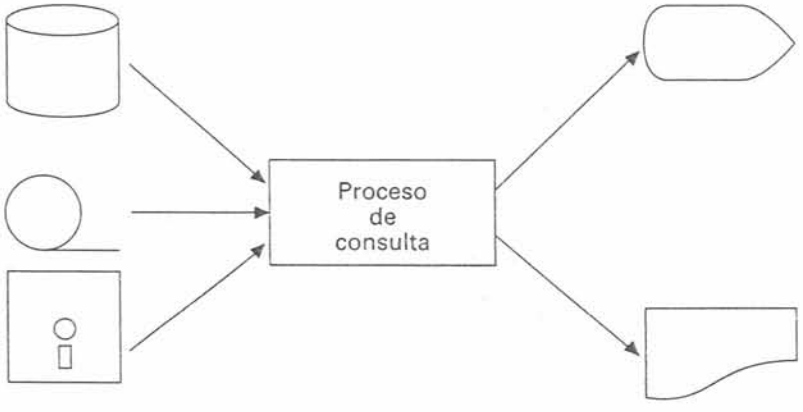


Figura 8.9. Consulta de un archivo.

8.5.3. Actualización de un archivo

Es la operación que permite tener actualizado (puesto al día) el archivo, de tal modo que sea posible realizar las siguientes operaciones con sus registros:

- consulta del contenido de un registro,
- inserción de un registro nuevo en el archivo,
- supresión de un registro existente,
- modificación de un registro.

Un ejemplo de actualización es el de un archivo de un almacén, cuyos registros contienen las existencias de cada artículo, precios, proveedores, etc. Las existencias, precios, etc. varían continuamente y exigen una actualización simultánea del archivo con cada operación de consulta.

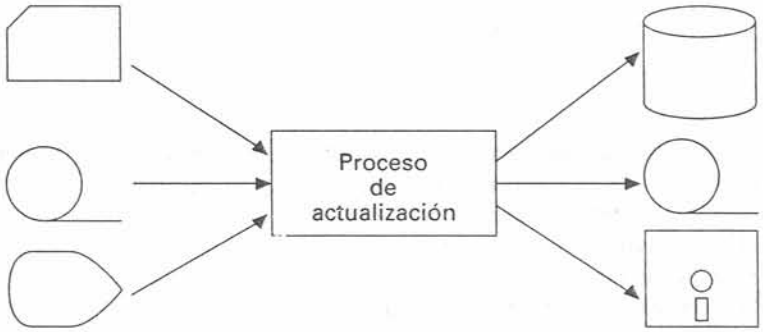


Figura 8.10. Actualización de un archivo (I).

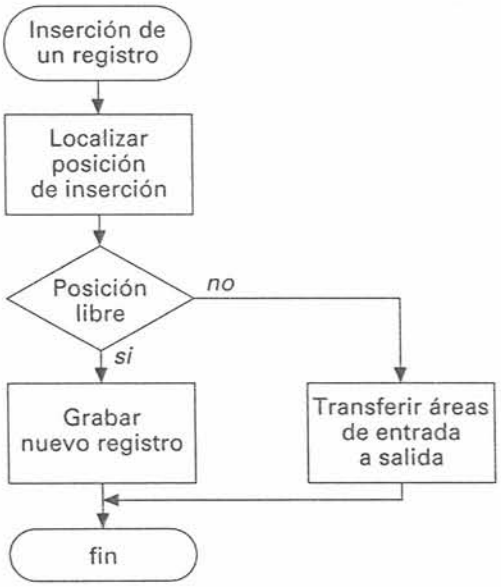


Figura 8.11. Actualización de un archivo (II).

8.5.4. Clasificación de un archivo

Una operación muy importante en un archivo es la *clasificación* u *ordenación* (*sort*, en inglés). Esta clasificación se realizará de acuerdo con el valor de un campo específico, pudiendo ser *ascendente* (creciente) o *descendente* (decreciente): alfabética o numérica.

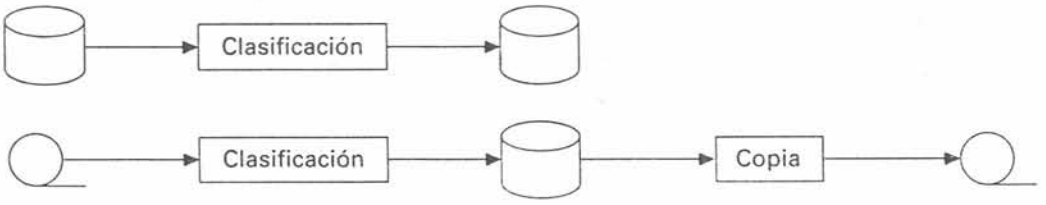


Figura 8.12. Clasificación de un archivo.

8.5.5. Reorganización de un archivo

Las operaciones sobre archivos modifican la estructura inicial o la óptima de un archivo. Los índices, enlaces (punteros), zonas de sinónimos, zonas de desbordamiento, etc. se modifican con el paso del tiempo, lo que hace a la operación de acceso al registro cada vez más lenta.

La reorganización suele consistir en la copia de un nuevo archivo a partir del archivo modificado, a fin de obtener una nueva estructura lo más óptima posible.

8.5.6. Destrucción de un archivo

Es la operación inversa a la creación de un archivo (*kill*, en inglés). Cuando se destruye (anula o borra) un archivo, éste ya no se puede utilizar y, por consiguiente, no se podrá acceder a ninguno de sus registros.

Figura 8.13. Reorganización de un archivo.

8.5.7. Reunión, fusión de un archivo

Reunión. Esta operación permite obtener un archivo a partir de otros varios.

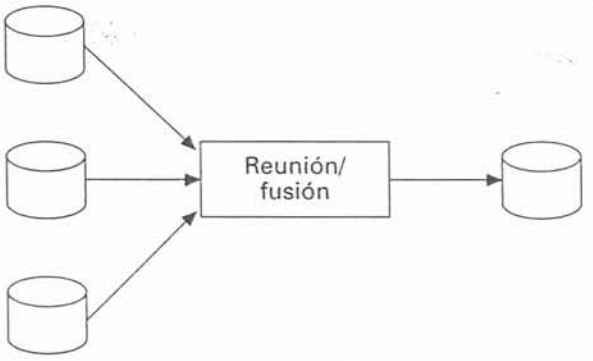


Figura 8.14. Fusión de archivos.

Fusión. Se realiza una fusión cuando se reúnen varios archivos en uno solo, intercalándose unos en otros, siguiendo unos criterios determinados.

8.5.8. Rotura/estallido de un archivo

Es la operación de obtener varios archivos a partir de un mismo archivo inicial.

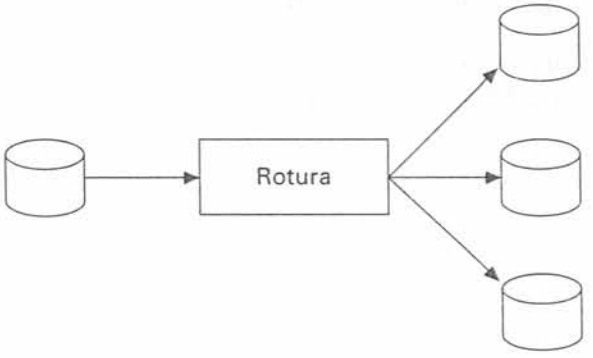


Figura 8.15. Rotura de un archivo.

8.6. GESTIÓN DE ARCHIVOS

Las operaciones más usuales en los registros son:

- *consulta*: lectura del contenido de un registro,
- *modificación*: alterar la información contenida en un registro,
- *inserción*: añadir un nuevo registro al archivo,
- *borrado*: suprimir un registro del archivo.

Las operaciones sobre archivos se realizan mediante programas, de modo tal que los archivos se identifican por un nombre externo al que están asociados. Pueden existir programas que procesen el mismo archivo de datos. La mayoría de los programas ejecutarán las siguientes clases de funciones:

- *crear archivos (create),*
- *abrir o arrancar (open)* un archivo que fue creado con anterioridad a la ejecución de este programa,
- *cerrar* el archivo después que el programa ha terminado de utilizarlo *(close),*
- *borrar (delete)* un archivo que ya existe,
- *transferir datos desde (leer) o a (escribir)* el dispositivo utilizado por el programa.

Antes de que cualquier dato pueda ser escrito o leído de un archivo, el archivo debe ser creado en el dispositivo correspondiente. *Las operaciones sobre archivos tratan con la propia estructura del archivo, no con los registros de datos dentro del archivo.*

Con anterioridad a la creación de un archivo, se requiere diseñar la estructura del mismo mediante los campos del registro, longitud y tipo de los mismos.

Para poder gestionar un archivo mediante un programa es preciso declarar el archivo, su nombre y la estructura de sus registros. La declaración se realizará con las siguientes instrucciones:

```
tipo
  registro: <tipo_registro>
      <tipo>:<nombre del campo>
       ....
    fin registro
archivo_<organizacion> de <tipo_de_dato>:<tipo_archivo>

var
  tipo_registro:nombre_registro
  tipo_archivo:identificador_archivo
```

Por ejemplo:

```
tipo
  registro:Rempleado
     cadena:nombre
     cadena:cod
     entero:edad
     real:salario
  fin registro
archivo_d de rempleado:empleado
var
  Rempleado:Re
  Empleado:E
```

8.6.1. Crear un archivo

La creación de un archivo es la operación mediante la cual se introduce la información correspondiente al archivo en un soporte de almacenamiento de datos.

Antes de que cualquier usuario pueda procesar un archivo es preciso que éste haya sido creado previamente. El proceso de creación de un archivo será la primera operación a realizar. Una vez que

el archivo ha sido creado, la mayoría de los usuarios simplemente desearán acceder al archivo y a la información contenida en él.

Para crear un nuevo archivo dentro de un sistema de computadora se necesitan los siguientes datos:

- *nombre dispositivo/usuario*: indica el lugar donde se situará el archivo cuando se cree,
- *nombre del archivo*: identifica el archivo entre los restantes archivos de una computadora,
- *tamaño del archivo*: indica el espacio necesario para la creación del archivo,
- *organización del archivo*: tipo de organización del archivo,
- *tamaño del bloque o registro físico*: cantidad de datos que se leen o escriben en cada operación de entrada/salida (E/S).

El proceso de creación de un archivo se suele incluir como un subprograma dentro de un programa principal. Al ejecutar el programa de creación de un archivo se pueden generar una *serie de errores*, entre los que se pueden destacar los siguientes:

- otro archivo con el mismo nombre ya existía en el soporte,
- el dispositivo no tiene espacio disponible para crear otro nuevo archivo,
- el dispositivo no está operacional,
- existe un problema de hardware que hace abortar el proceso,
- uno o más de los parámetros de entrada en la instrucción son erróneos.

La instrucción o acción en pseudocódigo que permite crear un archivo se codifica con la palabra `crear`.

```
crear (<var_tipo_archivo>, <nombre_fisico>)
```

8.6.2. Abrir un archivo

La acción de *abrir* (*open*) un archivo es permitir al usuario localizar y acceder a los archivos que fueron creados anteriormente.

La diferencia esencial entre una instrucción de *abrir* un archivo y una instrucción de *crear* un archivo reside en que el archivo no existe antes de utilizar `crear` y se supone que debe existir antes de utilizar `abrir`.

La información que un sistema de tratamiento de archivos requiere para abrir un archivo es diferente de las listas de información requerida para crear un archivo. La razón para ello reside en el hecho de que toda la información que realmente describe el archivo se escribió en éste durante el proceso de creación del archivo. Por consiguiente, la operación `crear` sólo necesita localizar y leer esta información conocida como atributos del archivo.

La instrucción de abrir un archivo consiste en la creación de un canal que comunica a un usuario a través de un programa con el archivo correspondiente situado en un soporte.

Los parámetros que se deben incluir en una instrucción de apertura (`abrir`) son:

- nombre del dispositivo,
- nombre del usuario o canal de comunicación,
- nombre del archivo.

Al ejecutar la instrucción **abrir** se pueden encontrar los siguientes errores:

- archivo no encontrado en el dispositivo especificado (nombre de archivo o identificador de dispositivo erróneo),
- archivo ya está en uso para alguna otra aplicación del usuario,
- errores hardware.

El formato de la instrucción es:

```
Abrir (<var_tipo_archivo>,<modo>,<nombre_fisico>)
```

La operación de abrir archivos se puede aplicar para operaciones de lectura (l), escritura (e), lectura/escritura (l/e).

```
abrir (id_archivo, l, nombre_archivo)
```

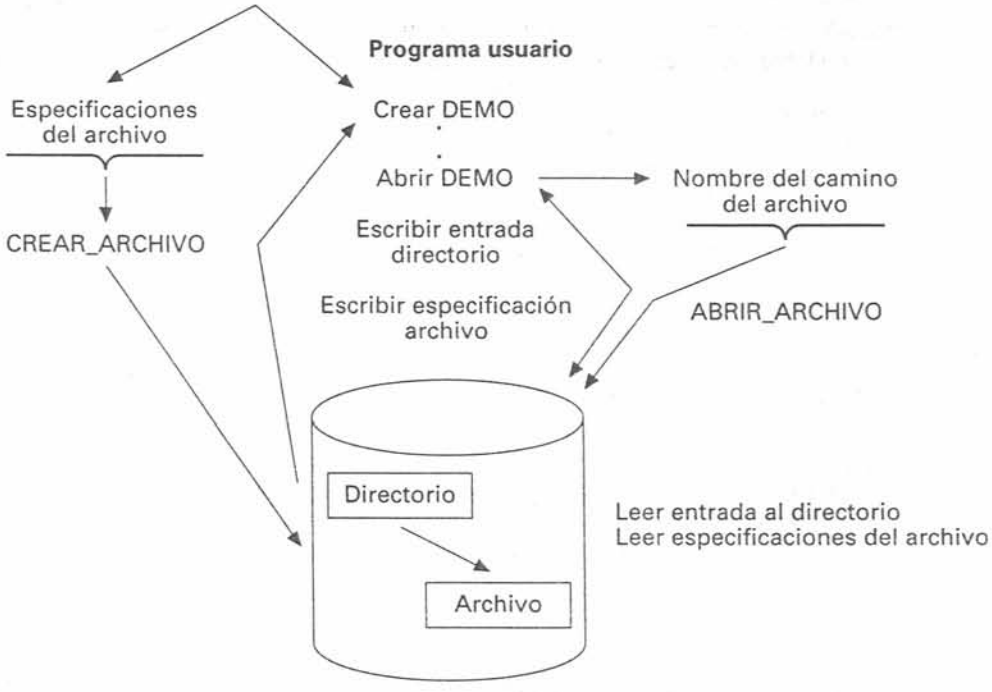


Figura 8.16. Abrir un archivo.

Para que un archivo pueda abrirse ha de haber sido previamente creado. Cuando un archivo se abre para lectura colocamos un hipotético puntero en el primer registro del archivo y se permitirán únicamente operaciones de lectura de los registros del archivo. La apertura para escritura coloca dicho hipotético puntero detrás del último registro del archivo y dispuesto para la adición de nuevos registros en él. Ambos modos se consideran propios de archivos secuenciales. Los archivos directos se abrirán en modo lectura/escritura, permitiéndose tanto la lectura como la escritura de nuevos registros.

8.6.3. Cerrar archivos

El propósito de la operación de cerrar un archivo es permitir al usuario cortar el acceso o detener el uso del archivo, permitiendo a otros usuarios acceder al archivo. Para ejecutar esta función, el sistema de tratamiento de archivos sólo necesita conocer el nombre del archivo que se debe cerrar, y que previamente debía estar abierto.

`cerrar` sitúa una marca fin de archivo a través de la función **FDA (EOF)** que se detectará en posteriores lecturas.

Formato:

```
cerrar(<var_tipo-archivo>
```

Estructura:

8.6.4. Borrar archivos

La instrucción de `borrar` tiene como objetivo la supresión de un archivo del soporte o dispositivo. El espacio utilizado por un archivo borrado puede ser utilizado para otros archivos.

La información necesaria para eliminar un archivo es:

- nombre del dispositivo,
- nombre del archivo.

Los *errores* que se pueden producir son:

- el archivo no se pudo encontrar bien porque el nombre no es válido o porque nunca existió,
- otros usuarios estaban actuando sobre el archivo y estaba activo,
- se detectó un problema de hardware.

8.7. MANTENIMIENTO DE ARCHIVOS

La operación de mantenimiento de un archivo incluye todas las operaciones que sufre un archivo durante su vida y desde su creación hasta su eliminación o borrado.

El mantenimiento de un archivo consta de dos operaciones diferentes:

- *actualización,*
- *consulta.*

La *actualización* es la operación de eliminar o modificar los datos ya existentes, o bien introducir nuevos datos. En esencia, es la puesta al día de los datos del archivo.

Las operaciones de actualización son:

- *altas,*
- *bajas,*
- *modificaciones.*

Las operaciones de consulta tienen como finalidad obtener información total o parcial de los datos almacenados en un archivo y presentados en dispositivos de salida: pantalla o impresora, bien como resultados o como listados.

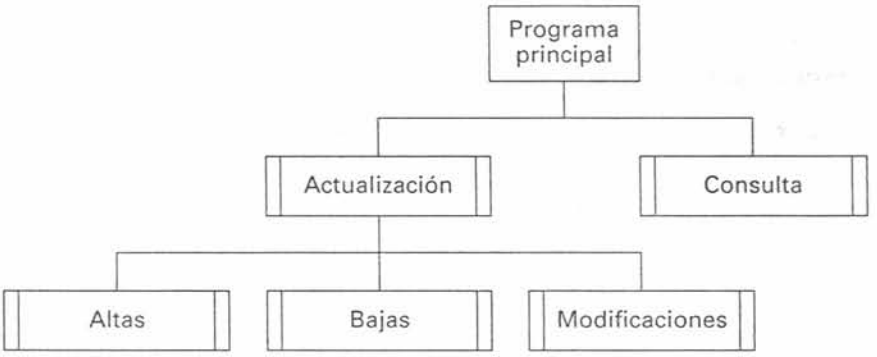
Todas las operaciones de mantenimiento de archivos suelen constituir módulos independientes del programa principal y su diseño se realiza con subprogramas (*subrutinas* o *procedimientos* específicos).

Así, los subprogramas de mantenimiento de un archivo constarán de:

Altas

Una operación de *alta* en un archivo consiste en la adición de un nuevo registro. En un archivo de empleados, un alta consistirá en introducir los datos de un nuevo empleado. Para situar correctamente un alta, se deberá conocer la posición donde se desea almacenar el registro correspondiente: al principio, en el interior o al final de un archivo.

El algoritmo del subprograma ALTAS debe contemplar la comprobación de que el registro a dar de alta no existe previamente.



Bajas

Una *baja* es la acción de eliminar un registro de un archivo. La baja de un registro se puede presentar de dos formas distintas: indicación del registro específico que se desea dar de baja o bien visualizar los registros del archivo para que el usuario elija el registro a borrar.

La baja de un registro puede ser *lógica* o *física*. Una *baja lógica* supone el no borrado del registro en el archivo. Esta baja lógica se manifiesta en un determinado campo del registro con una *bandera*, *indicador* o «*flag*» —carácter *, $, etc.—, o bien con la escritura o rellenado de espacios en blanco en el registro específico.

Una *baja física* implica el borrado y desaparición del registro, de modo que se crea un nuevo archivo que no incluye el registro dado de baja.

Modificaciones

Una *modificación* en un archivo consiste en la operación de cambiar total o parcialmente el contenido de uno de sus registros.

Esta fase es típica cuando cambia el contenido de un determinado campo de un archivo; por ejemplo, la dirección o la edad de un empleado.

La forma práctica de modificar un registro es la visualización del contenido de sus campos; para ello se debe elegir el registro o registros a modificar. El proceso consiste en la lectura del registro, modificación de su contenido y escritura, total o parcial del mismo.

Consulta

La operación de *consulta* tiene como fin visualizar la información contenida en el archivo, bien de un modo completo —bien de modo parcial—, examen de uno o más registros.

Las operaciones de consulta de archivo deben contemplar diversos aspectos que faciliten la posibilidad de conservación de datos. Los aspectos más interesantes a tener en cuenta son:

- opción de visualización en pantalla o listado en impresora,
- detención de la consulta a voluntad del usuario,
- listado por registros o campos individuales o bien listado total del archivo (en este caso deberá existir la posibilidad de impresión de listados, con opciones de saltos de página correctos).

8.7.1. Operaciones sobre registros

Las operaciones de transferencia de datos a/o desde un dispositivo a la memoria central se realizan mediante las instrucciones:

```
leer (<var_tipo_archivo>, lista de entrada de datos)
escribir (<var_tipo_archivo>, lista de salida de datos)
```

organización directa:

```
lista de entrada de datos = numero_registro, nombre_registro
lista de salida de datos  = numero_registro, nombre_registro
```

organización secuencial:

```
lista de entrada de datos = <lista_de_variables>
lista de salida de datos  = <lista_de_expresiones>
```

Las operaciones de acceso a un registro y de paso de un registro a otro se realiza con las acciones **leer** y **escribir**.

8.8. PROCESAMIENTO DE ARCHIVOS SECUENCIALES (ALGORITMOS)

En un archivo secuencial los registros se insertan en el archivo en orden cronológico de llegada al soporte, es decir, un registro de datos se almacena inmediatamente a continuación del registro anterior.

Los archivos secuenciales terminan con una marca final de archivo (FDA o EOF). Cuando se tengan que añadir registros a un archivo secuencial se añadirán en las marcas fin de archivos.

Las operaciones básicas que se permiten en un archivo secuencial son: *escribir su contenido, añadir un registro al final del archivo y consultar sus registros*. Las demás operaciones exigen una programación específica.

Los archivos secuenciales son los que ocupan menos memoria y son útiles cuando se desconoce a priori el tamaño de los datos y se requieren registros de longitud variable. También son muy empleados para el almacenamiento de información, cuyos contenidos sufran pocas modificaciones en el transcurso de su vida útil.

8.8.1. Creación

La *creación* de un archivo secuencial es un proceso secuencial, ya que los registros se almacenan consecutivamente en el mismo orden en que se introducen en el archivo.

El método de creación de un archivo consiste en la ejecución de un programa adecuado que permita la entrada de datos al archivo desde el terminal. El sistema usual es el *interactivo*, en el que el programa solicita los datos al usuario que los introduce por teclado, al terminar se introduce una marca final de archivo, que supone el final físico del archivo.

En los archivos secuenciales, EOF o FDA es una función lógica que toma el valor *cierto*, si se ha alcanzado el final de archivo, y *falso* en caso contrario.

La creación del archivo requerirá los siguientes pasos:

- abrir el archivo,
- leer datos del registro,
- grabar registro;
- cerrar archivo.

El algoritmo de creación es el siguiente:

```
algoritmo  Crea_sec
tipo
  registro:datos_personales
    <tipo_dato1>: nombre_campo1
    <tipo_dato2>: nombre_campo2
    ..........................
  fin_registro
  archivo_s de datos_personales: arch
var
  arch              :f
  datos_personales :persona
inicio
  crear (f,<nombre_en_disco>)
  abrir (f,e,<nombre_en_disco>)
  llamar_a leer_reg (persona)  //utilizamos un procedimiento para no
                               //tener que detallar la lectura
  mientras no ultimo_dato(persona) hacer
    llamar_a escribir_f_reg (f,persona) //la escritura se realizaria campo a campo
    llamar_a leer_reg(persona)
  fin_mientras
  cerrar(f)
fin
```

8.8.2. Consulta

El proceso de búsqueda o consulta de una información en un archivo de organización secuencial se debe efectuar obligatoriamente en modo secuencial. Por ejemplo, si se desea consultar la información contenida en el registro 50, se deberán leer previamente los 49 primeros registros que le preceden en orden secuencial. En el caso de un archivo de personal, si se desea buscar un registro determinado correspondiente a un determinado empleado, será necesario recorrer —leer— todo el archivo desde el principio hasta encontrar el registro que se busca o la marca final de archivos.

Así, para el caso de un archivo de n registros, el número de lecturas de registros efectuadas son:

- mínimo 1, si el registro buscado es el primero del archivo,
- máximo n, si el registro buscado es el último o no existe dentro del archivo.

Por término medio, el número de lecturas necesarias para encontrar un determinado registro es:

$$\frac{n+1}{2}$$

El tiempo de acceso será influyente en las operaciones de lectura/escritura. Así, en el caso de una lista o vector de n elementos almacenados en memoria central puede suponer tiempos de microsegundos o manosegundos; sin embargo, en el caso de un archivo de n registros los tiempos de acceso son de milisegundos o fracciones/múltiples de segundos, lo que supone un tiempo de acceso de 1.000 a 100.000 veces más grande una búsqueda de información en un soporte externo que en memoria central.

El algoritmo de consulta de un archivo requerirá un diseño previo de la presentación de la estructura de registros en el dispositivo de salida, de acuerdo al número y longitud de los campos.

```
algoritmo Consulta_sec
tipo
  registro: datos_personales
          <tipo_dato1>: nombre_campo1
          <tipo_dato2>: nombre_campo2
          ............: ............
  fin_registro
  archivo_s de datos_personales: arch
var
  arch:              f
  datos_personales: persona
```

o bien:

```
inicio
  abrir(f,1,<nombre_en_disco>)
  mientras no fda(f)hacer
    llamar_a leer_f_reg(f,persona)
    llamar_a escribir_reg (persona)
  fin_mientras
  cerrar(f)
fin
```

o bien:

```
inicio
  abrir(f,1,<nombre_en_disco>)
  llamar_a leer_f_reg(f, persona)
  mientras no fda(f) hacer
    llamar_a escribir_reg(persona)
    llamar_a leer_f_reg(f,persona)
  fin_mientras
  cerrar(f)
fin
```

En el caso de búsqueda de un determinado registro, con un campo clave *x*, el algoritmo de búsqueda se puede modificar en la siguiente forma, con

Consulta de un registro

Si el archivo no está ordenado:

```
algoritmo Consulta1_sec
tipo
  registro: datos_personales
    <tipo_dato1>: x
    <tipo_dato2>:nombre_campo2
    ..... : .....
fin_registro
archivo_s de datos_personales: arch

var
  arch              :f
  datos_personales:persona
  <tipo_dato1>      :clavebus
  logico            :encontrado

inicio
  abrir(f,1,<nombre en_disco>)
  encontrado ← falso
  leer(clavebus)
  mientras no encontrado y no fda(f) hacer
    llamar_a leer_f_reg(f, persona)
    si igual(clavebus, persona) entonces
      encontrado ← verdad
    fin_si
  fin_mientras

  si no encontrado entonces
    escribir ('No existe')
  si_no
    llamar_a escribir_reg(persona)
  fin_si
  cerrar(f)
fin
```

Si el archivo está indexado en orden creciente por el campo por el cual realizamos la búsqueda se podría acelerar el proceso, de forma que no sea necesario recorrer todo el fichero para averiguar que un determinado registro no está:

```
algoritmo Consulta2_sec
tipo
  registro: datos_personales
            <tipo_dato1>: x
            <tipo_dato2>: nombre_campo2
            .....: .....
  fin registro
  archivo_s de datos_personales: arch
var
  arch              : f
  datos_personales  : persona
  <tipo_dato1>      : clavebus
  logico            : encontrado, pasado
```

```
inicio
  abrir(f,l,<nombre_en_disco>)
  encontrado ← falso
  pasado ← falso
  leer(clavebus)
  mientras no encontrado y no pasado y no fda(f) hacer
    llamar_a leer_f_reg(f, persona)
    si igual(clavebus, persona) entonces
      encontrado ← verdad
    si_no
    fin_si
  fin_mientras
  si no encontrado entonces
    escribir ('No existe')
  si_no
    llamar_a escribir_reg(persona)
  fin_si
  cerrar(f)
fin
```

8.8.3. Actualización

La actualización de un archivo supone:

- añadir nuevos registros (*altas*),
- modificar registros ya existentes (*modificaciones*),
- borrar registros (*bajas*).

Altas

La operación de dar de alta un determinado registro es similar a la operación de añadir datos a un archivo.

```
algoritmo Añade_sec

tipo
    registro: datos_personales
          <tipo_dato1>: nombre_campo1
          <tipo_dato2: nombre_campo2
          ..........:.............
    fin_registro
    archivo_s de datos_personales:arch
var
    arch            : f
    datos_personales: persona

inicio
    abrir(f, e,<nombre_en_disco>)
    llamar_a leer_reg(persona)
    mientras no ultimo_dato(persona) hacer
        llamar_a escribir_f_reg (f,persona)
        llamar_a leer_reg (persona)
    fin_mientras
    cerrar (f)
fin
```

Bajas

Existen dos métodos para dar de baja un registro:

1. se utiliza un archivo transitorio,
2. almacenar en un array (vector) todos los registros del archivo, señalando con un indicador o bandera (*flag*) el registro que se desea dar de baja.

Método 1:

Se crea un segundo archivo auxiliar, también secuencial, copia del que se trata de actualizar. Se lee el archivo completo registro a registro y en función de su lectura se decide si el registro se debe dar de baja o no.

Si el registro se va a dar de baja, se omite la escritura en el archivo auxiliar o transitorio. Si el registro no se va a dar de baja, este registro se escribe en el archivo auxiliar.

Tras terminar la lectura del archivo original, se tendrán dos archivos: *original* (o maestro) y *auxiliar*.

El proceso de bajas del archivo concluye cambiando el nombre del archivo auxiliar por el de maestro y borrando previamente el archivo maestro original.

```
algoritmo Bajas_s

tipo
  registro: datos_personales
    <tipo_dato1>: nombre_campo1
    <tipo_dato2>: nombre_campo2
    ..........:.............
  fin_registro
    archivo_s de datos_personales:arch
var
  arch              :f,faux
  datos_personales: persona, personaaux
  logico            :encontrado

inicio
  abrir(f,1, 'antiguo')
  crear(faux, 'nuevo')
  abrir(faux, e, 'nuevo')
  leer(personaaux.nombre_campo1)
  encontrado ← falso
  mientras no fda (f) hacer
      llamar_a leer_f_reg (f, persona)
    si personaaux.nombre_campo1 = persona.nombre_campo1 entonces
    encontrado ← verdad
    si_no
      llamar_a escribir_f_reg (faux, persona)
    fin_si
```

```
  fin_mientras
  si no encontrado entonces
    escribir ('no esta')
  fin_si
  cerrar (f, faux)
  borrar ('antiguo')
  renombrar ('nuevo', 'antiguo')
fin
```

Método 2:

Este procedimiento consiste en señalar los registros que se desean dar de baja con un indicador o bandera; estos registros no se graban en el nuevo archivo secuencial que se crea sin los registros dados de baja.

Modificaciones

El proceso de modificación de un registro consiste en localizar este registro, efectuar dicha modificación y a continuación reescribir el nuevo registro en el archivo. El proceso es similar al de bajas:

```
algoritmo Modificacion_sec

tipo
  registro: datos_personales
      <tipo_dato1>: nombre_campo1
      <tipo_dato2>: nombre_campo2
      .............:.............
  fin_registro
  archivo_s de datos_personales: arch
var
  arch             :f, faux
  datos_personales: persona, personaaux
  logico           : encontrado

inicio
  abrir(f, 1, 'antiguo')
  crear(faux, 'nuevo')
  abrir(faux, e, 'nuevo')
  leer(personaaux.nombre_campo1)
  encontrado ← falso
  mientras_no fda(f) hacer
    llamar_a leer_f_reg (f, persona)
    si personaaux.nombre_campo1=persona.nombre_campo1 entonces
      encontrado ← verdad
      llamar_a modificar (persona)
    fin_si
    llamar_a escribir_f_reg (faux, persona)
  fin_mientras
  si no encontrado entonces
    escribir ('no esta')
  fin_si
  cerrar(f, faux)
  borrar('antiguo')
  renombrar ('nuevo', 'antiguo')
fin
```

El subprograma de modificación de su registro consta de unas pocas instrucciones en las que se debe introducir por teclado el registro completo con indicación de todos sus campos o, por el contrario, el campo o campos que se desea modificar. El subprograma en cuestión podría ser:

```
procedimiento modificar(E/S datos_personales: persona)
var caracter: opcion
    entero    : n

inicio
  escribir('R.- registro completo)
  escribir('C.- campos individuales')
  escribir('elija opcion:')
  leer(opcion)
  segun_sea opcion hacer
  'R': visualizar(persona)
       leer_reg(persona)
  'C': presentar(persona)
       solicitar_campo(n)
       introducir_campo(n, persona)
  fin_segun
fin_procedimiento
```

8.9. ARCHIVOS DE TEXTO

Los archivos de texto son un caso particular de archivos de organización secuencial. Un archivo de texto es una serie continua de caracteres que se pueden leer uno tras otro.

Un archivo de texto es un archivo en el que cada registro es del tipo cadena de caracteres.

El tratamiento de archivos de texto es elemental y en el caso de lenguajes como Pascal es posible detectar lectura de caracteres especiales como *fin de línea* o *fin de archivo*.

8.10. PROCESAMIENTO DE ARCHIVOS DIRECTOS (ALGORITMOS)

Se dice que un archivo es aleatorio o directo cuando cualquier registro es directamente accesible mediante la especificación de un índice, que da la posición del registro con respecto al origen del fichero. Los archivos aleatorios o directos tienen una gran rapidez para el acceso comparados con los secuenciales; los registros son fáciles de referenciar —número de orden del registro—, lo que representa una gran facilidad de mantenimiento.

La lectura/escritura de un registro es rápida, ya que se accede directamente al registro y no se necesita recorrer los anteriores.

8.10.1. Operaciones con archivos de acceso directo

Las operaciones con archivos directos son las usuales, ya vistas anteriormente.

8.10.1.1. Creación

El proceso de creación de un archivo directo o aleatorio consiste en ir introduciendo los sucesivos registros en el soporte que los va a contener en la posición deseada o en la dirección obtenida, resultante del algoritmo de conversión. Si al introducir un registro y aplicar a su clave un algoritmo de

conversión, se encuentra ocupada la dirección, el nuevo registro deberá ir a la zona de sinónimos o de excedentes.

```
algoritmo Crea_dir
  tipo
    registro: datos_personales
      <tipo_dato1> : nombre_campo1
      ..........   : ............
      <tipo_datoN> : nombre_campoN
    fin_registro
    archivo_d de datos_personales: arch
  var
    arch             : f
    datos_personales : persona

  inicio
    crear(f,<nombre_en_disco>)
    abrir(t,l/e,<nombre_en_disco>)
    ........................
    // las operaciones pueden variar con arreglo al modo como pensemos trabajar
    //posteriormente con el archivo (posicionamiento directo en un determinado
    //registro, transformacion de clave, indexacion)
    cerrar(f)
  fin
```

En los registros de un archivo directo se suele incluir un campo —ocupado— que pueda servir para distinguir un registro dado de baja o modificado de un alta o de otro que nunca contuvo información.

Dentro del proceso de creación del archivo podríamos considerar una inicialización de dicho campo en cada uno de los registros del archivo directo.

```
algoritmo Crea_dir

const
  max = <valor>
tipo
  registro: datos_personales
     <tipo_dato1>: cod
     <tipo_dato2>: ocupado            //tipo_dato2: podria ser caracter
     <tipo_datoN>: nombre_campoN
  fin_registro
  archivo_d de datos_personales: arch
var
  arch             : f
  datos_personales : persona

inicio
  crear(f,<nombre_en_disco>)
  abrir(f,l/e,<nombre_en_disco>)
  desde i ← 1 hasta Max hacer
    persona.ocupado ← ' '
    escribir(f, i, persona)
  fin_desde
  cerrar(f)
fin
```

8.10.1.2. Altas

La operación de altas en un archivo directo o aleatorio consiste en ir introduciendo los sucesivos registros en una determinada posición, especificada a través del índice. Mediante el índice nos posicionaremos directamente sobre el byte del fichero que se encuentra en la posición *(indice – 1) * tamaño_de (<tipo_registros_del_archivo>)* y escribiremos allí nuestro registro.

Altas. Tratamiento por transformación de clave

El método de transformación de clave consiste en transformar un número de orden (clave) en direcciones de almacenamiento por medio de un algoritmo de conversión.

Cuando las altas se realizan por el método de transformación de clave, la dirección donde introducir un determinado registro se conseguirá por la aplicación a la clave del algoritmo de conversión o dispersión (hash). Si encontráramos que dicha dirección ya está ocupada el nuevo registro deberá ir a la zona de sinónimos o de excedentes.

```
algoritmo Altas_dir_trcl

const
  findatos = <valor1>
  max      = <valor2>
tipo
  registro: datos_personales
      <tipo_dato1>: cod
      <tipo_dato2>: ocupado
      .......... : ...........
      <tipo_daton>: nombre_campon
      .......... : ...........
  fin_registro
  archivo_d de datos_personales: arch
var
  arch             : f
  datos_personales : persona, personaaux
  logico           : encontradohueco
  entero           : posi

inicio
  abrir(f,1/e,<nombre_en_disco>)
  leer(personaaux.cod)
  posi ← HASH(personaaux.cod)
  leer(f, posi, persona)
  si persona.ocupado ='*' entonces
    encontradohueco ← falso
    posi ← findatos
    mientras posi < Max y no encontradohueco hacer
      posi ← posi + 1
      leer(f, posi, persona)
      si persona.ocupado <> '*' entonces
        encontradohueco ← verdad
      fin_si
    fin_mientras
  si_no
    encontradohueco ← verdad
  fin_si
  si encontradohueco entonces
    llamar_a leer_otros_campos(personaaux)
```

```
    persona ← personaaux
    persona.ocupado ← '*'
  escribir(f, posi, persona)
  si_no
    escribir('no esta')
  fin_si
  cerrar(f)
fin
```

8.10.1.3. Consulta

El proceso de consulta de un archivo directo o aleatorio es rápido y debe comenzar con la entrada del índice correspondiente al registro que deseamos consultar.

El índice permitirá el posicionamiento directo sobre el byte del archivo que se encuentra en la *posición (indice – 1) * tamaño_de* (<var_de_tipo_registros_del_archivo>).

```
algoritmo Consultas_dir

const
  max       = <valor1>
tipo
  registro: datos_personales
    //Cuando el codigo coincide con el indice o posicion del registro en el
    //archivo, no resulta necesario su almacenamiento
    <tipo_dato1>: ocupado
    ..... : .....
    <tipo_daton>: nombre_campon
  fin_registro
  archivo_d de datos_personales: arch
var
  arch             : f
  datos_personales : persona
  logico           : encontrado
  entero           : posi

inicio
  abrir(f,l/e,<nombre_en_disco>)
  leer(posi)
  si (posi >= 1) y (posi <= Max) entonces
    leer(f, posi, persona)
      // como al escribir los datos marcamos el campo ocupado con *
    si persona.ocupado <>'*' entonces
      //para tener garantias en esta operacion es por lo que debemos inicializar
      //en todos los registros, durante el proceso de creacion, el campo ocupado a
      //un determinado valor, distinto de *
      encontrado ← falso
    si_no
      encontrado ← verdad
    fin_si
    si encontrado entonces
      llamar escribir_reg(persona)
    si_no
      escribir('no esta')
    fin_si
  si_no
    escribir('Numero de registro incorrecto')
  fin_si
  cerrar(f)
fin
```

Consulta. *Por transformación de clave*

Puede ocurrir que la clave o código por el que deseamos acceder a un determinado registro no coincida con la posición de dicho registro en el archivo, aunque guarden entre sí una cierta relación, pues, al escribir los registros en el archivo, la posición se obtuvo aplicando a la clave un algoritmo de conversión..

En este caso es imprescindible el almacenamiento de la clave en uno de los campos del registro y las operaciones a realizar para llevar a cabo una consulta serían:

- Definir clave del registro buscado.
- Aplicar algoritmo de conversión clave a dirección.
- Lectura del registro ubicado en la dirección obtenida.
- Comparación de las claves de los registros leído y buscado y, si son distintas, exploración secuencial del área de excedentes.
- Si tampoco se encuentra el registro en esta área es que no existe.

```
algoritmo Consultas_dir_trcl

const
  findatos = <valor1>
  max      = <valor2>
tipo
  registro: datos_personales
    <tipo_dato1>: cod
    <tipo_dato2>: ocupado
    ..... : .....
    <tipo_daton>: nombre_campon
    ..... : .....
  fin_registro
  archivo_d de datos_personales: arch
var
  arch             : f
  datos_personales : persona, personaaux
  logico           : encontrado
  entero           : posi

inicio
  abrir(f,l/e,<nombre_en_disco>)
  leer(personaaux.cod)
  posi ← HASH(personaaux.cod)
  leer(f, posi, persona)
  si persona.ocupado <>'*' o persona.cod <> personaaux.cod entonces
      encontrado ← falso
      posi ← Findatos
      mientras posi < Max y no encontrado hacer
        posi ← posi + 1
        leer(f, posi, persona)
        si persona.ocupado = '*' y persona.cod = personaaux.cod entonces
          encontrado ← verdad
    fin_si
      fin_mientras
  si_no
    encontrado ← verdad
  fin_si
```

```
    si encontrado entonces
      llamar escribir_reg(persona)
    si_no
      escribir('No esta')
    fin_si
    cerrar(f)
  fin
```

8.10.1.4. Bajas

En el proceso de bajas se considera el contenido de un campo indicador, por ejemplo, `persona.ocupado`, que, cuando existe información válida en el registro, está marcado con un *. Para dar de baja al registro, es decir considerar su información como no válida, eliminaremos dicho *. Este tipo de baja es una baja lógica.

Desarrollaremos a continuación un algoritmo que realice bajas lógicas y acceda a los registros a los que se desea dar la baja por el método de transformación de clave.

```
algoritmo Bajas_dir_trcl

const
  findatos = <valor1>
  max      = <valor2>
tipo
  registro: datos_personales
    <tipo_dato1>: cod
    <tipo_dato2>: ocupado
    .......... : ...........
    <tipo_daton>: nombre_campon
    .......... : ...........
  fin_registro
  archivo_d de datos_personales: arch

var
  arch             : f
  datos_personales : persona, personaaux
  logico           : encontrado
  entero           : posi

inicio
  abrir(f,1/e,<nombre_en_disco>)
  leer(personaaux.cod)
  posi ← HASH(personaaux.cod)
  leer(f, posi, persona)
  si persona.ocupado <>'*' o persona.cod <> personaaux.cod entonces
      encontrado ← falso
      posi ← findatos
      mientras posi < Max y no encontrado hacer
        posi ← posi + 1
        leer(f, posi, persona)
        si persona.ocupado='*' y persona.cod = personaaux.cod entonces
           encontrado ← verdad
        fin_si
      fin_mientras
    si_no
      encontrado ← verdad
  fin_si
```

```
  si encontrado entonces
    persona.ocupado ← '
    escribir(f, posi, persona)
  si_no
    escribir('No esta')
  fin_si
  cerrar(f)
fin
```

8.10.1.5. Modificaciones

En un archivo aleatorio se localiza el registro que se desea modificar —mediante la especificación del índice o aplicando el algoritmo de conversión clave a dirección y, en caso necesario, la búsqueda en la zona de colisiones— se modifica el contenido y se reescribe.

```
algoritmo Modificaciones_dir_trcl
const

  findatos = <valor1>
  max      = <valor2>
tipo
  registro: datos_personales
    <tipo_dato1>: cod
    <tipo_dato2>: ocupado
    .......... : ...........
    <tipo_daton>: nombre_campon
    .......... : ...........
  fin_registro
  archivo_d de datos_personales: arch
var
  arch             : f
  datos_personales : persona, personaaux
  logico           : encontrado
  entero           : posi

inicio
  abrir(f,l/e,<nombre_en_disco>)
  leer(personaaux.cod)
  posi ← HASH(personaaux.cod)
  leer(f, posi, persona)
  si (persona.ocupado <>'*') o (persona.cod <> personaaux.cod) entonces
    encontrado ← falso
    posi ← findatos
    mientras posi < max y no encontrado hacer
      posi ← posi + 1
      leer(f, posi, persona)
      si (persona.ocupado = '*') y (persona.cod = personaaux.cod) entonces
         encontrado ← verdad
      fin_si
    fin_mientras
  si_no
    encontrado ← verdad
  fin_si
  si encontrado entonces
    llamar_a leer_otros_campos(personaaux)
```

```
        personaaux.ocupado ← '*'
        escribir(f, posi, personaaux)
      si_no
        escribir('no esta')
      fin_si
      cerrar(f)
    fin
```

8.10.2. Clave-dirección

Con respecto a las transformaciones clave-dirección deberemos realizar todavía algunas consideraciones.

En un soporte direccionable —normalmente un disco—, cada posición se localiza por su dirección absoluta —número de pista y número de sector en el disco—. Los archivos directos manipulan direcciones relativas en lugar de absolutas, lo que hará al programa independiente de la posición absoluta del archivo en el soporte. Los algoritmos de conversión de clave transformarán las claves en direcciones relativas. Suponiendo que existen N posiciones disponibles para el archivo, los algoritmos de conversión de clave producirán una dirección relativa en el rango 1 a N por cada valor de la clave.

Existen varias técnicas para obtener direcciones relativas. En el caso en que dos registros distintos produzcan la misma dirección, se dice que se produce una colisión o sinónimo.

8.10.3. Tratamiento de las colisiones

Las colisiones son inevitables y, como se ha comentado, se originan cuando dos registros de claves diferentes producen la misma dirección relativa. En estos casos las colisiones se pueden tratar de dos formas diferentes.

Supongamos que un registro $e1$ produce una dirección $d1$ que ya está ocupada. ¿Dónde colocar el nuevo registro?

Existen dos métodos básicos:

- Considerar una zona de excedentes y asignar el registro a la primera posición libre en dicha zona. Fue el método aplicado en los algoritmos anteriores.
- Buscar una nueva dirección libre en la zona de datos del archivo.

8.10.4. Acceso a los archivos directos mediante indexación

La indexación es una técnica para el acceso a los registros de un archivo. En esta técnica, el archivo principal de registros está suplementado por uno o más índices. Los índices pueden ser archivos independientes o un array que se carga al comenzar en la memoria del ordenador; en ambos casos, estarán formados por registros con los campos código o clave y posición o número de registro.

El almacenamiento de los índices en memoria permite encontrar los registros más rápidamente que cuando se trabaja en disco.

Cuando utilicemos un archivo indexado localizaremos los registros en el índice a través del campo clave y éste nos devolverá la posición del registro en el archivo principal, directo.

Las operaciones básicas a realizar con un archivo indexado son:

- Crear las zonas de índice y datos como archivos vacíos originales.
- Cargar el archivo índice en memoria antes de utilizarlo.
- Reescribir el archivo índice desde memoria después de utilizarlo.
- Añadir registros al archivo de datos y al índice.
- Borrar registros del archivo de datos.
- Actualizar registros en el archivo de datos.

8.10.4.1. Consulta

Como ejemplo veamos la operación de consulta de un registro.

```
algoritmo Consulta_dir_ind

const
  max = <valor>
tipo
  registro: datos_personales
    <tipo_dato1>: cod
    <tipo_dato2>: nombre_campo2
    <tipo_daton>: nombre_campon
  fin_registro
  registro: datos_indice
    <tipo_dato1>: cod
    entero      : posi
  fin_registro
  archivo_d de datos_personales: arch
  archivo_d de datos_indice    : ind
  array[1..max] de datos_indice: arr
var
  arch             : f
  ind              : t
  arr              : a
  datos_personales : persona
  entero           : i, n, central
  <tipo_dato1>     : cod
  logico           : encontrado

inicio
  abrir(f,1/e,<nombre_en_disco1>)
  abrir(t,1/e,<nombre_en_disco2>)
  n ← lda(t)/tamaño_de(datos_indice)
  desde i ← 1 hasta n hacer
    leer(t,i,arr[i])
  fin_desde

  cerrar(t)
  //Debido a la forma de efectuar las altas el archivo indice siempre tiene sus
  //registros ordenados por el campo cod
  leer(cod)
  llamar_a busqueda_binaria(arr, n, cod, central, encontrado)
  // el procedimiento de busqueda_binaria en un array sera desarrollado en
  //capitulos posteriores del libro
```

```
    si encontrado entonces
      leer(f, arr [central].posi, persona)
      llamar_a escribir_reg(persona)
    si_no
      escribir('no esta')
    fin_si
  cerrar(f)
fin
```

Altas

El procedimiento empleado para dar las altas en el archivo anterior podría ser el siguiente:

```
procedimiento altas(E/S arr: a   E/S entero: n)
var
  reg           : persona
  entero        : p
  logico        : encontrado
  entero        : num

inicio
  si n = max entonces
    escribir('lleno')
  si_no
    llamar_a leer_reg(persona)
    encontrado ← falso
    busqueda_binaria(a, n, persona.cod, p, encontrado)
  si encontrado entonces
    escribir('Clave duplicada')
  si_no
    num ← lda(f)/tamaño_de(datos_personales) + 1
    //Insertamos un nuevo registro en la tabla sin que pierda su ordenacion
    alta_indice(a, n, p, persona.cod, num)
    n ← n + 1
    //Escribimos el nuevo registro al final del archivo principal
    escribir(f, num, persona)
  fin_si
  fin_si
  //en el programa principal, al terminar, crearemos de nuevo el archivo indice a
  //partir de los registros almacenados en el array a
fin_procedimiento
```

8.11. PROCESAMIENTO DE ARCHIVOS SECUENCIALES INDEXADOS

Los archivos de organización secuencial indexada contienen tres áreas: un área de datos, que agrupa a los registros, un área índice, que contiene los niveles de índice y una zona de desbordamiento o excedentes, para el caso de actualizaciones con adición de nuevos registros.

Los registros han de ser grabados obligatoriamente en orden secuencial ascendente por el contenido del campo clave y, simultáneamente a la grabación de los registros, el sistema crea los índices.

Una consideración adicional con respecto a este tipo de organización es que es posible usar más de una clave, hablaríamos así de la clave primaria y de una o más secundarias. El valor de la clave primaria es la base para la posición física de los registros en el archivo y debe ser única. Las claves secundarias pueden ser o no únicas y no afectan al orden físico de los registros.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

8.1. Escribir un algoritmo que permita la creación e introducción de los primeros datos en un archivo secuencial, PERSONAL, que deseamos almacene la información mediante registros de siguiente tipo.

```
tipo
  registro:   datos_personales
     <tipo_dato1>    :   nombre_campo1
     <tipo_dato2>    :   nombre-campo2
     ............    :   ............
  fin_registro
```

Análisis del problema

Tras la creación y apertura en modo conveniente del archivo, el algoritmo solicitará la introducción de datos por teclado y los almacenará de forma consecutiva en el archivo.
Se utilizará una función, `ultimo_dato(persona)`, para determinar el fin en la introducción de datos.

Diseño del algoritmo

```
algoritmo   Ejercicio_8_1
tipo
   registro:   datos_personales
       <tipo-dato1>  :  nombre_campo1
       <tipo-dato2>  :  nombre_campo2
       ............  :  ............
   fin_registro
   archivo_s de  datos_personales: arch
var
   arch              :  f
   datos_personales  :  persona
inicio
   crear (f,'Personal')
   abrir (f,e,'Personal')
   llamar_a leer_reg (persona)
   //  Procedimiento para la lectura de un
   //  registro campo a campo
   mientras no ultimo_dato(persona) hacer
     llamar_a escribir_f_reg (f,  persona)
     //  Procedimiento auxiliar, no desarrollado, para la
     //  escritura en el archivo del registro campo a campo
     llamar_a leer_reg(persona)
   fin_mientras
   cerrar (f)
fin
```

8.2. Supuesto que deseamos añadir nueva información al archivo PERSONAL, anteriormente creado, diseñar el algoritmo correspondiente.

Análisis del problema

Al abrir el archivo, para escritura se coloca el puntero de datos al final del mismo, permitiéndonos, con un algoritmo similar al anterior, la adición de nueva información al final del mismo.

Diseño del algoritmo

```
algoritmo Ejercicio_8_2
tipo
  registro: datos_personales
     <tipo_dato1> : nombre_campo1
     <tipo_dato2> : nombre_campo2
     .......... : ............
  fin_registro
  archivo_s de datos_personales: arch
var
  arch              : f
  datos_personales : persona
inicio
  abrir (f,e,'PERSONAL')
  llamar_a leer_reg (persona)
  mientras no ultimo_dato (persona) hacer
     llamar_a escribir_f_reg (f, persona)
     llamar_a leer_reg (persona)
  fin_mientras
  cerrar (f)
fin
```

8.3. Diseñar un algoritmo que muestre por pantalla el contenido de todos los registros del archivo PERSONAL.

Análisis del problema

Deberemos abrir el archivo para lectura y, repetitivamente, leer los registros y mostrarlos por pantalla hasta detectar el fin de fichero.
Vamos a considerar si la función FDA(id_arch) detecta el final de archivo con la lectura de su último registro.

Diseño del algoritmo

```
algoritmo Ejercicio_8_3
tipo
  registro: datos_personales
     <tipo_dato1>: nombre_campo1
     <tipo_dato2>: nombre_campo2
     .......... : ............
  fin_registro
  archivo_s de datos_personales: arch
var
  arch              : f
  datos_personales : persona
inicio
  abrir (f,l,'PERSONAL')
  mientras no fda (f) hacer
    llamar_a leer_f_reg (f, persona)
    llamar_a escribir_reg (persona)
  fin_mientras
  cerrar (f)
fin
```

Si consideráramos la existencia de un registro especial que marca el fin de archivo, la función FDA(id_arch) se activaría al leer este registro y nos veríamos obligados a modificar algo nuestro algoritmo.

```
inicio
   abrir (f,1,'PERSONAL')
   llamar_a leer_f_reg (f, persona)
   mientras no fda (f)  hacer
      llamar_a escribir_reg (persona)
      llamar_a leer_f_reg (f, persona)
  fin_mientras
  cerrar (f)
fin
```

8.4. Una librería almacena en un archivo secuencial la siguiente información sobre cada uno de sus libros: CODIGO, TITULO, AUTOR y PRECIO.
El archivo está ordenado ascendentemente por los códigos de los libros —de tipo cadena—, que no pueden repetirse.
Se precisa un algoritmo con las opciones:

1. Insertar: Permitirá insertar nuevos registros en el archivo, que debe mantenerse ordenado en todo momento.
2. Consulta: Buscará registros por el campo CODIGO.

Análisis del problema

El algoritmo comenzará presentando un menú de opciones a través del cual se haga posible la selección de un procedimiento u otro.

Insertar: Para poder colocar el nuevo registro en el lugar adecuado, sin que se pierda la ordenación inicial, necesitaremos utilizar un archivo auxilar. En dicho archivo auxiliar iremos copiando los registros hasta llegar al punto donde debe colocarse el nuevo, entonces, lo escribiremos y continuaremos con la copia de los restantes registros.

Consulta: Como el archivo está ordenado y los códigos no repetidos, el proceso de consulta se puede acelerar. Iremos recorriendo el archivo de forma secuencial hasta encontrar el código buscado, o hasta que éste sea menor que el código del registro que acabamos de leer desde el archivo, o bien, si nada de esto ocurre, hasta el fin del archivo.
Cuando el código buscado sea menor que el código del registro que acabamos de leer desde el archivo podremos deducir que de ahí en adelante ese registro ya no podrá estar en el fichero, por tanto, podremos abandonar la búsqueda.

Diseño del algoritmo

```
algoritmo Ejercicio_8_4
tipo
  registro : reg
    cadena : cod
    cadena : titulo
    cadena : autor
    entero : precio
  fin_registro
  archivo_s de reg : arch
var
  entero : op
inicio
  repetir
```

```
    escribir( 'MENU')
    escribir( '1.- INSERTAR')
    escribir( '2.- CONSULTA')
    escribir( '3.- FIN')
    escribir( 'Elija opcion ')
    leer (op )
    segun_sea op hacer
       1 : llamar_a insertar
       2 : llamar_a consulta
    fin_segun
  hasta_que op = 3
fin


procedimiento insertar
var
  arch     : f, f2
  reg      : rf,r
  logico   : escrito
  caracter : resp
inicio
  repetir
     abrir (f,1,'Libros.dat')
     crear (f2,'Nlibros.dat')
     abrir (f2,e, 'Nlibros.dat')
     escribir ('Deme el codigo')
     leer (r.cod)
     escrito ← falso
     mientras no FDA(f)
       llamar_a leer_arch_reg ( f, rf)
       si rf.cod > r.cod y no escrito entonces
       // Si leemos del fichero un registro con codigo
       // mayor que el nuevo y este aun no lo
       // hemos escrito, es el momento de insertarlo
          escribir( 'Deme otros campos ')
          llamar_a completar ( r )
          llamar_a escribir_arch_reg ( f2, r )
          escrito ← verdad
          // Deberemos marcar que lo hemos escrito
          // para que no siga insertandose, desde aqui
          // en adelante, todo el rato
       si_no
          si rf.cod = r.cod entonces
             escrito ← verdad
          fin_si
       fin_si
       llamar_a escribir_arch_reg ( f2, rf )
       // De todas formas escribimos el que
       // leemos del fichero
     fin_mientras
     si no escrito entonces
     // Si el codigo del nuevo es mayor que todos los del
     // archivo inicial, llegaremos al final sin haberlo
     // escrito
        escribir ('Deme otros campos')
        llamar_a completar (r)
        llamar_a escribir_arch_reg ( f2, r )
     fin_si
```

```
    cerrar (f, f2)
    borrar ( 'Libros.dat')
    renombrar ('Libros.dat', 'Libros.dat')
    escribir ('¿Seguir? (s/n) ')
    leer ( resp )
  hasta_que resp = 'n'
fin_procedimiento

procedimiento consulta
var
  reg: rf,r
  arch: f
  caracter: resp
  logico: encontrado, pasado
inicio
  resp ← 's'
  mientras resp <> 'n' hacer
    abrir (f, 1, 'Libros.dat')
    escribir ('Deme el codigo a buscar ')
    leer ( r.cod)
    encontrado ← falso
    pasado ← falso
    mientras no FDA (f) y no encontrado y no pasado hacer
      llamar_a leer_arch_reg (f, rf)
      si r.cod = rf.cod entonces
         encontrado ← verdad
         llamar_a escribir_reg ( rf )
      si_no
         si r.cod < rf.cod entonces
            pasado ← verdad
         fin_si
      fin_si
    fin_mientras
    si no encontrado entonces
      escribir ( 'Ese libro no esta')
    fin_si
    cerrar (f)
    escribir ('¿Seguir? (s/n)')
    leer ( resp )
  fin_mientras
fin_procedimiento
```

8.5. Diseñar un algoritmo que efectúe la creación de un archivo directo —PERSONAL—, cuyos registros serán del siguiente tipo:

```
tipo
registro:  datos_personales
   <tipo_dato1> : cod          // Campo clave
   ...........  : ............
   <tipo_datoN> : nombre_campoN
fin_registro
```

y en el que, posteriormente, vamos a introducir la información empleando el método de transformación de clave.

Análisis del problema

El método de transformación de claves consiste en introducir los registros, en el soporte que los va a contener, en la dirección que proporciona el algoritmo de conversión. Su utilización obliga al almacena-

miento del código en el propio registro y hace conveniente la inclusión en el registro de un campo auxiliar —ocupado— en el que se marque si el registro está o no ocupado. Durante el proceso de creación deberemos realizar un recorrido de todo el archivo inicializando el campo ocupado a vacío, por ejemplo a espacio.

Diseño del algoritmo

```
algoritmo Ejercicio_8_5
const
  Max = <valor>
tipo
  registro: datos_personales
     <tipo_dato1> : cod // Podria no ser necesario
                        // su almacenamiento, en el caso
                        // de que coincidiera con el
                        // indice
     ..... : .....
     <tipo_daton> : nombre_campon
  fin_registro
  archivo_d de datos_personales: arch
var
  arch             : f
  datos personales : persona
  entero           : i
inicio
  crear (f,'PERSONAL')
  abrir (f,l/e, 'PERSONAL')
  desde i ← 1 hasta Max hacer
     persona.ocupado ' '
     escribir (f, persona, i)
  fin_desde
  cerrar (f)
fin
```

8.6. Supuesto que deseamos introducir información, por el método de transformación de clave, en el archivo PERSONAL creado en el ejercicio anterior, diseñar el algoritmo correspondiente.

Análisis del problema

Como anteriormente explicábamos, el método de transformación de claves consiste en introducir los registros, en el soporte que los va a contener, en la dirección que proporciona el algoritmo de conversión. A veces, registros distintos, sometidos al algoritmo de conversión, proporcionan una misma dirección, por lo que deberemos tener previsto un espacio en el disco para el almacenamiento de los registros que han consolidado. Aunque se puede hacer de diferentes maneras, en nuestro caso, reservaremos espacio para las colisiones en el propio fichero a continuación de la zona de datos.
Supondremos que la dirección más alta capaz de proporcionar el algoritmo de conversión es Findatos y colocaremos las colisiones que se produzcan a partir de allí en posiciones consecutivas del archivo. La inicialización a espacio del campo ocupado se realizó hasta Max, damos por supuesto que Max es mayor que Findatos.

Diseño del algoritmo

```
algoritmo  Ejercicio_8_6
const
  Findatos = <valor1>
  Max      = <valor2>
```

```
tipo
  registro: datos_personales
    <tipo_dato1> : cod // Podria no ser necesario
                       // su almacenamiento, en el caso
                       // de que coincidiera con el
                       // indice
    ..... : .....
    <tipo_daton> : nombre_campon
  fin_registro
  [illegible] ssonales . rch
var
  arch             : f
  datos_personales : persona, personaaux
  logico           : encontradohueco
  entero           : posi
inicio
  abrir (f,1/e,'PERSONAL')
  leer (personaaux.cod)
  posi ← HASH (personaaux.cod)
  // HASH es el nombre de la funcion de transformacion de
  // claves. La cual devolvera valores
  // entre 1 y Findatos, ambos inclusive
  leer(f, persna, posi)
  si persona.ocupado ='*' entonces //El '*' indica que esta
                                   //ocupado
    encontradohueco ← falso
    posi ← Findatos
    mientras posi < Max y no encontradohueco hacer
      posi ← posi + 1
      leer(f, persona, posi)
      si persona.ocupado <> '*' entonces
        encontradohueco ← verdad
      fin_si
    fin_mientras
  si_no
    encontradohueco ← verdad
  fin_si
  si encontradohueco entonces
    llamar_a leer_otros_campos (personaaux)
    persona ← personaaux
    persona.ocupado ← '*' //Al dar un alta marcaremos
                          //el campo ocupado
    escribir(f, persona, posi)
  si_no
    escribir ('No esta')
  fin_si
  cerrar (f)
fin
```

EJERCICIOS

8.1. Diseñar un algoritmo que permita crear un archivo AGENDA de direcciones cuyos registros constan de los siguientes campos:

```
NOMBRE
DIRECCION
CIUDAD
CODIGO POSTAL
TELEFONO
EDAD
```

8.2. Realizar un algoritmo que lea el archivo AGENDA e imprima los registros de toda la agenda.

8.3. Diseñar un algoritmo que copie el archivo secuencial AGENDA de los ejercicios anteriores en un archivo directo DIRECTO_AGENDA, de modo que cada registro mantenga su posición relativa.

8.4. Se dispone de un archivo indexado denominado DIRECTORIO, que contiene los datos de un conjunto de personas y cuya clave es el número del DNI. Escribir un algoritmo capaz de realizar una consulta de un registro. Si no se encuentra el registro se emite el correspondiente mensaje de ERROR.

8.5. Se dispone de un archivo STOCK correspondiente a la existencia de artículos de un almacén y se desea señalar aquellos artículos cuyo nivel está por debajo del mínimo y que visualicen un mensaje 'hacer pedido'. Cada artículo contiene un registro con los siguientes campos: número del código del artículo, nivel mínimo, nivel actual, proveedor, precio.

8.6. El director de un colegio desea realizar un programa que procese un archivo de registros correspondiente a los diferentes alumnos del centro a fin de obtener los siguientes datos:

- nota más alta y número de identificación del alumno correspondiente,
- nota media por curso,
- nota media del colegio.

NOTA: Si existen varios alumnos con la misma nota más alta, se deberán visualizar todos ellos.

8.7. Diseñar un algoritmo que genere un archivo secuencial BIBLIOTECA, cuyos registros contienen los siguientes campos:

```
TITULO
AUTOR
EDITORIAL
AÑO DE EDICION
ISBN
NUMERO DE PAGINAS
```

8.8. Diseñar un algoritmo que permita modificar el contenido de alguno de los registros del archivo secuencial BIBLIOTECA mediante datos introducidos por teclado.

CAPÍTULO 9

ORDENACIÓN, BÚSQUEDA E INTERCALACIÓN

CONTENIDO

9.1. Introducción.
9.2. Ordenación.
9.3. Búsqueda.
9.4. Intercalación.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

Las computadoras emplean una gran parte de su tiempo en operaciones de *búsqueda, clasificación* y *mezcla de datos*. Las operaciones de cálculo numérico y, sobre todo, de gestión requieren normalmente operaciones de clasificación de los datos: ordenar fichas de clientes por orden alfabético, por direcciones o por código postal.

Existen dos métodos de ordenación: *ordenación interna* (de arrays) y *ordenación externa* (archivos). Los arrays se almacenan en la memoria interna o central, de acceso aleatorio y directo, y por ello su gestión es rápida.

Los *archivos* se sitúan adecuadamente en dispositivos de almacenamiento externo, que son más lentos y basados en dispositivos mecánicos: cintas y discos magnéticos.

Las técnicas de ordenación, búsqueda y mezcla son muy importantes y el lector deberá dedicar especial atención al conocimiento y aprendizaje de los diferentes métodos que en este capítulo se analizan.

9.1. INTRODUCCIÓN

Ordenación, *búsqueda* y, en menor medida, *intercalación* son operaciones básicas en el campo de la documentación y en las que, según señalan las estadísticas, las computadoras emplean la mitad de su tiempo.

Aunque su uso puede ser con vectores (arrays) y con archivos, este capítulo se referirá a vectores.

La *ordenación* (*clasificación*) es la operación de organizar un conjunto de datos en algún orden dado, tal como creciente o decreciente en datos numéricos, o bien en orden alfabético directo o inverso. Estructuras de datos típicas que suelen requerir ordenación son: lista de números, archivos de clientes de banco, nombres de una agenda telefónica, etc. En síntesis, la ordenación significa poner objetos en orden (orden numérico para los números y alfabético para los caracteres) ascendente o descendente.

Por ejemplo, las clasificaciones de los equipos de fútbol de la liga en la 1.ª división española se pueden organizar en orden alfabético creciente/decreciente o bien por clasificación numérica ascendente/descendente.

Los nombres de los equipos y los puntos de cada equipo se almacenan en dos vectores:

```
equipo[1] = 'Real Madrid'        puntos[1] = 10
equipo[2] = 'Barcelona'          puntos[2] = 14
equipo[3] = 'Valencia'           puntos[3] =  8
equipo[4] = 'Oviedo'             puntos[4] = 12
equipo[5] = 'Betis'              puntos[5] = 16
```

Si los vectores se ponen en orden decreciente de puntos de clasificación:

```
equipo[5] = 'Betis'              puntos[5] = 16
equipo[2] = 'Barcelona'          puntos[2] = 14
equipo[4] = 'Oviedo'             puntos[4] = 12
equipo[1] = 'Real Madrid'        puntos[1] = 10
equipo[3] = 'Valencia'           puntos[3] =  8
```

Consideremos ahora otros dos vectores con los nombres de los equipos y los puntos conseguidos en el Campeonato de liga, ordenados de modo alfabético:

```
equipo[1] = 'Barcelona'          puntos[1] =  5
equipo[2] = 'Cadiz'              puntos[2] = 13
equipo[3] = 'Malaga'             puntos[3] = 12
equipo[4] = 'Oviedo'             puntos[4] =  8
equipo[5] = 'Real Madrid'        puntos[5] =  4
equipo[6] = 'Valencia'           puntos[6] = 16
```

Se pueden situar en orden numérico decreciente:

```
equipo[6] = 'Valencia'           puntos[6] = 16
equipo[2] = 'Cadiz'              puntos[2] = 13
equipo[3] = 'Malaga'             puntos[3] = 12
equipo[4] = 'Oviedo'             puntos[4] =  8
equipo[1] = 'Barcelona'          puntos[1] =  5
equipo[5] = 'Real Madrid'        puntos[5] =  4
```

Los vectores anteriores comienzan en orden alfabético de equipos y se reordenan en orden descendente de 'puntos'. El listín telefónico se clasifica en orden alfabético de abonados; un archivo de clientes de una entidad bancaria normalmente se clasifica en orden ascendente de números de cuenta. El propósito final de la clasificación es facilitar la manipulación de datos en un vector o en un archivo.

Algunos autores diferencian entre un conjunto o *vector clasificado* (*sorted*) y *vector ordenado* (*ordered set*). Un *conjunto ordenado* es aquél en el que el orden de aparición de los elementos afecta al significado de la estructura completa de datos: puede estar clasificado, pero no es imprescindible. Un *conjunto clasificado* es aquél en que los valores de los elementos han sido utilizados para disponerlos en un orden particular: es, probablemente, un conjunto ordenado, pero no necesariamente.

Es importante estudiar la clasificación por dos razones. Una es que la clasificación de datos es tan frecuente que todos los usuarios de computadoras deben conocer estas técnicas. La segunda es que es una aplicación que se puede describir fácilmente, pero que es bastante difícil conseguir el diseño y escritura de buenos algoritmos.

La clasificación de los elementos numéricos del vector

```
7, 3, 2, 1, 9, 6, 7, 5, 4
```

en orden ascendente producirá

```
1, 2, 3, 4, 5, 6, 7, 7, 9
```

Obsérvese que pueden existir elementos de igual valor dentro de un vector.

Existen muchos algoritmos de clasificación, con diferentes ventajas e inconvenientes. Uno de los objetivos de este capítulo y del Capítulo 10 es el estudio de los métodos de clasificación más usuales y de mayor aplicación.

La *búsqueda* de información es, al igual que la ordenación, otra operación muy frecuente en el tratamiento de información. La búsqueda es una actividad que se realiza diariamente en cualquier aspecto de la vida: búsqueda de palabras en un diccionario, nombres en una guía telefónica, localización de libros en una librería. A medida que la información se almacena en una computadora, la recuperación y búsqueda de esa información se convierte en una tarea principal de dicha computadora.

9.2. ORDENACIÓN

Además de la búsqueda en un vector es necesario, con frecuencia, clasificar u ordenar sus elementos en un orden particular. Por ejemplo, clasificar un conjunto de números en orden creciente o una lista de nombres por orden alfabético.

La clasificación es una operación tan frecuente en programas de computadora que una gran cantidad de algoritmos se han diseñado para clasificar listas de elementos con eficacia y rapidez.

La elección de un determinado algoritmo depende del tamaño del vector o array a clasificar, el tipo de datos y la cantidad de memoria disponible.

La *ordenación* o *clasificación* es el proceso de organizar datos en algún orden o secuencia específica, tal como creciente o decreciente, para datos numéricos, o alfabéticamente, para datos de caracteres.

Los *métodos de ordenación* se dividen en dos categorías:

- ordenación de vectores, tablas (arrays),
- ordenación de archivos.

La ordenación de arrays se denomina también *ordenación interna*, ya que se almacena en la memoria interna de la computadora de gran velocidad y acceso aleatorio. La ordenación de archivos se suele hacer casi siempre sobre soportes de almacenamiento externo, discos, cintas, etc. y, por ello, se denomina también *ordenación externa*. Estos dispositivos son más lentos en las operaciones de entrada/salida, pero, por el contrario, pueden contener mayor cantidad de información.

Ordenación interna: clasificación de los valores de un vector según el orden creciente en memoria central: rápida.
Ordenación externa: clasificación de los registros de un archivo situado en un soporte externo: menos rápido.

Ejemplo: clasificación en orden ascendente del vector

```
7, 3, 2, 1, 9, 6, 7, 5, 4
```

se obtendrá el nuevo vector

```
1, 2, 3, 4, 5, 6, 7, 7, 9
```

Los métodos de clasificación se explicarán aplicados a vectores (arrays unidimensionales), pero se pueden extender a matrices o tablas (arrays bidimensionales), considerando la ordenación respecto a una fila o columna.

Los *métodos directos* son los que se realizan en el espacio ocupado por el array. Los más populares son:

- *intercambio,*
- *inserción,*
- *selección,*
- *Shell,*
- *ordenación rápida (quick-sort).*

9.2.1. Método de intercambio o de burbuja

El algoritmo de clasificación de *intercambio* o *de la burbuja* se basa en el principio de comparar pares de elementos adyacentes e intercambiarlos entre sí hasta que estén todos ordenados.

Supongamos que se desea clasificar en orden ascendente el vector o lista

```
 50      15      56      14      35      1       12      9
A[1]    A[2]    A[3]    A[4]    A[5]    A[6]    A[7]    A[8]
```

Los pasos a dar son:

1. Comparar `A[1]` y `A[2]`; si están en orden, se mantienen como están; en caso contrario, se intercambian entre sí.
2. A continuación se comparan los elementos 2 y 3; de nuevo se intercambian si es necesario.
3. El proceso continúa hasta que cada elemento del *vector* ha sido comparado con sus elementos adyacentes y se han realizado los intercambios necesarios.

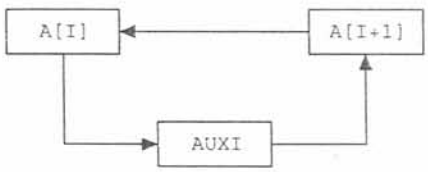
El método expresado en pseudocódigo en el primer diseño es:

```
desde I ← 1 hasta 7 hacer
  si elemento[I] > elemento[I + 1] entonces
    intercambiar (elemento [I], elemento [I + 1])
  fin_si
fin_desde
```

La acción *intercambiar* entre sí los valores de dos elementos `A[I]`, `A[I + 1]` es una acción compuesta que contiene las siguientes acciones, considerando una variable auxiliar `AUXI`:

```
AUXI ← A[I]
A[I] ← A[I + 1]
A[I + 1] ← AUXI:
```

En realidad, el proceso gráfico es:



El elemento cuyo valor es mayor va posición a posición hacia el final de la lista, al igual que las burbujas de aire en un depósito o botella de agua. Tras realizar un recorrido completo por todo el vector, el elemento mencionado habrá subido en la lista y ocupará la última posición. En el segundo recorrido, el segundo elemento llegará a la penúltima, y así sucesivamente.

En el ejercicio citado anteriormente los sucesivos pasos con cada una de las operaciones se muestran en las Figuras 9.1 y 9.2.

Vector inicial	*1.ª comp.*	*2.ª comp.*	...				
A[1]	50	15	15	15	15	15	15	15
A[2]	15	50	50	50	50	50	50	50
A[3]	56	56	56	14	14	14	14	14
A[4]	14	14	14	56	35	35	35	35
A[5]	35	35	35	35	56	1	1	1
A[6]	1	1	1	1	1	56	12	12
A[7]	12	12	12	12	12	12	56	9
A[8]	9	9	9	9	9	9	9	56

Figura 9.1. Método de la burbuja (paso 1).

Si se efectúa $n - 1$ veces la operación sobre una tabla de n valores se tiene ordenada la tabla. Cada operación requiere $n - 1$ comprobaciones o tests y como máximo $n - 1$ intercambios. La ordenación total exigirá un máximo de

$(n - 1) * (n - 1) = (n - 1)^2$ intercambios de elementos.

Los estados sucesivos del vector se indican en la Figura 9.2:

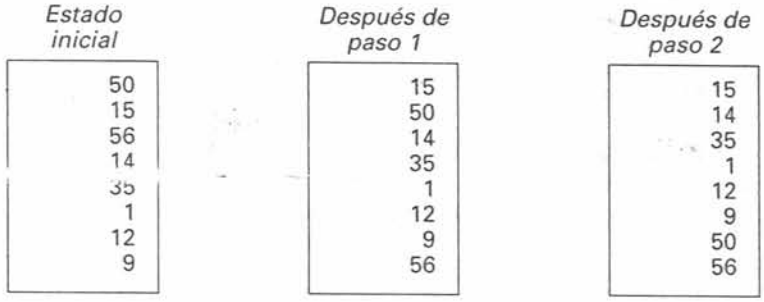


Figura 9.2. Método de la burbuja (paso 2).

Ejemplo 9.1

Describir los diferentes pasos para clasificar en orden ascendente el vector

```
72   64   50   23   84   18   37   99   45   8
```

Las sucesivas operaciones en cada uno de los pasos necesarios hasta obtener la clasificación final se muestran en la Tabla 9.1

MÉTODO 1:

El algoritmo se describirá, como siempre, con un diagrama de flujo y un pseudocódigo.

Pseudocódigo

```
algoritmo Burbuja1
//incluir las declaraciones precisas
inicio
  //lectura del vector
  desde i ← 1 hasta N hacer
    leer(X[I])
  fin_desde
```

Tabla 9.1. Pasos necesarios de la ordenación por burbuja

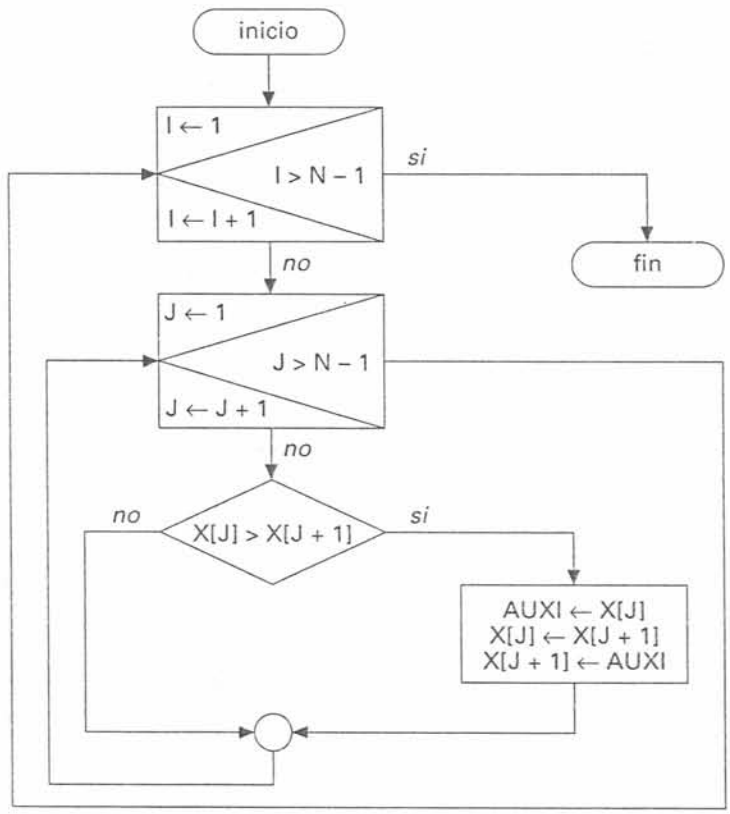
Vector desordenado	Número de paso 1	2	3	4	5	6	Fin de clasificación 7	8	9
72	64	50	23	23	18	18	18	18	8
64	50	23	50	18	23	23	23	8	18
50	23	64	18	37	37	37	8	23	23
23	72	18	37	50	45	8	37	37	37
84	18	37	64	45	8	45	45	45	45
18	37	72	45	8	50	50	50	50	50
37	84	45	8	64	64	64	64	64	64
99	45	8	72	72	72	72	72	72	72
45	8	84	84	84	84	84	84	84	84
8	99	99	99	99	99	99	99	99	99

```
    //clasificacion del vector
    desde I ← 1 hasta N-1 hacer
      desde J ← 1 hasta J ← N - 1 hacer
        si X[j] > X[J + 1] entonces
        //intercambiar
          AUXI ← X[J]
          X[J] ← X[J + 1]
          X[J+1] ← AUXI
        fin_si
      fin_desde
    fin_desde
    //imprimir lista clasificada
    desde J ← 1 hasta N hacer
      escribir(X[J])
    fin_desde
  fin
```

Diagrama de flujo 9.1

Para clasificar el vector completo se deben realizar las sustituciones correspondientes `(N - 1) * (N - 1)` o bien $N^2 - 2N + 1$ veces. Así, en el caso de un vector de cien elementos `(N = 100)`, se deben realizar casi 10.000 iteraciones.

El algoritmo de clasificación es:



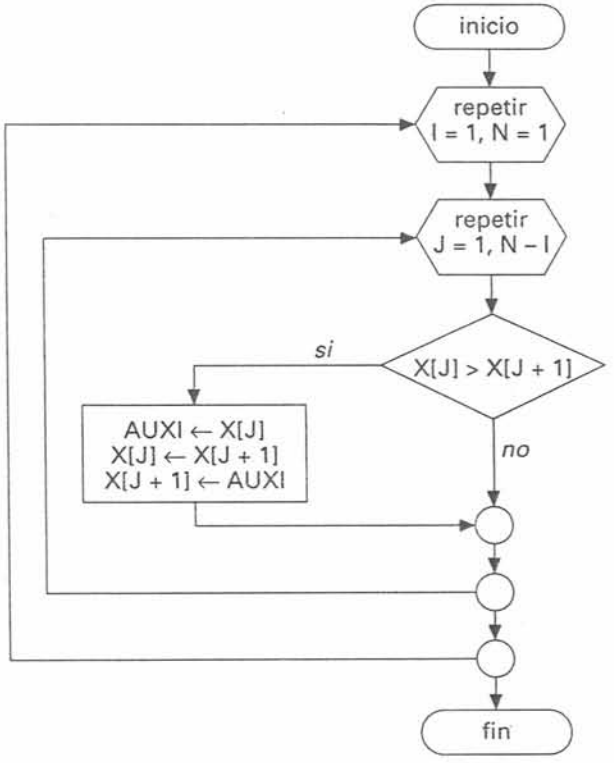
MÉTODO 2:

Se puede realizar una *mejora en la velocidad de ejecución del algoritmo*. Obsérvese que en el primer recorrido del vector (cuando `I = 1`) el valor mayor del vector se mueve al último elemento `X[N]`. Por consiguiente, en el siguiente paso no es necesario comparar `X[N - 1]` y `X[N]`. En otras palabras, el límite superior del bucle **desde** puede ser `N - 2`. Después de cada paso se puede decrementar en uno el límite superior del bucle **desde**. El algoritmo sería:

Pseudocódigo

```
algoritmo Burbuja2
inicio
  desde I ← 1 hasta N - 1 hacer
    desde J ← 1 hasta N - I hacer
      si X[J] > X[J + 1] entonces
        AUXI ← X[J]
        X[J] ← X[J + 1]
        X[J + 1] ← AUXI
      fin_si
    fin_desde
  fin_desde
fin
```

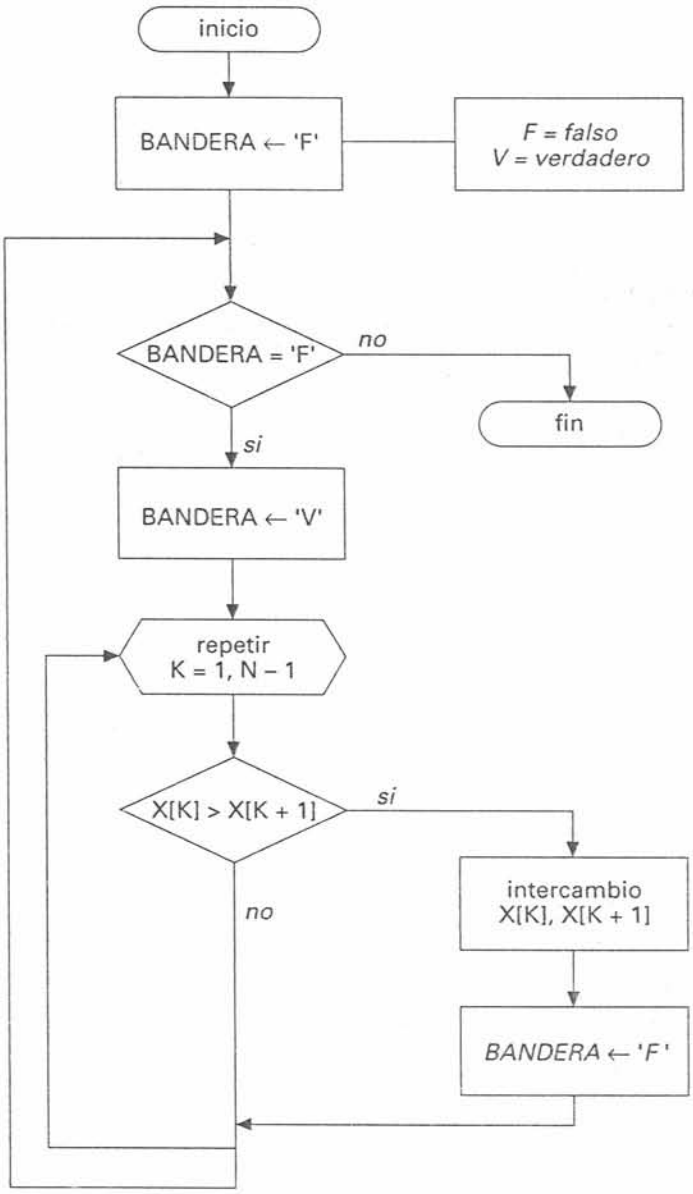
Diagrama de flujo 9.2



MÉTODO 3 (USO DE UNA BANDERA/INDICADOR):

Mediante una bandera/indicador o centinela (*switch*) o bien una variable lógica, se puede detectar la presencia o ausencia de una condición. Así, mediante la variable BANDERA se representa *clasificación terminada* con un valor verdadero, y *clasificación no terminada* con un valor falso.

Diagrama de flujo 9.3



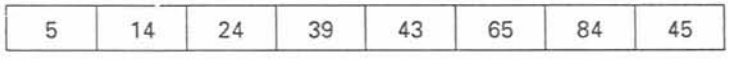
Pseudocódigo

```
algoritmo Burbuja3

inicio
  BANDERA ← 'F' //F, falso; V, verdadero
  mientras BANDERA = 'F' hacer
    BANDERA ← 'V'
    desde K ← 1 hasta N - 1 hacer
      si X[K] > X[K + 1] entonces
        intercambiar(X[K], X[K + 1]) //llamada a un procedimiento de intercambio
          BANDERA ← 'F'
      fin_si
    fin_desde
  fin_mientras
fin
```

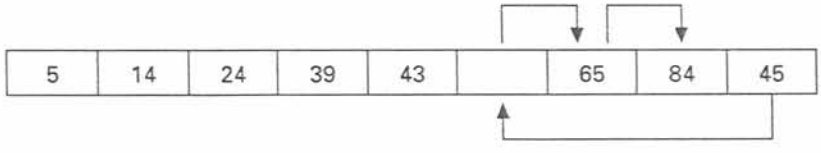
9.2.2. Ordenación por inserción

Este método consiste en insertar un elemento en el vector en una parte ya ordenada de este vector y comenzar de nuevo con los elementos restantes. Por ser utilizado generalmente por los jugadores de cartas se le conoce también por el nombre de *método de la baraja*.

Así, por ejemplo, suponga que tiene la lista desordenada



Para insertar el elemento 45, habrá que insertarlo entre 43 y 65, lo que supone desplazar a la derecha todos aquellos números de valor superior a 45, es decir, saltar sobre 65 y 84.



El método se basa en comparaciones y desplazamientos sucesivos. El algoritmo de clasificación de un vector X para N elementos se realiza con un recorrido de todo el vector y la inserción del elemento correspondiente en el lugar adecuado. El recorrido se realiza desde el segundo elemento al *n*-ésimo.

```
desde i ← 2 hasta N hacer
  insertar X[i] en el lugar adecuado
  entre x[1]..x[i - 1]
fin_desde
```

Esta acción repetitiva —*insertar*— se realiza más fácilmente con la inclusión de un valor centinela o bandera (SW).

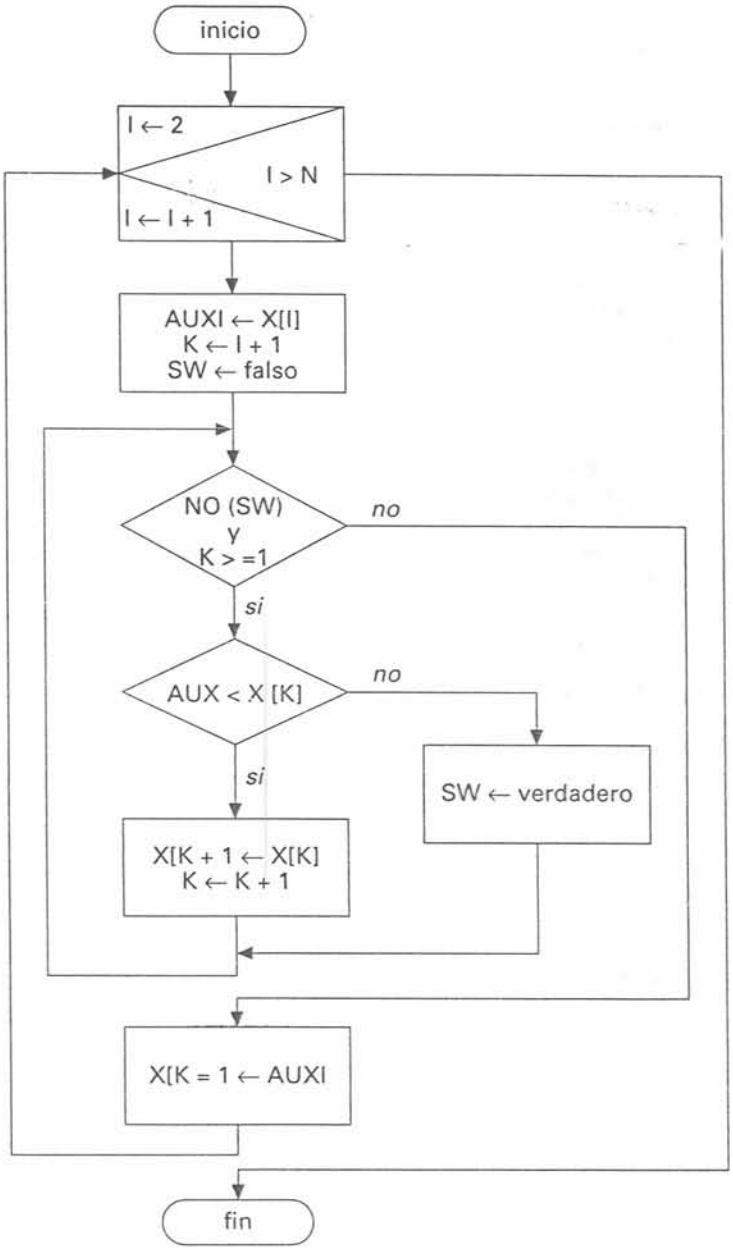
Pseudocódigo

```
algoritmo Clas_insercion1
//declaraciones

inicio
  .....
  //ordenacion
  desde I ← 2 hasta N hacer
    AUXI ← X[I]
    K ← I - 1
    SW ← falso
    mientras no (SW) y (K >= 1) hacer
      si AUXI < X[K] entonces
        X[K + 1] ← X[K]
        K ← K - 1
      si_no
        SW ← verdad
      fin_si
    fin_mientras
    X[K + 1] ← AUXI
  fin_desde
fin
```

Algoritmo inserción mejorado

El algoritmo de inserción directa se mejora fácilmente. Para ello se recurre a un método de búsqueda binaria —en lugar de una búsqueda secuencial— para encontrar más rápidamente el lugar de inserción. Este método se conoce como *inserción binaria*.

```
algoritmo Clas_insercion_binaria
//declaraciones

inicio
  desde I ← 2 hasta N hacer
    AUX ← X[I]
    P ← 1     {primero}
    U ← I-1   {ultimo}
    mientras P <= U hacer
      C ← (P + U) div 2
      si AUX < X[C] entonces
        U ← C - 1
      si_no
        P ← C + 1
      fin_si
    fin_mientras
    desde K ← I - 1 hasta P decremento 1 hacer
      X[K + 1] ← X[K]
    fin_desde
      X[P] ← AUX
  fin_desde
fin
```



Número de comparaciones

El cálculo del número de comparaciones `F(n)` que se realiza en el algoritmo de inserción se puede calcular fácilmente.

Consideraremos el elemento que ocupa la posición `X` en un vector de `n` elementos, en el que los `X - 1` elementos anteriores se encuentran ya ordenados ascendentemente por su clave.

Si la clave del elemento a insertar es mayor que las restantes, el algoritmo ejecuta sólo una comparación; si la clave es inferior a las restantes, el algoritmo ejecta `X - 1` comparaciones.

El número de comparaciones tiene por media `X/2`.

Veamos los casos posibles.

Vector ordenado en origen:

Comparaciones mínimas: $(n-1)$

Vector inicialmente en orden inverso:

Comparaciones máximas: $\dfrac{N(N-1)}{2}$

ya que

$$(n-1)+(n-2)+\ldots+3+2+1=\frac{(n-1)n}{2} \quad \textit{es una progresión aritmética}$$

Comparaciones medias: $\dfrac{(n-1)+(n-1)n/2}{2}=\dfrac{N^2+N-2}{4}$

otra forma dededucirlas sería:

$$Cmedias=\underbrace{\frac{N-1+1}{2}+\frac{N-2+1}{2}+\ldots+\frac{1+1}{2}}_{N-1\ veces}$$

y la suma de los términos de una progresión aritmética es:

$$Cmedias=(N-1)\frac{\left(\frac{N}{2}\right)+1}{2}=(N-1)\frac{N+2}{4}=\frac{N^2+2N-N-2}{4}=\frac{N^2+N-2}{4}$$

9.2.3. Ordenación por selección

Este método se basa en buscar el elemento menor del vector y colocarlo en primera posición. Luego se busca el segundo elemento más pequeño y se coloca en la segunda posición, y así sucesivamente.

Los pasos sucesivos a dar son:

1. Seleccionar el elemento menor del vector de n elementos.
2. Intercambiar dicho elemento con el primero.
3. Repetir estas operaciones con los $n-1$ elementos restantes, seleccionando el segundo elemento; continuar con los $n-2$ elementos restantes hasta que sólo quede el mayor.

Un ejemplo aclarará el método.

Ejemplo 9.2

Clasificar la siguiente lista de números en orden ascendente:

```
320   96   16   90   120   80   200   64
```

El método comienza buscando el número más pequeño.

```
320   96   16   90   120   80   200   64
```

La lista nueva será:

```
16   96   320   90   120   80   200   64
```

A continuación, se busca el siguiente número más pequeño, `64`, y se realizan las operaciones 1 y 2.

La nueva lista sería:

```
16   64   320   90   120   80   200   96
```

Si se siguen realizando las iteraciones, se encontrarán las siguientes líneas:

```
16   64   80   90   120   320   200   96
```

No se realiza ahora ningún cambio, ya que el número más pequeño del vector `V[4], V[5], ..., V[8]` está ya en la posición más a la izquierda. Las sucesivas operaciones serán:

```
16   64   80   90   96   320   200   120
16   64   80   90   96   120   200   320
16   64   80   90   96   120   200   320
```

y se habrán terminado las comparaciones, ya que el último elemento debe ser el más grande y, por consiguiente, estará en la posición correcta.

Desarrollemos ahora el algoritmo para clasificar el vector `V` de `n` componentes `V[1], V[2], ..., V[n]` con este método. El algoritmo se presentará en etapas y lo desarrollaremos con un refinamiento por pasos sucesivos.

La tabla de variables que utilizaremos será:

`I, J`	*enteras* y se utilizan como índices del vector `V`
`X`	*vector* (array unidimensional)
`AUXI`	variable auxiliar para intercambio
`N`	número de elementos del vector `V`

Nivel 1:

```
inicio
  desde I ← 1 hasta N - 1 hacer
    Buscar elemento menor de X[I], X[I + 1], ..., X[N] e     intercambiar con X[I]
  fin_desde
fin
```

Nivel 2:

```
inicio
  I ← 1
  repetir
    Buscar elemento más pequeño de X[I], X[I + 1], ..., X[N] y cambiar con X[I]
    I ← I + 1
  hasta_que I = N
fin
```

La selección e intercambio se realiza `N - 1` veces, ya que `I` se incrementa en `1` al final del bucle.

Nivel 3:

Dividamos el bucle repetitivo en dos partes:

```
inicio
  I ← 1
  repetir
    Buscar elemento mas pequeño X[I], X[I + 1], ..., X[N] //Supongamos que es X[K]
    Intercambiar X[K] y X[I]
  hasta_que I = N
fin
```

Nivel 4a:

Las instrucciones `'seleccion'` e `'intercambiar'` se refinan independientemente. El algoritmo con la estructura repetir es:

```
inicio
  I ← 1
  repetir
    AUXI ← X[I]     //AUXI representa el valor mas pequeño
    K ← I           //K representa la posicion
    J ← I
    repetir
      J ← J + 1
      si X[J] < AUXI entonces
        AUXI ← X[J]     //actualizar AUXI
        K ← J           //K, posicion
      fin_si
    hasta_que J = N     //AUXI = X[K] es ahora el mas pequeño
    X[K] ← X[I]
    X[I] ← AUXI
    I ← I + 1
  hasta_que I = N
fin
```

Nivel 4b:

El algoritmo con la estructura **`mientras`**.

```
inicio
  I ← 1
  mientras I < N hacer
    AUXI ← X[I]
    K ← I
    J ← I
    mientras J < N hacer
      J ← J + 1
      si X[J] < AUXI entonces
        AUXI ← X[J]
        K ← J
      fin_si
    fin_mientras
    X[K] ← X[I]
    X[I] ← AUXI
    I ← I + 1
  fin_mientras
fin
```

Nivel 4c:

El algoritmo de ordenación con estructura **desde**.

```
inicio
  desde I ← 1 hasta N - 1 hacer
    AUXI ← X[I]
    K ← I
    desde J ← I + 1 hasta N hacer
      si X[J] < AUXI entonces
        AUXI ← X[J]
        K ← J
      fin_si
    fin_desde
    X[K] ← X[I]
    X[I] ← AUXI
  fin_desde
fin
```

9.2.4. Método de Shell

Es una mejora del método de inserción directa que se utiliza cuando el número de elementos a ordenar es grande. El método se denomina «Shell» —en honor de su inventor Donald Shell— y también método de *inserción* con incrementos decrecientes.

En el método de clasificación por inserción, cada elemento se compara con los elementos contiguos de su izquierda, uno tras otro. Si el elemento a insertar es más pequeño —por ejemplo—, hay que ejecutar muchas comparaciones antes de colocarlo en su lugar definitivamente.

Shell modificó los saltos contiguos resultantes de las comparaciones por saltos de mayor tamaño y con eso se conseguía la clasificación más rápida. El método se basa en fijar el tamaño de los saltos constantes, pero de más de una posición.

Supongamos un vector de elementos

```
4   12   16   24   36   3
```

en el método de inserción directa, los saltos se hacen de una posición en una posición y se necesitarán cinco comparaciones. En el método de Shell, si los saltos son de dos posiciones, se realizan tres comparaciones.

```
4   12   16   24   36   3
```

El método se basa en tomar como salto `N/2` (siendo `N` el número de elementos) y luego se va reduciendo a la mitad en cada repetición hasta que el salto o distancia vale 1.

Considerando la variable *salto*, se tendría para el caso de un determinado vector `X` los siguientes recorridos:

```
Vector X     [X[1], X[2], X[3], ..., X[N]]
Vector X1    [X[1], X[1] + salto, X[2] + salto, ...]
Vector XN    [salto1, salto2, salto3, ...]
```

Ejemplo 9.3

Deducir las secuencias parciales de clasificación por el método de Shell para ordenar en ascendente la lista o vector

```
6,   1,   5,   2,   3,   4,   0
```

Recorrido	*Salto*	*Lista reordenada*	*Intercambio*
1	3	2,1,4,0,3,5,6	(6,2),(5,4),(6,0)
2	3	0,1,4,2,3,5,6	(2,0)
3	3	0,1,4,2,3,5,6	Ninguno
4	1	0,1,2,3,4,5,6	(4,2), (4,3)
5	1	0,1,2,3,4,5,6	Ninguno

Sea un vector `X`:

```
X[1], X[2], X[3], ..., X[N]
```

y consideremos el primer salto a dar que tendrá un valor de

$$\frac{N}{2}$$

por lo que, para redondear, se tomará la parte entera

```
N DIV 2
```

y se iguala a `salto`

```
salto = N div 2
```

Se elige como pivote el número 50. Los valores 30, 20, 10, 15 y 25 son más pequeños que 50 y constituirán la primera lista, y 80, 90, 70, 95, 85 y 75 se sitúan en la segunda lista. Se recorre la lista desde la izquierda para encontrar el primer número mayor que 50 y desde la derecha el primero menor que 25.

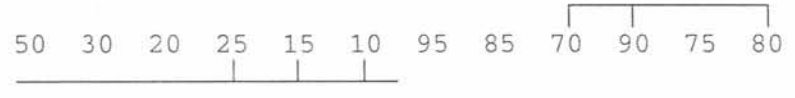
se localizan los dos números 80 y 25 y se intercambian

A continuación se reanuda la búsqueda desde la derecha para un número menor que 50, y desde la izquierda para un número mayor de 50.

Estos recorridos localizan los números 25 y 90, que se intercambian

Las búsquedas siguientes localizan 10 y 70.

El intercambio proporciona



Cuando se reanuda la búsqueda desde la derecha para un número menor que 50, localizamos el valor 10 que se encontró en la búsqueda de izquierda a derecha. Se señala el final de las dos búsquedas y se intercambian 50 y 10.

Lista de números < 50 *Lista de números > 50*

Algoritmos

El algoritmo de ordenación rápida se basa esencialmente en un algoritmo de división o partición de una lista. El método consiste en explorar desde cada extremo e intercambiar los valores encontrados. Un primer intento de algoritmo de partición es:

```
algoritmo Particion

inicio
  establecer x al valor de un elemento arbitrario de la lista
  mientras division no este terminada hacer
    recorrer de izquierda a derecha para un valor >= x
    recorrer de derecha a izquierda para un valor =< x
    si los valores localizados no estan ordenados entonces
      intercambiar los valores
    fin_si
  fin_mientras
fin
```

La lista que se desea partir es `A[1], A[2], ..., A[n]`. Los índices que representan los extremos izquierdo y derecho de la lista son `L` y `R`. En el refinamiento del algoritmo se elige un valor arbitrario `x`, suponiendo que el valor central de la lista es tan bueno como cualquier elemento arbitrario. Los índices `i, j` exploran desde los extremos. Un refinamiento del algoritmo anterior, que incluye mayor número de detalles, es el siguiente:

```
algoritmo Particion
  i ← L
  j ← R
  x ← A ((L + R) div 2)
  mientras i =< j hacer
    mientras A[i] < x hacer
      i ← i + 1
    fin_mientras
    mientras A[j] > x hacer
      j ← j - 1
    fin_mientras
      si i =< j entonces
        llamar_a intercambiar (A[i], A[j])
        i ← i + 1
        j ← j - 1
      fin_si
  fin_mientras
fin
```

En los bucles externos y la sentencia **si**, la condición utilizada es $i =< j$. Puede parecer que $i < j$ funcionan de igual modo en ambos lugares. De hecho, se puede realizar la partición con cualquiera de las condiciones. Sin embargo, si se utiliza la condición $i < j$, podemos terminar la partición con dos casos distintos, los cuales pueden diferenciarse antes de que podamos realizar divisiones futuras. Por ejemplo, la lista

```
1   7   7   9   9
```

y la condición $i < j$ terminará con $i = 3$, $j = 2$ y las dos particiones son `A[L]..A[I]` y `A[i]..A[R]`. Sin embargo, para la lista

```
1   7   7   9   9
```

y la condición $i < j$ terminaremos con $i = 3$, $j = 3$ y las dos particiones se solapan.

El uso de la condición $i <= j$ produce también resultados distintos para estos ejemplos. La lista

```
1   7   7   9   9
```

y la condición $i <= j$ se termina con $i = 3, j = 2$ como antes. Para la lista 1, 1, 7, 9, 9 y la condición $i =< j$ se termina con $i = 4, j = 2$. En ambos casos las particiones que requieren ordenación posterior son `A[L]..A[j]` y `A[i]..A[R]`.

En los bucles **mientras** internos la igualdad se omite de las condiciones. La razón es que el valor de partición actúe como *centinela* para detectar las exploraciones.

En nuestro ejemplo se ha tomado como valor de partición o pivote el elemento cuya posición inicial es el elemento central. Éste no es generalmente el caso. El ejemplo de la clasificación de la lista ya cit...

```
50   30   20   80   90   70   95   85   10   15   75   25
```

utilizaba como pivote el primer elemento.

El algoritmo de ordenación rápida en el caso de que el elemento pivote sea el primer elemento se muestra a continuación:

```
algoritmo Particion2
//lista a evaluar de 10 elementos
//array LISTA contiene los N elementos
//IZQUIERDO, indice de busqueda (recorrido) desde la izquierda
//DERECHO, indice de busqueda desde la derecha

inicio
  //inicializar indice para recorridos desde la izquierda y derecha
  IZQUIERDO ← ALTO  //ALTO parametro que indica principio de la sublista
  DERECHO ← BAJO    //BAJO parametro que indica final de la sublista
  A ← X [1]

  //realizar los recorridos
  mientras IZQUIERDO <= DERECHO hacer
  //busqueda o recorrido desde la derecha
    mientras X[IZQUIERDO]< A Y IZQUIERDO < BAJO
      IZQUIERDO ← IZQUIERDO + 1
    fin_mientras
    mientras X[DERECHO] > A
      DERECHO ← DERECHO - 1
    fin_mientras
    //intercambiar elemento
    si IZQUIERDO <= DERECHO entonces
      AUXI ← X [IZQUIERDO]
      X[IZQUIERDO] ← X[DERECHO]
      X[DERECHO] ← AUXI
      IZQUIERDO ← IZQUIERDO + 1
      DERECHO ← DERECHO - 1
    fin_si
  fin_mientras

  //fin busqueda; situar elemento seleccionado en su posicion
  //FINAL, posicion final del elemento
  si IZQUIERDO < BAJO + 1 entonces
      AUXI= X [DERECHO]
      X [DERECHO] = X [1]
      X [1] = AUXI
  si_no
        AUXI= X [BAJO]
        X [BAJO] = X[1]
        X[1] = AUXI
  fin_si
fin
```

9.3. BÚSQUEDA

La recuperación de información, como ya se ha comentado, es una de las aplicaciones más importantes de las computadoras.

La *búsqueda* (*searching*) de información está relacionada con las tablas para consultas (*lookup*). Estas tablas contienen una cantidad de información que se almacena en forma de listas de parejas de datos. Por ejemplo, un diccionario con una lista de palabras y definiciones; un catálogo con una lista de libros de informática; una lista de estudiantes y sus notas; un índice con títulos y contenido de los artículos publicados en una determinada revista, etc. En todos estos casos es necesario con frecuencia buscar un elemento en una lista.

Una vez que se encuentra el elemento, la identificación de su información correspondiente es un problema menor. Por consiguiente, nos centraremos en el proceso de búsqueda. Supongamos que se desea buscar el vector `X[1]..X[n]`, que tiene componentes numéricos, para ver si contiene o no un número dado `T`.

Si en vez de tratar sobre vectores se desea buscar información en un archivo, debe realizarse la búsqueda a partir de un determinado campo de información denominado *campo clave*. Así, en el caso de los archivos de empleados de una empresa, el campo clave puede ser el número de DNI o los apellidos.

La búsqueda por claves para localizar registros es, con frecuencia, una de las acciones que mayor consumo de tiempo conlleva y, por consiguiente, el modo en que los registros están dispuestos y la elección del modo utilizado para la búsqueda pueden redundar en una diferencia sustancial en el rendimiento del programa.

El problema de búsqueda cae naturalmente dentro de los dos casos típicos ya tratados. Si existen muchos registros, puede ser necesario almacenarlos en archivos de disco o cinta, externo a la memoria de la computadora. En este caso se llama *búsqueda externa*. En el otro caso, los registros que se buscan se almacenan por completo dentro de la memoria de la computadora. Este caso se denomina *búsqueda interna*.

En la práctica, la búsqueda se refiere a la operación de encontrar la posición de un elemento entre un conjunto de elementos dados: lista, tabla o fichero.

Ejemplos típicos de búsqueda son localizar nombre y apellidos de un alumno, localizar números de teléfono de una agenda, etc.

Existen diferentes algoritmos de búsqueda. El algoritmo elegido depende de la forma en que se encuentren organizados los datos.

La operación de búsqueda de un elemento `N` en un conjunto de elementos consiste en:

- determinar si `N` pertenece al conjunto y, en ese caso, indicar su posición en él,
- determinar si `N` no pertenece al conjunto.

Los métodos más usuales de búsqueda son:

- *búsqueda secuencial o lineal,*
- *búsqueda binaria,*
- *búsqueda por transformación de claves (hash).*

9.3.1. Búsqueda secuencial

Supongamos una lista de elementos almacenados en un vector (array unidimensional). El método más sencillo de buscar un elemento en un vector es explorar secuencialmente el vector o, dicho en

otras palabras, *recorrer el vector* desde el primer elemento al último. Si se encuentra el elemento buscado, visualizar un mensaje similar a 'Fin de busqueda'; en caso contrario, visualizar un mensaje similar a 'Elemento no existe en la lista'.

En otras palabras, la búsqueda secuencial compara cada elemento del vector con el valor deseado, hasta que éste se encuentra o termina de leer el vector completo.

La búsqueda secuencial no requiere ningún requisito por parte del vector y, por consiguiente, no necesita estar ordenado. El recorrido del vector se realizará normalmente con estructuras repetitivas.

Ejemplo 9.5

Se tiene un vector A *que contiene* n *elementos numéricos* (n) >= 1(A[1], A[2], A[3], ..., A[n]) *y se desea buscar un elemento dado* t. *Si el elemento* t *se encuentra, visualizar un mensaje* 'Elemento encontrado' *y otro que diga* 'posicion = '.

Si existen n elementos, se requerirán como media n/2 comparaciones para encontrar un determinado elemento. En el caso más desfavorable se necesitarán n comparaciones.

Método 1:

```
algoritmo Busqueda_secuencial_1

inicio
  leer(t)
  //recorrido del vector
  desde i ← 1 hasta n hacer
    si A[i] = t entonces
      escribir('Elemento encontrado')
      escribir('en posicion', i)
    fin_si
  fin_desde
fin
```

Método 2:

```
algoritmo Busqueda_secuencial_2

inicio
  leer(t)
  i ← 1
  mientras (A[i] <> t) y (i =< n) hacer
    i ← i + 1
    //este bucle se detiene bien con A[i] = t o bien con i > n
  fin_mientras

  si A[i] = t entonces      //condicion de parada
    escribir('El elemento se ha encontrado en la posicion', i)
  si_no                     //recorrido del vector terminado
    escribir('El numero no se encuentra en el vector')
  fin_si
fin
```

Este método no es completamente satisfactorio, ya que si t no está en el vector A, i toma el valor n + 1 y la comparación

```
A[i] <> t
```

producirá una referencia al elemento A[n + 1], que presumiblemente no existe. Este problema se resuelve sustituyendo i =< n por i < n en la instrucción **mientras**, es decir, modificando la instrucción anterior **mientras** por

```
mientras (A[i] <> t) y (i < n) hacer
```

Método 3:

```
algoritmo Busqueda_secuencial_3

inicio
  leer(t)
  i ← 1
  mientras (A[i] <> t) y (i < n) hacer
    i ← i + 1
    //este bucle se detiene cuando A[i] = t o i >= n
  fin_mientras

  si A[i] = t entonces
    escribir('El numero deseado esta presente y ocupa el lugar',i)
  si_no
    escribir(t, 'no existe en el vector')
  fin_si
fin
```

Método 4:

```
algoritmo Busqueda_secuencial_4

inicio
  llamar_a llenar(A)
  leer(t)
  i ← 1

  mientras i <= n hacer
    si t = A[i] entonces
      escribir('Se encontro el elemento buscado en la posicion',i)
    fin_si
      i ← i + 1
  fin_mientras

fin
```

Búsqueda secuencial con centinela

Una manera muy eficaz de realizar una búsqueda secuencial consiste en modificar los algoritmos anteriores utilizando un elemento centinela. Este elemento se agrega al vector al final del mismo. El valor del elemento centinela es el del argumento. El propósito de este elemento centinela, A[n + 1], es significar que la búsqueda siempre tendrá éxito. El elemento A[n + 1] sirve como centinela y se le asigna el valor de t antes de iniciar la búsqueda. En cada paso se evita la comparación de i con N y, por consiguiente, este algoritmo será preferible a los métodos anteriores, concretamente el método 4. Si el índice alcanzase el valor n + 1, supondría que el argumento no pertenece al vector original y, en consecuencia, la búsqueda no tiene éxito.

Método 5:

```
algoritmo Busqueda_secuencial_5

inicio
  leer(t)
  i ← 1
  A[n + 1] ← t
  mientras A[i] <> t hacer
    i ← i + 1
  fin_mientras

  si i = n + 1 entonces
    escribir('No se ha encontrado elemento')
  si_no
    escribir('Se ha encontrado el elemento')
  fin_si
fin
```

Una variante del método 5 es utilizar una variable lógica (interruptor o *switch*), que represente la existencia o no del elemento buscado.

Localizar si el elemento `t` existe en una lista `A[i]`, donde `i` varía desde `1` a `n`.

En este ejemplo se trata de utilizar una variable lógica `ENCONTRADO` para indicar si existe o no el elemento de la lista.

Método 6:

```
algoritmo Busqueda_secuencial_6

inicio
  leer(t)
  i ← 1
  ENCONTRADO ← falso
  mientras (no ENCONTRADO) y (i =< n) hacer
    si A[i] = t entonces
      ENCONTRADO ← verdadero
    si_no
      i ← i + 1
    fin_si
  fin_mientras

  si ENCONTRADO entonces
    escribir('El numero ocupa el lugar', i)
  si_no
    escribir('El numero no esta en el vector')
  fin_si
fin
```

NOTA: De todas las versiones anteriores, tal vez la más adecuada sea la incluida en el método 6. Entre otras razones, debido a que el bucle **`mientras`** engloba las acciones que permiten explorar el vector bien hasta que `t` se encuentre o bien cuando se alcance el final del vector.

Método 7:

```
algoritmo Busqueda_secuencial_7

inicio
  leer(t)
  i ← 1
```

```
  ENCONTRADO ← falso
  mientras i =< n hacer
    si A[i] = t entonces
      ENCONTRADO ← verdadero
      escribir ('El numero ocupa el lugar'; i)
    fin_si
    i ← i + 1
  fin_mientras
  si no (encontrado) entonces
    escribir('El numero no esta en el vector')
  fin_si
fin
```

Método 8:

```
algoritmo Busqueda_secuencial_8

inicio
  ENCONTRADO ← falso
  i ← 0
  leer(t)
  repetir
    i ← i + 1
    si A[i] = t entonces
      ENCONTRADO ← verdadero
    fin_si
  hasta_que ENCONTRADO o (i = n)
fin
```

Método 9:

```
algoritmo Busqueda_secuencial_9

inicio
  ENCONTRADO ← falso
  leer(t)
  desde i ← 1 hasta n hacer
    si A[i] = t entonces
      ENCONTRADO ← verdadero
    fin_si
  fin_desde

  si ENCONTRADO entonces
    escribir('Elemento encontrado')
  si_no
    escribir('Elemento no encontrado')
  fin_si
fin
```

Consideraciones sobre la búsqueda lineal

El método de búsqueda lineal tiene el inconveniente del consumo excesivo de tiempo en la localización del elemento buscado. Cuando el elemento buscado no se encuentra en el vector, se verifican o comprueban sus n elementos. En los casos en que el elemento se encuentra en la lista, el número podrá ser el primero, el último o alguno comprendido entre ambos.

Se puede suponer que el número medio de comprobaciones o comparaciones a realizar es de $(n + 1)/2$ (aproximadamente igual a la mitad de los elementos del vector).

La búsqueda secuencial o lineal no es el método más eficiente para vectores con un gran número de elementos. En estos casos, el método más idóneo es el de *búsqueda binaria*, que presupone una ordenación previa en los elementos del vector. Este caso suele ser muy utilizado en numerosas facetas de la vida diaria. Un ejemplo de ello es la búsqueda del número de un abonado en una guía telefónica; normalmente no se busca el nombre en orden secuencial, sino que se busca en la primera o segunda mitad de la guía; una vez en esa mitad, se vuelve a tantear en una de sus dos submitades, y así sucesivamente se repite el proceso hasta que se localiza la página correcta.

9.3.2. Búsqueda binaria

En una búsqueda secuencial se comienza con el primer elemento del vector y se busca en él hasta que se encuentra el elemento deseado o se alcanza el final del vector. Aunque éste puede ser un método adecuado para pocos datos, se necesita una técnica más eficaz para conjuntos grandes de datos.

Si el número de elementos del vector fuese grande, el algoritmo de búsqueda lineal se ralentizaría en tiempo de un modo considerable. Por ejemplo, si tuviéramos que consultar un nombre en la guía telefónica de una gran ciudad como Madrid, con una cifra aproximada de un millón de abonados, el tiempo de búsqueda —según el nombre— se podría eternizar. Naturalmente, las personas que viven en esa gran ciudad nunca utilizarán un método de búsqueda secuencial, sino un método que se basa en la división sucesiva del espacio ocupado por el vector en sucesivas mitades, hasta encontrar el elemento buscado.

Si los datos que se buscan están clasificados en un determinado orden, el método citado anteriormente se denomina *búsqueda binaria*.

La búsqueda binaria utiliza un método de «divide y vencerás» para localizar el valor deseado. Con este método se examina primero el elemento central de la lista; si éste es el elemento buscado, entonces la búsqueda ha terminado. En caso contrario, se determina si el elemento buscado está en el primera o la segunda mitad de la lista y, a continuación, se repite este proceso, utilizando el elemento central de esa sublista.

Supongamos la lista

1231
1473
1545
1834
1892
1898 *elemento central*
1983
2005
2446
2685
3200

Si está buscando el elemento 1983, se examina el número central, 1898, en la sexta posición. Ya que 1983 es mayor que 1898, se desprecia la primera sublista y nos centramos en la segunda:

1983
2005
2446 *elemento central*
2685
3200

El número central de esta sublista es 2446 y el elemento buscado es 1983, menor que 2446; eliminamos la segunda sublista y nos queda

1983
2005

Como no hay término central, elegimos el término inmediatamente anterior al término central, 1983, que es el buscado.

Se han necesitado tres comparaciones, mientras que la búsqueda secuencial hubiese necesitado siete.

La búsqueda binaria se utiliza en vectores ordenados y se basa en la constante división del espacio de búsqueda (recorrido del vector). Como se ha comentado, se comienza comparando el elemento que se busca, no con el primer elemento, sino con el elemento central. Si el elemento buscado —*t*— es menor que el elemento central, entonces *t* deberá estar en la mitad izquierda o inferior del vector; si es mayor que el valor central, deberá estar en la mitad derecha o superior, y si es igual al valor central, se habrá encontrado el elemento buscado.

El funcionamiento de la búsqueda binaria en un vector de enteros se ilustra en la Figura 9.3 para dos búsquedas: *con éxito* (localizado el elemento) y *sin éxito* (no encontrado el elemento).

El proceso de búsqueda debe terminar normalmente conociendo si la búsqueda *ha tenido éxito* (se ha encontrado el elemento) o bien *no ha tenido éxito* (no se ha encontrado el elemento) y normalmente se deberá devolver la posición del elemento buscado dentro del vector.

Ejemplo 9.6

Encontrar el algoritmo de búsqueda binaria para hallar un elemento K *en una lista de elementos* X_1, X_2, ..., X_n *previamente clasificados en orden ascendente.*

El array o vector X se supone ordenado en orden creciente si los datos son numéricos, o alfabéticamente si son caracteres. Las variables BAJO, CENTRAL, ALTO indican los límites inferior, central y superior del intervalo de búsqueda.

```
algoritmo Busqueda_binaria

inicio
  leer(K)
  //inicializar variables
  BAJO ← 1
  ALTO ← N
  CENTRAL ← ent ((BAJO + ALTO) / 2)
  mientras (BAJO =< ALTO) y (X[CENTRAL] <> K) hacer
    si K < X[CENTRAL] entonces
      ALTO ← CENTRAL - 1
    si_no
      BAJO ← CENTRAL + 1
    fin_si
    CENTRAL ← ent ((BAJO + ALTO) / 2)
  fin_mientras
  si K = X[CENTRAL] entonces
    escribir('Valor encontrado en', CENTRAL)
  si_no
    escribir('Valor no encontrado')
  fin_si
fin
```

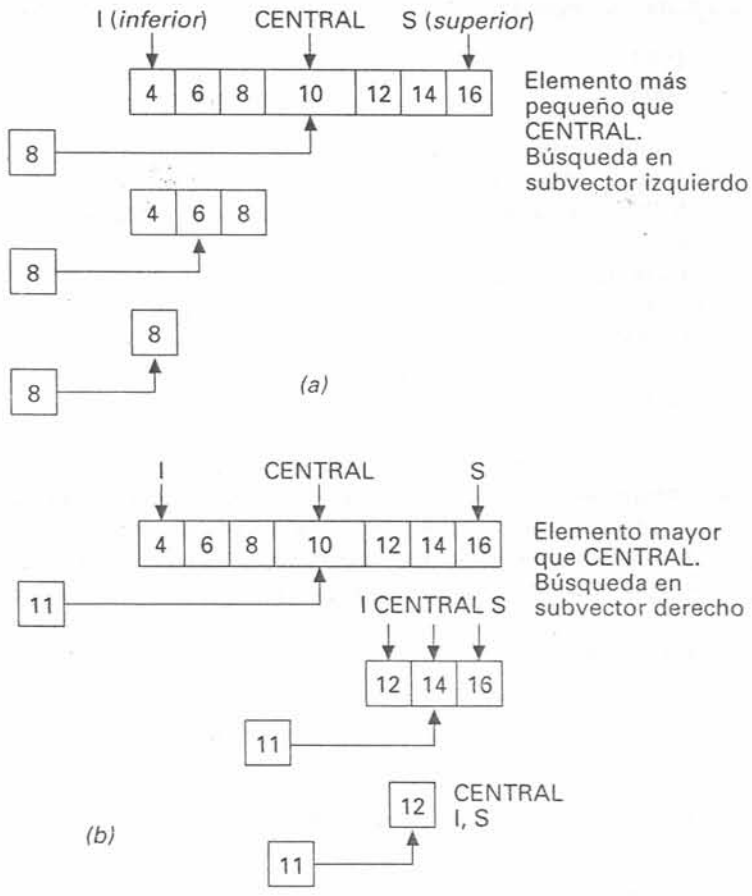


Figura 9.3. Ejemplo de búsqueda binaria: (a) con éxito, (b) sin éxito.

Ejemplo 9.7

Se dispone de un vector tipo carácter NOMBRE *clasificado en orden ascendente y de* N *elementos. Realizar el algoritmo que efectúe la búsqueda de un nombre introducido por el usuario.*

La variable N indica cuántos elementos existen en el array.
ENCONTRADO es una variable lógica que detecta si se ha localizado el nombre buscado.

```
algoritmo busqueda_nombre

//inicializar todas las variables necesarias
NOMBRE        //array de caracteres
N             //numero de nombres del array NOMBRE
ALTO          //puntero al extremo superior del intervalo
BAJO          //puntero al extremo inferior del intervalo
CENTRAL       //puntero al punto central del intervalo
X             //nombre introducido por el usuario
ENCONTRADO    // bandera o centinela
```

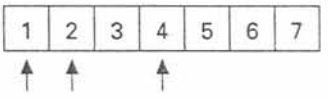
```
inicio
  leer(X)
  BAJO ← 1
  ALTO ← N
  ENCONTRADO ← falso

  mientras (no ENCONTRADO) y (BAJO =< ALTO) hacer
    CENTRAL ← ent ((BAJO + ALTO) / 2)
    //verificar nombre central en este intervalo
    si NOMBRE[CENTRAL] = X entonces
      ENCONTRADO ← verdad
    si_no
      si NOMBRE[CENTRAL] > X entonces
        ALTO ← CENTRAL - 1
      si_no
        BAJO ← CENTRAL + 1
      fin_si
    fin_si
  fin_mientras
  si ENCONTRADO entonces
    escribir('Nombre encontrado')
  si_no
    escribir('Nombre no encontrado')
  fin_si
fin
```

Análisis de la búsqueda binaria

La búsqueda binaria es un método eficiente siempre que el vector esté ordenado. En la práctica esto suele suceder, pero no siempre. Por esta razón la búsqueda binaria exige una ordenación previa del vector.

Para poder medir la velocidad de cálculo del algoritmo de búsqueda binaria, se deberán obtener el número de comparaciones que realiza el algoritmo.

Consideremos un vector de siete elementos ($n = 7$). El número 8 ($N + 1 = 8$) se debe dividir en tres mitades antes de que se alcance 1; es decir, se necesitan tres comparaciones.



El medio matemático de expresar estos números es:

$3 = \log_2 (8)$

en general, para n elementos:

$K = log_2 (n + 1)$

Recuerde que $\log_2 (8)$ es el exponente al que debe elevarse 2 para obtener 8. Es decir, 3, ya que $2^3 = 8$.

Si $n + 1$ es una potencia de 2, entonces $\log_2 (n + 1)$ será un entero. Si $n + 1$ no es una potencia de 2, el valor del logaritmo se redondea hasta el siguiente entero. Por ejemplo, si n es 12, entonces K será 4, ya que $\log_2 (13)$ (que está entre 3 y 4) se redondeará hasta 4 (2^4 es 16).

En general, en el mejor de los casos se realizará una comparación y, en el peor de los casos, se realizarán $\log_2 (n + 1)$ comparaciones.

Como término medio, el número de comparaciones es:

$$\frac{1+\log_2(n+1)}{2}$$

Esta fórmula se puede reducir para el caso de que n sea grande a

$$\frac{\log_2(n+1)}{2}$$

Para poder efectuar una comparación entre los métodos de búsqueda lineal y búsqueda binaria, realicemos los cálculos correspondientes para diferentes valores de n.

`n = 100` En la *búsqueda secuencial* se necesitarán:

$$\frac{100+1}{2}$$ **50 comparaciones**

En la *búsqueda binaria* $\log_2 (100) = 6...$:

$log_2 (100) = x$ donde $2^x = 100$ y $x = 6...$

$2^7 = 128 > 100$ **7 comparaciones**

`n = 1.000.000` En la *búsqueda secuencial:*

$$\frac{1.000.000+1}{2}$$ **50.000 comparaciones**

En la *búsqueda binaria:* $log_2 (1.000.000) = x$

$2^x = 1.000.000$ donde $x = 20$ y $2^{20} > 1.000.000$

20 comparaciones

Como se observa en los ejemplos anteriores, el tiempo de búsqueda es muy pequeño, aproximadamente siete comparaciones para 100 elementos y veinte para 1.000.000 de elementos. *(Compruebe el lector que para 1.000 elementos se requiere un máximo de diez comparaciones.)*

La búsqueda binaria tiene, sin embargo, inconvenientes a resaltar:

El *vector debe estar ordenado* y *el almacenamiento de un vector ordenado* suele plantear problemas en las inserciones y eliminaciones de elementos. (En estos casos será necesario utilizar listas enlazadas o árboles binarios. Véanse Capítulos 11 y 12.)

La Tabla 9.2 compara la eficiencia de la búsqueda lineal y búsqueda binaria para diferentes valores de n. Como se observará en dicha tabla, la ventaja del método de búsqueda binaria aumenta a medida que n aumenta.

Tabla 9.2. Eficiencia de las búsquedas lineal y binaria

Búsqueda secuencial		Búsqueda binaria
Número de comparaciones		Número máximo de comparaciones
n	Elemento no localizado	Elemento no localizado
1	1	3
100	100	7
1000	1000	10
1000000	100000	20

9.3.3. Búsqueda mediante transformación de claves (*hashing*)

La búsqueda binaria proporciona un medio para reducir el tiempo requerido para buscar en una lista. Este método, sin embargo, exige que los datos estén ordenados. Existen otros métodos que pueden aumentar la velocidad de búsqueda en los que los datos no necesitan estar ordenados; este otro método se conoce como transformación de claves (clave-dirección) o *hashing*.

El método de transformación de claves consiste en convertir la clave dada (numérica o alfanumérica) en una dirección (índice) dentro del array. La correspondencia entre las claves y la dirección en el medio de almacenamiento o en el array se establece por una función de conversión (función *hash*).

Así, por ejemplo, en el caso de una lista de empleados (100) de una pequeña empresa. Si cada uno de los cien empleados tiene un número de identificación (clave) del 1 al 100, evidentemente puede existir una correspondencia directa entre la clave y la dirección definida en un vector o array de 100 elementos.

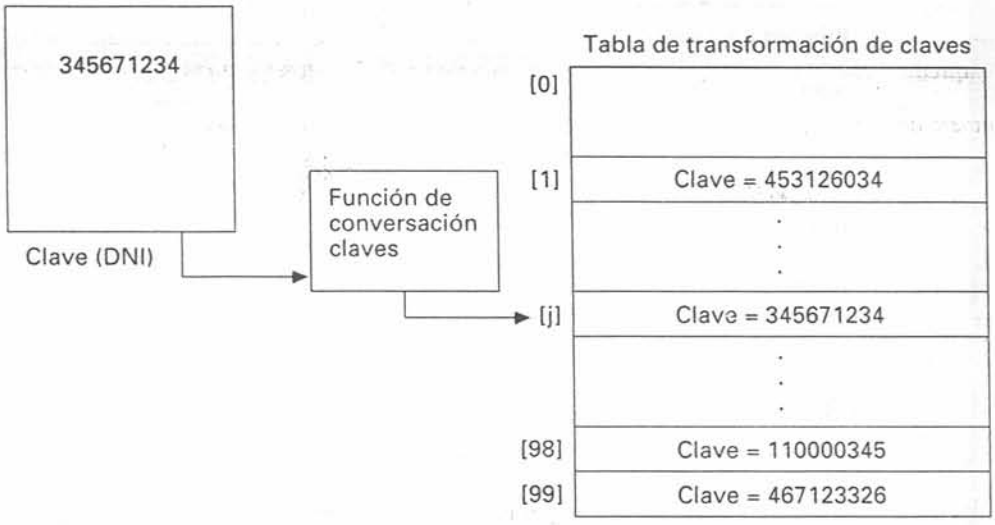
Supongamos ahora que el campo clave de estos registros o elementos es el número del DNI o de la Seguridad Social, que contenga nueve dígitos. Si se desea mantener en un array todo el rango posible de valores, se necesitarán 10^{10} elementos en la tabla de almacenamiento, cantidad difícil de tener disponible en memoria central, aproximadamente 1.000.000.000 de registros o elementos. Si el vector o archivo sólo tiene 100, 200 o 1.000 empleados, cómo hacer para introducirlos en memoria por el campo clave DNI. Para hacer uso de la clave DNI como un índice en la tabla de búsqueda, se necesita un medio para convertir el campo clave en una dirección o índice más pequeño. En la figura se presenta un diagrama de cómo realizar la operación de conversión de una clave grande en una tabla pequeña.

Los registros o elementos del campo clave no tienen por qué estar ordenados de acuerdo con los valores del campo clave, como estaban en la búsqueda binaria.

Por ejemplo, el registro del campo clave 345671234 estará almacenado en la tabla de transformación de claves (array) en una posición determinada; por ejemplo, 75.

La función de transformación de clave, $H(k)$ convierte la clave (k) en una dirección (d).

Imaginemos que las claves fueran nombres o frases de hasta dieciséis letras, que identifican a un conjunto de un millar de personas. Existirán 26^{16} combinaciones posibles de claves que se deben transformar en 10^3 direcciones o índices posibles. La función H es, por consiguiente, evidentemente una función de paso o conversión de múltiples claves a direcciones. Dada una clave k, el primer paso en la operación de búsqueda es calcular su índice asociado $d \leftarrow H(k)$ y el segundo paso —eviden-



temente necesario— es verificar *si* o *no* el elemento con la clave k es identificado verdaderamente por h en el array T; es decir, para verificar si la clave $T[H(K)] = K$, se deben considerar dos preguntas:

- ¿Qué clase de función H se utilizará?
- ¿Cómo resolver la situación de que H no produzca la posición del elemento asociado?

La respuesta a la segunda cuestión es que se debe utilizar algún método para producir una posición alternativa, es decir, el índice h', y si ésta no es aún la posición del elemento deseado, se produce un tercer índice h'', y así sucesivamente. El caso en el que una clave distinta de la deseada está en la posición identificada se denomina *colisión*; la tarea de generación de índices alternativos se denomina tratamiento de colisiones.

Un ejemplo de colisiones puede ser:

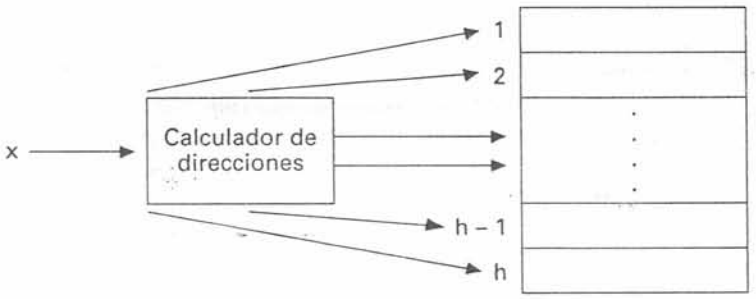
clave 345123124	
clave 416457234	función de conversión $H \rightarrow$ dirección 200
	función de conversión $H \rightarrow$ dirección 200

Dos claves distintas producen la misma dirección, es decir, *colisiones*. La elección de una buena función de conversión exige un tratamiento idóneo de colisiones, es decir, la reducción del número de colisiones.

9.3.3.1. Métodos de transformación de claves

Existen numerosos métodos de transformación de claves.

Todos ellos tienen en común la necesidad de convertir claves en direcciones. En esencia, la función de conversión equivale a una caja negra que podríamos llamar *calculador de direcciones*. Cuando se desea localizar un elemento de clave x, el indicador de direcciones indicará en qué posición del array estará situado el elemento.



Truncamiento

Ignora parte de la clave y se utiliza la parte restante directamente como índice (considerando campos no numéricos y sus códigos numéricos). Si las claves, por ejemplo, son enteros de ocho dígitos y la tabla de transformación tiene mil posiciones, entonces el primero, segundo y quinto dígitos desde la derecha pueden formar la función de conversión. Por ejemplo, 72588495 se convierte en 895. El truncamiento es un método muy rápido, pero falla para distribuir las claves de modo uniforme.

Plegamiento

La técnica del plegamiento consiste en la partición de la clave en diferentes partes y la combinación de las partes en un modo conveniente (a menudo utilizando suma o multiplicación) para obtener el índice.

La clave x se divide en varias partes, x_1, x_2, ..., x_n, donde cada parte, con la única posible excepción de la última parte, tiene el mismo número de dígitos que la dirección especificada.

A continuación, se suman todas las partes:

$$h(x) = x_1 + x_2 + \ldots + x_n$$

En esta operación se desprecian los dígitos más significativos que se obtengan de arrastre o acarreo.

Ejemplo 9.8

Un entero de ocho dígitos se puede dividir en grupos de tres, tres y dos dígitos, los grupos se suman juntos y se truncan si es necesario para que estén en el rango adecuado de índices.

Por consiguiente, si la clave es:

```
62538194
```

y el número de direcciones es `100`, la función de conversión será:

```
625 + 381 + 94 = 1100
```

que se truncará a `100` y que será la dirección deseada.

Ejemplo 9.9

Los números empleados —campo clave— de una empresa constan de cuatro dígitos y las direcciones reales son 100. Se desea calcular las direcciones correspondientes por el método de plegamiento de los empleados.

```
4205      8148      3355

h(4205) = 42 + 05 = 47
h(8148) = 81 + 48 = 129      y se convierte en 29 (129 – 100), es decir, se ignora el acarreo 1
h(3355) = 33 + 55 = 88
```

Si se desea afinar más, se podría hacer la inversa de las partes pares y luego sumarlas.

Aritmética modular

Convertir la clave a un entero, dividir por el tamaño del rango del índice y tomar el resto como resultado. La función de conversión utilizada es **mod** (módulo o resto de la división entera).

```
h(x) = x mod m
```

donde m es el tamaño del array con índices de 0 a $m - 1$. Los valores de la función —direcciones— (el resto) irán de 0 a $m - 1$, ligeramente menor que el tamaño del array. La mejor elección de los módulos son los números primos. Por ejemplo, en un array de 1.000 elementos se puede elegir 997. Otros ejemplos son:

```
18 mod 6    19 mod 6    20 mod 6
```

que proporcionan unos restos de 0, 1 y 2, respectivamente.

Si se desea que las direcciones vayan de 0 hasta m, la función de conversión debe ser:

```
h(x) = x mod (m+1)
```

Ejemplo 9.10

Un vector T tiene cien posiciones, 0..100. Supongamos que las claves de búsqueda de los elementos de la tabla son enteros positivos (por ejemplo, número del DNI).

Una función de conversión h debe tomar un número arbitrario entero positivo x y convertirlo en un entero en el rango 0..100, esto es, h es una función tal que para un entero positivo x:

$h(x) = n$, donde n es entero en el rango 0..100

El método del módulo, tomando `101`, será:

```
h(x) = x mod 101
```

Si se tiene el DNI número `234661234`, por ejemplo, se tendrá la posición `56`:

```
234661234 mod 101 = 56
```

Ejemplo 9.11

La clave de búsqueda es una cadena de caracteres —tal como un nombre—. Obtener las direcciones de conversión.

El método más simple es asignar a cada carácter de la cadena un valor entero (por ejemplo, `A = 1, B = 2, ...`) y sumar los valores de los caracteres en la cadena. Al resultado se le aplica entonces el módulo 101, por ejemplo.

Si el nombre fuese JONAS, esta clave se convertiría en el entero

```
10 + 15 + 14 + 1 + 19 = 63

63 mod 101 = 63
```

Mitad del cuadrado

Este método consiste en calcular el cuadrado de la clave x. La función de conversión se define como

$$h(x) = c$$

donde c se obtiene eliminando dígitos a ambos extremos de x^2. Se deben utilizar las mismas posiciones de x^2 para todas las claves.

Ejemplo 9.12

Una empresa tiene ochenta empleados y cada uno de ellos tiene un número de identificación de cuatro dígitos y el conjunto de direcciones de memoria varía en el rango de 0 a 100. Calcular las direcciones que se obtendrán al aplicar función de conversión por la mitad del cuadrado de los números empleados:

```
4205  7148  3350

x       4205            7148            3350
x²      17 682 025      51 093 904      11 222 500
```

Si elegimos, por ejemplo, el cuarto y quinto dígito significativo, quedaría:

$h(x)$ 82 93 22

9.3.3.2. Colisiones

La función de conversión $h(x)$ no siempre proporciona valores distintos puede suceder que para dos claves diferentes x_1 y x_2 se obtenga la misma dirección. Esta situación se denomina *colisión* y se deben encontrar métodos para su correcta resolución.

Los ejemplos vistos anteriormente de las claves DNI correspondientes al archivo de empleados, en el caso de cien posibles direcciones. Si se considera el método del módulo en el caso de las claves, y se considera el número primo 101.

```
123445678   123445880
```

proporcionarían las direcciones:

```
h (123445678) = 123445678 mod 101 = 44
h (123445880) = 123445880 mod 101 = 44
```

Es decir, se tienen dos elementos en la misma posición del vector o array, [44]. En terminología de claves, se dice que las claves `123445678` y `123445880` han *colisionado*.

El único medio para evitar el problema de las colisiones totalmente es tener una posición del array para cada posible número de DNI. Si, por ejemplo, los números de DNI son las claves y el DNI se representa con nueve dígitos, se necesitaría una posición del array para cada entero en el rango 000000000 a 999999999. Evidentemente, sería necesario una gran cantidad de almacenamiento. En general, el único método para evitar colisiones totalmente es que el array sea lo bastante grande para que cada posible valor de la clave de búsqueda pueda tener su propia posición. Ya que esto normalmente no es práctico ni posible, se necesitará un medio para tratar o resolver las colisiones cuando sucedan.

Resolución de colisiones

Consideremos el problema producido por una colisión. Supongamos que desea insertar un elemento con número nacional de identidad DNI 12345678, en un array `T`. Se aplica la función de conversión del módulo y se determina que el nuevo elemento se situará en la posición `T[44]`. Sin embargo, se observa que `T[44]` ya contiene un elemento con DNI 123445779.

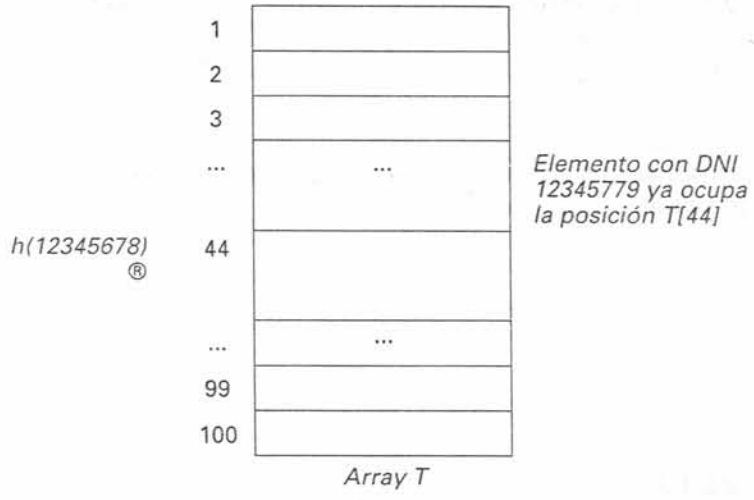


Figura 9.4. Colisión.

La pregunta que se plantea inmediatamente es, ¿qué hacer con el nuevo elemento?

Un método comúnmente utilizado para resolver una colisión es cambiar la estructura del array `T` de modo que pueda alojar más de un elemento en la misma posición. Se puede, por ejemplo, modificar `T` de modo que cada posición `T[i]` sea por sí misma un array capaz de contener `N` elementos. El problema, evidentemente, será saber la magnitud de `N`. Si `N` es muy pequeño, el problema de las colisiones aparecerá cuando aparezcan `N + 1` elementos.

Una solución mejor es permitir una lista enlazada o encadenada de elementos para formar a partir de cada posición del array. En este método de resolución de colisiones, conocido como *encadenamiento*, cada entrada `T[i]` es un puntero que apunta al elemento del principio de la lista de elementos (véase Capítulo 11), de modo que la función de transformación de clave lo convierte en la posición `i`.

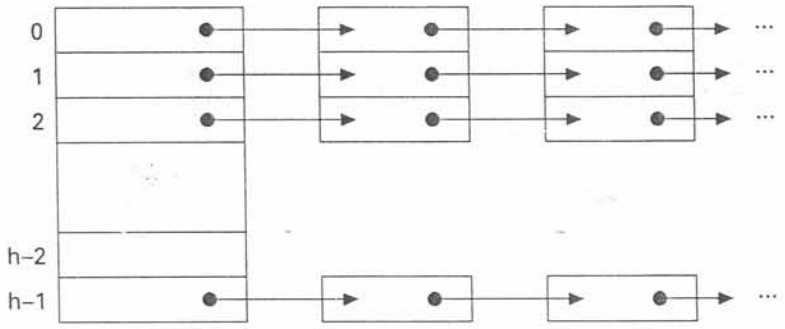


Figura 9.5. Encadenamiento.

9.4. INTERCALACIÓN

La *intercalación* es el proceso de mezclar (intercalar) dos vectores ordenados y producir un nuevo vector ordenado.

Consideremos los vectores (listas de elementos) ordenados:

```
A: 6  23  34
B: 5  22  26  27  39
```

El vector clasificado es:

```
C: 5     6     22     23     24     26     27     39
```

La acción requerida para solucionar el problema es muy fácil de visualizar. Un algoritmo sencillo puede ser:

1. poner todos los valores del vector A en el vector C,
2. poner todos los valores del vector B en el vector C,
3. clasificar el vector C.

Es decir, todos los valores se ponen en el vector C, con todos los valores de A seguidos por todos los valores de B. Seguidamente, se clasifica el vector C. Evidentemente, es una solución correcta. Sin embargo, se ignora por completo el hecho de que los vectores A y B están clasificados.

Supongamos que los vectores A y B tienen M y N elementos. El vector C tendrá, por tanto, M + N elementos.

El algoritmo comenzará seleccionando el más pequeño de los dos elementos A y B, situándolo en C. Para poder realizar las comparaciones sucesivas y la creación del nuevo vector C, necesitaremos dos índices para los vectores A y B. Por ejemplo, i y j. Entonces nos referiremos al elemento i en la lista A y al elemento j en la lista B. Los pasos generales del algoritmo son:

```
si elemento i de A es menor que elemento j de B entonces
  transferir elemento i de A a C
  avanzar i (incrementar en 1)
si_no
  transferir elemento j de B a C
  avanzar j
fin_si
```

Figura 9.6. Intercalación (`B[j] < A[j]`, de modo que `C[k]` se obtiene de `B[j]`).

Se necesita un índice `K` que represente la posición que se va rellenando en el vector `C`. El proceso gráfico se muestra en la Figura 9.6.

El primer refinamiento del algoritmo:

```
//estado inicial de los algoritmos
i ← 1
j ← 1
k ← 0
mientras (i <= M) y (j <= N) hacer
  //seleccionar siguiente elemento de A o B y añadir a C
  k ← k+1
  //incrementar K
  si A[i] < B[j] entonces
    C[k] ← A[i]
    i ← i + 1
  si_no
    C[k] ← B[j]
    j ← j + 1
  fin_si
fin_mientras
```

Si los vectores tienen elementos diferentes, el algoritmo anterior no requiere seguir haciendo comparaciones cuando el vector más pequeño se termine de situar en `C`. La operación siguiente deberá copiar en `C` los elementos que restan del vector más grande. Así, por ejemplo, supongamos:

```
A = 6    23    24                  i = 4
B = 5    22    26    27    39      j = 3
C = 5    6     22    23    24      k = 5
```

Todos los elementos del vector `A` se han relacionado y situados en el vector `C`. El vector `B` contiene los elementos no seleccionados y que deben ser copiados, en orden, al final del vector `C`. En general, será necesario decidir cuál de los vectores `A` o `B` tienen elementos no seleccionados y a continuación ejecutar la asignación necesaria.

El algoritmo de copia de los elementos restantes es:

```
si i <= M entonces
  desde r ← i hasta M hacer
    k ← k + 1
    C[k] ← A[r]
  fin_desde
si_no
  desde r ← j hasta N hacer
    k ← k + 1
    C[k] ← B[r]
  fin_desde
fin_si
```

El algoritmo total resultante de la intercalación de dos vectores A y B ordenados en uno C es:

```
algoritmo Intercalacion

inicio
  leer(A, B)  //A, B vectores de M y N elementos
  i ← 1
  j ← 1
  k ← 0

  mientras (i <= M) y (j <= N) hacer
    //seleccionar siguiente elemento de A o B y añadirlo a C
    k ← k + 1
    si A[i]< B[j] entonces
      C[k] ← A[i]
      i ← i + 1
    si_no
      C[k] ← B[j]
      j ← j + 1
    fin_si
  fin_mientras

  //copiar el vector restante
  si i <= M entonces
    desde r ← i hasta M hacer
      k ← k + 1
      C[k] ← A[r]
    fin_desde
  si_no
    desde r ← j hasta N hacer
      k ← k + 1
      C[k] ← B[r]
    fin_desde
  fin_si

  escribir(C)   //vector clasificado
fin
```

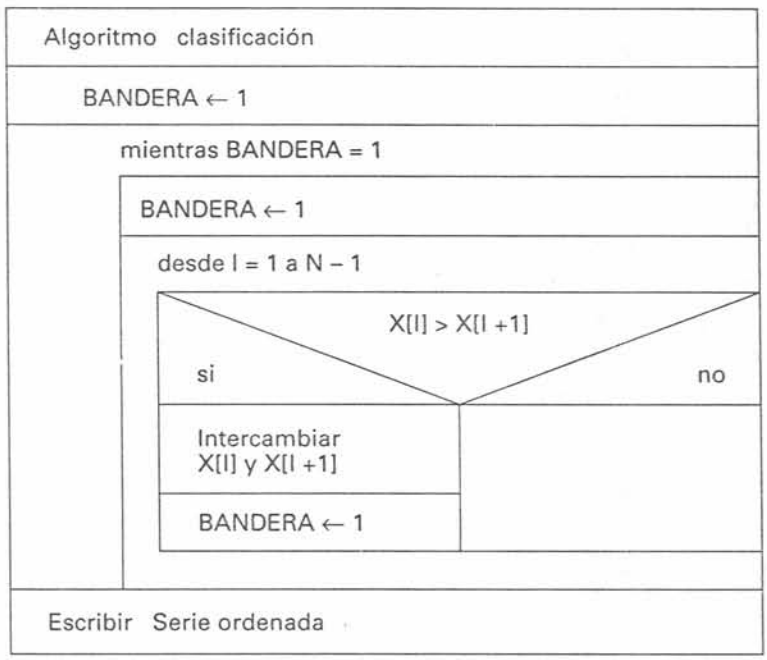
ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

9.1. Clasificar una serie de números X1, X2, ..., Xn en orden creciente por el método del intercambio o de la burbuja.

ANÁLISIS

Se utiliza un indicador (bandera) igual a 0 si la serie está bien ordenada, y a 1 en caso contrario. Como a priori la serie no está bien ordenada, se inicializa el valor de la bandera a 1 y después se repiten las siguientes acciones:

- se fija la bandera a 0,
- a partir del primero se comparan dos elementos consecutivos de la serie; si están bien ordenados, se pasa al elemento siguiente, si no se intercambian los valores de los dos elementos y se fija el valor de la bandera a 1; si después de haber pasado revista —leído— toda la serie, la bandera permanece igual a 0, entonces la clasificación está terminada.



9.2. Clasificar los números A y B.

Pseudocódigo

Método 1:

```
algoritmo Clasificar A,B
inicio
  leer(A, B)
  si A < B entonces
    permutar A, B
  fin_si
  escribir('Mas grande', A)
  escribir('Mas pequeña', B)
fin
```

Método 2:

```
algoritmo Clasificar
inicio
  leer(A)
  MAX ← A
  leer(B)
  MIN ← B
  si B > A entonces
    MAX ← B
    MIN ← A
  fin_si
  escribir('Maximo =', MAX)
  escribir('Minimo =', MIN)
fin
```

9.3. Se dispone de una lista de números enteros clasificados en orden creciente. Se desea conocer si un número dado introducido desde el terminal se encuentra en la lista. En caso afirmativo, averiguar su posición, y en caso negativo se desea conocer su posición en la lista e insertarlo en su posición.

ANÁLISIS

Como ya conoce el lector, existen dos métodos fundamentales de búsqueda: lineal y binaria. Resolvemos el problema con los dos métodos a fin de consolidar las ideas sobre ambos.

Búsqueda lineal

El método consiste en comparar el número dado en orden sucesivo con todos los elementos del conjunto de números, efectuando un recorrido completo del vector que representa la lista.

El proceso termina cuando se encuentra un número igual o superior al número dado.

El método de inserción o intercalación de un elemento en el vector será el descrito en el apartado 6.3.4.

La tabla de variables es la siguiente:

N	número de elementos de la lista: *entero*
J	posición del elemento en la lista: *entero*
K	contador del bucle de búsqueda: *entero*
X	número dado: *entero*
LISTA	conjunto de números enteros

Búsqueda dicotómica

La condición para realizar este método —más rápido y eficaz— es que la lista debe estar clasificada en orden creciente o decreciente.

Se obtiene el número de elementos de la lista y se calcula el número central de la lista. Si el número dado es igual al número central de la lista, la búsqueda ha terminado. En caso contrario, pueden suceder dos casos:

- el número está en la sublista inferior,
- el número está en la sublista superior.

Tras localizar la sublista donde se encuentra, se consideran variables MIN y MAX que contienen los elementos menor y mayor de cada sublista —que coincidirán con los extremos al estar ordenada la lista—, así como el término central (CENTRAL), de acuerdo al siguiente esquema:

Sublista inferior	L[1] L[2] ...L[CENTRAL]
Sublista superior	L[CENTRAL + 1]... L[N]

Los valores de las variables INF, SUP y CENTRAL serán:

Primera búsqueda

```
CENTRAL = (SUP-INF) / 2 + INF = N - 1 / 2 + 1 = N + 1 / 2
SUP = N
INF = 1
```

- *Si el número X está en la sublista inferior, entonces*

```
INF = 1
SUP = CENTRAL - 1
```

y se realiza una segunda búsqueda entre los elementos de orden 1 y CENTRAL.

- *Si el número X está en la sublista superior, entonces*

```
INF = CENTRAL + 1
SUP = N
```

y se realiza una segunda búsqueda entre los elementos de orden CENTRAL + 1 y N.

El proceso de variables es:

N	número de elementos de la lista: *entero*
I	contador del bucle de búsqueda: *entero*
SW	interruptor o bandera para indicar si el número dado está en la lista: *lógico*
LISTA	conjunto de números enteros: *entero*
X	número buscado: *entero*
INF	posición inicial de la lista o sublista: *entero*
SUP	posición superior de la lista o sublista: *entero*
POSICION	lugar del orden ocupado por el número buscado: *entero*

PSEUDOCÓDIGO

Búsqueda lineal

```
algoritmo Busqueda_1

var
  entero: Y, K, X, N
  array [1...50] de entero:lista
  //se supone dimension de la lista a 50 elementos y que se trata de una lista
  //ordenada
inicio
  leer(N)
  //lectura de la lista
  desde I ← 1 hasta N hacer
    leer(LISTA[I])
  fin_desde

  leer(X)
  I ← 0
  repetir
    I ← I + 1
  hasta_que (LISTA[I] >= X) o (I = 50)
  si LISTA[I] = X entonces
    escribir('se encuentra en',I)
```

```
  si_no
    escribir('El numero dado no esta en la lista')
    //insertar el elemento X en la lista
    si N < 50 entonces
      desde K ← N hasta I decremento 1 hacer
        LISTA[K + 1] ← LISTA[K]
      fin_desde
      LISTA[I] ← X
      N ← N + 1
      escribir('Insertado en',I)
      fin_si
  fin_si

  //escritura del vector LISTA
  desde I ← 1 hasta N hacer
    escribir(LISTA[I])
  fin_desde
fin
```

Búsqueda dicotómica

```
algoritmo Busqueda_b

var
  entero: Y, N, X, K, INF, SUP, CENTRAL
  logico: SW
  array [1...50] de entero:LISTA

inicio
  leer(N)
  desde I ← 1 hasta N hacer
    leer(LISTA[I])   //la lista ha de estar ordenada
  fin_desde
  leer(X)
  SW ← falso
  INF ← 1
  SUP ← N

  repetir
    CENTRAL ← (SUP-INF)DIV2 + INF
    si LISTA[CENTRAL] = X entonces
      escribir('Numero encontrado en la lista')
      P ← CENTRAL
      escribir(P)
      SW ← verdad
    si_no
      si X < LISTA[CENTRAL] entonces
        SUP ← CENTRAL
      si_no
        INF ← CENTRAL + 1
      fin_si
      si (INF = SUP) y (LISTA[INF] = X) entonces
        escribir('El numero esta en la lista')
        P ← INF
        escribir(P)
        SW ← verdad
      fin_si
    fin_si
  hasta_que (INF = SUP) o SW
```

```
  si no(SW) entonces
    escribir('Numero no existe en la lista')
    si X < lista(INF) entonces
      P ← INF
    si_no
      P ← INF+1
    fin_si
    escribir(P)
    desde K ← N hasta P decremento 1 hacer
    LISTA[K + 1] ← LISTA[K]
  fin_desde

    LISTA[P] ← X
    N ← N + 1
  fin_si
  //escritura de la lista L
  desde I ← 1 hasta N hacer
    escribir(LISTA[I])
  fin_desde
fin
```

9.4. Ordenar de mayor a menor un vector de N elementos (N <= 40), cada uno de los cuales es un registro con los campos día, mes y año de tipo entero.

Utilice una función ESMENOR(fecha1,fecha2), que nos devuelva si una fecha es menor que otra.

```
algoritmo Ordfechas
tipo registro: fechas
       entero: dia
       entero: mes
       entero: año
     fin_registro
     array[1..40] de fechas: arr
var arr    : f
    entero : n

inicio
  pedirfechas(f,n)
  ordenarfechas(f,n)
  presentarfechas(f,n)
fin

logico funcion esmenor(E fechas: fecha1,fecha2)
inicio
  si (fecha1.año<fecha2.año) o
     (fecha1.año=fecha2.año) y (fecha1.mes<fecha2.mes) o
     (fecha1.año=fecha2.año) y (fecha1.mes=fecha2.mes) y
     (fecha1.dia<fecha2.dia)
  entonces
     devolver(verdad)
  si_no
     devolver(falso)
  fin_si
fin_funcion
```

```
procedimiento pedirfechas(S  arr:f;  S entero:n)
var entero:i
    entero:dia

inicio
  i ← 1
  escribir('Deme la ',i,'ª fecha')
  escribir('Dia: ')
  leer(dia)
  mientras (dia<>0) y (i<=40) hacer
    f[i].dia ← dia
    escribir('Mes:')
    leer(f[i].mes)
    escribir('Año:')
    leer(f[i].año)
    n ← i
    i ← i+1
    si i<=40 entonces
      escribir ('Deme la ',i,'ª fecha')
      escribir ('Dia: ')
      leer(dia)
    fin_si
  fin_mientras
fin_procedimiento

procedimiento ordenarfechas(E/S arr:f;  E entero:n)
  var entero:salto
      logico:ordenada
      entero:j
      fechas:AUXI
inicio
  salto ← n
  mientras salto > 1 hacer
    salto ← salto div 2
    repetir
      ordenada ← verdad
      desde j ← 1 hasta n - salto hacer
        si esmenor(f[j], f[j + salto]) entonces
          AUXI ← f[j]
          f[j] ← f[j + salto]
          f[j + salto] ← AUXI
          ordenada ← falso
        fin_si
      fin_desde
    hasta ordenada
  fin_mientras
fin_procedimiento

procedimiento presentarfechas(E arr:f;  E entero:n)
var entero:i
inicio
  desde i←1 hasta n hacer
    escribir(f[i].dia,f[i].mes,f[i].año)
  fin_desde
fin_procedimiento
```

Considerar otras posibilidades, usando el mismo método de ordenación, para resolver el ejercicio.

9.5. Dada la lista de fechas ordenada en orden decreciente del ejercicio anterior, diseñar los procedimientos:

1.º Buscar, que nos informará sobre si una determinada fecha se encuentra o no en la lista:

— si no está, indicará la posición donde correspondería insertarla,
— si está, nos dirá la posición donde la hemos encontrado o, si estuviera repetida, a partir de qué posición y cuántas veces son las que aparece.

2.º Insertar, que nos permitirá insertar una fecha en una determinada posición. Se deberá utilizar en un algoritmo haciendo uso previo de buscar; así, cuando una fecha no se encuentre en la lista, la insertará en el lugar adecuado para que no se pierda la ordenación inicial.

```
algoritmo Buscar_insertar_fechas

tipo registro: fechas
       inicio
         entero: dia
         entero: mes
         entero: año
       fin
     array[1..40] de fechas: vector

var vector     : f
    entero     : n
    fechas     : fecha
    logico     : esta
    entero     : posic, cont

inicio
  pedirfechas(f,n)
  ordenarfechas(f,n)
  presentarfechas(f,n)
  escribir('Deme fecha a buscar (dd mm aa)')
  leer(fecha.dia,fecha.mes,fecha.año)
  buscar(f,n,fecha,esta,posic,cont)
  si esta entonces
    si cont > 1 entonces
       escribir('Aparece a partir de la posicion: ', posic,'
        , cont, ' veces')
    si_no
      escribir('Esta en la posicion: ', posic )
    fin_si
  si_no
    si n=40 entonces
      escribir('No esta. Array lleno')
    si_no
      insertar(f,n,fecha,posic)
      presentarfechas(f,n)
    fin_si
  fin_si
fin

logico funcion esmenor(E fechas: fecha1,fecha2)
inicio
    ..................
fin_funcion
```

```
logico funcion esigual(E fechas: fecha1,fecha2)
inicio
    si (fecha1.año=fecha2.año) y (fecha1.mes = fecha2.mes) y
       (fecha1.día=fecha2.dia)
    entonces
      devolver(verdad)
    si_no
      devolver(falso)
    fin_si
fin_funcion

procedimiento pedirfechas(S vector: f;  S entero: n)
var entero: i
  entero: dia
inicio
    ......................
fin_procedimiento

procedimiento ordenarfechas(E/S vector: f;  E entero: n)
var entero : salto
    logico : ordenada
    entero : j
    fechas : AUXI
inicio
    ........................
fin_procedimiento

procedimiento buscar(E vector: f;  E entero:n;  E fechas: fecha
                     S logico:esta;  S entero: posic, cont)
var entero : primero,ultimo,central,i
    logico : encontrado
inicio
    primero ← 1
    ultimo ← n
    esta ← falso
    mientras (primero <= ultimo) y (no esta) hacer
      central ← (primero + ultimo) div 2
      si esigual(f[central],fecha) entonces
        esta ← verdad
      si_no
        si esmenor(f[central],fecha) entonces
          ultimo ← central - 1
        si_no
          primero ← central + 1
        fin_si
      fin_si
    fin_mientras
  cont ← 0

  si esta entonces
       i ← central - 1
       encontrado ← verdad
       mientras (i >= 1) y (encontrado) hacer
         si esigual(f[i],f[central]) entonces
              i ← i - 1
         si_no
              encontrado ← falso
         fin_si
```

```
        fin_mientras
        i ← i + 1
        encontrado ← verdad
        posic ← i
        mientras (i <= 40) y encontrado hacer
          si esigual(f[i],f[central]) entonces
                cont ← cont + 1
                i ← i + 1
          si_no
                encontrado ← falso
          fin_si
        fin_mientras
      si_no
        posic ← primero
  fin_si
fin_procedimiento

procedimiento insertar(E/S vector: f;     E/S entero: n;
                       E fechas:fecha;    E entero:posic)
var entero:i

inicio
  desde i ← n hasta posic decremento 1 hacer
    f[i + 1] ← f[i]
  fin_desde
  f[posic] ← fecha
  n ← n + 1
fin_procedimiento

procedimiento presentarfechas(E vector:f;  E entero:n)
var entero:i
inicio
   .....
fin_procedimiento
```

9.6. Escribir el procedimiento de búsqueda binaria de forma recursiva.

```
algoritmo Busqueda_binaria
  tipo
       array[1..10] de entero: arr
  var
       arr    : a
       entero : num, posic, i
inicio
    desde i ← 1 hasta 10 hacer
      leer(a[i])
    fin_desde
    ordenar(a)
    escribir('Indique el numero a buscar en el array ')
    leer(num)
    busqueda(a,posic,1,10,num)
    si posic > 0 entonces
       escribir('Existe el elemento en la posicion ', posic)
    si_no
       escribir('No existe el elemento en el array.')
    fin_si
fin
```

```
procedimiento ordenar(E/S arr: a)
inicio
fin_procedimiento

procedimiento busqueda(E arr: a;   S entero: posic;
                       E entero: primero,ultimo,num)
//Este procedimiento devuelve 0 si no existe el elemento
//en el array, y si existe devuelve su posicion
var
  entero: central

inicio
    si primero > ultimo entonces
      posic ← 0
    si_no
      central ← (primero + ultimo) div 2
      si a[central] = num entonces
        posic ← central
      si_no
        si num > a[central] entonces
          primero ← central + 1
        si_no
          ultimo ← central - 1
        fin_si
        busqueda(a,posic,primero,ultimo,num)
      fin_si
    fin_si
fin_procedimiento
```

9.7. Partiendo de la siguiente lista inicial:

80	36	98	62	26	78	22	27	2	45

tomar como elemento pivote el contenido del que ocupa la posición central y realizar el seguimiento de los distintos pasos que llevarían a su ordenación por el método Quick-Sort. Implementar el algoritmo correspondiente.

1	2	3	4	5	6	7	8	9	10
80	**36**	**98**	**62**	**26**	**78**	**22**	**27**	**2**	**45**
2								80	
	22					36			
		26		98					
		j	i						

1	2	3	4	5	6	7	8	9	10
2	**22**	**26**	**62**	**98**	**78**	**36**	**27**	**80**	**45**
	22								
j		i							

1	2	3	4	5	6	7	8	9	10
2	**22**	**26**	**62**	**98**	**78**	**36**	**27**	**80**	**45**
			27				62		
				36		98			
				j	i				

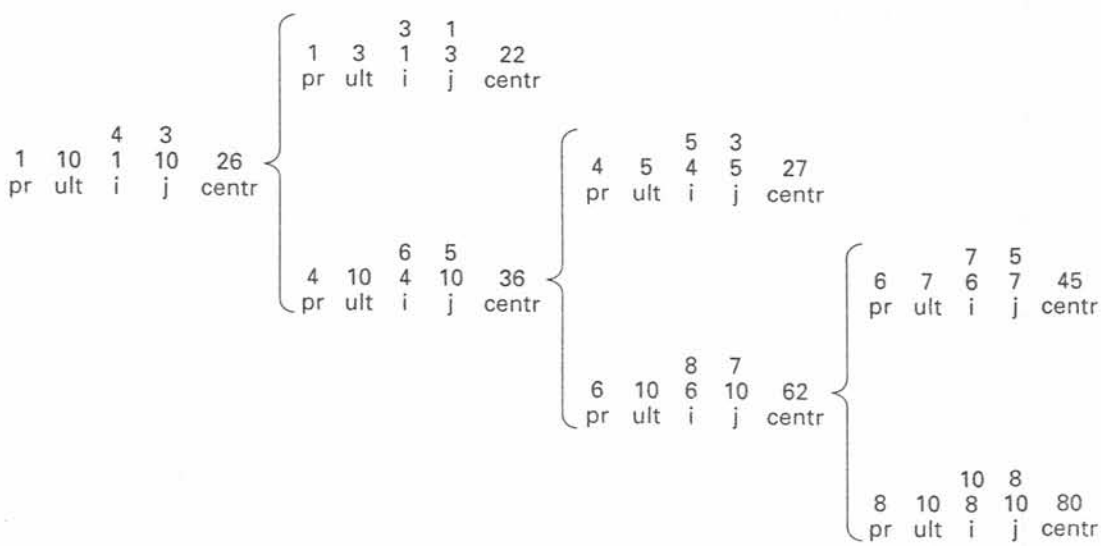
1	2	3	4	5	6	7	8	9	10
2	**22**	**26**	**27**	**36**	**78**	**98**	**62**	**80**	**45**
			27						
		j		i					

1	2	3	4	5	6	7	8	9	10
2	**22**	**26**	**27**	**36**	**78**	**98**	**62**	**80**	**45**
					45				78
						62	98		
						j	i		

1	2	3	4	5	6	7	8	9	10
2	**22**	**26**	**27**	**36**	**45**	**62**	**98**	**80**	**78**
					45				
				j		i			

1	2	3	4	5	6	7	8	9	10
2	**22**	**26**	**27**	**36**	**45**	**62**	**98**	**80**	**78**
							78		98
								80	
							j		i

1	2	3	4	5	6	7	8	9	10
2	**22**	**26**	**27**	**36**	**45**	**62**	**78**	**80**	**98**



```
algoritmo Quicksort
  tipo
    array[1..10] de entero: arr
  var
    arr    : a
    entero : k

inicio
  desde k ← 1 hasta 10 hacer
    leer (a[k])
  fin_desde
    rapido (a,10)
    desde k ← 1 hasta 10 hacer
      escribir (a[k])
    fin_desde
fin

procedimiento intercambiar (E/S ;entero: m,n)
  var
    entero: AUXI
  inicio
    AUXI ← m
    m ← n
    n ← AUXI
  fin_procedimiento

procedimiento partir (E/S; arr: a; E; entero: primero, ultimo)
var
  entero: i,j,central

inicio
  i ← primero
  j ← ultimo
  // encontrar elemento pivote, central, y almacenar su contenido
  central ← a[ (primero + ultimo) div 2 ]
  repetir
    mientras a[i] < central hacer
      i ← i + 1
    fin_mientras
      mientras a[j] > central hacer
        j ← j - 1
      fin_mientras
      si i <= j entonces
        intercambiar( a[i],a[j] )
        i ← i + 1
        j ← j - 1
      fin_si
    hasta_que i > j
    si primero < j entonces
      partir (a,primero,j)
    fin_si
    si i < ultimo entonces
      partir (a,i,ultimo)
  fin_si
fin_procedimiento

procedimiento rapido (E/S arr: a ;  E entero: n)
inicio
  partir (a,1,n)
fin_procedimiento
```

EJERCICIOS

9.1. Realizar el diagrama de flujo y el pseudocódigo que permuta tres enteros: $n1$, $n2$, y $n3$ en orden creciente.

9.2. Escribir un algoritmo que lea diez nombres y los ponga en orden alfabético utilizando el método de selección. Utilice los siguientes datos para comprobación: Sánchez, Waterloo, McDonald, Bartolomé, Jorba, Clara, David, Robinson, Francisco, Westfalia.

9.3. Clasificar el array (vector):

```
42    57    14    40    96    19    08    68
```

por los métodos: 1) selección, 2) burbuja. Cada vez que se reorganice el vector, se debe mostrar el nuevo vector reformado.

9.4. Supongamos que se tiene una secuencia de n números que deben ser clasificados:

1. Utilizando el método de selección, cuántas comparaciones y cuántos intercambios se requieren para clasificar la secuencia si:

   - ya está clasificado,
   - está en orden inverso.

2. Repetir el paso i para el método de selección.

9.5. Escribir un algoritmo de búsqueda lineal para un vector ordenado.

9.6. Un algoritmo ha sido diseñado para leer una lista de no más de 1.000 enteros positivos, cada uno menos de 100, y ejecutar algunas operaciones. El cero es la marca final de la lista. El programador debe obtener en el algoritmo:

1. Visualizar los números de la lista en orden creciente.
2. Calcular e imprimir la mediana (valor central).
3. Determinar el número que ocurre más frecuentemente.
4. Imprimir una lista que contenga:

   - números menores de 30,
   - números mayores de 70,
   - números que no pertenezcan a los dos grupos anteriores.

5. Encontrar e imprimir el entero más grande de la lista junto con su posición en la lista antes de que los números hayan sido ordenados.

9.7. Diseñar diferentes algoritmos para insertar un nuevo valor en una lista (vector). La lista debe estar ordenada en orden ascendente antes de proceder a la clasificación.

CAPÍTULO 10

ORDENACIÓN, BÚSQUEDA Y FUSIÓN EXTERNA (ARCHIVOS)

CONTENIDO

10.1. Introducción.
10.2. Archivos ordenados.
10.3. Fusión de archivos.
10.4. Partición de archivos.
10.5. Clasificación de archivos.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

Los sistemas de procesamiento de la información procesan normalmente gran cantidad de información. En estos casos los datos se almacenan sobre soportes de almacenamiento masivo (cintas y discos magnéticos).

Los algoritmos de ordenación presentados en el Capítulo 9 no son aplicables si la masa de datos no cabe en la memoria central de la computadora y se encuentran almacenados en su soporte como una cinta. En estos casos se suelen colocar en memoria central las fichas que se procesan y a las que se pueda acceder directamente.

Normalmente, estas técnicas no son muy eficaces, por lo que se utilizan técnicas distintas de ordenación. La técnica más importante es la *fusión* o *mezcla*.

Este capítulo realiza una introducción a las técnicas de ordenación, búsqueda y mezcla o fusión externas.

10.1. INTRODUCCIÓN

Cuando la masa de datos a procesar es grande y no cabe en la memoria central de la computadora, éstos se organizan en archivos que, a su vez, se almacenan en dispositivos externos de memoria auxiliar (discos, cintas magnéticas, etc.).

Las operaciones básicas estudiadas en el Capítulo 9, ordenación, búsqueda e intercalación o mezcla, sufren un cambio importante en su concepción, derivado esencialmente del hecho físico de que los datos a procesar no caben en la memoria principal de la computadora.

10.2. ARCHIVOS ORDENADOS

El tratamiento de los archivos secuenciales exige que éstos se encuentren ordenados respecto a un campo del registro, denominado *campo clave*.

Supongamos un archivo del personal de una empresa, cuya estructura de registros es la siguiente:

NOMBRE	*tipo cadena*	*(nombre del empleado)*
DIRECCION	*tipo cadena*	*(dirección)*
FECHA	*tipo cadena*	*(fecha de nacimiento)*
SALARIO	*tipo numérico*	*(salario)*
CATEGORIA	*tipo cadena*	*(categoría laboral)*
DNI	*tipo cadena*	*(número de DNI)*

La clasificación en orden ascendente o descendente se puede realizar con respecto a una clave (nombre, dirección, etc.). Sin embargo, puede ser interesante tener clasificado un fichero por categoría laboral, y a su vez se pueden tener por cada categoría laboral los registros agrupados por nombres o direcciones. Ello nos lleva a la conclusión de que un archivo puede estar ordenado por un campo clave o una jerarquía de campos.

Se dice que un *archivo* (estructura del registro: campos `C1, C2, ... Cn`) está ordenado principalmente por el campo `C1`, en orden secundario 1 por el campo `C2`, en orden secundario 2 por el campo `C3`, etc., en orden secundario n por el campo `Cn + 1`. El archivo tendrá la siguiente organización:

- los registros aparecen en el archivo según el orden de los valores del campo clave `m1`,
- si se considera un mismo valor `m1`, los registros aparecen en el orden de los valores del campo `m2`,
- para un mismo valor de $m(m_i)$ los registros aparecen según el orden de los valores del campo $m_i + 1$, siendo $1 =< i <= n$.

Si se desea ordenar un archivo principalmente por m_1, y en orden secundario 1 por m_2, se necesita:

- ordenar primero por m_2,
- ejecutar a continuación una ordenación estable por el campo m_1.

La mayoría de los Sistemas Operativos actuales disponen de programas estándar (utilidad) que realizan la clasificación de uno o varios archivos (*sort*). En el caso del Sistema Operativo MS-DOS existe la orden `SORT`, que permite realizar la clasificación de archivos según ciertos criterios específicos.

Los algoritmos de clasificación externa son muy numerosos y a ellos dedicaremos gran parte de este capítulo.

10.3. FUSIÓN DE ARCHIVOS

La *fusión* o *mezcla de archivos* (*merge*) consiste en reunir en un archivo los registros de dos o más archivos ordenados por un campo clave T. El archivo resultante será un archivo ordenado por el campo clave T.

Supongamos que se dispone de dos archivos ordenados sobre dos cintas magnéticas y que se desean mezclar o fundir en un solo archivo ordenado. Sean los archivos F1 y F2 almacenados en dos cintas diferentes. El archivo F3 se almacenará en una tercera cinta.

El algoritmo de fusión de archivos será:

```
inicio
  //fusion de dos archivos
  poner archivo 1 en cinta 1, archivo 2 en cinta 2
  seleccionar de los dos primeros registros de archivo
    1 y archivo 2 el registro de clave mas pequeña y
    almacenarlo en un nuevo archivo 3
  mientras (archivo 1 no vacio) y (archivo 2 no vacio) hacer
    seleccionar el registro siguiente con clave mas
    pequeña y almacenarlo en el archivo 3
  fin_mientras
  //uno de los archivos no esta aun vacio
  almacenar resto archivo en archivo 3 registro a registro
fin
```

Ejemplo 10.1

Se dispone de dos archivos, F1 *y* F2, *cuyos campos claves son:*

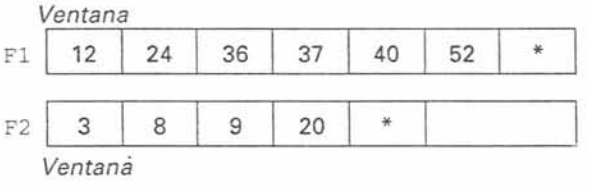
```
F1    12    24    36    37    40    52
F2     3     8     9    20
```

y se desea un archivo FR ordenado, que contenga los dos archivos F1 y F2.

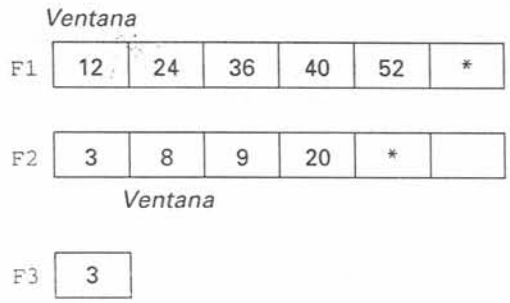
La estructura de los archivos F1 y C es:

```
F1    12   24   36   37   40   52    EOF(*)  Fin archivo (eof)
F2     3    8    9   20
```

Para realizar la fusión de F1 y F2 es preciso acceder a los archivos F1 y F2, que se encuentran en soportes magnéticos en organización secuencial. En cada operación de acceso a un archivo, sólo se puede acceder a un único elemento del archivo en un momento dado. Para realizar la operación, se utiliza una variable de trabajo del mismo tipo que los elementos del archivo. Esta variable representa al elemento actual del archivo y la denominaremos *ventana*, debido a que será la variable que nos permitirá ver el archivo, elemento tras elemento. El archivo se recorre en un único sentido y su final físico termina con una marca especial denominada `fin de archivo (EOF, end of file)`; por ejemplo, un asterisco (*).



Se comparan las claves de las ventanas y se sitúa la más pequeña 3 (F2) en el archivo de salida. A continuación, se avanza un elemento el archivo F2 y se realiza una nueva comparación de los elementos situados en las ventanas.



Cuando uno u otro de los archivos de entrada se ha terminado, se copia el resto del archivo sobre el archivo de salida y el resultado final será:

El algoritmo correspondiente de fusión de archivos será:

```
algoritmo Fusion_archivo

var
  entero:ventana1, ventana2, ventanaS
  archivo_s de entero: F1,F2,F3
  //ventana1,ventana2  claves de los archivos F1,F2
  //ventanaS           claves del archivo FR

inicio
  abrir (F1, l, 'nombre')
  abrir (F2, l, 'nombre2')
  crear (FR, 'nombre3')
  abrir (FR, e, 'nombre3')
  leer (F1, ventana1)
  leer (F2, ventana2)

  mientras no fda(F1) y no fda(F2) hacer
    si ventana1 <= ventana2 entonces
      ventanaS ← ventana1
      escribir(FR,ventanaS)
      leer (F1, ventana1)
    si_no
      ventanaS ← ventana2
      escribir (FR, ventanaS)
      leer (F2, ventana2)
    fin_si
  fin_mientras
```

```
  //lectura terminada de F1 o F2
  mientras no fda (F1) hacer
    ventanaS ← ventana1
    escribir(FR, ventanaS)
    leer(F1, ventana1)
  fin_mientras
  mientras no fda(F2)hacer
    ventanaS ← ventana2
    escribir (FR, ventanaS)
    leer(F2, ventana2)
  fin_mientras

  cerrar (F1,F2,FR)
fin
```

Consideremos ahora el otro caso posible en los archivos secuenciales. El final físico del archivo se detecta al leer el último elemento (no la marca de fin de archivo) y los ficheros son de registros con varios campos. El algoritmo correspondiente a la *fusión* sería:

```
tipo
  registro: datos_personales
    -: C              //campo por el que estan ordenados
    -:-
  fin registro
  archivo_s de datos_personales: arch
var
  datos_personales:r1, r2
  arch: f1, f2, f       //f es el fichero resultante
  logico: fin1, fin2

inicio
  abrir (f1, 1, 'nombre1')
  abrir (f2, 1, 'nombre2')
  crear (f, 'nombre3')
  abrir (f, e, 'nombre3')
  fin1← falso
  fin2← falso
  si fda (f1) entonces
    fin1← verdad
  si-no
    llamar_a leer_reg (f1, r1)
  fin_si
  si fda (f2) entonces
    fin2← verdad
  si_no
    llamar_a leer_reg (f2, r2)
  fin_si

  mientras no fin1 y no fin2 hacer
    si r1.c < r2.c entonces
      llamar_a escribir_reg (f, r1)
      si fda (f1) entonces
        fin1← verdad
      si_no
        llamar_a leer_reg (f1, r1)
      fin_si
```

```
    si_no
      llamar_a escribir_reg (f, r2)
    si fda (f2) entonces
      fin2← verdad
    si_no
      llamar_a leer_reg (f2,r2)
        fin_si
    fin_si
  fin mientras

  mientras no fin1 hacer
    llamar_a escribir_reg (f, r1)
    si fda (f1) entonces
      fin1← verdad
    si_no
      llamar_a leer_reg (f1, r1)
    fin_si
  fin_mientras

  mientras no fin2 hacer
    llamar_a escribir_reg (f, r2)
    si fda (f2) entonces
      fin2← verdad
    si_no
      llamar_a leer_reg (f2, r2)
    fin_si
  fin_mientras

  cerrar (f1, f2, f)
fin
```

10.4. PARTICIÓN DE ARCHIVOS

La *partición* o *división* de un archivo consiste en repartir los registros del archivo en otros dos o más archivos en función de una determinada condición.

Aunque existen muchos métodos de producir particiones a partir de un archivo no clasificado, consideraremos sólo los siguientes métodos:

- *clasificación interna,*
- *por el contenido,*
- *selección por sustitución,*
- *secuencias.*

Supongamos el archivo de entrada siguiente, en el que se indican las claves de los registros:

110	48	33	69	46	2	62	39	28	47	16	19	34	55
99	78	75	40	35	87	10	26	61	92	99	75	11	2
28	16	80	73	18	12	89	50	47	36	67	94	23	15
84	44	53	60	10	39	76	18	24	86				

10.4.1. Clasificación interna

El método más sencillo consiste en leer M registros a la vez de un archivo no clasificado, clasificarlos utilizando un método de clasificación interna y, a continuación, darles salida como partición. Obsérvese que todas las particiones producidas de este modo, excepto posiblemente la última, contendrán exactamente M registros. La tabla siguiente muestra las particiones producidas a partir del archivo de entrada utilizando un tamaño de memoria (M) de cinco registros.

33	46	48	69	110
2	28	39	47	62
16	19	34	55	99
35	40	75	78	87
10	26	61	92	99
2	11	16	28	75
12	18	73	80	89
36	47	50	67	94
15	23	44	53	84
10	18	39	60	76
24	86			

10.4.2. Partición por contenido

La partición del archivo de entrada se realiza en función del contenido de uno o más campos del registro.

Así, por ejemplo, supongamos un archivo F que se desea dividir en dos archivos, F1 y F2, tal que F1 contenga todos los registros que contengan, en el campo clave C, el valor V, y en el archivo F2 los restantes registros.

El algoritmo de partición se muestra a continuación:

```
algoritmo Particion_contenido
....
inicio
  abrir (f, l,'nombre')
  crear (f1, 'nombre1')
  abrir (f1, e, 'nombre1')
  crear (f2, 'nombre2')
  abrir ( f2, e, 'nombre2')
  leer(v)
  mientras no fda (f) hacer
    llamar_a leer_reg (f, r)
    si v = r.c entonces
      llamar_a escribir_reg (f1,r)
    si_no
      llamar_a escribir_reg (f2, r)
    fin_si
  fin_mientras
  cerrar (f, f1, f2)
fin
```

10.4.3. Selección por sustitución

La clasificación interna vista en el Apartado 10.4.1 no tiene en cuenta la ventaja que puede suponer cualquier ordenación parcial que pueda existir en el archivo de entrada. El algoritmo de selección por sustitución tiene en cuenta tal ordenación. Los pasos a dar para obtener particiones ordenadas son:

1. Leer N registros del archivo desordenado, poniéndolos todos a no congelados.
2. Obtener el registro R con clave más pequeña de entre los no congelados y escribirlo en partición.
3. Sustituir el registro por el siguiente del archivo de entrada. Este registro se congelará si su clave es más pequeña que la del registro R y no se congelará en otro caso. Si hay registro sin congelar volver al paso 2.
4. Comenzar nueva partición. Si se ha llegado a fin de fichero se repite el proceso sin leer.

NOTA: Al final de este método los ficheros con las particiones tienen secuencias ordenadas, lo que no quiere decir que ambos hayan quedado completamente ordenados.

F:	3	31	14	42	10	15	8	13	63	18	50
				F1							F2
3	31	14	42	3			13	50	8	18	8
10	31	14	42	10			13	50	8	18	13
15	31	14	42	14			13	50	8	18	18
15	31	8	42	15			13	50	8	18	50
13	31	8	42	31			13	50	8	18	
13	63	8	42	42							
13	63	8	18	63							
	50	8	18								

```
algoritmo Particion_s

const n= <valor>
tipo
  registro:datos_personales
    <tipo_dato>:c
    ..............:..
  fin_registro
  registro:datos
      datos_personales: dp
      logico          : congela
  fin_registro
  array[1..n] de datos: arr
  archivo_s de datos_personales: arch
var
  datos_personales: r
  arr             : a
  arch            : f1,f2, f
  logico          : sw
  entero          : numcongelados, i, posicionmenor
```

```
inicio
  abrir(f, 1, 'nombre')
  crear(f1, 'nombre1')
  abrir(f1, e, 'nombre1')
  crear(f2,'nombre2')
  abrir(f2, e, 'nombre32')
  numcongelados ← 0
  desde i ← hasta n hacer
    si no fda(f) entonces
      llamar_a leer_reg(f, r)
      a[i].dp ← r
      a[i].congela ← falso
    si_no
      a[i].congela ← verdad
      numcongelados ← numcongelados + 1
    fin_si
  fin_desde
  sw ← verdad

  mientras no fda(f) hacer
    mientras numcongelados < n y no fda(f) hacer
      llamar_a buscar_no_congelado_menor(a, posicionmenor)
      si sw entonces
        llamar_a escribir_reg(f1, a[posicionmenor].dp)
      si_no
        llamar_a escribir_reg(f2, a[posicionmenor].dp)
      fin_si
        llamar_a leer_reg(f, r)
      si r.c. > a[posicionmenor].dp.c entonces
        a[posicionmenor].dp ← r
      si_no
        a[posicionmenor].dp ← r
        a[posicionmenor].congela ← verdad
        numcongelados ← numcongelados + 1
      fin_si
    fin_mientras
    sw ← no sw
      llamar_a descongelar(a)
    numcongelados ← 0
  fin_mientras

  mientras numcongelados < n hacer
      llamar_a buscar_no_congelado_menor(a, posicionmenor)
    si sw entonces
      llamar_a escribir_reg(f1,a[posicionmenor].dp)
    si_no
      llamar_a escribir_reg(f2, a[posicionmenor].dp)
    fin_si
    a[posicionmenor].congela ← verdad
    numcongelados ← numcongelados + 1
  fin_mientras
  cerrar(f, f1, f2)
fin
```

10.4.4. Partición por secuencias

Los registros se dividen en secuencias alternativas con longitudes iguales o diferentes según los casos. Las secuencias pueden ser de diferentes diseños:

- El archivo F se divide en dos archivos, F1 y F2, copiando alternativamente en uno y otro archivo secuencias de registros de longitud *m*. (Algoritmo particion_1.)
- El archivo F se divide en dos archivos, F1 y F2, de modo que en F1 se copian los registros que ocupan las posiciones pares y en F2 los registros que ocupan las posiciones impares. (Algoritmo particion_2.)

```
algoritmo Particion_1
tipo
  registro = datos personales
    <tipodato> : C
  ..............
  fin_registro
  archivo_s de datos_personales : arch
var
  datos_personales: r
  arch            : f, f1, f2
  logico          : SW
  entero          : i, n
inicio
  abrir (f, 1, 'nombre')
  crear (f1, 'nombre1')
  abrir (f1, e, 'nombre1')
  crear (f2, 'nombre2')
  abrir (f2, e, 'nombre2')
  i ← o
  leer (n)
  SW ← verdad

  mientras no fda (f) hacer
    llamar_a leer_reg (f,r)
    si SW entonces
      llamar_a escribir_reg (f2, r)
    si_no
      llamar_a escribir_reg (f2,r)
    fin_si
      i ← i + 1
    si i = n entonces
      SW ← no SW
      i ← 0
    fin_si
  fin_mientras

  cerrar (f, f1, f2)
fin

algoritmo Particion_2
  tipo
  registro = datos_personales
    <tipo_dato> : C
    ..............
  fin_registro
  archivo_s de datos_personales : arch
```

```
var
  datos_personales : r
  arch             : f1, f2, f
  logico           : SW

inicio
  abrir (f, 1, 'nombre')
  crear (f1, 'nombre1')
  abrir (f1, e, 'nombre1')
  crear (f2, 'nombre2')
  abrir (f2, e, 'nombre2')
  SW← verdad
  mientras no fda (f) hacer
    llamar_a leer_reg (f, r)
    si SW entonces
      llamar_a escribir_reg (f1,r)
      si_no
      llamar_a escribir_reg (f2,r)
    fin_si
    SW ← no SW
  fin_mientras
  cerrar (f, f1, f2)
fin
```

10.5. CLASIFICACIÓN DE ARCHIVOS

Los archivos están *clasificados en orden ascendente* o *descendente* cuando todos sus registros están ordenados en sentido ascendente o descendente respecto al valor de un campo determinado, denominado *clave de ordenación*.

Si el archivo a ordenar cabe en memoria central, se carga en un vector y se realiza una clasificación interna, transfiriendo a continuación el archivo ordenado al soporte externo o copiando el resultado en el archivo original, si no se desea conservar.

En el caso de que el archivo no quepa en memoria central, la clasificación se realizará sobre el archivo almacenado en un soporte externo. El inconveniente de este tipo de clasificación reside en el tiempo, que será mucho mayor debido especialmente a las operaciones entrada/salida de información que la clasificación externa sigue.

Los algoritmos de clasificación son muy variados, pero muchos de ellos se basan en procedimientos mixtos consistentes en aprovechar al máximo la capacidad de la memoria central.

Como métodos de clasificación de archivos que no utilizan la memoria central y son aplicables a archivos secuenciales, tenemos la mezcla directa y la mezcla natural.

10.5.1. Clasificación por mezcla directa

El método más fácil de comprender es el conocido de *mezcla directa*. Veremos su aplicación a través de un breve ejemplo en el que se aplicará el método sobre un vector. Podemos pensar en los componentes del vector como las claves de los registros sucesivos del archivo.

El procedimiento consiste en una partición sucesiva del archivo y una fusión que produce secuencias ordenadas. La primera partición se hace para secuencias de longitud 1 y la fusión producirá secuencias ordenadas de longitud 2. A cada nueva partición y fusión se duplicará la longitud

de las secuencias ordenadas. El método terminará cuando la longitud de la secuencia ordenada exceda la longitud del archivo a ordenar.

Consideremos el archivo:

```
F: 19   27   2   8   36   5   20   15   6
```

El archivo `F` se divide en dos nuevos archivos `F1` y `F2`:

```
F1:   19   2   36   20   6
F2:   27   8    5   15
```

Ahora se funden los archivos `F1` y `F2`, formando pares ordenados:

```
F:  19   27   2   8   5   36   15   20   6
```

Se vuelve a dividir de nuevo en partes iguales y en secuencias de longitud 2:

```
F1: 19   27   5   36   6
F2: 2    8    15  20
```

La fusión de los archivos producirá

```
F: 2   8   19   27   5   15   20   36   6
```

La nueva partición será:

```
F1: 2   8   19   27   6
F2: 5   15  20   36
```

La nueva fusión será:

```
F: 2   5   8   15   19   20   27   36   6
```

Cada operación que trata por completo el conjunto de datos en su totalidad se denomina una *fase* y el proceso de ordenación se denomina *pasada*.

```
F1: 2   5   8   15   19   20   27   36
F2: 6
```

```
F: 2   5   6   8   15   19   20   27   36
```

Evidentemente, la clave de la clasificación es disminuir el número de pasadas e incrementar su tamaño; una secuencia ordenada es una que contiene sólo una pasada que, a su vez, contiene todos los elementos de la pasada.

Ejemplo 10.2

Para la implementación de los siguientes algoritmos no *se consideró la existencia de un registro especial que indicara el fin de archivo. La función* `FDA(id_arch)` *retorna cierto cuando se accede al último registro.*

Si consideráramos la existencia del registro especial que marca el fin de archivo se podría prescindir del uso de las variables lógicas `fin, fin1, fin2`.

```
algoritmo Ord_mezcla_directa
   ..........................
procedimiento ordmezcladirecta

var datos_personales: r,r1,r2
    arch            : f,f1,f2    // El tipo arch es archivo_s de datos_personales
    entero          : lgtud, long
    logico          : sw,fin1,fin2
    entero          : i,j

inicio
  //calcularlongitud(f) es una funcion definida por el usuario que nos devuelve
  //el numero de registros del archivo original
  long ← calcularlongitud(f)
  lgtud ← 1
  mientras lgtud < long hacer
    abrir(f,1,'fd')
    crear(f1,'f1d')
    crear(f2,'f2d')
    abrir(f1,e,'f1d')
    abrir(f2,e,'f2d')
    i ← 0
    sw ← verdad
    mientras no fda(f) hacer
      llamar_a leer_reg(f,r)
      si sw entonces
        llamar_a escribir_reg(f1,r)
        si_no
          llamar_a escribir_reg(f2,r)
      fin_si
      i ← i + 1
      si i=lgtud entonces
        sw ← no sw
        i ← 0
      fin_si
    fin_mientras

    cerrar(f,f1,f2)
    abrir(f1,1,'f1d')
    abrir(f2,1,'f2d')
    crear(f,'fd')
    abrir (f,e,'fd')
    i ← 0
    j ← 0
    fin1 ← falso
    fin2 ← falso
    si fda(f1) entonces
      fin1 ← verdad
    si_no
      llamar_a leer_reg(f1,r1)
    fin_si
    si fda(f2) entonces
      fin2 ← verdad
    si_no
      llamar_a leer_reg(f2,r2)
    fin_si
```

```
mientras no fin1 o no fin2 hacer
  mientras no fin1 y no fin2 y (i < lgtud) y (j < lgtud) hacer
    si menor(r1,r2) entonces
      llamar_a escribir_reg(f,r1)
      si fda(f1) entonces
        fin1 ← verdad
      si_no
        llamar_a leer_reg(f1,r1)
      fin_si
      i ← i + 1
    si_no
      llamar_a escribir_reg(f,r2)
      si fda(f2) entonces
        fin2 ← verdad
      si_no
        llamar_a leer_reg(f2,r2)
      fin_si
      j ← j + 1
    fin_si
  fin_mientras

  mientras no fin1 y (i < lgtud) hacer
    llamar_a escribir_reg(f,r1)
    si fda(f1) entonces
      fin1 ← verdad
    si_no
      llamar_a leer_reg(f1,r1)
    fin_si
    i ← i + 1
  fin_mientras

  mientras no fin2 y (j < lgtud) hacer
    llamar_a escribir_reg(f,r2)
    si fda(f2) entonces
      fin2 ← verdad
    si_no
    llamar_a leer_reg(f2,r2)
    fin_si
    j ← j + 1
  fin_mientras

  i ← 0
  j ← 0
fin_mientras                    // del mientras no fin1 o no fin2

cerrar(f,f1,f2)
lgtud ← lgtud * 2
fin_mientras                    // del mientras lgtud < long
borrar('f1d')
borrar('f2d')
fin_procedimiento
```

10.5.2. Clasificación por mezcla natural

Es uno de los mejores métodos de ordenación de ficheros secuenciales. Consiste en aprovechar la posible ordenación interna de las secuencias del archivo (F), obteniendo con ellas particiones ordenadas de longitud variable sobre dos ficheros auxiliares F1 y F2. A partir de estos ficheros auxiliares escribiremos un nuevo F mezclando los segmentos crecientes maximales de cada uno de ellos.

Ejemplo 10.3

Clasificar el vector

```
F: 19   27   2   8   36   5   20   15   6
```

Se divide F en dos vectores F1 y F2, donde se ponen alternativamente los elementos F1 y F2. F está ahora vacío.

Etapa 1, fase 1:

```
F1: 19   27/   5   20/   6
F2:  2    8   36/   15
```

Se selecciona el elemento más pequeño de F1 y F2, que pasan a estar en F3.

Etapa 1, fase 2:

```
F1: 19   27/   5   20/   6
F2:  8   36/   15
F3:  2
```

Ahora se comparan 8 y 19, se selecciona 8. De modo similar, 19 y 27:

```
F1:  5   20/   6
F2: 36/  15
F3:  2    8   19   27
```

En F1 se ha interrumpido la secuencia creciente y se continua con F2 hasta que también en él se termine la secuencia creciente.

```
F1:  5   20/   6
F2: 15
F3:  2    8   19   27   36
```

Ahora 5 y 15 son menores que 36. Finalmente, se tendrá:

```
F3: 2   8   19   27   36/   5   15   20/   6
```

F1 y F2 están ahora vacíos.

Etapa 2, fase 1:

Dividir F3 en dos:

```
F1: 2    8   19   27   36/   6
F2: 5   15   20
```

Etapa 2, fase 2:

Se mezclan F1 y F2 :

```
F3:   2   5   8   15   19   20   27   36   6
```

Etapa 3, fase 1:

```
F1:   2   5   8   15   19   20   27   36
F2:   6
```

Etapa 3, fase 2:

```
F3:  2   5   6   8   15   19   20   27   36
```

y el archivo F3 ya está ordenado.

Algoritmo

```
algoritmo Ord_mezcla_natural
procedimiento ordmezclanatural

var datos_personales: r,r1,r2,ant,ant1,ant2
    arch            : f,f1,f2        //El tipo arch es archivo_s
                                     //de datos_personales
    logico          : ordenado,crece,fin,fin1,fin2
    entero          : numsec

inicio
  ordenado ← falso
  mientras no ordenado hacer
    abrir(f,1,'fd')
    crear(f1,'f1d')
    crear(f2,'f2d');
    abrir(f1,e,'f1d')
    abrir(f2,e,'f2d')
    fin ← falso
    si fda (f) entonces
      fin ← verdad
    si_no
      llamar_a leer_reg(f,r)
    fin_si

    mientras no fin hacer
      ant ← r
      crece ← verdad
      mientras crece y no fin hacer
        si menorigual(ant,r) entonces
          llamar_a escribir_reg(f1,r)
          ant ← r
          si fda (f) entonces
            fin ← verdad
          si_no
            llamar_a leer_reg(f,r)
          fin_si
        si_no
          crece ← falso
        fin_si
      fin mientras

      ant ← r
      crece ← verdad

      mientras crece y no fin hacer
        si menorigual(ant,r) entonces
          llamar_a escribir_reg(f2,r)
```

```
      ant ← r
      si fda(f) entonces
        fin ← verdad
      si_no
        llamar_a leer_reg(f,r)
      fin_si
    si_no
      crece ← falso
  fin_si
fin_mientras
fin_mientras

cerrar(f,f1,f2)

abrir(f1,1,'f1d')
abrir(f2,1.'f2d')
crear(f,'fd')
abrir(f,e,'fd')
fin1 ← falso
fin2 ← falso
si fda(f1) entonces
  fin1 ← verdad
si_no
  llamar_a leer_reg(f1,r1)
fin_si
si fda(f2) entonces
  fin2 ← verdad
si_no
  llamar_a leer_reg(f2,r2)
fin_si
numsec ← 0

mientras no fin1 y no fin2 hacer
  ant1 ← r1
  ant2 ← r2
  crece ← verdad

  mientras no fin1 y no fin2 y crece hacer
    si menorigual(ant1,r1) y menorigual(ant2,r2) entonces
      si menorigual(r1,r2) entonces
        llamar_a escribir_reg(f,r1)
        ant1 ← r1
        si fda(f1) entonces
          fin1 ← verdad
        si_no
          llamar_a leer_reg(f1,r1)
        fin_si
      si_no
      llamar_a escribir_reg(f,r2)
      ant2 ← r2
      si fda(f2) entonces
        fin2 ← verdad
      si_no
        llamar_a leer_reg(f2,r2)
        fin_si
      fin_si
    si_no
      crece ← falso
    fin_si
  fin_mientras
```

```
        mientras no fin1 y menorigual(ant1,r1) hacer
          llamar_a escribir_reg(f,r1)
            ant1 ← r1
            si fda(f1) entonces
              fin1 ← verdad
            si_no
          llamar_a leer_reg(f1,r1)
        fin_si
      fin_mientras

      mientras no fin2 y menorigual(ant2,r2) hacer
        llamar_a escribir_reg(f,r2)
        ant2 ← r2
        si fda(f2) entonces
          fin2 ← verdad
        si_no
          llamar_a leer_reg(f2,r2)
        fin_si
      fin_mientras
      numsec ← numsec + 1
    fin_mientras                    // del mientras no fin1 y no fin2

    si no fin1 entonces
      numsec ← numsec + 1
      mientras no fin1 hacer
        llamar_a escribir_reg(f,r1)
        si fda(f1) entonces
          fin1 ← verdad
        si_no
          llamar_a leer_reg(f1,r1)
        fin_si
      fin_mientras
    fin_si
    si no fin2 entonces
      numsec ← numsec + 1
      mientras no fin2 hacer
        llamar_a escribir_reg(f,r2)
        si fda(f2) entonces
          fin2 ← verdad
        si_no
          llamar_a leer_reg(f2,r2)
        fin_si
      fin_mientras
    fin_si
    cerrar(f,f1,f2)

    si numsec <= 1 entonces
      ordenado ← verdad
    fin_si

  fin_mientras                      // del mientras no ordenado
  borrar('f1d')
  borrar('f2d')
fin_procedimiento
```

10.5.3. Clasificación por mezcla de secuencias equilibradas

Este método utiliza la memoria de la computadora para realizar clasificaciones internas y cuatro archivos secuenciales temporales para trabajar.

Supongamos un archivo de entrada F que se desea ordenar por orden creciente de las claves de sus elementos. Se dispone de cuatro archivos secuenciales de trabajo, F1, F2, F3 y F4, y que se pueden colocar *m* elementos en memoria central en un momento dado en una tabla T de *m* elementos. El proceso es el siguiente:

1. Lectura de archivo de entrada por bloques de *n* elementos.
2. Ordenación de cada uno de estos bloques y escritura alternativa sobre F1 y F2.
3. Fusión de F1 y F2 en bloques de $2n$ elementos, que se escriben alternativamente sobre F3 y F4.
4. Fusión de F3 y F4 y escritura alternativa en F1 y F2, de bloques con $4n$ elementos ordenados.
5. El proceso consiste en doblar cada vez el tamaño de los bloques y utilizando las parejas (F1, F2) y (F3, F4).

Fichero de entrada:

46 66 4 12 7 5 34 32 68 8 99 16 13 14 12 10

F1 [4 12 46 66] [8 16 68 99]
F2 [5 7 32 34] [10 12 13 14]
F3 *vacío*
F4 *vacío*

Fusión por bloques:

F1 *vacío*
F2 *vacío*
F3 4 5 7 12 32 34 46 66 /F4 8 10 12 13 14 16 68 99

La mezcla o fusión final es:

F1 4 5 7 8 10 12 12 13 14 16 32 34 46 66 68 99
F2 *vacío*
F3 *vacío*
F4 *vacío*

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

10.1. Realizar el algoritmo de partición de un archivo F en dos particiones, F1 y F2, según el contenido de campo clave C. El contenido debe tener el valor V.

```
algoritmo Particion_contenido
inicio
  abrir (F, e, 'f0')  // lectura
  abrir (F1, e, 'f1') // escritura
  abrir (F2, e, 'f2') // escritura
  mientras no fda (F) hacer
    leer (archivo F, registro R)
    si C = V
      entonces escribir (F1,R)
      sino     escribir (F2,R)
    fin_si
  fin_mientras
  cerrar (F1,F2,F)
fin
```

10.2. Realizar el algoritmo de partición por secuencias alternativas de longitud n.

Entrada: Archivo F Salida: Archivos F1 y F2
Secuencia: longitud n (n registros en cada secuencia)
Cada n registros se almacenan alternativamente en F1 y F2

```
algoritmo Partir_alternativa
inicio
  var SW : logica
      I: entero {contador de elementos de la secuencia}
  abrir archivo F para lectura
  abrir archivo F1 para escritura
  abrir archivo F2 para escritura
  SW ← falso
  i ← 0
  mientras no fda (F) hacer
      leer (F,R)
      si SW = falso
        entonces escribir (F1,R)
        sino     escribir (F2,R)
      fin_si
      i ← i + 1
      si i = n
        entonces SW ← verdadero
                 i ← 0          {se inicializa el contador de secuencia}
      fin_si
  fin-mientras
  cerrar (F,F1,F2)
fin
```

10.3. Aplicar el algoritmo de mezcla directa al archivo F de claves
F: 19 27 2 8 36 5 20 15 6

1. Primera división (partición)
   F1: 19 2 36 20 6
   F2: 27 8 5 15

2. Mezcla de F1 y F2, formando pares ordenados
   F: 19 27 || 2 8 || 5 36 || 15 20 || 6

3. Segunda división (participación en secuencias de longitud 2)
   F1: 19 27 || 5 36 || 6
   F2: 2 8 || 15 20

4. Mezcla de F1 y F2
   F: 2 8 19 27 || 5 15 20 36 || 6

5. Tercera división
   F1: 2 8 19 27 || 6
   F2: 5 15 20 36

6. F1: 2 5 8 15 19 20 27 36
   F2: 6

7. Mezcla de F1 y F2 (última)
   F: 2 5 6 8 15 19 20 27 37

10.4. Escribir el algoritmo de mezcla de los dos archivos ordenados.

Archivos de entrada: Archivo X de registros con claves en orden creciente.
Archivo Y de registros con claves en orden creciente.
Archivos de salida: Archivo Z con todos los registros de X e Y con claves en orden creciente.

```
algoritmo Fusion
inicio { Marcas FA no asociadas al ultimo registro }
  leer (registro x de X, registro y de Y)
    mientras X e Y contengan registros hacer
       si (clave de x) < (clave de y)
         entonces
             escribir (registro x en el archivo z)
             leer (registro x de X)
         sino
             escribir (registro y del archivo z)
             leer (registro y de Y)
      fin_si
    fin-mientras
    si archivo x es el que esta vacio
       entonces transferir resto de Y a Z
       sino transferir resto de X a Z
    fin_si
fin
```

10.5. Escribir el algoritmo de ordenación por mezcla de un archivo desordenado con n registros.

Entrada: Archivo desordenado R de n registros, cada uno con una clave identificadora.
Salida: Un archivo F3, de n registros con claves en orden creciente. Dos archivos intermedios F1 y F2 contienen las secuencias.

a) Ordenar en secuencias:

```
Elegir un numero K (el tamaño de la secuencia inicial)
repetir
  leer (K registros del archivo R)
  Ordenar en una secuencia (de longitud k) por un metodo de ordenacion interna.
hasta_que R este vacio
De modo alternativo escribir la secuencia sobre F1 y F2.
```

b) Mezcla:

```
repetir
  Para cada par de secuencias, una de F1 y otra de F2, mezclar las dos secuen-
  cias en una secuencia de doble longitud y escribir la secuencia en F3.
hasta_que F3 contenga solo una secuencia de longitud n distribuir las secuencias
  de F3 alternativamente sobre F1 y F2.
```

10.6. Dos sucursales, A y B, de una misma empresa tienen cada una un archivo con los artículos que tienen en stock. FICHA contiene NA registros y FICHB contiene NB registros.

Cada registro comprende el código de un artículo y la cantidad disponible en stock para la sucursal correspondiente. Las dos sucursales utilizan los mismos códigos para los mismos artículos, pero no tienen obligatoriamente los mismos artículos en stock.
Los dos archivos están dispuestos en orden creciente de códigos de artículos.
Las dos sucursales deciden fusionar su stock.

Escribir un algoritmo que permita sustituir FICHA y FICHB por un archivo único, FUSION, que tenga las características siguientes:

- El archivo FUSION está dispuesto en orden creciente de códigos de artículos.
- Cada registro comprenderá el código de un artículo y la cantidad disponible en stock en el conjunto de las dos sucursales.

Análisis del problema

Archivo A: (FICHA)	Codigo RA Cantidad Q1	Archivo B: (FICHB)	Codigo RB Cantidad QB

- RA = RB: artículo en stock en las dos sucursales en el archivo FUSION con el código RA y la cantidad $Q_A + Q_B$
- RA > R_b: artículos diferentes; es preciso escribir en el archivo FUSION con la referencia RB y la cantidad en stock QB
- $R_B > R_A$: intercambiar los papeles de A y B

Se introducen dos variables lógicas:

SWA: igual a verdadero cuando se hayan leído todos los registros de FICHA
SWB: igual a verdadero cuando se hayan leído todos los registros de FICHB

Primer refinamiento del algoritmo

- Leer número de artículos del archivo FICHA: NA
- Leer número de artículos del archivo FICHB: NB
- Leer el primer registro del archivo FICHA: R_A y Q_A
- Leer el primer registro del archivo FICHB: RB y QB

```
repetir
 contador de articulos del archivo FUSION
 si RA = RB
   entonces  escribir RA y QA + QB en el archivo FUSION
      si FICHA no se ha terminado
        entonces leer siguiente registro de FICHA
        sino salir del bucle
      fin_si
      si FICHB no se ha terminado
        entonces leer el registro siguiente de FICHB
        sino salir del bucle
      fin_si
 sino si RA > RB
   entonces  escribir RB y QB en el archivo FUSION
      si FICHB no se ha terminado
        entonces leer el registro siguiente de FICHB
        sino salir del bucle
      fin_si
   sino escribir RA y QA en el archivo FUSION
      si FICHA no se ha terminado
        entonces leer el registro siguiente de FICHA
        sino salir del bucle
      fin_si
 fin_si
 hasta_que indicacion de salida de bucle

si FICHA no se ha terminado de leer, copiar en FUSION el resto de FICHA
si FICHB no se ha terminado de leer, copiar en FUSION el resto de FICHB
escribir (el numero de articulos del archivo FUSION)
```

Tabla de variables

NA : número de artículos del archivo FICHA
NB : número de artículos del archivo FICHB
J : contador de número de artículos leídos en el archivo FICHB
K : contador de número de artículos leídos en el archivo FICHA
N : contador de número de artículos del archivo FUSION
RA : código del artículo leído en el archivo FICHA
QA : cantidad en stock leída en el archivo FICHB
RB : código del artículo leído en el archivo FICHA
QB : cantidad en stock leída en el archivo FICHB

Algoritmo

```
algoritmo Mezcla
var J,K,N,NA,NB,RB,RA,QB,QB,R,Q: entero
    SW1, SW2: boolean

inicio
  leer (NA y NB)
  leer (RA, QA en FICHA)
  leer (RB,AB en FICHB)
  J ← 1
  K ← 1
  N ← 0
  SW1 ← falso
  SW ← falso
repetir
     N ← N + 1
     si RA = RB
       entonces escribir (RA, Q_A + Q_B en FUSION)
           si J < NA
              entonces leer (R_A, Q_A en FICHA)
              sino SW1 ← verdadero
           si K < NB
              entonces leer (R_A, Q_A en FICHB)
              sino SW2 ← verdadero
       sino
         si R_A > R_B
            entonces escribir (R_B,Q_B en FUSION)
                     si k < N_B entonces leer (R_B,Q_B en FICHB)
                        K ← K + 1
                     sino
                        SW2 ← verdadero
            sino escribir (R_A,Q_A en FUSION)
                     si < J NA entonces leer (R_A,Q_A en FICHA)
                        J ← J + 1
                     sino
                        SW2 ← verdadero
       hasta_que SW1 = verdadero o SW2 = falso
       si SW1 = falso
            entonces escribir (FUSION, RA,QA))
                     repetir
                        leer (FICHA,RA,QA)
                        escribir (FUSION,RA,QA)
                        N ← N + 1
                        J ← J + 1
                     hasta_que J > NA
       fin_si
```

```
        si SW2 = falso
             entonces (FUSION,RB,QB)
               repetir
                 leer (FICHB, R_B, Q_B)
                 escribir (FUSION,RB,QB)
                 N ← NH
                 K ← K + 1
               hasta_que K > NB
        fin_si
        escribir (N)
fin
```

10.7. Describir el método de ordenación por mezcla de secuencias equilibradas para el archivo de claves:

44 66 4 12 7 5 34 32 68 8 99 16 13 14 12 10

Archivo de entrada: F *Archivos de trabajo:* F1, F2, F3, F4

44 66 4 12 7 5 34 32 68 8 99 16 13 14 12 10

$$F1 \begin{bmatrix} 4 & 12 & 46 & 66 \\ 5 & 7 & 32 & 34 \end{bmatrix} \quad \begin{bmatrix} 8 & 16 & 68 & 99 \\ 10 & 12 & 13 & 14 \end{bmatrix}$$

F2

F3 vacío
F4 vacío

La fusión por bloques o secuencias es:

F1 vacío
F2 vacío

F3 4 5 7 12 32 34 46 66 || F4 8 10 12 13 14 16 68 99

La mezcla o fusión final:

F1 4 5 7 8 10 12 12 13 14 16 32 34 46 66 68 99
F2 vacío
F3 vacío
F4 vacío

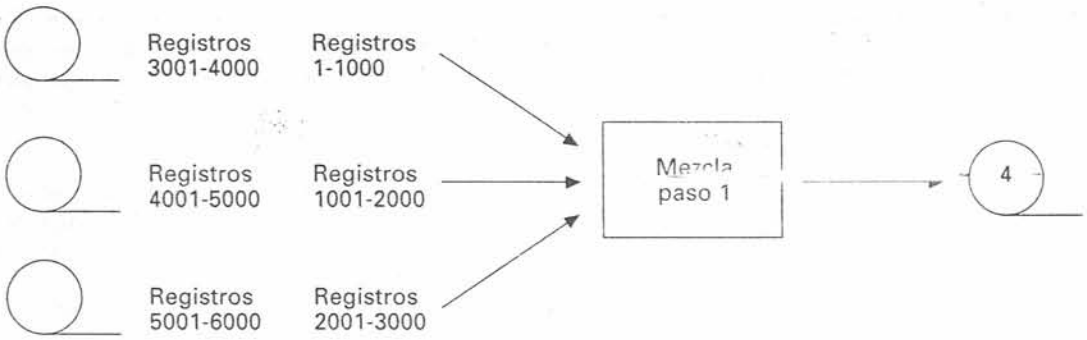
10.8. Representar gráficamente el proceso de ordenación de un archivo de 6.000 registros por el método de mezcla natural de tres caminos.

Entrada: Tres archivos *Salida:* 1 archivo

Fases:

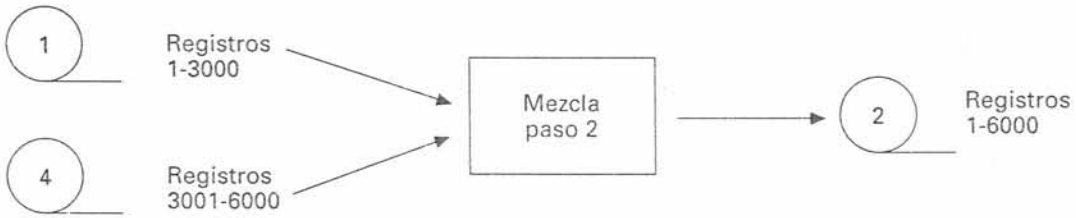
(1) Distribuir los seis archivos en dos series de tres archivos cada una. La ordenación interna de los archivos se puede realizar simultáneamente si se desea.

(2) Mecla, paso 1



(3) Distribución de las series de archivos (4) en 3 archivos (en ese ejemplo sólo se utilizan 2).

(4) Mezcla, paso 2



EJERCICIOS

10.1. Se desea intercalar los registros del archivo `P` con los registros del archivo `Q` y grabarlos en otro archivo `R`.

NOTA: Los archivos `P` y `Q` están clasificados en orden ascendente por una determinada clave y se desea que el archivo `R` quede también ordenado en modo ascendente.

10.2. Los archivos `M`, `N` y `P` contienen todas las operaciones de ventas de una empresa en los años 1985, 1986 y 1987 respectivamente. Se desea un algoritmo que intercale los registros de los tres archivos en un solo archivo `Z`, teniendo en cuenta que los tres archivos están clasificados en orden ascendente por el campo clave ventas.

10.3. Se dispone de dos archivos secuenciales `F1` y `F2` que contienen cada uno de ellos los mismos campos. Los dos archivos están ordenados de modo ascendente por el campo clave (alfanumérico) y existen registros comunes a ambos archivos. Se desea diseñar un programa que obtenga: a) un archivo `C` a partir de `F1` y `F2`, que contenga todos los registros comunes, pero sólo una vez; b) un archivo que contenga todos los registros que no son comunes a `F1` y `F2`.

10.4. Se desea intercalar los registros del archivo `A` con los registros del archivo `B` y grabarlos en un tercer archivo `C`. Los archivos `A` y `B` están clasificados en orden ascendente por su campo clave. Y se desea también que el archivo `C` quede clasificado en orden ascendente.

10.5. El archivo `A` contiene los números de socios del Club Deportivo Esmeralda, y el archivo `B` los códigos de los socios del Club Deportivo Diamante. Se desea crear un archivo `C` que contenga los números de los

socios que pertenecen a ambos clubes. Así mismo, se desea saber cuántos registros se han leído y cuántos se han grabado.

10.6. Los archivos F1, F2 y F3 contienen todas las operaciones de ventas de una compañía informática los años 1985, 1986 y 1987, respectivamente. Se desea un programa que intercale todos los registros de los tres archivos en un solo archivo F, suponiendo que todo registro posee un campo clave y que F1, F2 y F3 están clasificados en orden ascendente de ese campo clave.

10.7. Se desea actualizar un archivo maestro de la nómina de la compañía Aguas del Pacífico con un archivo MODIFICACIONES que contiene todas las incidencias de empleados (altas, bajas, modificaciones). Ambos archivos están clasificados en orden ascendente del código de empleado (campo clave). El nuevo archivo maestro actualizado debe conservar la clasificación ascendente por código de empleado y sólo debe existir un registro por empleado.

10.8. Se tiene un archivo maestro de inventarios con los siguientes campos:

```
CODIGO DE ARTICULO    DESCRIPCION    EXISTENCIAS
```

Se desea actualizar el archivo maestro con los movimientos habidos durante el mes (altas/bajas). Para ello se incluyen los movimientos en un archivo OPERACIONES, que contiene los siguientes campos:

```
CODIGO DE ARTICULO    CANTIDAD    OPERACION (1-Alta, 2-Baja)
```

Los dos archivos están clasificados por el mismo campo clave.

CAPÍTULO 11

ESTRUCTURAS DINÁMICAS LINEALES DE DATOS (PILAS, COLAS Y LISTAS ENLAZADAS)

CONTENIDO

11.1. Introducción a las estructuras lineales de datos.
11.2. Listas.
11.3. Listas enlazadas.
11.4. Procesamiento de listas enlazadas.
11.5. Listas circulares.
11.6. Listas doblemente enlazadas.
11.7. Pilas.
11.8. Colas.
11.9. Doble cola.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

Los datos estudiados hasta ahora se denominan *estáticos*. Ello es debido a que las variables son direcciones simbólicas de posiciones de memoria; esta relación entre nombres de variables y posiciones de memoria es una relación estática que se establece por la declaración de las variables de una unidad de programa y que se fija durante la ejecución de esa unidad. Aunque el contenido de una posición de memoria asociada con una variable puede cambiar durante la ejecución, es decir, el valor de la variable puede cambiar, las variables por sí mismas no se pueden crear ni destruir durante la ejecución. En consecuencia, las variables consideradas hasta este punto se denominan *variables estáticas*.

En algunas ocasiones, sin embargo, no se conoce por adelantado cuánta memoria se requerirá para un programa. En esos casos es conveniente disponer de un método para adquirir posiciones adicionales de memoria a medida que se necesiten durante la ejecución del programa y liberarlas cuando no se necesitan. Las variables que se crean y están disponibles durante la ejecución de un programa se llaman *variables conti-*

nuas. Estas variables se representan con un tipo de datos conocido como *puntero*. Las variables dinámicas se utilizan para crear *estructuras dinámicas de datos* que se pueden ampliar y comprimir a medida que se requieran durante la ejecución del programa. Una estructura de datos dinámica es una colección de elementos denominados nodos de la estructura —normalmente de tipo registro— que son enlazados juntos. Las estructuras dinámicas de datos se clasifican en lineales y no lineales. El estudio de las estructuras lineales, listas, pilas y colas, es el objetivo de este capítulo.

11.1. INTRODUCCIÓN A LAS ESTRUCTURAS LINEALES DE DATOS

En capítulos anteriores se ha introducido a las estructuras de datos, definiendo tipos y estructuras de datos primitivos tales como *enteros*, *real* y *carácter*, utilizados para construir tipos más complicados como *arrays* y *registros*, denominados estructuras de datos compuestos. Tienen una estructura porque sus datos están relacionados entre sí. Las estructuras compuestas, tales como *arrays* y registros están soportadas en la mayoría de los lenguajes de programación, debido a que son necesarias en casi todas las aplicaciones.

La potencia y flexibilidad de un lenguaje están directamente relacionadas con las estructuras de datos que posee. La programación de algoritmos complicados puede resultar muy difícil en un lenguaje con estructuras de datos limitados, caso de BASIC, FORTRAN y COBOL.

Cuando una aplicación particular requiere una estructura de datos no soportada por el lenguaje, se hace necesaria una labor de programación para representarla. Se dice que necesitamos *implementar* la estructura de datos. Esto naturalmente significa más trabajo para el programador. Si la programación no se hace bien, se puede malgastar tiempo de programación y —naturalmente— de computadora. Por ejemplo, supongamos que tenemos un lenguaje que permite arrays de una dimensión de números enteros y reales, pero no arrays multidimensionales. Para implementar una tabla con cinco filas y diez columnas podemos utilizar

```
tipo array [1..10] de real: FILA
var
FILA: FILA1, FILA2, FILA3, FILA4, FILA5
```

La llamada al elemento de la tercera fila y sexta columna se realizará con la instrucción

```
FILA3 [6]
```

Un método muy eficaz es diseñar procedimientos y funciones que ejecuten las operaciones realizadas por las estructuras de datos. Sin embargo, con las estructuras vistas hasta ahora, arrays y registros, tienen dos inconvenientes: 1) *la reorganización de una lista si ésta implica movimiento de muchos elementos de datos puede ser muy costosa*, y 2) *son estructuras de datos estáticas*.

Una estructura de datos se dice que es *estática* cuando el tamaño ocupado en memoria es fijo, es decir, siempre ocupa la misma cantidad de espacio en memoria. Por consiguiente, si se representa una lista como vector, se debe anticipar (*declarar* o *dimensionar*) la longitud de esa lista cuando se escribe un programa; es imposible ampliar el espacio de memoria disponible (algunos len-

guajes permiten dimensionar dinámicamente el tamaño de un array durante la ejecución del programa, como es el caso de BASIC). En consecuencia, puede resultar difícil diferenciar estructuras de datos.

Los arrays unidimensionales son estructuras estáticas lineales ordenadas secuencialmente. Las estructuras se convierten en dinámicas cuando los elementos pueden ser insertados o suprimidos directamente sin necesidad de algoritmos complejos. Se distinguen las estructuras dinámicas de las estáticas por los modos en que se realizan las inserciones y borrados de elementos.

11.1.1. Estructuras dinámicas de datos

Las estructuras dinámicas de datos son estructuras que «crecen a medida que se ejecuta un programa». Una *estructura dinámica de datos* es una colección de elementos —llamados *nodos*— que son normalmente registros. Al contrario que un array, que contiene espacio para almacenar un número fijo de elementos, una estructura dinámica de datos se amplía y contrae durante la ejecución del programa, basada en los registros de almacenamiento de datos del programa.

Las estructuras dinámicas de datos se pueden dividir en dos grandes grupos:

lineales { pilas, colas, listas enlazadas

no lineales { árboles, grafos

Las estructuras dinámicas de datos se utilizan para almacenamiento de datos del mundo real, que están cambiando constantemente. Un ejemplo típico ya lo hemos visto como estructura estática de datos: la lista de pasajeros de una línea aérea. Si esta lista se mantuviera en orden alfabético en un array, sería necesario hacer espacio para insertar un nuevo pasajero por orden alfabético. Esto requiere utilizar un bucle para copiar los datos del registro de cada pasajero en el siguiente elemento del array. Si en su lugar se utilizara una estructura dinámica de datos, los nuevos datos del pasajero se pueden insertar simplemente entre dos registros existentes con un mínimo esfuerzo.

Las estructuras dinámicas de datos son extremadamente flexibles. Como se ha descrito anteriormente, es relativamente fácil añadir nueva información creando un nuevo nodo e insertándolo entre nodos existentes. Se verá que es también relativamente fácil modificar estructuras dinámicas de datos, eliminando o borrando un nodo existente.

En este capítulo examinaremos las tres estructuras dinámicas lineales de datos: listas, colas y pilas, dejando para el próximo capítulo las estructuras no lineales de datos: árboles y grafos.

> Una *estructura estática de datos* es aquélla cuya estructura se especifica en el momento en que se escribe el programa y no puede ser modificada por el programa. Los valores de sus diferentes elementos pueden variar, pero no su estructura, ya que ésta es fija.
>
> Una *estructura dinámica de datos* puede modificar su estructura mediante el programa. Puede ampliar o limitar su tamaño mientras se ejecuta el programa.

11.2. LISTAS

Una *lista lineal* es un conjunto de elementos de un tipo dado que se encuentran ordenados y pueden variar en número. Ésta es una definición muy general, que incluye los ficheros y vectores.

Los elementos de una lista lineal se almacenan normalmente contiguos —un elemento detrás de otro— en posiciones consecutivas de la memoria. Las sucesivas entradas en una guía o directorio telefónico, por ejemplo, están en líneas sucesivas, excepto en las partes superior e inferior de cada columna. Una lista lineal se almacena en la memoria principal de una computadora en posiciones sucesivas de memoria; cuando se almacenan en cinta magnética, los elementos sucesivos se presentan en sucesión en la cinta. Esta asignación de memoria se denomina *almacenamiento secuencial*. Posteriormente, se verá que existe otro tipo de almacenamiento denominado *encadenado* o *enlazado*.

Las líneas así definidas se denominan *contiguas*. Las operaciones que se pueden realizar con listas lineales contiguas son:

1. Insertar, eliminar o localizar un elemento.
2. Determinar el tamaño —número de elementos— de la lista.
3. Recorrer la lista para localizar un determinado elemento.
4. Clasificar los elementos de la lista en orden ascendente o descendente.
5. Unir dos o más listas en una sola.
6. Dividir una lista en varias sublistas.
7. Copiar la lista.
8. Borrar la lista.

Una lista lineal se almacena en la memoria de la computadora en posiciones sucesivas o adyacentes y se procesa como un array unidimensional. En este caso, el acceso a cualquier elemento de la lista y la adición de nuevos elementos es fácil; sin embargo, la inserción o borrado requiere un desplazamiento de lugar de los elementos que le siguen y, en consecuencia, el diseño de un algoritmo específico.

Para permitir operaciones con listas como arrays se deben dimensionar éstos con tamaño suficiente para que contengan todos los posibles elementos de la lista.

Ejemplo 11.1

Se desea leer el elemento j-ésimo de una lista `P`.

El algoritmo requiere conocer el número de elementos de la lista (su longitud, `L`). Los pasos a dar son:

1. **conocer** longitud de la lista `L`,
2. **si** `L = 0` visualizar 'error lista vacía'
   si_no comprobar si el elemento *j*-ésimo está dentro del rango permitido de elementos `1 <= j <= L`; en este caso, asignar el valor del elemento `P(j)` a una variable `B`; si el elemento *j*-ésimo no está dentro del rango visualizar un mensaje de error 'elemento solicitado no existe en la lista',
3. **fin**.

El pseudocódigo correspondiente sería:

```
procedimiento acceso(E lista: P; S elementolista: B; E entero: L, J)
inicio
  si L = 0 entonces
    escribir('Lista vacia')
  si_no
    si (j >= 1) y (j <= L) entonces
      B ← P[j]
    si_no
      escribir('ERROR: elemento no existente')
    fin_si
  fin_si
fin
```

Ejemplo 11.2

Borrar un elemento `j` *de la lista* `P`.

Variables

```
L     longitud de la lista
J     posicion del elemento a borrar
I     subindice del array P
P     lista
```

Las operaciones necesarias son:

1. Comprobar si la lista es vacía.
2. Comprobar si el valor de `J` está en el rango del subíndice I de la lista `1 <= J <= L`.
3. En caso de `J` correcto, mover los elementos `j+1, j+2, . . .`, a las posiciones `j, j+1, . . .`, respectivamente, con lo que se habrá borrado el antiguo elemento `j`.
4. Decrementar en uno el valor de la variable `L`, ya que la lista contendrá ahora `L - 1` elementos.

El algoritmo correspondiente será:

```
inicio
  si L = 0 entonces
    escribir('lista vacia')
  si_no
    leer(J)
    si (J >= 1) y (J <= L) entonces
      desde I ← J hasta L - 1 hacer
        P(I) ← P(I + 1)
      fin_desde
        L ← L - 1
    si_no
        escribir('Elemento no existe')
    fin_si
  fin_si
fin
```

Una *lista contigua* es aquélla en que los elementos son adyacentes en la memoria o soporte direccionable. Tiene unos límites izquierdo y derecho o inferior/superior que no pueden ser rebasados cuando se le añade un elemento.

La inserción o eliminación de un elemento, excepto en la cabecera o final de la lista, necesita una traslación de una parte de los elementos de la misma: la que precede o sigue a la posición del elemento modificado.

Las operaciones directas de añadir y eliminar se efectúan únicamente en los extremos de la lista. Esta limitación es una de las razones por las que esta estructura es poco utilizada.

Las *listas enlazadas* o de almacenamiento enlazado o encadenado son mucho más flexibles y potentes, y su uso es mucho más amplio que las listas contiguas.

11.3. LISTAS ENLAZADAS

Los inconvenientes de las listas contiguas se eliminan con las listas enlazadas. Se pueden almacenar los elementos de una lista lineal en posiciones de memoria que no sean contiguas o adyacentes.

Una *lista enlazada* o *encadenada* es un conjunto de elementos en los que cada elemento contiene la posición —o dirección— del siguiente elemento de la lista. Cada elemento de la lista enlazada debe tener al menos dos campos: un campo que tiene el valor del elemento y un campo (enlace, *link*) que contiene la posición del siguiente elemento, es decir, su conexión, enlace o encadenamiento. Los elementos de una lista son enlazados por medio de los campos enlaces.

Las listas enlazadas tienen una terminología propia que se suele utilizar normalmente. Primero, los valores se almacenan en un *nodo* (Figura 11.1).

Figura 11.1. Nodo con dos campos.

Una lista enlazada se muestra en la Figura 11.2.

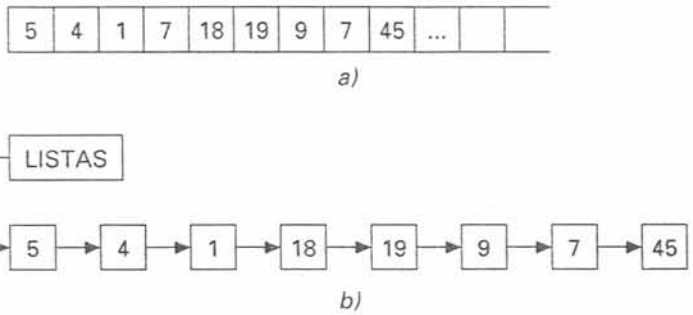


Figura 11.2. (a) Array representado por una lista; (b) lista enlazada representada por una lista de enteros.

Los componentes de un nodo se llaman *campos*. Un nodo tiene al menos un campo *dato* o *valor* y un *enlace* (indicador o puntero) con el siguiente campo. El campo enlace apunta (proporciona la dirección de) al siguiente nodo de la lista. El último nodo de la lista enlazada, por convenio, se suele representar por un enlace con la palabra reservada `nil` (*nulo*), una barra inclinada (/) y, en ocasiones, el símbolo eléctrico de tierra o masa (Figura 11.3).

Figura 11.3. Representación del último nodo de una lista.

La implementación de una lista enlazada depende del lenguaje. Pascal, C, C++, Ada y Modula-2 utilizan como enlace una *variable puntero*, o *puntero* simplemente. Los lenguajes como FORTRAN y BASIC no disponen de este tipo de datos y se debe simular con una variable entera que actúa como indicador o cursor. En nuestro libro utilizaremos a partir de ahora el término *puntero* para describir el enlace entre dos elementos o nodos de una lista enlazada.

Un **puntero** (*apuntador*)[1] es una variable cuyo valor es la dirección o posición de otra variable.

En las listas enlazadas no es necesario que los elementos de la lista sean almacenados en posiciones físicas adyacentes, ya que el puntero indica dónde se encuentra el siguiente elemento de la lista, tal como se indica en la Figura 11.4.

Por consiguiente, la inserción y borrado no exigen desplazamiento como en el caso de las listas contiguas.

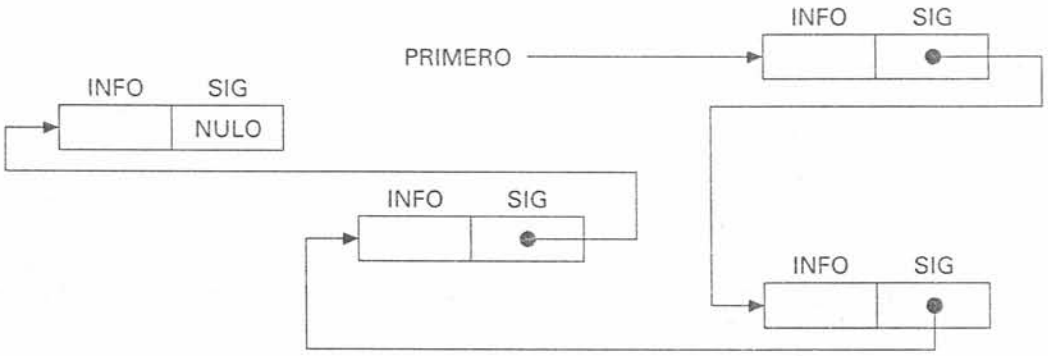


Figura 11.4. Elementos no adyacentes de una lista enlazada.

Para eliminar el 45° elemento (`'INES'`) de una lista lineal con 2.500 elementos (Figura 11.5 (a)) sólo es necesario cambiar el puntero en el elemento anterior, 44°, y que apunte ahora al elemento 46° (Figura 11.5 (b)). Para insertar un nuevo elemento (`'HIGINIO'`) después del 43° elemento, es necesario cambiar el puntero del elemento 43° y hacer que el nuevo elemento apunte al elemento 44° (Figura 11.5 (c)).

[1] En Latinoamérica, el término *puntero* (en inglés, *pointer*) se suele traducir por **apuntador**.

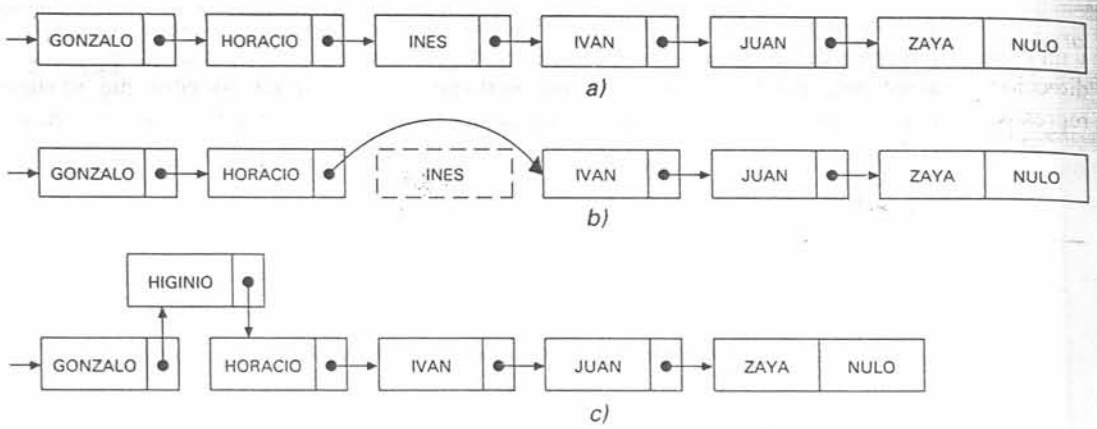


Figura 11.5. Inserción y borrado de elementos.

Una lista enlazada sin ningún elemento se llama *lista vacía*. Su puntero inicial o de cabecera tiene el valor nulo (`nil`).

Una lista enlazada se define por:

- el tipo de sus elementos: campo de información (datos) y campo enlace (*puntero*),
- un puntero de cabecera que permite acceder al primer elemento de la lista,
- un medio para detectar el último elemento de la lista: *puntero nulo* (**nil**).

Ejemplo 11.3

El director de un hotel desea registrar el nombre de cada cliente a medida de su llegada al hotel, junto con el número de habitación que ocupa —el antiguo libro de entradas—. También desea disponer en cualquier momento de un lista de sus clientes por orden alfabético.

Ya que no es posible registrar los clientes alfabética y cronológicamente en la misma lista, se necesita o bien listas alfabéticas independientes o bien añadir punteros a la lista existente, con lo que sólo se utilizará una única lista. El método manual en el libro requería muchos cruces y re-escrituras; sin embargo, una computadora mediante un algoritmo adecuado lo realizará fácilmente.

Por cada nodo de la lista el campo de información o datos tiene dos partes: nombre del cliente y número de habitación. Si *x* es un puntero a uno de estos nodos, *l[x].nombre* y *l[x].habitación* representarán las dos partes del campo información.

El listado alfabético se consigue siguiendo el orden de los punteros de la lista (campo puntero). Se utiliza una variable `CABECERA(S)` para apuntar al primer cliente.

```
CABECERA ← 3
```

Así, `CABECERA(S)` es 3, ya que el primer cliente, `Antolin`, ocupa el lugar `3`. A su vez, el puntero asociado al nodo ocupado por `Antolin` contiene el valor `10`, que es el segundo nombre de los clientes en orden alfabético, y éste tiene como campo puntero el valor `7`, y así sucesivamente. El campo puntero del último cliente, `Tomas`, contiene el puntero nulo indicado por un `0` o bien una `Z`.

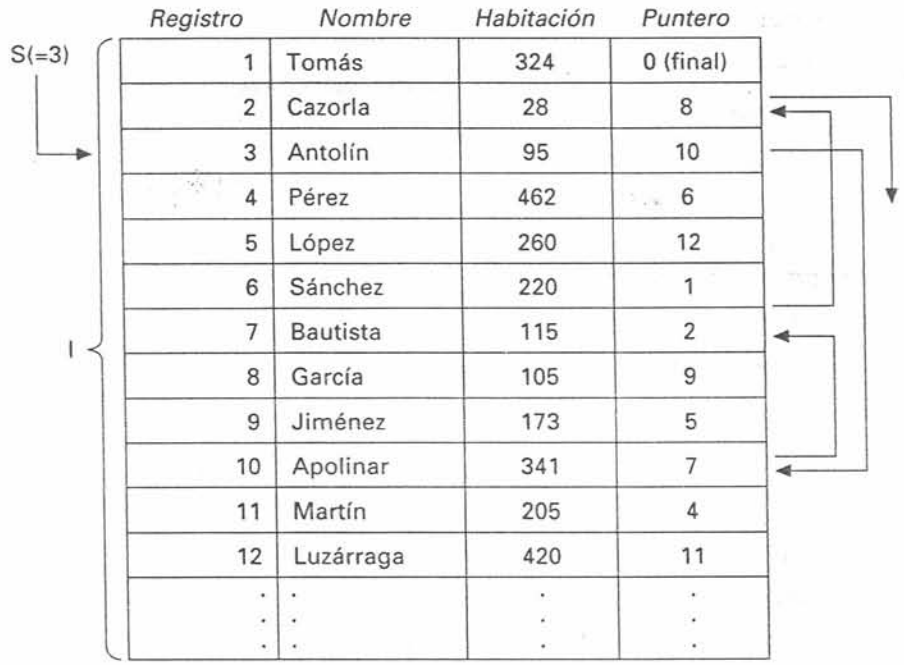


Figura 11.6. Lista enlazada clientes de un hotel.

11.4. PROCESAMIENTO DE LISTAS ENLAZADAS

Para procesar una lista enlazada son imprescindibles las siguientes informaciones:

- primer nodo (cabecera de la lista),
- el tipo de sus elementos.

Las operaciones que normalmente se ejecutan con listas incluyen:

1. Recuperar información de un nodo específico (acceso a un elemento).
2. Encontrar el nodo que contiene una información específica (localizar la posición de un elemento dado).
3. Insertar un nuevo nodo en un lugar específico de la lista.
4. Insertar un nuevo nodo en relación a una información particular.
5. Borrar (eliminar) un nodo existente que contiene información específica.

11.4.1. Implementación de listas enlazadas con punteros

Como ya hemos visto, la representación gráfica de un puntero consiste en una flecha que sale del puntero y llega a la variable dinámica apuntada.

Para declarar una variable de tipo puntero

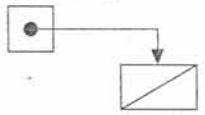
```
tipo
  puntero_a <tipo_dato>: punt
var
  punt : p, q
```

El `<tipo_dato>` podrá ser simple o estructurado.

Operaciones con punteros:

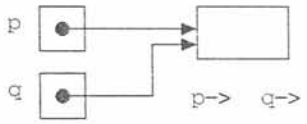
Inicialización:

`p ← nulo` A nulo para indicar que no apunta a ninguna variable



Comparación:

`p = q` Con los operadores = o <>



Asignación:

`p ← q` Implica hacer que el puntero `p` apunte a donde apuntaba `q`

Creación de variables dinámicas:

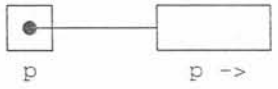
`Reservar(p)` Reservar espacio en memoria para la variable dinámica

Eliminación de variables dinámicas:

`Liberar(p)` Liberar el espacio en memoria ocupado por la variable dinámica

Variables dinámicas

Variable simple o estructura de datos sin nombre y creada en tiempo de ejecución.



Para acceder a una variable dinámica apuntada, utilizar el símbolo –>, de modo que resultará `p ->`.

Las variables `p ->` podrán intervenir en toda operación o expresión de las permitidas para una variable estática de su mismo tipo.

Nodo

Las estructuras dinámicas de datos están formadas por nodos.

Un nodo es una variable dinámica constituida por al menos dos campos:

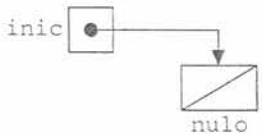
- el campo dato o valor (`elemento`),
- el campo enlace, de tipo puntero (`sig`).

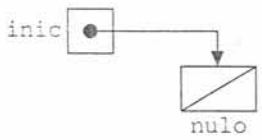
```
tipo
  registro: nodo
  <tipo_elemento>: elemento      //campo que contiene la informacion apunta al
  punt           : sig           //siguiente elemento de la estructura segun la
                                 //estructura de que se trate podra haber uno o
                                 //varios campos de tipo punt
  fin registro
```

11.4.1.1. Creación de la lista

La creación de la lista conlleva la inicialización a nulo del puntero (`inic`), que apunta al primer elemento de la lista.

```
tipo
      puntero_a nodo: punt
      registro: tipo_elemento
       ... : ...
       ... : ...
      fin registro
      registro: nodo
        inicio
          tipo_elemento : elemento
          punt          : sig
      fin registro
 var  punt           : inic
      punt           : posic
      punt           : anterior
      tipo_elemento  : elemento
      logico         : encontrado
 inicio
  inicializar(inic)
fin

procedimiento inicializar(S punt: inic)
inicio
  inic ← nulo
fin_procedimiento
```





uo que resultará

idas para una

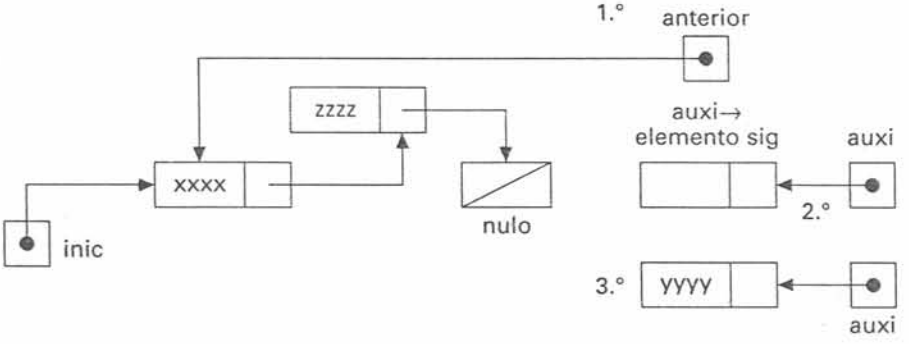
11.4.1.2. Inserción de un elemento

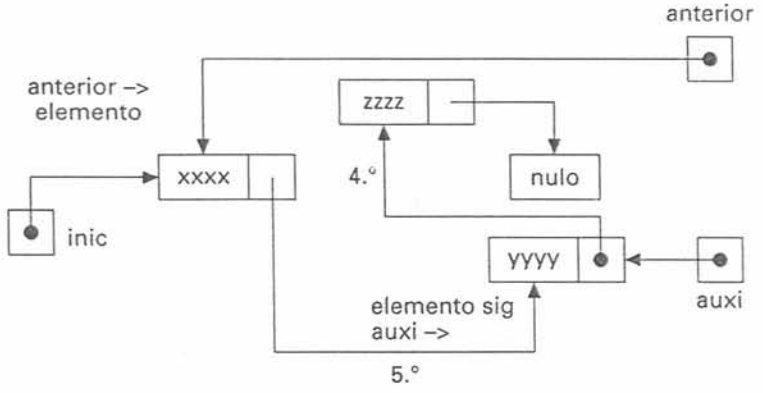
La inserción tiene dos casos particulares:

- Insertar el nuevo nodo en el frente, principio de la lista.
- Insertar el nuevo nodo en cualquier otro lugar de la lista.

El procedimiento `insertar`, inserta un nuevo elemento a continuación de `anterior`; si `anterior` fuera nulo significa que ha de insertarse al comienzo de la lista.

```
insertar(inicio,anterior,elemento)
```






```
1.°  Situacion de partida
2.°  Reservar(auxi)
3.°  Introducir la nueva informacion en auxi->elemento
4.°  Hacer que auxi->sig apunte a donde lo hacia anterior->sig
5.°  Conseguir que anterior->sig apunte a donde lo hace auxi

procedimiento insertar(E/S punt: inic,anterior  E tipo_elemento: elemento)
.................
  var punt: auxi
inicio
  reservar(auxi)
  auxi ->elemento ← (elemento)
  si anterior = nulo entonces
    auxi->sig ← (inic)
    inic ← (auxi)
```

```
  si_no
    auxi->sig ← (anterior->sig)
    anterior->sig ← (auxi)
  fin_si
  anterior ← (auxi)  // Opcional
fin_procedimiento
```

11.4.1.3. Eliminación de un elemento de una lista enlazada

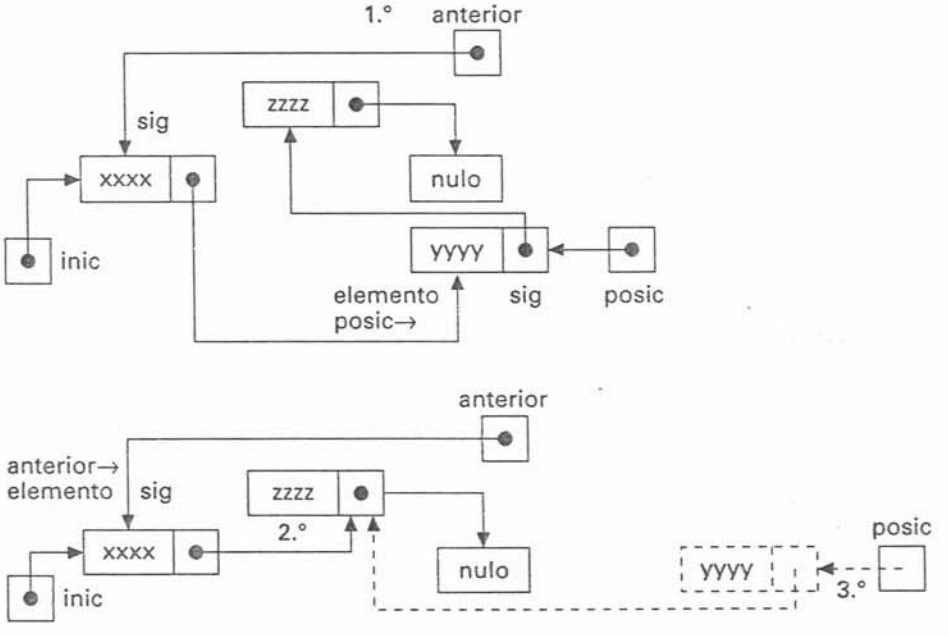
Antes de proceder a la eliminación de un elemento de la lista, deberemos comprobar que no está vacía. Para lo que podremos recurrir a la función vacía.

```
logico funcion vacia(E punt: inic)
inicio
  si inic = nulo entonces
    devolver(verdad)
  si_no
    devolver(falso)
  fin_si
fin_funcion
```

Al suprimir un elemento de una lista consideraremos dos casos particulares:

- el elemento a suprimir está al principio de la lista,
- el elemento se encuentra en cualquier otro lugar de la lista.

Suprimir **(inic,anterior,posic)**




```
1.° Situacion de partida
2.° anterior ->sig apunta a donde posic ->sig
3.° liberar(posic)
```

```
procedimiento suprimir(E/S punt: inic,anterior,posic)
inicio
  si anterior = nulo entonces
    inic ← (posic ->sig)
  si_no
    anterior ->sig ← (posic ->sig)
  fin_si
  liberar(posic)
  anterior ← (nulo) // Opcional
  posic ← (inic)    // Opcional
fin_procedimiento
```

11.4.1.4. Recorrido de una lista enlazada

Para recorrer la lista utilizaremos una variable de tipo puntero auxiliar.

```
procedimiento recorrer(E punt:inic)
  var punt: posic
inicio
  posic ← (inic)
  mientras posic <> nulo hacer
    llamar_a proc_escribir(posic ->elemento)
    posic ← posic ->sig
  fin_mientras
fin_procedimiento
```

Ejemplo 11.4

Cálculo del número de elementos de una lista enlazada.

```
procedimiento contar(E punt: primero; S entero: n)
  var punt: p
inicio
  n ← 0                              //contador de elementos
  p ← (primero)
  mientras p <> nulo hacer
    n ← n + 1
    p ← (p ->sig)
  fin_mientras
fin_procedimiento
```

11.4.1.5. Acceso a un elemento de una lista enlazada

La búsqueda de una información en una lista simplemente enlazada sólo puede hacerse mediante un proceso secuencial o recorrido de la lista elemento a elemento, hasta encontrar la información buscada o detectar el final de la lista.

```
procedimiento consultar(E punt: inic;  S punt: pos,anterior;  E tipo_elemento:
                        elemento;  S logico: encontrado)
inicio
  anterior ← nulo
  pos ← (inic)
  mientras no igual(pos->elemento,elemento)y (pos<>nulo) hacer
```

```
  //igual es una funcion que compara los elementos que le pasamos como parametros,
  //recurrimos a ella porque,si se tratara de registros, compararíamos unicamente
  //la informacion almacenada en un determinado campo
    anterior ← (pos)
    pos ← (pos->sig)
  fin_mientras
  si igual(pos->elemento,elemento) entonces
    encontrado ← verdad
  si_no
    encontrado ← falso
  fin_si
fin_procedimiento
```

Ejemplo 11.5

Encontrar el nodo de una lista que contiene la información de valor `t`*, suponiendo que la lista almacena datos de tipo entero.*

```
procedimiento encontrar(E punt: primero;  E entero: t)
var punt  : p
    entero: n
inicio
  n ← 0
  p ← primero
  mientras (p->info <> t) y (p <> nulo) hacer
    n ← n + 1
    p ← (p ->sig)
  fin_mientras
  si p->info = t entonces
    escribir('Se encuentra en el nodo ',n,'de la lista')
  si_no
    escribir('No encontrado')
  fin_si
fin_procedimiento
```

Considerar que la información se encuentra almacenada en la lista de forma ordenada, orden creciente, y mejorar la eficacia del algoritmo anterior.

```
procedimiento encontrar(E punt: primero;  E entero: t)
  var punt  : p
      entero: n
inicio
  n ← 0
  p ← (primero)
  mientras (p->info < t) y (p <> nulo) hacer
    n ← n + 1
    p ← (p->sig)
  fin_mientras
  si p ->info = t entonces
    escribir('Se encuentra en el nodo ',n,' de la lista')
  si_no
    escribir('No encontrado')
  fin_si
fin_procedimiento
```

11.4.2. Implementación de listas enlazadas con arrays

Las listas enlazadas deberán implementarse de forma dinámica, pero si el lenguaje no lo permite, lo realizaremos a través de arrays, con lo cual impondremos limitaciones en cuanto al número de elementos que podrá contener la lista y estableceremos una ocupación en memoria constante.

Los nodos podrán almacenarse en arrays paralelos o arrays de registros.

Cuando se empleen arrays de registros, el valor (dato o información) del nodo se almacenará en un campo y el enlace con el siguiente elemento se almacenará en otro.

Otra posible implementación, como ya se ha dicho antes, es con dos arrays: uno para los datos y otro para el enlace.

Un valor de puntero `0`, o bien `Z`, indica el final de la lista.

	ELEMENTO	SIG
1	xxxxxxxxxxxxx	2
2	xxxxxxxxxxxxx	4
3		
4	xxxxxxxxxxxxx	6
5		
6	xxxxxxxxxxxxx	0

	ELEMENTO	SIG
1	xxxxxxxxxxxxx	2
2	xxxxxxxxxxxxx	4
3		
4	xxxxxxxxxxxxx	6
5		
6	xxxxxxxxxxxxx	0

En la definición de una lista se debe especificar la variable que apunta al primer nodo(cabecera), que en nuestro caso denominaremos `inic`:

```
inic ← 1
```

M

1	xxxxxxxxxxxxx	3
2	xxxxxxxxxxxxx	4
3	xxxxxxxxxxxxx	2
4	xxxxxxxxxxxxx	6
5		
6	xxxxxxxxxxxxx	0

Para insertar un nuevo elemento, que siga a `m[1]` y sea seguido por `m[2]`, lo único que se hará es modificar los punteros.

Como el nuevo elemento se coloca en la primera posición libre deberemos tener un puntero vacío que apunte a dicha primera posición libre.

Es decir, utilizaremos el array para almacenar dos listas, la lista de elementos y la lista de vacíos.

Es pues necesario, al comenzar a trabajar, crear la lista de vacíos de la forma que a continuación se expone:

- `vacio` apunta al primer registro libre,
- en el campo `sig` de cada registro se almacena información sobre el siguiente registro disponible,
- cuando lleguemos al último registro libre, su campo `sig` recibirá el valor `0`, para indicar que ya no quedan más registros disponibles.

```
vacio ← 1
inic ← 0
```

	ELEMENTO	SIG
1		2
2		3
3		4
4		5
5		6
6		0

Insertar el primer elemento

```
vacio ← 2
inic ← 1
```

1	xxxxxxxxxxxxx	0
2		3
3		4
4		5
5		6
6		0

Al implementar una lista a través de arrays necesitaremos los procedimientos:

```
inicializar(...) iniciar(...) consultar(...) insertar(...) suprimir(...)
reservar(...) liberar(...)
```

y las funciones:

```
vacia(...)  llena(...)
```

El procedimiento `reservar(...)` nos proporcionará la primera posición vacía para almacenar un nuevo elemento y la eliminará de la lista de vacíos, pasando el puntero de vacío (`vacio`) a la posición siguiente. `Vacio` toma el valor del siguiente vacío de la lista.

`Liberar(...)` inserta un nuevo elemento en la lista de vacíos. Se podrían adoptar otras soluciones, pero nuestro procedimiento liberar insertará el nuevo elemento en la lista de vacíos por delante, sobrescribiendo el campo `m[posic].sig` para que apunte al que antes era el primer vacío. El puntero de inicio de los vacíos (`vacio`) lo cambiará al nuevo elemento.

11.4.2.1. Creación de la lista

Consideraremos el array como si fuera la memoria del ordenador y guardaremos en él dos listas: la lista de elementos y la de vacíos.

El primer elemento de la lista de elementos está apuntado por `inic` y, por `vacio`, el primero de la lista de vacíos.

```
const max = <expresion>
tipo
  registro: tipo_elemento
    ... : ...
    ... : ...
    fin registro
  registro: tipo_nodo
      tipo_elemento : elemento
      entero        : sig
      // actua como puntero, almacenando la posicion donde se encuentra el
      // siguiente elemento de una lista
    fin registro
  array[1..Max] de tipo_nodo: arr

var
  entero  : inic, posic, anterior, vacio
  arr     : m                  //representa la memoria de nuestra computadora
  tipo_elemento : elemento
  logico        : encontrado

inicio
  iniciar(m,vacio)
  inicializar(inic)
fin
```

Al comenzar

`vacio ← 1` Indica que el primer registro libre es `m[1]`

`inic ← 0` Inic señala que no hay elementos en la lista

	Elemento	Sig
1		2
2		3
3		4
4		5
5		6
6		0

```
procedimiento inicializar(S entero: inic)     //lista de elementos
inicio
  inic ← 0
fin_procedimiento
procedimiento iniciar(S arr: m;  S entero: vacio) //lista de vacios
  var entero: i
inicio
  vacio ← 1
```

```
desde i ← 1 hasta Max-1 hacer
  m[i].sig ← i + 1
fin_desde
m[Max].sig ← 0  //Como ya no hay mas posiciones libres a las que apuntar,
                //recibe un 0
fin_procedimiento
```

Al trabajar de esta manera conseguiremos que la inserción o borrado de un determinado elemento, *n*-ésimo, de la lista no requiera el desplazamiento de otros.

11.4.2.2. Inserción de un elemento

Al actualizar una lista se pueden presentar dos casos particulares:

- `desbordamiento` (*overflow*),
- `subdesbordamiento` o desbordamiento negativo (*underflow*).

El `desbordamiento` se produce cuando la lista está llena y la lista de espacio disponible está vacía.

El `subdesbordamiento` se produce cuando se tiene una lista vacía y se desea borrar un elemento de la misma.

Luego, para poder insertar un nuevo elemento en una lista enlazada, es necesario comprobar que se dispone de espacio libre para ello. Al insertar un nuevo elemento en la lista deberemos recurrir al procedimiento `reservar(...)` que nos proporcionará, a través de `auxi`, la primera posición vacía para almacenar en ella el nuevo elemento, eliminando dicha posición de la lista de vacíos.

Por ejemplo al insertar el primer elemento:

- `vacio` señala que la primera posición libre es la 1: `auxi ← 1`,
- el campo `sig` del registro `m[vacio]` proporciona la siguiente posición vacía y `reservar` hará que `vacio` apunte a esta nueva posición: `vacio ← 2`.

1	xxxxxxxxxxxxx	~~2~~ 0
2		3
3		4
4		5
5		6
6		0

Al insertar un segundo elemento:

- como vacio tiene el valor 2, `auxi ← 2`
- `vacio ← m[2].sig`, es decir, `vacio ← 3`

	Elemento	Sig
1	xxxxxxxxxxxxx	~~2~~ 0
2	xxxxxxxxxxxxx	~~3~~ 0
3		4
4		5
5		6
6		0

```
procedimiento reservar(S entero:auxi;E ; arr:m‖E/S entero: vacio)
inicio
  si vacio = 0 entonces
    // Memoria agotada
    auxi ← 0
  si_no
    auxi ← vacio
    vacio ← m[vacio].sig
  fin_si
fin_procedimiento
```

El procedimiento `insertar` colocará un nuevo elemento a continuación de `anterior`, si `anterior` fuera 0 significa que ha de insertarse al comienzo de la lista.

```
procedimiento insertar(E/S entero:inic,anterior‖E tipo_elemento: elemento;
                       E/S arr: m;E/S entero: vacio)
var entero: auxi
inicio
  llamar_a reservar(auxi,m,vacio)
  m[auxi].elemento ← elemento
  si anterior = 0 entonces
    m[auxi].sig ← inic
    inic ← auxi
  si_no
    m[auxi].sig ← m[anterior].sig
    m[anterior].sig ← auxi
  fin_si
  anterior ← auxi  // Opcional
  //Prepara anterior para que, si no especificamos otra cosa, la siguiente
  //insercion se realice a continuacion de la actual
fin_procedimiento
```

Consideremos la siguiente situación y analicemos el comportamiento que en ella tendrían los procedimientos `reservar` e `insertar`:

Se desea insertar un nuevo elemento en la lista a continuación del primero y la situación actual, tras sucesivas inserciones y eliminaciones, es como se muestra a continuación:

```
vacio ← 3
inic  ← 1
```

1	xxxxxxxxxxxxx	2
2	xxxxxxxxxxxxx	4
3		5
4	xxxxxxxxxxxxx	6
5		0
6	xxxxxxxxxxxxx	0

El nuevo elemento se colocará en el array en la primera posición libre y lo único que se hará es modificar los punteros.

```
reservar(...) proporciona la primera posición libre
auxi ← 3      vacio ← 5
```

1	xxxxxxxxxxxxx	3
2	xxxxxxxxxxxxx	4
3	nuevo_elemento	2
4	xxxxxxxxxxxxx	6
5		0
6	xxxxxxxxxxxxx	0

```
m[3].elemento ← nuevo_elemento
```

como queremos insertar el nuevo elemento a continuación del primero de la lista, su anterior será el apuntado por `inic`:

```
anterior ← 1
m[3].sig ← 2
m[1].sig ← 3
```

11.4.2.3. Eliminación de un elemento

Para eliminar un elemento de la lista deberemos recurrir al procedimiento `suprimir(...)`, que, a su vez, llamará al procedimiento `liberar(...)` para que inserte el elemento eliminado en la lista de vacíos.

Supongamos que se trata de eliminar el elemento marcado con `**********` y, tras sucesivas inserciones de elementos, el aspecto de nuestra lista es el siguiente:

la posición del elemento a eliminar es la 3:

```
posic ← 3
```

	Elemento	Sig
1	xxxxxxxxxxxxx	2
2	xxxxxxxxxxxxx	3
3	**********	4
4	xxxxxxxxxxxxx	0
5		6
6		0

el elemento anterior al 3 ocupa en el array la posición 2:

```
anterior ← 2
```

el primer `vacio` está en 5:

```
vacio ← 5
inic ← 1
```

Al suprimir el elemento 3 la lista quedaría:

```
m[2].sig ← 4
```

	Elemento	Sig
1	xxxxxxxxxxxxx	2
2	xxxxxxxxxxxxx	4
3	**********	5
4	xxxxxxxxxxxxx	0
5		6
6		0

mediante el procedimiento `liberar(...)` incluimos el nuevo `vacio` en la lista de vacíos:

```
m[3].sig ← 5      vacio ← 3
```

como el que se suprime no es el primer elemento de la lista, el valor de `inic` no varía:

```
inic ← 1
```

```
procedimiento liberar(E entero:pos;E/S arr:m;E/S entero:vacio)
inicio
  m[pos].sig ← vacio
  vacio ← pos
fin_procedimiento

procedimiento suprimir(E/S entero: inic,anterior,posic;
                       E/S arr: m; E/S entero: vacio)
inicio
  si anterior = 0 entonces
    inic ← m[posic].sig
  si_no
    m[anterior].sig ← m[posic].sig
  fin_si
  llamar_a liberar(posic,m,vacio)
  anterior ← 0    // Opcional
  pos ← inic      // Opcional
    // Preparan los punteros para que, si no especificamos otra cosa, la proxima
    //eliminacion se realice por el principio de la lista
fin_procedimiento
```

11.4.2.4. Recorrido de una lista

El recorrido de la lista se realizará siguiendo los punteros a partir de su primer elemento, el señalado por `inic`.

Nuestro procedimiento `recorrer(...)`, al recorrer la lista, irá mostrando por pantalla los diferentes elementos que la componen.

```
procedimiento recorrer(E entero: inic)
var entero: pos
inicio
  pos ← inic
    mientras pos <> 0 hacer
      // Recurrimos a un procedimiento, proc_escribir(...), para presentar por
      //pantalla los campos del registro pasado como parametro
    llamar_a proc_escribir(m[posic].elemento)
    pos ← m[pos].sig
  fin_mientras
fin_procedimiento
```

11.4.2.5. Búsqueda de un determinado elemento en una lista

El procedimiento `consultar` nos informará sobre si un determinado elemento se encuentra o no en la lista, la posición que ocupa dicho elemento en el array y la que ocupa el elemento anterior. Si la información se encontrara colocada en la lista de forma ordenada y creciente por el campo de búsqueda el procedimiento de consulta podría ser el siguiente:

```
procedimiento consultar(E entero:inic;S entero: posic,anterior;
                        E tipo_elemento: elemento; S logico:encontrado ; E arr: m)
inicio
  anterior ← 0
  posic ← inic  // Las funciones menor(...) e igual(...)comparan los registros por
                // un determinado campo
```

```
  mientras menor(m[posic].elemento,elemento)y(posic<>0) hacer
    anterior ← posic
    posic ← m[posic].sig
  fin_mientras
    si igual(m[posic].elemento,elemento) entonces
      encontrado ← verdad
    si_no
      encontrado ← falso
    fin_si
fin_procedimiento
```

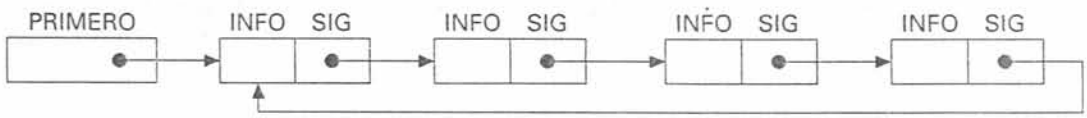
11.4.2.6. Funciones

Cuando implementamos una lista enlazada utilizando arrays, necesitamos las siguientes funciones:

```
logico funcion vacia(E entero: inic)
inicio
  si inic = 0 entonces
    devolver(verdad)
  si_no
    devolver(falso)
  fin_si
fin_funcion

logico funcion llena(E entero: vacio)
inicio
  si vacio = 0 entonces
    devolver(verdad)
  si_no
    devolver(falso)
  fin_si
fin_funcion
```

11.5. LISTAS CIRCULARES

Las listas simplemente enlazadas no permiten a partir de un elemento acceder directamente a cualquiera de los elementos que le preceden. En lugar de almacenar un puntero NULO en el campo SIG del último elemento de la lista, se hace que el último elemento apunte al primero o principio de la lista. Este tipo de estructura se llama *lista enlazada circular* o simplemente *lista circular* (en algunos textos se les denomina listas en anillo).



Las listas circulares presentan las siguientes *ventajas* respecto de las listas enlazadas simples:

- Cada nodo de una lista circular es accesible desde cualquier otro nodo de ella. Es decir, dado un nodo se puede recorrer toda la lista completa. En una lista enlazada de forma simple sólo es posible recorrerla por completo si se parte de su primer nodo.
- Las operaciones de concatenación y división de listas son más eficaces con listas circulares.

Los *inconvenientes*, por el contrario, son:

- Se pueden producir lazos o bucles infinitos. Una forma de evitar estos bucles infinitos es disponer de un nodo especial que se encuentre permanentemente asociado a la existencia de la lista circular. Este nodo se denomina *cabecera* de la lista.

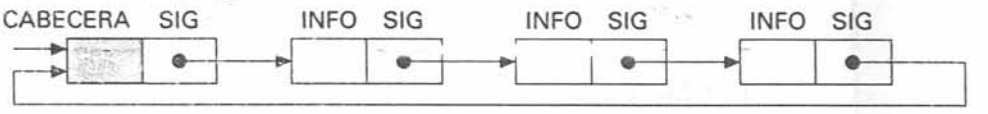


Figura 11.7. Nodo cabecera de la lista.

El nodo cabecera puede diferenciarse de los otros nodos en una de las dos formas siguientes:

- puede tener un valor especial en su campo `INFO` que no es válido como datos de otros elementos,
- puede tener un indicador o bandera (*flag*) que señale cuándo es nodo cabecera.

El campo de la información del nodo cabecera no se utiliza, lo que se señala con el sombreado de dicho campo.

Una lista enlazada circularmente vacía se representa como se muestra en la Figura 11.8.

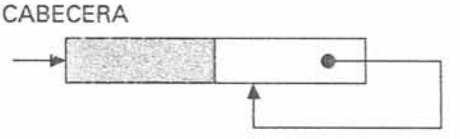


Figura 11.8. Lista circular vacía.

11.6. LISTAS DOBLEMENTE ENLAZADAS

En las listas lineales estudiadas anteriormente el recorrido de ellas sólo podía hacerse en un único sentido: *de izquierda a derecha* (principio a final). En numerosas ocasiones se necesita recorrer las listas en ambas direcciones.

Las listas que pueden recorrerse en ambas direcciones se denominan *listas doblemente enlazadas*. En estas listas cada nodo consta del campo `INFO` de datos y dos campos de enlace o punteros: `ANTERIOR(ANT)` y `SIGUIENTE(SIG)` que apuntan hacia adelante y hacia atrás:

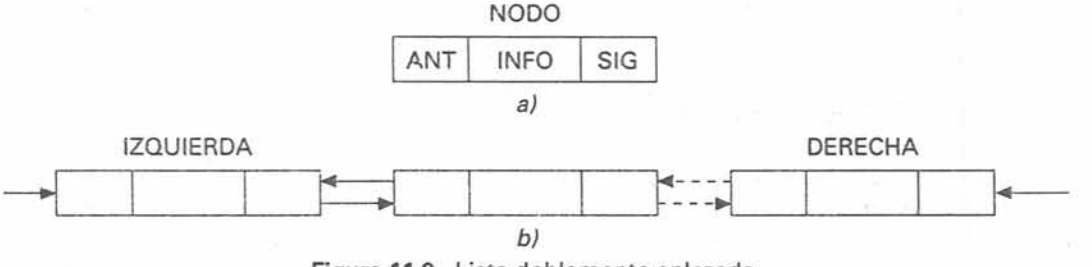


Figura 11.9. Lista doblemente enlazada.

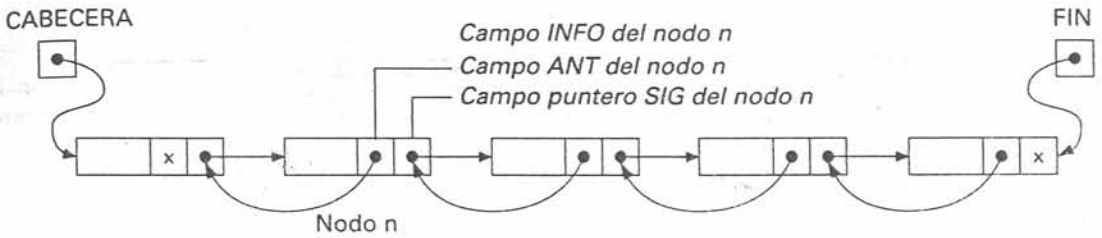


Figura 11.10. Lista doble.

Cada elemento tiene dos punteros y por consiguiente una lista doblemente enlazada ocupa más espacio en memoria que una lista simplemente enlazada para una misma cantidad de información.

La lista necesita dos punteros CABECERA y FIN[2] que apuntan hacia el primero y último nodo.

La variable CABECERA y el puntero SIG permiten recorrer la lista en el sentido normal, y la variable FIN y el puntero ANT permiten recorrerla en sentido inverso.

Como se ve en la Figura 11.10, una propiedad fundamental de las listas doblemente enlazadas es que para cualquier puntero P de la lista:

```
nodo [nodo[p]. sig].ant = p
```

y

```
nodo [nodo[p]. ant].sig = p
```

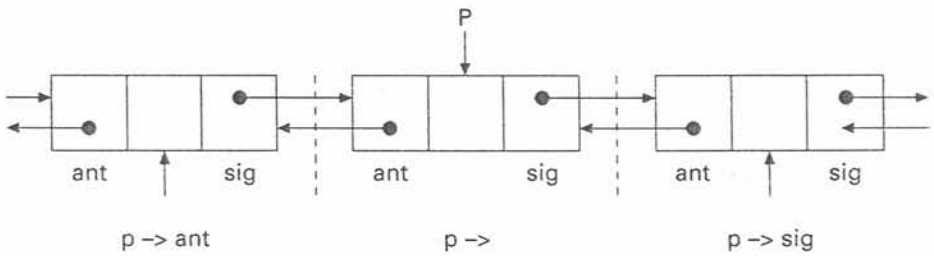


Figura 11.11.

11.6.1. Inserción

La inserción de un nodo a la derecha de un nodo especificado, cuya dirección está dada por la variable M, puede presentar varios casos:

1. La lista está vacía; se indica mediante M = NULO, y CABECERA y FIN son también NULO. Una inserción indica que CABECERA se debe fijar con la dirección del nuevo nodo y los campos ANT y SIG también se establecen en NULO.

[2] Se adoptan estos términos a efectos de normalización, pero el lector puede utilizar otros términos como IZQUIERDA y DERECHA.

2. Insertar dentro de la lista: existe un elemento anterior y otro posterior de X.

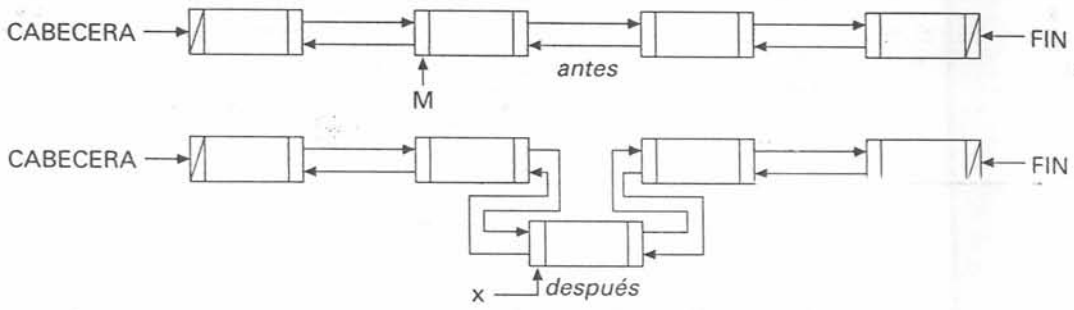


Figura 11.12. Inserción en una lista doblemente enlazada.

3. Insertar a la derecha del nodo de la extrema derecha de la lista. Se requiere que el apuntador FIN sea modificado.

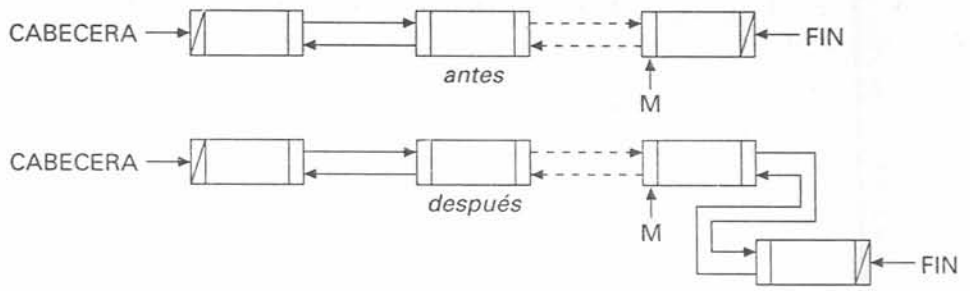


Figura 11.13. Inserción en el extremo derecho de una lista doblemente enlazada.

11.6.2. Eliminación

La operación de eliminación es directa.

Si la lista tiene un simple nodo, entonces los punteros de los extremos izquierdo y derecho asociados a la lista se deben fijar en NULO. Si el nodo del extremo derecho de la lista es el señalado para la eliminación, la variable FIN debe modificarse para señalar el predecesor del nodo que se va a borrar de la lista.

La eliminación se puede realizar dentro de la lista (Figura 11.14).

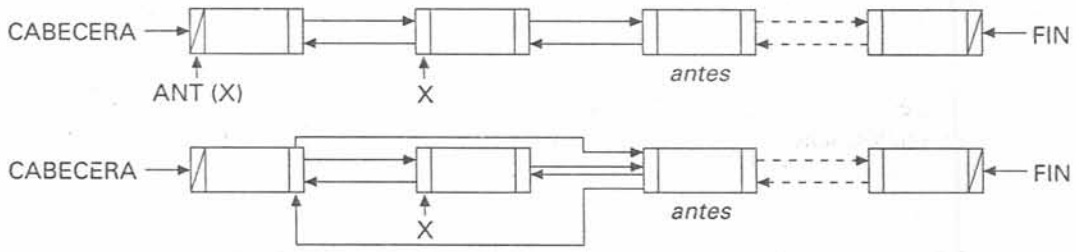


Figura 11.14. Eliminación de un nodo en una lista doblemente enlazada.

11.7. PILAS

Una *pila* (*stack*) es un tipo especial de lista lineal en la que la inserción y borrado de nuevos elementos se realiza sólo por un extremo que se denomina *cima* o *tope* (*top*).

La pila es una estructura con numerosas analogías en la vida real: una pila de platos, una pila de monedas, una pila de cajas de zapatos, una pila de camisas, una pila de bandejas, etc.

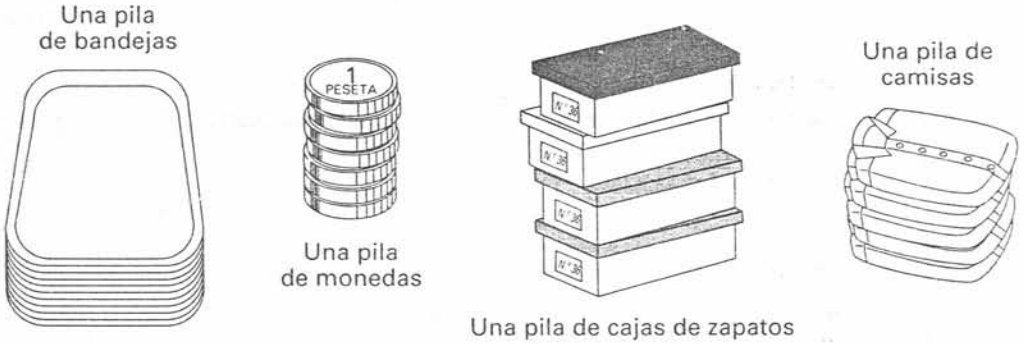


Figura 11.15. Ejemplos de tipos de pilas.

Dado que las operaciones de insertar y eliminar se realizan por un solo extremo (el superior), los elementos sólo pueden eliminarse en orden inverso al que se insertan en la pila. El último elemento que se pone en la pila es el primero que se puede sacar; por ello, a estas estructuras se les conoce por el nombre de **LIFO** (*last-in, first-out*, último en entrar, primero en salir).

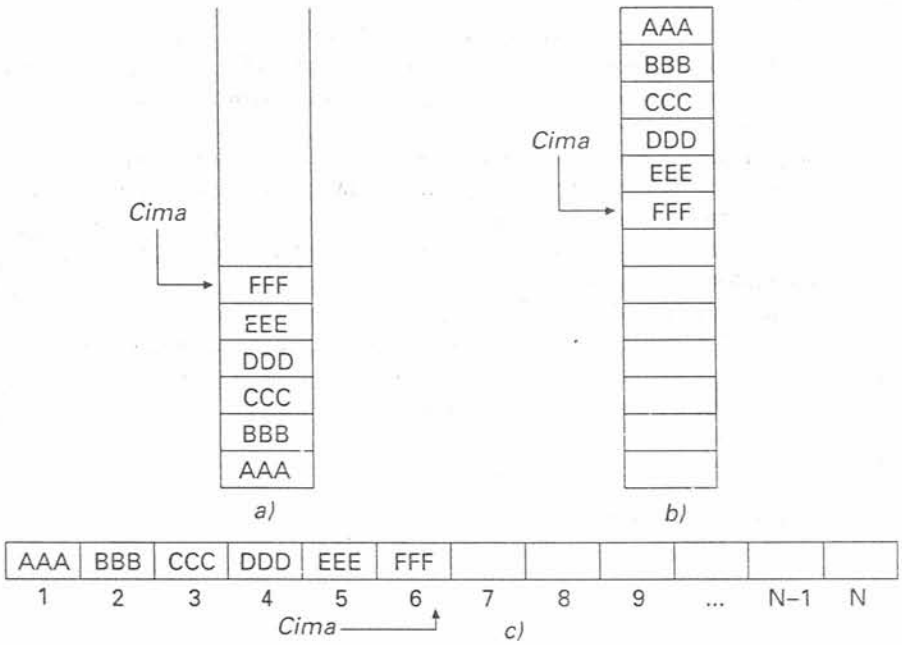


Figura 11.16. Representación de las pilas.

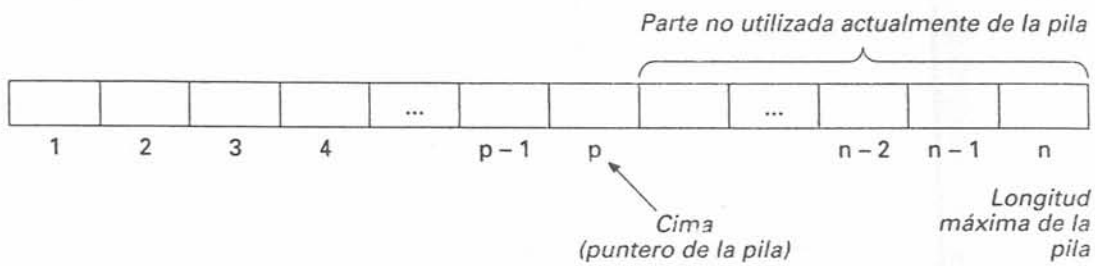
Las operaciones más usuales asociadas a las pilas son:

`push`	*meter o poner*: operación de insertar un elemento en la pila
`pop`	*sacar o quitar*: operación de eliminar un elemento de la pila

Las pilas se pueden representar en cualquiera de las tres formas representadas en la Figura 11.16. Para representar una pila `S`, se puede definir un vector con un determinado tamaño (longitud máxima):

```
var array [1..n] de <tipo_dato> : S
```

Se considerará un elemento entero `P` como el puntero de la pila (*stack pointer*). `P` es el subíndice del array correspondiente al elemento cima de la pila (esto es, el que ocupa la última posición). Si la pila está vacía, `P = 0`.



En principio, la pila está vacía y el puntero de la pila o `CIMA` está a cero. Al meter un elemento en la pila, se incrementa el puntero en una unidad. Al sacar un elemento de la pila se decrementa en una unidad el puntero.

Al manipular una pila se deben realizar algunas comprobaciones. En una pila vacía no se pueden sacar datos (`P = 0`). Si la pila se implementa con un array de tamaño fijo, se puede llenar cuando `P = n` (`n`, longitud total de la pila) y el intento de introducir más elementos en la pila producirá un *desbordamiento de la pila*.

Idealmente, una pila puede contener un número ilimitado de elementos y no producir nunca desbordamiento. En la práctica, sin embargo, el espacio de almacenamiento disponible es finito. La codificación de una pila requiere un cierto equilibrio, ya que si la longitud máxima de la pila es demasiado grande, se gasta mucha memoria, mientras que un valor pequeño de la longitud máxima producirá desbordamientos frecuentes.

Para trabajar fácilmente con pilas, es conveniente diseñar subprogramas de *poner* (*push*) y *quitar* (*pop*) elementos. También es necesario con frecuencia comprobar si la pila está vacía; esto puede conseguirse con una variable o función booleana `VACIA`, de modo que cuando su valor sea *verdadero*, la pila está vacía, y falso en caso contrario.

`P = CIMA`	*puntero de la pila*
`VACIA`	*función booleana «pila vacía»*
`PUSH`	*subprograma para añadir, poner o insertar elementos*
`POP`	*subprograma para eliminar o quitar elementos*
`LONGMAX`	*longitud máxima de la pila*
`S[i]`	*elemento i-ésimo de la pila* `S`
`X`	*elemento a añadir o quitar de la pila*

Implementación con punteros

Si el lenguaje tiene punteros, deberemos implementar las pilas con punteros.

Para la manipulación de una pila mediante punteros deberemos diseñar los siguientes procedimientos:

```
Consultar(...), Meter(...), Sacar(...), Inicializar(...)
```

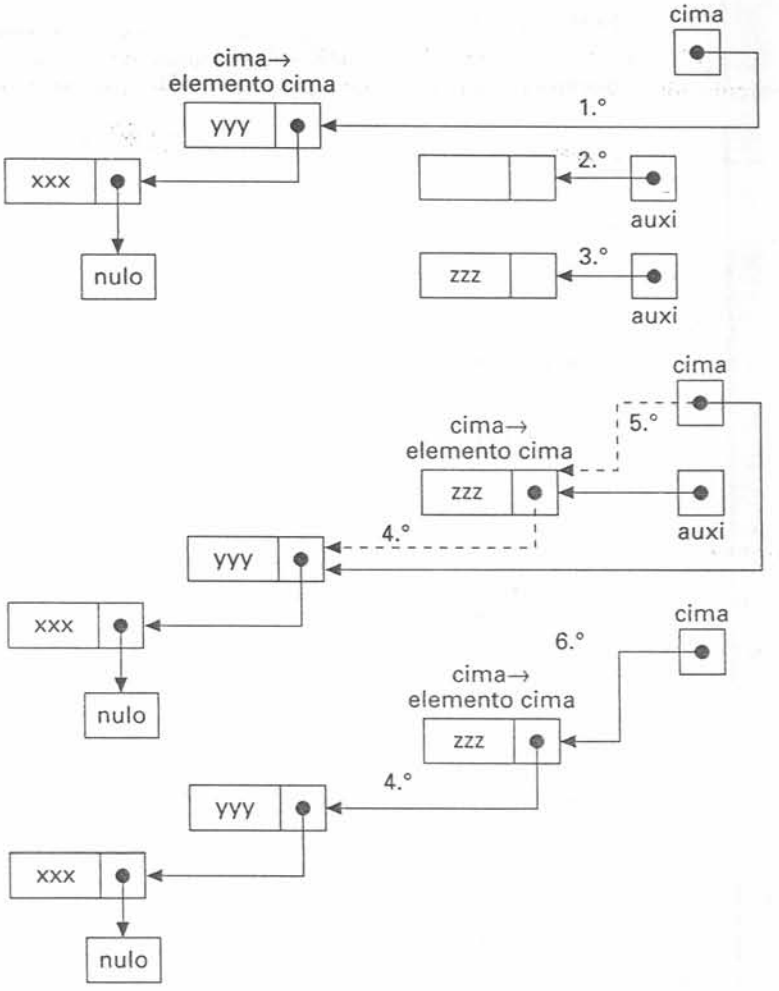
y la función

```
Vacia(...) de resultado lógico
```

```
algoritmo Pilas_con_punteros
tipo
  puntero_a nodo: punt
  registro : tipo_elemento
   .... : ....
   .... : ....
  fin_registro
  registro : nodo
    tipo_elemento : elemento
    punt          : cima
  fin_registro
var
  punt     : cima
  elemento : tipo_elemento

inicio
  inicializar(cima)
fin

procedimiento inicializar(S punt: cima)
inicio
  cima ← nulo
fin_procedimiento

logico funcion vacia(E punt: cima)
inicio
  si cima = nulo entonces
    devolver (verdad)
  si_no
    devolver (falso)
  fin_si
fin_funcion

procedimiento consultar(E punt:cima; S tipo_elemento:elemento)
inicio
  elemento ← (cima ->elemento)
fin_procedimiento
```

Los elementos se incorporan siempre por un extremo, `cima`.

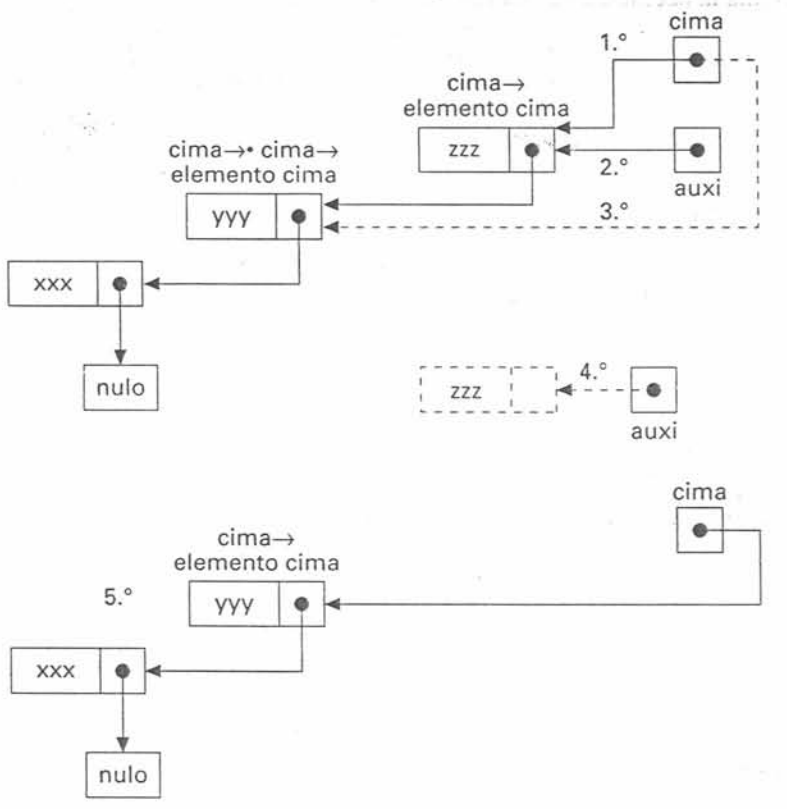
Meter(cima, elemento)




```
1.° Cima apunta al ultimo elemento de la pila
2.° Reservar(auxi)
3.° introducimos la información en auxi ->elemento
4.° hacemos que auxi ->cima apunte a donde cima
5.° cambiamos cima para que apunte a donde lo hace auxi
6.° la pila tiene un elemento mas

procedimiento meter(E/S punt:cima E tipo_elemento: elemento)
var punt: auxi
inicio
  reservar(auxi)
  auxi->elemento ← elemento
  auxi->cima ← (cima)
  cima ← (auxi)
fin_procedimiento
```

Los elementos se recuperan en orden inverso a como fueron introducidos.

sacar(cima, elemento)




```
1.° cima apunta al ultimo elemento de la pila
2.° hacemos que auxi apunte a donde apuntaba cima
3.° y que cima pase a apuntar a donde cima ->cima
4.° liberar(auxi)
5.° la pila tiene un elemento menos
```

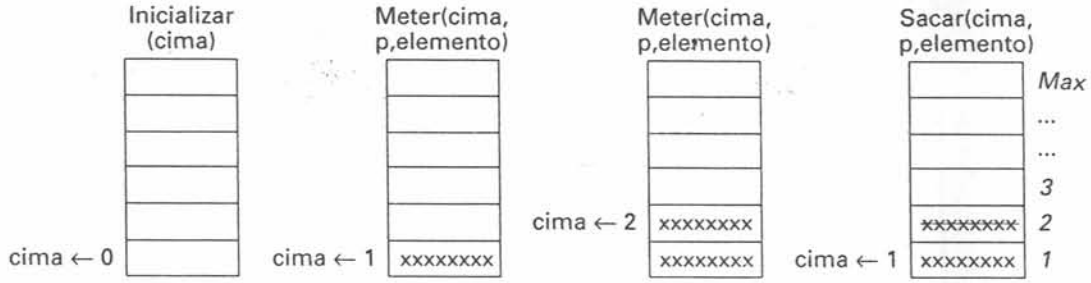
```
procedimiento sacar(E/S punt:cima; S tipo_elemento: elemento)
var punt:auxi
inicio
  auxi ← (cima)
  elemento ← (cima->elemento)
  cima ← (cima->cima)
  liberar(auxi)  //Procedimiento para la eliminacion de variables dinamicas
fin_procedimiento
```

Implementación con arrays

Necesitaremos un array y una variable numérica, cima que apunte al último elemento colocado en la pila.

La inserción o extracción de un elemento se realizará siempre por la parte superior.

Su implementación mediante arrays limita el máximo número de elementos que la pila puede contener y origina la necesidad de una función más, `Llena(...)`, `de resultado lógico`.




```
Const Max = <expresion>
tipo  registro: tipo_elemento
      ..... : .....
      ..... : .....
      fin registro
      array[1..Max] de tipo_elemento: arr
var
  entero        : cima
  arr           : p
  tipo_elemento : elemento

inicio
  inicializar(cima)
fin

procedimiento inicializar(S entero: cima)
inicio
  cima ← 0
fin_procedimiento

logico funcion vacia(E entero: cima)
inicio
  si cima = 0 entonces
    devolver (verdad)
  si_no
    devolver (falso)
  fin_si
fin_funcion

logico funcion llena(E entero: cima)
inicio
  si cima = Max entonces
    devolver (verdad)
  si_no
    devolver (falso)
  fin_si
fin_funcion

procedimiento consultar(E entero:cima;E arr:p;S tipo_elemento: elemento)
inicio
  elemento ← p[cima]
fin_procedimiento
```

```
procedimiento meter(E/S entero: cima;E/S arr: p;E tipo_elemento: elemento)
inicio
  cima ← cima + 1
  p[cima] ← elemento
fin_procedimiento
procedimiento sacar(E/S entero:cima;E arr:p;S tipo_elemento:elemento)
inicio
  elemento ← p[cima]
  cima ← cima - 1
fin_procedimiento
```

11.7.1. Aplicaciones de las pilas

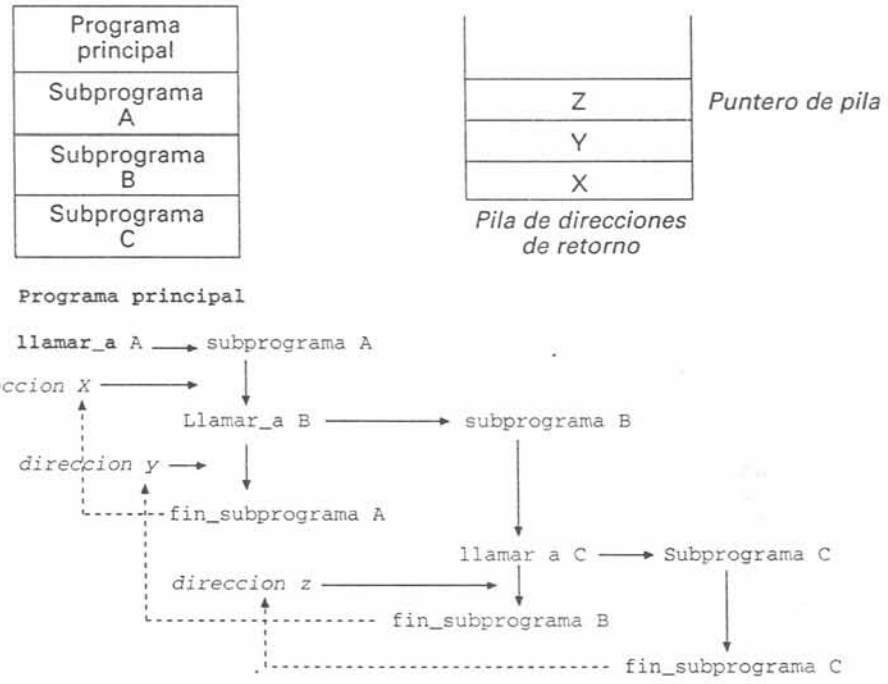
Las pilas son utilizadas para solucionar una amplia variedad de problemas. Se utilizan en compiladores, sistemas operativos y en programas de aplicación. Veamos algunas de las aplicaciones más interesantes.

Llamadas a subprogramas

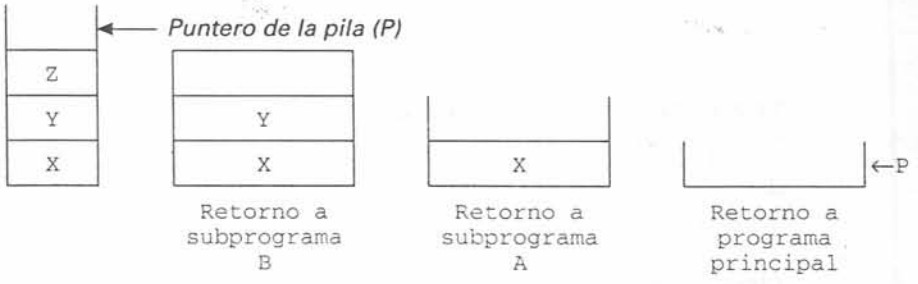
Cuando dentro de un programa se realizan llamadas a subprogramas, el programa principal debe recordar el lugar desde donde se hizo la llamada, de modo que pueda retornar allí cuando el subprograma se haya terminado de ejecutar.

Supongamos que tenemos tres subprogramas, llamados A, B, C, y supongamos también que A invoca a B y B invoca a C. Entonces B no terminará su trabajo hasta que C haya terminado y devuelto su control a B. De modo similar, A es el primero que arranca su ejecución, pero es el último que la termina, tras la terminación y retorno de B.

Esta operación se consigue disponiendo las direcciones de retorno en una pila.



Cuando un subprograma termina, debe retornar a la dirección siguiente a la instrucción que le llamó (`llamar_a`). Cada vez que se invoca un subprograma, la dirección siguiente (x, y o z) se introduce en la pila. El vaciado o limpieza de la pila se realizará por los sucesivos retornos; decrementándose el puntero de pila queda libre siempre apuntando a la siguiente dirección de retorno.



Ejemplo 11.6

Se desea leer un texto y separar los caracteres letras, dígitos y restantes caracteres para ser utilizados posteriormente.

Utilizaremos tres pilas (LETRAS, DIGITOS, OTROSCAR) para contener los diferentes tipos de caracteres. El proceso consiste en leer carácter a carácter, comprobar el tipo de carácter y según el resultado introducirlo en su pila respectiva.

```
algoritmo Lecturacaracter
  const Max = <valor>
tipo
  array [1..Max] de caracter:pila
var
  entero    : cima1, cima2, cima3
  pila      : pilaletras, piladigitos, pilaotroscaracteres
  caracter  : elemento

inicio
  crear (cima1)
  crear (cima2)
  crear (cima3)
  elemento ← leercar
  mientras (codigo(elemento)<> (26)y no llena (cima1) y no llena(cima2)y no llena
           (cima3) hacer
    //saldremos del bucle en cuanto se llene alguna de las pilas o pulsemos ^Z
    si (elemento>='A') y (elemento<= 'Z') o (elemento>= 'a') y(elemento>='z')entonces
    meter (cima1, pilaletras,elemento)
    si_no
    si (elemento>= '0') y (elemento<='9') entonces
      meter (cima2, piladigitos, elemento)
        si_no
      meter (cima3, pilaotroscaracteres, elemento)
        fin_si
      fin_si
    elemento ← leercar
  fin_mientras
fin
```

```
procedimiento crear (S entero: cima)
inicio
  cima ← 0
fin_procedimiento

logico funcion llena (E entero: cima)
inicio
  si cima = Max entonces
    devolver (verdad)
  si_no
    devolver (falso)
  fin_si
fin_funcion

procedimiento meter(E/S entero:cima; E/S tipo_elemento:elemento)
inicio
  cima ← cima + 1
  p [cima] ← elemento
fin_procedimiento
```

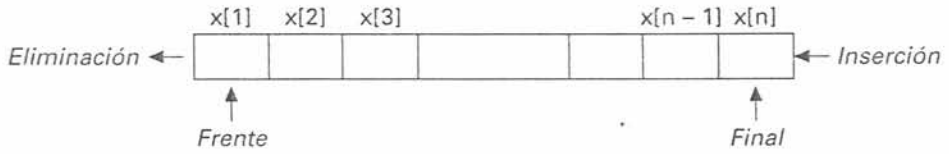
11.8. COLAS

Las colas son otro tipo de estructura lineal de datos, similar a las pilas, diferenciándose de ellas en el modo de insertar/eliminar elementos.

Una *cola* (*queue*) es una estructura lineal de datos

```
var array [1..n] de <tipo_dato> : C
```

en la que las *eliminaciones* se realizan al principio de la lista, *frente* (*front*), y las *inserciones* se realizan en el otro extremo, *final* (*rear*). En las colas el elemento que entró el primero sale también el primero; por ello se conocer como listas **FIFO** (*first-in, first-out*, «primero en entrar, primero en salir»). Así pues, la diferencia con las pilas reside en el modo de entrada/salida de datos; en las colas las inserciones se realizan al final de la lista, no al principio. Por ello, las colas se usan para almacenar datos que necesitan ser procesados según el orden de llegada.



En la vida real se tienen ejemplos numerosos de colas: la cola de un autobús, la cola de un cine, una caravana de coches en una calle, etc. En todas ellas el primer elemento (pasajero, coche, etc.) que llega es el primero que sale.

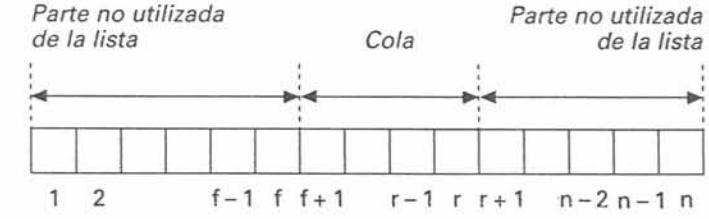
En informática existen también numerosas aplicaciones de las colas. Por ejemplo, en un sistema de tiempo compartido suele haber un procesador central y una serie de periféricos compartidos: discos, impresoras, etc. Los recursos se comparten por los diferentes usuarios y se utiliza una cola para almacenar los programas o peticiones de los diferentes usuarios que esperan su turno de ejecución. El procesador central atiende —normalmente— por riguroso orden de llamada del usuario; por tanto,

todas las llamadas se almacenan en una cola. Existe otra aplicación muy utilizada que se denomina *cola de prioridades*; en ella el procesador central no atiende por riguroso orden de llamada, aquí el procesador atiende por prioridades asignadas por el sistema o bien por el usuario, y sólo dentro de las peticiones de igual prioridad se producirá una cola.

11.8.1. Representación de las colas

Las colas se pueden representar como estructuras dinámicas de datos o como arrays.

Se necesitan dos punteros: `frente(f)` y `final(r)`, y la lista o array de n elementos (LONGMAX).



Si la cola está vacía	`frente = 0`	o bien `f ← 0`
Eliminar elementos	`frente ← frente + 1`	o bien `f ← f + 1`
Añadir elementos	`final ← final + 1`	o bien `r ← r + 1`

La Figura 11.17 muestra la representación de una cola mediante un array o mediante una lista enlazada.

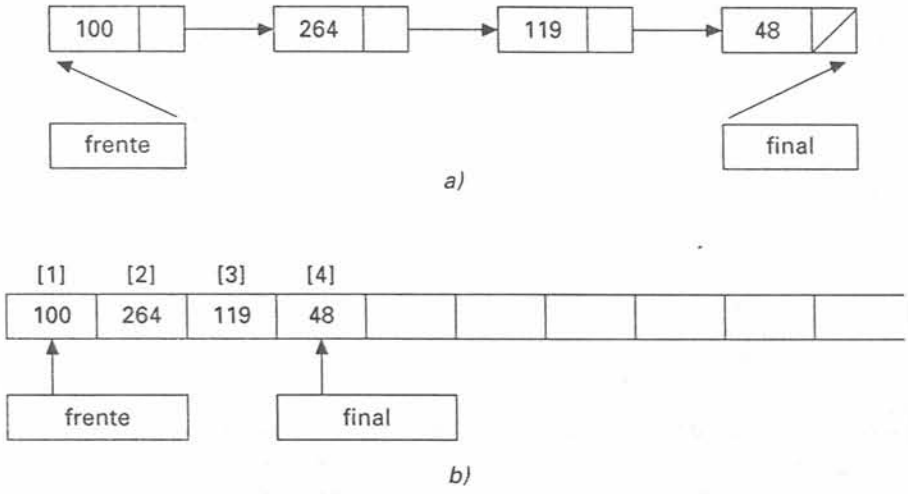


Figura 11.17. Representación de una cola: (b) mediante un array, (a) mediante una lista enlazada.

Las operaciones que se pueden realizar con una cola son:

- acceder al primer elemento de la cola,
- añadir un elemento al final de cola,
- eliminar el primer elemento de la cola,
- vaciar la cola,
- verificar el estado de la cola: vacía:llena.

Implementación como estructuras dinámicas

Aunque, como posteriormente veremos, las colas se pueden simular mediante un array y dos variables numéricas (frente, final), deberemos, si el lenguaje lo permite, implementarlas mediante punteros.

En una cola las eliminaciones se realizarán por el extremo denominado frente y las inserciones por el final. Para la manipulación de una cola necesitaremos: los procedimientos

```
Inicializar(...), Consultar(...), Meter(...), Sacar(...)
```

la función

```
Vacia(...)
```

y los siguientes tipos de datos:

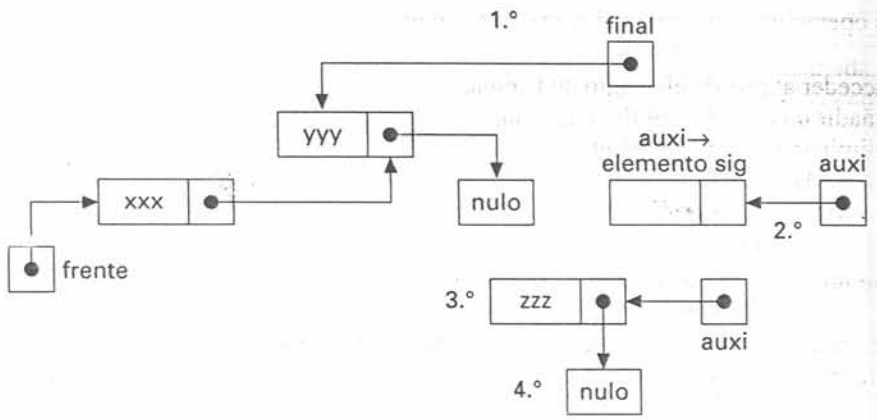
```
tipo
  puntero_a nodo : punt
  registro       : tipo_elemento
   .... : .....
   .... : .....
  fin registro
  registro       : nodo
    tipo_elemento : elemento
    punt          : sig
  fin registro
var
  punt          : frente,final
  tipo_elemento : elemento
```

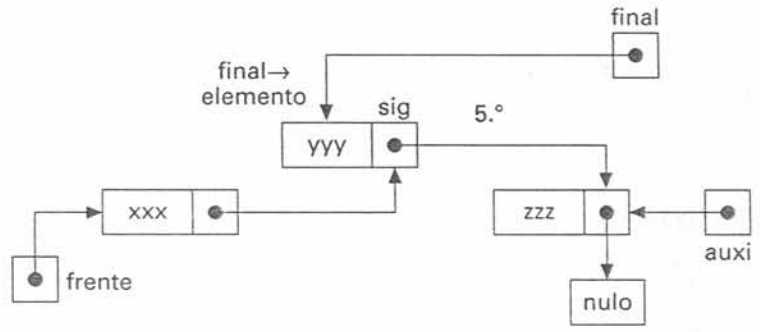
Cuando no hay elementos en la cola:

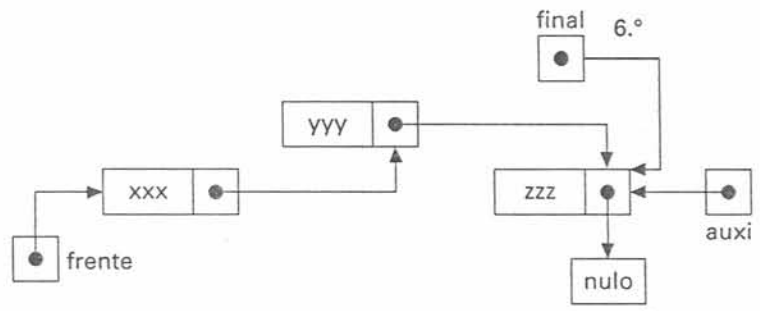
```
frente ← nulo          final ← nulo
```

de lo que deducimos

```
procedimiento inicializar(S punt: frente,final)
  inicio
    frente ← nulo
    final ← nulo
  fin_procedimiento
Meter(final, frente, elemento)
```







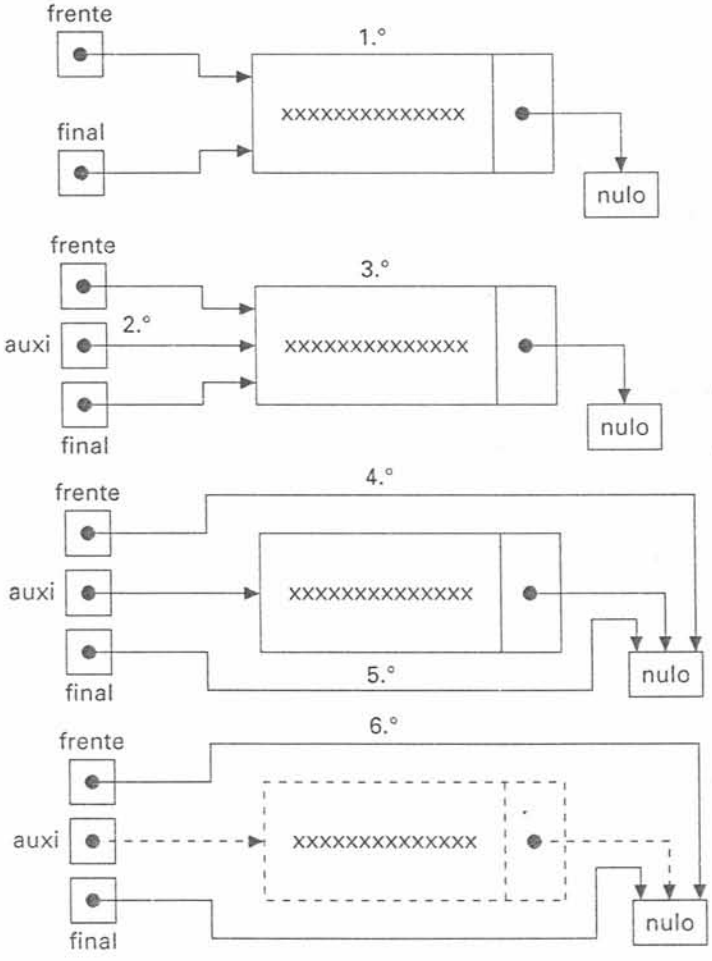

```
1.°  Situacion de partida
2.°  Reservar (auxi)
3.°  Introducir la nueva informacion en auxi ->elemento
4.°  Hacer que auxi ->sig apunte a nulo
5.°  Conseguir que final ->sig apunte a donde lo hace auxi
6.°  Por ultimo final debe apuntar tambien a donde auxi
```

```
procedimiento meter(E/S punt:final;S punt:frente;E tipo_elemento:elemento)
var punt: auxi
inicio
  reservar(auxi)
  auxi ->elemento ← (elemento)
```

```
  auxi ->sig       ← (nulo)
  si final = nulo entonces
    frente ← auxi
  si_no
    final ->sig ← auxi
  fin_si
  final ← auxi
fin_procedimiento
```

Sacar(frente, final, elemento)

Imaginemos que la cola tiene un único elemento. Los elementos se extraen siempre por el frente.




```
1.° Estado inicial
2.° Hacemos que auxi apunte a donde lo hace frente
3.° Extraemos la informacion de auxi ->elemento
4.° Hacemos que frente apunte a donde lo hace auxi ->sig
5.° Como frente toma el valor nulo a final le damos nulo
6.° Liberar(auxi)
```

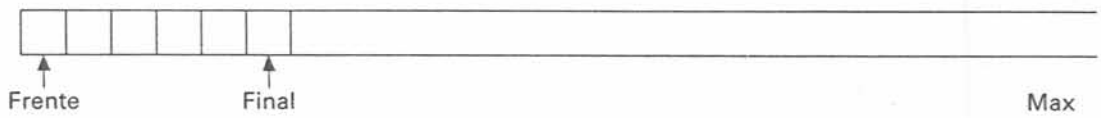
```
procedimiento sacar(E/S punt: frente  S;punt: final;S tipo_elemento: elemento)
var punt: auxi
inicio
  auxi     ← (frente)
  elemento ← (auxi ->elemento)
  frente   ← (frente ->sig)
  si frente = nulo entonces
    final ← (nulo)
  fin_si
  liberar(auxi)
fin_procedimiento
```

Como los elementos se extraen siempre por el frente, la cola estará vacía cuando: `frente = nulo`.

```
logico funcion vacia(E punt: frente)
inicio
  si frente = nulo entonces
    devolver(verdad)
  si_no
    devolver(falso)
  fin_si
fin_funcion

procedimiento consultar(E punt:frente;S tipo_elemento:elemento)
inicio
  elemento ← (frente ->elemento)
fin_procedimiento
```

Implementación utilizando estructuras de tipo array

Podemos representar las colas mediante arrays.




```
const Max  = <expresion>  //longitud maxima
tipo
  registro: tipo_elemento
   .... : ....
   .... : ....
  fin_registro
  array[1..Max] de tipo_elemento: arr
var
  entero        : frente, final
  arr           : c        //la cola se define como un array
  tipo_elemento : elemento
```

Cuando la cola esté vacía, `frente ← 0` `final ← 0`

```
procedimiento inicializar(S entero: frente,final)
inicio
  frente ← 0
  final ← 0
fin_procedimiento
```

El procedimiento para la inserción de un nuevo elemento deberá verificar, en primer lugar, que la cola no está totalmente llena y, por consiguiente, no se producirá error de desbordamiento.
La condición de desbordamiento se produce cuando `final = Max`.

```
procedimiento meter(E/S entero: final;S entero: frente;E/S arr: c;
                    E;tipo_elemento: elemento)
inicio
  si final = 0 entonces
    frente ← 1
  fin_si
  final ← final + 1
  c[final] ← elemento
fin_procedimiento
```

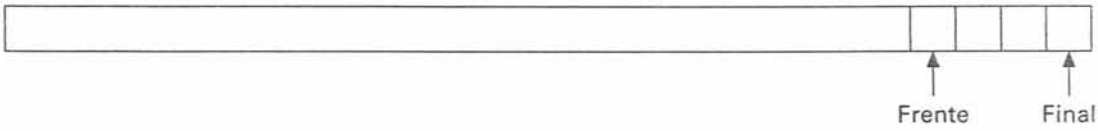
Para eliminar un elemento será preciso verificar, en primer lugar, que la cola no está vacía.

```
procedimiento sacar(E/S entero: frente,final;E arr: c;S tipo_elemento: elemento)
inicio
  elemento ← c[frente]
  frente ← frente + 1
  si frente = final + 1 entonces
    frente ← 0
    final ← 0
  fin_si
fin_procedimiento
```

La cola estará vacía cuando `frente = 0`.

```
logico funcion vacia(E entero: frente)
inicio
  si frente = 0 entonces
    devolver(verdad)
  si_no
    devolver(falso)
  fin_si
fin_funcion
```

Esta implementación tiene el inconveniente de que puede ocurrir que la variable final llegue al valor máximo de la tabla, con lo cual no se puedan seguir añadiendo elementos a la cola, aun cuando queden posiciones libres a la izquierda de la posición frente por haber sido eliminados algunos de sus elementos.



Existen diversas soluciones a este problema:

1) *Retroceso:*
   Consiste en mantener fijo a 1 el valor de frente, realizando un desplazamiento de una posición para todas las componentes ocupadas cada vez que se efectúa una supresión.

2) *Reestructuración:*

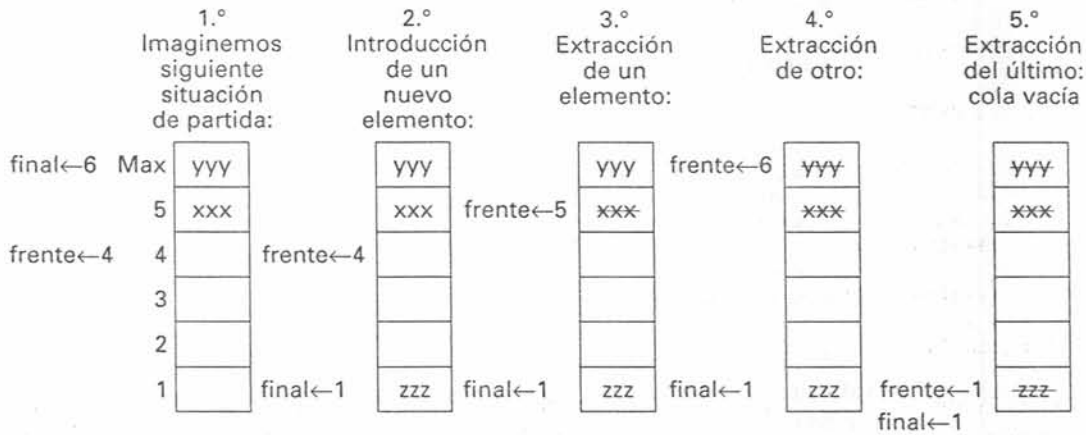
Cuando final llega al máximo de elementos se desplazan las componentes ocupadas hacia atrás las posiciones necesarias para que el principio coincida con el principio de la tabla.

3) *Mediante un array circular:*

Un array circular es aquél en el que se considera que la componente primera sigue a la componente última.
Esta implementación obliga a dejar siempre una posición libre para separar el principio y el final del array.
Evidentemente, seguirá existiendo la limitación de que pueda llenarse completamente el array, `Max - 1` posiciones ocupadas.

11.8.2. Aprovechamiento de la memoria

El mejor método para evitar el desaprovechamiento de espacio es el diseño de la cola mediante un array circular.



Deberemos efectuar las siguientes declaraciones:

```
const Max = <expresion>
tipo
  registro : tipo_elemento
  inicio
      .... : ....
      .... : ....
  fin_registro
  array[1..Max] de tipo_elemento : arr
var
  entero        : frente, final
  arr           : c
  tipo_elemento : elemento
```

En `c[frente]` estará siempre libre, sirviendo para separar el principio y el final del array.

```
procedimiento inicializar(S entero: frente,final)
inicio
  frente ← 1
  final ← 1
fin_procedimiento
```

Los elementos se añaden por el final.

```
procedimiento meter(E/S entero: final; E/S arr: c; E tipo_elemento: elemento)
inicio
  final ← final mod max + 1
  c[final] ← elemento
fin_procedimiento
```

Los elementos se eliminan por el frente. El elemento a eliminar se encuentra siempre en la posición del array siguiente a la especificada por frente.

```
procedimiento sacar(E/S entero: frente; E arr: c; S tipo_elemento: elemento)
inicio
  elemento ← c[frente mod max + 1]
  frente ← frente mod max + 1
fin_procedimiento

logico funcion vacia(E entero: frente,final)
inicio
  si frente = final entonces
    devolver(verdad)
  si_no
    devolver(falso)
  fin_si
fin_funcion
```

Cuando la posición siguiente a final sea frente no podremos añadir más información, pues habría que hacerlo en

`c[final mod max + 1]` *es decir,* `c[frente]`

y la cola se encontrará llena:

```
logico funcion llena(E entero: frente,final)
inicio
  si frente = final mod Max + 1 entonces
    devolver(verdad)
  si_no
    devolver(falso)
  fin_si
fin_funcion

procedimiento consultar(E entero: frente; E arr: c; S tipo_elemento: elemento)
inicio
  elemento ← c[frente mod max + 1]
fin_procedimiento
```

11.9. DOBLE COLA

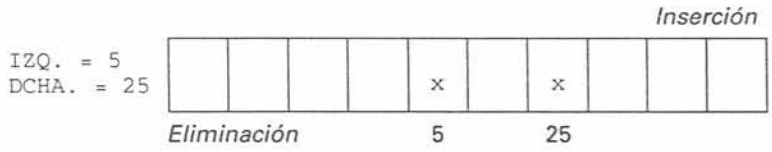
Existe una variante de la cola simple estudiada anteriormente y que es la *doble cola*. La *doble cola* o *bicola* es una cola bidimensional en la que las inserciones y eliminaciones se pueden realizar en cualquiera de los dos extremos de la lista.

Figura 11.24. Doble cola (bicola).

Existen dos variantes de la doble cola:

doble cola de entrada restringida: acepta inserciones sólo al final de la cola,
doble cola de salida restringida: acepta eliminaciones sólo al frente de la cola.

Los procedimientos de inserción y eliminación de las dobles colas son variantes de los procedimientos estudiados para las colas simples y se dejan como ejercicio al lector.



ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

11.1. Una tienda de artículos deportivos desea almacenar en una lista enlazada, con un único elemento por producto, la siguiente información sobre las ventas realizadas: `Código` del artículo, `Cantidad` y `Precio`. Usando estructuras de tipo array, desarrollar un algoritmo que permita tanto la creación de la lista como su actualización al realizarse nuevas ventas o devoluciones de un determinado producto.

Análisis del problema

El algoritmo contemplará la creación de la lista y colocará los elementos clasificados por código, para que las búsquedas puedan resultar algo más rápidas. Al producirse una venta deberemos considerar las siguientes posibilidades:

- Es la primera vez que se vende ese artículo y esto nos lleva a la inserción de un nuevo elemento en la lista.
- Ya se ha vendido alguna otra vez dicho artículo, por tanto, es una modificación de un elemento de la lista, incrementándose la cantidad vendida.

Una devolución nos hará pensar en las siguientes situaciones:

- El comprador devuelve parte de lo que se había vendido de un determinado artículo, lo que representa una modificación de la cantidad vendida, decrementándose con la devolución.
- Se devuelve todo lo que llevábamos vendido de un determinado artículo y, en consecuencia, el producto debe desaparecer de la lista de ventas.

Diseño del algoritmo

```
algoritmo  Ejercicio_11_1
const  max = .....
tipo
  registro  : tipo_elemento
     cadena : cod
     entero : cantidad
     real   : precio
  fin_registro
  registro  : tipo_nodo
     tipo_elemento : elemento
     entero        : sig
  fin_registro
  array[1..Max] de tipo_nodo : lista
var
  entero : inic, vacio
  lista : m
  caracter : opcion

inicio
  Iniciar(m,vacio)
  Inicializar(inic)
  repetir
     escribir('1.- Ventas')
     escribir('2.- Devoluciones')
     escribir('3.- Mostrar lista')
     escribir('4.- Fin')
     escribir('Elija opcion:')
     leer (opcion)
     segun_sea opcion hacer
        '1': nuevasventas (inic,vacio,m)
        '2': devoluciones(inic,vacio,m)
        '3': recorrer (inic,m)
     fin_segun
  hasta_que opcion = '4'
fin

procedimiento inicializar (S entero: inic)
inicio
  inic ← 0
fin_procedimiento
logico funcion vacia (E entero: inic)
inicio
  si inic = 0 entonces
     devolver (verdad)
  si_no
     devolver (falso)
  fin_si
fin_funcion
```

```
procedimiento iniciar (E/S lista: m; E/S entero: vacio)
var
  entero : i
inicio
  vacio ← 1
  desde i ← 1 hasta Max   -1 hacer
    m[i].sig ← i + 1
  fin_desde
  m[Max].sig ← 0
fin_procedimiento


procedimiento reservar (S entero: auxi;  E lista: m E/S entero: vacio)
inicio
  si vacio = 0 entonces
     escribir('Memoria agotada')
     auxi ← 0
  si_no
     auxi ← vacio
     vacio ← m[vacio].sig
  fin_si
fin_procedimiento


logico funcion llena (E entero: vacio)
inicio
  si vacio = 0 entonces
     devolver (verdad)
  si_no
     devolver (falso)
  fin_si
fin_funcion
procedimiento consultar (E entero : inic; S entero : posic,anterior;
                         E tipo_elemento : elemento;
                         S logico : encontrado; E lista: m)

inicio
  anterior ← 0
  posic ← inic
  mientras (m[posic].elemento.cod < elemento.cod) y (posic <> 0) hacer
     anterior ← posic
     posic ← m[posic].sig
  fin_mientras
  si m[posic].elemento.cod = elemento.cod entonces
     encontrado ← verdad
  si_no
     encontrado ← falso
  fin_si
fin_procedimiento


procedimiento  insertar (E/S entero: inic, anterior;
                         E tipo_elemento: elemento;
                         E/S lista: m; E/S entero: vacio)
var
  entero: auxi
inicio
  llamar_a reservar(auxi,m,vacio)
  m[auxi].elemento ← elemento
```

```
  si anterior = 0 entonces
     m[auxi].sig ← inic
     inic ← auxi
  si_no
     m[auxi].sig ← m[anterior].sig
     m[anterior].sig ← auxi
  fin_si
  anterior ← auxi
fin_procedimiento

procedimiento recorrer (E entero: inic; E lista: m)
var
  entero: posic
inicio
  posic ← inic
  mientras posic <> 0 hacer
     escribir (m[posic].elemento.cod,'    ', m[posic].elemento.cantidad,'
          ➥ ',m[posic].elemento.precio))
     posic ← m[posic].sig
  fin_mientras
fin_procedimiento

procedimiento suprimir (E/S entero : inic,anterior, posic;
                        E/S lista: m; E/S entero: vacio)
inicio
  si anterior = 0 entonces
     inic ← m[posic].sig
  si no
     m[anterior].sig ← m[posic].sig
  fin_si
  liberar(posic,m,vacio)
  anterior ← 0
  posic  ← inic
fin_procedimiento

procedimiento nuevasventas (E/S entero : inic,vacio; E/S lista: m)
var
  tipo_elemento : elemento
  logico        : encontrado
  entero        : anterior, posic
inicio
  repetir
     escribir('Codigo:')
     leer(elemento.cod)
     si elemento.cod <> '*' entonces
        si vacia (inic) entonces
          anterior ← 0
          escribir('Cantidad:')
          leer (elemento.cantidad)
          escribir('Precio:')
          leer(elemento.precio)
          insertar(inic,anterior,elemento,m,vacio)
        si_no
          consultar(inic,posic,anterior,elemento,encontrado,m)
          si no encontrado entonces
            si no llena (vacio) entonces
              escribir('Cantidad:')
```

```
                leer (elemento.cantidad)
                escribir('Precio:')
                leer(elemento.precio)
                insertar(inic,anterior,elemento,m,vacio)
              si_no
                escribir('Llena')
              fin_si
            si_no
              escribir('Cantidad:')
              leer(elemento.cantidad)
              m[posic].elemento.cantidad ←
                   ➥ m[posic].elemento.cantidad + elemento.cantidad
            fin_si
          fin_si
        fin_si
        hasta_que (elemento.cod='*')
fin_procedimiento

procedimiento devoluciones (E/S entero: inic,vacio;  E/S lista: m)
var
  tipo_elemento : elemento
  entero        : posic,anterior
  entero        : cantidad
  logico        : encontrado
inicio
  si no vacia(inic) entonces
     escribir('Deme elemento y * para fin')
     escribir ('Codigo:')
     leer(elemento.cod)
  si_no
     escribir('No hay ventas, no puede haber devoluciones')
  fin_si
  mientras (elemento.cod <> '*') y no vacia (inic) hacer
     consultar (inic,posic,anterior,elemento,encontrado,m)
     si encontrado entonces
        repetir
           escribir('Deme cantidad devuelta')
           leer(cantidad)
           si cantidad > m[posic].elemento.cantidad entonces
             escribir('No se puede devolver mas de lo que se compro')
           fin_si
        hasta_que cantidad <= m[posic].elemento.cantidad
        m[posic].elemento.cantidad ← m[posic].elemento.cantidad-cantidad
        si m[posic].elemento.cantidad = 0 entonces
           suprimir(inic,anterior,posic,m,vacio)
        fin_si
     si_no
        escribir('No existe')
     fin si
     si no vacia (inic) entonces
        escribir('Deme elemento y * para fin')
        escribir('Codigo:')
        leer (elemento.cod)
     si_no
        escribir('No hay ventas, no puede haber devoluciones')
     fin_si
  fin_mientras
fin_procedimiento
```

11.2. Diseñar las operaciones primitivas necesarias para trabajar con pilas utilizando estructuras de tipo array.

Definición de las estructuras de datos

Si los datos los vamos a almacenar en un array, la estructura pila será un registro compuesto por un array del tipo base de la pila (`TipoElemento`) y una variable entera, `cima`, que indica la posición del último elemento de la pila. El número máximo de elementos de la pila vendrá determinado por una constante (`MaxPila`).

```
const MaxPila = ...
tipo
  registro : TipoElemento
    ... : ...
    ... : ...
  fin_registro
  registro : Pila
    entero : cima
    array[1..MaxPila] de TipoElemento : el
  fin_registro
var
  pila : p
```

Procedimientos y funciones

Vamos a considerar cinco primitivas para trabajar con pilas:

`Pila Vacia`	Inicializa una pila, crea una pila como vacía.
`EsPilaVacia`	Es una función lógica que indica si una pila está vacía.
`PInsertar`	Inserta un elemento en la pila.
`Tope`	Devuelve el elemento situado en la cima de la pila.
`PBorrar`	Borra un elemento de la pila.

El procedimiento `PilaVacia`, cuando se trabaja con arrays, se limita a inicializar el entero cima a cero —podemos considerar el cero como un puntero `nulo` a arrays.

```
procedimiento PilaVacia(S pila : p)
inicio
  p.cima ← 0
fin_procedimiento
```

La función lógica `EsPilaVacia`, indica si una pila está vacía, es decir, si un puntero cima es nulo. Esta función va a ser útil cuando queramos recorrer una pila hasta el final.

```
logico funcion EsPilaVacia (E pila : p)
inicio
  devolver(p.cima = 0)
  ptr : aux
inicio
  si EsColaVacia (c) entonces
     // Error, la cola esta vacía
  si_no
     aux ← c.p
     c.p ← c.p ← .sig
     liberar(aux)
     si c.p  = nulo entonces
        c.f ← nulo
     fin_si
fin_procedimiento
```

11.3. Diseñar un procedimiento que realice una copia de una pila en otra.

ANÁLISIS DEL PROBLEMA

Entendemos por copiar la acción de rellenar otra pila con los mismos elementos y en el mismo orden. Por tanto, si simplemente sacamos los elementos de la pila y los metemos en otra, tendrá los mismos elementos, pero en orden distinto. Tenemos dos soluciones, una sería utilizar una pila auxiliar: sacaremos los elementos de la pila principal y los meteremos en la auxiliar, para después volcarlos en la pila de salida. La otra solución sería recursiva: sacaremos los elementos de la pila mediante llamadas recursivas; cuando la pila esté vacía inicializaremos la copia y a la vuelta de la recursividad iremos metiendo los elementos en orden inverso a como han salido.
Observar que el procedimiento valdrá tanto para la implementación con arrays como con estructuras dinámicas de datos. Sin embargo, en este segundo caso, al llamar al procedimiento `PBorrar`, liberaremos los nodos de la memoria, con lo que perderemos la información de éstos. En este caso, el procedimiento `PBorrar`, no debería incluir la llamada al procedimiento estándar `liberar` para no perder la información. En el caso de los arrays, al pasar la pila original como entrada no perdemos su información.

DISEÑO DEL ALGORITMO

Solución iterativa

```
procedimiento CopiarPila (E pila : p; S pila : copia)
var
  pila : aux
  TipoElemento : e
inicio
  PilaVacia(aux)
  mientras no EsPilaVacia (p) hacer
    Tope(p,e)
    PInsertar(aux,e)
    PBorrar(p)
  fin_mientras
  PilaVacia(copia)
  mientras no EsPilaVacia(aux) hacer
    Tope(aux,e)
    PInsertar(copa,e)
    PBorrar(aux)
  fin_mientras
fin_procedimiento
```

Solución recursiva

```
procedimiento CopiarPila (E pila: p; S pila : copia)
var
  TipoElemento : e
inicio
  si no EsPilaVacia(p) hacer
    Tope(p,e)
    PBorrar(p)
    CopiarPila(p,salida)
    PInsertar(salida,e)
  si_no
    PilaVacia(salida)
  fin_si
fin_procedimiento
```

11.4. Diseñar un procedimiento que elimine el elemento *n*-simo de una pila.

ANÁLISIS DEL PROBLEMA

También en este caso debemos utilizar una pila auxiliar o recursividad para poder restaurar los elementos en el mismo orden. Debemos ir borrando elementos e insertarlos en la pila auxiliar hasta llegar al elemento n. En este punto, sacaremos todos los elementos de la pila auxiliar y los introduciremos en la pila original.

Obsérvese que en este caso es totalmente indistinto utilizar estructuras de datos dinámicas o estáticas.

DISEÑO DEL ALGORITMO

Solución iterativa

```
procedimiento BorrarElementoN (S pila : p; E entero : n)
var
  pila : aux
  TipoElemento : e
  entero : i
inicio
  i ← 1
  PilaVacia(aux)
  mientras no EsPilaVacia(p) y i < n hacer
     i ← i + 1
     Tope(p,e)
     PInsertar(aux,e)
     PBorrar(p)
  fin mientras
  PBorrar(p)
  mientras no EsPilaVacia(aux) hacer
     Tope(aux,e)
     PInsertar(p,e)
     PBorrar(aux)
  fin_mientras
fin_procedimiento
```

Solución recursiva

```
procedimiento BorrarElementoN (S pila :p; E entero : n)
var
  TipoElemento : e
inicio
  si i > 0 y no EsPilaVacia (p) entonces
     Tope(p,e)
     PBorrar(p)
     BorrarElementoN(p,n - 1)
     PInsertar(p,e)
  si_no
     PBorrar(p)
  fin_si
fin_procedimiento
```

11.5. Diseñar un algoritmo para, utilizando pilas y colas, comprobar si una frase es un palíndromo (un palíndromo es una frase que se lee igual de izquierda a derecha que de derecha a izquierda).

Análisis del problema

Se puede aprovechar el distinto orden en que salen los elementos de una pila y una cola para averiguar si una frase es igual a ella misma invertida. Para ello, una vez introducida la frase, insertamos en una pila y una cola todos los caracteres (vamos a evitar los signos de puntuación, y se podría mejorar si se convierten todos los caracteres a mayúsculas o minúsculas y se eliminan los acentos).
A continuación, se van sacando elementos de la pila y de la cola. Si aparece algún carácter distinto, es que la frase no es igual a ella misma invertida, y por tanto no será un palíndromo. Si al acabar de sacar los elementos, todos han sido iguales, se trata de un palíndromo.

Diseño del algoritmo

```
algoritmo Ejercicio_11_5
// Aqui deberian incluirse las declaraciones, procedimientos y
// funciones para trabajar con pilas y colas.
// Es indistinto trabajar con estructuras estaticas o dinamicas.
tipo
  caracter : TipoElemento
var
  pila : p
  cola : c
  caracter : car1, car2
  cadena : frase
  entero : i
inicio
  ColaVacia(c)
  PilaVacia(p)
  leer(frase)
  desde i ← 1 hasta longitud (frase) hacer
     car1 ← subcadena (frase, i, 1)
     // si no es un signo de puntuacion
     si posicion (car1, ',:;.') = 0 entonces
        CInsertar(c,car1)
        PInsertar(p,car1)
     fin_si
  fin_desde
```

EJERCICIOS

11.1. Dada una lista lineal cuya estructura de nodos consta de los campos INFO y ENLACE, diseñar un algoritmo que cuente el número de nodos de la lista.

11.2. Diseñar un algoritmo que cambie el campo INFO del *n*-ésimo nodo de una lista enlazada simple por un valor dado x.

11.3. Dadas dos listas enlazadas, cuyos nodos frontales se indican por los apuntadores PRIMERO y SEGUNDO, respectivamente, realizar un algoritmo que una ambas listas. El nodo frontal de la lista nueva se almacenará en TERCERO.

11.4. Se dispone de una lista enlazada DEMO 1 almacenada en memoria. Realizar un algoritmo que copie la lista DEMO 1 en otra denominada DEMO 2.

11.5. Escribir un algoritmo que realice una inserción contigua a la izquierda del n-ésimo nodo de una lista enlazada y repetir el ejercicio para una inserción también contigua a la derecha de n-ésimo nodo.

11.6. Escribir un algoritmo que divide una lista enlazada determinada en dos listas enlazadas independientes. El primer nodo de la lista principal es `PRIMERO` y la variable `PARTIR` es la dirección del nodo que se convierte en el primero de los nodos de la segunda lista enlazada resultante.

11.7. Como aplicación de pilas, obtener un subalgoritmo función recursiva de la función de Ackermann.

Función de Ackermann:

$$A(m, n) = \begin{cases} n + 1 & \text{si } m = 0 \\ A(m - 1,1) & \text{si } n = 0 \\ A(m - 1)\, A(m, n - 1) & \text{restantes casos} \end{cases}$$

11.8. Escribir un subalgoritmo que permita insertar un elemento en una doble cola —representada por un vector—. Tengan en cuenta que debe existir un parámetro que indique el extremo de la doble cola en que debe realizarse la inserción.

11.9. Realizar un algoritmo que cuente el número de nodos de una lista circular que tiene una cabecera.

11.10. Diseñar un algoritmo que inserte un nodo al final de una lista circular.

CAPÍTULO 12

ESTRUCTURA DE DATOS NO LINEALES (ÁRBOLES Y GRAFOS)

CONTENIDO

12.1. Introducción.
12.2. Árboles.
12.3. Árbol binario.
12.4. Árbol binario de búsqueda.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

Las *estructuras dinámicas lineales de datos —listas enlazadas, pilas y colas—* tienen grandes ventajas de flexibilidad sobre las representaciones contiguas; sin embargo, tienen un punto débil: son listas secuenciales, es decir, están dispuestas de modo que es necesario moverse a través de ellas una posición cada vez (cada elemento tiene un siguiente elemento). Esta linealidad es típica de cadenas, de elementos que pertenecen a una sola dimensión: campos en un registro, entradas en una pila, entradas en una cola y de nodos en una lista enlazada simple. En este capítulo se tratarán las *estructuras de datos no lineales* que resuelven los problemas que plantean las listas lineales y en las que cada elemento puede tener diferentes «siguientes» elementos, que introducen el concepto de estructuras de bifurcación. Estos tipos de datos se llaman *árboles*.

Así mismo este capítulo introduce a una estructura matemática importante que tiene aplicaciones en ciencias tan diversas como la sociología, química, física, geografía y electrónica. Estas estructuras se denominan *grafos*.

12.1. INTRODUCCIÓN

Las estructuras de datos que han sido examinadas hasta ahora en este libro son lineales. A cada elemento le correspondía siempre un «siguiente» elemento. La linealidad es típica de cadenas, de elementos de arrays o listas, de campos en registros, entradas en pilas o colas y nodos en listas enlazadas.

En este capítulo se examinarán las estructuras de datos *no lineales*. En estas estructuras cada elemento puede tener diferentes «siguientes» elementos, que introduce el concepto de estructuras de bifurcación.

Las estructuras de datos no lineales son *árboles* y *grafos*. A estas estructuras se les denomina también *estructuras multi-enlazadas*.

12.2. ÁRBOLES

El árbol es una estructura de datos fundamental en informática, muy utilizada en todos sus campos, porque se adapta a la representación natural de informaciones homogéneas organizadas y de una gran comodidad y rapidez de manipulación. Esta estructura se encuentra en todos los dominios (campos) de la informática, desde la pura *algorítmica* (métodos de clasificación y búsqueda...) a la *compilación* (árboles sintácticos para representar las expresiones o producciones posibles de un lenguaje) o incluso los dominios de la inteligencia artificial (árboles de juegos, árboles de decisiones, de resolución, etc.).

Las estructuras tipo árbol se usan principalmente para representar datos con una relación jerárquica entre sus elementos, como son árboles genealógicos, tablas, etc.

Un árbol A es un conjunto finito de uno o más nodos tales que:

1. Existe un nodo especial denominado RAIZ(v_1) del árbol.
2. Los nodos restantes (v_2, v_3, . . . , v_n) se dividen en $m >= 0$ conjuntos disjuntos denominado A_1, A_2, . . . , A_m, cada uno de los cuales es, a su vez, un árbol. Estos árboles se llaman *subárboles* del RAIZ.

La definición de árbol implica una estructura recursiva. Esto es, la definición del árbol se refiere a otros árboles. Un árbol con ningún nodo es un *árbol nulo*; no tiene raíz.

La Figura 12.1 muestra un árbol en el que hemos rotulado cada nodo con una letra dentro de un círculo. Ésta es una notación típica para dibujar árboles.

Los tres subárboles del raíz A son B, C y D, respectivamente. B es la raíz de un árbol con un subárbol E. Este subárbol no tiene subárbol conectado. El árbol C tiene dos subárboles, F y G.

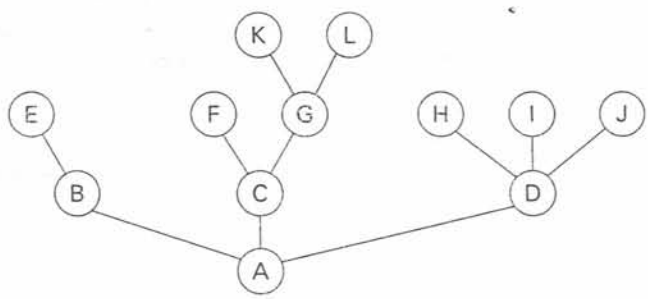


Figura 12.1. Árbol.

12.2.1. Terminología y representación de un árbol general

La representación y terminología de los árboles se realiza con las típicas notaciones de las relaciones familiares en los árboles genealógicos: padre, hijo, hermano, ascendente, descendiente, etc. Sea el árbol general de la Figura 12.2.

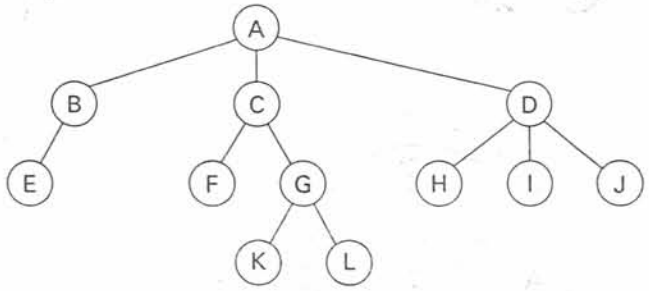


Figura 12.2. Árbol general.

Las definiciones a tener en cuenta son:

- *Raíz* del árbol. Todos los árboles que no están vacíos tienen un único nodo raíz. Todos los demás elementos o nodos se derivan o descienden de él. El nodo raíz no tiene *padre* —es decir, no es el hijo de ningún elemento.
- *Nodo*, son los vértices o elementos del árbol.
- *Nodo terminal u hoja* (*leaf node*) es aquel nodo que no contiene ningún subárbol (los nodos terminales u hojas del árbol de la Figura 12.1 son E, F, K, L, H, I y J).
- A cada nodo que no es hoja se asocia uno o varios subárboles llamados *descendientes* (*offspring*) o *hijos*. De igual forma, cada nodo tiene asociado un antecesor o *ascendiente* llamado *padre*.
- Los nodos de un mismo padre se llaman *hermanos*.
- Los nodos con uno o dos subárboles —no son hojas ni raíz— se llaman *nodos interiores* o *internos*.
- Una colección de dos o más árboles se llama *bosque* (*forest*).
- Todos los nodos tienen un solo padre —excepto el *raíz*— que no tiene padre.
- Se denomina *camino* el enlace entre dos nodos consecutivos, y *rama* es un camino que termina en una hoja.
- Cada nodo tiene asociado un número de *nivel* que se determina por la longitud del camino desde el raíz al nodo específico. Por ejemplo, en el árbol de la Figura 12.1.

```
nivel 0    A
nivel 1    B, C, D
nivel 2    E, F, G, H, I, J
nivel 3    K, L
```

- La *altura* o *profundidad* de un árbol es el número máximo de nodos de una rama. Equivale al nivel más alto de los nodos más uno. El *peso* de un árbol es el número de nodos terminales. La *altura* y el *peso* del árbol de la Figura 12.1 son 4 y 7, respectivamente.

Las representaciones gráficas de los árboles —además de las ya expuestas— pueden ser las mostradas en la Figura 12.3.

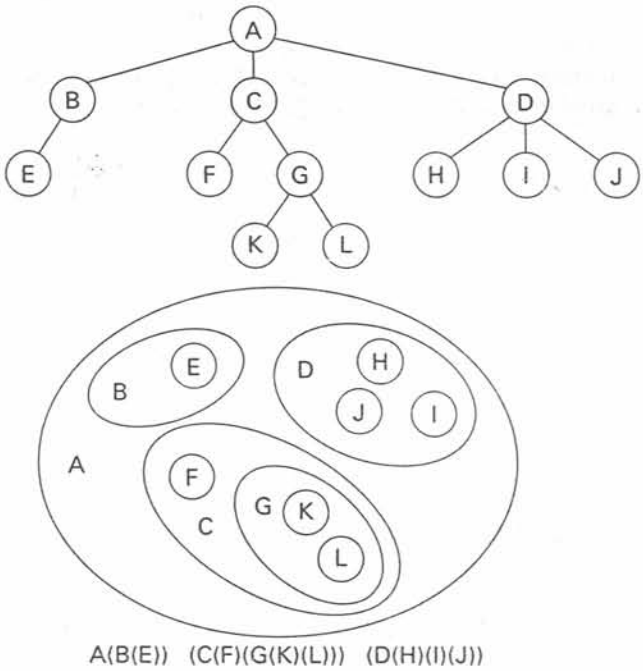


Figura 12.3. Representaciones de árboles.

12.3. ÁRBOL BINARIO

Existe un tipo de árbol denominado *árbol binario* que puede ser implementado fácilmente en una computadora.

Un *árbol binario* es un conjunto finito de cero o más nodos tales que:

- existe un nodo denominado raíz del árbol,
- cada nodo puede tener 0, 1 o 2 subárboles, conocidos como *subárbol izquierdo* y *subárbol derecho*.

La Figura 12.4 representa diferentes tipos de árboles binarios:

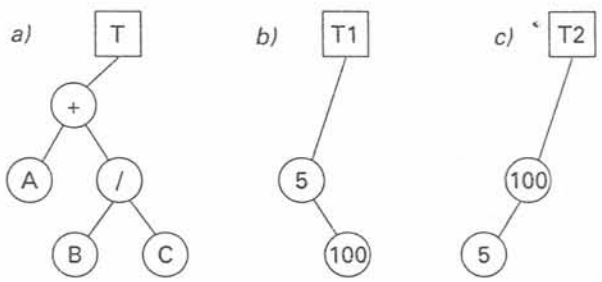


Figura 12.4. Ejemplos de árboles binarios: (a) expresión árbol a + b/c; (b) y (c) dos árboles diferentes con valores no enteros.

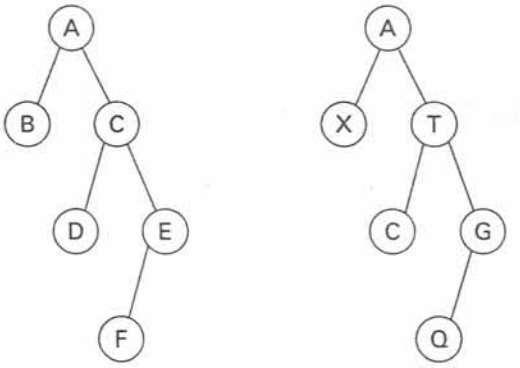
12.3.1. Terminología de los árboles binarios

Dos árboles binarios se dice que son *similares* si tienen la misma estructura, y son *equivalentes* si son similares y contienen la misma información.

Un árbol binario está *equilibrado* si las alturas de los dos subárboles de cada nodo del árbol se diferencian en una unidad como máximo.

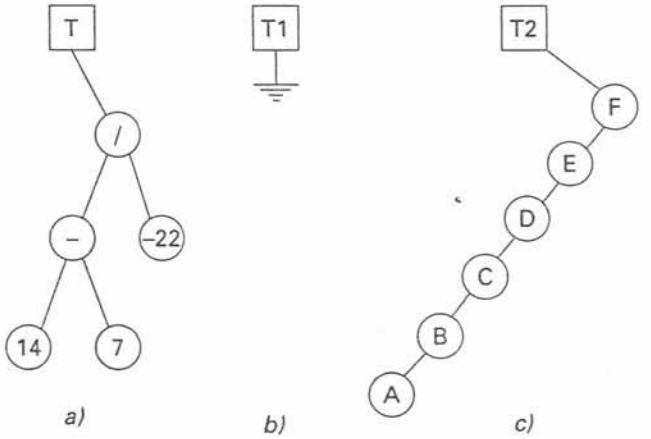
altura (subarbol izquierdo) – altura (subarbol derecho) ≤ 1

El procesamiento de árboles binarios equilibrados es más sencillo que los árboles no equilibrados. En la Figura 12.6 se muestran dos árboles binarios de diferentes alturas y en la Figura 12.7, árboles equilibrados y sin equilibrar.



a) *b)*

Figura 12.5. Árboles binarios similares.



a) *b)* *c)*

Figura 12.6. Árboles binarios de diferentes alturas: (a) altura 3, (b) árbol vacío, altura 0, (c) altura 6.

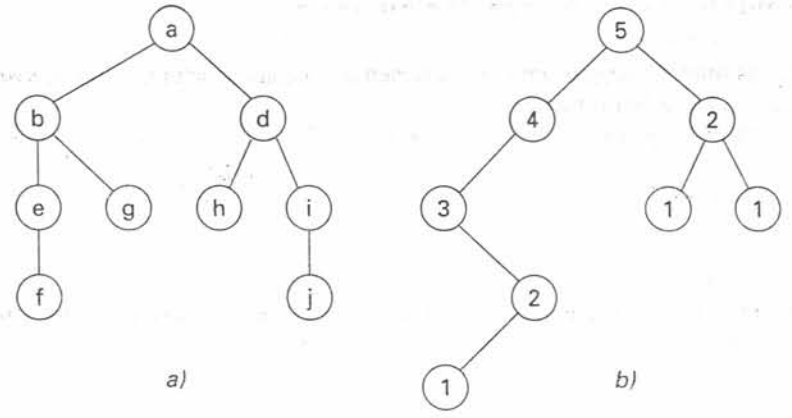


Figura 12.7. Árboles binarios: (a) equilibrados, (b) no equilibrados.

12.3.2. Árboles binarios completos

Un árbol binario se llama *completo* si todos sus nodos, excepto las hojas, tienen exactamente dos subárboles. Un árbol binario completo, tal que todos los niveles están llenos, se llama *árbol binario lleno*.

En la Figura 12.8 se ilustran ambos tipos de árboles.

Un árbol binario T de nivel h puede tener como máximo $2^h - 1$ nodos.

La altura de un árbol binario lleno de n nodos es $log_2(n + 1)$. A la inversa, el *número máximo de nodos* de un árbol binario de altura h será $2^h - 1$. En la Figura 12.9 se muestra la relación matemática que liga los nodos de un árbol.

Por último, se denomina *árbol degenerado* un árbol en el que todos sus nodos tienen solamente un subárbol, excepto el último.

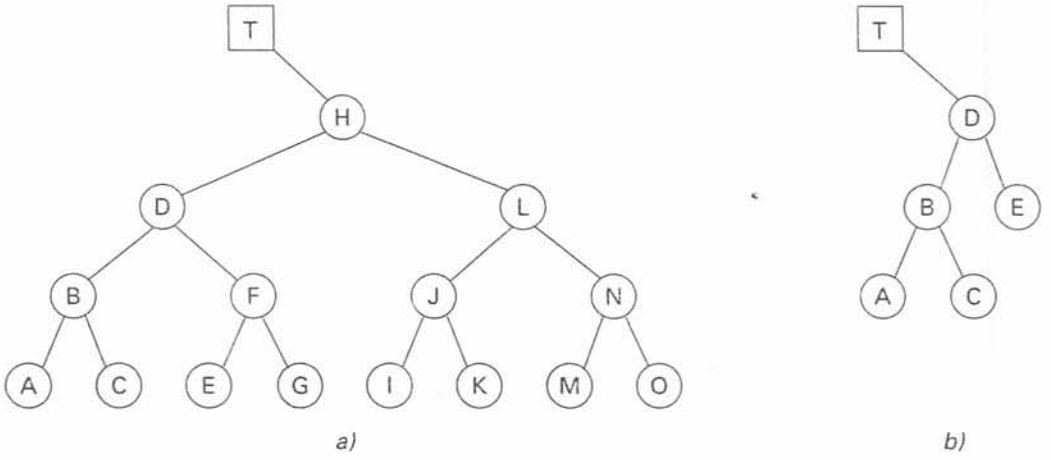


Figura 12.8. (a) Árbol binario lleno de altura 4, (b) árbol binario completo de altura 3.

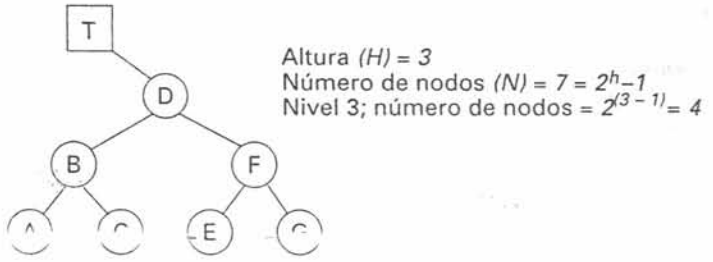


Figura 12.9. Relaciones matemáticas de un árbol binario.

12.3.3. Conversión de un árbol general en árbol binario

Dado que los árboles binarios son la estructura fundamental en la teoría de árboles, será preciso disponer de algún mecanismo que permita la conversión de un árbol general en un árbol binario.

Los árboles binarios son más fáciles de programar que los árboles generales. En éstos es imprescindible deducir cuántas ramas o caminos se desprenden de un nodo en un momento dado. Por ello, y dado que de los árboles binarios siempre se cuelgan como máximo dos subárboles, su programación será más sencilla.

Afortunadamente, existe una técnica para convertir un árbol general a formato de árbol binario. Supongamos que se tiene el árbol A y se quiere convertir en un árbol binario B. El algoritmo de conversión tiene tres pasos fáciles:

1. La raíz de B es la raíz de A.
2. *a)* Enlazar al nodo raíz con el camino que conecta el nodo más a la izquierda (su hijo).
   b) Enlazar este nodo con los restantes descendientes del nodo raíz en un camino, con lo que se forma el nivel 1.
   c) A continuación, repetir los pasos *a)* y *b)* con los nodos del nivel 2, enlazando siempre en un mismo camino todos los hermanos —descendientes del mismo nodo—. Repetir estos pasos hasta llegar al nivel más alto.
3. Girar el diagrama resultante 45° para diferenciar entre los subárboles izquierdo y derecho.

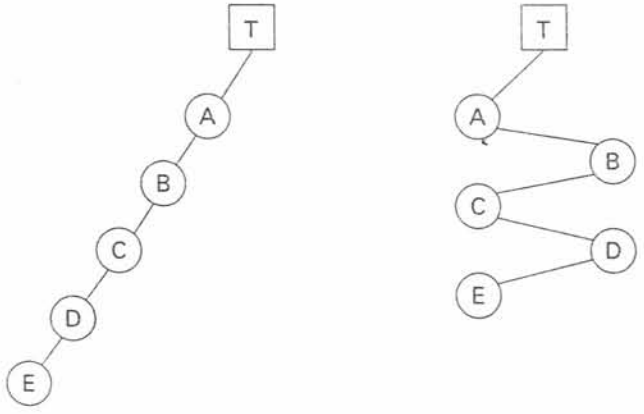
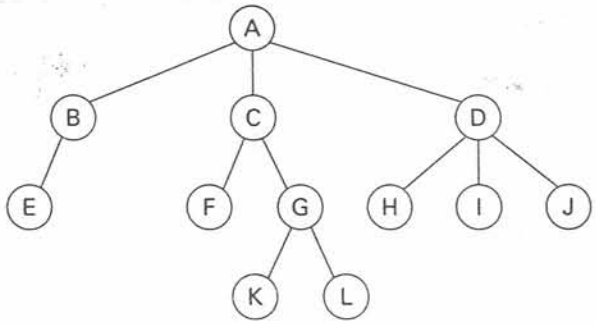


Figura 12.10. Árboles degenerados.

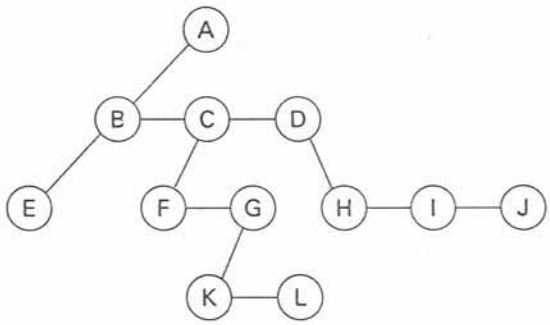
Ejemplo 12.1

Convertir el árbol general T *en un árbol binario.*

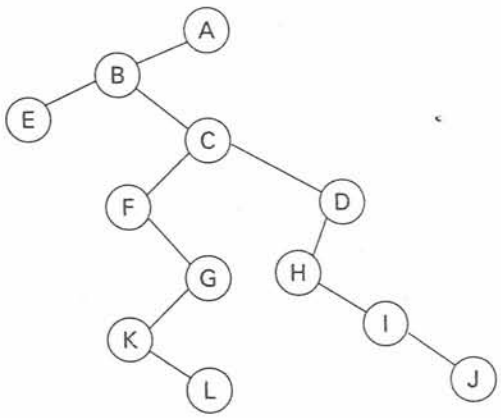


Siguiendo los pasos del algoritmo.

Paso 1:



Paso 2:



Paso 3:

Obsérvese que no existe camino entre E y F, debido a que no son descendientes del árbol original, ya que ellos tienen diferentes padres, B y C.

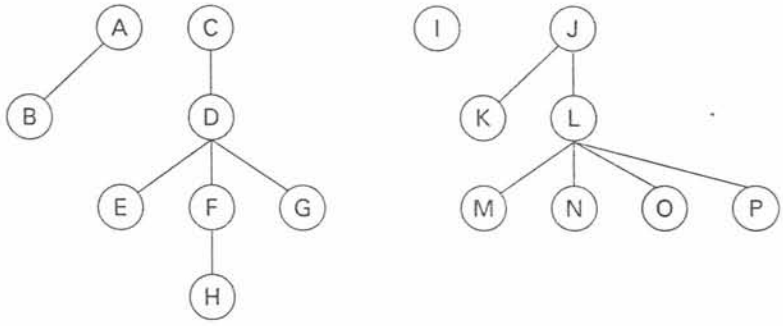
En el árbol binario resultante los punteros izquierdos son siempre desde un nodo padre a su primer hijo (más a la izquierda) en el árbol general original. Los punteros derechos son siempre desde un nodo de sus descendientes en el árbol original.

Ejemplo 12.2

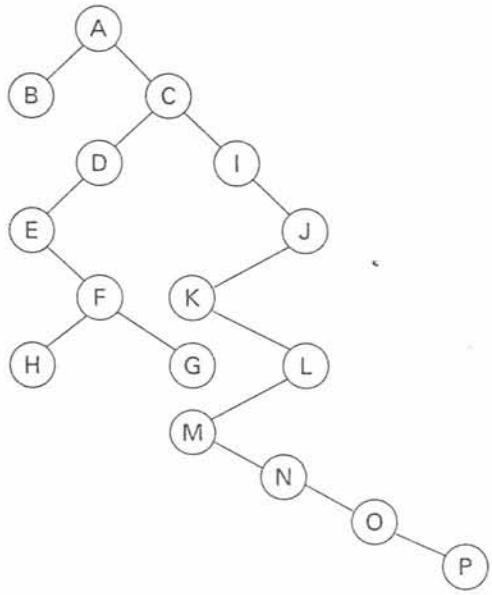
El algoritmo de conversión puede ser utilizado para convertir un bosque de árboles generales a un solo árbol binario.

El bosque siguiente puede ser representado por un árbol binario.

Bosque de árboles:

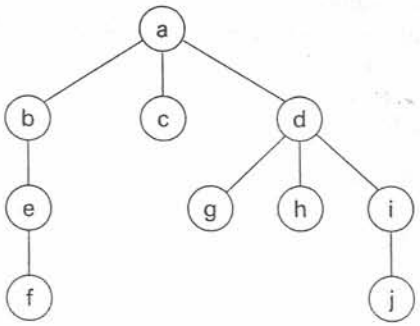


Árbol binario equivalente:

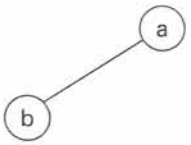


Ejemplo 12.3

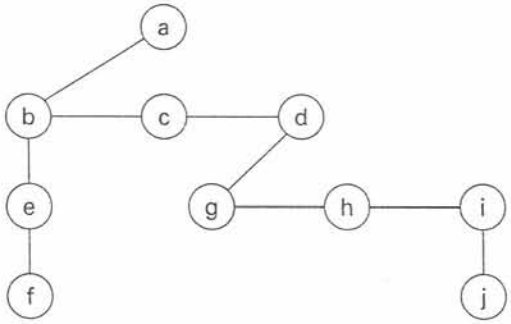
Convertir el árbol general siguiente en árbol binario:



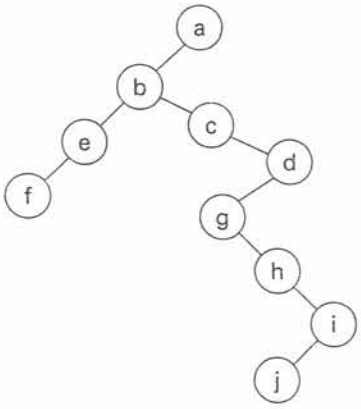
Paso 1:



Paso 2:



Paso 3:



12.3.4. Representación de los árboles binarios

Los árboles binarios pueden ser representados de dos modos diferentes:

- *mediante punteros* (lenguaje Pascal),
- *mediante arrays*.

12.3.4.1. Representación por punteros

Cada nodo de un árbol será un registro que contiene al menos tres campos:

- un campo de datos con un tipo de datos,
- un puntero al nodo del subárbol izquierdo (que puede ser **nulo**-*nil*),
- un puntero al nodo del subárbol derecho (que puede ser **nulo**-*nil*).

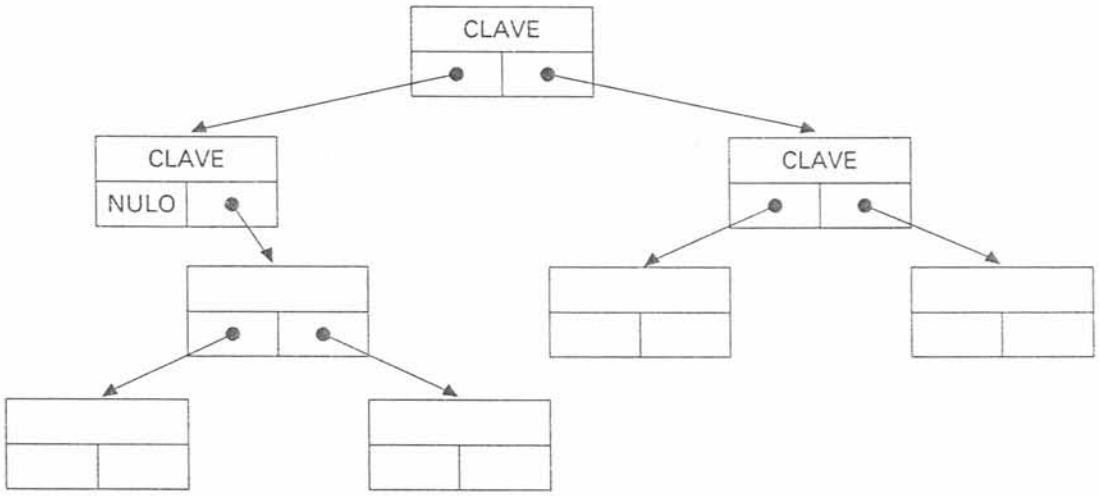
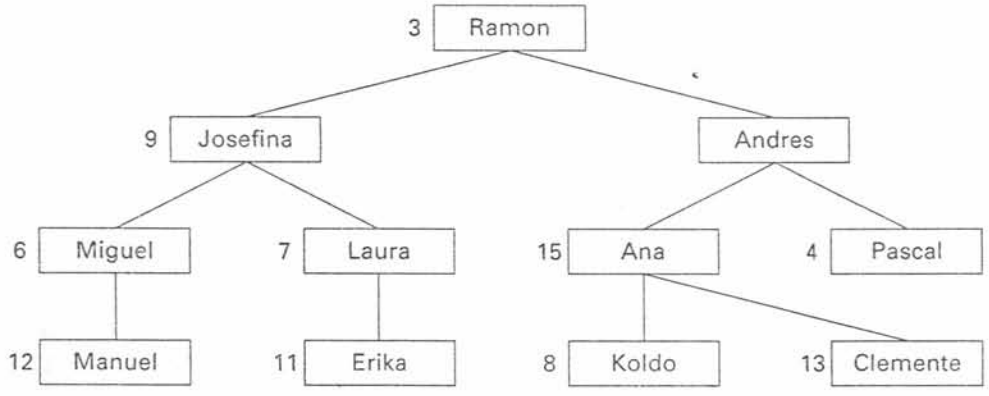


Figura 12.11. Representación de un árbol con punteros.



En lenguaje algorítmico se tendrá:

```
tipo
  puntero_a nodo_arbol: punt
  registro : nodo_arbol
    <tipo_elemento> : elemento
    puntero: subiz, subder
  fin registro
```

12.3.4.2. Representación por listas enlazadas

Mediante una lista enlazada se puede siempre representar el árbol binario de la Figura 12.12.

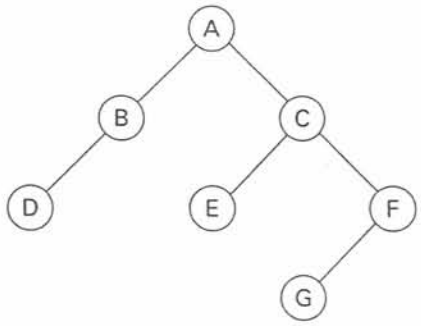


Figura 12.12. Árbol binario.

Nodo del árbol: campo 1 `INFO` (nodo)
campo 2 `IZQ` (nodo)
campo 3 `DER` (nodo)

El árbol binario representado como una lista enlazada se representa en la Figura 12.13.

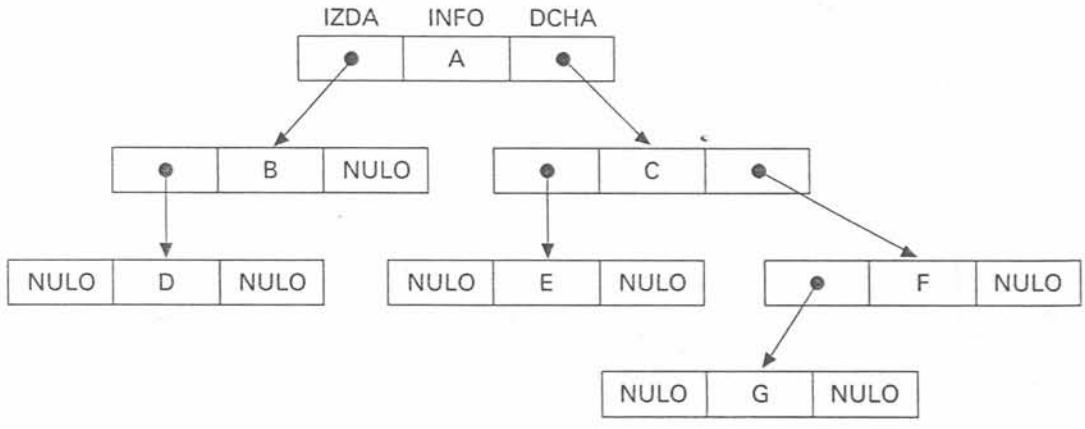


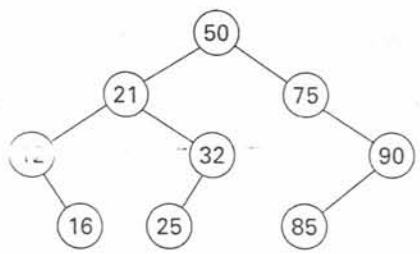
Figura 12.13. Árbol binario como lista enlazada.

12.3.4.3. Representación por arrays

Existen diferentes métodos; uno de los más fáciles es mediante tres arrays lineales paralelos que contemplan el campo de información y los dos punteros de ambos subárboles. Así, por ejemplo, el nodo raíz `RAMON` tendrá dos punteros `IZQ:(9) -JOSEFINA-` y `DER:(16) -ANDRES-`, mientras que el nodo `PASCAL`, al no tener descendientes, sus punteros se consideran cero (`IZQ:0, DER:0`).

Figura 12.14. Árbol binario como arrays.

Otro método resulta más sencillo —un array lineal—. Para ello se selecciona un array lineal ARBOL.



El algoritmo de transformación es:

1. La raíz del árbol se guarda en `ARBOL [1]`.
2. **`Si`** un nodo *n* está en `ARBOL[i]` **`entonces`**
   su hijo izquierdo se pone en `ARBOL[2*i]`
   y su hijo derecho en `ARBOL[2*i+1]`
   Si un subárbol está vacío, se le da el valor `NULO`.

Este sistema requiere más posiciones de memoria que nodos tiene el árbol. Así, la transformación necesitará un array con 2^{h+2} elementos, si el árbol tiene una profundidad *h*. En nuestro caso, como la profundidad es 3, requerirá 32 posiciones (2^5), aunque si no se incluyen las entradas nulas de los nodos terminales, veremos cómo sólo necesita catorce posiciones.

Árbol

1	50
2	21
3	75
4	12
5	32
6	0
7	90
8	0
9	16
10	25
11	
12	
13	
14	85
15	
16	
17	
18	
	. . .

Un tercer método, muy similar al primero, sería la representación mediante un array de registros.

P [3]

	INFO	IZQ.	DER.
1			
2			
3	RAMON	9	16
4	PASCAL	0	0
5			
6	MIGUEL	12	0
7	LAURA	11	0
8	KOLDO	0	0
9	JOSEFINA	6	7
10			
11	ERIKA	0	0
12	MANUEL	0	0
13	CLEMENTE	0	0
14			
15	ANA	8	13
16	ANDRES	15	4

12.3.5. Recorrido de un árbol binario

Se denomina *recorrido de un árbol* el proceso que permite acceder una sola vez a cada uno de los nodos del árbol. Cuando un árbol se recorre, el conjunto completo de nodos se examina.

Existen muchos modos para recorrer un árbol binario. Por ejemplo, existen seis diferentes recorridos generales en un árbol binario, simétricos dos a dos.

Los algoritmos de recorrido de un árbol binario presentan tres tipos de actividades comunes:

- visitar el nodo raíz,
- recorrer el subárbol izquierdo,
- recorrer el subárbol derecho.

Estas tres acciones repartidas en diferentes órdenes proporcionan los diferentes recorridos del árbol. Los más frecuentes tienen siempre en común recorrer primero el subárbol izquierdo y luego el subárbol derecho. Los algoritmos anteriores se llaman *pre-orden*, *post-orden*, *in-orden* y su nombre refleja el momento en que se visita el nodo raíz. En el in-orden el raíz está en el medio del recorrido, en el pre-orden el raíz está el primero y en el post-orden el raíz está el último:

Recorrido pre-orden:

1. Visitar el raíz.
2. Recorrer el subárbol izquierdo en pre-orden.
3. Recorrer el subárbol derecho en pre-orden.

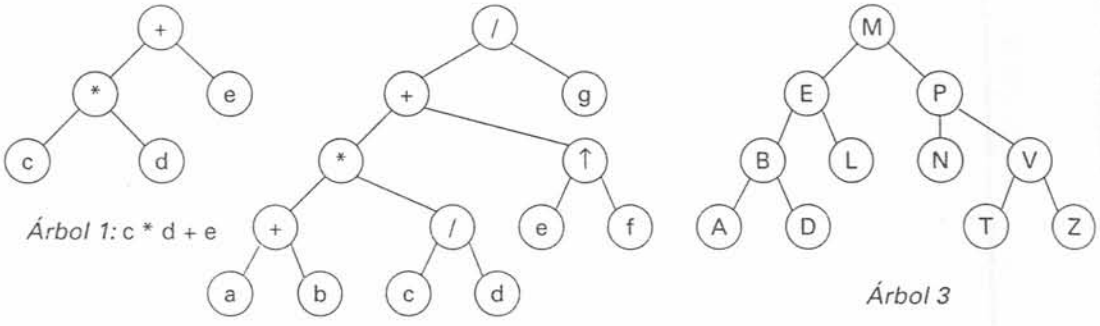
Recorrido en-orden:

1. Recorrer el subárbol izquierdo en in-orden.
2. Visitar el raíz.
3. Recorrer el subárbol derecho en in-orden.

Recorrido post-orden:

1. Recorrer el subárbol izquierdo en post-orden.
2. Recorrer el subárbol derecho en post-orden.
3. Visitar el raíz.

Obsérvese que todas estas definiciones tienen naturaleza recursiva.
En la Figura 12.15 se muestran los recorridos de diferentes árboles binarios.



Árbol 1: c * d + e

Árbol 2: (((a + b) * c/d) + e ↑ f)/g

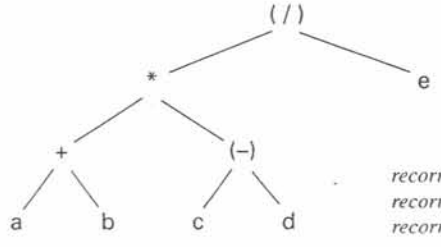
Árbol 3

Figura 12.15. Recorrido de árboles binarios.

Árbol 1	*Pre-orden*	`+ * cde`
	In-orden	`c * d + e`
	Post-orden	`cd * e +`
Árbol 2	*Pre-orden*	`/ + * + ab/cd ^ efg`
	In-orden	`a + b * c/d + e ^ f/g`
	Post-orden	`ab + cd/ * ef ^ + g/`
Árbol 3	*Pre-orden*	`mebadlpnvtz`
	In-orden	`abdelmnptvz`
	Post-orden	`adblentzvpm`

Ejemplo 12.4

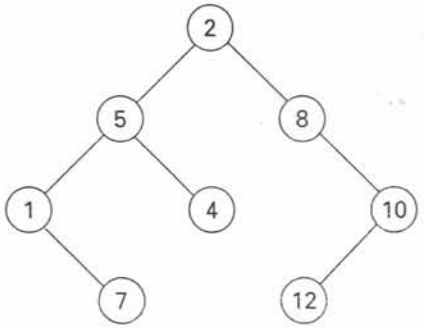
Calcular los recorridos del árbol binario.



recorrido pre-orden	`/ * + ab - cde`
recorrido in-orden	`a + b * c - d/e`
recorrido post-orden	`ab + cd - * e/`

Ejemplo 12.5

Realizar los recorridos del árbol binario.



recorrido pre-orden	2	5	1	7	4	8	10	12
recorrido in-orden	1	7	5	4	2	8	12	10
recorrido post-orden	7	1	4	5	12	10	8	2

12.4. ÁRBOL BINARIO DE BÚSQUEDA

Recordará del Capítulo 9, «*Ordenación, búsqueda e intercalación*», que para localizar un elemento en un array se podía realizar una búsqueda lineal; sin embargo, si el array era grande, una búsqueda lineal era ineficaz por su lentitud, especialmente si el elemento no estaba en el array, ya que requería la lectura completa del array. Se ganaba tiempo si se clasificaba el array y se utilizaba una búsqueda binaria. Sin embargo, en un proceso de arrays las inserciones y eliminaciones son continuas, por lo que el proceso de búsqueda se hará complejo en cualquier método.

En los casos de gran número de operaciones sobre arrays o listas, lo que se necesita es una estructura donde los elementos puedan ser eficazmente localizados, insertados o borrados. Una solución a este problema es una variante del árbol binario que se conoce como *árbol binario de búsqueda* o *árbol binario clasificado* (*binary search tree*).

El árbol binario de búsqueda se construirá teniendo en cuenta las siguientes premisas:

- el primer elemento se utiliza para crear el nodo raíz,
- los valores del árbol deben ser tales que pueda existir un orden (entero, real, lógico o carácter e incluso definido por el usuario si implica un orden),
- en cualquier nodo todos los valores del subárbol izquierdo del nodo son menores o iguales al valor del nodo. De modo similar, todos los valores del subárbol derecho deben ser mayores que los valores del nodo.

Si estas condiciones se mantienen, es sencillo probar que el recorrido *in-orden* del árbol produce los valores clasificados por orden. Así, por ejemplo, en la Figura 12.16 se muestra un árbol binario.

Los tres recorridos del árbol son:

pre-orden	P	F	B	H	G	S	R	Y	T	W	Z
in-orden	B	F	G	H	P	R	S	T	W	Y	Z
post-orden	B	G	H	F	R	W	T	Z	Y	S	P

En esencia, un árbol binario contiene una clave en cada nodo que satisface las tres condiciones anteriores.

Un árbol con las propiedades anteriores se denomina *árbol binario de búsqueda.*

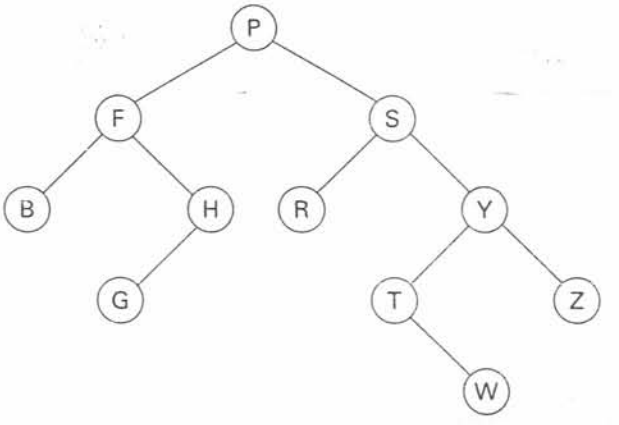


Figura 12.16. Árbol binario.

Ejemplo 12.6

Supongamos que se dispone de un array que contiene los siguientes caracteres:

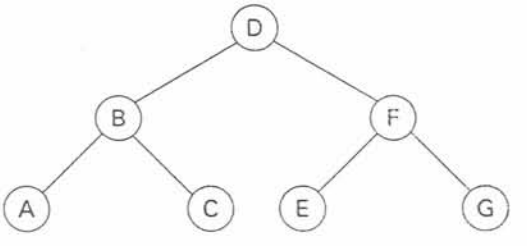
```
D F E B A C G
```

Construir un árbol binario de búsqueda.

Los pasos para la construcción del algoritmo son:

1. Nodo raíz del árbol: `D`.
2. El siguiente elemento se convierte en el descendente derecho, dado que `F` alfabéticamente es mayor que `D`.
3. A continuación, se compara `E` con el raíz. Dado que `E` es mayor que `D`, pasará a ser un hijo de `F` y como `E < F` será el hijo izquierdo.
4. El siguiente elemento `B` se compara con el raíz `D` y como `B < D`, y es el primer elemento que cumple esta condición, `B` será el hijo izquierdo de `D`.
5. Se repiten los pasos hasta el último elemento.

El árbol binario de búsqueda resultante sería:



Ejemplo 12.7

Construir el árbol binario de búsqueda correspondiente a la lista de números

```
4  19  -7  49  100  0  22  12
```

El primer valor, como ya se ha comentado, es el raíz del árbol: es decir, 4. El siguiente valor, 19, se compara con 4; como es más grande, se lleva al subárbol derecho de 4. El siguiente valor, –7, se compara con el raíz y es menor que su valor, 4; por tanto, se mueve al subárbol izquierdo. La Figura 12.17 muestra los sucesivos pasos.

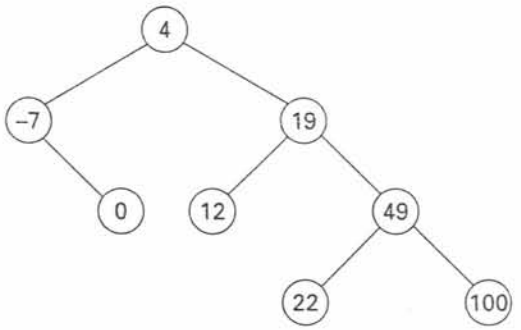
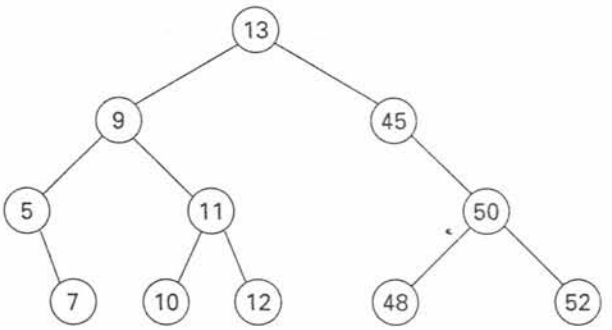


Figura 12.17. Construcción de un árbol binario.

12.4.1. Búsqueda de un elemento

La búsqueda en un árbol binario ordenado es dicotómica, ya que a cada examen de un nodo se elimina aquel de los subárboles que no contiene el valor buscado (valores todos inferiores o todos superiores).



El algoritmo de búsqueda del elemento —clave x— se realiza comparándolo con la clave del raíz del árbol. Si no es el mismo, se pasa al subárbol izquierdo o derecho, según el resultado de la comparación, y se repite la búsqueda en ese subárbol. La terminación del procedimiento se producirá cuando:

- se encuentra la clave,
- no se encuentra la clave; se continúa hasta encontrar un subárbol vacío.

```
procedimiento buscar (E punt: RAIZ E <tipo_elemento>: elemento;
                      S punt: ACTUAL, ANTERIOR)
var
  logico:encontrado

inicio
  encontrado ← falso
  anterior ← (nulo)
  actual ← (raiz)
  mientras no encontrado y (actual<>nulo) hacer
    si ACTUAL -> elemento = elemento entonces
      encontrado ← verdad
    si_no
      anterior ← actual
      si actual  ->elemento > elemento entonces
        actual ← (actual  -> izdo)
      si_no
        actual ← (actual  -> dcho)
      fin_si
    fin_si
  fin_mientras
  si no encontrado entonces
    escribir('no existe', elemento)
  si_no
    escribir( elemento, 'existe')
  fin_si
fin_procedimiento
//< tipo_elemento> en este algoritmo es un tipo de dato SIMPLE
```

12.4.2. Insertar un elemento

Para insertar un elemento en el árbol A se ha de comprobar, en primer lugar, que el elemento no se encuentra en el árbol, ya que su caso no precisa ser insertado. Si el elemento no existe, la inserción se realiza en un nodo en el que al menos uno de los dos punteros IZQ o DER tenga valor **nil**.

Para realizar la condición anterior se desciende en el árbol a partir del nodo `raiz`, dirigiéndose de izquierda a derecha de un nodo, según que el valor a insertar sea inferior o superior al valor del campo clave INFO de este nodo. Cuando se alcanza un nodo del árbol en que no se puede continuar, el nuevo elemento se engancha a la izquierda o derecha de este nodo en función de que su valor sea inferior o superior al del nodo alcanzado.

El algoritmo de inserción del elemento x es:

```
procedimiento insertar (E/S punt:raiz; E <tipo_elemento> : elemento)
var
  punt : nuevo, actual, anterior

inicio
  llamar_a buscar (raiz, elemento,actual, anterior)
  si ACTUAL<> NULO entonces
    escribir ('elemento duplicado')
  si_no
    reservar (nuevo)
    nuevo -> elemento ← (elemento)
    nuevo -> izdo ← (nulo)
    nuevo -> dcho ← (nulo)
```

```
    si anterior = nulo entonces
      raiz ← nuevo
    si_no
      si anterior -> elemento > elemento entonces
        anterior -> izdo ← (nuevo)
      si_no
        anterior -> dcho ← (nuevo)
      fin_si
    fin_si
  fin_si
fin_procedimiento
//< tipo_elemento> es un tipo SIMPLE
```

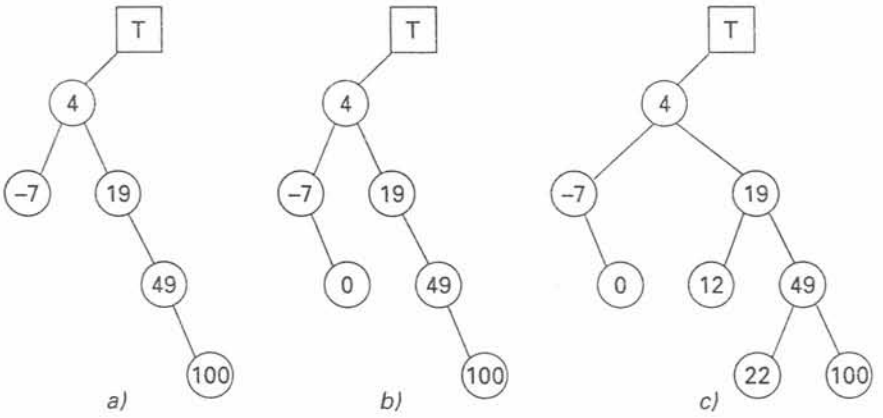


Figura 12.18. Inserciones en un árbol de búsqueda binaria: (a) insertar 100, (b) insertar (0), (c) insertar 22 y 12.

Para insertar el valor X en el árbol binario ordenado se necesitará llamar al subprograma **insertar**.

12.4.3. Eliminación de un elemento

La eliminación de un elemento debe conservar el orden de los elementos del árbol. Se consideran diferentes casos, según la posición del elemento o nodo en el árbol:

- si el elemento es una hoja, se suprime simplemente y se marca nulo en el padre;
- si el elemento no tiene más que un descendiente, se sustituye entonces por ese descendiente;
- si el elemento tiene dos descendientes, se sustituye por el elemento inmediato inferior situado lo más a la derecha posible de su subárbol izquierdo.

Para poder realizar estas acciones, será preciso conocer la siguiente información del nodo a eliminar:

- conocer su posición en el árbol,
- conocer la dirección de su padre,
- conocer si el nodo a eliminar tiene hijos, si son 1 o 2 hijos, y en el caso de que sólo sea uno, si es hijo derecho o izquierdo.

La Figura 12.19 muestra los tres posibles casos de eliminación de un nodo: *a)* eliminar C, *b)* eliminar F, y *c)* eliminar B.

En la Figura 12.20 se muestra el caso de eliminación de un nodo con un subárbol en un gráfico comparativo, antes y después de la eliminación.

En la Figura 12.21 se muestra el caso de la eliminación de un nodo (27) que tiene dos subárboles no nulos.

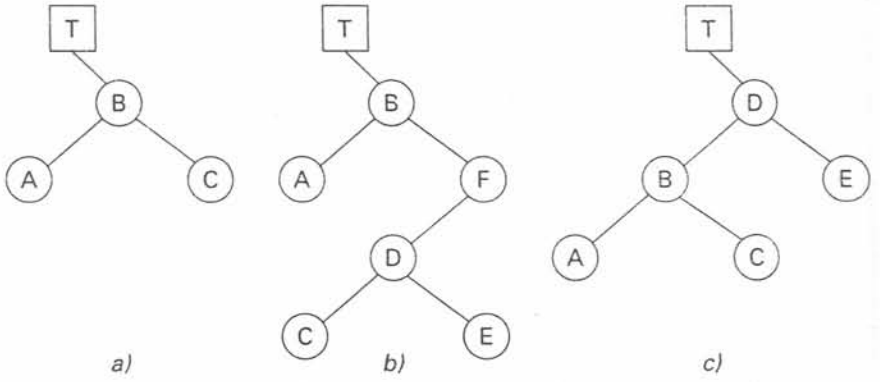


Figura 12.19. Casos posibles de eliminación de un nodo.

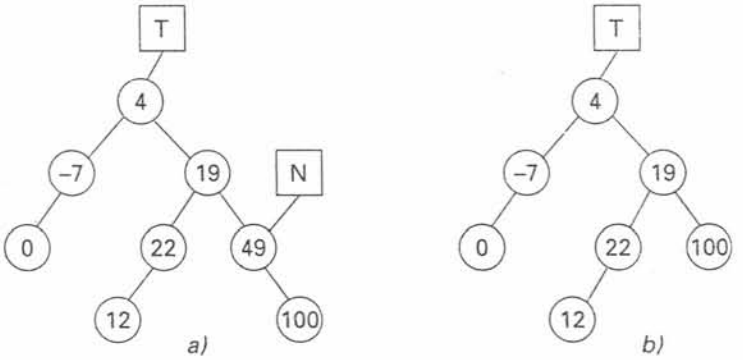


Figura 12.20. Eliminación de un nodo con un subárbol.

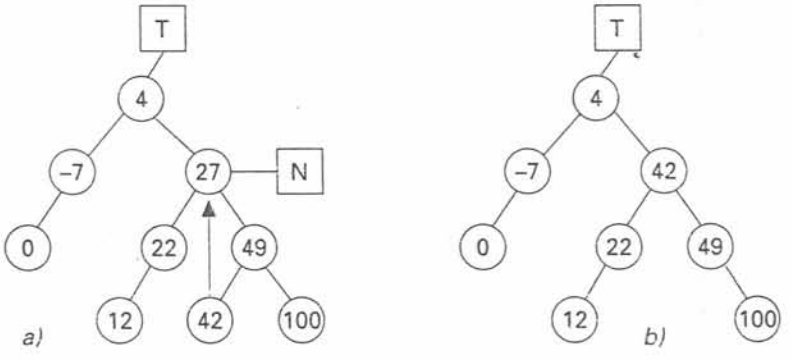
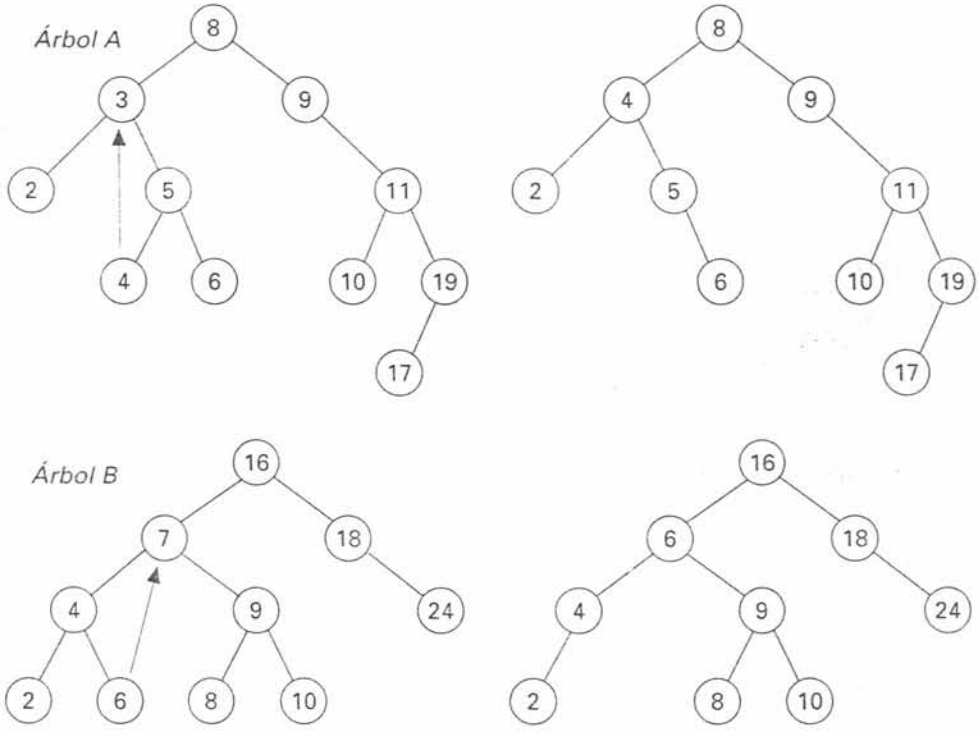


Figura 12.21. Eliminación de un nodo con dos subárboles no nulos.

En este caso se busca el nodo sucesor que, en principio, ha de ser 49 o uno de sus descendientes. Lógicamente el nodo cuyo campo de información le sigue en orden ascendente, es decir, 42, se intercambia entonces con el elemento que se desea borrar, 27.

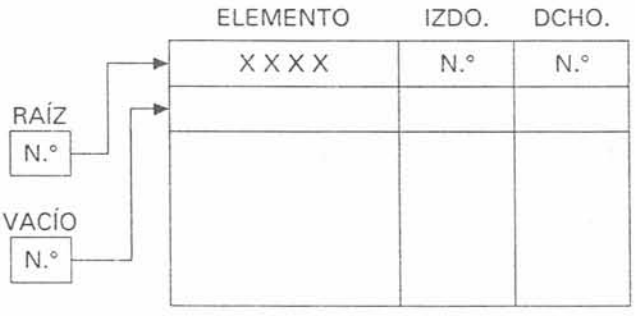
Ejemplo 12.8

Deducir los árboles resultantes de eliminar el elemento 3 *en el árbol* A *y el elemento* 7 *en el árbol* B.



12.4.4. Árbol binario mediante arrays

Los árboles deberán ser tratados como estructuras dinámicas. No obstante, si el lenguaje no tiene punteros podremos simularlos mediante arrays.



IZDO y DCHO serán dos campos numéricos para indicar la posición en que están los hijos izquierdo y derecho. El valor 0 indicaría que no tiene hijo.

Trataremos el array como una lista ENLAZADA y necesitaremos una LISTA DE VACÍOS y una variable VACIO que apunte al primer elemento de la lista de vacíos. Para almacenar la lista de vacíos es indiferente que utilicemos el campo Izdo o Dcho.

Ejemplo 12.9

```
algoritmo arbol_binario_mediante_arrays

const
  Max = <expresion>
tipo
  registro:      TipoElemento
   .... : ....
   .... : ....
  fin_registro
  registro:      TipoNodo
    inicio
     TipoElemento : Elemento
     Entero       : Izdo,Dcho
  fin_registro
  array[1..Max] de TipoNodo : Arr

var
  Arr          : a
  Entero       : opcion
  TipoElemento : elemento
  Entero       : raiz
  Entero       : vacio

inicio
  llamar_a iniciar(a,raiz,vacio)
  repetir
    llamar_a menu
    escribir ('OPCION: ')
    repetir
      leer (opcion)
    hasta_que (opcion >= 0) Y (opcion <= 3)
    segun_sea opcion hacer
      1 :
            llamar_a listado (a,raiz)
            escribir ('INTRODUZCA NUEVO ELEMENTO: ')
            llamar_a proc_leer (elemento)
            llamar_a altas (a, elemento, raiz, vacio)
            llamar_a listado (a,raiz)
            llamar_a pausa
      2 :
            llamar_a listado (a,raiz)
            escribir ('INTRODUZCA ELEMENTO A DAR DE BAJA: ')
            llamar_a proc_leer (elemento)
            llamar_a bajas (a, elemento, raiz, vacio)
            llamar_a listado (a,raiz)
            llamar_a pausa
      3:
            llamar_a listado (a,raiz)
            llamar_a pausa
    fin_segun
  hasta_que opcion = 0
fin
```

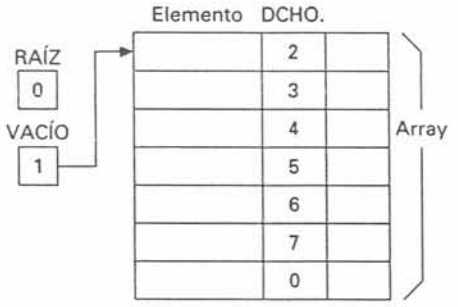
```
procedimiento pausa
var
  cadena : c
inicio
  escribir('PULSE RETURN PARA CONTINUAR')
  leer(c)
fin_procedimiento

procedimiento menu
inicio
  escribir ('1.- ALTAS')
  escribir ('2.- BAJAS')
  escribir ('3.- LISTADO')
  escribir ('0.- FIN')
fin_procedimiento

procedimiento iniciar(S Arr: a ; S Entero: raiz, vacio)
var
  Entero: i
inicio
  raiz ← 0
  vacio ← 1
  desde I ← 1 hasta Max - 1 hacer
    a[i].dcho ← i + 1
  fin_desde
  a[Max].dcho ← 0
fin_procedimiento

logico funcion ArbolVacio(E Entero: raiz)
inicio
  si raiz=0 entonces
      devolver(verdad)
  si_no
      devolver(falso)
  fin_si
fin_funcion

logico funcion ArbolLleno(E Entero: vacio)
inicio
  si vacio=0 entonces
      devolver(verdad)
    si_no
      devolver(falso)
  fin_si
fin_funcion

procedimiento inorden(E/S Arr: a ; E Entero: raiz)
inicio
  si raiz<>0 entonces
    inorden(a,a[raiz].Izdo)
    llamar_a proc_escribir(a[raiz].elemento)
    inorden(a,a[raiz].Dcho)
  fin_si
fin_procedimiento

procedimiento preorden(E/S Arr: A ; E Entero: raiz)
inicio
  si raiz<>0 entonces
    llamar_a proc_escribir(a[raiz].elemento)
    preorden(a,a[raiz].Izdo)
    preorden(a,a[raiz].Dcho)
  fin_si
fin_procedimiento
```

```
procedimiento postorden(E/S Arr: A ; E Entero: raiz)
inicio
  si raiz<>0 entonces
    postorden(a,a[raiz].Izdo)
    postorden(a,a[raiz].Dcho)
    llamar_a proc_escribir(a[raiz].elemento)
  fin_si
fin_procedimiento

procedimiento buscar(E/S Arr: A ; E TipoElemento: elemento S ; Entero: act,ant)
var
  logico: encontrado
inicio
  Encontrado ← falso
  act ← raiz
  ant ← 0
  mientras no encontrado y (act<>0) hacer
    si igual(elemento, a[act].elemento) entonces
      encontrado ← verdad
    si_no
        ant ← act
        si mayor(a[act].elemento, elemento) entonces
          act ← a[act].Izdo
        si_no
          act ← a[act].Dcho
        fin_si
    fin_si
  fin_mientras
fin_procedimiento

procedimiento altas(E/S Arr: A ; E TipoElemento: elemento ;
                    E/S Entero: raiz,vacio)
var
  Entero: act,ant,auxi
inicio
  si vacio <> 0 entonces
    buscar(a, elemento, act, ant)
    si act <> 0 entonces
      escribir('ESE ELEMENTO YA EXISTE')
    si_no
      auxi ← vacio
      vacio ← a[auxi].Dcho
      a[auxi].elemento ← elemento
      a[auxi].Izdo ← 0
      a[auxi].Dcho ← 0
      si ant = 0 entonces
        raiz ← auxi
      si_no
        si mayor(a[ant].elemento, elemento) entonces
          a[ant].Izdo ← auxi
      si_no
          a[ant].Dcho ← auxi
        fin_si
      fin_si
    fin_si
  fin_si
fin_procedimiento
```

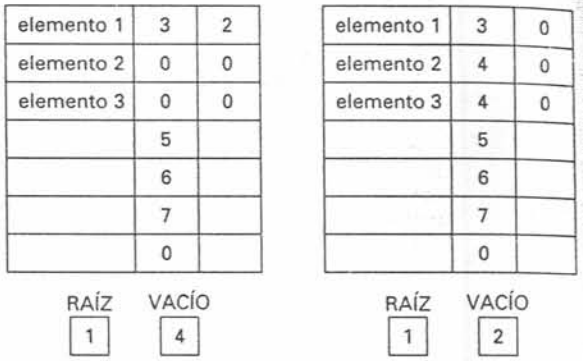
```
procedimiento bajas(E/S Arr: A; E TipoElemento: elemento; E/S Entero: raiz,vacio)
var
  Entero: act,ant,auxi
inicio
  buscar(a, elemento, act, ant)
  si act = 0 entonces
    escribir('ESE ELEMENTO NO EXISTE')
  si_no
    si (a[act].Izdo = 0) y (a[act].Dcho = 0) entonces
      si ant = 0 entonces
        raiz ← 0
      si_no
        si a[ant].Izdo = act entonces
          a[ant].Izdo ← 0
        si_no
          a[ant].Dcho ← 0
        fin_si
      fin_si
   si_no
     si (a[act].Izdo <> 0) y (a[act].Dcho <> 0) entonces
       ant ← act
       auxi ← a[act].Izdo
       mientras a[auxi].Dcho <> 0 hacer
         ant ← auxi
         auxi ← a[auxi].Dcho
       fin_mientras
       a[act].Elemento ← a[auxi].Elemento
       si ant = act entonces
         a[ant].Izdo ← a[auxi].Izdo
       si_no
         a[ant].Dcho ← a[auxi].Izdo
       fin_si
         act  ←  auxi
       si_no
         si a[act].Dcho <> 0 entonces
           si ant = 0 entonces
             raiz ← a[act].Dcho
           si_no
             si a[ant].Izdo = act entonces
               a[ant].Izdo ← a[act].Dcho
             si_no
               a[ant].Dcho ← a[act].Dcho
             fin_si
           fin_si
         si_no
           si ant = 0 entonces
             raiz ← a[act].Izdo
           si_no
             si a[ant].Dcho = act entonces
               a[ant].Dcho ← a[act].Izdo
             si_no
               a[ant].Izdo ← a[act].Izdo
           fin_si
         fin_si
       fin_si
     fin_si
   fin_si
   a[act].Dcho ← vacio
   vacio ← act
 fin_si
fin_procedimiento
```

```
procedimiento listado (E Arr: a ; E Entero: raiz)
inicio
  escribir ('INORDEN: ')
  inorden (a, raiz)
  escribir ('PREORDEN: ')
  preorden (a, raiz)
  escribir ('POSTORDEN: ')
  postorden (a, raiz)
fin_procedimiento
```

Iniciar



Dada la siguiente situación eliminar el elemento 2



12.5. GRAFOS

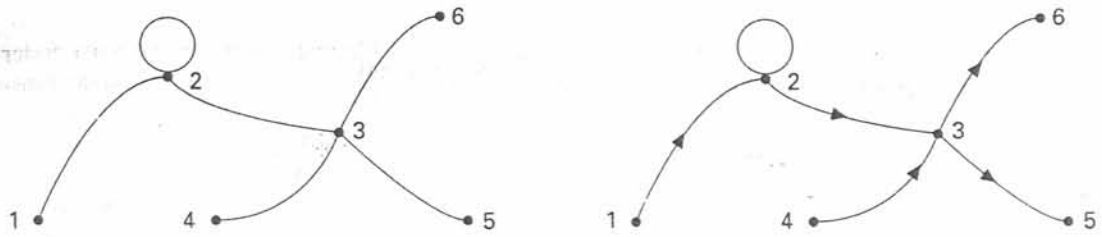
Los grafos son otra estructura de datos no lineal y que tiene gran número de aplicaciones. El estudio del análisis de grafos ha interesado a los matemáticos durante siglos y representa una parte importante de la teoría combinatoria en matemáticas. Aunque la teoría de grafos es compleja y amplia, en esta sección se realizará una introducción a la teoría de grafos y a los algoritmos que permiten su solución por computadora.

Los árboles binarios representan estructuras jerárquicas con limitaciones de dos subárboles por cada nodo. Si se eliminan las restricciones de que cada nodo puede apuntar a dos nodos —como máximo— y que cada nodo puede estar apuntado por otro nodo —como máximo— nos encontramos con un grafo.

Ejemplos de grafos en la vida real los tenemos en la red de carreteras de un estado o región, la red de enlaces ferroviarios o aéreos nacionales, etc.

En una red de carreteras los nudos de la red representan los *vértices* del grafo y las carreteras de unión de dos ciudades los arcos, de modo que a cada arco se asocia una información, tal como la distancia, el consumo en gasolina por automóvil, etc.

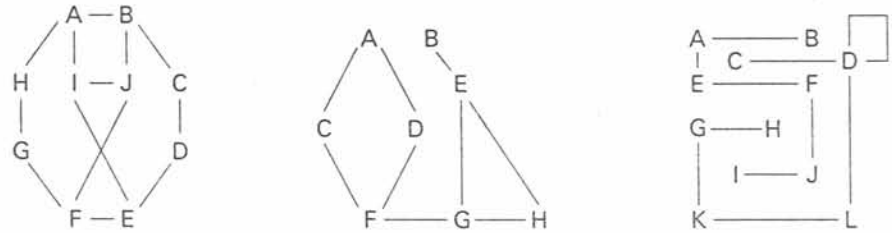
Los grafos nos pueden ayudar a resolver problemas como éste. Supongamos que ciertas carreteras del norte del Estado han sido bloqueadas por una reciente tormenta de nieve. ¿Cómo podemos saber si todas las ciudades de ese Estado se pueden alcanzar por carretera desde la capital o si existen ciudades aisladas? Evidentemente, existe la solución del estudio de un mapa de carreteras; sin embargo, si existen muchas ciudades, la obtención de la solución puede ser ardua y costosa en tiempo. Una computadora y un algoritmo adecuado de grafos solucionarán fácilmente el problema.



a) no dirigido *b)* dirigido

Figura 12.25. Grafo: (a) no dirigido, (b) dirigido.

grafo conectado: existe siempre un camino que une dos vértices cualesquiera.
grafo desconectado: existen vértices que no están unidos por un camino.

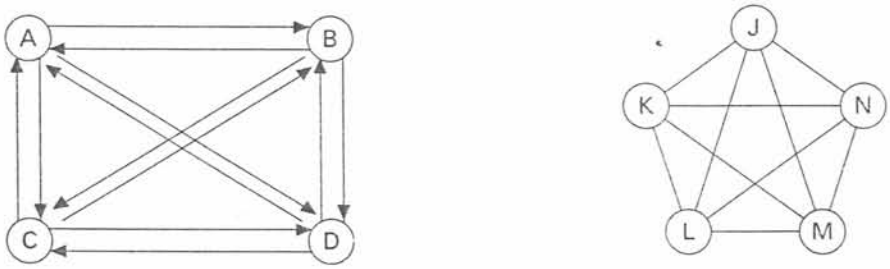


a) conectado *b)* conectado *c)* no conectado

Figura 12.26. Grafos conectados y no conectados.

Otros tipos de grafos de gran interés se muestran en la Figura 12.27. Un *grafo completo* es aquél en que cada vértice está conectado con todos y cada uno de los restantes nodos. Si existen n vértices, habrá $n(n-1)$ aristas en un grafo completo y dirigido, y $n(n-1)/2$ aristas en un grafo no dirigido completo.

Un *grafo ponderado o con peso* es aquél en el que cada arista tiene un valor. Los grafos con peso suelen ser muy importantes, ya que pueden representar situaciones de gran interés; por ejemplo, los



a) grafo completo dirigido *b)* grafo completo no dirigido

Figura 12.27. Grafos completos.

vértices pueden ser ciudades y las aristas distancias o precios del pasaje de ferrocarril o avión entre ambas ciudades. Eso nos puede permitir calcular cuál es el recorrido más económico entre dos ciudades, sumando los importes de los billetes de las ciudades existentes en el camino y así poder tomar una decisión acertada respecto al viaje e incluso estudiar el posible cambio de medio de transporte: avión o automóvil, si éstos resultan más baratos.

La solución de encontrar el camino más corto, el de menor precio o más económico entre dos vértices de un grafo es un algoritmo importante en la teoría de grafos. (El *algoritmo de Dijkstra* es un algoritmo tipo para la solución de dichos problemas.)

12.5.2. Representación de grafos

Existen dos técnicas estándar para representar un grafo G: *la matriz de adyacencia* (mediante arrays) y *la lista de adyacencia* (mediante punteros/listas enlazadas).

12.5.2.1. Matriz de adyacencia

La matriz de adyacencia M es un array de dos dimensiones que representa las conexiones entre pares de verticales. Sea un grafo G con un conjunto de nodos V_G y un conjunto de aristas A_G. Supongamos que el grafo es de orden N, donde N >= 1. La matriz de adyacencia M se representa por un array N × N, donde:

$$M(i,j) = \begin{cases} 1 \text{ si existe una arista } (V_i, V_j) \text{ en } A_G, V_i \text{ es adyacente a } V_j, \\ 0, \text{ en caso contrario.} \end{cases}$$

Las columnas y las filas de la matriz representan los vértices del grafo. Si existe una arista desde i a j (esto es, el vértice i es adyacente a j), el coste o peso de la arista de i a j se introduce; si no existe la arista, se introduce un 0; lógicamente, los elementos de la diagonal principal son todos ceros, ya que el coste de la arista i a i es 0.

Si G es un grafo no dirigido, la matriz es simétrica M(i,j) = M(j,i). La matriz de adyacencia del grafo de la Figura 12.25 se indica en la Figura 12.28.

i \ j	1	2	3	4	5	6
1	0	1	0	0	0	0
2	1	0	1	0	0	0
3	0	1	0	1	1	1
4	0	0	1	0	0	0
5	0	0	1	0	0	0
6	0	0	1	0	0	0

Figura 12.28. Matriz de adyacencia.

12.5.1. Terminología de grafos

Formalmente, un *grafo* es un conjunto de puntos —una estructura de datos— y un conjunto de líneas, cada una de las cuales une un punto a otro. Los puntos se llaman *nodos* o *vértices* del grafo, y las líneas se llaman *aristas* o *arcos* (*edges*).

Se representan el conjunto de vértices de un grafo dado G por V_G, y el conjunto de arcos por A_G. Por ejemplo, en el grafo G de la Figura 12.23:

$V_G = \{a, b, c, d\}$

y

$A_G = \{1, 2, 3, 4, 5, 6, 7, 8\}$

El número de elementos de V_G se llama *orden* del grafo. Un *grafo nulo* es un grafo de orden cero.

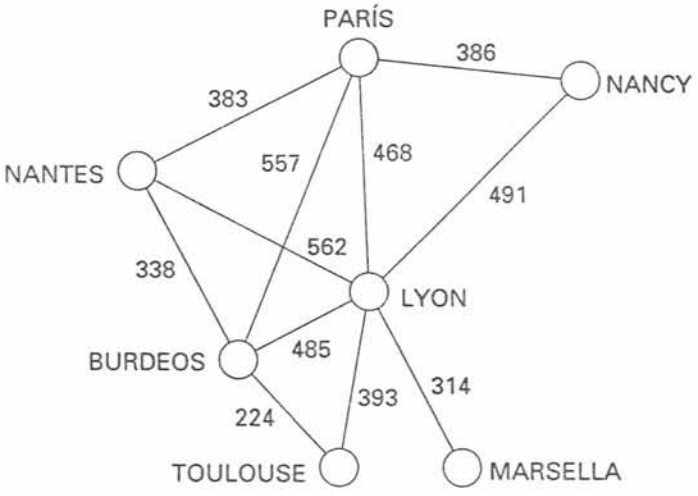


Figura 12.22. Grafo de una red de carreteras.

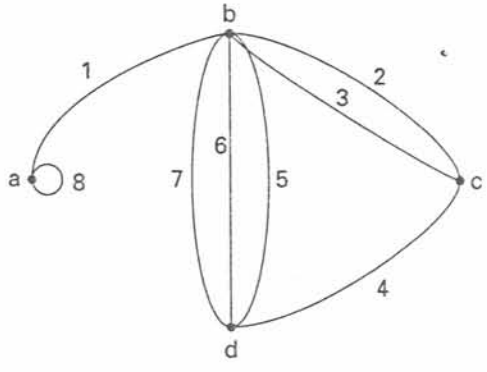


Figura 12.23. Grafo G.

Una arista se representa por los vértices que conecta. La arista 3 conecta los vértices b y c, y se representa por A(b, c). Algunos vértices pueden conectar un nodo consigo mismo; por ejemplo, el vértice 8 tiene el formato V(a, a). Estas aristas se denominan *bucles* o *lazos*.

Un grafo G se denomina *sencillo* si se cumplen las siguientes condiciones:

- no tiene lazos, no existe un arco en A_G de la forma (V, V), donde está en V_G.
- no existe más que un arco para unir dos nodos, es decir, no existe más que un arco (V_1, V_2) para cualquier par de vértices V_1, V_2.

En la Figura 12.24 se representa un grafo sencillo.

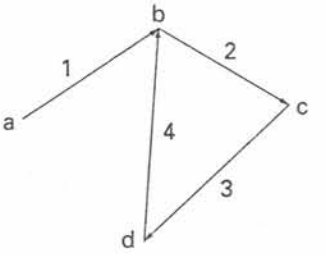


Figura 12.24. Grafo sencillo.

Un grafo que no es sencillo se denomina *grafo múltiple*.

Un *camino* es una secuencia de uno o más arcos que conectan dos nodos. Representaremos por C(V_i, V_j) un camino que conecta los vértices V_1 y V_2.

La *longitud* de un camino es el número de arcos que comprende. En el grafo de la Figura 12.24 existen los siguientes caminos entre los nodos b y d.

```
C(b,d) = (b,c) (c,d)                    longitud = 2
C(b,d) = (b,c) (c,b) (b,c) (c,d)        longitud = 4
C(b,d) = (b,d)                          longitud = 1
C(b,d) = (b,d) (c,b) (b,d)              longitud = 3
```

Dos vértices se dice que son *adyacentes* si hay un arco que los une. Así, V_i y V_j son adyacentes si existe un camino que los une. Esta definición es muy general y normalmente se particulariza; si existe un camino desde A a B, decimos que A *es adyacente a* B, y B *es adyacente desde* A. Así, en el grafo de la Figura 12.25, Las Vegas es adyacente a Nueva York, pero Nueva York no es adyacente a Las Vegas.

Se consideran dos tipos de grafos:

dirigidos: los vértices apuntan unos a otros; los arcos están dirigidos o tienen dirección,

no-dirigidos: los vértices están relacionados, pero no se apuntan unos a otros; la dirección no es importante.

En la Figura 12.5 el grafo es dirigido, dado que la dirección es importante; así, existe un vuelo entre Las Vegas y Nueva York, pero no en sentido contrario.

Si el grafo fuese dirigido, su matriz resultante sería:

i \ j	1	2	3	4	5	6
1	0	1	0	0	0	0
2	0	0	1	0	0	0
3	0	0	0	0	1	1
4	0	0	1	0	0	0
5	0	0	0	0	0	0
6	0	0	0	0	0	0

Ejemplo 12.10

Deducir la matriz de adyacencia del grafo siguiente:

La matriz de adyacencia resultante de este grafo, cuyos vértices representan ciudades y los pesos de las aristas, los precios de pasajes de avión en dólares, es:

	SF	LA	LV	KC	NY
SF		1000			
LA	1000		390	2000	
LV		390		3000	2500
KC		2000	3000		
NY	1500			3500	

Ejemplo 12.11

Sea un grafo con aristas ponderadas. Los vértices representan ciudades y las aristas las rutas utilizadas por los camiones de una empresa de transporte de mercancías. Cada arista está rotulada con la distancia entre las parejas de ciudades enlazadas directamente.

En este caso utilizaremos una matriz triangular.

NOTA: Obsérvese como consecuencia de este ejemplo que las aristas ponderadas tienen una gran aplicación.

- En transporte, comúnmente, representan distancias, precios de billetes, tiempos.
- En hidráulica, capacidades. Por ejemplo, el caudal de un oleoducto entre diferentes ciudades, litros/segundo.

12.5.2.2. Lista de adyacencia

El segundo método utilizado para representar grafos es útil cuando un grafo tiene muchos vértices y pocas aristas; es la *lista de adyacencia*. En esta representación se utiliza una lista enlazada por cada vértice *v* del grafo que tenga vértices adyacentes *desde* él.

El grafo completo incluye dos partes: un directorio y un conjunto de listas enlazadas. Hay una entrada en el directorio por cada nodo del grafo. La entrada en el directorio del nodo *i* apunta a una lista enlazada que representa los nodos que son conectados al nodo *i*. Cada registro de la lista enlazada tiene dos campos: uno es un identificador de nodo, otro es un enlace al siguiente elemento de la lista; la lista enlazada representa arcos.

Una lista de adyacencia del grafo de la Figura 12.25 se da en la Figura 12.29.

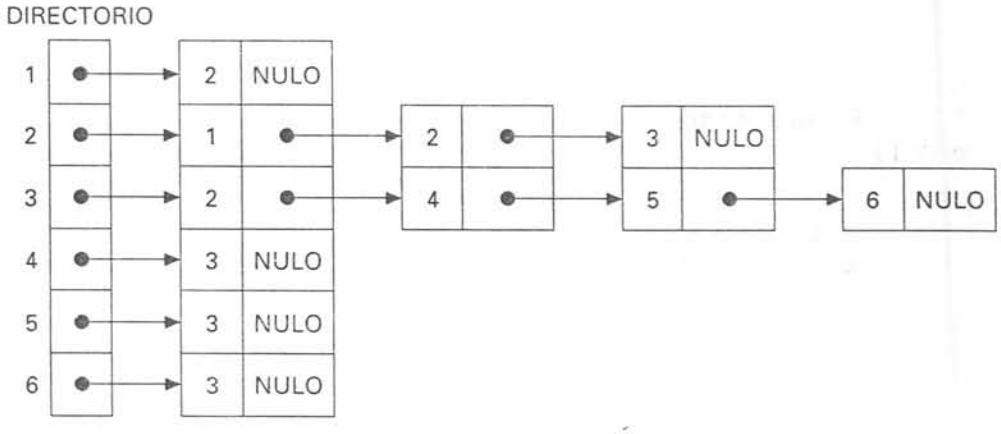


Figura 12.29. Lista de adyacencia.

Un grafo no dirigido de orden N con A arcos requiere N entradas en el directorio y 2 * A entradas de listas enlazadas, excepto si existen bucles que reducen el número de listas enlazadas en 1.

Un grafo dirigido de orden N con A arcos requiere N entradas en el directorio y A entradas de listas enlazadas.

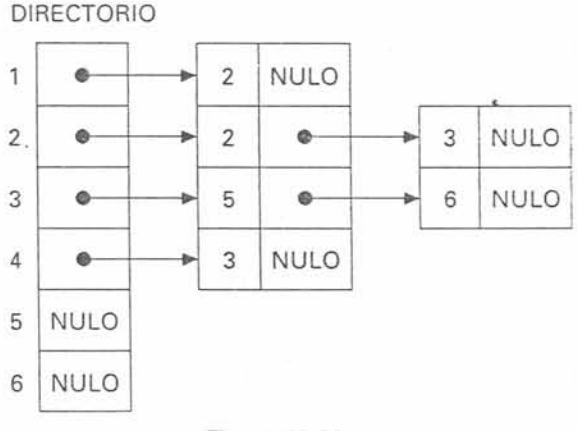
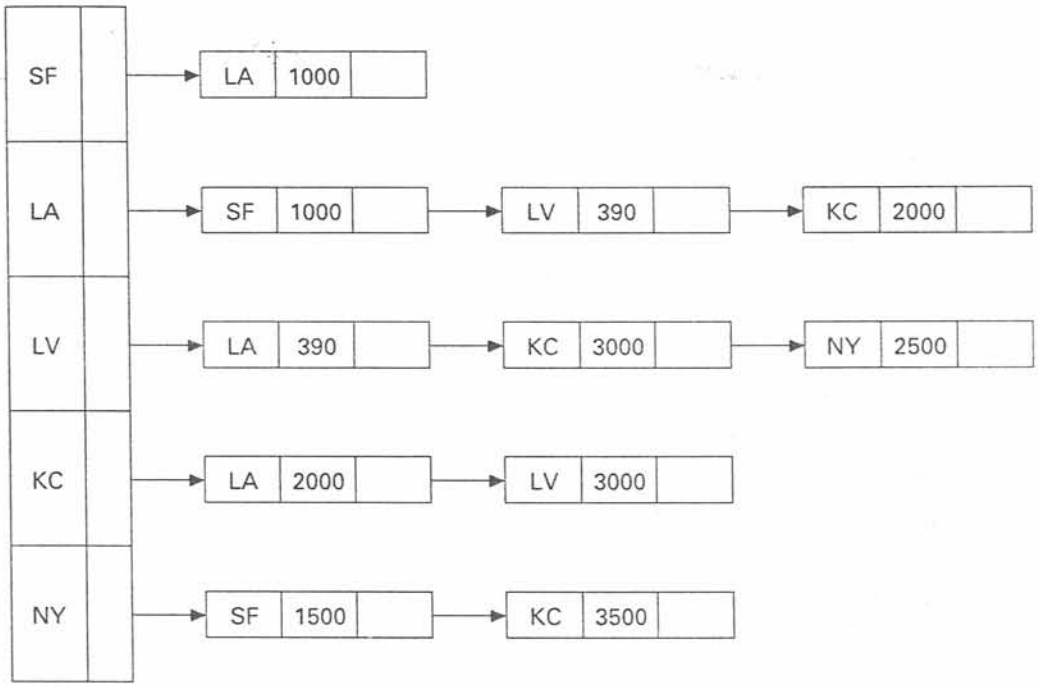


Figura 12.30.

Ejemplo 12.12

La lista de adyacencia del grafo del ejemplo 12.9 es:



La elección de la representación depende del algoritmo particular que se vaya a implementar y si el grafo es «disperso» o «denso». Un grafo disperso es un grafo en el que el número de vértices N es mucho mayor que el número de arcos. En un grafo denso el número de arcos se acerca al máximo.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

12.1. Teniendo en cuenta que nuestro lenguaje de programación no maneja estructuras dinámicas de datos escribir un procedimiento que inserte un nuevo nodo en un árbol binario en el lugar correspondiente según su valor. Escribir otro procedimiento que permita conocer el número de nodos de un árbol binario. Utilizar ambos procedimientos desde un algoritmo que cree el árbol y nos informe sobre su número de nodos.

Análisis del problema

El procedimiento de inserción será análogo al de altas que aparece en el primer ejercicio.
Para conocer el número de nodos del árbol realizaremos su recorrido, por uno cualquiera de los métodos ya comentados —inorden, preorden y postorden— y los iremos contando.
El programa principal comenzará con un proceso de inicialización, a continuación utilizará una estructura repetitiva que permita la inserción de número indeterminado de nodos en el árbol y, por último, llamará al procedimiento para contarlos.

Al no especificarse en el enunciado el tipo de información que se almacena en los registros del árbol, trataremos ésta de forma genérica, recurriendo a procedimientos y funciones auxiliares, no desarrollados, que nos permitan manipularla, como `leerelemento (elemento)`, `escribirelemento(elemento)`, `distinto(elemento,'0')`.

Diseño del algoritmo

```
algoritmo Ejercicio_12_1_ap
const
  Max = ....
tipo
  registro : tipoelemento
    .... : ....
    .... : ....
  fin_registro
  registro : tiponodo
    tipoelemento : elemento
    entero       : izdo,dcho
  fin_registro
  array[1..Max] de tiponodo : arr
var
  arr          : a
  tipoelemento : elemento
  entero       : raiz, vacio
inicio
  iniciar(a,raiz,vacio)
  escribir ('Introduzca nuevo elemento :')
  si no arbollleno(vacio) entonces
    llamar_a leerelemento (elemento)
  fin_si
  mientras distinto(elemento,'0') y no arbollleno(vacio) hacer
    altas (a, elemento, raiz, vacio)
    si no arbollleno(vacio) entonces
      escribir ('Introduzca nuevo elemento:')
      llamar_a leerelemento (elemento)
    fin_si
  fin_mientras
  listado (a,raiz)
fin

procedimiento iniciar(S arr: a ; S entero: raiz, vacio)
var
  entero : i
inicio
  raiz ← 0
  vacio ← 1
  desde i ← 1 hasta max 1 hacer
    a[i].dcho ← i + 1
  fin desde
  a[max].dcho ← 0
fin_procedimiento

logico funcion arbolleno (E entero : vacio)
inicio
  si vacio = 0 entonces
     devolver(verdad)
  si_no
     devolver(falso)
```

```
  fin_si
fin_funcion

procedimiento inorden(E arr: a; E entero: raiz; E/S entero: cont)
inicio
  si raiz<>0 entonces
     inorden(a, a[raiz].izdo, cont)
     cont ← cont + 1
     llamar_a escribirelemento(a[raiz].elemento)
     // Ademas de contar los nodos visualiza la
     // informacion almacenada en ellos
     inorden(a, a[raiz].dcho, cont)
  fin_si
fin_procedimiento

procedimiento buscar(E arr: a; E tipoelemento:elemento; S entero: act,ant)
var
  logico: encontrado
inicio
  encontrado ← falso
  act ← raiz
  ant ← 0
  mientras no encontrado y (act<>0) hacer
    si igual(elemento, a[act].elemento) entonces
       encontrado ← verdad
    si_no
       ant ← act
       si mayor(a[act].elemento, elemento) entonces
          act ← a[act].izdo
       si_no
          act ← a[act].dcho
       fin_si
    fin_si
  fin_mientras
fin_procedimiento

procedimiento altas (E/S arr: a; E tipoelemento: elemento;
                     E/S entero : raiz, vacio)
var
  entero: act, ant, axi
inicio
  si_no arbollleno(vacio) entonces
     buscar(a, elemento, act, ant)
     si act<>0 entonces
        escribir('Ese elemento ya existe')
     si_no
        auxi ← vacío
        vacio ← a[auxi].dcho
        a[auxi].elemento ← elemento
        a[auxi].izdo ← 0
        a[auxi].dcho ← 0
        si ant = 0 entonces
           raiz ← auxi
        si_no
           si mayor(a[ant].elemento, elemento) entonces
             a[ant].izdo ← auxi
           si_no
             a[ant].dcho ← auxi
```

```
            fin_si
          fin_si
        fin_si
    fin_si
fin_procedimiento

procedimiento listado (E arr: a; E entero: raiz)
var
  cont:entero
inicio
  escribir ('inorden:')
  cont ← 0
  inorden (a, raiz, cont)
  escribir('El numero de nodos es', cont)
fin_procedimiento
```

12.2. Escribir un procedimiento que nos permita contar las hojas de un árbol mediante estructuras dinámicas.

Análisis del problema

Recorrer el árbol, contando, únicamente, los nodos que no tienen hijos.

```
procedimiento contarhojas (E punt: raiz; E/S entero: cont)
inicio
  si raiz <> nulo entonces
     si (raiz ←.izdo=nulo) y (raiz ←.dcho=nulo) entonces
           cont ← cont + 1
     fin_si
      contarhojas(raiz ->.izdo,cont)
      contarhojas(raiz ->.dcho,cont)
  fin_si
fin_procedimiento
```

12.3. Diseñar una función que nos permita comprobar si son iguales dos árboles cuyos nodos tienen la siguiente estructura:

```
tipo
  puntero_a nodo: punt
  registro : nodo
    entero : elemento
    punt : izdo,dcho
  fin_registro
```

Análisis del problema

Se trata de una función recursiva que compara nodo a nodo la información almacenada en ambos árboles. Las condiciones de salida del proceso recursivo serán:

- Que se termine de recorrer uno de los dos árboles.
- Que terminen de recorrerse ambos.
- Que encontremos diferente información en los nodos comparados.

Si los árboles terminaron de recorrerse simultáneamente es que ambos tienen el mismo número de nodos y nunca ha sido diferente la información comparada, por tanto, la función devolverá `verdad`, en cualquier otro caso la función devolverá `falso`.

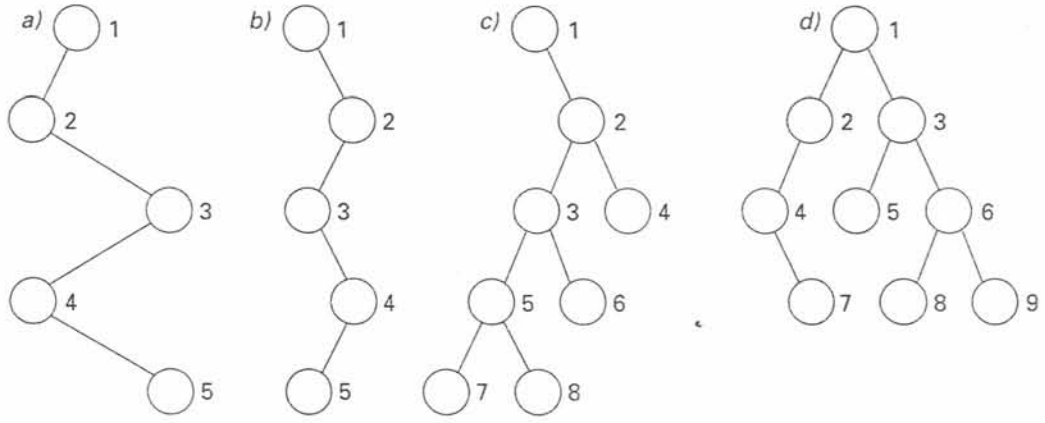
```
logico funcion iguales (E punt: raiz1, raiz2)
inicio
    si raiz1 = nulo entonces
      si raiz2 = nulo entonces
         devolver(verdad)
      fin_si
    si_no
      si raiz2 = nulo entonces
         devolver(falso)
      si_no
      si raiz1 ->.elemento<> raiz2 ->.elemento entonces
         devolver(falso)
      si_no
         devolver (iguales(raiz1 ->.izdo, raiz2 ->.izdo)
            ➥ y iguales(raiz1 ->.dcho, raiz2 ->.dcho))
      fin_si
    fin_si
  fin_si
fin_función
```

EJERCICIOS

12.1. Dado un árbol binario de números enteros ordenados, se desea un subalgoritmo que busque un elemento con un proceso recursivo.

12.2. Diseñar un subalgoritmo que busque un elemento en un árbol binario de números enteros ordenados. realizado con un proceso repetitivo.

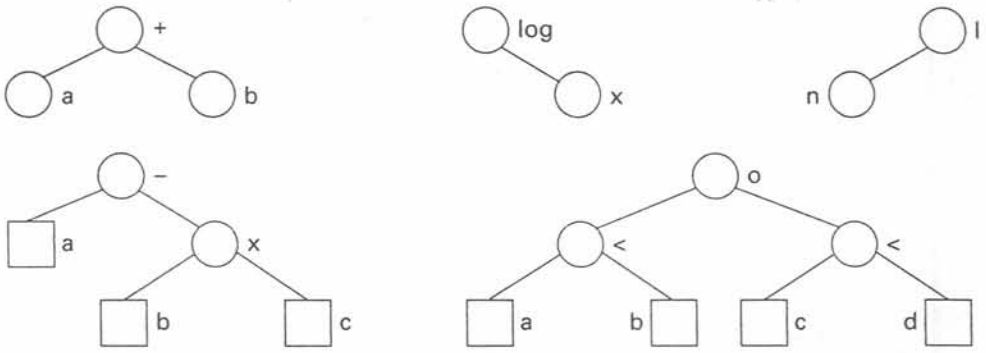
12.3. Describir el orden en el que los vectores de los siguientes árboles binarios serán visitados en: a) pre-orden, b) en-orden, y c) post-orden.



12.4. Dibujar la expresión árbol para cada una de las siguientes expresiones y dar el orden de visita a los nodos en: a) pre-orden, b) en-orden, y c) post-orden:

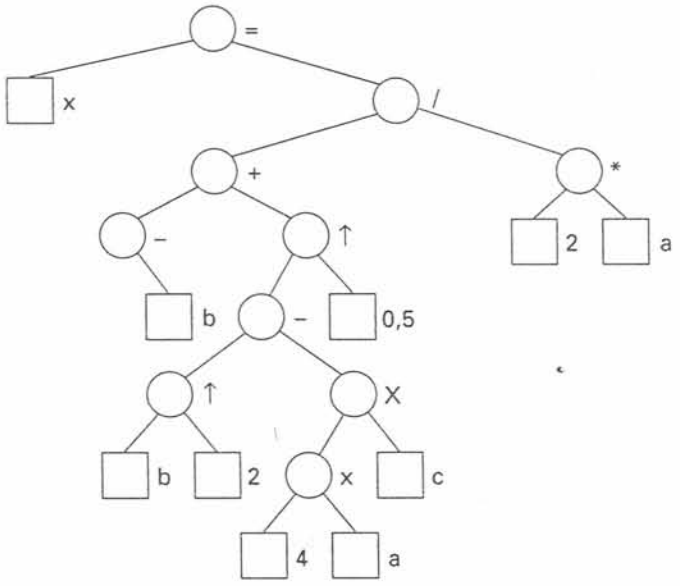
1. log n!
2. (a - b) - c
3. a - (b - c)
4. (a < b) y (b < c) y (c < d)

12.5. Escribir un subalgoritmo recursivo que liste los nodos de un árbol binario en pre-orden.
12.6. Escribir un subalgoritmo que elimine un nodo determinado de un árbol de enteros.
12.7. Se dispone de un árbol de números reales desordenados y se desea escribir un subalgoritmo que inserte un nodo en el lugar correspondiente de acuerdo a su valor.
12.8. Escribir un subalgoritmo que permita conocer el número de nodos de un árbol binario.
12.9. Deducir las fórmulas de las expresiones representadas por los siguientes árboles de expresión:




```
a + b
log x
n!
a - (b * c)
(a < b) o (c < d)
```

12.10. Deducir la fórmula que representa el siguiente árbol expresión:

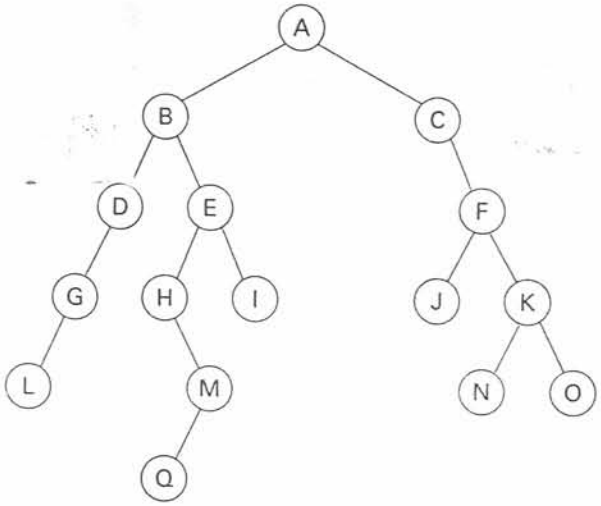


$x \leftarrow (-b + (b2 - 4 * a * c)^{0.5}) / (2 * a)$

es decir, las raíces solución de la ecuación cuadrática o de segundo grado:

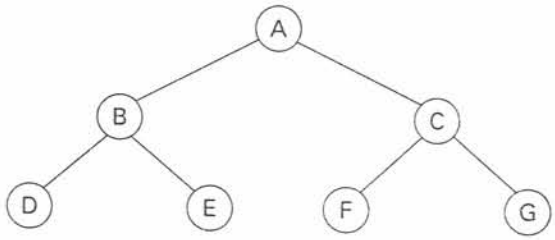
$ax^2 + bx + c = 0$.

12.11. Considerar el árbol binario



Listar los nodos del árbol en: a) pre-orden, b) en-orden, y c) post-orden.

12.12. Transformar el siguiente árbol general en un árbol binario:



CAPÍTULO 13

DISEÑO EFECTIVO DE PROGRAMAS Y APLICACIONES ALGORÍTMICAS

CONTENIDO

13.1. Estilo de programación.
13.2. Estructura del programa.
13.3. Estructura modular y diseño descendente.
13.4. Diseño de programas.
13.5. Aplicaciones de gestión (análisis).

EJERCICIOS.

El profesor Niklaus Wirt —diseñador del Pascal y Modula-2— considera la programación como *un método a seguir y unas técnicas de aprendizaje*, y como soporte creó Pascal, un lenguaje de programación para enseñar buenos métodos y hábitos de programación. El profesor D. E. Knuth —autor de la más completa obra de algoritmos, *The Art of Computer Programming*— prefiere considerar la programación como un *arte*. ¿Qué camino seguir si dos genios de la informática nos plantean esa disyuntiva? Sin duda, pensamos que ambos en el fondo nos indican el mismo camino: *la programación requiere un estilo*. El buen estilo de la programación exige del programador el uso adecuado de las técnicas de programación y una buena escritura del programa que facilite el mantenimiento del mismo durante su vida útil.

La calidad del programa requiere esencialmente un funcionamiento correcto, una buena documentación y ser eficiente. *El diseño descendente y un refinamiento sucesivo* debe conducir a buenos programas. Hoy día la *legibilidad* es, sin duda, uno de los criterios más sobresalientes a la hora de decidir si un programa es o no bueno. La superación de la barrera de los 640K en memoria central con las nuevas computadoras basadas en el microprocesador 80386 y su gran velocidad de cálculo, como la familia PS/2 de IBM, hacen que el *estilo de la programación* se considere hoy más importante que el ahorro de memoria en programas y el tiempo de ejecución.

13.1. ESTILO DE PROGRAMACIÓN

El estilo en la construcción de programas, al igual que el estilo de escritura, no es una cosa que pueda adquirirse sólo a partir del conocimiento de las reglas de sintaxis de un lenguaje y las técnicas básicas de programación. El estilo en programación, como en otras actividades de la vida, es una cuestión de gustos personales; sin embargo, en programación es posible unir la inventiva y la ingeniosidad del programador con unas reglas de disciplina y orden en el diseño de programas.

El estilo de la buena programación está íntimamente unido con la legibilidad de los programas. La *legibilidad de un programa es el criterio más sencillo para evaluar la calidad de un programa; si el programa es fácil de leer, probablemente será un buen programa; si es difícil de leer, no es un buen programa* (Kernighan[1] y Plauger. *Software Tools*. Addison-Wesley, 1976).

La legibilidad es la clave para la comprensión de un programa; un programa que no se puede comprender no se podrá modificar ni mantener actualizado. Es muy importante distinguir entre legibilidad y facilidad de escritura. Las técnicas de programación deben facilitar la escritura de programas y posteriormente facilitar su lectura.

La escritura de algoritmos y su posterior codificación conduce a la creación de hábitos de programación. La formación de buenos hábitos de programación, desde el inicio de su aprendizaje será, sin duda, uno de los puntos más importantes en los que deberá prestar atención.

La calidad de un programa se puede medir con diferentes parámetros; entre ellos, destacaremos:

- un programa debe funcionar; obviamente, es la característica más simple e importante,
- la documentación,
- la eficiencia,
- la corrección,
- la flexibilidad,
- la fiabilidad,
- la presentación.

El programa debe funcionar

Un programa se dice que funciona correctamente cuando proporciona unos resultados correctos para todo el rango posible de datos de entrada y en todos sus estados.

La documentación

Es muy importante que el programa esté documentado. La documentación sirve para ayudar a comprender y utilizar un programa. Puede ser *interna* al programa o *externa* al mismo.

La eficiencia

La eficiencia de un programa viene determinada normalmente por el producto espacio-tiempo. Antiguamente los programadores debían gastar mucho tiempo en ahorrar espacio de memoria y disminución de tiempos de ejecución, debido a los escasos recursos hardware y software existentes. Hoy día, aunque se debe tener presente siempre el ahorro de memoria y el tiempo de ejecución, los avances en capacidades de memorias centrales y la velocidad de cálculo de los microprocesadores hacen que el producto espacio-tiempo no sea tan exigente como en años pasados.

---

[1] Creador del lenguaje C.

La corrección

El programa debe coincidir exactamente con las especificaciones: proporcionar la respuesta correcta para cualquier rango válido de datos.

La flexibilidad

Los programas deben permitir cambios con ligeros retoques.

La fiabilidad

La exactitud y precisión de los resultados debe estar acorde con los tipos y rangos de datos y las características de proceso de la computadora.

La presentación

El formato de la presentación de los listados de un programa es decisivo a la hora de mejorar su legibilidad. Dos características se pueden destacar: *comentarios* y *sangrado*.

13.1.1. Nombres de variables

La elección de los nombres de las variables juega un importante papel en la comprensión y legibilidad de los programas. El uso de nombres de variables que describan claramente el propósito de las mismas ayuda en la documentación del programa (*nombres significativos*).

Las variables son partes fundamentales en el programa, mientras que los comentarios son partes auxiliares.

Así, por ejemplo, la instrucción:

```
V ← E / T
```

no es ilustrativa del sentido de la asignación. Sin embargo,

```
VELOCIDAD ← ESPACIO / TIEMPO
```

sólo requiere un poco más de esfuerzo, pero es representativa de las magnitudes que representa.

El nombre de las variables difiere de un lenguaje a otro, aunque en general todos ellos admiten nombres de longitud variable y el carácter subrayado (_).

```
NOMBRE_APELLIDO          COBOL3          PASCAL2
```

Existen dos tipos de variables:

implícitas: nombres de variables que no han sido explícitamente definidas en el programa, pero que pueden utilizarse,
explícitas: las variables deben ser declaradas siempre en el programa antes de usarlas.

Como regla general de programación —aunque el lenguaje permite variables implícitas—, es conveniente declarar siempre todas las variables y evitar siempre que sea posible las variables implícitas o por omisión.

3.1.2. Comentarios

Los comentarios constituyen el principal componente de la documentación interna de los programas. Aunque es importante usar comentarios para ayudar al lector del programa a determinar lo que éste hace en ese momento, no es necesario comentar cada sentencia. El uso frecuente de comentarios como

```
SUMA ← 0      //SUMA se inicializa a cero
N ← N + 1     //N se incrementa en 1
```

sólo produce confusión en el programa, aumento del tamaño del programa y escritura innecesaria.

La escritura de buenos comentarios es muy importante en los programas. Un buen comentario facilita la legibilidad del programa —naturalmente no influye en la codificación—; sin embargo, un mal comentario puede deteriorar la codificación.

Los comentarios deben apoyar la codificación, y aunque no existen sitios fijos donde situarlos, se pueden considerar deseables comentarios en los siguientes puntos:

- *declaración de variables,*
- *estructuras del programa (selectivas o repetitivas),*
- *llamadas y definiciones a/de subprogramas,*
- *codificaciones complejas.*

13.2. ESTRUCTURA DEL PROGRAMA

La *construcción* o *estructura* real de un programa es una de las fases más importantes de la programación. Aunque se conozca la solución del problema y el diseño del programa, la construcción de un programa nunca podrá resultar directa.

En la década de los años setenta apareció un nuevo método de programación conocido como *programación estructurada* —en los Capítulos 4 y 5 se trata con detalle este método— que, aunque en apariencia elimina problemas en la construcción de programas, no es así totalmente, pero sí es cierto que el método enseña a diseñar estructuras que sustituyen a la programación instintiva y de aspecto personal.

La *estructura* de un programa está determinada por las construcciones que se han utilizado para controlar el flujo de control. Una característica notable de la programación estructurada es que hace al flujo de control muy similar al flujo o secuencia de lectura. Esto se facilita con el uso de las estructuras básicas por el programa. Esta obra se ha escrito en la perspectiva de la programación estructurada.

13.3. ESTRUCTURA MODULAR Y DISEÑO DESCENDENTE

En la etapa de análisis del programa se determina —como ya sabemos— lo que *hace* el programa. En la etapa de diseño se determina *cómo* realiza el programa la solución del problema. Existen diferentes métodos para diseñar el proceso correspondiente, y la mayoría de ellos se basan en *una estructura modular* y *en el diseño descendente* (*top-down*).

El problema a resolver se descompone en módulos o tareas más simples. Cada tarea representa una actividad completa y se codifica independientemente.

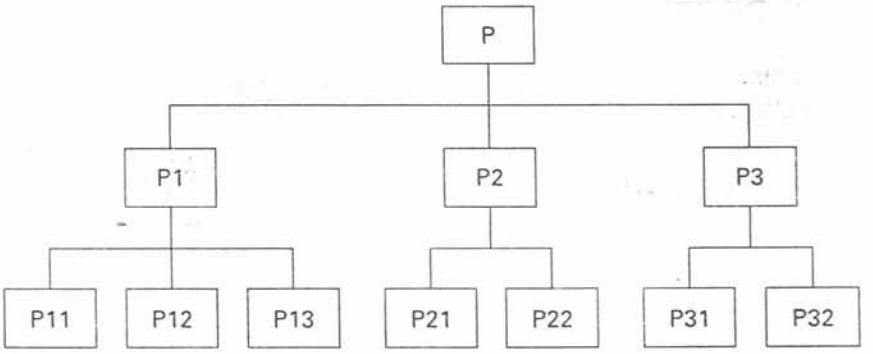


Figura 13.1. Descomposición en módulos del problema P.

Un *módulo* —como ya conocemos— es un conjunto de instrucciones que tienen sólo un punto de entrada y un punto de salida.

El término *diseño descendente* se refiere al orden cronológico en que los módulos de un programa se desarrollan.

La estructura modular se basa en la descomposición de un problema en una serie de subproblemas (módulos o subprogramas). Dado un problema P, éste se puede descomponer en subproblemas P_1, P_2, P_3, ... A su vez, cada módulo o subproblema se puede dividir en otros módulos o subprogramas.

La descomposición del problema principal P en niveles sucesivos de detalle se llama también *refinamiento por pasos*.

Existe otro método, que es el *ascendente* (*botton-up*), el cual desarrolla, en primer lugar, los módulos de nivel más bajo y a continuación construye el programa hacia arriba desde niveles inferiores. En la práctica real, el desarrollo de un programa puede ser una mezcla de diseños descendente y ascendente.

13.3.1. Programación modular

Cualquier programa bien diseñado consta de un *programa principal* (el módulo de nivel superior) que llama a los *subprogramas* (módulos de nivel inferior), los cuales, a su vez, pueden llamar a otros subprogramas, y así sucesivamente. Los programas estructurados de este modo se dice que tienen un *diseño modular*; el método de romper el programa grande en módulos más pequeños se denomina *programación modular*. Los módulos pueden ser planificados, codificados, comprobados y depurados independientemente (incluso con programadores diferentes), y a continuación se combinan uno a uno con otros módulos. El proceso implica los siguientes pasos hasta que el programa esté completo:

1. programar un módulo,
2. comprobar el módulo,
3. si es necesario, depurar el módulo,
4. combinar el módulo con módulos completados anteriormente —si existen— y comprobar la combinación.

13.4. DISEÑO DE PROGRAMAS

El diseño de un programa como método para la resolución de problemas con computadoras, como ya se vio en el Capítulo 2, contempla tres fases básicas:

1. análisis del problema,
2. diseño o desarrollo del algoritmo,
3. solución del algoritmo por computadora.

A su vez, cada fase se compone de varias subfases. Dado que en todos los capítulos anteriores se han estudiados las fases 1 y 2, desarrollaremos con más detalle la fase 3 y dejaremos un apartado para profundizar también en la fase de análisis del problema.

13.4.1. Codificación

La *codificación* es la etapa de conversión del algoritmo en programa, escrito en un lenguaje de programación de alto nivel, como C/C++, Pascal, FORTRAN, BASIC, COBOL, etc. La codificación del programa suele ser una tarea pesada que requiere un conocimiento completo de las características del lenguaje elegido para conseguir un programa eficaz. Sin embargo, si el diseño del algoritmo se ha realizado en detalle con acciones simples y con buena legibilidad, el proceso de codificación puede reducirse a una simple tarea mecánica.

Las reglas de sintaxis que regulan la codificación variarán de un lenguaje a otro y el programador deberá conocer en profundidad dichas reglas para poder diseñar buenos programas.

13.4.2. Ejecución de programas

Una vez finalizada la fase de codificación, el usuario dispone de un programa fuente que podrá ser introducido en la computadora; normalmente, vía teclado y mediante un programa editor.

Normalmente, un programa casi nunca funciona —*corre*— bien a la primera y se producirán diferentes errores. Para poder llegar a tener programas correctos sin ningún tipo de error, necesitará corregir estos errores. La detección de errores y la posterior eliminación de errores constituyen la fase de depuración del programa.

13.4.3. Puesta a punto de programas

La fase de *puesta a punto de un programa* consiste en localizar, verificar y corregir los errores de programación para obtener un programa que funcione correctamente. La puesta a punto de un programa consta de las siguientes fases:

- detección de errores,
- depuración de errores:
  — localización,
  — eliminación,
- prueba del programa.

El objeto final de la puesta a punto de un programa será prevenir tantos errores como sea posible a la hora de ejecutar un programa, así como facilitar la detección y corrección de errores. Dijkstra dijo: «*La verificación puede sólo mostrar la presencia, no la ausencia, de errores (bugs)*».

Los errores típicos durante la ejecución de un programa son:

- *errores de sintaxis,*
- *errores de lógica.*

Errores de sintaxis

Se originan en la fase de compilación/interpretación del programa y se deben a causas propias de la sintaxis del lenguaje, como escrituras incorrectas de instrucciones, omisión de signo, etc.

Normalmente, los errores de sintaxis se descubren con facilidad, aunque en ocasiones puede suceder que un error de sintaxis cometido en un programa no sea detectado como tal, sino que se pone de manifiesto como otro tipo de error en la etapa de ejecución del programa. Por ejemplo, si en una instrucción del tipo `M/N*P` se omite el símbolo de división (`/`) y la instrucción se convierte en `MN*P`, la omisión de este símbolo no se reconoce como error de sintaxis en la fase de compilación o interpretación, ya que `MN` se considera como una variable. Sin embargo, como se comentó en el Apartado 13.1, con la mayoría de los compiladores se producirá un error durante la ejecución del programa, ya que la variable `MN` no ha sido declarada anteriormente.

Errores de lógica

En general, los errores sintácticos son relativamente fáciles de localizar y corregir. Los *errores lógicos* que suceden durante la ejecución de un programa normalmente son más difíciles de detectar.

Se pueden considerar dos categorías de errores: los que detienen la ejecución del programa y los que no la detienen, pero producen resultados erróneos.

Los errores en la lógica del programa se pueden producir durante la fase de compilación o ejecución.

Un ejemplo claro de error lógico puede ser:

```
A ← M/N
```

Si `N` es cero, muchos de los errores lógicos sólo se manifiestan en el momento de la ejecución; por ello se denominan también *errores de ejecución.*

Prueba de programas

Para saber si un programa funciona correctamente, será preciso realizar pruebas con conjuntos de datos de muestra, cuya solución sea conocida y correcta.

Validación de datos

La validación de datos supone la verificación de que los datos de entrada son correctos y están dentro del rango válido.

Ejemplo 13.1

Escribir el algoritmo que verifique los datos de entrada (`HORAS`) *en el rango de* `0` *a* `40` *horas inclusive.*

```
//lectura de datos de entrada

leer (HORAS)
mientras (HORAS < 0) o (HORAS > 40) hacer
  escribir('Numero de horas trabajadas (0-40)')
  leer(HORAS)
fin_mientras
```

13.4.4. Manipulación de entrada/salida

En la fase de análisis del problema existen diferentes subfases (Apartado 2.2), entre las que destacan las especificaciones de los datos de entrada y salida, que condicionarán el diseño del programa.

Estas subfases son completamente independientes del equipo físico en que se va a implementar o mecanizar la aplicación. La complejidad del estudio de los datos de entrada y salida dependerá de la propia complejidad del problema o aplicación a resolver. Así, puede resultar elemental, como en el caso del Ejemplo 2.1, o sumamente complicada, en el caso de gestionar las nóminas de una gran empresa. En general, se deberán considerar una serie de características durante el diseño de las especificaciones de entrada y salida.

Datos de entrada

Se entiende como *datos de entrada* toda aquella información a partir de la cual se deben producir o elaborar los resultados.

Los puntos que se deben describir son esencialmente:

- volumen de la información (medido en unidades de memoria *K-bytes* o *M*-bytes o, a lo sumo, en palabras),
- características de los datos a tratar:
  — numéricos,
  — alfabéticos,
  — alfanuméricos,
  — lógicos,
  — etc.,
- rango de los valores que tomarán los datos,
- formato de los datos.

Si se consideran operaciones con archivos —operaciones usuales—, especificar cuáles son:

- archivos de nueva creación,
- archivos de consulta,
- archivos de actualización,
- etc.

En el caso de aplicaciones de gestión, será preciso considerar los documentos que contienen la información: albaranes, facturas, órdenes de petición/envío, listados de stocks, etc.

Datos de salida

Se deben describir los resultados deseados como solución final, especificando en ellos:

- tipo de datos,
- rangos de valores,
- formatos,
- magnitud esperada,
- precisión en los cálculos (caso de problemas con soluciones numéricas).

13.4.5. Escritura de programas: la documentación

La *escritura del programa*, aunque no es la fase más difícil en el proceso de programación, es, sin duda, la fase que más directamente afectará a la consecución de resultados reales correctos. Esta escritura de programa en el lenguaje seleccionado vendrá influenciada notablemente por el grado de detalle al que se haya descendido en la fase de análisis del problema. La programación y su fase de escritura como exponente fiel de ella puede ser un trabajo de simple conversión o traducción a instrucciones del lenguaje o requerir del programador la realización de complejos diagramas o pseudocódigos.

La escritura de programas se debe ajustar a unas reglas básicas que puedan ser entendibles fácilmente. Los programas solucionan problemas cuando se ejecutan en una computadora; sin embargo, es frecuente que sucedan situaciones como las citadas a continuación:

1. El programa ha sido escrito por un programador y debe ser consultado por personas diferentes al mismo.
2. Los programas pueden contener errores —pese a que el programa en apariencia funcione— y deberán ser verificados por personas distintas al autor usual.
3. Los programas no contienen errores, pero deben ser modificados para actualizarlos: añadirles, modificarles o quitarles determinadas cualidades.

Las actividades 2 y 3 constituyen la operación conocida como *mantenimiento del programa*. Este mantenimiento exige ineludiblemente la necesidad de tener los programas muy documentados. Esto significa que cualquier programa debe tener su propia *documentación*, es decir, descripciones inteligibles que digan *lo que hace* el programa y *cómo lo hace*. La documentación de un programa puede ser *interna* y *externa*.

13.4.6. Documentación interna

La *documentación interna* de un programa está constituida por:

- los comentarios:
  — codificación auto-documentada,
- la presentación:
  — sangrías en los párrafos,
  — listados, impresiones agradables a la vista.

El objetivo de todas estas características es hacer el programa legible, comprensible y fácilmente modificable.

Comentarios

Los comentarios constituyen el principal componente de la documentación interna de los programas. La mayoría de los lenguajes proporcionan la posibilidad de incluir comentarios en los programas.

En Pascal, un comentario es todo lo que se encuentra entre los símbolos `'(*'` y `'*)'` o bien `'{'` y `'}'`. En **BASIC**, es cualquier línea que comience con `REM` (abreviatura de *remark*, comentario). En BASIC estándar, con apóstrofo `(')`, o signo de admiración `(!)` en otras versiones de BASIC.

En **COBOL**, un asterisco comienza una línea de comentario.

En **FORTRAN**, un comentario comienza con la letra `C` (de *comment*, comentario) o con un asterisco. En **C**, un comentario comienza con `/*` y termina con `*/`. En **C++** los comentarios comienzan con un símbolo `//` y terminan con el fin de línea.

En nuestro lenguaje algorítmico, y como ya conoce el lector, utilizamos el símbolo `//` y en algunos casos `{ }`.

Algunos comentarios se pueden haber incluido en el pseudocódigo. Caso de ser así, se deberán incluir también en el programa.

La escritura de buenos comentarios puede ser difícil y en ocasiones tan importante como la escritura de buenos programas.

No es conveniente que los programadores noveles al principio incluyan comentarios, para evitar confusiones innecesarias o situaciones evidentes que sólo implican ilegibilidad en el programa y ocupación innecesaria de memoria central. Comentarios innecesarios y no recomendables —ni en el principio del aprendizaje de programación— son:

```
RESTA ← 0      //RESTA se inicializa a cero
N ← N + 1      //sumar 1 a N
```

Los comentarios se utilizarán libremente para indicar las operaciones que se van a realizar. Como norma general, se deben incluir comentarios en los siguientes puntos:

Cabecera del programa

- nombre del programa,
- tarea que realiza,
- nombre del programador, destino del programa,
- fecha de creación,
- número de versión o actualización (*update*, en inglés),
- fechas de sucesivas actualizaciones,
- estructuras de los datos de entrada y salida, así como rangos aceptables de variables de entrada,
- mensaje de error producido por el programa,
- lista de módulos del programa y una indicación de quién llama a quién.

Declaraciones

Como ya se ha comentado en este capítulo, aun en los casos en que el lenguaje admita declaración implícita de variables, como FORTRAN o BASIC, se recomienda el uso de declaraciones de variables, constantes, subprogramas, tipos o etiquetas. Se debe comentar el uso de tales elementos declarados o, al menos, de los más notables.

```
var
  entero: notas, medias
```

Entradas/salidas

Puede ser interesante, en ocasiones, incluir comentarios que faciliten la identificación y tipo de datos.

```
leer(n)     //n es el numero de alumnos
```

Estructuras de programas

En el principio o a continuación de cada bifurcación o bucle.

```
//determina si divisor es un divisor exacto de numero
desde divisor ← 1 hasta 50 hacer
  cociente ← numero div divisor
  producto ← cociente * divisor
  si producto = numero entonces
    escribir(divisor, 'es un divisor de', numero)
  fin_si
fin_desde     //bucle desde

indice ← 1
//si se encuentra valor devuelve índice para proceso, si no comprobar
//siguiente valor
repetir
si valor = lista[indice] entonces
    indicelista ← indice
    encontrado ← verdad
  si_no
    encontrado _ falso
  fin_si
hasta_que (indice > maxindice) o encontrado
```

Llamadas a subprogramas

Se debe indicar un comentario antes de cada llamada a un subprograma que explique su función y efecto sobre los datos.

```
//NEWTON, calcula interpolaciones
//SORT, clasifica en orden ascendente la lista
//BLANCOS, salta 5 lineas en blanco
```

Presentación

La presentación final de un programa viene influida no sólo por los comentarios, sino por otra serie de consideraciones de tipo práctico muy importantes y que debe tener en cuenta el programador.

Aunque existen lenguajes como Pascal, BASIC o FORTRAN que tienen reglas estrictas de escritura, se debe tender siempre a escribir los programas en cualquier lenguaje, con el estilo de escritura que hemos utilizado a lo largo de esta obra con el pseudocódigo: *sangrado* de las instrucciones interiores a las estructuras. Las sangrías sirven de gran ayuda al revelar la estructura lógica del algoritmo o programa.

Los modelos de sangrado pueden variar de uno a otro programador. El método que hemos utilizado en el libro nos parece el más coherente, pero existen otros que se pueden utilizar y que no por ello dejarán de ser útiles.

```
algoritmo MAXIMO_3_NUMEROS
inicio
//entrada de datos
  leer(A, B, C)
  //cálculo valor mayor de los 3 números
  si A < B entonces
    si A < C entonces
      MIN ← A
      si B < C entonces
        MAX ← B  //A < C < B
        si_no
          MAX ← C  //A < B < C
        fin_si
      si_no
      MIN ← C
      MAX ← B
    fin_si
  si_no
    si A > C entonces
      MAX ← A  //A > B > C
      si B > C entonces
        MIN ← C  //A > C > B
      si_no
        MIN ← B
      fin_si
    si_no
      MAX ← C
      MIN ← B
    fin_si
  fin_si
  //escritura de resultados
  escribir('Valor maximo', MAX)
  escribir('Valor minimo', MIN)
fin
```

Listados/impresiones agradables a la vista

La *impresión agradable* al lector de programas influye notablemente a la hora de modificar un programa. Algunas técnicas que se deben utilizar son:

- líneas y espacios en blanco que facilitan la comprensión de la estructura lógica de un programa,
- líneas de blanco de separación de unidades o bloques lógicos de un programa,
- agrupamiento de sentencias de entradas y sentencias de salida,
- tratar de que la secuencia en que el programa se escribe o lista posteriormente se aproxime lo máximo posible a la secuencia en que se ejecutará el programa.

Codificación autodocumentada

Los programas deben utilizar identificadores con significados que enfaticen sobre la función de los datos y de las variables y constantes.

Por ejemplo,

```
var real: x, y
Z ← M + N
```

no dice nada sobre el uso de las variables. Sin embargo,

```
var real: precio, unidades, total
.
.
.
total ← precio * unidades
```

proporciona buena información.

El Ejemplo 13.2 muestra un programa (algoritmo fácil de leer y comprender).

Ejemplo 13.2

Cálculo de la hipotenusa de un triángulo rectángulo.

```
algoritmo Hipotenusa
//programa de calculo de la hipotenusa de un triangulo rectangulo
//autor J.R. Mackoy
//fecha 8-10-1995
//version 1.0
//declaración de variables
var real: CAT1, CAT2, HIP //CAT1, CAT2, catetos del triangulo
                          //HIP, hipotenusa del triangulo
inicio
   //lectura datos
  leer(CAT1, CAT2)
  //verificacion de errores/validacion de datos positivos
  mientras CAT1 <= 0 o CAT2 <= 0 hacer
    escribir('error: Dato => 0')
    leer(CAT1, CAT2)
  fin_mientras
  HIP ← raiz2(CAT1 * CAT1 + CAT2 * CAT2)
  escribir(HIP)
fin             //fin de calculo de hipotenusa
```

13.4.7. Documentación externa

La documentación asociada con un programa, pero no contenida en él se denomina *externa*.

Una buena documentación externa debe incluir al menos:

1. Manual del usuario.
2. Manual del operador.
3. Manual de mantenimiento del programa.
4. Especificaciones del programa.
5. Lista de datos de prueba (*test*) y resultados.
6. Historia del desarrollo del programa, modificaciones posteriores.
7. Diseño descendente con detalle de módulos y submódulos.
8. Versiones y diferencias entre ellas.

Como detalle final a los comentarios de documentación, queda citar que E. W. Dijkstra *recomienda diez páginas de documentación por cada página de código*. Sin duda, la opinión de Dijkstra, uno de los *padres* de la informática moderna en el campo de la programación, puede no ser signifi-

cativa para los programadores de tipo medio, pero no cabe duda de que si el porcentaje puede resultar alto para ellos, no así la idea que trata de inculcarnos relativa a la preponderancia de la documentación externa sobre la codificación.

Existe otra anécdota contada por Jourdan en *Techniques of Program Structure and Designe* (Prentice-Hall, 1975), y que también puede resultar ilustrativa en lo relativo a la documentación interna y la necesidad de que sea significativa. Cuenta Jourdan que un programador escribió un complicado programa en código máquina y se marchó de vacaciones. Tras las primeras ejecuciones del programa aparecieron diferentes errores (*bugs*), que exigieron su inmediata atención. Como no se le localizó, se encomendó la tarea a un joven programador que encontró un solo comentario en el programa: 'RIPLVB', que acompañaba a una constante octal; la instrucción era:

```
CONST23:        3443        ;RIPLVB
```

Después de muchas horas de estudio, comprobó que 3443 (en base 8) era equivalente a 1827 (en base 10), que coincidía con el año de la muerte (RIP) de Ludwig Van Beethoven (LVB). Evidentemente, este comentario sólo tenía sentido para su autor.

13.5. APLICACIONES DE GESTIÓN (ANÁLISIS)

La resolución de problemas con computadora es el objetivo de este libro; sin embargo, la mayor parte de su contenido se dedica al diseño de algoritmos y a la introducción en estructuras de datos. En el caso de aplicaciones de gestión, las fases que hemos enunciado en este capítulo y los anteriores sufren ciertas modificaciones, debido esencialmente a las características propias de la resolución de problemas de gestión. Aunque las fases prácticamente coinciden, podríamos considerar unas nuevas etapas en el proceso de programación conducente a la resolución de problemas de gestión.

Las fases que podemos considerar son:

1. Análisis de oportunidad.
2. Análisis funcional o descripción del problema.
3. Análisis orgánico o análisis del problema.
4. Programación (diseño del algoritmo y codificación).
5. Depuración y puesta a punto.
6. Explotación.

13.5.1. Análisis de oportunidad

Es la primera fase a desarrollar y su objetivo es definir el coste que supone la mecanización de un determinado problema. Para la definición del coste es preciso tener en cuenta:

- rendimiento económico,
- tiempo de estudio o análisis,
- personal informático necesario,
- medios a utilizar.

Al final del estudio de oportunidad se debe tener la información necesaria para que las personas responsables puedan decidir sobre la conveniencia o no de realizar la aplicación informática y para ello se le debe indicar:

- coste de la mecanización,
- plazos de instalación,
- organización necesaria para el desarrollo de la aplicación.

13.5.2. Análisis funcional

Esta fase de denomina por algunos autores descripción del problema. El objeto del análisis funcional es realizar un estudio global de las soluciones propuesto en el análisis de oportunidad.

Es una fase totalmente independiente del equipo físico en el que se va a implantar la mecanización de la aplicación; en ella se debe incluir una descripción del problema lo más completa posible, analizando métodos y procedimientos utilizados hasta ese momento y que puedan ser útiles para la resolución del problema.

En esta fase se debe realizar un estudio analítico de:

- *datos de entrada,*
- *desarrollo o proceso lógico,*
- *datos de salida.*

13.5.3. Análisis orgánico

El objeto del análisis orgánico es detallar el estudio de todos los puntos desarrollados durante el análisis funcional de una aplicación y definirlos de modo tal que se pueda realizar la programación.

Las características a tener en cuenta en el desarrollo del análisis son:

- descripción de los archivos y tablas:
  — datos: tipos y disposición,
- definición de las funciones del teclado,
- tipos y propiedades de los periféricos utilizados,
- disponibilidad de memoria.

Las herramientas utilizadas en esta fase son: diagramas de flujo y de sistemas y tablas de decisión, y sirven como base para la programación. Los diagramas de flujo ya han sido estudiados en los primeros capítulos y en el capítulo siguiente afrontaremos el estudio de las tablas de decisión.

En esta fase se deben tener en cuenta consideraciones relativas a la computadora o sistema de computadora y al proceso.

Consideraciones sobre la computadora

Se deben seleccionar las siguientes características:

- lenguaje de programación,
- soporte de entrada (ficha, cinta perforada, cinta magnética, disco magnético, etc.):
  — pequeño volumen de datos (se pueden teclear los datos desde el terminal),
  — gran volumen de datos (es conveniente tener almacenados los datos en dispositivos de memoria auxiliar),
- soporte de salida,
- métodos de cálculo, especialmente en el caso de problemas de tipo científico.

Consideraciones sobre el proceso de programación

Los puntos a tener en cuenta en esta fase son:

- definición de variables,
- subprogramas y/o programas especiales,
- definición del juego de datos de ensayo,
- tomas de decisión.

Las fases 4 y 5 ya han sido analizadas en este y anteriores capítulos, por ello terminaremos este apartado con la fase de explotación.

13.5.4. Explotación

Es la fase de ejecución definitiva del programa. Para ello se requiere probar el programa con datos reales, deduciendo los posibles errores de lógica no detectados en la fase de puesta a punto y depuración que deben ser corregidos en este momento.

Un programa, para que sea considerado totalmente acabado requiere haber sido probado totalmente mediante la elección de juegos de ensayo que abarquen todo el rango posible de valores y todas las posibles situaciones que se pueden presentar.

En la explotación real del programa se deberán detectar las posibles situaciones extrañas, de modo que permitan al usuario la seguridad plena de funcionamiento del mismo.

EJERCICIOS

13.1. Realizar el diagrama que calcule la derivación estándar de un vector.

13.2. Se desea procesar los resultados de un examen. La salida será la nota media de la clase y las notas más alta y más baja, junto con los nombres de los estudiantes que han obtenido esas notas. La entrada será una línea para cada estudiante, con su nombre seguido por la nota (clasificación), un entero de 0 a 100 inclusive.

NOTA: Suponer que los nombres no tienen más de 40 caracteres de longitud y que el número de estudiantes de la clase es como máximo 200.

CAPÍTULO 14

TABLAS DE DECISIÓN

CONTENIDO

14.1. Introducción a las tablas de decisión.
14.2. Definición de una Tabla de Decisión (TD).
14.3. Tipos de tablas de decisión.
14.4. Tipos de reglas de decisión.
14.5. Construcción de tablas de decisión.
14.6. Encadenamiento de tablas de decisión.
14.7. Conversión de tablas de decisión en programas.

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS.
EJERCICIOS.

Un método que se ha utilizado como alternativa a los diagramas de flujo han sido las *tablas de decisión*. Esta herramienta sencilla se basa en la construcción de una tabla en la que se recojan las condiciones que se pueden dar en el problema y las acciones que se deben tomar cuando se cumplan las condiciones.

Las Tablas de Decisión (TD) son una herramienta típica de gestión y cuyo uso —si bien ya restringido— puede solucionar y facilitar la resolución de problemas que, utilizando otras herramientas de programación, resultarían notablemente difíciles.

Este capítulo describe las tablas de decisión, sus tipos, el método de construcción y el procedimiento para conversión de una TD en un diagrama de flujo.

14.1. INTRODUCCIÓN A LAS TABLAS DE DECISIÓN

Los ordinogramas estudiados en los capítulos anteriores presentan una serie de inconvenientes:

- el ordinograma no es único para un problema determinado,
- son difíciles de modificar,
- en lenguajes muy evolucionados, los ordinogramas pierden eficacia,
- el proceso secuencial del ordinograma obliga a establecerlo de manera progresiva antes de tener la totalidad del proceso a realizar.

El uso de las tablas de decisión en Informática es reciente —años 1958 a 1960—, aunque anteriormente se habían utilizado en numerosas actividades, como tablas de tarifas, contabilidades, etc.

Las tablas de decisión (TD) constituyen una herramienta poderosa para definir la lógica de un programa complejo. Constituyen un medio de representación de la información en forma tabular, cuyo objetivo primordial es el de aportar información en un formato que sea fácil de leer y comprender.

El uso de las tablas de decisión, aunque no tan extendido como los ordinogramas, suele constituir en algunas ocasiones una técnica para capturar datos y la primera operación de análisis del problema. Una vez planteada la TD, se suele realizar la construcción del ordinograma o algoritmo correspondiente. En ocasiones, las TD sustituyen a los diagramas de flujo u ordinogramas.

14.2. DEFINICIÓN DE UNA TABLA DE DECISIÓN (TD)

La secuencia de un programa puede ser descrita por una estructura de decisión, la *Tabla de Decisión* (TD).

Una TD es un *documento de comunicación entre usuarios, analistas, programadores, etc.* y un instrumento de análisis y programación que se puede aplicar a numerosos y diversos problemas, permitiendo la representación de las diferentes situaciones de un modo fácil y lógico.

Una TD es una herramienta que permite presentar de forma concisa las reglas lógicas que hay que utilizar para decidir acciones a ejecutar en función de las condiciones y la lógica de decisión de un problema específico.

Una TD es un tipo de tabla que muestra lo que debe realizar el programa cuando se cumplen ciertas condiciones. Son especialmente útiles para describir procesos que incluyan muchas decisiones múltiples.

En esencia, las TD constituyen una técnica fácil de aprender y emplear, que no requiere grandes esfuerzos de imaginación para su comprensión e interpretación.

Una tabla de decisión se representa en una cuadro de cuatro bloques (Figura 14.1).

A la izquierda de la Figura 14.1 se tiene la *matriz* y a la derecha las *entradas*. La *matriz de condiciones* recoge las condiciones de todo o parte del problema; dicho en otras palabras, las preguntas que deben probarse para alcanzar una decisión. La *matriz de acciones* refleja todas o parte de las acciones del problema a realizar o las acciones que han de emprenderse cuando se presenta un conjunto dado de condiciones.

La entrada de las condiciones (combinación de las posibles condiciones) permiten reflejar en la TD si se cumple o no tal condición o si es indiferente, es decir, sea cual sea su condición de entrada, no tiene influencia sobre la acción que debería efectuarse.

MATRIZ DE CONDICIONES	ENTRADA DE CONDICIONES (*Situación*)
MATRIZ DE ACCIONES	ENTRADA DE ACCIONES (*Decisión*)

Figura 14.1. Representación gráfica de una tabla de decisión (TD).

La entrada de las acciones (decisión) indica efectuar la acción correspondiente a un conjunto de condiciones cumplimentadas.

14.2.1. Reglas de decisión

Cada combinación de entrada de condiciones y su correspondiente entrada de acciones constituyen una relación denominada *regla de decisión*. Existen tantas reglas como pares distintos de entradas condiciones/acciones. El *número de reglas de decisión* debe cubrir todos los casos posibles, sin repeticiones ni omisiones.

Notas de construcción:

- El número de reglas de decisión es 2^n, siendo n el número de condiciones posibles.
- A una condición de entrada sólo le corresponde una decisión o condición de salida.
- A una condición de salida le pueden corresponder varias condiciones de entrada.
- Para que una TD esté bien construida es necesario que en cada momento sea cierta una y sólo una de las situaciones.
- El resultado de una regla no varía si se permuta el orden de las líneas de condición, ni la lógica de decisión cambia si se permuta el orden de las columnas (reglas).
- Cada regla de decisión (columna) es una estructura `si-entonces` (si se cumple la condición de entradas, entonces realizar tal o tales acciones) del tipo booleano de condiciones y se le pueden aplicar las leyes del Algebra de Boole.
- La entrada de condiciones se representa con los símbolos lógicos `S`, `N` o —(`S`, afirmativa; `N`, negativa—, indiferente).
- La entrada de acciones se representa con una `X` si se realiza, y en blanco o un guión (-) si no se realiza.
- La lista de condiciones puede ponerse en cualquier orden.
- La lista de acciones debe ponerse en el orden en que se tengan que ejecutar.
- Cada columna (regla de decisión) de una TD equivale a una ruta o camino de un diagrama de flujo.

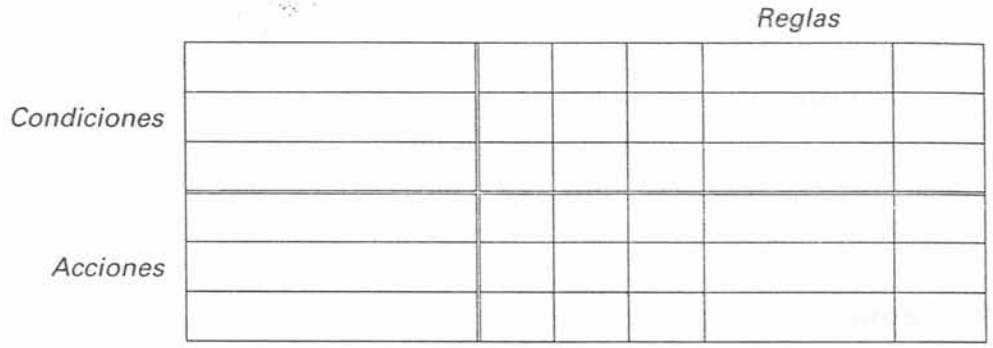
14.2.2. Modos de representación de las tablas de decisión

Existen dos modos de presentación: horizontal y vertical.

Presentación horizontal

En este modo de presentación:

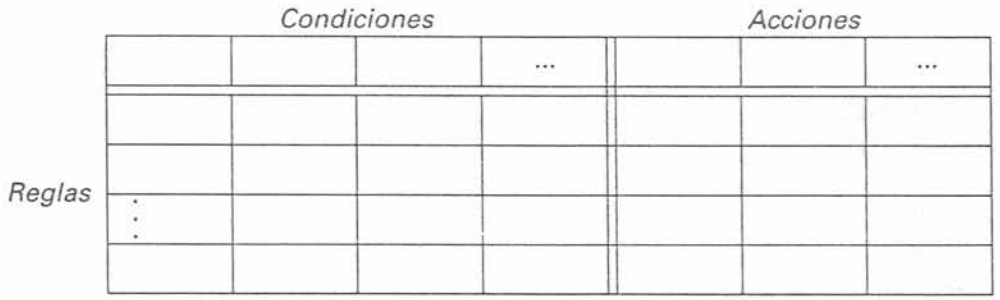
- Las condiciones y las acciones se escriben horizontalmente.
- Las reglas se leen verticalmente.



Presentación vertical

En este modo de presentación:

- Las condiciones y las acciones se escriben verticalmente.
- Las reglas se leen horizontalmente.



La representación de una TD completa se representa en la Figura 14.2.

Las TD se leen siempre de izquierda a derecha y las reglas de arriba a abajo. Las líneas horizontales y verticales dobles sirven como límite o frontera: las condiciones aparecen encima de la doble línea horizontal y los tratamientos debajo de dicha doble línea.

Ejemplo 14.1

Tabla de decisión que contiene los pedidos de una empresa y emite facturas a los clientes de acuerdo con las siguientes hipótesis:

1. Los pedidos pueden ser de Madrid o exteriores a Madrid y, en cualquier caso, se expedirá la factura calculando el precio total de acuerdo con las siguientes condiciones.
2. Pedido exterior de Madrid: imprimir etiquetas de direcciones para envío por correo; cargar portes en la factura, si el pedido es superior a 50.000 pesetas.
3. Pedido de Madrid: si el pedido es superior a 50.000 pesetas, realizar un descuento del 5 por 100.

Envío a Madrid Pedido superior a 50.000	S N	S S	N S	N N
Imprimir etiqueta Calcular precio pedido Añadir portes Descuento 5 %	X	X X	X X X	X X

La lectura de la TD (izquierda-derecha) arroja los siguientes resultados:

1. **Si** *el envío es a Madrid* y **no** *superior a 50.000 pesetas*, **entonces** *calcular precio pedido* para emitir factura por ese precio.
2. **Si** *el envío es a Madrid* y **si** *superior a 50.000 pesetas*, **entonces** *calcular precio pedido* y hacer un *descuento del 5 por 100.*
3. **No** *es de Madrid el envío* y **si** *es superior a 50.000 pesetas*, **entonces** *imprimir etiquetas, calcular precio pedido y añadir portes* para emitir factura total.
4. **No** *es de Madrid el envío* y **no** *es superior a 50.000 pesetas*, **entonces** *imprimir etiqueta y calcular precio pedido* para emitir factura total.

La TD es denominada *propia* por indicarse en ella todo el conjunto de condiciones posibles. En el caso de que una o varias combinaciones de condiciones den lugar a las mismas acciones, es posible simplificar la TD, obteniéndose en ese caso la TD *impropia* o *reducida*, que tiene menor número de reglas.

Situación

Condiciones	C1	S	S	S	N		N
	C2	S	N	N	S		N
	①					③	
	Cn	N	N	S	N		S
Acciones	A1	X			X		X
	A1		X				X
	②					④	
	An	X			X		

Regla de decisión

S: Si
N: No
X: Acción en ejecución
En blanco: No se realiza

Figura 14.2. Tabla de decisión completa.

Así, por ejemplo, en la TD siguiente:

	Reglas							
	R1	R2	R3	R4	R5	R6	R7	R8
C1	S	S	S	S	N	N	N	N
C2	S	S	N	N	S	S	N	N
C3	S	N	S	N	S	N	S	N
A1	X	X	X	X	X		X	X
A2		X		X				X
A3	X		X			X	X	
A4	X				X			X

Las reglas 2 y 4 producen las mismas acciones:

```
S    S
S    N
N    N
```

por consiguiente, en la condición `C2`, *`Sí`* o *`No`*, se producen las mismas acciones; eso significa que se pueden simplificar las dos reglas `R2` y `R4`, dejándolas reducidas a una sola `R24` con la segunda condición representada por un guión (-).

```
S                    S    S
-        equivale a  S    N
N                    N    N
```

De igual forma, las reglas 3 y 7 producen las mismas acciones:

```
S    N
N    N
S    S
```

por consiguiente, se pueden sustituir por una sola regla, `R37`.

```
-
N
S
```

Así pues, la TD resultante sería:

	R1	R24	R37	R5	R6	R8
C1	S	S	-	N	N	N
C2	S	-	N	S	S	N
C3	S	N	S	S	N	N
A1	X	X	X	X		X
A2		X				X
A3	X		X		X	
A4	X			X		X

Ejemplo 14.2

Tabla de decisión para la expedición de billetes de ferrocarril (dos clases, 1.ª y 2.ª) y con dos listas de espera (1.ª y 2.ª).

		R1	R2	R3	R4	R5	R6	R7	R8	Otras
C1	Petición billete 1.ª	S	S	S	S	N	N	N	N	
C2	Petición billete 2.ª	N	N	N	N	S	S	S	S	
C3	Existen plazas en 1.ª	S	S	N	N	N	S	N	S	
C4	Existen plazas en 2.ª	N	S	N	S	S	S	N	N	
A1	Expedir billete 1.ª	X	X							
A2	Expedir billete 2.ª					X	X			
A3	Poner en lista de espera 1.ª			X	X					
A4	Poner en lista de espera 2.ª							X	X	

En esta TD se consideran condiciones excluyentes: si el viajero pide billete de 1.ª, no pide de 2.ª, y caso de no existir 1.ª, pasa a lista de espera de 1.ª, igual para el caso de billetes de 2.ª.

Las reglas de condición R1 y R2, R3 y R4, R5 y R6, R7 y R8 producen las mismas acciones; por consiguiente, la TD se puede reducir a:

		R12	R34	R56	R78	Otras
C1	Petición billete 1.ª	S	S	N	N	
C2	Petición billete 2.ª	N	N	S	S	
C3	Existen plazas en 1.ª	S	N	-	-	
C4	Existen plazas en 2.ª	-	-	S	N	
A1	Expedir billete 1.ª	X				
A2	Expedir billete 2.ª			X		
A3	Poner en lista de espera 1.ª		X			
A4	Poner en lista de espera 2.ª				X	

Para la lectura de esta tabla —como ya se ha comentado— se examinará una sola regla cada vez. Así, por ejemplo, la regla R34 se puede describir así:

```
si la peticion es de un billete de 1ª y no existen plazas en 1ª
entonces poner en lista de 1ª
```

14.3. TIPOS DE TABLAS DE DECISIÓN

Las tablas de decisión se pueden clasificar atendiendo a dos criterios:

Tipos de entrada de condición:

- limitadas,
- ampliadas,
- mixtas.

Procesos complejos (encadenamiento o enlace de tablas):

- abiertas,
- cerradas.

14.3.1. Tablas de decisión limitadas

Son aquéllas en las que la evaluación de todas las condiciones está limitada a dos posibles valores: `SI`, `NO` o la reducción de ambas, el guión (`-`). En síntesis:

- las casillas situadas frente a las condiciones se rellenan con `SI`, `NO` o bien indiferente (`-`),
- las casillas situadas frente a las acciones sólo contienen aspas `(X)`.

Ejemplo 14.3

Tabla de decisión para las ventas de unos grandes almacenes, donde el cliente puede pagar al contado o con tarjeta de compras, y el pedido se le remitirá a su domicilio, cobrándole portes si el pedido es inferior a 10.000 pesetas. (En cualquier pedido se incluirá un IVA del 16 por 100.)

Condiciones: Tiene tarjeta de crédito? `(C1)`
Importe de la compra < `10.000?` `(C2)`

Acciones:

Contado	`(C1 no)`	`(A1)`
Cargar en cuenta	`(C1 si)`	`(A2)`
Cargar portes	`(C2 si)`	`(A3)`
Cargar IVA (16 por 100)	`(siempre)`	`(A4)`

		R1	R2	R3	R4
C1	¿Tiene tarjeta de crédito?	S	S	N	N
C2	¿Importe de la compra < 10.000?	S	N	S	N
A1	Pago contado			X	X
A2	Cargar cuenta	X	X		
A3	Cargar IVA	X	X	X	X
A4	Cargar portes	X		X	

14.3.2. Tablas de decisión de entradas ampliadas o extendidas

Son aquellas tablas en donde las distintas condiciones pueden tener un rango de alternativas superiores a dos valores (si o no).

Ejemplos claros de aplicación de TD ampliadas son:

1. Puesta al día de un archivo. Las posibles alternativas son: altas, bajas y modificaciones.
2. Valores numéricos múltiples correspondientes a un solo elemento:

   1 peseta = 1/135 dólares = 1/220 libras = 1/60 francos...

3. Valores acotados:

compras entre 0 y 1.000 pesetas	$0 < C < 1.000$
compras entre 1.000 y 5.000 pesetas	$1.000 < C < 5.000$
compras entre 5.000 y 10.000 pesetas	$5.000 < C < 10.000$

Ejemplo 14.4

La tabla de decisión limitada anterior se debe modificar debido a que la empresa de transportes ha variado sus tarifas y es necesario un reajuste en las facturas de pedidos, de acuerdo a las siguientes hipótesis:

- Si el importe de la compra < 5.000 pesetas.
  — Cargar total de portes.
- Si el importe de la compra está entre 5.000 y 10.000 pesetas.
  — Cargar 75 por 100 portes.
- Si el importe de la compra está entre 10.000 y 50.000 pesetas.
  — Cargar 50 por 100 portes.
- Si el importe de la compra es > 50.000 pesetas.
  — No cobrar portes.

El número de condiciones se eleva a seis, lo que significa 2^6 combinaciones o reglas de decisión diferentes. Por otra parte, las nuevas condiciones son excluyentes.

Si el pedido es menor de 5.000 pesetas, nunca podrá cumplir las otras condiciones (> 50.000 pesetas, ...).

Al elevado número de condiciones habría que sumar las nuevas acciones, lo que hace la resolución del problema con TD limitada sumamente compleja, y por ello es más sencillo recurrir a una TD ampliada.

Tiene tarjeta de crédito	SI	NO	SI	NO	SI	NO	SI	NO
Importe de la compra	< 5.000		5.000 y 10.000		10.000 y 50.000		> 50.000	
Pago contado		X		X		X		X
Cargar cuenta	X		X		X		X	
Cargar IVA	X	X	X	X	X	X	X	X
Cargar portes	100%	100%	75%		50%			

En este caso ha sido necesario desdoblar la condición (Tarjeta de Crédito) en dos subcondiciones, pero en cualquier caso esta tabla siempre será más fácil de leer y entender que su equivalente tabla limitada.

14.3.3. Tablas de decisión mixtas

Se denominan así las tablas que resultan de la combinación de las tablas vistas en los párrafos anteriores.

Una aplicación de este tipo de tabla podrá ser una ampliación del ejemplo de expedición de billetes de la siguiente forma:

- comprobar plazas libres en 1.ª y 2.ª,
- petición del viajero,
- acepta cambio de clase si no existe la solicitada.

La TD ampliada resultante sería la siguiente:

	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	R12
Billetes disponibles	Sólo 1.ª			1.ª y 2.ª		Sólo 2.ª			No hay plazas			
Petición viajero	1.ª	2.ª	2.ª	1.ª	2.ª	1.ª	1.ª	2.ª	1.ª	1.ª	2.ª	2.ª
Cambio de clase	–	S	N	–	–	S	N	–	S	N	S	N
Expedición billete 1.ª	X	X		X								
Expedición billete 2.ª					X	X		X				
Lista de espera 1.ª							X		X	X	X	
Lista de espera 2.ª		X							X		X	X

Esta tabla resuelta, en representación gráfica limitada, sería enormemente compleja, ya que de cada condición actual se derivarían las escritas en la entrada de condiciones.

14.3.4. Comparación y transformación de los tipos de tablas

La *comparación* de las TD produce las siguientes características:

- Las TD de entradas ampliadas son más legibles que las TD limitadas, por ser menos voluminosas que sus equivalentes. La razón, como ya se ha visto, reside en el hecho de que una condición de una tabla de entrada ampliada corresponde a varias condiciones de la tabla de entradas limitadas equivalentes, debido a que éstas sólo tienen dos posibles alternativas, S o N.
- Las TD de entradas limitadas son más fáciles de codificar en programa (las reglas de decisión constituyen caminos de un diagrama de flujo) que las tablas implicadas.

Para convertir una TD de entradas ampliadas en una TD de entradas limitadas, será necesario ejecutar las siguientes tareas:

- Formular tantas condiciones binarias (S/N) como alternativas consideradas existan para cada condición de la tabla de entrada ampliada.
- Formular tantas acciones binarias (S/N) como posibilidades consideradas en las reglas para cada una de las acciones de la tabla de entrada ampliada.

La tabla mixta anterior podría quedar así:

	R1		R2	R3
Billetes disponibles sólo 1.ª	S	Excluyentes	S	S
Billetes disponibles 1.ª y 2.ª	–		–	–
Billetes disponibles sólo 2.ª	–		–	–
No hay plaza	–		–	–
Petición de viajero 1.ª	S		–	–
Petición de viajero 2.ª	–		S	S
Cambio de clase	–		S	N
Expedición billete 1.ª	X		X	
Expedición billete 2.ª				
Lista de espera 1.ª				
Lista de espera 2.ª				X

Procediendo en forma inversa, se convertirá una TD limitada a una TD ampliada.

14.4. TIPOS DE REGLAS DE DECISIÓN

Las reglas de decisión según las relaciones que existan entre las condiciones de una tabla de decisión pueden ser:

```
Reglas < <Y> > (AND)
```

En cada situación se necesita que se satisfaga la primera condición y la segunda y la tercera, ... (operador lógico **AND**).

```
si condicion 1 y condicion 2 y ... entonces
```

```
Reglas < <O> > (OR)
```

Para que se satisfaga una regla, basta con que se cumpla cualquiera de las condiciones de la situación (operador lógico **OR**). Existen dos tipos de reglas **<<O>>**.

OR inclusivo

Se cumple una u otra condición o simultáneamente.

OR exclusivo

Entre dos condiciones se cumple una u otra, pero no ambas simultáneamente.

En la siguiente tabla se pueden apreciar las reglas **<<Y>>**, **<<OR>>**.

	R1	R2	R3	R4
C1	N	N	S	S
C2	N	S	N	S
Acciones	...	...	...	...

Las reglas R2, R3 y R4 son de tipo O inclusivo, las reglas R2 y R3 **<<Or exclusivo>>**. La regla R4 constituye una de tipo **<<Y>>**.

Reglas ELSE (si_no)

Entre las diversas situaciones posibles, sólo algunas de ellas conducen a tratamientos diversos, mientras que el resto de las reglas conducen a un único tratamiento. Las reglas que conducen a un tratamiento único se agrupan en una sola regla denominada **ELSE (si_no o en otro caso)**. Se suele colocar al final de la TD sin evaluar sus condiciones.

14.4.1. Conversión de reglas <<O>> y <<SI_NO>>

Una regla **<<O>>** se transforma en *n* reglas «y», siendo *n* el número de condiciones que intervienen en la regla (las *n* reglas se suponen con indiferencias, ya que sin ellas se tendría $2 \uparrow n - 1$).

Supongamos la situación de una regla OR extraída de una TD de cuatro condiciones.

```
N     si no se cumple la primera condición,
S     o si se cumple la segunda,
N     o no se cumple la tercera,
S     o si se cumple la cuarta.
```

Esta regla **OR** equivale a cuatro reglas **AND**:

```
R1 R2 R3 R4
N     S     S     S
-     S     N     N
-     -     N     S
-     -     -     S
```

(–) significa indiferencia, que procede de la fusión de dos situaciones similares:

```
N     N
N     S
N     N
S     S
```

equivale a:

```
N
-
N
S
```

14.5. CONSTRUCCIÓN DE TABLAS DE DECISIÓN

Para construir una TD son necesarios los siguientes pasos:

1. Estudio de condiciones, acciones y sus relaciones.
2. Rellenar la TD.
3. Analizar la TD.
4. Simplificar la TD.

14.5.1. Estudio de condiciones, acciones y relaciones

Tras la lectura del enunciado del problema, se localizan las condiciones, que se han de extraer del enunciado, mediante la búsqueda de palabras similares a `<<Si, cuando, si no, etc.>>`.

Las acciones, como el diccionario de la lengua indica, se han de localizar en el enunciado con los verbos que implican acción, es decir, hacer algo: realizar, ejecutar, sumar, restar, etc.

Una vez indicadas las condiciones y acciones, se comienza la *escritura de la TD* de acuerdo a las siguientes reglas:

Escritura de las condiciones:

- Realizar un listado de todas las condiciones.
- Clasificar las condiciones siguiendo el orden lógico —si existe— en el que se consideran. Así, en el caso de tres condiciones, órdenes lógicas pueden ser:

```
N  N  N  N  S  S  S  S          S  S  S  S  N  N  N  N
N  N  S  S  N  N  S  S   o bien  S  S  N  N  S  S  N  N
N  S  N  S  N  S  N  S          S  N  S  N  S  N  S  N
```

Una elección lógica del orden de las condiciones simplifica considerablemente la escritura de las reglas de la tabla.

Ejemplo 14.5

En la suscripción de una revista de información, los suscriptores antiguos tienen un descuento del 10 por 100 y los estudiantes, del 20 por 100. Los descuentos no pueden acumularse y, caso de coincidir dos descuentos en una sola persona, se le aplicará el mayor descuento a que tenga lugar.

La tabla de decisión puede ser la siguiente:

		R1	R2	R3	R4
C1	¿Suscriptor antiguo?	S	S	N	N
C2	¿Estudiante?	S	N	S	N
A1	Descuento 10 %		X		
A2	Descuento 20 %	X		X	
A3	Sin descuento				X

Sin embargo, eligiendo las condiciones `C1` y `C2` en sentido contrario, como el descuento de estudiante es mayor que el de suscriptor antiguo, haciendo la pregunta de estudiantes antes, cuando la contestación es afirmativa, será indiferente la situación como suscriptor.

Estudiante	S	N	N
Suscriptor antiguo	–	S	N
Descuento 10 %		X	
Descuento 20 %	X		
Sin descuento			X

Regla: El orden lógico de considerar las condiciones de una tabla es el siguiente:

- de arriba abajo para las tablas de presentación horizontal,
- de izquierda a derecha para las tablas de presentación vertical.

Escritura de las acciones

- Realizar el listado de todas las acciones a tomar.
- Clasificar las acciones mediante un orden cronológico —si existe— según el orden en que deben considerarse. En ocasiones, el orden no es importante, pero en otras puede afectar al desarrollo de las acciones.
- El orden de considerar las acciones es:
  — de arriba abajo para las tablas de presentación horizontal,
  — de izquierda a derecha para las tablas de presentación vertical.

Escritura de la entrada de condiciones (situaciones):

- Las condiciones se deben formular Si(S), No(N) o Indiferente (-).
- Seguir un orden lógico en las entradas. Por ejemplo, primero todos los SIes (o todos los NOes) y a continuación hacer que vayan apareciendo los NOes progresivamente. Si una tabla de decisión contiene *n* condiciones, debe tener 2^n reglas, en el caso de no utilizar signos de indiferencia.

Así, para la TD de tres condiciones:

Condición 1	S	S	S	S	N	N	N	N
Condición 2	S	S	N	N	S	S	N	N
Condición 3	S	N	S	N	S	N	S	N
Acciones	.....							

Escritura de entrada de acciones

Se van colocando aspas (`X`) frente a las acciones que deben ejecutarse para cada situación o conjunto de condiciones consideradas.

Escritura de las reglas de las TD ampliadas

Se van escribiendo las condiciones de modo que a cada una de ellas se corresponderá un determinado número de alternativas. Para evitar olvidar reglas es conveniente seguir un método para rellenar las tablas.

Para rellenar las entradas de acciones bastará ir colocando las pre[illegible] relativas a las acciones que se han de ejecutar en la casilla correspondiente.

14.5.2. Análisis y requisitos de una TD

Antes de convertir la TD en programa, se necesita efectuar un análisis de sus reglas y comprobar que la TD diseñada cumple los siguientes requisitos:

- completa,
- no tiene redundancias,
- no tiene contradicciones.

Una TD es completa cuando se han establecido todas las situaciones posibles —todas las combinaciones matemáticamente posibles de las entradas o estados de las condiciones—, o dicho de otra forma: no se ha olvidado ninguna posible entrada.

Una TD no tiene redundancias cuando no existe ninguna situación idéntica que conduzca a tratamientos idénticos. Existe redundancia cuando aparece dos o más veces la misma entrada de condiciones con igual tratamiento.

Una TD no es contradictoria cuando, siendo redundante, los tratamientos son los mismos, o dicho de otra manera, es contradictoria cuando a un mismo estado de condiciones le corresponden tratamientos diferentes. Si el conjunto de reglas supera 2^n, existirán errores.

14.5.3. Simplificación de una tabla de decisión

La simplificación de una TD se debe realizar siempre que sea posible. Un método consiste en agrupar reglas simples en reglas compuestas (reglas simples son las que no contienen ninguna indiferencia y reglas compuestas, las que contienen al menos una indiferencia).

Dos reglas pueden simplificarse siempre que se produzcan las mismas acciones y un estado de condiciones difiere sólo en la situación de una condición.

	R1	R2	
C1	S	S	
C2	S	N	
C3	N	N	
A1	X	X	
A2			
A3			

equivale a

	R12
C1	S
C2	–
C3	N
A1	X
A2	
A3	

La siguiente tabla no se puede simplificar (difiere en más de dos situaciones):

	R1	R2
C1	S	S
C2	N	S
C3	S	N
A1	X	X
A2		
A3		

Si son más de dos reglas las que contienen iguales acciones, existe un teorema, denominado *<<del paraguas>>* por el tipo de dibujo necesario para la obtención de la simplificación, que permite la sustitución de varias reglas simples por una compuesta, siempre que las reglas tengan las mismas acciones: *<<Se unen mediante un arco aquellas reglas en que sólo cambia una condición. Si todos los arcos forman un circuito cerrado, se puede obtener una regla compuesta que sustituya a las reglas simples>>.*

En la siguiente TD es posible simplificar las cuatro reglas simples en una sola compuesta, debido a que se cumple el teorema anterior.

	R1	R2	R3	R4
C1	S	S	S	S
C2	S	S	S	S
C3	S	N	S	N
C4	S	S	N	N
A1				
A2	X	X	X	X

La tabla resultante se obtiene viendo las condiciones que permanecen fijas y poniendo indiferencias en las condiciones que varían. En el caso de `R14` se tiene

	R14
C1	S
C2	S
C3	–
C4	–
A1	
A2	X

Una forma de comprobar la simplificación sería hacer estos dos pasos:

	R12	R34
C1	S	S
C2	S	S
C3	–	–
C4	S	N
A1		
A2	X	X

equivale a

	R1234
C1	S
C2	S
C3	–
C4	–
A1	
A2	X

14.6. ENCADENAMIENTO DE TABLAS DE DECISIÓN

Cuando un problema de lógica es complejo, la TD equivalente resultará así mismo compleja. En estos casos es preferible dividir el problema en problemas secundarios, de modo que cada uno de ellos dé lugar a la creación de una tabla particular. Las TD pequeñas son más fáciles de realizar, utilizar, modificar y leer que una TD voluminosa. Posteriormente, las diferentes tablas se encadenan o enlazan para la solución del problema global.

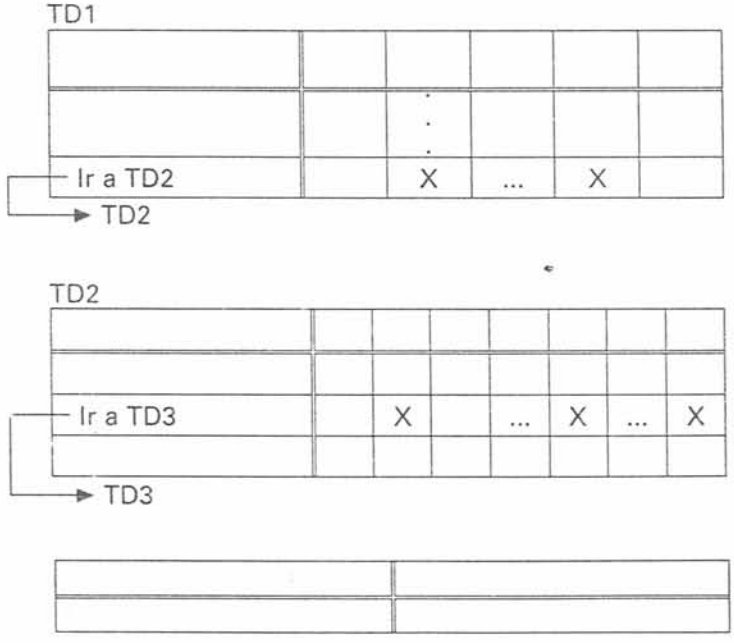
Los encadenamientos se realizan con las ya conocidas *tablas abiertas* y *tablas cerradas*.

14.6.1. Tablas abiertas

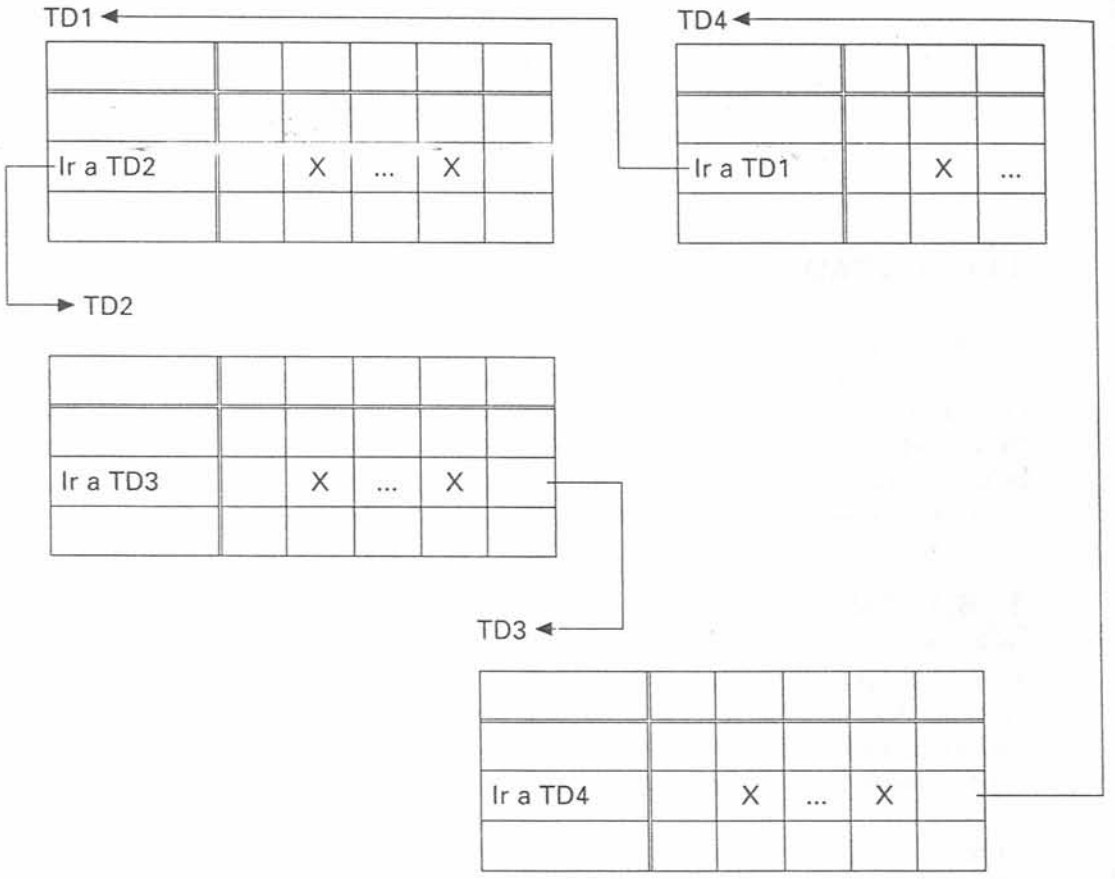
Estas tablas presentan una conexión al principio de la tabla llamada (n) sin necesidad de retorno a la tabla que llama, después de la ejecución de la tabla n. Las llamadas se realizan mediante una bifurcación incondicional de tipo `GO TO` y, una vez ejecutada, no retorna a la tabla anterior.

Se clasifican en dos grandes grupos:

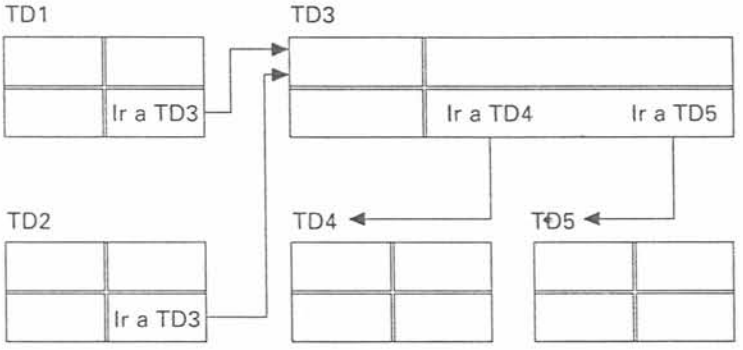
Encadenamiento en cascada:



Encadenamiento en anillo:



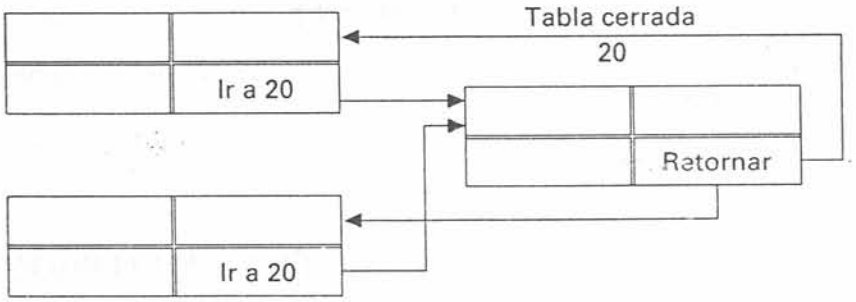
Otros ejemplos de encadenamientos serían:



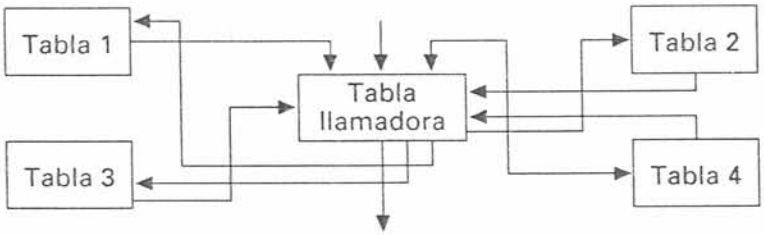
14.6.2. Tablas cerradas

Son aquéllas en que, una vez ejecutada la tabla de llamada, devuelve el control a la tabla que la llamó. La llamada se realiza mediante una estructura DO `(hacer)` y el retorno del control a la tabla precedente con RETURN `(volver a)`.

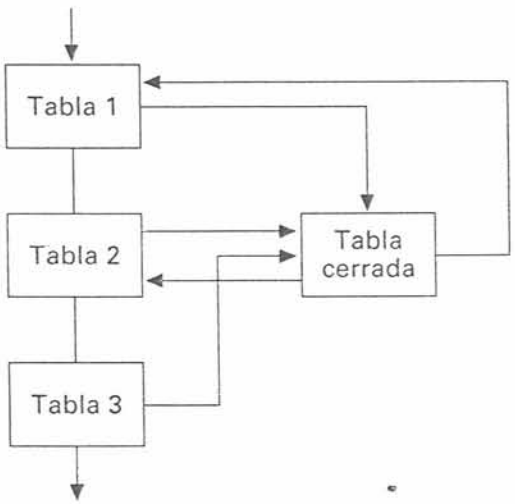
Existen numerosos casos de tablas cerradas:



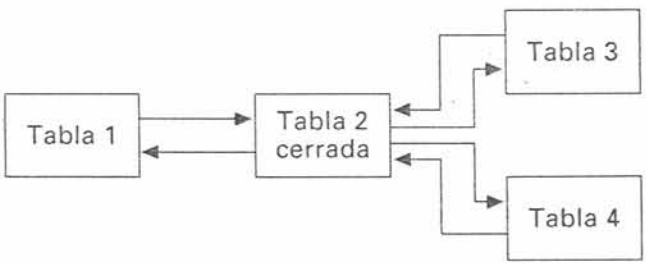
1. Se puede llamar desde una tabla a varias tablas cerradas:



2. Una tabla cerrada puede ser llamada desde diferentes tablas:



3. Una tabla cerrada puede llamar a su vez a otras tablas cerradas:



Las condiciones que se deben cumplir para que pueda existir un encadenamiento son:

1. Cada tabla se debe identificar mediante una etiqueta (número o serie de caracteres).
2. La tabla invocada se ejecutará totalmente.
3. Cada regla de decisión debe indicar lo que se ha de hacer a continuación (*bifurcar, hacer, terminar, etc.*).

14.7. CONVERSIÓN DE TABLAS DE DECISIÓN EN PROGRAMAS

La conversión de las TD en programas se pueden realizar con diferentes técnicas:

1. Programación directa en lenguajes de alto nivel como COBOL, FORTRAN o BASIC.
2. Transformación de la tabla en diagramas de flujo.
3. Convertidores de TD en programas.
4. Métodos matriciales.

14.7.1. Programación directa en lenguajes de alto nivel

Las reglas de decisión se convierten en instrucciones de los lenguajes. En el caso de la siguiente regla:

	R1
C1	S
C2	N
C3	S
A1	X
A2	X

```
IF C1 AND NOT C2 AND C3 THEN A1 A2 ELSE...
```

Esta técnica es cómoda, pero tiene el inconveniente de preguntar en la misma pasada varias veces la misma condición.

Si se han definido tablas encadenadas, cada una de ellas dará lugar a subprogramas abiertos o cerrados, según el tipo de tabla.

En el caso de tablas abiertas, la acción *ir a la tabla n* se traduce, según el caso, por una instrucción de bifurcación incondicional del tipo `GOTO` (BASIC o Pascal), `GO TO` (COBOL), o bien por una instrucción de llamada a subprograma abierto o subrutina mediante la instrucción `CALL` en COBOL, BASIC, etc.

Si son tablas cerradas, la acción *ejecutar la tabla n* se traduce por una instrucción de llamada de subprograma cerrado (instrucción `PERFORM` en COBOL).

14.7.2. Transformación de la tabla en diagrama de flujo

Esta técnica fue desarrollada por S. L. Pollack y establece como hipótesis que la evaluación de cada condición va a ocupar siempre las mismas posiciones de memoria. A continuación, se debe minimizar el número de evaluaciones a efectuar, o sea, lograr un mínimo número de pruebas de condiciones.

El método se basa en la localización de las indiferencias de cada regla, siguiendo los siguientes pasos:

1. Cálculo del número de reglas simples a las que equivale cada regla (una regla con n indiferencias equivale a 2^n reglas simples) y consigue el mismo número al final de cada columna.
2. Por cada condición se calcula un coeficiente D sumando los valores calculados en 1 y se registra en una columna a la derecha de la TD; el coeficiente de cuenta mide el número de reglas en que la condición en estudio no interviene (su valor será tanto más importante cuanto menor sea su valor). *Se evalúa en primer lugar la condición cuyo coeficiente sea menor.*
3. Si existen varias condiciones con una cuenta mínima igual, se elige un nuevo valor (coeficiente) que es el que mide en valor absoluto la diferencia entre el número de `SI` y `NO` que aparece en dichas condiciones, eligiéndose el de mayor valor absoluto.
4. Si subsiste la igualdad, se elige indiferentemente cualquiera de las condiciones.
5. Se repite el proceso (pasos 1 a 4) hasta la última condición.

Sea, por ejemplo, una TD de cuatro condiciones y siete reglas con las indiferencias siguientes:

	R1	R2	R3	R4	R5	R6	R7
C1	N	S	–	S	S	N	S
C2	S	N	S	S	S	N	N
C3	–	–	–	N	–	–	N
C4	S	N	N	S	S	–	S

Paso 1:

R4 y R7	Reglas simples (coeficiente `1/R4 = 1, R7 = 1`).
R1, R2 y R5	Reglas compuestas con una indiferencia (equivale cada una a dos reglas simples (coeficiente `2/R1 = R2 = R5 = 2`).
R3 y R6	Reglas compuestas con dos indiferencias (equivale cada una a cuatro reglas simples; coeficiente `4/R3 = R6 = 4`).

Paso 2:

Condiciones	*Indiferencias*	*Coeficientes*
C1	R3	4
C2	R5	2
C3	R1, R2, R3, R6	2 + 2 + 4 + 4 = 12
C4	R6	4

Se elige la condición C2 por ser la de coeficiente mínimo.

	R1	R2	R3	R4	R5	R6	R7	*Coeficiente*	Δ
C1	N	S	–	S	S	N	S	4	2
C2	S	N	S	S	–	N	N	2	0
C3	–	–	–	N	–	–	N	12	10
C4	S	N	N	S	S	–	S	4	2
	2	2	4	1	2	4	1		

Al evaluar la condición C2 (S y N), la TD se transforma en dos tablas derivadas de la eliminación de la condición C2. La primera tabla contiene las reglas en las que la condición C2 tiene el valor S, y la segunda el valor N (las indiferencias tienen los valores dobles S y N).

Paso 3:

Las evaluaciones siguientes a partir de C2 es para C1 y C4 por orden de coeficiente (C1 para C2 con evaluación N y C4 para evaluación S).

C2 con evaluación S

Regla	*Coeficiente*	*Condiciones*	*Indiferencia*	*Coeficiente*
R1	2	C1	R3	4
R3	4	C3	R1, R3	2 + 4 = 6
R4	1	C4	Ninguna	0
R5	1			

La siguiente condición es C4 (coeficiente 0).

C2 con evaluación N

Regla	*Coeficiente*	*Condiciones*	*Indiferencia*	*Coeficiente*
R2	2	C1	Ninguna	0
R5	1	C3	R2, R6	2 + 4 = 6
R6	4	C4	R6	4
R7	1			

La siguiente condición es C1 (coeficiente 0).

	R1	R3	R4	R5	Coeficientes
C1	N	–	S	S	4
C3	–	–	N	S	6
C4	S	N	S	S	0
	2	4	1	1	

	R2	R5	R6	R7	Coeficientes
C1	S	S	N	S	0
C3	–	S	–	N	6
C4	N	S	–	S	4
	2	1	4	1	

Paso 4 y repeticiones:

A partir de C2 se analizarán los dos posibles estados o respuestas S, N.

Para C2 con evaluación S se toma la condición C4, ya que los coeficientes y las condiciones son:

Coeficientes		*Condiciones*	
	C1 - 4		R1 - 2
	C3 - 6		R2 - 4
	C4 - 0		R3 - 1
			R3 - 1

Para C2 con evaluación N se toma la condición C1, ya que los coeficientes son:

Coeficientes		*Condiciones*	
	C1 - 0		R1 - 2
	C3 - 6		R5 - 1
	C4 - 4		R6 - 4
			R7 - 1

El diagrama resultante será:

	R1	R4	R5	*Coeficientes*
C1	N	S	S	0
C3	-	N	S	2

	R2	R5	R7	*Coeficientes*
C3	-	S	S	2
C4	N	S	S	0

R3	
C1	-
C3	-

R6	
C3	-
C4	-

A partir de la condición C4 se derivarán C1 y de la condición SI, C3 con sus dos estados. De la condición C1 se derivarán C4, y de la condición SI de C4 la condición C3.

Diagrama de flujo derivado de C4:

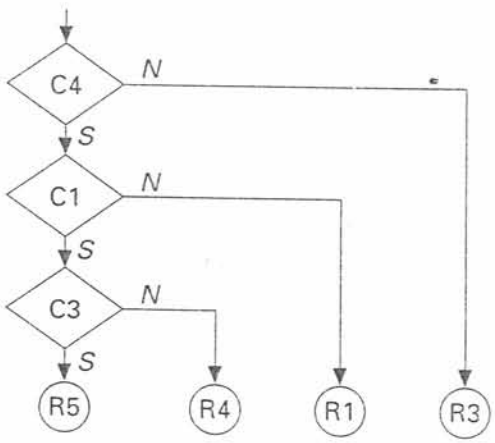
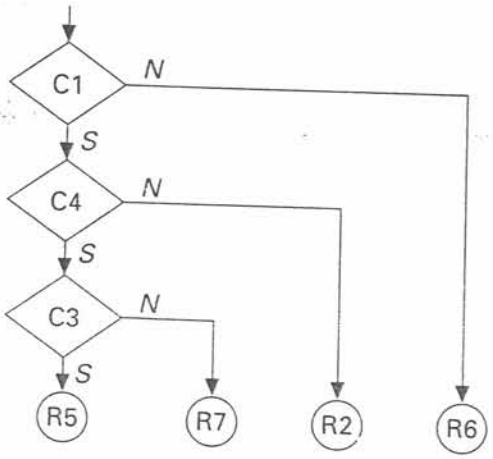
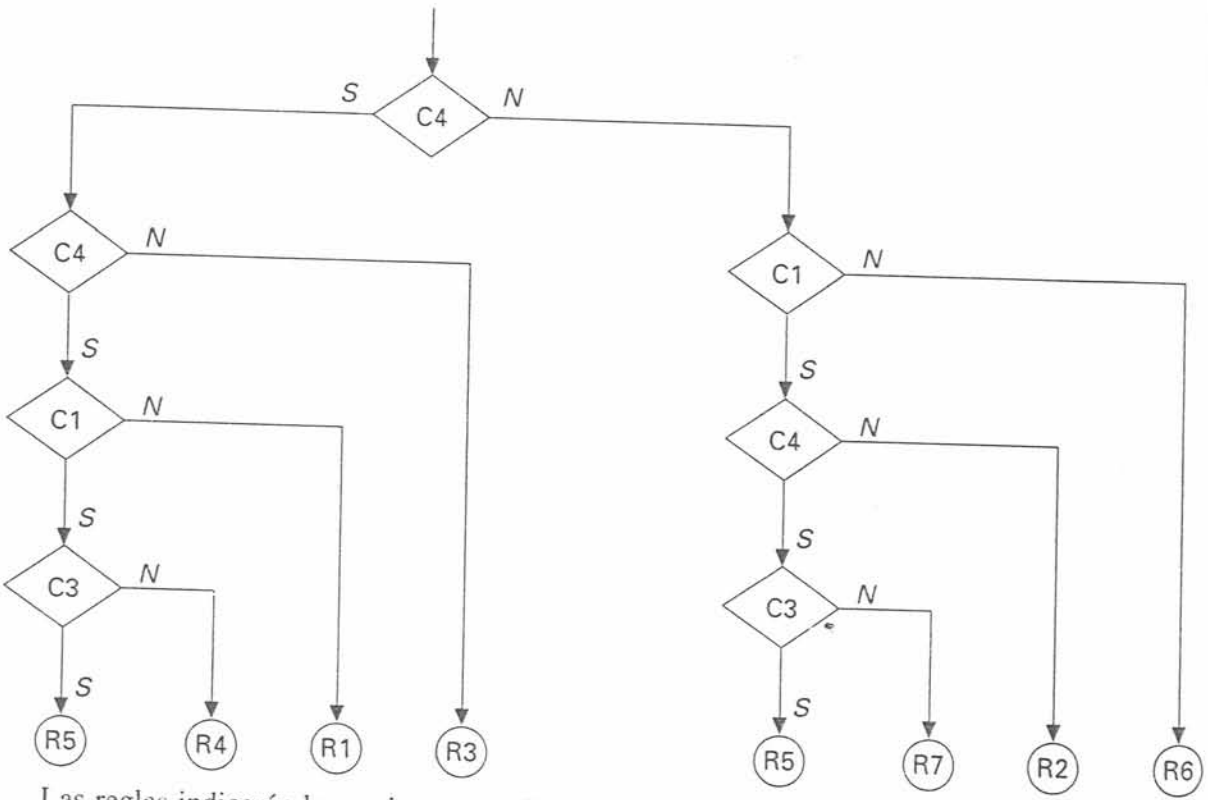


Diagrama de flujo derivado de C1:



Para obtener el diagrama de flujo final, bastará ir uniendo los diagramas de flujo parciales.

Diagrama de flujo total:



Las reglas indicarán las acciones a realizar en cada caso según la TD inicial. La codificación resultante en pseudocódigo será:

```
si C2 entonces
  si C4 entonces
    si C1 entonces
      si C3 entonces
        <R5>
```

```
        si_no <R4>
        fin_si
      si_no <R1>
    fin_si
  si_no <R3>
  fin_si
si_no
  si C1 entonces
    si C4 entonces
      si C3 entonces
        <R5>
      si_no <R7>
      fin_si
    si_no <R2>
    fin_si
  si_no <R6>
  fin_si
fin_si
```

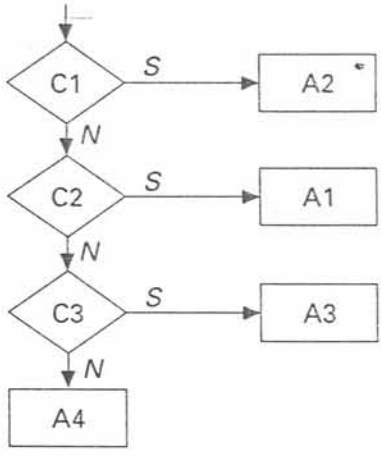
Ejemplo 14.6

Convertir en diagrama de flujo la siguiente TD:

C1 C2 C3	S – –	N S –	N N S	N N N
A1		X		
A2	X			
A3			X	
A4				X

El orden de las condiciones se realizará por su importancia. En este caso el orden es C1, C2 y C3, ya que ése es el orden en donde más reglas aparecen.

Así pues, el diagrama de flujo correspondiente será:



ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

14.1. Una empresa tiene las siguientes condiciones de pago para emitir las facturas de los pedidos de sus clientes:

1. El cliente paga al contado y el pedido es superior a 100.000 pesetas; se emite factura con un descuento del 15 por 100.
2. El cliente paga al contado y el pedido no es superior a 100.000 pesetas; se emite factura con un descuento del 15 por 100, pero se cargan los portes al cliente.

Las condiciones a evaluar son:

C1 Pago al contado.
C2 Pedido superior a 100.000 pesetas.

El conjunto total de reglas de condición es $2^2 = 4$.
El conjunto de acciones es:

- Cálculo del importe total del pedido.
- Cargar porte.
- Descuento: 15 por 100.

La configuración de la TD es:

	R1	R2	R3	R4
Pagar contado	S	S	N	N
Pedido superior a 100.000 pesetas	S	N	S	N
Cálculo importe	X	X	X	X
Cargar portes		X		
Descuento 15 %	X	X		

14.2. Realizar una tabla de decisión que permita la distribución de los pedidos de una empresa y la expedición de facturas, de acuerdo con las siguientes hipótesis:

A) *Si existe saldo y existencias del artículo solicitado:*

- y el pedido es superior o igual al mínimo (1.000) y el cliente es mayorista, hacer un descuento del 20 por 100, enviar pedido y factura,
- si el pedido es inferior al mínimo (1.000), facturar sin descuento, sea el cliente mayorista o detallista.

B) *Si existe saldo, pero no hay existencias disponibles suficientes:*

- facturar existencias,
- dejar pendiente de envío el resto pedido,
- no hacer descuento,
- emitir factura.

C) *Si no existe saldo:*

- remitir presupuesto del pedido al cliente.

Saldo suficiente	S	S	S	N
Existencias suficientes	S	S	N	–
Cliente mayorista	S	–	–	–
Pedido >= 1.000 unidades	S	N	–	–
Enviar total pedido	X	X		
Enviar total existencias			X	
Emitir factura	X	X	X	
Remitir presupuesto de pedido al cliente				X
Dejar pendiente el resto del pedido en espera de nuevas existencias			X	
Aplicar descuento del 20 %	X			

14.3. Una persona desea comprar un automóvil, para lo cual tiene tres opciones posibles para conseguir el importe (de acuerdo a sus medios económicos: actuales o futuros):

- solicitar un préstamo bancario,
- confiar en que le toque la lotería primitiva (loto),
- conseguir un adelanto de la empresa.

Condiciones:
- Préstamo bancario (C1)
- Le toca la lotería primitiva (C2)
- Adelanto de la empresa (C3)

Acciones:
- Comprar automóvil (T1)
- Esperar y seguir con el viejo (T2)

		R1	R2	R3	R4	R5	R6	R7	R8
C1	Préstamo bancario	S	S	S	S	N	N	N	N
	Lotería	S	S	N	N	S	S	N	N
	Adelanto empresa	S	N	S	N	S	N	S	N
	Comprar automóvil	X	X	X	X	X	X	X	
	Seguir con el viaje								X

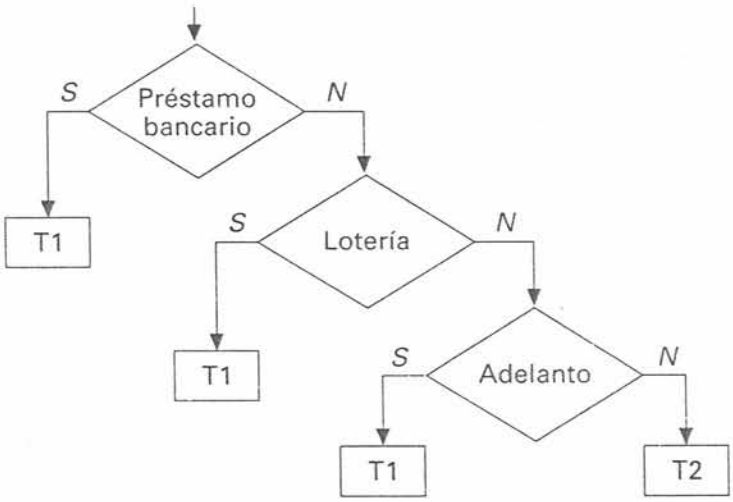
La simplificación de las reglas se deducirá por la siguiente matriz de cumplimiento de las condiciones:

	R12	R34	R56	R7	R8
Préstamo bancario	S	S	N	N	N
Lotería	S	N	S	N	N
Adelanto empresa	–	–	–	S	N
Comprar automóvil	X	X	X	X	
Seguir con el viaje					X

La tabla anterior puede simplificarse de nuevo, ya que existen nuevas indiferencias (S-N) para iguales condiciones:

R1234	R56	R7	R8
S	N	N	N
–	S	N	N
–	–	S	N
X	X	X	
			X

La conversión de la TD en diagrama de flujo:



14.4. Realizar una tabla de decisión limitada y su equivalente ampliada que indique el salario a pagar a los empleados por cada hora extraordinaria, dependiendo de la división del empleado (A o B) y su categoría (1 o 2):

- Hora extra categoría 1 2.500 pesetas
- Hora extra categoría 2 1.500 pesetas
- División A 500 pesetas
- División B 250 pesetas

TD limitada:

Categoría 1	S	S	N	N
Categoría 2	N	N	S	S
División A	S	N	S	N
División B	N	S	N	S
Hora extra 1.750				X
Hora extra 2.000			X	
Hora extra 2.750		X		
Hora extra 3.000	X			

TD ampliada:

Categoría	1	1	2	2
División	A	B	A	B
Hora extra	3.000	2.750	2.000	1.750

14.5. Diseñar una tabla de decisión que permita calcular el salario total a pagar a un empleado de una empresa de acuerdo con las siguientes proposiciones:

- Si el empleado está autorizado a realizar horas extraordinarias y ha trabajado durante la semana más de 40 horas, se le debe pagar su salario normal y, además, el importe de las horas extraordinarias.

Empleado autorizado a realizar horas extraordinarias	N	S	S
Horas trabajadas durante la semana > 40	–	N	S
Pagar salario normal	X	X	X
Pagar horas extras			X

14.6. Transformar la siguiente tabla de decisión en un diagrama de flujo:

	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	R12	R13	R14	R15	R16
C1	S	S	S	S	S	S	S	S	N	N	N	N	N	N	N	N
C2	S	S	S	S	N	N	N	N	S	S	S	S	N	N	N	N
C3	S	S	N	N	S	S	N	N	S	S	N	N	S	S	N	N
C4	S	N	S	N	S	N	S	N	S	N	S	N	S	N	S	N
T1	X	X			X	X	X									
T2			X	X					X		X	X				
T3										X			X	X	X	X
T4								X					X		X	

	T1	T2	T3	T4	T3 y T4
C1	S S S S S	S S N N N	N N	S N	N N
C2	S S N N N	S S S S S	S N	N N	N N
C3	S S S S N	N N S N N	S S	N N	S N
C4	S N S N S	S N S S N	N N	N N	S S

Simplificando, obtendremos:

	T1	T2	T3	T4	T3 y T4
C1	S S	– N	N	–	N
C2	– N	S S	–	N	N
C3	S S	N S	S	N	–
C4	– S	– S	N	N	S

Disponiéndolo en forma de tabla, considerando sus reglas, queda como sigue:

	R1	R2	R3	R4	R5	R6	R7
C1	S	S	–	N	N	–	N
C2	–	N	S	S	–	N	N
C3	S	N	N	S	S	N	–
C4	–	S	–	S	N	N	S
T1	X	X					
T2			X	X			
T3					X		X
T4						X	X

Construcción del diagrama:

	R1	R2	R3	R4	R5	R6	R7	Δ	
C1	S	S	–	N	N	–	N	6	
C2	–	N	S	S	–	N	N	6	
C3	S	N	N	S	S	N	–	2	→ **C3**
C4	–	S	–	S	N	N	S	8	
	4	1	4	1	2	2	2		

Para C3 = N o –

	R2	R3	R6	R7		
C1	S	–	–	N	6	
C2	N	S	N	N	0	→ **C2**
C4	S	–	N	S	4	
	1	4	2	1		

Para C2 = S o –

	R3		
C1	–	**R3** →	**T2**
C4	–		

Para C2 = N o –

	R2	R6	R7		
C1	S	–	N	2	
C4	S	N	S	0	→ **C4**
	1	2	1		

Para C4 = S o –

	R2	R7		
C1	S	N	R2 →	**T1**
			R7 →	**T3 Y 74**

Para C4 = N o –

	R6		
C1	–		
		R7 →	**T4**

Para C3 = S o –

	R1	R4	R5	R7		
C1	S	N	N	N	0	→ **C1**
C2	–	S	–	N	6	
C4	–	S	N	S	4	
	4	1	2	1		

Para C1 = S o –

	R1		
C2	–	→	**T1**
C4	–		

Para C1 = N o –

	R4	R5	R7		
C1	S	–	N	2	
C4	S	N	S	0	→ **C4**
	1	2	1		

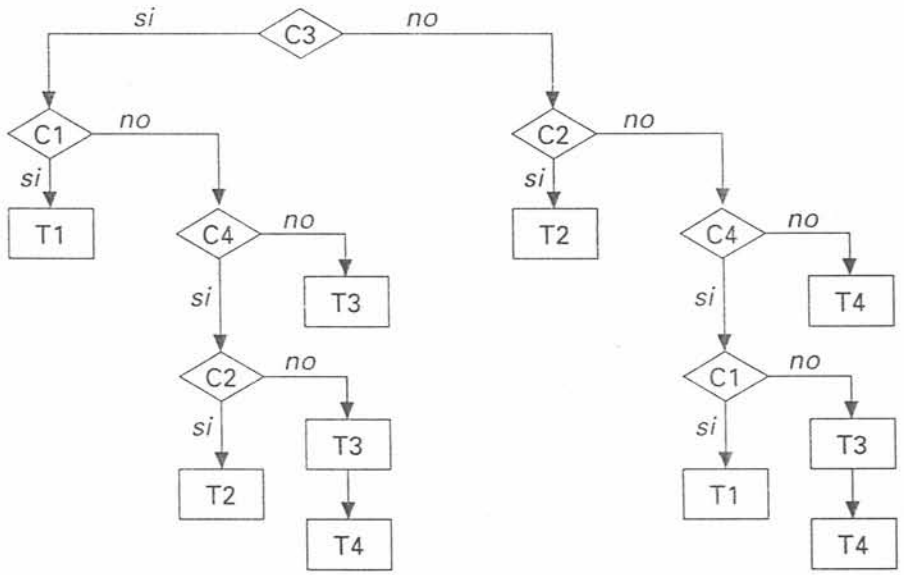
Para C4 = S o –

	R4	R7		
C2	S	N	R4 →	T2
			R7 →	T3 Y 74

Para C4 = N o –

	R5		
C2	–	R5 →	T3

Y sin más que ir trasladando reglas, de acuerdo con la importancia de las condiciones, ya definidas, llegaremos al diagrama adjunto.



14.7. Se desea diseñar una TD que permita controlar el stock de un almacén. Si el número de identificación del artículo es correcto y existe stock del mismo en almacén, servir pedido; en caso contrario realizar pedido.

La TD correspondiente es:

		R1	R2	R3	R4
C1	Número identificación correcto	S	S	N	N
C2	Existe stock	S	N	S	S
A1	Servir pedido	X			
A2	Realizar pedido		X	X	X

Las reglas R2, R3 y R4 producen la misma acción y la tabla simplificada será:

Número identificación correcto	S	S	N
Existe stock	S	N	–
Servir pedido	X		
Realizar pedido		X	X

14.8. Se desea representar en una TD la conducta a seguir por el robot R2D2 para cruzar una calle.

La situación donde se encuentra situado el robot es la siguiente:

- robot en la acera,
- pasan automóviles por la calle,
- existe un semáforo y un guardia urbano,
- el robot debe pasar en verde o bien con semáforo en rojo o en ámbar, siempre y cuando no lo vea el guardia.

Las condiciones son:

C1 Semáforo verde
C2 Guardia ve el robot
C3 Pasan automóviles

Las acciones son:

A1 Pasar la calle
A2 No pasar la calle

La TD requerirá $2^3 = 8$ reglas.

	R1	R2	R3	R4	R5	R6	R7	R8
C1	N	N	N	N	S	S	S	S
C2	N	N	S	S	N	N	S	S
C3	N	S	N	S	N	S	N	S
A1	X				X		X	
A2		X	X	X		X		X

Las reglas que se pueden simplificar son las que producen las mismas acciones:

R3	R4	R5	R7	R6	R8
N	N	S	S	S	S
S	S	N	S	N	S
N	S	N	N	S	S

y se simplifican en

R34	R57	R68
N	S	S
S	-	-
-	N	S

el resto de las reglas permanecen inalterables:

	R1	R2	R34	R57	R68
C1	N	N	N	S	S
C2	N	N	S	-	-
C3	N	S	-	N	S
A1	X			X	
A2		X	X		X

14.9. Diseñar una TD que tenga en cuenta las decisiones a tomar por el señor Mortimer al salir de casa una mañana de invierno en función de que llueva —coger paraguas— o no llueva —no coger paraguas.

Las condiciones son:

C1 llueve
C2 desea salir

Las acciones a realizar en función de las condiciones:

A1 coger paraguas
A2 no coger paraguas
A3 salir a la calle

Al ser dos condiciones, se necesitan 2^2 reglas:

	R1	R2	R3	R4
C1	N	S	N	S
C2	N	N	S	S
A1				X
A2	X	X	X	
A3			X	X

Las reglas R1 y R2 se pueden reducir a una R12, y la tabla simplificada queda:

C1	–	N	S
C2	N	S	S
A1			X
A2	X	X	
A3		X	X

14.10. En la empresa ACEROS ANDALUCES algunos empleados están autorizados a realizar horas extraordinarias sobre la base de treinta y ocho horas semanales. Diseñar la TD que permita el pago de salarios normales y los incrementos correspondientes a las horas extraordinarias.

Las reglas de la tabla son:

Regla 1 y 2:
(R1 y R2) El empleado **no** está autorizado a realizar horas extraordinarias con independencia de trabajar (R2) o no (R1) más de treinta y ocho horas y, por consiguiente, se le pagará el salario normal.

Regla 2:
(R2) **Si** el empleado está autorizado a realizar horas extraordinarias y **no** ha trabajado más de treinta y ocho horas en la semana se le pagará el salario normal.

Regla 3:
(R3) **Si** el empleado está autorizado a realizar horas extraordinarias y **si** ha trabajado más de treinta y ocho horas en la semana, se le pagará el salario normal y el salario de las horas extras.

Las condiciones son:

C1 Empleado autorizado a realizar horas extraordinarias
C2 Empleado ha trabajado más de treinta y ocho horas en la semana

Las acciones a tomar en función de las condiciones y las reglas son dos:

A1 Pagar salario normal
A2 Pagar horas extraordinarias

La TD resultante será:

	R1	R2	R3	R4
C1	N	N	S	S
C2	N	S	N	S
A1	X	X	X	X
A2				X

Las reglas R1 y R2 se pueden simplificar, resultando:

	R12	R3	R4
C1	N	S	S
C2	–	N	S
A1	X	X	X
A2			X

EJERCICIOS

14.1. Construir la TD de las decisiones a tomar por un conductor frente a un semáforo.

14.2. Construir una TD para la expedición de facturas a los clientes de una empresa de acuerdo con las siguientes hipótesis:

a) los clientes deudores no tienen descuento,
b) si de un artículo se facturan más de 5.000 unidades se realizará un 10 por 100 de descuento a los artículos del tipo M y un 5 por 100 a los del tipo N,
c) sólo en el caso de los artículos del tipo M, se añadirá al descuento del apartado b) un 5 por 100 más si la venta se realizó en la zona de Extremadura, y un 3 por 100 más para la zona de Valencia.

14.3. Realizar una TD que permita la distribución de los pedidos de una empresa y la expedición de facturas, de acuerdo con las siguientes hipótesis:

a) Si existe saldo y existencias del artículo solicitado:
   - y el pedido es superior o igual al mínimo (1.000) y el cliente es mayorista, hacer un descuento del 20 por 100, enviar pedido y factura,
   - si el pedido es inferior al mínimo (1.000), facturar sin descuento, sea el cliente mayorista o detallista.

b) Si existe saldo, pero no hay existencias disponibles suficientes:
   - facturar existencias,
   - dejar pendiente de envío el resto del pedido,

- no hacer descuento,
- emitir factura.

c) Si no existe saldo:
- remitir presupuesto del pedido al cliente.

14.4. Realizar una TD limitada y su equivalente ampliada que indique el salario a pagar a los empleados por cada hora extraordinaria, dependiendo de la categoría del empleado (`A` o `B`) y su división (`1.ª` o `2.ª`).

Hora extra categoría `1.ª`	2.500 pesetas
Hora extra categoría `2.ª`	1.500 pesetas
División `A`	500 pesetas
División `B`	250 pesetas

14.5. Diseñar una TD que permita calcular el salario total a pagar a un empleado de una empresa de acuerdo con las siguientes proposiciones:

- Si el empleado está autorizado a realizar horas extraordinarias y ha trabajado durante la semana más de cuarenta horas, se le debe pagar su salario normal y, además, el importe de las horas extraordinarias.

14.6. Transformar la siguiente TD en un diagrama de flujo:

Horas extras	S	S	S	S	S	S	S	S	N	N	N	N	N	N	N	N
Casado	S	S	S	N	N	N	N	N	S	S	N	N	S	S	N	N
Categoría laboral	S	S	N	N	S	N	N	N	S	S	N	N	N	N	N	N
Antigüedad	S	N	S	N	S	N	S	N	S	N	S	N	S	N	S	N
Acción A1						X		X								
Acción A2		X			X									X		
Acción A3	X		X	X			X		X	X	X	X	X		X	X

APÉNDICE A

ESPECIFICACIONES DEL LENGUAJE ALGORÍTMICO UPSAM[1]

A.1. ELEMENTOS DEL LENGUAJE

Identificadores

- Longitud: máximo 32 caracteres.
- Se admiten todos los caracteres alfabéticos (estándar y extendidos), dígitos y el símbolo de subrayado.
- No se hace distinción entre mayúsculas y minúsculas.
- Se debe empezar siempre por un carácter alfabético.

Comentarios

- Se recomienda el uso de la doble barra inclinada (//) al comienzo del comentario. Puede ir al principio de una línea del pseudocódigo, o después de alguna instrucción.
- Para comentarios multilínea, se puede encerrar el comentario entre llaves ({}).

Tipos de datos estándar

- Numérico (`entero` y `real`).
- Lógicos (`logico`).

[1] Las especificaciones del pseudolenguaje UPSAM (su denominación original fue UPS) han sido experimentadas desde el año 1990, en el Departamento de Lenguajes y Sistemas Informáticos de la Facultad y Escuela Universitaria de Informática de la Universidad Pontificia de Salamanca en Madrid y, con anterioridad, desde 1986 en el CESIES de la misma Universidad.

Las primeras especificaciones del pseudocódigo se publicaron en 1986 en nuestro libro *Metodología de la Programación*, editado por McGraw-Hill. El resultado que ahora se publica es fruto de todos los profesores que han impartido la asignatura de *Fundamentos de Programación*, y esta síntesis ha sido redactada por los profesores Luis Rodríguez, Matilde Fernández y Luis Joyanes, con las sugerencias y mejoras de los profesores Luis Villar y Joaquín Abeger. Han colaborado también los profesores Antonio Muñoz, Ángel Hermoso, Isabel Torralvo, Ángela Carrasco, Miguel Sánchez, Víctor Martín, M.ª del Mar García, Rafael Ojeda, Lucas Sánchez y Joaquín Abeger.

- Caracteres (`caracter`).
- Cadenas (`cadena`). Se considera como un dato estándar, pero estructurado, es decir, como un array de caracteres acabado en un carácter nulo.
- Se admiten cadenas abiertas o limitadas a un número determinado de caracteres, de la forma **`cadena`** `nombre_variable [n]`.

Constantes estándar

- Numéricas enteras: están compuestas por los dígito (`0..9`), y el signo (`+,-`).
- Reales en coma fija: están compuestas por los dígitos (`0..9`), el signo (`+.-`) y la coma decimal (`,`).
- Reales en coma flotante: están compuestas por los dígitos (`0..9`), el signo (`+,-`) y la coma decimal (`,`) de la mantisa, la letra **E**, antes del exponente y los dígitos (`0..9`), el signo (`+,-`) y la coma decimal (`,`) del exponente.
- Lógicos: son las constantes que representan a verdadero **`(Verdad)`** y falso **`(Falso)`**.
- Caracteres: cualquier carácter del juego de caracteres utilizado, delimitado por el separador de caracteres (`'`).

Operadores

Tabla A.1. Operadores aritméticos

–	Menos unario
*	Multiplicación
/	División real
↑	Exponenciación
+	Adición
–	Resta
`mod`	Módulo de la división entera
`div`	División entera

Tipos de resultados de las expresiones aritméticas:

La división real siempre da un resultado real. Los operadores `mod` y `div` sólo operan con enteros y el resultado es entero. El menos unario devuelve el mismo tipo que el operando. En el resto de operadores aritméticos, si alguno de los operadores es real, el resultado es real.

Tabla A.2. Operadores de relación

=	Igual a
<>	Distinto de
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que

Tipos de resultados de expresiones de relación:

Siempre devuelven un dato de tipo lógico. Los operandos deben ser del mismo tipo.

Tabla A.3. Operadores lógicos

`no`	Negación lógica
`Y`	Multiplicación lógica
`O`	Suma lógica
`O_X`	O exclusivo
`eqv`	Equivalencia
`imp`	Implicación

Tipos de resultados de expresiones lógicas:

Siempre devuelven un dato de tipo lógico. Los operandos deben ser del mismo tipo lógico.

Tabla A.4. Prioridad de los operandos

Paréntesis	( )
Exponenciación	↑
Operadores unarios	`no, -`
Operadores multiplicativos	`*./, div, mod, y,` ↑
Operadores auditivos	`+,-,o,o_x`
Operadores de relación	`=,<, >, < >, >=, <=`

Las operaciones se evalúan siempre de izquierda a derecha.

Operador de asignación: (←). Sólo se pueden asignar datos del mismo tipo.

Definición de tipos

- Registros:

```
registro: <nombre_del_tipo>
    <tipo>: <nombre_del_campo>
    ....
fin_registro
```

  Una línea por campo.

- Archivos de organización secuencial

```
archivo_s de <tipo_de_dato> : <nombre_del_tipo>
```

- Archivos de organización relativa

```
archivo_d de <tipo_de_dato> : <nombre_del_tipo>
```

- Arrays

  array[dimensiones] **de** <tipo_de_dato>: <nombre_del_tipo>

  dimensiones *sería un subrango.*

- Conjunto

  conjunto de <tipo_de_dato> : <nombre_del_tipo> [es (<lista de valores>)]

- Enumerados

  <nombre_del_tipo> **es** (<lista de valores>)

- Subrangos

  subrango de <tipo_de_dato> : <nombre_del_tipo> **es** [<inf>..<sup>]

Definición de constantes

- **const** <nombre_de_constante> = <expresion>

Definición de variables

- <tipo_de_dato> : <lista_de_variables>

Funciones aritméticas de biblioteca

`abs()`	*valor absoluto de un número*
`ent()`	*valor entero (se redondea al entero menor)*
`trunc()`	*valor entero (se truncan los decimales)*
`redondeo()`	*redondea un número*
`aleatorio()`	*valor aleatorio*
`exp()`	e^x
`ln()`	*logaritmo neperiano (base e)*
`log10()`	*logaritmo decimal (base 10)*
`raiz2()`	*raíz cuadrada*
`cuadrado()`	*cuadrado de*
`sen()`	*seno*
`cos()`	*coseno*
`arctan()`	*arco tangente*
`tipo()`	*convierte la expresión en el tipo base especificado*

Funciones de cadena de biblioteca

`longitud()`	*Longitud de la cadena*
`posicion(<c>, <cb>)`	*Posición de* `cb` *dentro de* `c`
`subcadena(<c>, <ini>, [<long>])`	*Extrae de* `c`*, a partir del carácter* `ini` *una subcadena de longitud* `long`

Funciones de conversión de número a cadena

`codigo()`	*Devuelve el código ASCII de un carácter*
`car()`	*Devuelve el carácter asociado a un código ASCII*
`valor()`	*Convierte una cadena en un valor numérico*
`cad()`	*Convierte un número en cadena*

Funciones de información

`tamaño_de()`	*Devuelve la longitud en bytes de una variable o tipo de dato*

Estructura de un programa

```
algoritmo <nombre_algoritmo>
[incluir <modulo>]…
// declaraciones de constantes, tipos y variables
[const <declaraciones de constante]
[tipo <declaraciones de tipos de datos>]
[var <declaraciones de variables>]
// Se recomienda declarar primero las constantes,
// luego los tipos y por ultimo las variables
// Cualquier objeto debe ser declarado antes de ser utilizado
inicio
...
fin
[declaraciones de procedimientos y funciones]
```

A.2. ESTRUCTURAS DE CONTROL

Estructuras selectivas

```
si condicion entonces
   <acciones>
[si_no
   <acciones>]
fin_si
segun_sea <expresion_ordinal> hacer
   <lista_de_valores_ordinales> : <acciones>
   ...
```

```
[si_no
   <acciones>]
fin_segun
```

- La expresión que se evalúa debe ser de tipo ordinal.
- Las listas de valores, son uno o más valores ordinales separados por comas.

Estructuras repetitivas

```
mientras <expresion_logica> hacer
   <acciones>
fin_mientras
```

```
repetir
   <acciones>
hasta_que <expresion_logica>
```

```
desde <variable_ordinal> ← <valor_final>
                          incremento|decremento <valor> hacer
   <acciones>
fin_desde
```

```
iterar
   <acciones>
   [salir_si]
fin_iterar
```

A.3. PROGRAMACIÓN MODULAR

A.3.1. Inclusión de archivos o modulos

- **incluir** *<modulo>*, permite incluir archivos con subprogramas.

A.3.2. Procedimientos

- Declaración

```
procedimiento <nombre_proc>[lista_de_parametros_formales)]
[declaraciones locales]
inicio
...
fin_procedimiento
```

<lista_de_argumentos> *es uno o más argumentos de la siguiente forma:*

{{E|S|E/S}[1] <tipo_de_dato> : <lista_de_argumentos>}...

[1] E: Entrada; S: Salida; E/S: Entrada/Salida.

- El tipo de dato debe estar definido por anticipado.
- Llamada a un procedimiento:

```
[llamar_a] <nom_proc>[lista_parametros_actuales)]
```

- Los parámetros deben coincidir en posición, tipo y número.

A.3.3. Funciones

- Declaración

```
<tipo_de_dato> funcion
<nombre_fun>[(lista_de_parametros_formales)]
[declaraciones locales]
inicio
  ...
  devolver (<expresion>)
fin_funcion
```

- Lista de parámetros es igual que en los procedimientos.
- `devolver` nos dice el valor de retorno de la función.
- Sólo se pueden devolver datos estándar.

A.3.4. Cuestiones generales

- El ámbito de las variables es igual que en Pascal.
- Todas las variables locales, para el uso de variables globales utilizar el modificador `global`, `global entero a.`
- No se admiten subprogramas anidados.
- Los archivos se pasan siempre como **E/S**.

A.3.5. Procedimientos de entrada/salida

- **leer** (*<lista_de_variables>*)
- **escribir** ([*<lista_de_expresiones>*])
- **leercar** (*<variable_tipo_caracter>*)

A.4. ARCHIVOS

A.4.1. Archivos secuenciales (texto)

- Crear y abrir es distinto. Para abrir es necesario tener creado el archivo (esto también es válido para indexados y directos). Crear inicializa el archivo; abrir, permite acceder a un archivo existente.
- **crear** (*<var_tipo_archivo>, <nombre_fisico>*), nombre físico es una expresión de cadena.
- **abrir** (*<var_tipo_archivo>, <modo>, <nombre_fisico>*), nombre físico es una expresión de cadena.
- <modo> en archivos secuenciales puede ser: **lectura** (coloca el puntero de datos al inicio

del archivo y abre para la lectura) o **escritura** (coloca el puntero de datos al final del archivo y abre para escritura).

- Escritura: **escribir** *(<var_tipo_archivo>, <lista_de_expresiones>)*.
- Lectura: **leer** *(<var_tipo_archivo>, <lista_de_variables>)*.
- Lectura: **leercar** *(<var_tipo_archivo>, <var_tipo_caracter>)*.
- Fin de línea: **fdl** [*(var_tipo_archivo)*].
- Fin de archivo: **fda** [*(var_tipo_archivo)*]. Es verdadero si se ha intentado la lectura después del último registro.
- Cerrar un archivo **cerrar** *(<lista var_tipo archivo>)*.
- Otros procedimientos útiles pueden ser **borrar** *(<nombre_fisico>)*, que borra un archivo que debe estar cerrado, y **renombrar** *(<nombre fisico>, <nombre_fisico>)*, que renombra el archivo primero por el nombre del segundo. Ambos deben estar cerrados.
- Otra función útil puede ser **lda** *(<nombre_fisico>)*, que devuelve la longitud del archivo en bytes.

A.4.2. Archivos directos (relativos)

- **crear** *(<var_tipo_archivo>, <nombre_fisico>)*, nombre físico es una expresión de cadena.
- **abrir** *(<var_tipo_archivo>, <modo>, <nombre_fisico>)*, nombre físico es una expresión de cadena.
- <modo> en archivos directos sólo es **l_e** para lectura y escritura.
- Escritura: **escribir** *(<var_tipo_archivo>, <var_tipo_base>, <posicion>)*. Escribe una variable del tipo base del archivo en la posición relativa especificada.
- Lectura: **leer** *(<var_tipo_archivo>, <var_tipo_base>, <posicion>)*. Lee una variable del tipo base del archivo en la posición relativa especificada.
- Fin de línea y Fin de archivo, no tienen efecto en archivos relativos (*si consideramos la lectura secuencia, Fin de archivo sí tendría sentido).*
- También se admiten los procedimientos **borrar** y **renombrar**, así como la función **lda**.
- El número de registros del archivo se puede obtener con

```
lda(<var_tipo_archivo>) / tamaño_de(var_tipo_base_del_archivo).
```

A.4.3. Archivos indexados

- Definición:

```
archivo_i de <tipo_dato>: <nombre_del_tipo>
            clave_p <lista_de_campos>//Clave primaria sin duplicados
            [clave_s <lista_de_campos> [duplicada]]...
```

- **crear** *(<var_tipo_archivo>, <nombre_fisico>)*, nombre físico es una expresión de cadena.
- **abrir** *(<var_tipo_archivo>, <modo>, <nombre_fisico>)*, nombre físico es una expresión de cadena.
- <modo> en archivos indexados sólo puede ser **l_e**.
- Escritura: **escribir** (<var_tipo_archivo>, <var_tipo_base>). Escribe el registro en la posición que tenga en ese momento la clave primaria. Da error si existe, o si tiene claves secundarias sin duplicados.

- Lectura directa: **leer** *(<var_tipo_archivo>, <var_tipo_base>, [<campo clave])*. Por omisión lee el registro cuyo valor coincida con el que en ese momento tenga la clave primaria. Para leer por claves alternativas se incluirá el argumento <campo_clave>.
- Una función **Existe** *(<var_tipo_archivo>)* devolvería verdadero si la lectura ha sido correcta.
- Lectura secuencial: **leersec** *(<var_tipo_archivo>, <var_tipo_base>, [<campo clave])*. Lectura secuencial del siguiente registo de la clave especificada. Por omisión, lee la siguiente clave secuencial.
- Fin de archivo: **fda** *[(var_tipo_archivo)]*. Es verdadero si se ha intentado la lectura después del último registro.
- Otros procedimientos útiles pueden ser: **borrar** *(<nombre_fisico>)*, que borra un archivo que debe estar cerrado, y **renombrar** *(<nombre fisico>, <nombre_fisico>)*, que renombra el archivo primero por el nombre del segundo. Ambos deben estar cerrados.
- Otra función útil puede ser **ida** *(<nombre_fisico>)*, que devuelve la longitud del archivo en bytes.

A.5. VARIABLES DINÁMICAS

- Declaración de tipos dinámicos

  puntero_a *<tipo_de_dato> : <nombre_del_tipo>*

- Acceso al contenido de una variable

  <var_tipo_ptr> ->

- Asignar memoria a una variable de tipo puntero: **reservar** (<var_tipo_ptr>).
- Liberar memoria de una variable de tipo puntero: **liberar**(<var_tipo_ptr>).
- Puntero nulo: **nulo**.

A.6. PALABRAS RESERVADAS, OPERADORES, CARACTERES ESPECIALES, FUNCIONES Y PROCEDIMIENTOS ESTÁNDAR

Tabla A.5. Palabras reservadas, símbolos y operadores

Carácter / Palabra reservada	Significado / Traducción
'	delimitador de caracteres
→	variable apuntada
←	asignación
*	multiplicación
+	suma, concatenación
+	signo positivo
-	signo negativo
, (coma decimal)	
-	menos unario
-	resta

Tabla A.5. Palabras reservadas, símbolos y operadores *(Cont.)*

Carácter / Palabra reservada	Significado / Traducción
/	división real
//	inicio de comentario
:	dos puntos
<	menor que
< =	menor o igual que
< >	distinto de
=	igual a
>	mayor que
> =	mayor o igual a
^	exponenciación
{	inicio de comentario
}	fin de comentario
&	concatenación
abrir	*open*
abs()	*abs*
aleatorio()	*random*
algoritmo	*program*
arctan()	*atm*
archivo_d	definición de archivos directos
archivo_i	definición de indexados
archivo_s	definición de archivos secuenciales
array	*array*
borrar	*delete*
cad()	*str*
cadena	*string*
car()	*chr*
caracter	*char*
clave_p	clave primaria de archivos indexados
clave_s	clave secundaria de archivos indexado
codigo()	*asc*
conjunto	*set*
const	*const*
cos()	*cos*
crear()	inicializa un archivo
cuadrado()	*sqr*
de	*of*
decremento	decremento para bucles desde
desde	*for*
devolver	*return*
div	*div*
duplicada	se admiten duplicados en claves secundarias de archivos indexados
E	inicio del exponente en un real de coma flotante
e	modo de apertura de archivos para escritura
E	parámetro de entrada
E/S	parámetro de entrada/salida

Tabla A.5. Palabras reservadas, símbolos y operadores.*(Cont.)*

Carácter / Palabra reservada	Significado / Traducción
ent()	*int*
entero	*integer*
entonces	*then*
eqv	*eqv*
es	*is*
escribir	*write*
existe()	indica en archivos indexados si una operación de lectura ha sido correcta
exp()	*exp*
Falso	*false*
fda	*eof*
fdl	*eoln*
fin	*end*
fin_desde	*next, end for*
fin_funcion	*end function*
fin_iterar	*loop*
fin_mientras	*end while*
fin_procedimiento	*end sub*
fin_segun	*end select*
fin_si	*end if*
funcion	*function*
hacer	*do*
hasta	*to*
hasta_que	*until*
imp	*imp*
incluir	*include*
incremento	para bucles desde
inicio	*begin*
iterar	*do*
l/d	*I-O*
lda	*lof*
l	*input*
leer	*read*
leercar	*readkey, input$*
leersec	lectura secuencial en archivos indexados
liberar	*dispose*
ln()	*log*
log10()	*log10*
logico	*boolean*
longitud()	*len, length*
llamar_a	*call*
mientras	*while*
mod	*mod*
no	*not*
nulo	*nil*
o	*or*
o_x	*xor*

Tabla A.5. Palabras reservadas, símbolos y operadores.*(Cont.)*

Carácter / Palabra reservada	Significado / Traducción
posicion (<c>, <cb>)	*pos, index, instr*
procedimiento	*procedure*
puntero_a	declaración de tipos de puntero
raiz2	*sqrt*
real	*real*
redondeo()	*round()*
registro	*record, struct*
renombrar	*rename*
repetir	*repeat*
reservar	*new*
S	parámetro de salida
salir	*exit*
segun_sea	*case of*
sen ()	*sin*
si	*if*
si_no	*else*
subcadena (<c>, <ini>, [lond>])	*mid, copy*
subrango	declaración de subrangos
Tamaño_de	*sizeOf*
tipo	*type*
trunc ()	*trunc*
var	*var*
valor ()	*val*
verdad	*true*
y	*and*

APÉNDICE B

GUÍA DEL USUARIO ISO/ANSI PASCAL ESTÁNDAR

Pascal es un lenguaje creado por Niklaus Wirth del Instituto Tecnológico de Zurich a principios de los años setenta, con el propósito de disponer de un lenguaje de alto nivel para la enseñanza de la programación estructurada.

La definición original del lenguaje se encuentra en el *Manual de usuario e Informe*, de Kathleen Jensen y el propio Niklaus Wirth, y ha sido la definición del estándar durante muchos años.

Sin embargo han existido de hecho varias definiciones del estándar. Por ejemplo, en Estados Unidos se ha tomado como estándar la definición del lenguaje hecha conjuntamente por el American National Standards Institute (**ANSI**) y el Institute of Electrical and Electronics Engineers (**IEEE**); esta definición es la que se ha denominado ANSI Pascal.

B.1. ELEMENTOS DEL LENGUAJE PASCAL

Los elementos del lenguaje Pascal nos permiten construir desde las sentencias más simples hasta las más complejas, así como los distintos tipos de datos, tanto predefinidos como aquellos que son definidos por el usuario. Estos elementos incluyen conjunto de caracteres o alfabeto de caracteres, palabras reservadas, identificadores estándar, tipos de datos numéricos, carácter, etc.

B.1.1. Conjunto de caracteres

Pascal reconoce letras, dígitos, símbolos especiales y símbolos palabras (palabras reservadas).

B.1.2. Símbolos especiales

```
+       =       ]       ;       >=      &       &
-       <       .       ^       )       :=      !
*       >       ,       <>      (       «»      {
/       [       :       <=      -       ,       }
```

B.1.3. Palabras reservadas

Las palabras reservadas o palabras clave son aquéllas a las que Pascal da un significado fijo y predeterminado, y que el programador no puede cambiar. Las palabras reservadas de Pascal estándar son las siguientes:

```
and         end         nil         set
array       file        not         then
begin       for         of          to
case        function    or          type
const       goto        packed      until
div         if          procedure   var
do          in          program     while
downto      label       record      with
else        mod         repeat
```

B.1.4. Identificadores

Un identificador es una combinación de letras y dígitos, que debe comenzar con una letra y que permite al usuario especificar constantes, variables, procedimientos, etc.:

```
Nombre    NombreApellidos      Nomina      NominaMes
```

Pascal contiene ciertos identificadores predeclarados o estándar, pero que, a diferencia de las palabras reservadas, pueden redefinirse por el usuario, aunque no se recomienda esta posibilidad.

B.1.5. Cadena de caracteres

Secuencia de caracteres encerrados entre apóstrofos. Para incluir un apóstrofo en una cadena, escriba el apóstrofo dos veces.

```
'Esta es una cadena'

'Esta cadena no contiene apostrofos, "esta si", fin'
```

B.1.6. Separadores

Espacios, retornos de carro y sentencias comentario.

B.1.7. Comentarios

Los comentarios se utilizan para documentar programas. Los comentarios se encierran entre llaves «{}» o entre símbolos «(*» y «*)».

```
{esto es un comentario}
(*esto tambien es un comentario*)
```

Los comentarios pueden incluir también cualquier carácter excepto una llave }.

```
{esto es {esto es comentario} esto no es}
```

B.1.8. Etiquetas

Las etiquetas son enteros con signo utilizados para señalar una sentencia Pascal.

```
Etiqueta
  15    100    9999
```

B.1.9. Directivas

Las directivas son nombres que sustituyen a bloques de funciones y procedimientos.

B.2. TABLA DE IDENTIFICADORES ESTÁNDAR

Los identificadores estándar son de muy diversos tipos. La característica fundamental es que pueden ser redefinidos por el usuario, aun cuando esta posibilidad no se recomienda porque podría inducir a confusión a otros usuarios o programadores que emplearán un código fuente con los identificadores estándar redefinidos.

B.2.1. Constantes

```
False       Maxint       True
```

B.2.2. Tipos estándares

```
Boolean     Char         Integer      Real         Text
```

B.2.3. Variables archivo

```
Input       Output
```

B.2.4. Funciones

```
Abs         ArcTan      Chr         Cos         Eof(F)      Eoln(F)
Exp         Ln          Odd         Ord         Pred        Round
Sin         Sqrt        Succ        Trunc
```

B.2.5. Procedimientos

```
Dispose     Get         New         Pack        Page        Put         Read
ReadLn      Reset       Rewrite     Unpack      Write       WriteLn
```

B.2.6. Listado alfabético

```
Abs         False       Pack        Sin
ArcTan      Get         Page        Sqr
Boolean     Input       Pred        Sqrt
Char        Integer     Put         Succ
Chr         Ln          Read        Text
Cos         MaxInt      ReadLn      True
Dispose     New         Real        Trunc
Eof         Odd         Reset       Unpack
Eoln        Ord         Rewrite     Write
Exp         Output      Round       Writeln
```

B.3. ESTRUCTURA DE UN PROGRAMA

La estructura general de un programa en Pascal es:

Cabecera de programa
...
Declaraciones
...
Bloque
...

B.3.1. Cabecera de un programa

```
program identificador (input, output);
program Media (input, output);
program Tablas (output);
program HojaDeCalculo;
```

B.3.2. Declaraciones

Como su propio nombre indica, en las declaraciones se definen todos aquellos elementos que luego se utilizarán durante la ejecución del programa: etiquetas, constantes, tipos de datos, variables, subprogramas (procedimientos y funciones), etc.

Declaraciones de etiquetas

```
Etiqueta
  900;
```

Se utilizan para el empleo de la sentencia de transferencia incondicional *goto*.

Declaración de constantes

```
const
  Pi       = 3.14159
  Epsilon  = -0.00001;
  Estrella = '*';
  Bandera  = TRUE
  Maximo   = 100;
```

Una constante debe ser definida antes de ser referenciada.

Declaración de tipos

```
type
  lista_datos = array[1..Maximo] of real;
```

En la declaración de tipos se incluyen todos los tipos de datos no predefinidos en el lenguaje.

Declaración de variables

```
var
  Altura, Hipotenusa : real;
  Pulgadas           : real;
  Cuenta             : integer;
  Encontrado, Final  : boolean;
```

Todas las variables de un programa deben ser declaradas.

Declaración de procedimientos y funciones

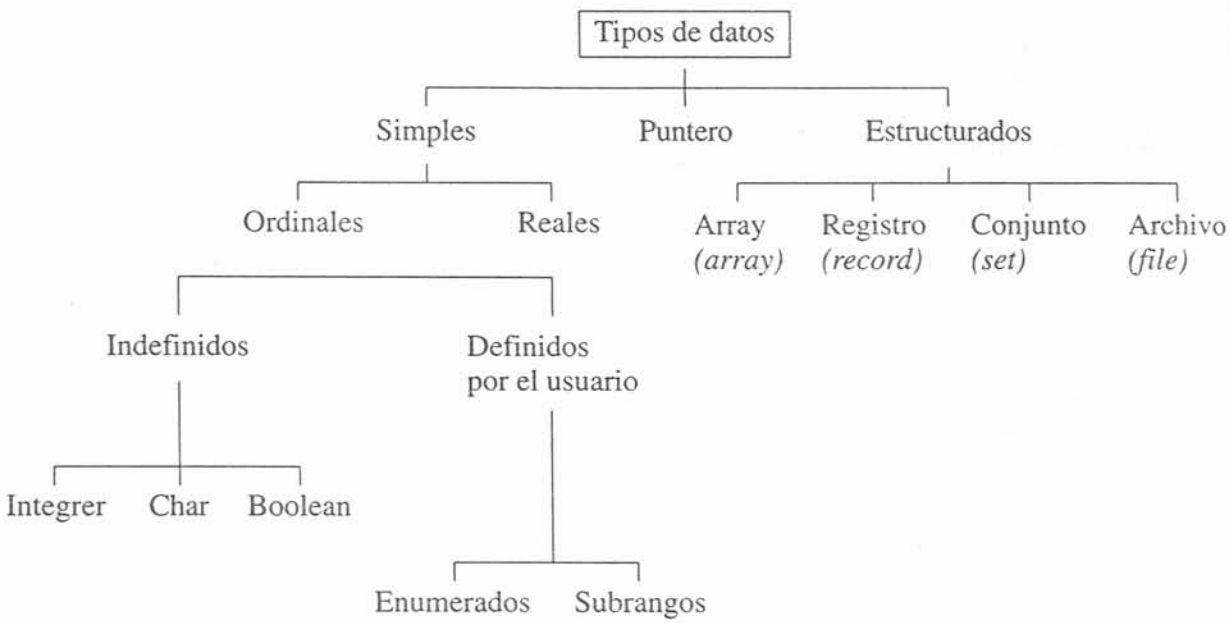
```
procedure Cambiar (var a,b : integer);
var
  aux : integer;
begin
  aux :=a;
  a := b;
  b := aux
end;

function media (A : TipoArray; N : integer) : real;
var
  i : integer;
  suma : real;
begin
  suma := 0;
  for i := 1 to N do
    suma : = suma + A[i];
  media := suma / N
end;
      {otras declaraciones de funciones y procedimientos}
      ...
```

B.3.3. Bloque o cuerpo principal

```
begin
  sentencias;
end.
```

```
begin              {cuerpo del programa}
  Cabecera;        {llamada al procedimiento Cabecera}
  LeerDatos(Datos);
  CalMedia;
  CalDesvEstandar;
  Write('Prueba de test');
  ReadLn(TestUno);
end.               {programa}
```

B.4. TIPOS DE DATOS



B.4.1. Definición de tipos

Una sentencia de definición de tipos crea un tipo de dato definido por el usuario.

nuevo_identificador = identificador denotador_tipo;

```
type
  estado = boolean;

var
  terminado, final : estado;
```

Una definición de tipo define los valores permisibles para un nuevo tipo de variable.

Las definiciones de tipo se pueden utilizar para crear tipos enumerados, tipos subrango, tipos estructurados y tipos puntero, así como sinónimos de tipos simples.

B.4.2. Tipos simples

Los tipos simples que permite Pascal estandar son los tipos numéricos *integer* (enteros) y *real* (reales), el tipo *char* (carácter), el tipo *boolean* (lógicos o booleanos), todos ellos predefinidos por el lenguaje, y los tipos enumerado y subrango, que al poderse definir por el usuario carecen de identificadores específicos.

Integer

Rango de enteros válido: `32768..32767 (MaxInt)`

```
var
   Cuenta : Integer;
```

Real

El rango de valores reales dependerá de cada computador.

```
var
   Media, Datos : Real;
```

Boolean

Un valor boolean es un valor lógico. El valor de una variable boolean puede ser uno de los dos identificadores predefinidos true o false. Los números ordinales asociados a true y false son 1 y 0 respectivamente.

```
var
   Estado : Boolean;
```

Char

Las variables pueden representar caracteres simples. Los valores carácter se representan por un carácter encerrado entre comillas simples.

```
'i'   'e'   'a'

var
   Letra : Char;
```

Tipo enumerado

Un tipo enumerado es una lista de identificadores que determina un conjunto ordenado de valores. Los identificadores en la lista se convierten en constantes. El orden de los valores se determina por el orden en que están en la lista, asignando al primer elemento el orden 0.

```
type
  dias  = (lunes, martes, miercoles, jueves, viernes);
  sexo  = (macho, hembra);
  meses = (ene, feb, mar, abri, may, jun, jul, ago, sep, oct, nov, dic);
```

Tipo subrango

Un subrango es un rango o intervalo de valores consecutivos de un tipo ordinal. Un subrango se especifica por los valores iniciales y finales separados por el símbolo «..».

```
type
  dela10   = 1 .. 10;
  pruebas  ='a' .. 'b';
  numorden = 1 .. maxint;

var
  numero : dela10;
```

Los subtipos se pueden especificar directamente en declaraciones de variables. Por ejemplo:

```
var
  unidades : 0..9;
  verano   : jun..sep;
```

B.4.3. Tipos estructurados o compuestos

Son todos aquellos que se forman por unión de otros de tipo más simple, ya sean homogéneos o heterogéneos. Los más importantes son los arrays, registros, conjuntos y archivos.

Tipos estructurados empaquetados

Las estructuras de datos grandes requieren grandes cantidades de espacio de almacenamiento. El empaquetamiento reduce el espacio necesario para tales estructuras, almacenando los datos de modo más eficiente, ocupando el menor espacio posible[1].

Un tipo de dato se empaqueta si está precedido por la palabra reservada **packed** en una definición de tipo o en una declaración de variable. Por ejemplo:

```
type
  nombre = packed array [1..10] of char;
```

El inconveniente de las estructuras empaquetadas es la ralentización de la ejecución del programa.

Arrays

Un array es una variable indexada que representa un grupo de variables del mismo tipo de dato. Un elemento (o componente) es un miembro de array. Un array consta de un número fijo de componentes, todos ellos del mismo tipo. A cada componente del array se accede explícitamente por el nombre del array y su índice correspondiente.

---

[1] El empaquetamiento de los datos depende fundamentalmente de la implementación. En aquellas computadoras en que una posición de memoria (palabra) es lo suficientemente grande como para almacenar, por ejemplo, más de un carácter (byte), el empaquetamiento ahorra gran cantidad de memoria. Esto es lo que sucede en grandes ordenadores. Aquellas otras en que el tamaño de la memoria es igual que el tamaño del dato que en ella se quiere almacenar, el empaquetamiento no ahorra memoria.

La definición de un tipo array sigue este formato:

```
type
  A = array [T] of T2;

    A    nombre del tipo array
    T    tipo de indice
    T2   cualquier tipo valido
```

Algunos ejemplos son:

```
type
  TipoLista = array [1..100] of Integer;

var
Vector = TipoLista;
```

o bien se puede declarar directamente:

```
var
  Vector : array [1..100] of Integer;
```

El acceso a un elemento del array se hará:

```
Vector[I];  Vector[7];
```

Arrays multidimensionales

Si el tipo de dato T2 es también un tipo array, entonces el tipo array original se dice que es *multidimensional*. Por consiguiente, la declaración de un array multidimensional `M` es:

```
var
  M : array [A..B] of array [C..D] of T;
```

donde `M[I][J]` representa el componente `J` (de tipo `T`) del componente `I` de `M`. En arrays multidimensionales, no obstante se representa la siguiente abreviatura en la notación:

```
var
M: array: [A..B, C..D] of T;
```

donde `M[I,J]` representa el elemento `J` del componente `I` de `M` o el elemento de la fila `I` y columna `J` del array —matriz— `M`.

Un array puede tener índices de distinto tipo. Por ejemplo:

```
type
  Colores = (rojo, blaco, amarillo, azul);

var
  Puntos : array [Colores, 1..10, 1..5] of Real;
```

Un elemento del array `Puntos` tiene el formato:

```
Puntos[nombre_del_color, numero, numero]
```

Las tres definiciones siguientes son equivalentes:

```
array [Colores, 1..10, 1..5] of Real;
array [Colores] of array [1..10, 1..5] of Real;
array [Colores] of array [1..10] of array [1..5] of Real;
```

Tipos Cadena (*string*)

Las cadenas son una secuencia de caracteres encerrados entre comillas y son una clase especial de arrays empaquetados con componentes de tipo `Char`. Las cadenas de un solo carácter son el tipo estándar `Char`. Una cadena se considera como una constante del tipo.

```
packed array [1...n] of Char;
```

donde `n` es la longitud de la cadena.

Ejemplos de cadenas:

```
'Hola Mundo'
'123456789012345'
```

Pascal estándar no soporta el tipo cadena (*string*) que soportan otras versiones de Pascal (como Turbo Pascal). Una cadena debe ser leída carácter a carácter, aunque puede ser escrita por una sentencia `Write` o `WriteLn`. El siguiente programa ilustra el uso de cadenas:

```
program Cadenas(input, output);
const
  palabra = 'Hola Mundo';

type
  Cadena10 = packed array [1..10] of char;
  Cadena20 = packed array [1..20] of char;

var
  indice : integer;
  palentrada : cadena20;
  palsalida  : cadena10;

begin
  for indice := 1 to 20 do
    read (palentrada[indice]);          {lectura de un caracter}
  palsalida := palabra;
  WriteLn (palsalida, ';', palentrada)  {escritura cadena completa}
end.
```

Empaquetar y desempaquetar

El acceso a los elementos de un array empaquetado suele ser laborioso y a veces requiere el empaquetamiento o desempaquetamiento del array. Esta operación es posible mediante los productos predeclarados de transferencia: `Pack` y `Unpack`.

```
pack (array_sin_empaquetar, indice, array_empaquetado)
unpack (array_empaquetado, array_sin_empaqueter, indice)

pack(ArrayU, 6, ArrayP);
unpack(palabra, letras, 2);
```

Registros

Un registro es una estructura de datos compuesta por campos de diferentes tipos. Su sintaxis es:

```
record
  declaraciones de campos
end;
```

donde *declaraciones de campos* consta de una secuencia de lista de identificadores de campos, que a su vez consta de una lista de identificadores seguida por un símbolo dos puntos (:) y el tipo.

```
record
  identificador_campo1 : tipo1;
    ...
  identificador_campoN : tipoN
end;      {descripcion de registro}
```

Ejemplo:

```
record
  Nombre  : packed array [1..20] of char;
  Horas   : real;
  Salario : real
end;

type
  Complejo = record
               Real, Imaginaria : Real;
             end;

var
  Z : Complejo;
```

Operaciones definidas en registros

Las operaciones definidas para el tipo registro son:

1. Asignación.
2. Selección de un componente del registro.
3. Lectura de un archivo de registros.
4. Escritura de un archivo de registros.

```
const
  LongNombre = 20;
type
  TipoNombre = packed array[1..LongNombre] of char;
  RegPersona = record
                 Nombre   : TipoNombre;
                 Horas,
                 Salario,
                 Tasas    : real;
               end;  {descripcion registro}
var
  Personal, Persona2 : RegPersona;
```

Asignación

Dos variables registro del mismo tipo de registro se pueden asignar una a otra.

```
Personal := Persona2; {valida}
Persona2 := Personal; {valida}
```

Acceso a un campo

Se selecciona un componente (campo) de un registro con la notación:

VariableRegistro.NombreCampo

```
Personal.Nombre
Personal.Nombre[5]  selecciona el quinto caracter del campo Nombre del registro Persona
```

La sentencia **with** ayuda a procesar campos de una variable registro, de acuerdo a la siguiente sintaxis:

```
with Variable_Registro do
  sentencia
```

La sentencia controlada por `with` es normalmente una sentencia compuesta.

```
with Personal do
  begin
    Nombre  := 'Pepe Mortimer'
    Horas   := 5000;
    Salario := 200000;
    Tasas   := 35;
  end;
```

La sentencia **with** ahorra esfuerzo de escritura.

Registros variantes

Un tipo de registro variante es aquel que incluye campos adicionales que dependen del valor de otro campo. La estructura de un registro que contiene una parte variante consta de una parte fija que viene en primer lugar, seguida de una parte variante.

La parte variante se construye mediante la palabra reservada **case**[2]. Una estructura de datos que se acopla perfectamente a un registro con variante es:

```
case IdentificadorEtiqueta : TipoIdentificador of
  Lista_de_constantes : ( ListaCampos );
  Lista_de_constantes : ( ListaCampos );
   ...
  Lista_de_constantes : ( ListaCampos );
```

[2] No debe confundirse con la sentencia **case-of**, ya que no lleva el **end** correspondiente.

Así, una estructura típica:

Profesor
1. Nombre
2. Altura
3. Sexo
4. Fecha de nacimiento
5. Estado
   Si está casado o viudo
      a) Fecha de matrimonio (año, mes y día)
   Si está divorciado
      a) Fecha del divorcio (año, mes y día)
      b) Primer divorcio (verdadero, falso)
   Si está soltero
      no hacer nada

se representa por la sintaxis:

```
type
   Cadena20 = packed array [1..20] of char;
   Estado   = (Casado, Viudo, Divorciado, Soltero);
   Fecha    = packed record
                Anyo: 1900..2002;
                Mes : (Ene, Feb, Mar, Abr, May, Jun, Jul, Ago, Sep, Oct, Nov,
                       Dic);
                Dia : 1..31
                Natural : 1..MaxInt
              ebd;
   Profesor = record
                Nombre : record
                           NombrePila,
                           Apellido    : Cadena20;
                         end;
                Altura : Natural;
                Sexo : (Macho, Hembra);
                Nacimiento : Fecha;
                case EM : Estado of
                           Casado, Viudo : (FechaM  : Fecha);
                           Divorciado    : (FechaD, : Fecha;
                                            PrimerD : Boolean);
                           Soltero : ()
   end; {Profesor}
```

Unión libre

Cuando se define un registro sin poner una etiqueta que identifique la parte variante, el formato de los campos variantes se denomina *unión libre.*

```
const
  LongNombre    = 20;
  LongDireccion = 40;
```

```
type
  TipoNombre = packed array[1..LongNombre] of char;
  TipoDirec  = packed array[1..LongDireccion] of char;
  RegPersona = record
                 Nombre : TipoNombre;
                 case Boolean of
                   true  : (Horas, Salario, Tasas : real);
                   false : (Direccion : TipoDirec)
               end;  {descripcion de registro}
```

Conjuntos

Un conjunto es una colección de valores de un único tipo ordinal. El tipo de los valores se llama «tipo base» del conjunto.

Un conjunto se representa por una lista de elementos encerrados por corchetes.

```
['A'..'Z']
[15]
['a', 'b', 'c', 'd', 'e']
[]  conjunto vacio
```

Reglas de construcción de conjuntos

1. Los conjuntos pueden ser vacíos (no tener elementos).
2. Un elemento puede aparecer sólo una vez dentro de un conjunto.
3. El orden de los elementos del conjunto no es significativo.

Definición de un conjunto

Un tipo conjunto se declara como cualquier otro tipo:

```
type
  IdentificadorTipoConjunto = set of TipoOrdinal;

type
Menor100 = set of 1..100;
caracvis = set of char;
colores  = (rojo, naranja, amarillo, azul, verde);

var
alfabeto : set of 'a'..'z';
pinturas : set of colores;
```

Archivos

Un archivo es una estructura de acceso secuencial y cuyos componentes son todos del mismo tipo. Un tipo archivo especifica el tipo de los componentes del campo. El tipo de componente es el tipo de los valores que se pueden almacenar en el archivo. El tipo de componente puede ser cualquier tipo excepto un tipo archivo o un tipo estructurado que contiene un tipo archivo. El identificador predefinido **text** especifica un tipo especial de archivo de caracteres organizado en líneas de texto.

Sintaxis de declaración de archivos

Un tipo archivo se declara como sigue:

```
type
  nombre_archivo : text;
type
  identificador = file of identificador de tipo;
```

o bien:

```
var
  nominas : file of salarios;
  informe : text;
```

Para utilizar un archivo externo, la variable archivo debe ser declarada globalmente. Esto es, el archivo debe aparecer en la sección de declaración de variables del programa. Por ejemplo, para utilizar un archivo externo denominado `archdatos` que contiene valores reales, se utilizará la siguiente declaración:

```
var
  archdatos : file of real;
```

Los archivos externos se pueden especificar como parámetros de programa, en la cabecera del programa, pero no es obligatorio.

Punteros

Una variable de tipo puntero es una variable cuyo valor es una dirección. Esta dirección es la posición de un valor en memoria. Las variables se pueden crear y destruir durante un bloque. A las variables dinámicas se accede mediante punteros.

La sintaxis para declarar un tipo puntero es:

identificador = ^*tipo;*

Por ejemplo:

```
var
  test : ^integer;
```

define la variable `test` como una variable puntero que apunta a valores enteros.

Para acceder al entero al que apunta `test`, se utiliza la sintaxis

```
test^  o bien test↑

test   es un puntero a un entero
test↑  es un entero
```

B.5. TIPOS COMPATIBLES

Los tipos deben ser compatibles en ciertas operaciones. Los tipos son compatibles si una de las siguientes condiciones es verdadera:

1. Los tipos han sido definidos equivalentes en una sentencia de declaración de tipos. Así:

   ```
   type
     estado = boolean;
   ```

   hace a `estado` y `boolean` tipos compatibles.

2. Un tipo es un subrango de otro. Por ejemplo, si:

```
type
  alfabeto = 'a'..'z';
```

`alfabeto` y `char` son tipos compatibles.

3. Ambos tipos y subrangos son del mismo tipo base. Por ejemplo, si:

```
type
  bajo = 1..50;
  alto = 51..500;
```

`bajo` y `alto` son tipos compatibles, ya que ambos son subrango de tipo entero.

4. Ambos tipos son tipos cadena con el mismo número de componentes.

```
type
  nombre  = packed array [1..5] of char;
  palabra = packed array [1..5] of char;
```

`nombre` y `palabra` son compatibles, ya que ambos son cadenas de caracteres de cinco caracteres de longitud.

5. Los tipos son tipos conjunto con tipos base compatibles. Por ejemplo, los tipos definidos por:

```
type
digitos   = set of '0'..'9';
intervalo = set of '4'..'6';
```

`digitos` e `intervalo` son compatibles, ya que tanto las variables como los tipos conjunto son del tipo base `char`.

6. Los tipos no pueden ser compatibles si uno está empaquetado y el otro no. Sólo pueden ser compatibles si ambos están empaquetados.

B.6. OPERADORES

Constituyen los elementos esenciales que transforman y permiten procesar la información. Atendiendo al tipo de operando y al resultado obtenido se dividen en: aritméticos, relacionales y lógicos:

Aritméticos

Operador	Operación	Tipos de operandos	Tipos de resultado
+ (*unario*)	identidad	Integer o Real	igual que operando
– (*unario*)	inversión de signo	Integer o Real	igual que operando
+	suma	Integer o Real	Integer o Real
–	resta	Integer o Real	Integer o Real
*	multiplicación	Integer o Real	Integer o Real
/	división real	Integer o Real	Integer o Real
`div`	división entera	Integer	Integer
`mod`	módulo	Integer	Integer

Operadores relacionales

Operador	Operación	Tipos de operandos	Tipos de resultado
=	igualdad	simple, puntero, conjunto, cadena	Boolean
<>	desigualdad	simple, puntero, conjunto, cadena	Boolean
<=	menor que o igual	simple o cadena	Boolean
>=	mayor que o igual	simple o cadena	Boolean
<=	inclusión de conjunto	conjunto	Boolean
>=	inclusión de conjunto	conjunto	Boolean
<	menor que	simple o cadena	Boolean
>	mayor que	simple o cadena	Boolean
in	pertenencia a conjunto	ordinal y conjunto	Boolean

Operadores lógicos

Operador	Operación	Tipos de operandos	Tipos de resultado
not	negación	Boolean	Boolean
and	conjunción	Boolean	Boolean
or	disyunción	Boolean	Boolean

Operadores de conjuntos

Se emplean con la estructura de datos conjunto. Permiten realizar las operaciones fundamentales de la teoría matemática de los conjuntos.

Operador	Operación	Tipos de operandos	Tipos de resultado
+	unión	conjunto de T	conjunto de T
-	diferencia	conjunto de T	conjunto de T
*	intersección	conjunto de T	conjunto de T

Prioridad de operadores en expresiones

Operador	Clasificación
not	negación lógica
*/ div mod and	operadores de multiplicación
+- or	operadores de adición
= < > < > >= <= in	operadores de relación

Otras operaciones

Operador	Operación	Tipos de operandos	Tipos de resultado
Asignación			
:=	asignación	cualquier tipo asignable	ninguno
Acceso a variables			
[,]	índice de array	array	tipo componente
.	selección de campo	record (registro)	tipo de campo
↑	identificación	puntero	tipo dominio
↑	acceso a buffer	archivo	tipo de componente
Construcción			
[,]	construcción de conjunto	tipo base	conjunto
, ,	construcción de cadenas	char	cadena

B.7. ENTRADAS Y SALIDAS BÁSICAS

La entrada de un programa es la información real o datos que el programa procesa. La salida constituye el resultado del programa.

B.7.1. Salida con *WriteLn* y *Write*

Los formatos generales son:

```
Write (lista de elementos);
WriteLn (lista de elementos);
WriteLn;
```

Algunas sentencias de salida son:

```
Write('Sr. Juan Mackoy');
WriteLn('Calle del Pez, 33');
WriteLn;
WriteLn('Carchelejo');
```

que producen el resultado en la pantalla siguiente:

```
Sr. Juan Mackoy
Calle del Pez, 33

Carchelejo
```

La sentencia `WriteLn;` produce una línea en blanco.

B.7.2. Entrada (Lectura) con *Read* y *ReadLn*

Los formatos generales son:

```
Read (lista de variables);
ReadLn (lista de variables);
ReadLn;
```

Algunas sentencias válidas son:

```
Read(x, y, x);
ReadLn(x, y, z);
ReadLn(n);
Read(n);
```

B.8. SENTENCIAS

Las sentencias constituyen el verdadero código ejecutable de un programa Pascal. Existen diferentes sentencias que se organizan en grupos con características afines.

B.8.1. Sentencia de asignación

Asigna un valor a una variable. Su formato es:

variable := expresión;

en el que se asigna el valor de una expresión a la variable situada a la izquierda.

```
temperatura := 24;
p := p + 24;
q := q + 1;
test := (a < 0) or (t > limite);   {test debe ser Boolean}
```

B.8.2. Sentencia compuesta

Grupos de sentencias encerrados entre las palabras reservadas **begin** y **end**.

```
begin
  sentencias
end
```

La siguiente sentencia es un ejemplo de sentencia compuesta:

```
begin
  aux := x;
  x   := y;
  y   := aux
end;
```

B.8.3. Sentencia procedimiento

Las sentencias procedimiento se utilizan para llamar a procedimientos estándar y definidos por el usuario. Los procedimientos se llaman por su nombre, seguido por una lista de argumentos entre paréntesis, que se pasan al procedimiento. El número de elementos en la lista de argumentos debe corresponder con el número de parámetros en la definición del procedimiento.

```
WriteLn;
SumaDatos;
Invertir (a, b);
Read(fl, a, b, c);
```

B.8.4. Sentencias condicionales

Las sentencias condicionales son sentencias que realizan diferentes acciones en función de una condición o expresión lógica.

Sentencias *if—then—else*

```
if expresion_logica then sentencia [else sentencia];
```

`sentencia:` puede ser una sentencia simple o una sentencia compuesta

Si la `expresion logica` se evalúa verdadera, se ejecuta la sentencia; en caso contrario se ejecuta la sentencia `else` o se termina la sentencia `if`.

```
if x < 10.5
   then z := x + y
   else Z := 10.5;
```

```
if j = 0
   then
   begin
     ...
   end
else
  WriteLn( i / j );
```

Las sentencias **if** pueden estar anidadas.

```
if C1 then S1
  else if C2 then S2
    else ....
      else if C(n-1) then S(n-1) else Sn
```

Sentencia *case*

Una sentencia **case** ejecuta una sentencia entre varias, en función del valor de una expresión especificada. La sintaxis de la sentencia **case** es:

```
case indice_caso of
  lista_constantes1 : sentencia;
  lista_constantes2 : sentencia;
  ...
  lista_constantesn : sentencia; {este; es opcional }
  [otherwise sentencia]
end;
```

`indice_caso` es la variable o expresión cuyo valor se comprueba. Una `lista_constantes` es una lista de valores. Si el valor de un `indice_caso` es igual a uno de los valores en la lista de constantes, la sentencia especificada a continuación se ejecutará, `indice_caso` y la lista de constantes deben ser compatibles.

Algunas versiones de Pascal como LPI-Pascal (además de Turbo Pascal) tienen una cláusula **otherwise** para expresar los valores las acciones que se ejecutan cuando el valor de `indice_caso` no está en ninguna de las listas de constantes.

Los siguientes ejemplos muestran el uso de **case**:

```
case operador of
  mas   : x := x + y;
  menos: x := x - y;
  por   : x := x * y
end;

case PUNTOS of
  10    : WritenLn('Excelente');
  7,8,9: WriteLn('Bueno');
  5,6,  : WriteLn('Normal');
  otherwise WriteLn('Deficiente')
end;
```

Sentencia de bifurcación incondicional: *goto*

Una sentencia **goto** bifurca durante la ejecución de una parte de un programa a una sentencia etiquetada. La sintaxis es:

```
goto etiqueta_sentencia;
```

Una etiqueta es un entero en el rango 0 a 9999 inclusive, seguido por un carácter dos puntos (:).

```
  label
  10;

begin
  if b=0 then goto 10;
  resultado := a/b;
  ...
10 : WriteLn('No se puede dividir por cero');
```

Precaución: El uso de la sentencia *goto* es peligroso. Tenga mucho cuidado si ha de emplearla. Prácticamente no se utiliza nunca.

B.8.5. Sentencias repetitivas (bucles)

Los bucles de programa son series de sentencias que se pueden ejecutar varias veces en sucesión. Existen dos tipos de bucles: bucles indeterminados, que se ejecutan un número no especificado de veces hasta que ocurre una condición específica, y bucles definidos, que se ejecutan un número específico de veces.

Las sentencias **while** y **repeat** crean bucles indeterminados y la sentencia **for** crea un bucle definido.

Sentencia *while*

La sentencia **while** repite una sentencia mientras que una condición sea verdadera. La condición se comprueba antes de que se ejecute la sentencia. Por consiguiente, si la condición es falsa la primera vez que se evalúe la sentencia no se ejecutarán las sentencias del interior del bucle. La sintaxis es:

```
while expresion_logica do sentencia
```

y su estructura típica será:

```
inicializar la condicion o expresion logica
while condicion do
  begin
    sentencias que realizan el bucle
    y cambian la condicion
  end;
```

Seguidamente se muestran dos ejemplos:

```
ReadLn (a,b);
while a <> b do
  if a > b then
    a := a -b
  else b := b - a;
WriteLn('FN');

Suma := 0;
i := 10;
while i <= 25 do
  begin
    Suma := Suma + i;
    i := i + 1
  end;
```

Sentencia *repeat*

La sentencia **repeat** ejecuta una serie de sentencias hasta que una condición especificada sea verdadera. La condición se comprueba después que la secuencia de sentencias se ejecuta. Por consiguiente, las sentencias dentro de una sentencia **repeat** se ejecutarán al menos una vez. La sintaxis de la sentencia **repeat** es:

```
repeat
  secuencia_sentencias
until expresion_logica;
```

La estructura de la sentencia es:

```
repeat
  sentencias que realizan el bucle
  y cambian las condiciones
until condiciones;
```

Un ejemplo de una sentencia **repeat** es:

```
repeat
  k := i mod j;
  i := j;
  j := k
until j = 0;
```

Una sentencia **repeat** puede estar anidada con otra sentencia **repeat**.

Sentencia *for*

La sentencia **for** ejecuta una sentencia un número especificado de veces. La sintaxis de la sentencia **for** es:

```
for variable_control:=valor_inicial [to downto] valor_final do sentencia
```

Su estructura es la siguiente:

```
for var_control := valor_infer to valor_sup do
  begin
    sentencias
  end;
```

O bien:

```
for var_control := valor_sup downto valor_infer do
  begin
    sentencias
  end;
```

Ejemplos de sentencias **for** y sus sentencias **while** equivalentes:

```
Suma : 0;
for Num := 1 to n do
  Suma := Suma + n
```

```
Suma := 0;
Num := 1;
while Num <= n ddo
  begin
    Suma := Suma + Num;
    Num  : = Num + 1;
  end
```

Otro ejemplo de uso de la sentencia **for** es:

```
k := 0
for i := 1 to 1 * n - 1 do
  begin
    WriteLn (i);
    k := k + 1
  end;
```

B.9. PROCEDIMIENTOS Y FUNCIONES DEFINIDOS POR EL USUARIO

Los procedimientos y funciones son rutinas (subprogramas) que realizan acciones específicas. Las rutinas se llaman por su nombre y están seguidas de una lista de parámetros.

La diferencia entre procedimientos y funciones es simple. Las funciones devuelven un valor y los procedimientos no.

B.9.1. Declaración de un procedimiento

```
procedure nombre (lista de declaracion de parametros):
declaraciones
begin      { cuerpo del procedimiento }
    sentencias
end;
lista de declaracion de parametros
lista de parametros : tipo;
```

Algunos ejemplos de declaraciones de procedimientos son:

```
type tabla = array [11..9, 1..9] of Real;
procedure Trasponer(var matriz1 : Tabla);
  { cuerpo del procedimiento }

procedure CabeceraPagina;
begin
  WriteLn ('programa de ayuda');
  ...
end;
```

B.9.2. Llamada a un procedimiento

Se realiza mediante el nombre del procedimiento con los parámetros actuales encerrados entre paréntesis y separados por comas.

```
nombre_procedimiento (lista de parametros);
```

Ejemplo:

```
Trasponer (Lista);
```

B.9.3. Declaración de una función

La declaración de una función tiene el mismo formato que la declaración de un procedimiento, con la diferencia de que, como las funciones devuelven un valor, la cabecera va seguida por dos puntos (:) y un tipo de resultado. Su formato es:

```
function nombre (declaracion de parametros formales) : tipo;
  declaraciones
begin
  sentencias
  nombre_funcion := valor_de_la_funcion
end;
```

Ejemplo:

```
function mod_complejo (parte_real, parte_imag : real) : real;
begin
  mod_complejo := sqrt(sqr(parte_real) + sqr(parte_imag))
end;

function Sigma (n : integer) : integer;
var
  i, total : integer;
begin
  total := 0;
  for i := 1 to non do
    total := total + i;
  Sigma := total;
end;   { Sigma }
```

B.9.4. Llamada a una función

```
1. Variable := Nombre_funcion ( lista de parametros reales );
2. Llamada dentro de una sentencia
```

Ejemplo:

```
function digitos (n : integer) : integer;
...
begin
...
  { llamada a la funcion digitos }
  if cuenta + digitos(numero) > 100 ...
```

B.9.5. Parámetros variables

Los parámetros variables son parámetros formales utilizados para representar un parámetro real cuyo valor puede cambiar durante la ejecución del procedimiento. Los parámetros variables se declaran incluyendo la palabra reservada **var** antes que las variables argumento en la cabecera.

Declaración de parámetros variables (var)

```
var identificadores de parametros formales : identificador de tipo;
```

Ejemplo:

```
procedure cambio(var m, n : integer);
var
  aux : integer;
  begin
  if n > mm then
    begin
      aux := n;
      n := m;
      m := aux;
    end;
end;
{ llamada al procedimiento cambio }
cambio (x, y);
```

B.9.6. Parámetros valor

Los parámetros valor transfieren un valor a una rutina. Los parámetros valor se declaran en la cabecera de la rutina por una lista de identificadores seguidos por sus tipos. Este tipo puede ser cualquier tipo excepto un archivo o un tipo estructurado que contenga un archivo como componente.

Declaración de parámetros valor

```
identificadores de parametros formales : identificador de tipo;
```

Ejemplos:

```
function escala(lista_en : tipo_array; facot_escala : real) : real;

function Superficie(longitud, anchura : real) : real;
  begin
    { convertir pies a pulgadas }
    longitud := 12 * longitud;
    anchura := 12 * anchura;
    Superficie := longitud * anchura;
  end;

{ llamada a la funcion }
espacio := Superficie(base, altura);
```

B.9.7. Parámetros función

Los parámetros funcionales son parámetros formales que se utilizan para representar una función real que se puede utilizar durante la ejecución de la rutina que se declara. Los parámetros funcionales permiten pasar una función a una rutina. Por ejemplo, el procedimiento siguiente aplica un parámetro funcional para transformar una matriz:

```
type
  array_real = array[1..9] of real;
procedure matriz_op(function op(a:real): real;
                    var arraya : array_real);
var
  i : integer;
begin
  for i := 1 to 9 do
    arraya [i] := op(arraya[i]);
end;
```

B.9.8. Parámetros procedimentales

Los parámetros procedimentales son a los parámetros funcionales como los procedimientos son a las funciones. Los parámetros procedurales son parámetros formales que representan un procedimiento actual que se puede utilizar durante la ejecución de la rutina que se ha declarado.

Los parámetros procedimentales tienen el mismo formato que una cabecera de procedimiento.

La siguiente función ejecuta un procedimiento y devuelve el valor 1 cuando se termina:

```
function ejecutar(procedure trabajo): integer;
  begin
    trabajo;
    ejecutar := 1
  end;
```

B.9.9. Directiva *forward*

Las rutinas definidas por el usuario deben ser declaradas antes de ser referenciadas. La directiva **forward** constituye una excepción a esta norma, permitiendo declarar las rutinas sin definir la rutina, de modo que otras rutinas pueden referirse a ella; **forward** indica al compilador Pascal que la definición de la rutina vendrá más tarde en el programa. Si una rutina necesita parámetros o un tipo de retorno, ellos se incluyen en la declaración **forward**.

El siguiente ejemplo muestra la directiva **forward** utilizada en dos definiciones de procedimiento. Estos procedimientos deben ser declarados utilizando la directiva **forward** debido a que son mutuamente recursivos.

```
procedure e(lista de parametros); forward;
procedure a(lista de parametros);
  begin
    ...
    b( lista de argumentos );
    ...
  end;

procedure b(lista de parametros);
  begin
    ...
    a(lista de argumentos);
    ...
  end;
```

B.10. FUNCIONES INTRÍNSECAS

Las funciones intrínsecas o predefinidas se llaman por identificadores predefinidos y se dividen en cuatro categorías:

- *numéricas,*
- *transferencia,*
- *ordinales,*
- *Boolean (lógicas).*

Las funciones numéricas realizan cálculos sobre valores de tipos numéricos. Las funciones de transferencia traducen valores de un tipo a otro. Las funciones ordinales hacen cálculos basados en la posición ordinal de un valor dentro de su tipo. Las funciones Boolean comprueban ciertas condiciones y devuelven un valor Boolean que indica si la condición es verdadera o no.

B.10.1. Funciones numéricas

Las funciones numéricas tienen un argumento que es una expresión (x) simple. Esta expresión puede ser un tipo real o un entero. Las funciones **abs** y **sqr** devuelven un resultado que es del mismo tipo que la expresión. Las otras funciones numéricas siempre devuelven resultados reales.

Función	Descripción
`abs(x)`	Devuelve el valor absoluto de x (|x|)
`arctan(x)`	Devuelve el ángulo en radianes (entre –pi/2 y pi/2)
`cos(x)`	Devuelve el coseno de *x* (*x* en radianes)
`exp(x)`	Devuelve e^x (e = 2.718282) La función `exp` es la inversa de la función `ln`
`sin(x)`	Devuelve el seno de *x* (*x* en radianes)
`sqr(x)`	Devuelve el cuadrado de *x*. La función `sqr` es más eficiente que la expresión, *x* * *x*
`sqtr(x)`	Devuelve la raíz cuadrada positiva de *x*, *x* debe ser mayor o igual a cero

B.10.2. Funciones de transferencia

Las funciones de transferencia toman argumentos reales y devuelven valores enteros.

Función	Descripción
`round(x)`	Redondea *x* (de tipo real) al entero más próximo
`trunc(x)`	Trunca la expresión *x* (de tipo real) y devuelve la pate entera

B.10.3. Funciones ordinales

Las funciones ordinales se basan en el orden de los elementos dentro de un tipo enumerado. El primer elemento es el ordinal 0, el segundo es el ordinal 1, etc.

Función	Descripción
`chr(x)`	Devuelve el carácter cuyo ordinal es el valor de *x*
`ord(x)`	Devuelve el entero que es el número ordinal del valor de *x*. *x* puede ser cualquier tipo ordinal
`pred(x)`	Devuelve el valor cuyo número ordinal es el precedente al valor de *x*
`succ(x)`	Devuelve el valor cuyo número ordinal es el siguiente al valor de *x*

B.10.4. Funciones boolean (lógicas)

Las funciones lógicas verifican la existencia de una condición dada y devuelven *true (verdadero)* o *false (falso).*

Función	Descripción
`eof(f)`	Devuelve *true* (*verdadero*) si se alcanza el final del archivo `f`. En caso contrario, devuelve *falso*
`eoln(f)`	Devuelve *true* si está al final de una línea en `f`. En caso contrario, devuelve falso (*f debe ser un fichero de texto)*
`odd(x)`	Devuelve *true* si la expresión es impar, *false* (falso) si es par. *x* debe ser una expresión entera

B.11. PROCEDIMIENTOS INTRÍNSECOS

Los procedimientos intrínsecos (o estándar) se llaman utilizando identificadores predefinidos. Existen tres categorías: tratamiento de archivos, asignación dinámica y transferencia.

B.11.1. Procedimientos de tratamiento de archivos

Gestionan la entrada y salida de archivos. En la tabla siguiente, el símbolo `f` representa una variable de archivo y el símbolo `f^` representa su variable de *buffer* de archivo.

Función	Descripción
`close(f)`	Cierra el archivo `f`. Una vez que un archivo se ha cerrado, ningún dato se puede leer o escribir en él, a menos que un procedimiento `reset` o `rewrite` se utilice para reabrirlo. Esta extensión no pertenece a ISO Pascal estándar, pero la implementan muchas versiones de compiladores de Pascal
`get(f)`	Obtiene el valor del siguiente componente del archivo `f` en su variable de *buffer* de archivo
`page(f)`	Sitúa el siguiente texto escrito en un archivo `f` en una nueva página
`put(f)`	Escribe el buffer del archivo, `f^`. en el archivo `f`. Después de una llamada a `put (f)`, el valor de la función `eof(f)` será verdadera y el valor de `f^` indefinido
`read(f,vl)`	Lee elementos de un archivo o del terminal. La lista de variables son de tipo entero, real, char o un subrango de éstos tipos. No está permitida la entrada directa de tipos enumerados, Boolean y cadena
`readln(f,vl)`	Funciona igual que `read`. El puntero del archivo avanza al primer carácter de la siguiente línea. Se utiliza sólo con archivos de texto. Si todos los parámetros se omiten, sólo se avanza el puntero del archivo

Función	Descripción
`reset(f)`	Abre un archivo `f` para lectura y pone el puntero del archivo a la primera posición del archivo
`rewrite(f)`	Crea un archivo `f` (o borra `f` si ya existe), lo abre para escritura y pone el puntero a la primera posición del archivo
`write(f,el)`	Escribe un elemento o una lista de elementos al terminar o al final del archivo
`writeln(f,el)`	Funciona igual que `write` añadiendo un retorno de carro. Sólo se utiliza con archivos de textos. Si todos los parámetros se omiten, sólo se escribe un retorno de carro

B.11.2. Procedimientos de asignación dinámica

Cambia el espacio de memoria asignado durante la ejecución de un programa.

Función	Descripción
`new(p)`	Asigna dinámicamente espacio de almacenamiento a una nueva variable de puntero. *p* es una variable puntero. *p*^ es la variable creada por el procedimiento `new`
`dispose(p)`	Elimina una variable creada por el procedimiento `new`. *p* es una variable puntero. El procedimiento dispose suprime la variable, *p*^, a la que está apuntando el puntero

B.11.3. Procedimientos de transferencia (empaquetamiento)

Los procedimientos de transferencia, transfieren datos de una variable de un tipo a otro. Convierten datos entre arrays empaquetados y sin empaquetar.

Función	Descripción
`pack(a,i,b)`	Empaqueta el array `a`, comenzando en la posición `i`, en el array empaquetado `b`
`unpack(b,a,i)`	Desempaqueta el array empaquetado `b` en el array `a`, comenzando en la posición `i`

ACTIVIDADES DE PROGRAMACIÓN RESUELTAS

B.1. Codificar un programa que muestre el dibujo de un pino en pantalla.

Codificación

```
program pino (output);
var
  i,j:integer;
begin
  for i:= 1 to 25 do
    writeln;
  for i:= 1 to 10 do
    begin
      write (' ':40 - i);
      for J;=1 to (2 * i - 1) do
        write ('*');
      writeln
    end;
  for i:= 1 to 5 do
    begin
      write ('**':40);
      writeln
    end
end.
```

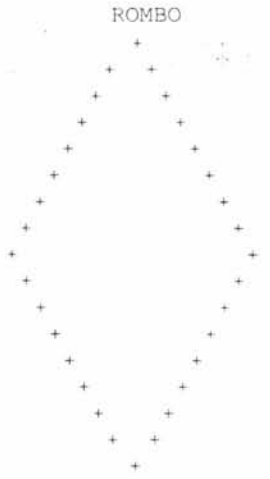
B.2. Codificar un programa que muestre un rombo en pantalla.

Codificación

```
program  rombol (input, output);
var
  eje, vcb, altura: integer;
  motivo : char;
procedure clrscr;
var
  i : integer;
begin
  for i := 1 to 25 do
    writeln
end;

begin
  clrscr;
  write ('introduce eje y altura impar:)'
  readln (eje, altura);
  write ('introduzca motivo: ');
  readln (motivo);
  writeln ('ROMBO': 42);
  writeln (motivo: eje);
  for vcb := 2 to altura div 2 + 1 do
    writeln (motivo: eje + 1 - vcb, motivo: 2 * (vcb - 1));
  for vcb := altura div 2 + 2 to altura - 1 do
    writeln (motivo: eje + vcb - altura, motivo: 2 * (altura - vcb));
  writeln (motivo:eje)
end.
```

Ejecución

```
introduce eje y altura impar: 41 21
introduce motivo: +
```



B.3. Diseñar un programa que muestre el número de días de un mes dado e indicar el ordinal de la semana del día de la fecha conociendo el correspondiente al 1 de enero de ese mismo año.

Codificación

```
program fechas (input,output);
(* calcula el numero de dias de un mes correspondiente a una fecha
   dada y determina el ordinal de la semana correspondiente al dia
   propuesto, conociendo el ordinal de la semana para
   el dia 1 de enero de ese mismo año.
   Se supone que el año es bisiesto si es multiplo de 4.*)
type
  diasmes = array [1..12] of 28..31;
var
  mes,
  mesactual,
  unoenero,
  pico,
  diactual,
  totdias : integer;
  dias:diasmes;
  agnoactual:1..2000;
begin
  for pico := 1 to 25 do
    writeln;
  write('introduce dia mes y año problema: ');
  readln (diactual, mesactual, agnoactual);
  for mes := 1 to 12 do
    case mes of
     1,3,5,7,8,10,12: dias[mes]:=31;
     4,6,9,11:        dias[mes]:=30;
     2:               if agnoactual mod 4 = 0 then
                        dias[2]:= 29
                      else
                        dias[2]:= 28;
    end;
```

```
  writeln ('El mes', mesactual,'del año', agnoactual,
         ' tiene ',dias[mesactual]:3, ' dias');
  writeln ('introduce ordinal semana del 1 de enero: ');
  readln (unoenero);
  totdias:= diactual;
  for mes:= 1 to mesactual -1 do
    totdias:= totdias + dias[mes];
  writeln ('dias del año transcurridos ',totdias:4);
  pico:= totdias mod 7; (*calcula dias sobrantes*)
  pico:= (pico + unoenero -1) mod 7;
  writeln ('el dia propuesto es el ',pico:2,' de la semana')
end.
```

Ejecución

```
introduce dia mes y año problema: 12 8 1994
El mes 8 del año 1994 tiene 31 dias
introduce ordinal semana del 1 de enero: 6
dias del año transcurridos 224
el dia propuesto es el 5 de la semana
```

B.4. Construir un programa para generar una agenda con números de teléfono y que permita realizar consultas.

Codificación

```
program agenda (input,output);
const
  maximo=10;
type
  nombre = packed array [1..15] of char;
  telefono = packed array [1..11] of char;
  conocidos = array [1..maximo] of nombre;
  tlfnos = array [1..maximo] of telefono;
var
  amigo : conocidos;
  numero : tlfnos;
  persona : nombre;
  tfo : telefono;
  vcb : 1..maximo;
  i,j : 1..maximo;
  camigo : nombre;
  ctfno : telefono;
  terminado : boolean;
  otra : char;

procedure buscar
          (humano :nombre; lista1 :conocidos; lista2 :telefonos);
var
  hallado :boolean;
  primero,ultimo,mitad :1..maximo;
begin
  hallado:= false;
  primero:= 1;
  ultimo:= maximo;
  while not hallado ADN (primero <= ultimo) do
```

```
    begin
      mitad := (primero + ultimo) div 2;
      if humano = lista1[mitad] then
        hallado:= true
      else
        if humano > lista[1mitad] then
          primero:= mitad + 1
        else
          ultimo:= mitad - 1;
    end;
  if hallado then
    writeln('su telefono es',lista2[mitad])
  else
    writeln(' no consta en la agenda');
end;

begin
  for vcb:= 1 to maximo do
    begin
      write ('introduce nombre');
      readln(amigo[vcb]);
      write('introduce telefono');
      readln(numero[vcb])
    end;
        (*ORDENACION DE LAS MATRICES DE NOMBRES Y TELEFONOS*)
  for i:= 1 to maximo - 1 do
    for j:= i + 1 to maximo do
      if amigo[i] > amigo [j] then
        begin
          camigo:= amigo[i];
          ctfno:= numero [i];
          amigo[i]:= amigo [j];
          numero[i]:= numero [j];
          amigo[j]:= camigo;
          numero[j] := ctfno
        end;
         (*BUSQUEDA DE DATOS PERSONALES*)
  repeat
    write('introduce nombre de la persona a buscar:');
    readln(persona);
    buscar (persona,amigo,numero);
    writeln ('otra persona? (s/n): ');
    readln (otra);
    terminado := otra ='n'
  until terminado
end.
```

B.5. Codificar un programa que cuente con el número de apariciones de un carácter en una línea de texto introducida por teclado.

Codificación

```
program buscaletra (input,output);
{Este programa cuenta el numero de veces que aparece un caracter}
var
  buscado,
  car: char;
  numero: integer;
```

```
begin
  numero:=0;
  writelin ('introduce caracter a buscar');
  readln (buscado);
  writeln ('introduce una linea de texto');
  while not eoln(input) do
    begin
      read(car);
      if car = buscado then
      numero := numero + 1
    end;
  writeln;
  writeln (buscado:1,' aparece', numero:1, 'veces')
end.
```

Ejecución

```
introduce caracter a buscar A
introduce una linea de texto
EN UN LUGAR DE LA MANCHA DE CUYO NOMBRE NO QUIERO ACORDARME…
A aparece 6 veces

introduce caracter a buscar O
introduce una linea de texto
EN UN LUGAR DE LA MANCHA DE CUYO NOMBRE NO QUIERO ACORDARME…
O aparece 5 veces

introduce caracter a buscar?
introduce una linea de texto
¿QUIEN HA SIDO EL CULPABLE DE SEMEJANTES DESMANES?
? aparece 1 veces

introduce caracter a buscar *
introduce una linea de texto
BLAISE PASCAL: FILOSOFO, FISICO Y MATEMATICO DEL SIGLO XVII
* aparece 0 veces
```

B.6. Codificar un programa que indique si un número es capicúa (414, 3113, 45254, …).

Codificación

```
program capicua (input, output);
{ Este programa determina si un numero entero es capicua}
var
  numero,
  copia,
  alreves,
  resto : integer;
begin
  writeln ('introduce numero a investigar');
  readln (numero);
  copia := numero;
  alreves := 0; {Para ir introduciendo el numero invertido}
  while numero 0 do
    begin
      resto := numero mod 10;
      alreves := alreves * 10;          { corre a la izquierda }
```

```
      alreves := alreves + resto;          { aumenta las unidades }
      writeln ('alreves =',alreves: 1);
      numero:= numero div 10
    end;
  write (copia: 1);          { escribe el número introducido }
  if al reves = copia then
    writeln ('es capicua')
  else
    writeln ('no es capicua')
end.
```

Ejecución

```
introduce numero a investigar
16832
alreves =2
alreves =23
alreves =238
alreves =2386
alreves =23861

16832 no es capicua

introduce numero a investigar
2442
alreves =2
alreves =24
alreves =244
alreves =2442

2442 es capicua
```

ESTRUCTURA DE UN PROGRAMA PASCAL

```
program nombre (input, output) {cabecera}
{estructura general de un programa}
{seccion de declaraciones}
label etiquetas
const identificador = valor constante;
type nombre_tipo = definición_tipo;
var identificador = tipo de datos;
{declaraciones}
procedure y function
{sentencias del programa principal}
begin
   <sentencia 1>;
   <sentencia 2>;
            .
            .
            .
   <sentencia n>
end
```

Programa fuente en Pascal: Demo.Pas

```
{Formato general de un programa Pascal. Cabecera del programa.
}
Program nombre_programa (entrada,salida,otros_ficheros);
{
Seccion de declaraciones
}
const
  nombre_const = valor;
   *      *      *
   *      *      *
tipe
  nombre_tipo = espec_tipo;
   *      *      *
   *      *      *
var
  nombre_var,
  nombre_var,
  nombre_var,
                :nombre_tipo;
   *      *      *
   *      *      *
function nombre_función (parametros): nombre_tipo;
  const
  type
  var
  value
  begin        {Cuerpo de la funcion}
                  *      *      *
                  *      *      *
  end;
value      {valido en PASCAL de IBM PC}
  nombre_var := valor_inicial;
   *      *      *
   *      *      *
procedure nombre_procedimiento (parametros: nombre_tipo);
  const
  type
  var
  value
  begin        {Cuerpo del procedimiento}
                  *      *      *
                  *      *      *
  end;
{
{Seccion de sentencias ejecutables}
begin          {Cuerpo del programa principal}
                  *      *      *
                  *      *      *
end
```

APÉNDICE C

GUÍA DE SINTAXIS TURBO/BORLAND PASCAL 7.0

C.1. ELEMENTOS BÁSICOS DEL LENGUAJE

Palabras reservadas

and	file	not	then
array	for	object	to
asm	function	of	type
begin	goto	or	unit
case	if	packet	until
const	implementation	procedure	uses
constructor	in	program	var
div	inherited	record	while
do	inline	repeat	with
downto	interface	set	xor
else	label	shl	
end	mod	shr	
	nil	string	

Directivas (se pueden redefinir)

absolute	far	near	virtual
assembler	forward	private	
external	interrupt	public	

C.1.1. Identificadores

WriteLn	Exit	CadenaReal	System Rosas
Un_mes	Nombre_Apellidos	Dia_del_Mes	Pedro_uno

C.1.2. Cadenas de caracteres

```
'Turbo'      {Turbo}
''''         {'}
''           {cadena nula}
' '          {espacio}
```

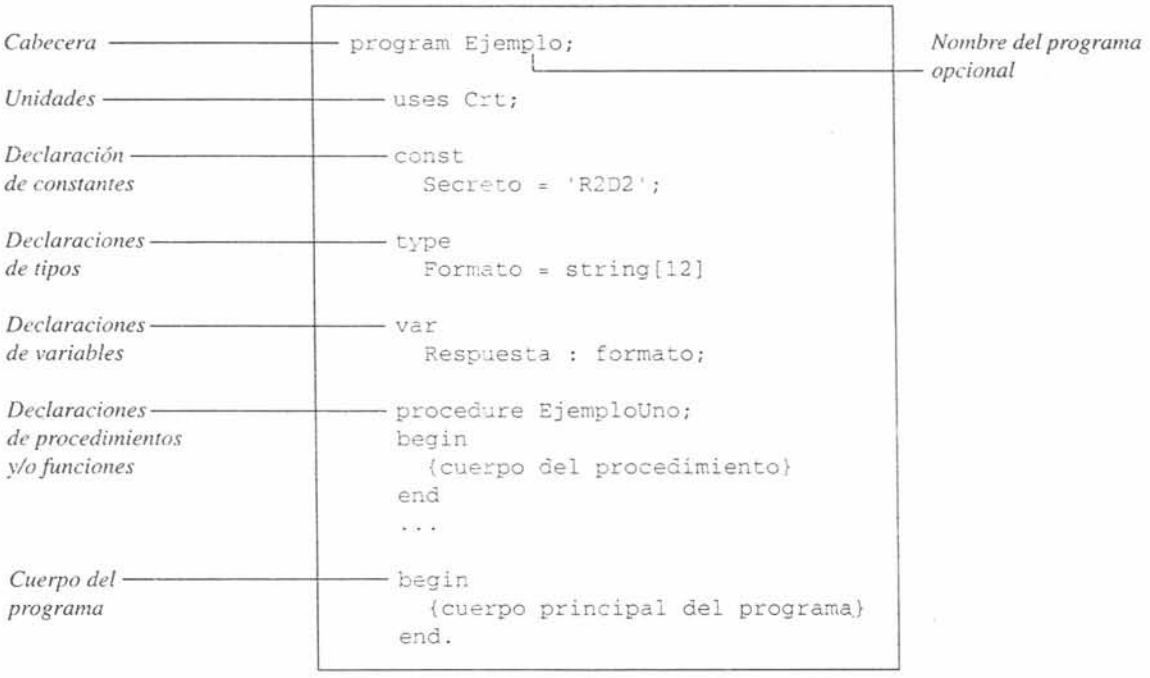
C.1.3. Comentarios

```
{Cualquier texto encerrado entre llaves}
(*Es tambien un comentario valido*)
```

C.1.4. Línea de programa

Las líneas de programas Turbo Pascal tienen una longitud máxima de 126 caracteres.

C.1.5. Estructura de un programa



C.1.6. Procedimientos y funciones

Cabecera procedimiento

```
procedure identificador (lista de parámetros);
  {declaraciones de datos}
begin
  {cuerpo del procedimiento}
end;
```

Cabecera función

```
function identificador (lista de parámetros);
  {declaraciones de datos}
begin
  {cuerpo de la función}
end;
```

C.2. UNIDADES

C.2.1. Unidades estándar

`Crt`	Soporte de pantalla y teclados
`Dos, WinDos`	Funciones de propósito general DOS
`Graph`	Rutinas gráficas
`Graph3`	Gráficos compatibles versión 3
`Overlay`	Implementa el administrador de solapamientos
`Printer`	Acceso a la impresora
`Strings`	Tratamiento de cadenas terminadas en nulo
`System`	Rutinas de la biblioteca en tiempo de ejecución
`Turbo3`	Mantener compatibilidad con Turbo Pascal 3.0

C.2.2. Sintaxis de la unidad

```
unit identificador
interface lista de unidades;                          opcional
  {declaraciones publicas}
  {cabeceras de todas las subrutinas publicas}
implementation
uses lista de unidades                                opcional
  {declaraciones privadas}
  {implementacion de procedimientos y funciones}
begin
  {codigo de inicializacion}                          opcional
end.
```

C.3. DISPOSITIVOS

`CON`	Terminal
`PRN`	Impresora
`AUX`	Dispositivo auxiliar
`LPT1`	Impresora
`NUL`	Dispositivo nulo

C.4. TIPOS DE DATOS

C.4.1. Tipos simples

Boolean (lógicos) `true  false`

Números enteros:

```
Byte                    0 .. 255
Integer              -32768 .. 32767
LongInt       2147483648 .. 2147483647
ShortInt             -128 .. 127
Word                    0 .. 65535
```

Números reales:

Real	$2\ 9 \times 10^{-39}$ ..1.7×10^{38}
Single	1.5×10^{-45} ..3.4×10^{38}
Double	5.0×10^{-324} ..1.7×10^{308}
Extended	3.4×10^{-4932}..1.1×10^{4932}
Comp	$-2^{63} + 1..2^{63} - 1$

Carácter:

Char	Cualquier carácter ASCII

Cadenas:

String	Secuencia de hasta 255 caracteres ASCII. Si no se especifica un tamaño, se utilizará por defecto 255

Punteros:

Pointer	Dirección de un elemento dato, procedimiento o función

C.4.2. Tipos subrango

```
    0..99
  -128..127
Lunes..Domingo
```

C.4.3. Tipos enumerados

```
type
  Naipes = (Oros, Espadas, Bastos, Copas);
```

C.4.4. Tipos estructurados

Array

```
array[1.100] of Real
array[Boolean] of array [1..10] of array [Longitud] of Real;
packed array [1..10, 1..15] of Boolean
```

Registro

```
type Reg = record
  Nombre  : String [40];
  Edad    : Integer;
  Salario : Real;
      end;
```

Objeto

```
type Punto = object
               x,y : Integer;
                     end;
```

Conjunto

```
Letras   :=['A','B','C','D','E']
NoLetras := []
```

Puntero

```
type PunteroReal = Real
```

Pchar

```
type Pchar = Char;
```

C.4.5. Tipos procedimiento

```
type
  Proc     = procedure;
  InterX   = procedure (var X,Y : Integer);
  procCX   = procedure (S : string);
  FuncUna  = function (X : Real) : Real;
```

C.5. VARIABLES Y CONSTANTES

C.5.1. Constantes predefinidas

cto 255

```
true        verdadero
false       falso
maxint      entero mayor disponible en maquina
```

C.5.2. Declaración de variables

```
var
  x,y,z   : Real;
  i,j,k   : Integer;
  Digito : 0..9;
```

C.5.3. Variables absolutas

```
var
  Cad : string[40]
  LongCad : Byte absolute Cad;
```

C.5.4. Variables predefinidas

Variable	*Tipo*
Input	Archivo Text
Output	Archivo Text
ExitCode	Integer
FileMode	Byte
InOutRes	Integer
RandSeed	LongInt
StrackLimit	Word

C.5.5. Constantes con tipos

Tipo simple:

```
const
  Maximo : Integer = 425;
  Factor : Real = -12.5;
```

```
const
  Min : Integer = 0;
  Max : Integer = 100;
type  declaración no válida
  Lista = array [Min..Max] of Integer;
```

la declaración `Lista` *no es válida.* `Min` *y* `Max` *son constantes de tipos*

Tipo cadena:

```
const
  Cabecera     : string[10] = 'Ciudades';
  Nueva Linea : string[2] = #13#10;
  Respuesta    : string[5] = 'Si';
```

Tipo estructurado:

```
const
  Digitos : array[0..0] of Char=
            ('0','1','2','3','4','5','6','7','8','9');
const
  Digitos = array[0..0] of Char = '0123456789';
type
  Matriz = array [0..1, 0..1, 0..1] of Integer;
const
  Lista : Matriz = (((0,1), (2,3)), ((4,6), (7,9)));
```

Tipo registro:

```
type
  Punto = record
            x,y : Real;
          end;
  Vector = array [0..1] of Punto;
const
  Origen : Punto  = (X:0.0; Y:00);
  Linea  : Vector = ((X:-4,5; Y:1.5), (X:6.4; Y:5.0));
```

Tipo objeto:

```
type
  Punto = object
            x,y : Integer;
          end;
const
  Origen : Punto = (x:0; y:0);
```

Tipo conjunto:

```
type
  Digitos = set of 0..9;
  Letras  = set of 'A'..'Z';
const
  Pares   : Digitos = [0,2,4,6,8];
  Vocales : Letras = ['A','E','I','O','U'];
```

Tipo puntero:

```
type
  Direccion = (Izda, Dcha, Arriba, Abajo);
  pNodo = ^NodoT;
  NodoT = record
            ...
          end;
const
  S1 : string[5] = 'ABAJO';
  N1 : NodoT = (Siguiente : nil; Symbolo : @S1; Valor : Abajo);
```

Tipo procedimiento:

```
type
  {SP*) = procedure (CodigoError: Integer);
procedure ErrorPorOmision (CodigoError : Integer);
begin
  WriteLn ('Error', CodigoError, '.');
end;
const
  ManipuladorErrores : ProcError = ErrorPorOmision;
```

C.6. OPERADORES

C.6.1. Aritméticos

Operador	Sintaxis	Significado
+	+expresion	Positivo (unitario)
+	expresion1 + expresion2	Suma (binaria)
-	-expresion	Negativo (unitario)
-	expresion1 - expresion2	Resta (binario)
*	expresion1 * expresion2	Multiplicación
/	expresion1 / expresion2	División real
DIV	expresion1 **DIV** expresion2	División entera
MOD	expresion1 **MOD** expresion2	Resto (módulo)

C.6.2. Lógicos

Operador	Sintaxis	Significado
NOT	NOT expresion	Complemento
AND	expresion1 **AND** expresion2	AND
OR	expresion1 **OR** expresion2	OR inclusiva
XOR	expresion1 **XOR** expresion2	OR exclusiva

C.6.3. Relacionales

Operador	Sintaxis	Devuelve verdadero si:
=	expresion1 = expresion2	Expresiones son iguales
< >	expresion1 < > expresion2	Expresiones no son iguales
<	expresion1 < expresion2	expresion1 es menor que expresion2
< =	expresion1 < = expresion2	expresion1 es menor que o igual a expresion2
>	expresion1 > expresion2	expresion1 es mayor que expresion2
> =	expresion1 > = expresion2	expresion1 es mayor o igual que expresion2

C.6.4. De conjuntos

Operador	Sintaxis	Devuelve verdadero si:
=	Conjunto1 = Conjunto2	Conjunto1 y Conjunto2 son idénticos. Cada elemento de Conjunto1 está contenido en Conjunto2, y cada elemento en Conjunto1 está contenido en Conjunto1
< >	Conjunto1 < > Conjunto2	Uno de los conjuntos contiene al menos un elemento que no está en el otro conjunto
< =	Conjunto1 < = Conjunto2	Cada elemento del Conjunto1 está también en Conjunto2
<	Conjunto1 < Conjunto2	Cada elemento de Conjunto1 está también en Conjunto2. Además, Conjunto2 contiene al menos otro elemento que no se encuentra en Conjunto1
> =	Conjunto1 > = Conjunto2	Cada elemento de Conjunto2 está también en Conjunto1.
>	Conjunto1 > Conjunto2	Cada elemento de Conjunto2 está también en Conjunto1. Además, Conjunto1 contiene al menos otro elemento que no se encuentra en Conjunto2
IN	elemento In Conjunto2	El elemento elemento se encuentra en Conjunto1

C.6.5. Dirección

@ Dirección de variable, procedimiento o función.

C.6.6. Concatenación

+ Concatena dos cadenas.

C.6.7. Prioridad de operadores

Prioridad	Operadores
1 (*alta*)	@NOT*unitario* + −
2	*, /, DIV, MOD, AND, SHL, SHR
3	*binario* +, −, OR, XOR
4 (*baja*)	=, <>, <, >, <=, >=, IN

C.7. SENTENCIAS

C.7.1. Asignación

<identificador> := *<expresion>*

```
rango  := alto-bajo;
cuenta := cuenta + 1;
```

C.7.2. Compuesta

begin *<sentencias>* **end**

```
begin
  z := 5
  GetReal ('Valor', ValorReal);
  WriteLn (ValorReal);
end
```

C.7.3. Selectiva (if)

if *<condicion>* **then**
  <sentencia>
[**else**
  <sentencia>];

```
if Cad1 <> Cad2 then
begin
  Cad1 := Cad1 + Cad2;
  WriteLn ('nueva cadena', Cad1);
end;
```

C.7.4. Selectiva (if-*anidada*)

if (*condicion*) **then**
  sentencia
else if (*condicion*) **then**
  sentencia
else if (*condicion*) **then**
  sentencia
else
  sentencia

```
if x > y then
  if x > z then
    Write (x)
  else
    Write (z)
else
  if y > z then
    Write (y)
  else
    Write (z);
WriteLn ( es el Mayor');
```

C.7.5. Selectiva múltiple (*case*)

```
case <selector> of
  <lista de constantes> :
    <sentencias 1>;
  <lista de constantes>:
    ...
  [else
    <sentencia por defecto>]
end;
```

```
case N of
    1,2 : WriteLn ('Primero');
    3,4,5 : WriteLn ('Bronce');
    7 : WriteLn ('Fin');
    else
      WriteLn ('Final')
end;
```

C.7.6. Sentencia repetitiva *for*

1. **for** <*índice*> := <*valor inicial*> **to** <*valor inicial*> **do** <*sentencia*>

```
for k := 1 to 10 do
  WriteLn ('cuerpo de bucle');
```

```
for j := 1 to 10 do
begin
  j := j + z;
  WriteLn ('pasos ;', pasos : 2);
  paso := paso + 1
end;
WriteLn ('fuera del buclé');
```

2. **for** <*índice*> := <*valor final*> **downto** <*valor inicial*> **do** <*sentencia*>

```
for i := 10 downto 10 do
begin
  Resultado := i * Resultado;
  WriteLn('Bucle', i, Resultado);
end;
```

C.7.7. Sentencia repetitiva *while*

```
while <condición> do
    <sentencia>;
    <sentencia> = sentencia simple
                  sentencia compuesta
```

```
paso :=1;
while paso <= 10 do
begin
  WriteLn ('Pasos:', paso:2);
  paso := paso + 1
end;
WriteLn('fuera del bucle');
while ch <> '' do Read(ch);
```

C.7.8. Sentencia repetitiva *repeat*

```
repeat
  <sentencia>
until <condición>
```

```
paso := 1;
repeat
  WriteLn('Pasos:',paso:2);
  paso := paso +1;
until paso > 10;
WriteLn('Fuera del bucle');
```

C.8. ÁMBITO Y LOCALIDAD

El *ámbito* de un identificador es el conjunto de módulos (programas o subprogramas) en el que está legalmente declarado el identificador.

```
program Principal

  var                                          variables globales
    w,x,y,z:...                                parámetros locales w, a, x
  procedure Uno (var w:...; a:...);
    var
     x :...;
  begin
    ...
  end;

  procedure Dos (c:...; var x...);             parámetros locales c, y, x, z, w
    var
     z,w:...;
  begin
   ...
  end;

begin {Principal}
  ...
  Uno (y,z);
  ...
  Dos (w,y,x);
  ...
end.
```

> Para eliminar posibles efectos laterales, evite utilizar variables globales en subprogramas o identificadores idénticos para cantidades diferentes.

C.9. PROCEDIMIENTOS Y FUNCIONES ESTÁNDAR

Turbo Pascal contiene procedimientos y funciones estándar (incorporadas o predefinidas) y variables predeclaradas en la unidad *System*. Todos ellos están declarados en la unidad *System*, en consecuencia, no necesita ninguna sentencia **uses** cuando desee utilizar alguno/a de ellos/as.

C.9.1. Funciones aritméticas

Nombre	Sintaxis	Descripción
Abs	`Abs(x);`	Devuelve el valor (positivo) absoluto de su argumento
ArcTan	`ArcTan(x:Real):Real;`	Arco tangente expresado en radianes
Cos	`Cos(x:Real):Real;`	Coseno del argumento en radianes
Exp	`Exp(x:Real):Real;`	Potencia exponencial del argumento (e^x)

Nombre	Sintaxis	Descripción
Frac	`Frac(x:Real):Real;`	Parte decimal de un número real
Int	`Int(x:Real):Real;`	Parte entera del argumento
Ln	`Ln(x:Real):Real;`	Logaritmo neperiano (base *e*) del argumento
Pi	`Pi:Real;`	Valor de *Pi* (3,1415926535897932385)
Sin	`Sin(x:Real):Real;`	Seno del argumento
Sqr	`Sqr(x);`	Cuadrado del argumento
Sqrt	`Sqrt(x:Real):Real;`	Raíz cuadrada del argumento

Ejemplo:

```
Y   := abs(x)* 2                z := Int(345.678);
z   := ArcTan(1.75);            t := Ln(1.25);
Tan := Sin(X)/cos(x);           z := ArcTan(Pi);
Pot := Exp(3);                  z := Sin(Pi);
R   := Frac)-245.123);          f := Sqr(3.45);
                                f := sqrt(1.2345);
```

C.9.2. Funciones de transferencia

Nombre	Sintaxis	Descripción
Chr	`Chr (x:Byte):Char;`	Devuelve el carácter correspondiente al código ASCII
Ord	`Ord(x):LongInt;`	Número ordinal de un tipo ordinal
Round	`Round (x:Real):LongInt;`	Redondea un valor real a entero largo
Trunc	`Trunc (x:Real):LongInt;`	Trunca un valor de tipo real a entero

Ejemplos:

```
Write(Chr(i));          Round(5.449)   devuelve 5
Ord('A')                Trunc(-3.14)   devuelve -3
                        Trunc(6.5)     devuelve 6
```

C.9.3. Procedimientos de flujo de control

Nombre	Sintaxis	Descripción
Break	`Break;`	Termina una sentencia **for, while** o **repeat**
Continue	`Continue;`	Continúa con la siguiente iteración de una sentencia **for, while** o **repeat**
Exit	`Exit;`	Termina inmediatamente el bloque actual (procedimiento, función o programa)
Halt	`Halt[CodigoSalida: Word)];`	Detiene la ejecución del programa y retorna al sistema operativo
RunError	`RunError[(CodigoError: Byte)];`	Detiene la ejecución del programa y genera un error en tiempo de ejecución

Ejemplos:

```
if t = 's' then Exit;
```

```
if p = nil then
  RunError(294);
```

C.9.4. Procedimientos o funciones ordinales

Nombre	Sintaxis	Descripción
Dec	Dec(var x[;n:LongInt]);	Decrementa una variable
Inc	Inc(var x[;n:LongInt]);	Incrementa una variable
High	High(x);	Devuelve el valor más alto en el rango del argumento que debe ser un tipo array o un tipo cadena
Low	Low(x)	Devuelve el valor más bajo en el rango del argumento (tipo array o cadena)
Odd	Odd(x:LongInt):Boolean;	Devuelve true si el argumento es impar
Pred	Pred(x);	Predecesor del argumento (*x* de tipo ordinal)
Succ	Succ(x);	Sucesor del argumento (*x* de tipo ordinal)

Ejemplos:

```
Dec(z)
Inc(z);

for i := 0 to High (x) do
  s := s + x[i];
```

```
for Dia := Low(ADia) to High (ADia) do
begin
...
end;

Pred('z')    devuelve 'Y'
Succ(1946)   devuelve 1947
```

C.9.5. Tratamiento de cadenas

Procedimientos

Nombre	Sintaxis	Descripción
Delete	Delete(var s:string; Pos,Len:Integer);	Borra una subcadena a partir de una posición en una cadena
Insert	Insert(var s:string; var D: string; Pos:Integer);	Inserta una subcadena en una posición de una cadena
Str	Str(I:Integer; var s: string); Str(R:Real; var s:string);	Convierte un valor numérico a cadena
Val	Val(s:string; var R:Real,P: Integer); Val(s:string; var I,P:Integer);	Convierte una cadena a su valor numérico

Funciones

Nombre	Sintaxis	Descripción
Concat	`Concat(s1,s2,...,sn: string):string;`	Concatena (une) cadenas
Copy	`Copy(s:string;Pos,Long: Integer):string;`	Copia una cadena dada
Length	`Length(s:string):Integer;`	Longitud de una cadena
Pos	`Pos(Patron,Fuente:string): Integer;`	Posición de la primera ocurrencia de una subcadena

Ejemplos:

```
Cad1 := 'computadora';
Delete(Cad1,4,3);
Insert('put'Cad1,4);
s := Concat ('xyz','LUIS');
```

```
Str(Maxint,Cad);
WriteLn(Cad);
Val('32425',r,e);
s := Copy(s,2,3);
t := Length(Cad);
```

C.9.6. Punteros y direcciones

Nombre	Sintaxis	Descripción
Addr	`Addr(x):Pointer;`	Devuelve la dirección de un objeto especificado
Assigned	`Assigned(var P):Boolean;`	Comprueba si una variable procedimiento o puntero es **nil**
CSeg	`CSeg:Word;`	Valor actual del registro CS
DSeg	`DSeg:Word;`	Valor actual del registro DS
Ofs	`Ofs(x):Word;`	Desplazamiento de un objeto especificado
Ptr	`Ptr(Seg,Ofs:Word):Pointer;`	Convierte una base segmento y una dirección de desplazamiento a un valor tipo puntero
Seg	`Seg(x):Word;`	Dirección del segmento de una variable o rutina
Sptr	`Sptr:Word;`	Valor actual del registro SP
Sseg	`SSeg:Word;`	Valor actual del registro SS

Ejemplos:

```
p := Addr(p);
if Assigned(p) then WriteLn('ok);
p := Ptr(Cseg,I);
p := Ptr(Dseg,I);
WriteLn('offset='Ofs(p2));
bajo := Ptr(0000,$046c);
WriteLn('seg(I)=',seg(I));
```

C.9.7. Asignación dinámica

Procedimientos

Nombre	Sintaxis	Descripción
Dispose	`Dispose(var p:pointer[,Destructor]);`	
FreeMem	`FreeMem(var p:pointer; Tamaño:Word);`	
GetMem	`GetMem(var p:pointer; Tamaño:Word);`	
New	`New(var p:pointer[,Init:Constructor]);`	

Funciones

Nombre	Sintaxis	Descripción
MaxAvail	`MaxAvail:LongInt;`	Tamaño del bloque disponible mayor del montículo (*heap*)
MemAvail	`MemAvail:LongInt;`	Cantidad de memoria libre en el montículo (*heap*)
BlockRead	`BlockRead(var f:file; var Buf:tipo Cuenta: Word[;var Resultado:Word]);`	Lee uno o más registros en un archivo sin tipos
BlocWrite	`BlocWrite(var f:file; var Buf:tipo Cuenta: Word[;var Resultado:Word]);`	Escribe uno o más registros en un archivo sin tipos
Close	`Close (var F);`	Cierra un archivo abierto
Erase	`Erase (var F);`	Borra un archivo externo
FileSize	`FileSize(var F):LongInt;`	Devuelve el tamaño actual de un archivo (no de texto)
Flush	`Flush(var F:Text);`	Limpia el *buffer* de un archivo de texto de salida
Read	`Read(F,V1[,V2,...Vn]);`	Lee uno o más valores de un archivo
ReadLn	`ReadLn(var F:Text;]V1[,V2,... Vn]);`	Igual que Read y después salta al principio de la línea siguiente
Rename	`Rename(var F:NuevoNombre);`	Renombra un archivo externo
Reset	`Reser (var F[:file;TamaReg: Word]);`	Abre un archivo existente
ReWrite	`ReWrite(var F[:file;TamaReg: Word]);`	Crea y abre un archivo nuevo
Seek	`Seek(var F;N:LongInt);`	Mueve la posición actual de un archivo a un componente especificado
SetTextBuf	`SetTextBuf(var F:Text; varBuf:tipo [;Tamaño:Word]);`	Asigna una memoria intermedia de E/S a un archivo de texto
Truncate	`Truncate (var F);`	Trunca un archivo con tipos o sin tipo

Funciones

Nombre	Sintaxis	Descripción
Write	Write ([var F:Text;]p1[,p2, ..., pn1]);	Escribe en un archivo de texto
	.Write (f,v1,[v2,...,vn]);	Archivos con tipos
WriteLn	WriteLn ([var F:Text;]p1[p2, ...,pn]);	Igual que Write y luego escribe un fin de línea
Eof	Eof (var F):Boolean;	Devuelve el estado fin de archivo
Eoln	Eoln[(var F:Text)]Boolean;	Devuelve el estado fin de línea de un archivo de texto
FilePos	FilePos(var F):LongInt;	Posición actual de un archivo con o sin tipos
IOResult	IOResultInteger;	Estado de la última operación de E/S realizada
SeekEof	SeekEof[(var F:Text)]: Boolean;	Estado fin de archivo de un archivo
SeekEoln	SeekEoln[(var F:Text)];	Estado fin de línea de un archivo

Ejemplos:

```
var Tf:Text;
    F:File;

Assign(Tf,'TEX.TXT');

Rewrite(Tf);
Close(Tf);
Append(Tf);
BlockRead(F,A,10,Resultado);
BlockWrite(F,A,I,Resultado);
Erase(F)

Seek(F,FileSize(F));
Flush(F);

while not Eof(Tf) do
begin
  if Eoln(Tf)
     then writeLn;
if Length (s) > 0
  then Rename(F,S);
```

```
Reset(F);

Seek(Tf,12);

Read(Tf,ch);

SetTextBuf(F,Buffer,512);

Existe := (IOResult = 0);
while not SeekEof(Tf) do
begin
  ReadLn(Tf,S);

  Inc(C2);
end

while not SeekEoln(Tf) do
begin
  ReadLn(Tf,Ch);
  write(ch);
end;
```

C.10. FUNCIONES DEFINIDAS POR EL USUARIO

C.10.1. Declaración

```
function <identificador> (<parametros formales>): <tipo>;
      <declaraciones locales>
begin
      <cuerpo de la funcion>
end;
```

C.11. ARCHIVOS

C.11.1. Preparación de un archivo

- Enlace entre un archivo interno y un archivo externo.

```
Assing(<identificador de archivo>, <nombre archivo externo>);
```

C.11.2. Creación de un archivo de registros

- Preparar el sistema para enviar datos de la estructura interna del archivo externo creando y abriendo un nuevo archivo para recibir salida de la computadora.

```
Rewrite(<identificador de archivo>);
```

C.11.3. Almacenar datos en un archivo

```
Write(<identificador de archivo>, <identificador de salida>);
```

C.11.4. Cerrar un archivo

```
Close(<identificador de archivo>);
```

C.11.5. Lectura de un archivo de registros

Procedimiento Reset (Inicializar un archivo)

```
Reset(<identificador de archivo>);
```

Lectura de un archivo de registros

```
Read(<identificador de archivo>, <identificador de entrada>);
```

Final de archivo

```
Eof(<identificador de archivo>;
```

Borrar archivos

```
Erase(<identificador de archivo>);
```

C.11.6. Cambiar registros en un archivo de registros

Función de posición FilePos

```
FilePos(<identificador de archivo>);
```

Posicionamiento en registro

```
Seek(<identificador de archivo>,<numero de registro>);
```

C.12. DIRECTIVAS DE COMPILACIÓN

C.12.1. Directivas de conmutación

Directiva	Significado	Valor por defecto	Tipo
A	Alinear datos	{$A+}	Global
B	Evaluación lógica (*booleana*)	{$B-}	Local
D	Información de depuración	{$D+}	Global
E	Emulación	{$E+}	Global
F	Force far calls	{$F-}	Local
G	Generación de código 80286	{$G-}	Global
I	Verificación de E/S	{$I+}	Local
L	Generación de información de símbolos locales	{$L+}	Global
N	Proceso numérico	{$N-}	Global
P	Parámetros de cadenas abiertas	{$P-}	Local
O	Generación de código de solapamiento	{$O-}	Local
Q	Generación de código de verificación de desbordamientos	{$Q-}	Local
R	Verificación de rango	{$R-}	Local
S	Verificación de desbordamiento de pila	{$S+}	Local
T	Tipos de valores puntero, generados, por el operador @	{$T-}	Global
V	Verificación de tipos en cadenas pasados como parámetros variables	{$V+}	Local
X	Activa o desactiva sintaxis extendida de Turbo Pascal	{$X+}	Global
Y	Información de referencias a símbolos	{$Y+}	Global

C.12.2. Directivas de parámetro

Directiva	Significado	Sintaxis
I	Archivo de inclusión	{$I *nombrearchivo*}
L	Enlaza el archivo reforzado con el programa o unidad que se está compilando	{$L *nombrearchivo*}
M	Requisitos de asignación de memoria; *long*, tamaño de la pila; *min* y *max*, tamaños mínimo y máximo del montículo	{$M *long,min,max*}
O	Nombre de unidad de recubrimiento (solapamiento)	{$O *nombreunidad*}

C.12.3. Directivas de compilación condicional

Directiva	Significado
{DEFINE *nombre*}	Define un símbolo condicional con el nombre dado
{$ELSE}	Conmuta entre compilación
{$ENDIF}	Termina la compilación condicional iniciada por la última directiva {$IFxxx}

Directiva	Significado
{$IFDEF *nombre*}	Compila el texto fuente que le sigue si está definido *nombre*
{$IFNDEF}	Compila el texto fuente que le sigue si *nombre* no está definido
{$IFOPT *conmutador*}	Verdadero o falso de acuerdo a que la directiva *conmutador* esté activada o desactivada
{$UNDEF *nombre*}	Indefine un símbolo condicional definido anteriormente

```
{$IFxxx}          {$IFxxx}          {$IFDEFnombre}
  ...               ...               <sentencias-1>
{$ENDIF}          {$ELSE}           {$ELSE}
                    ...             <sentencias-?>
                  {$ENDIF}          {$ENDIF}
```

C.12.4. Símbolos condicionales

Símbolo	Significado
VER70	Siempre definido, indicando versión 7.0
MSDOS	Siempre definido, indicando que el sistema operativo es MS-DOS o PC-DOS
CPU86	Siempre definido, indicando que la CPU pertenece a la familia de procesadores 80 × 86
CPU87	Definido si un coprocesador matemático está presente en tiempo de compilación

C.13. CONTROL DE DISPOSITIVOS

Las unidades `Crt` y `Printer` son utilizadas con mucha frecuencia por los programadores. `Printer` envía salida a su impresora. La unidad `Crt` implementa una amplia y potente gama de rutinas que le proporcionan un control completo de características de su PC tales como: control del modo de pantalla, códigos de teclado extendido, colores, ventanas y sonido. Por su importancia práctica para el programador, seleccionamos las características más notables utilizadas en programación profesional.

Recuerde que para utilizar la unidad `Crt` o `Printer`, en su programa, debe incluir la cláusula **uses** como cualquier otra unidad:

```
uses Crt, Printer;
```

C.13.1. Caracteres de control

Carácter	Nombre	Descripción
#7	BELL	Emite un sonido (pitido) del altavoz interno
#8	BS	Retrocede el cursor una columna
#10	LF	Avanza el cursor una línea abajo
#13	CR	Retorna el cursor al extremo izquierdo de la línea siguiente

C.13.2. Teclas de edición de entrada de líneas

Tecla de edición	Descripción
RETROCESO (←)	Borra el último carácter introducido
ESC	Borra la línea de entrada completa
ENTER (*Intro*)	Termina línea de entrada y almacena en almacenamiento temporal Genera marca fin de línea
CTRL+S	Igual que Retroceso (*Backspace*)
CTRL+D	Llama un carácter de la última línea introducida
CTRL+A	Igual que *Esc*
CTRL+F	Llama a la última línea introducida
CTRL+Z	Termina la última línea introducida y genera una marca fin de archivo

Funciones

Nombre	Sintaxis	Descripción
KeyPressed	KeyPressed	Devuelve *true* si se ha pulsado una tecla del teclado: *false* en caso contrario
ReadKey	ReadKey	Lee un carácter del teclado
WhereX	WhereX	Devuelve coordenada *x* de la posición actual del cursor
WhereY	WhereY	Devuelve coordenada *y* de la posición actual del cursor

Ejemplos:

```
repeat
  Write('Zz');
until KeyPressed;
```

```
Car := Readkey;

gotoxy(1, whereY-1);
```

Nombre	Sintaxis	Descripción
InsLine	InsLine	Inserta una línea vacía en posición del cursor
LowVideo	LowVideo	Selecciona caracteres de baja intensidad
NormVideo	NormVideo	Selecciona caracteres normales
NoSound	NoSound	Desactiva altavoz interno de la computadora
Sound	Sound(HZ:Word)	Arranca altavoz interno
TextBackground	TextBackground(Color:Byte)	Selecciona color de fondo
TextColor	TextColor(Color:Byte)	Selecciona color de carácter (primer plano)
TextMode	TextMode(Modo:Word)	Selecciona un modo de texto especificado
Window	Window(x1,y1,x2,y2:Byte)	Define una ventana de texto en pantalla

Procedimientos

Nombre	Sintaxis	Descripción
AssignCrt	AssignCrt(**var** F:Text)	Asocia un archivo de texto con la ventana Crt
ClrEol	ClrEol	Borra todos los caracteres desde la posición del cursor hasta el final de la línea
ClrScr	ClrScr	Borra la pantalla y cursor a posición inicial
Delay	Delay(ms:Word)	Retarda un número especificado de segundos
DelLine	DelLine	Borra la línea que contiene el cursor y mueve todas las líneas, inferiores a él, una línea hacia arriba
GotoXY	GotoXY(x,y:Byte)	Posiciona el cursor en coordenadas (*x*, *y*)
HighVideo	HighVideo	Selecciona caracteres de alta intensidad

```
Gotoxy(1,10);
Write('Mackoy');
ClrEol;
ReadLn(Car);
Sound(440);
Delay(500);
NoSound;
```

```
Window(1,10,60,20);
DelLine;
Write('Mortimer');
InsLine;
repeat
  x := Succ(Random(80));
  y := Succ(Random(80));
  Window(x,y,x + Random(10);
  y + Random(10));
  TexBackground(Random(15));
  ClrScr;
until KeyPressed;
```

C.14. PASCAL ESTÁNDAR (ANSI PASCAL) *versus* TURBO PASCAL 7.0

Turbo Pascal sigue las normas de Pascal estándar, con una serie de excepciones y mejoras que lo configuran como un entorno de programación especial. La versión 5.5 de Turbo Pascal ha añadido las características propias de la *programación orientada a objetos.*

C.14.1. Mejoras de Turbo Pascal (no existen en Pascal estándar)

Las siguientes características son propias de Turbo Pascal 5.5 a 7.0 y no existen en Pascal ANSI (estándar):

- Nuevas palabras reservadas en Turbo Pascal:

```
absolute        interface       string
external        interrupt       unit
implementation  shl             uses
inline          shr             xor
```

- Un identificador puede contener caracteres subrayados (–):

```
Paga_mes        Nombre_Apellidos      Interes_Anual
```

- Constantes:

  Enteras (se pueden escribir en notación hexadecimal, con prefijo `$`).
  De *cadena;* son compatibles con tipos *string* y pueden contener caracteres de control y otros caracteres no imprimibles.
- Nuevos tipos enteros:

  shortint longint byte word
- Nuevos tipos reales:

  single double extended comp
- Turbo Pascal incorpora tipos *string*.
- Las declaraciones de constantes, tipos, variables, procedimientos y funciones pueden ocurrir cualquier número de veces en cualquier orden en un bloque.
- Turbo Pascal incorpora tipos procedimiento.
- Las variables se pueden declarar en direcciones de memoria absoluta utilizando una cláusula **absolute.**
- Las variables tipo *string* se pueden indexar como *arrays* para acceder individualmente a caracteres de una cadena.
- Turbo Pascal incorpora tres nuevos operadores lógicos: **xor, shl** y **shr**.
- Los operadores **not, and, or** y **xor** se pueden utilizar con operandos tipo entero para ejecutar operaciones lógicas *bitwise.*
- El operador + se puede utilizar para comparar cadenas.
- Los operadores relacionales se pueden utilizar para comparar cadenas.
- Turbo Pascal incorpora el operador @, que sirve para obtener la dirección de una variable, un procedimiento o una función.
- Los valores tipo *string* pueden ser entrada y salida con los procedimientos estándar *Read, ReadLn, Write* y *WriteLn.*
- La sentencia **case** permite rangos de constantes en la lista de etiquetas **case**, y posee una parte opcional **else**.
- Turbo Pascal incorpora unidades para programación modular y compilación separada.
- Turbo Pascal incorpora los siguientes nuevos procedimientos y funciones específicas de tratamiento de archivos:

```
Append      Clase      Fush       RmDir
BlockRead   Erase      GetDir     Seek
BlockWhite  FilePos    MkDir      SeekEos
ChDir       FileSize   Rename     SeekEoln
```

- Turbo Pascal incorpora las siguientes nuevas funciones y procedimientos:

```
Addr        GetMen     Move       Sptr
Concat      Halt       Ofs        Seg
Copy        Hi         ParamCount SizeOf
CSeg        Inc        ParamStr   SSeg
DSeg        Insert     Pi         Str
Dec         Int        Pos        Swap
Delete      Lenght     Ptr        UpCase
Exit        Lo         Random     Val
```

```
FillChar     Mark         Randomize
Frac         MaxAvail     Release
FreeMem      MemAvail     RunError
```

- Arrays abiertos en Turbo Pascal 7.0 y nuevas funciones `High` y `Low`.
- Cadenas terminadas en nulo, similares a las cadenas en C. *Sólo en versión Turbo Pascal 7.0.*
- Nueva sentencia de salto incondicional: `break` y `continue`. *Sólo en versión Turbo Pascal 7.0.*

C.14.2. Características de Pascal estándar no existentes en Turbo Pascal

- Identificadores:

  Pascal estándar: un identificador puede ser de cualquier longitud y todos los caracteres son significativos.
  Turbo Pascal: un identificador puede ser de cualquier longitud, pero sólo son significativos los 63 primeros caracteres.

- En Pascal estándar, un comentario puede comenzar con { y finalizar *) o viceversa. En cambio, en Turbo Pascal los comentarios deben comenzar y terminar con el mismo conjunto de símbolos.
- En Pascal estándar, una variable *file* (archivo) tiene una variable *buffer* (memoria intermedia) asociada, que se identifica por el símbolo ^ después de la variable *file*. En Turbo Pascal, una variable archivo no tiene una variable *buffer* asociada, y si escribe un ^ después de la variable, cometerá un error.
- Pascal estándar define los procedimientos `Get` y `Put`, que se utilizan para leer y escribir en archivos. No están definidos en Turbo Pascal.
- En Pascal estándar, si el valor del selector en una sentencia `case` no es igual a cualquiera de las constantes `case`, se produce un error. En Turbo Pascal no se produce error; en su lugar se ignora la sentencia `case`, excepto que tenga la cláusula `else`.
- En Pascal, una lectura (`Read`) de un archivo de texto con una variable tipo `char` asigna un blanco a la variable si `Eoln` era *true* antes de `Read`. En Turbo Pascal se asigna a la variable un carácter retorno de carro (ASCII 13).
- En Pascal, una lectura (`Read`) de un archivo de texto con variables tipos entero o real cesa tan pronto como el siguiente carácter del archivo no es parte de un número o entero con signo. En Turbo Pascal la lectura cesa cuando el siguiente carácter del archivo es un blanco o un carácter de control.
- En Pascal está definido el procedimiento estándar `Page`, que produce un salto de página (pasa a una nueva página). Este procedimiento no está definido en Turbo Pascal; sin embargo, se puede realizar con la sentencia `Write (F, Chr(12))`.

  `Page(F)` *equivale a* `Write(E, Chr(12))`.

REFERENCIAS BIBLIOGRÁFICAS

BORLAND: *Language Guide, Programmer's Reference, User's Guide 6.0 y 7.0. Turbo Pascal Reference Guide 5.0 y 5.5, Borland Pascal Reference Guide.*

EJERCICIOS

C.1. Diseñar una unidad con los procedimientos y funciones específicos de las cadenas de caracteres implementándolos mediante arrays de caracteres.

C.2. Diseñar una unidad con las cuatro operaciones aritméticas básicas, para poder sumar, restar, multiplicar y dividir números reales de longitud arbitraria (por ejemplo, 20 dígitos).

C.3. Diseñar una unidad que contenga un procedimiento o función capaz de medir intervalos de tiempo utilizando las funciones SetTime y GetTime de Turbo Pascal.

C.4. Diseñar una unidad para sumar, restar y multiplicar, calcular inversas, transpuestas, etc., de matrices numéricas.

C.5. Se implementan conjuntos de números mediante arrays numéricos de una dimensión. Construir una unidad con los procedimientos y funciones necesarios para emular las operaciones entre conjuntos: unión, intersección, diferencia, inclusión, igualdad, pertenencia, etc.

C.6. Las estructuras lista, pila y cola disponen de una serie de primitivas que se utilizan con gran frecuencia. Diseñar las unidades necesarias para emular estas operaciones cuando las estructuras mencionadas se realizan sobre arrays.

C.7. Diseñar una unidad para emular árboles binarios de búsqueda mediante arrays de una dimensión, definiendo las operaciones elementales de búsqueda, eliminación, recorrido, etc., de nodos del árbol.

C.8. Construir una unidad que permita calcular perímetros, áreas, volúmenes, etc. de las figuras más frecuentes en dos y en tres dimensines (polígonos, circunferencia y círculo, esfera, cono, cilindro, etc.).

C.9. Construir una unidad que sume, reste, multiplique y evalúe derivadas de polinomios de una variable independiente y exponente natural.

APÉNDICE D

GUÍA DE REFERENCIA LENGUAJE C ANSI

D.1. ELEMENTOS BÁSICOS DE UN PROGRAMA

El lenguaje C fue desarrollado en *Bell Laboratories* para su uso en investigación, y se caracteriza por un gran número de propiedades que lo hacen ideal para usos científicos y de gestión.

Una de las grandes ventajas del lenguaje C es ser *estructurado*. Se pueden escribir bucles que tienen condiciones de entrada y salida claras y se pueden escribir funciones cuyos argumentos se verifican siempre para su completa exactitud.

Su excelente biblioteca estándar de funciones convierten a C en uno de los mejores lenguajes de programación que los profesionales informáticos pueden utilizar.

D.2. ESTRUCTURA DE UN PROGRAMA C

Un programa típico en C se organiza en uno o más *archivos fuentes* o *módulos*. Cada archivo tiene una estructura similar con comentarios, directivas de preprocesador, declaraciones de variables y funciones y sus definiciones. Normalmente se sitúa cada grupo de funciones y variables relacionadas en un único archivo fuente. Dentro de cada archivo fuente, los componentes de un programa suelen colocarse en un determinado modo estándar. La Figura D.1 muestra la organización típica de un archivo fuente en C.

Los componentes típicos de un archivo fuente de programa son:

1. El archivo comienza con algunos comentarios que describen el propósito del módulo e información adicional, tal como el nombre del autor y fecha, nombre del archivo. Los comentarios comienzan con /* y terminan con */.
2. Órdenes al preprocesador, conocidas como *directivas del preprocesador.* Normalmente incluyen archivos de cabecera y definición de constantes.
3. Declaraciones de variables y funciones son visibles en todo el archivo. En otras palabras, los nombres de estas variables y funciones se pueden utilizar en cualquiera de las funciones de este archivo. Si se desea limitar la visibilidad de las variables y funciones *sólo* a ese módulo,

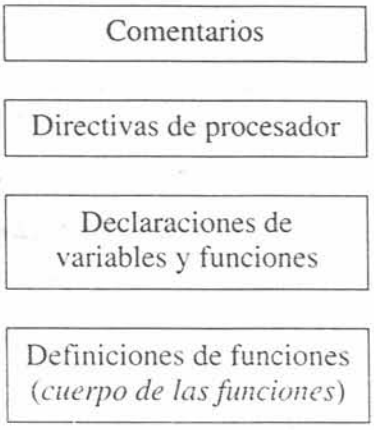


Figura D.1. Organización de un programa C.

ha de poner delante de sus nombres el prefijo `static`; por el contrario, la palabra reservada `extern` indica que los elementos se declaran y definen en otro archivo.

4. El resto del archivo incluye definiciones de las funciones (su cuerpo). Dentro de un cuerpo de una función se pueden definir variables que son locales a la función y que sólo existen en el código de la función que se está ejecutando.

D.3. EL PRIMER PROGRAMA C ANSI

```
#include <studio.h>

main ()
{
   printf ("¿Hola mundo!");
   return 0;
}
```

D.4. PALABRAS RESERVADAS ANSI C

```
auto        double      int         struct
break       else        long        switch
case        enum        register    typedef
char        extern      return      union
const       float       short       unsigned
continue    for         signed      void
default     goto        sizeof      volatile
do          if          static      while
```

Las palabras reservadas `const`, `enum`, `void` y `volatile` son nuevas en C ANSI.

D.5. DIRECTIVAS DEL PREPROCESADOR

El *preprocesador* es la parte del compilador que realiza la primera etapa de traducción o compilación de un archivo C ANSI en instrucciones de máquina. El preprocesador procesa el archivo fuente y

actúa sobre las órdenes, denominadas *directivas de preprocesador*, incluidas en el programa. Estas directivas comienzan con el signo de libra # (almohadilla). Normalmente, el compilador invoca automáticamente al preprocesador antes de comenzar la compilación. Se puede utilizar el preprocesador de tres formas distintas para hacer sus programas más modulares, más legibles y más fáciles de personalizar:

1. Mediante la directiva `#include` para insertar el contenido de un archivo en su programa.
2. Mediante la directiva `#define`, se pueden definir macros que permiten reemplazar una cadena por otra. Se puede utilizar la directiva `#define` para dar nombres significativos a constantes numéricas, mejorando la legibilidad de sus archivos fuente.
3. Con directivas tales como `#if`, `#ifdef`, `#else` y `#endif`, pueden compilar sólo partes de su programa. Se puede utilizar esta característica para escribir archivos fuente con código para dos o más sistemas, pero compilar sólo aquellas partes que se aplican al sistema informático en que se compila el programa.

D.6. ARCHIVOS DE CABECERA

Directivas tales como `#include <stdio.h>` indican al compilador que lea el archivo `studio.h` de modo que sus líneas se sitúan en la posición de la directiva. C ANSI soporta dos formatos para la directiva `#include`:

1. `#include <stdio>`
2. `#include "demo.h"`

El primer formato de `#include`, lee el contenido de un archivo (el archivo estándar de C, `stdio.h`). El segundo formato visualiza el nombre del archivo encerrado entre las dobles comillas que está en el directorio actual.

D.7. DEFINICIÓN DE MACROS

Una macro define un símbolo equivalente a una parte de código C y se utiliza para ello la directiva `#define`. Por ejemplo, se puede representar constantes tales como `PI`, `IVA` y `BUFFER`.

```
#define PI 3.14159
#define IVA 16
#define BUFFER 1024
```

que toman los valores `3.14159`, `16` y `1024`, respectivamente. Una macro también puede aceptar un parámetro y reemplazar cada ocurrencia de ese parámetro con el valor proporcionado cuando la macro se utiliza en un programa. Por consiguiente, el código que resulta de la expansión de una macro puede cambiar dependiendo del parámetro que se utilice cuando se ejecuta la macro. El código que resulta de la expansión de una macro puede cambiar dependiendo del parámetro que se utilice cuando se ejecuta la macro. Por ejemplo, la macro siguiente acepta un parámetro y expande a una expresión diseñada para calcular el cuadrado del parámetro:

```
#define cuadrado (x) ((x) * (x))
```

D.8. COMENTARIOS

El compilador ignora los comentarios encerrados entre los símbolos `/*` y `*/`.

```
/*Mi primer programa*/
```

Se pueden escribir comentarios multilínea:

```
/* Mi segundo programa C
escrito el dia 15 de Agosto de 1985 durante las fiestas
de Carchelejo - Jaen - España */
```

Los comentarios no pueden anidarse. La línea siguiente no es legal:

```
/*Comentario /* comentario interno */ externo */
```

D.9. TIPOS DE DATOS

Los tipos de datos básicos incorporados a C son *enteros, reales* y *carácter.*

Tabla D.1. Tipos de datos enteros

Tipo de dato	Tamaño en bytes	Tamaño en bits	Valor mínimo	Valor máximo
`signed char`	1	8	-128	127
`unsigned char`	1	8	0	255
`signed short`	2	16	-32.768	32.767
`unsigned short`	2	16	0	65.535
`signed int`	2	16	-32.768	32.767
`unsigned int`	2	16	0	65.535
`signed long`	4	32	-2.147.483.648	2.147.483.647
`unsigned long`	4	32	0	4.294.967.295

El tipo `char` se utiliza para representar caracteres o valores integrales. Las constantes de tipo `char` pueden ser caracteres encerrados entre comillas (`'A'`, `'b'`, `'p'`). Caracteres no imprimibles (tabulación, avance de página...) se pueden representar con secuencias de escape (`'\t'`, `'\f'`).

Tabla D.2. Secuencias de escape

Carácter	Significado	Código ASCII
`\a`	Carácter de alerta (timbre)	7
`\b`	Retroceso de espacio	8
`\f`	Avance de página	12
`\h`	Nueva línea	10

Carácter	Significado	Código ASCII
\r	Retorno de carro	13
\t	Tabulación (horizontal)	9
\v	Tabulación (vertical)	11
\\	Barra inclinada	92
\?	Signo de interrogación	63
\'	Comilla	39
\''	Doble comilla	34
\nnn	Número octal	—
\xnn	Número hexadecimal	—
'\0'	Carácter nulo (terminación de cadena)	—

Tabla D.3. Tipos de datos de coma flotante

Tipo de dato	Tamaño en bytes	Tamaño en bits	Valor mínimo	Valor máximo
float	4	32	3.4E-38	3.4E+38
double	8	64	1.7E-308	1.7E+308
long double	10	80	3.4E-4932	1.1E+4932

Todos los números sin un punto decimal en programas C se tratan como enteros y todos los números con un punto decimal se consideran reales de coma flotante de doble precisión. Si se desea representar números en base 16 (hexadecimal) o en base 8 (octal), se precede al número con el carácter '0X' para hexadecimal y '0' para octal. Si se desea especificar que un valor entero se almacena como un entero largo se debe seguir con una ''L''.

```
025   /* octal 25 o decimal 21 */
0x25  /* hexadecimal 25 o decimal 37 */
250L  /* entero largo 250 */
```

D.10. VARIABLES

Todas las variables en C se declaran o definen antes de que sean utilizadas. Una *declaración* indica el tipo de una variable. Si la declaración produce también almacenamiento (se inicia), entonces en una *definición*.

D.10.1. Nombres de variables en C

Los nombres de variables en C constan de letras, números y carácter subrayado. Pueden ser mayúsculas o minúsculas o una mezcla de tamaños. El tamaño de la letra es significativo, las variables siguientes son *todas diferentes*.

```
Temperatura TEMPERATURA temperatura
```

A veces se utilizan caracteres subrayados y mezcla de mayúsculas y minúsculas para aumentar la legibilidad:

```
Dia_de_Semana    DiaDeSemana    Nombre_ciudad
PagaMes

int x;                          //declara x variable entera
char nombre, conforme;          //declara nombre, conforme de tipo char

int x=, no = 0;                 //definen las variables x y no
flotat total = 42.125           //define la variable total
```

Se pueden declarar variables múltiples del mismo tipo en dos formas: Así una declaración

```
int v1; int v2; int v3; int v4;
```

o bien:

```
int v1;
int v2;
int v3;
int v4;
```

pudiéndose declarar también de la forma siguiente:

```
int v1, int v2, int v3, int v4;
```

C no soporta tipos de datos lógicos, pero mediante enteros se pueden representar: 0, significa *falso*; distinto de cero, significa *verdadero* (cierto).

La palabra reservada `const` permite definir determinadas variables con valores constantes, que no se pueden modificar. Así, se declara

```
const int z = 4350
```

y si se trata de modificar su valor

```
z = 3475
```

el compilador emite un mensaje de error similar a `"Cannot modify a const object in function main"` (`"No se puede modificar un objeto const en la funcion main"`). Las variables declaradas como `const` pueden recibir valores iniciales, pero no puede modificarse su valor con otras sentencias.

D.10.2. Variables tipo `char`

Las variables de tipo `char` (carácter) pueden almacenar caracteres individuales. Por ejemplo la definición

```
char car = 'M';
```

declara una variable `car` y le asigna el valor ASCII del carácter `M`. El compilador convierte la constante carácter `'M'` en un valor entero `(int)`, igual al código ASCII de `'M'` que se almacena a continuación en el byte reservado para `car`.

Dado que los caracteres literales se almacenan internamente como valores `int`, se puede cambiar la línea.

```
char car;
```

por

```
int car;
```

y el programa funcionará correctamente.

D.10.3. Constantes de cadena

Las cadenas de caracteres constan de cero o más caracteres separados por dobles comillas. La cadena se almacena en memoria como una serie de valores ASCII de tipo `char` de un solo byte y se termina con un byte cero, que se llama carácter nulo

```
'Sierra Magina en Jaen'
```

Además de los caracteres que son imprimibles, se pueden guardar en constantes cadena, códigos de escape, símbolos especiales que representan códigos de control y otros valores ASCII no imprimibles. Los códigos de escape se representan en la tabla como un carácter único, almacenado internamente como un valor entero y compuesto de una barra inclinada seguida por una letra, signo de puntuación o dígitos octales o hexadecimales. Por ejemplo, la declaración

```
char c = '\n';
```

asigna el símbolo nueva línea a la variable `c`. En los PC, cuando se envía un carácter `\n` a un dispositivo de salida, o cuando se escribe `\n` en un archivo de texto, el símbolo nueva línea se convierte en un retorno de carro y un avance de línea.

D.10.4. Tipos enumerados

El tipo `enum` es una *«lista ordenada»* de elementos como constantes enteras. A menos que se indique lo contrario, el primer miembro de un conjunto enumerado de valores toma el valor 0, pero se pueden especificar valores. La declaración:

```
enum diasSemana {Lunes, Martes, Miercoles, Jueves, Viernes, Sabado, Domingo};
```

significa que `Lunes = 0, martes = 1`, etc. Sin embargo, si se hace `Viernes = 10`, entonces `Lunes` sigue siendo 0, `Martes` es igual a 2, etc.; pero ahora `Viernes = 11, Sabado = 12`, etc. Un tipo enumerado se puede utilizar para declarar una variable

```
enum diasSemana Laborable;
```

y a continuación utilizarla con

```
Laborable = Jueves;
```

o bien:

```
Laborable = Sabado;
if (Laborable >= Viernes)
  printf ("Hoy no es laborable /n",Laborable);
```

D.11. EXPRESIONES Y OPERADORES

Las expresiones son operaciones que realiza el programa.

```
a + b + c;
```

Tabla D.4. Operadores aritméticos

Operador	Descripción	Ejemplo
*	Multiplicación	(a * b)
/	División	(a / b)
+	Suma	(a + b)
-	Resta	(a - b)
%	Módulo	(a % b)

Tabla D.5. Operadores relacionales

Operador	Descripción	Ejemplo
<	Menor que	(a < b)
<=	Menor que o igual	(a <= b)
>	Mayor que	(a > b)
>=	Mayor o igual que	(a >= b)
==	Igual	(a = = b)
!=	No igual	(a != b)
++	Incremento en i	++i, i++
--	Decremento en i	-- j, j --

Ejemplos:

```
i = i + 1;      //Sumar uno a i
i++;            //Igual que anterior

i = i - 1:      //Resta uno a i
i--;            //Igual que anterior
```

Tabla D.6. Operadores de manipulación de bits (bitwise)

Operador	Descripción	Ejemplo
&	AND bit a bit	C = A&B;
|	OR inclusiva bit a bit	C = A | B;
^	OR exclusiva bit a bit	C = A ^ B;
<<	Desplazar bits a izquierda	C = A << B;
>>	Desplazar bits a derecha	C = A >> B;
~	Complemento a uno	C = ~B

D.11.1. Operadores de asignación

Los operadores de asignación son binarios y combinaciones de operadores y del signo = utilizado para abreviar expresiones:

```
A = B        /* asigna el valor de B a A
C = (A = B) /* C y A son iguales a B
C =  A = B  /* asigna B a A y a C
```

```
A = A + 45;  equivale a  A += 45;
```

El compilador puede generar código más eficiente, recurriendo a operadores de asignación compuestos del tipo *=, +=, etc.; cada operador compuesto (*op*) reduce la expresión en pseudo-código:

```
O = a op b
```

a la forma abreviada

```
a op = b;
```

Tabla D.7. Operadores de asignación

Operador	Descripción y ejemplo		
=	Operación de asignación simple		a = b;
*=	z *= 10;	*equivale a*	z = z * 10;
/=	z /= 5;	*equivale a*	z = z / 5;
%=	z %= 2;	*equivale a*	z = z % 2;
+=	z += 4;	*equivale a*	z = z + 4;
-=	z -= 5;	*equivale a*	z = z - 5;
<<=	z <<= 3;	*equivale a*	z = z << 3;
>>=	z >>= 4;	*equivale a*	z = z > 3;
&=	z &= j;	*equivale a*	z = z $ j;
^=	z ^= j;	*equivale a*	z = z ^ j;
|=	z |= j;	*equivale a*	z = z | j;

D.11.2. Operador serie

El operador en serie, la coma, indica una serie de sentencias ejecutadas de izquierda a derecha. Se utilizan normalmente en bucles `for`. Por ejemplo:

```
for (cuenta=1; cuenta<100; ++cuenta, ++lineasporpagina);
```

produce el incremento de la variable cuenta y de la variable `lineasporpagina` cada vez que se ejecuta el bucle (se realiza una iteración).

D.11.3. Prioridad (precedencia) de operadores

Las expresiones C constan de diversos operandos y operadores. En expresiones complejas, las subexpresiones con operadores de prioridad (precedencia) más alta se evalúan antes que las subexpresiones con operadores de menor prioridad.

Tabla D.8. Orden de evaluación y prioridad de operadores (asociatividad)

Nivel	Operadores	Orden de evaluación
1	`( ) . [ ] →`	izquierda-derecha
2	`* & ! ~ ++ -- + -` *(conversion de tipo sizeof)*	derecha-izquierda
3	`* / %`	izquierda-derecha
4	`+ -`	izquierda-derecha
5	`<< >>`	izquierda-derecha
6	`< <= > >=`	izquierda-derecha
7	`== !=`	izquierda-derecha
8	`&`	izquierda-derecha
9	`^`	izquierda-derecha
10	`|`	izquierda-derecha
11	`&&`	izquierda-derecha
12	`||`	izquierda-derecha
13	`? :`	derecha-izquierda
14	`= *= /= += -= %= <<= >>= &= ^= |=`	derecha-izquierda
15	`,`	izquierda-derecha

D.12. FUNCIONES DE ENTRADA Y SALIDA

Las funciones `printf()` y `scanf()` permiten comunicarse con un programa y se denominan funciones de E/S. `printf()` es una *función de salida* (S) y `scanf()` es una *función de entrada* y ambas utilizan una cadena de control y una lista de argumentos.

D.12.1. `printf`

La función escribe en el dispositivo de salida, los argumentos incluidos en la lista de argumentos. Requiere el archivo de cabecera *`stdio.h`* La salida de `printf` se realiza con formato, y el mismo consta de una cadena de control y una lista de variables.

```
printf (cadena de control [, item1, item2,…item]);
```

El primer argumento es la *cadena de control* (o formato propiamente dicho) y determina el formato de escritura de los datos. Los argumentos restantes son los datos o variables de datos a escribir

```
printf ("Esto es una prueba %d\n", prueba);
```

La cadena de control tiene tres componentes: texto, identificadores y secuencias de escape. Se puede utilizar cualquier texto, identificadores y secuencias de escape. El número de identificadores ha de corresponder con el número de variables o valores a escribir.

Los identificadores de la cadena de formato determinan cómo se escriben cada uno de los argumentos:

```
printf ("Mi pueblo favorito es Cazorla%s", msg);
```

Cada identificador comienza con un signo porcentaje (%) y un código que indica el formato de salida de la variable.

Tabla D.9. Códigos de identificadores

Identificador	Formato
%d	Entero decimal
%c	Carácter simple
%s	Cadena de caracteres
%f	Coma flotante (decimal)
%e	Coma flotante (notación exponencial)
%g	Usa el %f o el %e más corto
%u	Entero decimal sin signo
%o	Entero octal sin signo
%x	Entero hexadecimal sin signo

Las secuencias de escape son las indicadas en la tabla

```
printf ("Mi flor favorita es la %s\n", msg);
printf ("La temperatura es %f grados centigrados \n", centigrados);
```

Ejemplo D.1

```
#include <stdio.h>
#define PI 3.141593
#define SIERRA "Sierra Magina"

int main (void)
{
    printf ("El valor de pi es % f.\n", PI);
    printf ("/%2s/ \n", SIERRA);
    printf ('/%22s/ \n", SIERRA);
    return 0;
}
```

La salida producida al ejecutar el programa es:

```
El valor de PI es 3.141593.
/Sierra Magina/
/              Sierra Magina/
```

D.12.2. scanf

La función `scanf()` es la función de entrada con formato. Esta función se puede utilizar para introducir números con formato de máquina, caracteres o cadena de caracteres, a un programa

```
scanf ("%f", &fahrenheit);
```

El formato general de la función `scanf()` es una cadena de formato y uno o más variables de entrada. La cadena de control consta sólo de identificadores.

Tabla D.10. Identificadores de formato de `scanf`

Identificador	Formato
`%d`	Entero decimal
`%c`	Carácter simple
`%s`	Cadena de caracteres
`%f`	Coma flotante
`%e`	Coma flotante
`%u`	Entero decimal sin signo
`%o`	Entero octal sin signo
`%x`	Entero hexadecimal sin signo
`&h`	Entero corto

Un ejemplo típico de uso de `scanf` es:

```
printf ("Introduzca ciudad y provincia :");
scanf ("%s %s", ciudad, provincia);
```

otros ejemplos son:

```
scanf ("%d", &cuenta);
scanf ("%20s", direccion)
scanf ("%d%s", &r, &c);
scanf ("%d*c%d", &x, &y);
```

D.13. SENTENCIAS DE CONTROL

Una *sentencia* consta de palabras reservadas, expresiones y otras sentencias; cada sentencia termina con su punto y coma `(;)`.

Un tipo especial de sentencia, la *sentencia compuesta* o *bloque,* es un grupo de sentencias encerradas entre llaves `({...})`. El cuerpo de una función es una sentencia compuesta. Una sentencia compuesta puede tener variables locales.

D.13.1. Sentencia if

1. `if (expresion)`
   `sentencia;`

o bien: `if (expresion)` *`sentencia;`*

2. Si la sentencia es *compuesta*

```
if (expresion) {                    o bien:     if (expresion)
                                                {
    sentencia1;                                     sentencia1;
    sentencia2;                                     sentencia2;
    ...                                             ...
}                                               }
```

3.
```
if (expresion == valor)
    sentencia;
```

4.
```
if (expresion != 0)          equivale a    if (expresion)
    sentencia;
```

NOTA: (expresion != 0) y (*expresion*) son equivalentes, ya que cualquier valor distinto de cero representa *cierto*.

D.13.2. Sentencia `if-else`

1.
```
if (expresion)
    sentencia1;
else
    sentencia2;
```

2.
```
if (expresion) {                    o bien:     if (expresion)
                                                {
    sentencia1;                                     sentencia1;
    sentencia2;                                     sentencia2;
    ...                                         }
} else                                          else
                                                {
    sentencia3;                                     sentencia3
    sentencia4;                                     sentencia4;
    ...
}                                               }
```

D.13.2.1. Sentencias `if` anidadas

1.
```
if (expresion1)
    sentencia1;
else if (expresion2)
    sentencia2;
else
    sentencia3;
```

```
2. if (expresion1)
       sentencia1;
   else if (expresion2)
       sentencia2;
   else if (expresion3)
       sentencia3;
   else if (expresionN)
       sentenciaN;
   else        /* opcional */
       sentenciaPorOmision;    /* opcional */
```

D.13.3. Expresión condicional (?:)

Expresión condicional es una simplificación de una sentencia `if-else`. Su sintaxis es:

```
expresion1 ? expresion2 : expresion3;
```

que equivale a

```
if (expresion1)
    expresion2;
else
    expresion3;
```

Ejemplo D.2

```
if (opcion == 'S')
   premio = 1000
else
   premio = 0
```

equivale a

```
Premio=(opcion =='S')? 1000 = 0
```

D.13.4. Sentencia `switch`

La sentencia `switch` realiza una bifurcación múltiple, dependiendo del valor de una expresión

```
switch (expresion) {
  case valor1:
    sentencia1; /* se ejecuta si expresion igual a 1*/
    break;      /* salida de sentencia switch       */
  case valor2:
    sentencia2;
    break;
  case valor3:
    sentencia3;
    break;
  ...
  default:
  sentencia por omision /* se ejecuta si ningun valor coincide
                           con expresion */
  }
```

Ejemplo D.3

```
switch (op)
{
    case 'a':
       func1();
       break;
    case 'b' :
       func2 ();
       break;
    case 'H':
       printf ("Hola \n");
    default:
       printf ("Salida \n");
}
```

D.13.5. Sentencia while

```
while (expresion)
  sentencia;
```

o bien:

```
while (expresion) {
  sentencia1;
  sentencia2;
  ...
}
```

D.13.6. Sentencia do-while

```
do {
  sentencia;
  ...
} while (expresion);
```

o bien:

```
do {
  sentencia1;
  sentencia2;
  ...
} while (expresion)
```

Las sentencias do-while monosentencias se pueden escribir también así:

```
do sentencia; while  (expresion);
```

D.13.7. Sentencia for

1.
```
for (expresion1; expresion2; expresion3) {
  sentencia;
  }
```

2.
```
for (expresion1; expresion2; expresion3) sentencia;
```

La sentencia for equivale a

```
expresion1;
while (expresion2) {
   sentencia;
   expresion3;
}
```

Ejemplo D.4

a)
```
for (i = 1; i <= 100; i++)
    printf ("i == %d\n", i);
```

b)
```
for (i = 0, suma = 0; i <= 100; suma + = i, i++);
```

sentencia nula

D.13.8. Sentencia nula

La *sentencia nula* representada por punto y coma no hace nada. Se utilizan sentencias nulas en bucles, cuando todo el proceso se hace en las expresiones del bucle en lugar del cuerpo. Por ejemplo, localizar el byte cero que marca el final de una cadena:

```
char cad[80] = "Prueba";
int i;

for (i = o; cad [i] != '\0'; i++)
    ;   /* sentencia nula */
```

D.13.9. Sentencia `break`

La sentencia `break` se utiliza para salir incondicionalmente de un bucle `for`, `while`, `do-while` o de una sección `case` de una sentencia `switch`.

```
for (;;) {
...
  if (expresion)
  break;
}
```

D.13.10. Sentencia `continue`

La sentencia `continue` salta en el interior del bucle hasta el principio del bucle para proseguir con la *ejecución*, dejando sin ejecutar las líneas restantes después de la sentencia `continue` hasta el final del bucle; `continue` es similar a la sentencia `break`.

```
while (expresion1) {
  ...
  if (expresion2)
    continue;
  ...
}
```

D.13.11. Sentencia goto

Una sentencia `goto` con una etiqueta dirige el programa a una sentencia específica que contiene dicha etiqueta. Las etiquetas deben terminar con un símbolo de dos puntos.

```
salto:
   ...
   if (expresion) goto salto;
```

D.14. FUNCIONES

Las funciones son los bloques de construcción de programas C. Una *función* es una colección de declaraciones y sentencias. Cada programa C tiene al menos una función: la función `main`; ésta es la función donde comienza la ejecución de un programa C. La biblioteca C ANSI contiene gran cantidad de funciones estándar.

D.14.1. Sentencia return

La sentencia `return` detiene la ejecución de la función actual y devuelve al control a la función llamadora. La sintaxis es:

```
return expresion
```

en donde el valor de *expresion* se devuelve como valor de la función.

```
#include <stdio.h>

main ()
{
   printf ("Sierra de Cazorla y \n");
   printf ("Sierra Magina \n")N
   printf ("son dis bonitas sierras andaluzas \n");
   return 0;
}
```

D.14.2. Prototipos de funciones

En C ANSI se debe declarar una función antes de utilizarla. La declaración de la función indica al compilador el tipo de valor que devuelve la función y el número y tipos de argumentos que acepta.

El *prototipo de una función* es el nombre de la función, la lista de sus argumentos y el tipo de dato que devuelve.

```
int SeleccionarMenu(void);
double Area(int x, int y);
void salir(int estado);
```

D.14.3. El tipo `void`

C ANSI ha incorporado el tipo de dato `void`, para declarar funciones que no devuelven *nada* ni aceptan parámetros, así como para describir punteros que pueden apuntar a cualquier tipo de dato.

```
void exit (int estado);
void CuentaArriba (void);
void CuentaAbajo (void);
```

Errores típicos de funciones

1. *Ningún retorno de valor.* La función carece de una sentencia `return`. Si una función termina sin ejecutar `return`, devuelve un valor impredecible, que puede producir errores serios.
2. *Retornos omitidos.* Si hay funciones que tienen sentencias `if`, hay que asegurarse de que existe una sentencia `return` por cada camino de salida posible.
3. *Ausencia de prototipos.* Las funciones que carecen de prototipos se considera que devuelven `int`, incluso aunque estén definidas para devolver valores de otro tipo. Como *regla general, declarar prototipos para todas las funciones.*
4. *Efectos laterales.* Este problema se produce normalmente por una función que cambia el valor de una o más variables globales.

D.14.4. Detener un programa con `exit`

Un medio para terminar un programa sin mensajes de error en el compilador es utilizar la sentencia

```
return valor;
```

donde *valor* es cualquier expresión entera.

Otro medio para detener un programa es llamar a `exit`. La ejecución de la sentencia

```
exit (0);
```

termina el programa inmediatamente y cierra todos los archivos abiertos.

D.15. ESTRUCTURAS DE DATOS

Los tipos complejos o estructuras de datos en C son: *arrays, estructuras, uniones, cadenas* y *campos de bits.*

D.15.1. Arrays

Todos los arrays en C comienzan con el índice [0].

```
int notas[25];          /* declara array de 25 elementos */
float lista[10][25];    /* declara array de 10 por 25 dimensiones */
```

El número de elementos de un array se puede determinar dividiendo el tamaño del array completo por el tamaño de uno de los elementos

```
noelementos = sizeof(lista) / sizeof(lista[0])
```

Un *array estático* se puede iniciar cuando se declara con

```
static char nombre[] ="Sierra Magina";
```

Un *array auto* se puede iniciar sólo con una expresión constante en C ANSI.

```
int listamenor [2][2] = {{25, 4}, {100, 75}};
int digitos[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, };
```

Un *array de punteros* a caracteres se puede iniciar con

```
char *colores[]= {"verde}, "rojo", "amarillo", "rosa", "azul"};
```

D.16. CADENAS

Las *cadenas* son simplemente arrays de caracteres que terminan siempre con un valor nulo ('\0')

```
char cadena[80];  /* declara una cadena de 80 caracteres */
char mensaje[] = "Carchelejo esta en Sierra Magina";
char frutas[][10] = {"naranja", "platano", manzana"};
```

D.17. ESTRUCTURAS

Las *estructuras* son colecciones de datos, normalmente de tipos diferentes que actúan como un todo. Puede contener tipos de datos simples (*carácter, float, array* y *enumerado*) o compuestos (estructuras, arrays o uniones)

```
struct coordenada {
  int x;
  int y;
};

struct signatura {
  char titulo[40];
  char autor[30];
  int paginas;
  int anyopubli;
};
```

Una variable de tipo estructura se puede declarar así:

```
struct coordenada punto;0
struct signatura librosinfantiles;
```

para asignar valores a los miembros de una estructura se utiliza la notación punto (.)

```
punto.x = 12
punto.y = 15;
```

Existe otro modo de declarar estructuras y variables de tipo estructura.

```
struct coordenada {
  int x;
  int y;
} punto;
```

Para evitar tener que escribir `struct` cada vez que se declara la estructura, se puede usar `typedef` en la declaración:

```
typedef struct coordenada {
  int x;
  int y;
} Coordenadas;
```

y, a continuación, se puede declarar la variable `Punto`:

```
Coordenadas punto;
```

y utilizar la notación punto (.)

```
punto.x =12
punto.y = 15;
```

D.18. UNIONES

Las *uniones* son casi idénticas a las estructuras en su sintaxis; proporcionan un medio de almacenar más de un tipo en una posición de memoria. Se define un tipo unión

```
union tipodemo {
  short x;
  long l;
  float f;
};
```

y se declara una variable con

```
union tipodemo demo;
```

pudiendo utilizarse mediante

```
demo.x = 345;
```

o bien mediante

```
demo.y = 324567;
```

La unión anterior se puede iniciar así:

```
union tipodemo { short x; long l; float f; } = {75};
```

D.19. CAMPOS DE BITS

Los *campos de bits* se utilizan con frecuencia para poner enteros en espacios más pequeños de los que el compilador pueda normalmente utilizar y son, por consiguiente, dependientes de la implementación. Una estructura de campos de bits especifica el número de bits que cada miembro ocupa.

Una declaración de una estructura `Persona`

```
struct persona {
  unsigned edad;     /* 0::99 */
  unsigned sexo;     /* 0 = varon, 1 = hembra */
  unsigned hijos;    /* 0 .. 15 */
};
```

y de una estructura de campos de bits

```
struct persona {
  unsigned edad: 7;    /* 0..1127 */
  unsigned sexo: 1;    /* 0 = varon, 1 = hembra */
  unsigned hijos: 4;   /* 0..15 */
}
```

D.20. PUNTEROS (APUNTADORES [1])

El *puntero* es un tipo de dato especial que contiene la dirección de otra variable. Un puntero se declara utilizando el asterisco (*) delante de un nombre de variable

```
float *longitudOnda;     /* puntero a datos float */
char * indice;           /* puntero a datos char */
int *p;                  /* puntero a datos int */
```

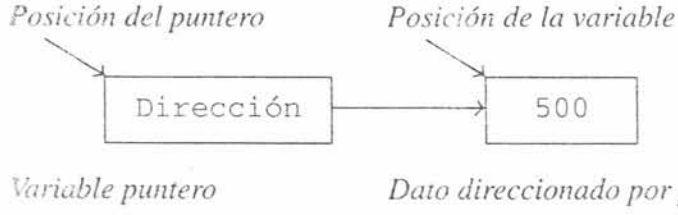


Figura D.2. Un puntero es una variable que contiene una dirección.

D.20.1. Declaración e indirección de punteros

```
int m;  /* una variable entera un */
int *p  /* un puntero p apunta a un valor entero */
p = &i; /* se asigna a p la direccion de la variable m*/
```

[1] En Latinoamérica se utiliza el término *apuntador*.

El operador de indirección (&) se utiliza para acceder al valor de la dirección contenida en un puntero.

```
float *longitudOnda;
longitudOnda = &cable1;
*longitudOnda = 40.5;
```

D.20.2. Punteros nulos y void

Un puntero *nulo* apunta a ninguna parte específica; es decir, no direcciona a ningún dato válido en memoria:

```
fp = NULL;    /* asigna Nulo a fp */
fp = 0;       /* asigna Nulo a fp */

if (fp != NULL)  /* el programa verifica si el puntero es valido */
```

Al igual que una función void que no devuelve ningún valor, un puntero void apunta a un tipo de dato no especificado

```
void *noapuntafijo;
```

D.20.3. Punteros a valores

Para utilizar un puntero que apunte a un valor, se utiliza el operador de indirección (*) delante de la variable puntero

```
double *total;
*total = 34750.75;
```

El operador de dirección (&) asigna el puntero a la dirección de la variable:

```
double *total;
double sctotal;
total = &ctotal;
```

D.20.4. Punteros y arrays

Si un programa declara

```
float *punterolista;
float listafloat [50];
```

entonces a punterolista se puede asignar la dirección del principio de listafloat (el primer elemento).

```
punterolista = listafloat;
```

o bien:

```
punterolista = &listafloat [0];

listafloat[4]     es lo mismo que     *(listafloat + 4)
listafloat[2]     es lo mismo que     *(listafloat + 2)
```

D.20.5. Punteros a punteros

Los punteros pueden apuntar a otros punteros (*doble indirección*):

```
char **lista;   /* se declara un puntero a otro puntero char */
```

equivale a

```
char *lista[];
```

y

```
char ****ptr   /* un puntero a un puntero a char */
```

D.20.6. Punteros a funciones

La declaración

```
int (*ptrafuncion ((void);
```

crea un puntero a una función que devuelve un entero, y la declaración

```
float (*ptrafuncion1) (int x, int y);
```

declara `ptrafuncion1` como un puntero a una función que devuelve un valor `float` y requiere dos argumentos enteros. Otros ejemplos:

```
float *ptr1;         /* ptr apunta a un valor float */
float *mifun(void); /* funcion mifun devuelve un puntero a un valor float */
```

APÉNDICE E

GUÍA DE SINTAXIS DEL LENGUAJE C++ (ESTÁNDAR C++ ANSI)

C++ es considerado un C más grande y potente. La sintaxis de C++ es una extensión de C, al que se han añadido numerosas propiedades, fundamentalmente orientada a objetos. ANSI C ya adoptó numerosas características de C++, por lo que la emigración de C a C++ no suele ser difícil.

En este apéndice se muestran las reglas de sintaxis del estándar clásico de C++ recogidas en al *Annotated Manual* (ARM) de Stroustrup y Ellis, así como las últimas propuestas incorporadas al nuevo borrador de C++ ANSI, que se incluyen en las versiones 3 (actual) y 4 (futura de AT&T C++).

E.1. ELEMENTOS DEL LENGUAJE

Un programa en C++ es una secuencia de caracteres que se agrupan en componentes léxicos (*tokens*) que comprenden el vocabulario básico del lenguaje. Estos componentes de léxico son: palabras reservadas, identificadores, constantes, constantes de cadena, operadores y signos de puntuación.

E.1.1. Caracteres

Los caracteres que se pueden utilizar para construir elementos del lenguaje (componentes léxicos o *tokens*) son:

```
a b c d e f g h i j k l m n o p q r s t u v w x y z
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0 1 2 3 4 5 6 7 8 9
+ - * / = ( ) { } [ ] < > ' " ! @ =/ $ ^ & % -: . , ; ? \ |
```

caracteres espacio (blancos y tabulaciones)

E.1.2. Comentarios

C++ soporta dos tipos de comentarios. Las líneas de comentarios al estilo C y C ANSI, tal como:

```
/* Comentario estilo C */
/* Comentario mas extenso, pero tambien es estilo C y ANSI C */
```

El otro tipo de comentarios se pueden utilizar por los programadores de C++: La versión /* . . . */ se utiliza para comentarios que excedan una línea de longitud y la versión // . . . se utiliza, sólo, para comentarios de una línea. *Los comentarios no se anidan.*

E.1.3. Identificadores

Los *identificadores* (nombres de variables, constantes,...) deben comenzar con una letra del alfabeto (mayúscula o minúscula) o con un carácter subrayado, y pueden tener uno o más caracteres. Los caracteres segundo y posteriores pueden ser: letras, dígitos o un subrayado, no permitiéndose caracteres no alfanuméricos ni espacios.

```
test_prueba    //legal
X123           //legal
multi_palabra  //legal
var25          //legal
15var          //no legal
```

C++ es sensible a las mayúsculas.

```
Paga_mes          es un identificador distinto      a paga_mes
```

Una buena práctica de programación aconseja utilizar identificadores significativos que ayudan a documentar un programa.

```
nombre  apellidos  salario  precio_neto
```

E.1.4. Palabras reservadas

Las palabras reservadas o claves no se deben utilizar como identificadores, debido a su significado estricto en C++; tampoco se deben redefinir. La Tabla E.1 enumera las palabras reservadas de C++ según el ARM[1].

Tabla E.1. Palabras reservadas de C++

asm*	continue	float	new*	signed	try
auto	default	for	operator*	sizeof	typedef
break	delete*	friend*	private*	static	union
case	do	goto	protected*	struct	unsigned
catch*	double	if	public*	switch	virtual*
char	else	inline*	register	template*	void
class*	enum	int	return	this*	volatile
const	extern	long	short	throw*	while

* Estas palabras no existen en ANSI C.

[1] Siglas del libro de BJARNE STROUSTRUP en el que se definen las reglas de sintaxis del lenguaje C++ estándar. *Annotated Reference Manual*, Addison-Wesley, 1992.

Los diferentes compiladores comerciales de C++ pueden incluir, además, nuevas palabras reservadas. Éstos son los casos de Borland, Microsoft y Sysmantec.

Tabla E.2. Palabras reservadas de Turbo/Borland C++

asm	_ds	interrup	short
auto	else	_loadds	signed
break	enum	long	sizeof
case	_es	_near	_ss
catch	_export	near	static
_cdecl	extern	new	struct
cdecl	_far	operator	switch
char	far	pascal	template
class	float	pascal	this
const	for	private	typedet
continue	friend	protected	union
_cs	goto	pubic	unsigned
default	huge	register	virtual
delete	if	return	void
do	inline	_saveregs	volatile
_double	int	_seg	while

Tabla E.3. Palabras reservadas de Microsoft Visual C/C++ 1.5/2.0

_asm	else	int	signed
auto	enum	_interrupt	sizeof
based	_except	_leave	static
break	_export	_loadds	_stcall
case	extern	long	struct
_cdecl	_far	maked	switch
char	_fastcall	_near	thread
const	_finally	_pascal	_try
continue	float	register	typedef
_declspec	for	return	union
default	_fortran	_saveregs	unsigned
dllexport	goto	_self	void
dllimport	_huge	_segment	volatile
do	if	_segname	while
double	_inline	short	

El comité ANSI ha añadido nuevas palabras reservadas (Tabla E.4).

Tabla E.4. Nuevas palabras reservadas de ANSI C++

bool	false	reinterpret_cast	typeid
cons_cast	mutable	static_cast	using
dynamic_cast	namespace	true	wchar_t

E.2. TIPOS DE DATOS

Los tipos de datos simples en C++ se dividen en dos grandes grupos: integrales (datos enteros) y de coma flotante (datos reales). La Tabla E.5. muestra los diferentes tipos de datos en C++.

Tabla E.5. Tipos de datos simples en C++

`char`	`signed char`	`unsigned char`
`short`	`int`	`long`
`usigned short`	`unsigned`	`unsigned long`
`float`	`double`	`long double`

Los tipos derivados en C++ pueden ser:

- enumeraciones (`enum`),
- estructuras (`struct`),
- uniones (`union`),
- arrays,
- clases (`class` y `struct`),
- uniones y enumeraciones anónimas,
- punteros,

E.2.1. Verificación de tipos

La verificación o comprobación de tipos en C++ es más rígida (estricta) que en C. Algunas consideraciones a tener en cuenta son:

- *Usar funciones declaradas.* Esta acción es ilegal en C++, y está permitida en C:

```
int main ()
{
    //...
    printf(x);        //C: int printf();
                      //C++ es ilegal, ya que printf no esta declarada
    return 0;
}
```

- *Fallo al devolver un valor de una función.* Una función en C++ declarada con un tipo determinado de retorno, ha de devolver un valor de ese tipo. En C, está permitido no seguir la regla.
- *Asignación de punteros* `void`. La asignación de un tipo `void*` a un puntero de otro tipo se debe hacer con una conversación explícita en C++. En C, se realiza implícitamente.
- *Inicialización de constantes de cadena.* En C++ se debe proporcionar un espacio para el carácter de terminación nulo cuando se inicializan constantes de cadena. En C, se permite la ausencia de ese carácter.

```
int main ()
{
     //...
     char car [7] = "Cazorla";    //legal en C
                                  //error en C++
    //...
    return 0;
}
```

Una solución al problema que funciona tanto en C como en C++ es:

```
char car[] = "Cazorla";
```

E.3. CONSTANTES

C++ contiene constantes para cada tipo de dato simple (integer, char,...). Las constantes pueden tener dos sufijos, `u`, `l` y `f`, que indican tipos `unsigned`, `long` y `float`, respectivamente. Así mismo, se pueden añadir los prefijos `o` y `ox`, que representan constantes octales y hexadecimales.

```
456  0456 0x456          //constantes enteras:decimal, octal, hexadecimal

1231 123ul               //constantes enteras:long, unsigned long

'B'  'b'  '4'            //constantes tipo char

3.1415f  3.14159L        //constantes reales de diferente posicion

"cadena de caracteres" //constante de cadena
```

Las cadenas de caracteres se encierran entre comillas, y las constantes de un solo carácter se encierran entre comillas simples.

```
" " //cadena vacia, '\0'
```

E.3.1. Declaración de constantes

En C++, los identificadores de variables/constantes se pueden declarar *constantes,* significando que su valor no se puede modificar. Esta declaración se realiza con la palabra reservada `const`.

```
const double PI = 3.1416;
const char BLANCO = ' ';
const double PI_EG = -I;
const double DOBLE_I = 2 * PI;
```

El modificador de tipos `const` se utiliza en C++, también para proporcionar protección de sólo lectura para variables y parámetros de funciones. Las funciones miembro de una clase que no modifican los miembros dato a que acceden pueden ser declarados `const`. Este modificador evita también que parámetros por referencia sean modificados:

```
void copy (const char* fuente, char* destino);
```

E.4. CONVERSIÓN DE TIPOS

Las conversiones explícitas se fuerzan mediante *moldes* (`casts`). La conversión forzosa de tipos de C tiene el formato clásico:

```
(tipo) expresion
```

C++ ha modificado la notación anterior por una notación funcional como alternativa sintáctica:

```
nombre del tipo (expresion)
```

Las notaciones siguientes son equivalentes:

```
z = float(x);    //notacion de moldes en C++
z = (float)x;    //notacion de moldes en C
```

E.5. DECLARACIÓN DE VARIABLES

En ANSI C, todas las declaraciones de variables y funciones se deben hacer al principio del programa o función. Si se necesitan declaraciones adicionales, el programador debe volver al bloque de declaraciones al objeto de hacer los ajustes o inserciones necesarios. Todas las declaraciones deben hacerse antes de que se ejecute cualquier sentencia. Así, la declaración típica en C++,

```
NombreTipo, NombreVariable1, NombreVariable2, ...
```

proporciona declaraciones tales como:

```
int saldo, meses;
double clipper, salario;
```

Al igual que en C, se pueden asignar valores a las variables en C++:

```
int mes = 4, dia, anyo = 1995;
double salario = 45.675;
```

En C++, las declaraciones de variables se pueden situar en cualquier parte de un programa. Esta característica hace que el programador declare sus variables en la proximidad del lugar donde se utilizan las sentencias de su programa. El siguiente programa es legal en C++, pero no es válido en C:

```
#include <stdio.h>
int main ()
{
   int i;
   for (i = o; i < 100; ++i)
      printf ("%d\n",i);

   double j;
   for (j = 1.7547; j < 25.4675; j + = .001)
        printf ("%1f\n",y);
}
```

El programa anterior se podría reescribir, haciendo la declaración y la definición dentro del mismo bucle:

```
int main ()
{
  for (int i = 0; i < 100; ++i)
      cout << i << end1;

  for (double j = 1.7545; j < 25.4675; j + = .001)
      cout << j << end1;
}
```

E.6. OPERADORES

C++ es un lenguaje muy rico en operadores. Se clasifican en los siguientes grupos:

- Aritméticos.
- Relacionales y lógicos.
- Asignación.
- Acceso a datos y tamaño.
- Manipulación de bits.
- Varios.

Como consecuencia de la gran cantidad de operadores, se producen también una gran cantidad de expresiones diferentes.

E.6.1. Operadores aritméticos

C++ proporciona diferentes operadores que relacionan operaciones aritméticas.

Tabla E.6. Operadores aritméticos en C++

Operador	Nombre	Propósito	Ejemplo
+	Más unitario	Valor positivo de x	x = +y, +5
-	Negación	Valor negativo de x	x = -y;
+	Suma	Suma x e y	z = x + y;
-	Resta	Resta y de x	z = x - y;
*	Multiplicación	Multiplica x por y	z = x * y;
/	División	Divide x por y	z = x / y;
%	Módulo	Resto de x dividido por y	z = x % y;
++	Incremento	Incrementa x después de usar	x++
--	Decremento	Decrementa x antes de usar	--x

Ejemplo:

```
-i + w;          //menos unitario mas unitario
a * b/c % t;     //multiplicacion, division, modulo
a - b a -b       //suma y resta binaria
a = 5/2;         //a toma el valor 2, si se considera a entero
a = 5/2;         //a toma el valor 2.5, si a es real
```

Los operadores de incremento y decremento sirven para incrementar y decrementar en uno los valores almacenados en una variable.

```
variable ++      //postincremento
++variable       //preincremento
variable --      //postdecremento
-- variable      //predecremento
++a;             equivale a      a = a + 1;
--b;             equivale a      b = b - 1;
```

Los formatos postfijos se conforman de modo diferente según la expresión en que se aplica:

```
b = ++a;          equivale a      a = a + 1; b = a
b = a++;          equivale a      b = a; a = a + 1;

int i, j, k = 5;
k++;              //k vale 6, igual efecto que ++k
--k;              //k vale ahora 5, igual efecto que k--
k = 5;
i = 4 * k++;      //k es ahora 6 e i es 20
k = 5;
j = 4* ++k;       //k es ahora 6 e i es 24
```

E.6.2. Operadores de asignación

El operador de asignación (=) hace que el valor situado a la derecha del operador se adjudique a la variable situada a su izquierda. La asignación suele ocurrir como parte de una expresión de asignación y las conversiones se producen implícitamente.

```
z = b + 5;        //asigna (b + 5) a variable z
```

C++ permite asignaciones múltiples en una sóla sentencia. Así:

```
a = b +(c=10);
```

equivale a:

```
c = 10;
a = b + c;
```

otros ejemplos de expresiones válidas y no válidas son:

```
//expresiones legales     //expresiones no legales
a = 5 * (b + a);            a + 3 = b;
double x = y;               PI = 3;
a = b = 6;                  x++ = y;
```

C++ proporciona operadores de asignación que combinan operadores de asignación y otros diferentes, produciendo operadores tales como +=, /=, -=, * = y %=. C++ soporta otros tipos de operadores de asignación para manipulación de bits.

Tabla E.7. Operadores aritméticos de asignación

Operador	Formato largo	Formato corto
+=	x = x + y;	x += y;
-=	x = x - y;	x -= y;
*=	x = x * y;	x *= y;
/=	x = x / y;	x /= y;
%=	x = x % y;	x %= y;

Ejemplos

```
a + b;          equivale a  a = a + b;
a * = a + b;    equivale a  a = a * (a + b);
v +- e;         equivale a  v = v + e;
v - = e;        equivale a  v = v%e;
```

Expresiones equivalentes:

```
n = n + 1
n += 1;
n++;
++n;
```

E.6.3. Operadores lógicos y relacionales

Los operadores lógicos y relacionales son los bloques de construcción básicos para constructores de toma de decisión en un lenguaje de programación. La Tabla E.8. muestra los operadores lógicos y relacionales:

Tabla E.8. Operadores lógicos y relacionales

Operador	Descripción	Ejemplo
&&	AND (*y*) lógico	a && b
||	OR (*o*) lógico	c || b
!	NOT (*no*) lógico	! c
<	Menor que	i < 0
<=	Menor o igual que	i <= 0
>	Mayor que	j > 50
>=	Mayor o igual que	j >=8.5
==	Igual a	x =='\0'
!=	No igual a	c != '\n'
?:	Asignación condicional	k = (i < 5) ? 1 = i;

El operador ?: se conoce como *expresión condicional.* La expresión condicional es una abreviatura de la sentencia condicional if_else. La sentencia if

```
if (condicion)
   variable = expresion1;
else
   variable = expresion2;
```

es equivalente a

```
variable = (condicion) ? expresion1 : expresion2;
```

La expresión condicional comprueba la condición. Si esa condición es verdadera, se asigna *expresion1* a *variable*; en caso contrario, se asigna *expresion2* a *variable.*

Reglas prácticas

Los operadores lógicos y relacionales actúan sobre valores lógicos: el valor *falso* puede ser o bien 0, o bien el puntero nulo, o bien 0.0; el valor *verdadero* puede ser cualquier valor distinto de cero. La siguiente tabla muestra los resultados de diferentes expresiones.

`x > y`	1, si x excede a y, sino 0
`x >= y`	1, si x es mayor o igual a y, sino 0
`x < y`	1, si x es menor que y, sino 0
`x <= y`	1, si x es menor que o igual a y, sino 0
`x == y`	1, si x es igual a y, sino 0
`x != y`	1, si x e y son distintos, sino 0
`!x`	1, si x es 0, sino 0
`x || y`	0, si ambos x e y son 0, sino 0

Evaluación en cortocircuito

C++, igual que C, admite reducir el tiempo de las operaciones lógicas; la evaluación de las expresiones se reduce cuando alguno de los operandos toman valores concretos.

1. *Operación lógica AND (&&).* Si en la expresión `expr1&& expr2, expr1` toma el valor cero y la operación lógica AND (y) siempre será cero, sea cual sea el valor de `expr2`. En consecuencia, `expr2` no se evaluará nunca.
2. *Operación lógica OR (||).* Si `expr1` toma un valor distinto de cero, la expresión `expr1 || expr2` se evaluará a 1, cualquiera que sea el valor de `expr2`; en consecuencia, `expr2` no se evaluará.

E.6.4. Operaciones de manipulación de bits

C++ proporciona operadores de manipulación de bits, así como operadores de asignación de manipulación de bits.

Tabla E.9. Operadores de manipulación de bits (*bitwise*)

Operador	Descripción	Ejemplo
`&`	AND bit a bit	`x & 128`
`|`	OR bit a bit	`j | 64`
`^`	XOR bit a bit	`j ^ 12`
`~`	NOT bit a bit	`~ j`
`<<`	Desplazar a la izquierda	`i << 3`
`>>`	Desplazar a la derecha	`j >> 4`

Tabla E.10. Operadores de asignación de manipulación de bits

Operador	Formato largo	Formato corto
&=	x = x & y;	x & = y;
|=	x = x | y;	x| = y;
^=	x = x ^ y;	x ^ = y;
<<=	x = x << y;	x << = y;
>>=	x = x >> y;	x >> = y;

Ejemplo:

```
~ x       Cambia los bits 1 a 0 y los bits 0 a 1
x & y     Operacion logica AND (y) bit a bit de x e y
x | y     Operacion logica OR (o) bit a bit de x e y
x << y    x se desplaza a la izquierda (en y posiciones)
x >> y    x se desplaza a la derecha (en y posiciones)
```

E.6.5. El Operador *sizeof*

El operador `sizeof` proporciona el tamaño en bites de un tipo de dato o variable `sizeof` toma el argumento correspondiente (tipo escalar, *array, record,* etc.). La sintaxis del operador es:

```
sizeof (nombre_variable |tipo_de_dato)
```

Ejemplo:

```
int m, n[12];
sizeof(m)       //proporciona 4, en maquinas de 32 bits
sizeof(n)       //proporciona 48
sizeof(15)      //proporciona 4
Tamanyo = sizeof(long) - sizeof(int);
```

E.6.6. Prioridad y asociatividad de operadores

Cuando se realizan expresiones en las que se mezclan operadores diferentes es preciso establecer una *precedencia* (prioridad) de los operadores y la *dirección* (o secuencia) de evaluación (orden de evaluación: izquierda-derecha, derecha-izquierda), denominada *asociatividad.*

La Tabla E.11 muestra la precedencia y asociatividad de operadores.

Ejemplo:

```
a * b/c + d    equivale a    (a * b) / (c + d)
```

E.6.7. Sobrecarga de operadores

La mayoría de los operadores de C++ pueden ser sobrecargados o redefinidos para trabajar con nuevos tipos de datos. La Tabla E.12. lista los operadores que pueden ser sobrecargados.

Tabla E.11. Precedencia y asociatividad de operadores

Operador	Asociatividad	Prioridad
`::` `()` `[]` `.` `->`	Izquierda-Derecha	1
`--`	Derecha-Izquierda	2
`&` *(direccion)* `~` *(tipo)* `!` `-` `+` `sizeof` *(tipo)* `new` `delete` `*` *(indireccion)*		
`.*` `->*`	Izquierda-Derecha	3
`*` `/` `%`	Izquierda-Derecha	4
`+` `-`	Izquierda-Derecha	5
`<<` `>>`	Izquierda-Derecha	6
`<` `<=` `>` `>=`	Izquierda-Derecha	7
`==` `!=`	Izquierda-Derecha	8
`&`	Izquierda-Derecha	9
`^`	Izquierda-Derecha	10
`|`	Izquierda-Derecha	11
`&&`	Izquierda-Derecha	12
`||`	Izquierda-Derecha	13
`?:`	Derecha-Izquierda	14
`=` `+=` `-=` `*=` `/=` `%=` `>>=` `<<=` `&=` `|=` `^=`	Derecha-Izquierda	15
`,` *(operador coma)*	Izquierda-Derecha	16

Tabla E.12. Operadores que se pueden sobrecargar

`+`	`-`	`*`	`\`	`%`	`^`	`&`	`!`
`~`	`!`	`=`	`<`	`>`	`+=`	`-=`	`*=`
`/=`	`%=`	`^=`	`&=`	`|=`	`<<`	`>>`	`>>=`
`<<=`	`==`	`!=`	`<=`	`>=`	`&&`	`||`	`++`
`--`	`,`	`->*`	`->()`	`[]`			

E.7. ENTRADAS Y SALIDAS BÁSICAS

Al contrario que muchos lenguajes, C++ no tiene facilidades incorporadas para manejar entrada o salida. En su lugar, se manejan por rutinas de bibliotecas. Las clases que C++ utiliza para entrada y salida se conocen como *flujos*. Un *flujo (stream)* es una secuencia de caracteres junto con una colección de rutinas para insertar caracteres en flujos (a pantalla) y extraer caracteres de un flujo (de teclado).

E.7.1. Salida

El flujo `cout` es el flujo de salida estándar que corresponde a `stdout` en C. Este flujo se deriva de la clase `ostream` construida en `isostream`.

Figura E.1. Uso de flujos para salida de caracteres.

Si se desea visualizar el valor del objeto `int` llamado `i`, se escribe la sentencia:

```
cout << i;
```

El siguiente programa visualiza en pantalla una frase:

```
#include <isostream.h>
int main ()
{
   cout <<"hola, mundo\n";
}
```

Las salidas en C++ se pueden conectar en cascada, con una facilidad de escritura mayor que en C.

```
#include <isostream.h>
int main ()
{
   int i;
   i = 1099;
   cout <<"El valor de i es" << i <<'\n";
}
```

Otro programa que muestra la conexión en cascada es:

```
#include <iostream.h>
int main ()
{
   int x = 45;
   double y = 495.125;
   char *c = "y multiplicada por x=";
   cout << c << y * x << "\n";
}
```

E.7.2. Entrada

La entrada se maneja por la clase `istream`. Existe un objeto predefinido `istream`, llamado `cin`, que se refiere al dispositivo de entrada estándar (el teclado). El operador que se utiliza para obtener un valor del teclado es el *operador de extracción* >>. Por ejemplo, si `i` era un objeto `int`, se escribirá:

```
cin >> i;
```

que obtiene un número del teclado y lo almacena en la variable `i`.

Un programa simple que lee un dato entero y lo visualiza en pantalla es:

```
#include <iostream.h>
int main ()
{
   int i;
   cin >> i; "\n";
   cout << i <<
}
```

Al igual que en el caso de `cout`, se pueden introducir datos en cascada:

```
#include <iostream.h>
int main ()
{
   char c[60];
   int x,y;

   cin >> c >> x >> y;
   cout << c << " "<< x <<" "<< y << "\n";
}
```

E.7.3. Manipuladores

Un método fácil de cambiar la anchura del flujo y otras variables de formato es utilizar un operador especial denominado *manipulador*. Un manipulador acepta una referencia de flujo como un argumento y devuelve una referencia al mismo flujo.

El siguiente programa muestra el uso de manipuladores específicamente para conversiones de número (`dec`, `oct`, y `hex`)

```
#include <iostream.h>
int main ()
{
   int i = 36;
   cout << dec << i << oct << i << " "<< hex << i << "\n";
}
```

La salida de este programa es:

```
36   44   24
```

Otro manipulador típico es `endl`, que representa al carácter de nueva línea (*salto de línea*), es equivalente a `'\n'`. El programa anterior se puede escribir también así:

```
#include <isostream.h>
int main ()
{
   int i = 36;

   cout << dec << i <<" " << oct << i << " " << hex << i << endl;
}
```

E.8. SENTENCIAS

Un programa en C++ consta de una secuencia de sentencias. Existen diversos tipos de sentencias. El punto y coma se utiliza como elemento terminal de cada sentencia.

E.8.1. Sentencia de declaración

Se utilizan para establecer la existencia y, opcionalmente, los valores iniciales de objetos identificados por nombre.

```
NombreTipo identificador, ...;
NombreTipo identificador = expresion, ...;
const NombreTipo identificador = expresion, ...;
```

Algunas sentencias válidas en C++ son:

```
char c1;
int p, q = 5, r = a + b;    //suponiendo que a y b han
                            //sido declaradas e
                            //inicializadas antes
const double IVA = 16.0;
```

E.8.2. Sentencias expresión

Las sentencias de expresiones hacen que la expresión sea evaluada. Su formato general es:

```
expresion;
```

Ejemplo:

```
n++;
425;                    //legal, pero no hace nada
a + b;                  //legal, pero no hace nada
n = a < b || b != 0;
a += b = 3;              //sentencia compleja
```

C++ permite asignaciones múltiples en una sentencia.

```
m = n + (p = 5);     equivale a      p = 5;
                                     m = n + p;
```

E.8.3. Sentencias compuestas

Una *sentencia compuesta* es una serie de sentencias encerradas entre llaves. Las sentencias compuestas tienen el formato:

```
{
   sentencia
   sentencia
   sentencia
   ...
}
```

Las sentencias encerradas pueden ser cualquiera: declaraciones, expresiones, sentencias compuestas, etc. Un ejemplo es:

```
{
   int i = 5;
   double x = ...., y = ....;
   int j = 4 - i;
   x = 4.5 * (x - y);
}
```

El cuerpo de una función C++ es siempre una sentencia compuesta.

E.9. SENTENCIAS CONDICIONALES: *if*

El formato general de una sentencia `if` es:

```
if (expresion) if (expresion) {
  sentencia <secuencia de sentencias>
            }
```

Si *expresion* es verdadera (distinta de cero), entonces se ejecuta *sentencia* o *secuencia de sentencias;* en caso contrario, se salta la sentencia. Después que la sentencia `if` se ha ejecutado, el control pasa a la siguiente sentencia:

Ejemplo 1:

```
if (a < 0)
   negativos++;
```

Si la variable `a` es negativa, se incrementa la variable `negativos`.

Ejemplo 2:

```
if (numeroDeDias < 0)
   numeroDeDias = 0;

if ((altura - 5) < 4) {
   area = 3.14 * radio * radio;
   volumen = area * altura;
```

Ejemplo 3:

```
if (temperatura >= 45)
   cout << " Estoy en Sonora - Hermosillo, en Agosto";
cout << " Estoy en Veracruz " << temperatura <<endl;
```

La frase `"Estoy en Sonora-Hermosillo, en Agosto"` se visualiza cuando `temperatura` es mayor o igual que `45`. La sentencia siguiente se ejecuta siempre.

La sentencia `if_else` tiene el formato siguiente:

```
1. if (expresion)          2. if (expresion) {
      sentencia1                  <secuencia de sentencias1>
   else                       else
    sentencia2                   <secuencia de sentencias2>
```

Si *expresion* es distinto de cero, la *sentencia 1* se ejecuta y *sentencia2* se salta; si *expresion* es cero, la *sentencia1* se salta y *sentencia2* se ejecuta. Una vez que se ha ejecutado la sentencia `if_else`, el control pasa a la siguiente sentencia.

Ejemplo 4:

```
if (Numero ==0)
   cout << "No se calculara la media";
else
   media = total / Numero;
```

Ejemplo 5:

```
if (cantidad > 10) {
   descuento = 0.2;
   precio = n * precio (1 - descuento);
}
else {
   descuento = 0;
   precio = n * Precio;
}
```

E.9.1. Sentencias *if_else* anidadas

C++ permite anidar sentencias `if_else` para crear una sentencia de alternativa múltiple:

```
if (expresion1)
   sentencia1; | {sentencia compuesta}
else if (expresion2)
   sentencia2; | {sentencia compuesta}
...
else if (expresion N)
   sentencia N; | {sentencia compuesta}
[else
   sentencia N + 1; | {sentencia compuesta}]
```

Ejemplo:

```
if (a > 100)
if (b <= 0)
   SumaP = 1;
else
   SumaN = 1;
else
   Numero = 1;
```

E.9.2. Sentencias de alternativa múltiple: *switch*

La sentencia switch ofrece una forma de realizar decisiones de alternativas múltiples. El formato de switch es:

```
switch (expresion)
{
   case constante 1:
     sentencias
     break;
   case constante 2:
     sentencias
     .
     .
     .
     break;
   case constante n:
     sentencias
     break;
   default: //opcional
     sentencias
}
```

La sentencia switch requiere una expresión cuyo valor sea entero. Este valor puede ser una constante, una variable, una llamada a función o una expresión. El valor de *constante* ha de ser una constante. Al ejecutar la sentencia se evalúa *expresion* y si su valor coincide con una *constante* se ejecutan las sentencias a continuación de ella, en caso contrario se ejecutan las sentencias a continuación de default.

```
switch (Puntos)
{
   case 10:
    nota = 'A';
    break;
   case 9:
    nota = 'B';
    break;
   case 7,8:
    nota = 'C';
    break;
   case 5,6:
    nota = 'D';
    break;
   default:
   nota = 'F';
}
```

E.10. BUCLES: SENTENCIAS REPETITIVAS

Los *bucles* sirven para realizar tareas repetitivas. En C++, existen tres diferentes tipos de sentencias repetitivas:

- `while`
- `do`
- `for`

E.10.1. Sentencia *while*

La sentencia `while` es un bucle condicional que se repite mientras la condición es verdadera. El bucle `while` nunca puede iterar si la condición comprobada es inicialmente falsa. La sintaxis de la sentencia `while` es:

```
while (expresion)
   sentencia;
```

o bien:

```
while (expresion) {
  < secuencia de sentencias >
}
```

Ejemplo:

```
int n, suma = 0;
int i = 1;
while (i <= 100)
{
   cout << "Entrar";
   cin >> n;
   suma += n;
   i++;
}
cout << "La media es" << double (suma)/100.0;
```

E.10.2. Sentencia *do*

La sentencia `do` actúa como la sentencia `while`. La única diferencia real es que la evaluación y la prueba de salida del bucle se hace después que el cuerpo del bucle se ha ejecutado, en lugar de antes. El formato es:

```
do
   sentencias
while (expresion);
sentencia siguiente
```

Se ejecuta `sentencia` y a continuación se evalúa `expresion` y, si es verdadero (distinto de cero), el control se pasa de nuevo al principio de la sentencia `do` y el proceso se repite, hasta que `expresion` es falso (cero) y el control pasa a la `sentencia siguiente`.

Ejemplo:

```
int n, suma = 0;
int i = 1;
do
{
    cout <<"Entrar";
    cin >>n;
    suma += n;
    i++;
}while (i <= 100);
cout << "La media es" << double (suma)/100.0;
```

El siguiente ejemplo visualiza los cuadrados de 2 a 10:

```
int i = 2;
do
{
    cout << i << "12 = " << i * i++ << endl;
}while (i < 11);
```

E.10.3. La sentencia *for*

Una sentencia `for` ejecuta la iteración de un bucle un número determinado de veces. `for` tiene tres componentes: *expresion1*, inicializa las variables de control del bucle; *expresion2*, es la condición que determina si el bucle realiza otra iteración; la última parte del bucle `for` es la cláusula que incrementa o decrementa las variables de control del bucle. El formato general de `for` es:

```
for (expresion1; expresion2; expresion3)
    sentencia;|<secuencia de sentencias>;
```

expresion1 se utiliza para inicializar la variable de control de bucle; a continuación *expresion2* se evalúa, si es verdadera (distinta de cero), se ejecuta la sentencia y se evalúa *expresion3* y el control pasa de nuevo al principio del bucle. La iteración continúa hasta que *expresion2* es falsa (cero), en cuyo momento el control pasa a la sentencia siguiente al bucle.

Ejemplos:

```
1. for (int i = 0; i < N; i++)  //se relaciona N iteraciones
       sentencias1

2. Suma de 100 numeros
   int n, suma = 0;
   for (int i = 0; i < 100; i++)
   {
      cout << "Entrar";
      cin >> n;
      suma += n;
   }
```

E.10.4. Sentencias *break* y *continue*

El flujo de control ordinario de un bucle se puede romper o interrumpir mediante las sentencias `break` y `continue`.

La sentencia `break` produce una salida inmediata del bucle `for` en que se encuentra situada:

```
for (i = 0; i < 100; --i)
{
   cin >> x;
   if (x < 0.0) )
     cout << "salir del bucle" <<endl;
     break;
   }
   cout << sqrt(x) << endl;
}
```

La sentencia `break` también se utiliza para salir de la sentencia `switch`.

La sentencia `continue` termina la iteración que se está realizando y comenzará de nuevo la siguiente iteración:

```
for (i = 0; i < 100; --i)
   cin >> x;
   if (x < 0.0)
     continue;
}
```

> *Advertencia:*
>
> - Una sentencia `break` puede ocurrir únicamente en el cuerpo de una sentencia `for`, `while`, `do` o `switch`.
> - Una sentencia `continue` sólo puede ocurrir dentro del cuerpo de una sentencia `for`, `while` o `do`.

E.10.5. Sentencia nula

La `sentencia nula` se representa por un punto y coma, y no hace ninguna acción.

```
char cad[80]="Cazorla";
int i;

for (i = 0; cad[i] != '0'; i++)
     ;
```

E.10.6. Sentencia *return*

La *sentencia return* detiene la ejecución de la función actual y devuelve el control a la función llamada. Su sintaxis es:

```
return expresion;
```

donde el valor de *expresion* se devuelve como el valor de la función.

E.11. PUNTEROS (APUNTADORES)[2]

Un puntero o apuntador es una referencia indirecta a un objeto de un tipo especificado. En esencia, un puntero contiene la posición de memoria de un tipo dado.

E.11.1. Declaración de punteros

Los punteros se declaran utilizando el operador unitario *. En las sentencias siguientes se declaran dos variables: n es un entero, y p es un puntero a un entero.

```
int n;  //n es un tipo de dato entero
int *p; //p es un puntero a un entero
```

Una vez declarado un puntero, se puede fijar la dirección o posición de memoria del tipo al que apunta.

```
p = &n; //p se fija a la direccion de a
```

> Un puntero se declara escribiendo:
>
> *NombreTipo * Nombre Variable*

Una vez que se ha declarado un puntero, p, el objeto al que apunta se escribe *p y se puede tratar como cualquier otra variable de tipo NombreTipo.

```
int *p, *q, n;    //dos punteros a int, y un int
n = -25;          //n se fija a -16
*p = 105;         //*p a 101
*q = n + *p;      //*q a 80
```

C++ trata los punteros a tipos diferentes como tipos diferentes,

```
int *ip;
double *dp;
```

Los punteros ip y dp son incompatibles, de modo que es un error escribir

```
dp = ip;            //Error, no se pueden asignar punteros a tipos diferentes
```

Se pueden, sin embargo, realizar asignaciones entre contenidos, ya que se realizaría una conversión explícita de tipos.

```
*dp = *ip;
```

[2] El termino *puntero* es el más utilizado en España, mientras que en Latinoamérica se suele utilizar *apuntador*.

Existe un puntero especial (*nulo*) que se suele utilizar con frecuencia en programas C++. El puntero NULL tiene un valor cero, que lo diferencia de todas las direcciones válidas. El conocimiento nos permite comprobar si un puntero p es el puntero NULL evaluando la expresión (p==0). Los punteros NULL se utilizan sólo como señales de que ha sucedido algo. En otras palabras, si p es un puntero NULL, es correcto referenciar *p.

E.11.2. Punteros a arrays

A los arrays se accede a través de los índices:

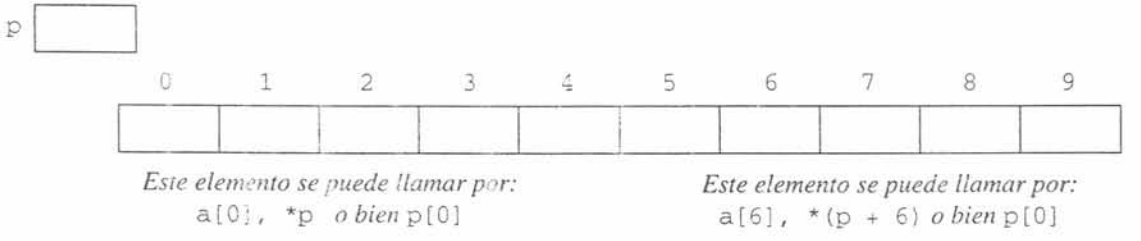
```
int lista [5];
lista [3] = 5;
```

A los arrays también se puede acceder a través de punteros:

```
int lista [5]; //array de 5 elementos
int *ptr;      //puntero a entero
ptr = lista;   //fija puntero al primer elemento del array
ptr += 3;      //suma 3 a ptr; ptr apunta al 4° elemento
*ptr = 5;      //establece el 4° elemento a 5.
```

> El nombre de un array se puede utilizar también como si fuera un puntero al primer elemento del array.

```
double a [10];
double *p = a;    //p y a se refieren al mismo array
```



Si nombre apunta al primer elemento del array, entonces nombre + 1 apunta al segundo elemento. El contenido de lo que se almacena en esa posición se obtiene por la expresión:

```
*(nombre + 1)
```

> Aunque las funciones no pueden modificar sus argumentos, si un array se utiliza como un argumento de una función, la función puede modificar el contenido del array.

E.11.3. Punteros a estructuras

Los punteros a estructuras son similares y funcionan de igual forma que los punteros a cualquier otro tipo de dato.

```
struct familia
{
   char *marido;
   char *esposa;
   char *hijo;
};

familia Mackoy;     //Mackoy estructura de tipo familia
familia *p;         //p, un puntero a familia
p = &Mackoy         //p, contiene direccion de mackoy

p marido = "Luis Mackoy";     //inicializacion
p esposa = "Vilma Gonzalez"; //inicializacion
p hijo = "Luisito Mackoy";    //inicializacion
```

E.11.4. Punteros a objetos constantes

Cuando se pasa un puntero a un objeto grande, pero se trata de que la función no modifique el objeto (por ejemplo, el caso de que sólo se desea visualizar el contenido de un array), se declara el argumento correspondiente de la función como puntero a un objeto constante.

La declaración:

```
const NombreTipo *v
```

establece v como un puntero a un objeto que no puede ser modificado. Un ejemplo puede ser:

```
void Visualizar(const ObjetoGrande *v);
```

E.11.5. Punteros a *void*

El tipo de dato `void` representa un valor nulo. En C++, sin embargo, el tipo de puntero `void` se suele considerar como un puntero a cualquier tipo de dato. La idea fundamental que subyace en el puntero `void` en C++ es la de un tipo que se puede utilizar adecuadamente para acceder a cualquier tipo de objeto, ya que es más o menos independiente del tipo.

Un ejemplo ilustrativo de la diferencia de comportamiento en C y C++ es el siguiente segmento de programa:

```
int main()
{
   void *vptr;
   int *iptr;

   vptr = iptr;
   iptr = vptr;                //Incorrecto en C++, correcto en C
   iptr = (int *) vprr;        //Correcto en C++
   ...
}
```

E.11.6. Punteros y cadenas

Las cadenas en C++ se implementan como arrays de caracteres, como constantes de cadena y como punteros a caracteres.

Constantes de cadena

Su declaración es similar a

```
char *Cadena = "Mi profesor";
```

o bien su sentencia equivalente

```
char VarCadena[] = "Mi profesor";
```

Si desea evitar que la cadena se modifique, añada `const` a la declaración:

```
const char *VarCadena = "Mi profesor";
```

> Los puntos a cadena se declaran:
>
> `char s []` *o bien:* `char *s`

Punteros a cadenas

Los punteros a cadenas no son cadenas. Los punteros que localizan el primer elemento de una cadena almacenada.

```
char *varCadena;
const char *Cadenafija;
```

Consideraciones prácticas

Todos los arrays en C++ se implementan mediante punteros:

```
char cadena1[16]= "Concepto Objeto";
char * cadena2 = cadena1;
```

Las declaraciones siguientes son equivalentes y se refieren al carácter `'C'`:

```
cadena1[0]  cadena1  cadena2
```

E.11.7. Aritmética de punteros

Dado que los punteros son números (direcciones), puden ser manipulados por los operadores aritméticos. Las operaciones que están permitidas sobre punteros son: suma, resta y comparación. Así, si las sentencias siguientes se ejecutan en secuencia:

```
char *p;     //p, contiene la direccion de un caracter
char a[10]; //array de diez caracteres
p = &a[0];  //p, apunta al primer elemento del array
p++;        //p, apunta al segundo elemento del array
p++;        //p, apunta al tercer elemento del array
p--;        //p, apunta al segundo elemento del array
```

Un elemento de comparación de punteros, es el siguiente programa:

```
#include <iostream.h>
main (void)
{
  int *ptr1, *ptr2;
  int a[2] = {10,10};
  ptr1 =a;
  cout << "ptr1 es" << "ptr1<< "*ptr1 es" << *ptr1 <<endl;
  ptr2 = ptr1 + 1;
  cout << "ptr2 es" << ptr2 << "*ptr2 es" << *ptr2 <<endl;

  //comparar dos punteros
  if (ptr1 == ptr2)
      cout << " ptr1 no es igual a ptr2 \n";

  if (*ptr1 == *ptr2)
      cout <<"*ptr1 es igual a *ptr2 \n";
  else
      cout <<"*ptr1 no es igual a *ptr2 \n";
}
```

E.12. LOS OPERADORES *NEW* Y *DELETE*

C++ define un método para realizar asignación dinámica de memoria, diferente del utilizado en C, mediante los operadores **new** y **delete**.

El operador new sustituye a la función malloc tradicional en C, y el operador delete sustituye a la función free tradicional, también en C. new, asigna memoria, y devuelve un puntero al objeto últimamente creado; su sintaxis es:

```
new NombreTipo
```

y un ejemplo de su aplicación es:

```
int *ptr1
double *ptr2;
ptr1 = new int;     //memoria asignada para el objeto ptr1
ptr2 = new double; //memoria ampliada para el objeto ptr2
*ptr1 = 5;
*ptr2 = 6.55;
```

Dado que new devueve un puntero, se puede utilizar ese puntero para inicializar el puntero de una sola definición, tal como:

```
int* p = new int;
```

Si new no puede ocupar la cantidad de memoria solicitada devuelve un valor NULL. El operador delete libera la memoria asignada mediante new.

```
delete ptr1;
```

Un pequeño programa que muestra el uso combinado de new y delete es:

```
#include <iostream.h>
void main (void)
{
   char *c;

   c = new char[512];
   cin >> c;
   cout << c << endl;

   delete c;
}
```

Los operadores new y delete se pueden utilizar para asignar memoria a arrays, clases y otro tipo de datos.

```
int *i;
i = new int[2][35];     //crear el array
//asignar el array
...
delete i;               //destruir el array
```

Sintaxis de new y delete

```
new nombre-tipo                          new Int       new char[100]
new nombre-tipo inicializador            new int(99)   new char('C')
new nombre-tipo                          new (char*)
delete expresion                         delete p
delete[] expresion                       delete []p
```

E.13. ARRAYS

Un *array*[3] (*matriz, tabla*) es una colección de elementos dados del mismo tipo que se identifican por medio de un índice. Los elementos comienzan con el índice 0.

Declaración de arrays

Un declaración de un array tiene el siguiente formato:

```
nombreTipo   nombreVariable[n]
```

Algunos ejemplos de arrays unidimensionales:

```
int ListaNum[2];        //array de dos enteros
char ListaNombres[10]; //array de 10 caracteres
```

3 En Latinoamérica, este término se traduce al español por la palabra *arreglo*.

Arrays *multidimensionales* son:

```
nombretipo      nombreVariable[n1][n2]...[nx];
```

El siguiente ejemplo declara un array de enteros $4 \times 10 \times 3$

```
int multidim [4][10][3];
```

El ejemplo tabla declara un array de 2×3 elementos

```
int tabla [2][3]; //array de enteros de 2 ×3 = 6 elementos
```

E.13.1. Definición de arrays

Los arrays se inicializan con este formato:

```
int a [3]         = {5, 10, 15};
char cad[5]       = {'a', 'b', 'c', 'd', 'e'};
int tabla [2][3] = {{1,2,3} {3,4,5}};
```

Las tres siguientes definiciones son equivalentes:

```
char saludo [5] = "hola";
char saludo []  = "hola";
char saludo [5] = {'h', 'o', 'l', 'a', '\0'};
```

1. Los arrays se pueden pasar como argumentos a funciones.
2. Las funciones no pueden devolver arrays.
3. No está permitida la asignación entre arrays. Para asignar un array a otro, se debe escribir el código para realizar las asignaciones elemento a elemento.

E.14. ENUMERACIONES, ESTRUCTURAS Y UNIONES

En C++, un nombre de una enumeración, estructura o unión es un nombre de un tipo. Por consiguiente, la palabra reservada `struct`, `union`, o `enum` no son necesarias cuando se declara una variable.

El tipo de dato *enumerado* designa un grupo de constantes enteros con nombres. La palabra reservada `enum` se utiliza para declarar un tipo de dato enumerado o *enumeración*. La sintaxis es:

```
enum nombre
{
   lista-simbolos
}
```

donde *nombre* es el nombre de la variable declarada enumerada, *lista-simbolos* es una lista de tipos enumerados, a los que se asigna valores cuando se declara la variable enumerada y puede tener un valor de inicialización. Se puede utilizar el nombre de una enumeración para declarar una variable de ese tipo (variable de enumeración).

```
nombre var;
```

Considérese la siguiente sentencia:

```
enum color {Rojo, Azul, Verde, Amarillo};
```

Una variable de tipo enumeración color es:

```
color pantalle = Rojo; //Estilo C++
```

Una *estructura* es un tipo de dato compuesto que contiene una colección de elementos de tipos de datos diferentes combinados en una única construcción del lenguaje. Cada elemento de la colección se llama `miembro` y puede ser una variable de un tipo de dato diferente. Una estructura representa un nuevo tipo de dato en C++.

La sintaxis de una estructura es:

```
struct nombre
{
   miembros
};
```

Un ejemplo y una variable tipo estructura se muestran en las siguientes sentencias:

```
struct cuadro {
  init i;
  float f;
};

struct cuadro nombre; //Estilo C
cuadro nombre;        //Estilo C++
```

Una *unión* es una variable que puede almacenar objetos de tipos y tamaños diferentes. Una unión puede almacenar tipos de datos diferentes, sólo puede almacenar uno cada vez, en oposición a una estructura que almacena simultáneamente una colección de tipos de datos. La sintaxis de una unión es:

```
union nombre {
  miembros
};
```

Un ejemplo de una unión es:

```
union alfa {
  int x;
  char c;
};
```

Una declaración de una variable estructura es:

```
alfa w;
```

El modo de acceder a los miembros de la estructura es mediante el operador punto.

```
u.x = 145;
u.c = 'z';
```

C++ admite un tipo especial de unión llamada *unión anónima*, que declara un conjunto de miembros que comparten la misma dirección de memoria. La unión anónima no tiene asignado un nombre y, en consecuencia, se accede a los elementos de la unión directamente.

La sintaxis de una unión anónima es:

```
union {
  int nuevoID;
  int contador;
};
```

Las variables de la unión anónima comparten la misma posición de memoria y espacio de datos.

```
int main()
{
  union {
    int x;
    float y;
    double z;
  };
  x = 25;
  y = 245.245;     //el valor en y sobreescribe el valor de x

  z = 9.41415;     //el valor en z sobreescribe el valor de z
}
```

E.15. CADENAS

Una *cadena* es una serie de caracteres almacenados en bytes consecutivos de memoria. Una cadena se puede almacenar en un array de caracteres (`char`) que termina en un carácter nulo (cero, `'\0'`).

```
char perro[5] = {'m','o','r','g','a','n'}; // no es una cadena
char gato[5] = {'f','e','l','i','s','\o'}; // es una cadena
```

Lectura de una cadena del teclado

```
#include <isostream.h>
main()
{
   char cad [80];
   cout <<"Introduzca una cadena:";     // lectura del teclado
   cin >> cad;
   cout << "Su cadena es:";
   cout << cad;

   return 0;
}
```

Esta lectora del teclado lee una cadena hasta que se encuentra el primer carácter blanco. Así, cuando se lee "Sierra Magina. Jaen" la primera cadena, en cad sólo se almacena Sierra. Para resolver el problema, utilizará la función gets () que lee una cadena completa leída del teclado. El programa anterior se lee así:

```
#include <iostream.h>
#include <stdio.h>

main()
{
   char cad[80];

   cout << "Introduzca una cadena:";
   gets(cad);
   cout << "Su cadena es:";
   cout << cad;

   return 0;
}
```

E.16. FUNCIONES

Una *función* es una colección de declaraciones y sentencias que realizan una tarea única. Cada función tiene cuatro componentes: (1) su nombre, (2) el tipo de valor que devuelve cuando termina su tarea, (3) la información que toma al realizar su tarea, y (4) la sentencia o sentencias que realizan su tarea. Cada programa C++ tiene al menos una función: la función main.

E.16.1. Declaración de funciones

En C++, se debe declarar una función antes de utilizarla. La declaración de la función indica al compilador el tipo de valor que devuelve la función y el número y tipo de argumentos que toma. La declaración en C++ se denomina *prototipo*:

```
tipo   NombreFuncion (lista argumentos);
```

Ejemplos válidos son:

```
double Media(double x, double y);
void print(char* formato,...);
extern max(const int*, int);
char LeerCaracter();
```

E.16.2. Definición de funciones

La definición de una función es el cuerpo de la función que se ha declarado con anterioridad.

```
double Media(double x, double y)
//Devuelve la media de x e y
{
   return (x + y)/2.0
}
```

```
char LeerCaracter();
//Devuelve un caracter de la entrada estandar
{
   char c;
   cin >> c;
   return c;
}
```

E.16.3. Argumentos por omisión

Los parámetros formales de una función pueden tomar valores por omisión, o argumentos cuyos valores se inicializan en la lista de argumentos formales de la función.

```
int Potencia (int n, int k = 2);
Potencia(256);                    //256 elevado al cuadrado
```

Los parámetros por omisión no necesitan especificarse cuando se llama a la función. Los parámetros con valores por omisión deben estar al final de la lista de parámetros:

```
void func1 (int i =3, int j);              //ilegal
void func2 (int i, int j = o, int k = 0);  //correcto
void func3 (int i = 2, int j, int k = 0);  //ilegal
void ImprimirValores (int cuenta, double cantidad = 0.0);
```

Llamadas a la función `ImprimirValores`:

```
ImprimirValores (n,a);
ImprimirValores (n);    //equivalente a ImprimirValores (n, 0.0)
```

Otras declaraciones y llamadas a funciones son:

```
double f1(int n, int m, int p=0);           //legal
double f2(int n, int m = 1, int p = 0);     //legal
double f3(int n = 2, omt m = 1, int p = 0); //legal
double f4(int n, int m = 1, int p);         //ilegal
double f5(int n = 2, int m, int p = 0);     //ilegal
```

E.16.4. Funciones en línea (*inline*)

Si una declaración de función está precedida por la palabra reservada `inline`, el compilador sustituye cada llamada a la función con el código que implementa la función.

```
inline int Max(in a, int b)
{
   if (a > b) return a;
      return b;
}

main()
{
   int x = 5, y = 4;
   int z = Max(x,y);
}
```

Los parámetros con valores por omisión deben entrar al final de la lista de parámetros: Las funciones `f1`, `f2` y `f3` son válidas, mientras que las funciones `f4` y `f5` no son válidas.

Las funciones *en línea* (*inline*) evitan los tiempos suplementarios de las llamadas múltiples a funciones. Las funciones declaradas en línea deben ser simples con sólo unas pocas sentencias de programa; sólo se pueden llamar un número limitado de veces y no son recursivas.

```
inline int abs(int i);
inline int min(int v1, int v2);
int mcd(int v1, int v2);
```

Las funciones en línea deben ser definidas antes de ser llamadas.

```
#include <isostream.h>
int incrementar (int i);

inline incrementar (int i)
{
   i++
   return;
}
main(void)
{
   int i = 0;
   while (i < 5)
   {
       i = incrementar (i);
      cout << "i es: " << i <<endl;
   }
}
```

E.16.5. Sobrecarga de funciones

En C++, dos o más funciones distintas pueden tener el mismo nombre. Esta propiedad se denomina *sobrecarga*. Un ejemplo es el siguiente:

```
int max (int, int);
double max (double, double);
```

o bien este otro:

```
void sumar (char i);
void sumar (float J);
```

Las funciones sobrecargadas se diferencian en el número y tipo de argumentos, o en el tipo que devuelven las funciones, y sus cuerpos son diferentes en cada una de ellas.

```
#include <iostream.h>

void suma (char);
void suma (float);
main (void)
{
   int i = 65;
   int j = 6.5;
```

```
    suma(i);
    suma(j);
}

void suma(char i)
{
    cout << "Suma interior(char)" <<endl;
}

void suma(float j)
{
    cout << Suma interior (float) " << endl;
}
```

E.16.6. El modificador *const*

El modificador de tipos `const` se utiliza en C++ para proporcionar protección de sólo lectura para variables y parámetros de funciones. Cuando se hace preceder un tipo de argumento con el modificador `const` para indicar que este argumento no se puede cambiar, el argumento al que se aplica no se puede asignar un valor ni cambiar.

```
void copia (const char * fuente, char* dest);
void func_demo (const int i);
```

E.16.7. Paso de parámetros a funciones

En C++ existen tres formas de pasar parámetros a funciones:

1. *Por valor.* La función llamada recibe una copia del parámetro y este parámetro no se puede modificar dentro de la función

```
void intercambio (intx, int y)
{
   int aux = y;
         y = x;
         x = aux;
}
//...
intercambio (i, j); //las variables i, j, no se intercambian
```

2. *Por dirección.* Se pasa un puntero al parámetro. Este método permite simular en C/C++ la llamada por referencia, utilizando tipos punteros en los parámetros formales en la declaración de prototipos. Este método permite modificar los argumentos de una función.

```
void intercambio (int*x, int*y)
{
   int aux = *y;
        *y = *x;
        *x = aux;
}
//...
intercambio (&i, &j); //i, j se intercambian sus valores
```

3. *Por referencia.* Se pueden pasar tipos referencia como argumentos de funciones, lo que permite modifi car los argumentos de una función.

```
void intercambio (int&x, int&y);
{
  int aux = y;
      y = x;
      x = aux;
}
//...
intercambio (i, j);     //i, j intercambian sus valores
```

> Si se necesita modificar un argumento de una función en su interior, el argumento debe ser un tipo de referencia en la declaración de la función.

E.16.8. Paso de arrays

Los arrays se pasan por referencia. La dirección del primer elemento del array se pasa a la función; los elementos individuales se pasan por valor. Los arrays se pueden pasar indirectamente por su valor si el array se define como un miembro de una estructura.

```
//Paso del array completo. Ejemplo 1
#include <iostream.h>
void func1 (int x []);      //prototipo de funcion

void main() {
   int a[3] = {1,2,3};
   func1(a);               //sentencias
   func1(&a[0];            //equivalentes
}

void func(int x[]) {
   int i;
   for (i = 0; i < 3; i = 1)
     cout << i << x[i] << '\n';
}
```

El siguiente ejemplo pasa un elemento de un array:

```
#include <iostream.h>

{
const int N = 3;
void func2(int x);
void main() {
   int a[N] = {1,2,3};
   func2(a[2]);
}

void func2(int x) {
   cout << x << '\n';
}
```

APÉNDICE F

CODIFICACIÓN DE ALGORITMOS EN LENGUAJES DE PROGRAMACIÓN

F.1. CONVERSIÓN DE ALGORITMOS

Para convertir un algoritmo escrito en lenguaje natural en alguna de las herramientas de programación *diagramas de flujo, diagramas N-S* y *pseudocódigo,* podrá utilizar las tablas siguientes, donde se consideran las correspondientes equivalencias. Para su correcta conversión, deberá buscar en la columna de instrucciones el correspondiente elemento del algoritmo y en las siguientes columnas sus equivalentes en herramientas de programación.

Tabla F.1. Conversión de algoritmos en herramientas de programación

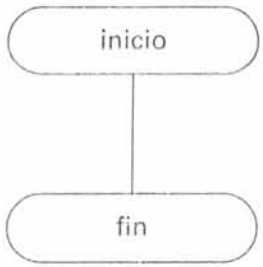
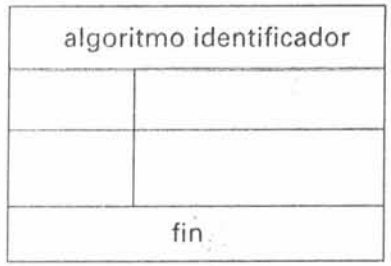
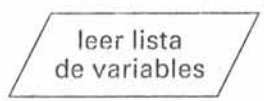
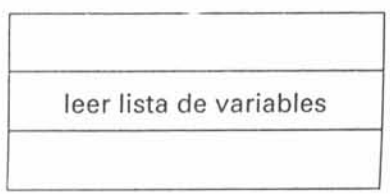
Instrucciones (acciones)	Pseudocódigo	Diagrama de flujo	Diagrama N-S
Estructura del algoritmo	**algoritmo** identificador [sección de declaración de variables y constantes] **inicio** . . . **fin**		
Declaración de variables	*Modelo1* (dentro del algoritmo en la sección de declaración). **var** nombre11, nombre12, ...: tipo 1; nombre21, nombre22...: tipo2 *Modelo 2*. Descripción de variables y tipos en la tabla de variables	Se escriben las variables y tipos en la *tabla de variables* junto con el diagrama de flujo	Escritura de una tabla de variables
Asignación	V ← E V = variable E = expresión (constante, variable o expresión)	V ← E / V = E	V ← E
Entrada de datos	**leer** lista de variables		

Tabla F.1. Conversión de algoritmos en herramientas de programación. *(Cont.)*

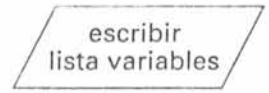
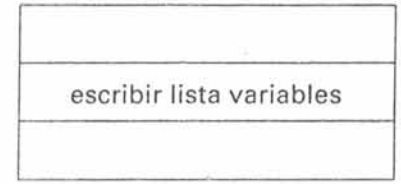
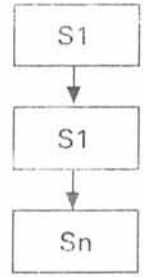
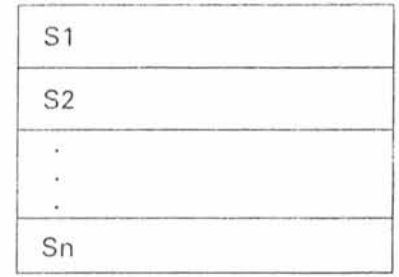
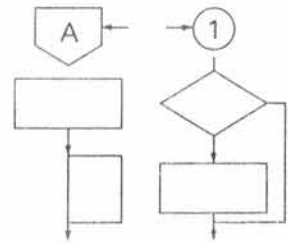
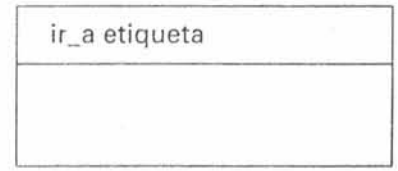
Instrucciones (acciones)	Pseudocódigo	Diagrama de flujo	Diagrama N-S
Salida de datos	escribir lista de expresiones		
Instrucción compuesta	S1 S2 S3 . . . Sn		
Comentarios	1. {fase descriptiva del comentario} 2. {fase descriptiva del comentario}		
Bifurcación incondicional	ir_a etiqueta		

Tabla F.1. Conversión de algoritmos en herramientas de programación *(Cont.)*

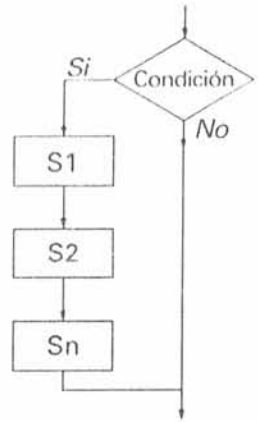
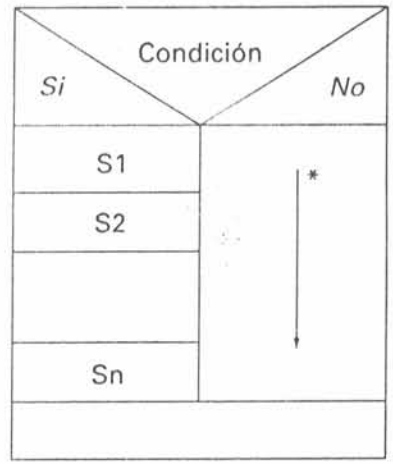
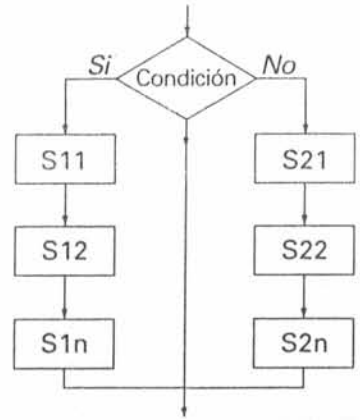
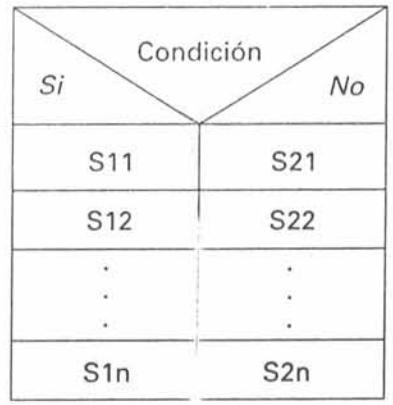
Instrucciones (acciones)	Pseudocódigo	Diagrama de flujo	Diagrama N-S
Selectiva alternativa simple	*Modelo 1:* si *condicion* entonces *S1* *S2* . . *Sn* fin_si *Modelo 2:* si *condicion* entonces *S1* *S2* . . *Sn* fin_si *Modelo 3:* si *condicion* entonces S1		 * Flecha o Blanco
Selectiva alternativa doble	*Modelo 1:* si *condicion* entonces *S11* *S12* . . *S1n* sino *S21* *S22* . . *S2n* fin_si *Modelo 2:* si *condicion* entonces S1 sino S2 fin_si		

Tabla F.1. Conversión de algoritmos en herramientas de programación, *(Cont.)*

Instrucciones (acciones)	Pseudocódigo	Diagrama de flujo	Diagrama N-S

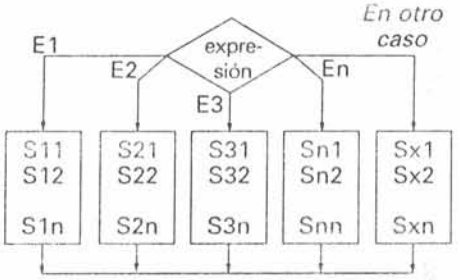
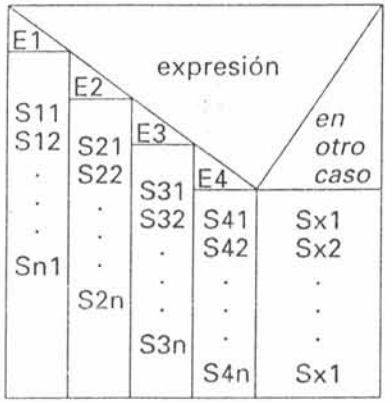
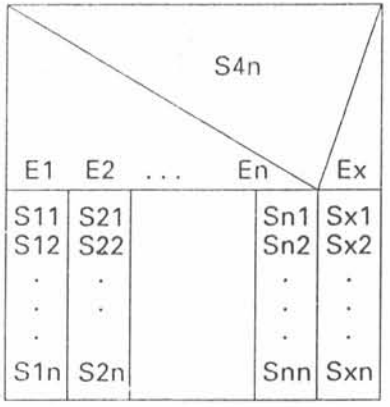
Selectiva alternativa múltiple	`segun_sea expresion hacer`
	E1:S11
	S12
	.
	.
	.
	S1n
	E2:S21
	22
	.
	.
	.
	S2n
	En:Sn1
	Sn2
	.
	.
	S1n
	`en_otro_caso`
	Sx1
	Sx2
	.
	.
	Sxn
	`fin_segun`
			*Modelo 1:*
			 
			Modelo 2:
			 

Tabla F.1. Conversión de algoritmos en herramientas de progra...

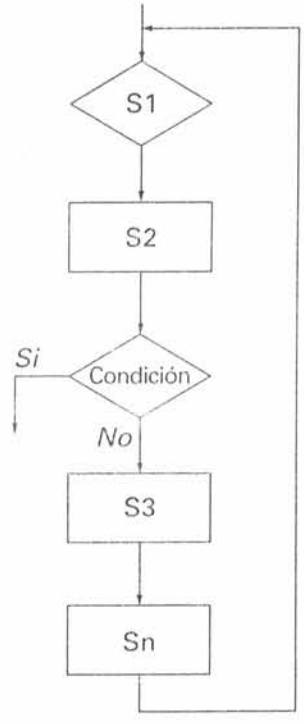
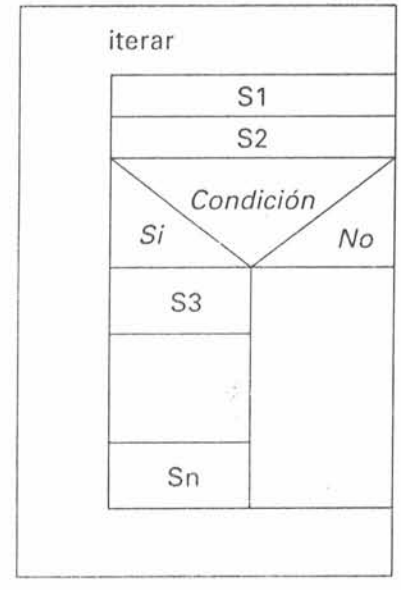
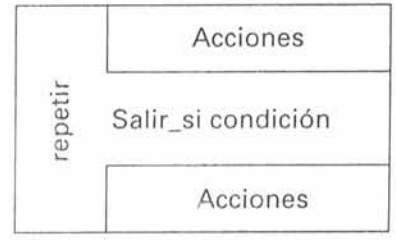
Instrucciones (acciones)	Pseudocódigo	Diagrama de flujo	Diagrama N-S
repetitiva *iterar*	iterar *S1* *S2* . . salir_si *condicion* . . *S3* . . *Sn* fin-iterar		*Modelo 1:*  *Modelo 2:* 

Tabla F.1. Conversión de algoritmos en herramientas de programación. *(Cont.)*

Instrucciones (acciones)	Pseudocódigo	Diagrama de flujo	Diagrama N-S

Tabla F.1. Conversión de algoritmos en herramientas de programación. *(Cont.)*

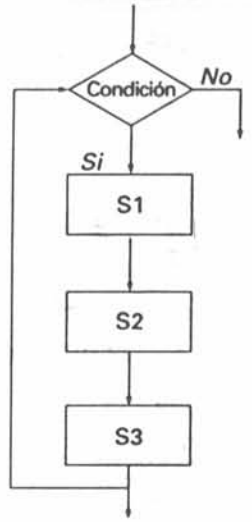
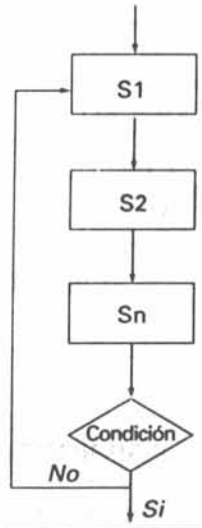
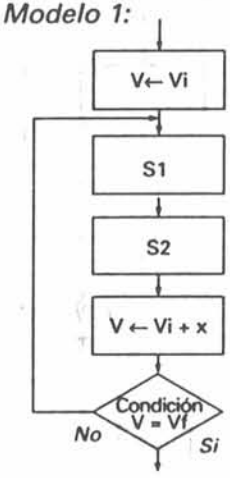
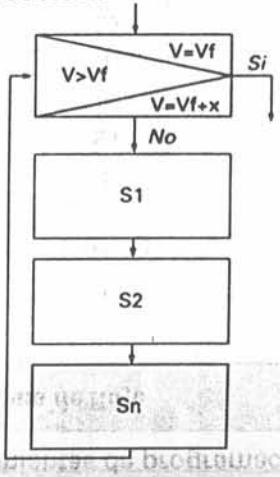
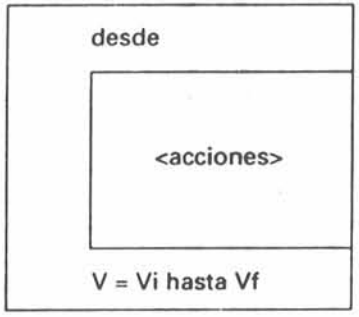
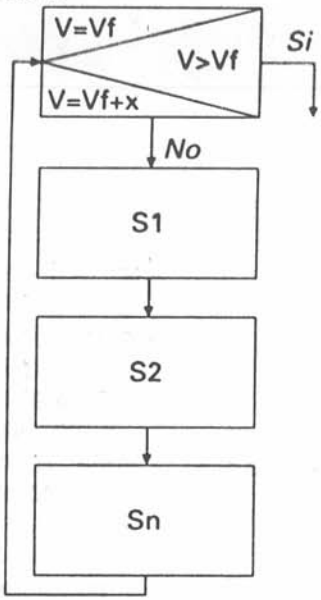
Instrucciones (acciones)	Pseudocódigo	Diagrama de flujo	Diagrama N-S
repetitiva *mientras*	***mientras*** *condicion* *S1* *S2* . . *Sn* **fin_mientras**		
repetitiva *repetir*	**repetir** S1 S2 . . . Sn **hasta_que** *condicion*	*Modelo 1:* 	

Tabla F.1. Conversión de algoritmos en herramientas de programación. *(Cont.)*

Instrucciones (acciones)	Pseudocódigo	Diagrama de flujo	Diagrama N-S

`repetitivas` *desde*	*Modelo 1:*
	`desde` V=Vi `hasta` Vf [*incremento* X] `hacer`
	S1
	S2
	.
	.
	Sn
	`fin_desde`
	
	Modelo 2:
	`para` *V* `de` *Vi* `a` *Vf* `hacer`
	S1
	S2
	.
	.
	Sn
	`fin_para`
		Modelo 1:
		 
		
		Modelo 2:
		 
		
			

Modelo 3:



Modelo 4:

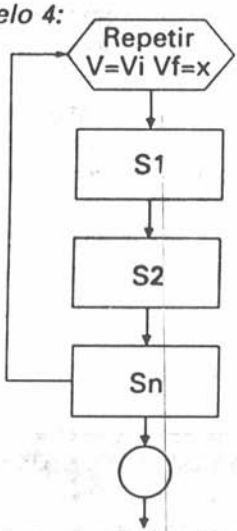


Tabla F.2. Conversión de pseudocódigos a programas

Pseudocódigo	Pascal	FORTRAN	Modula-2
Estructura del algoritmo			
algoritmo *identificador*	**program** identificador (input-output)	**PROGRAM** identificador (opcional)	**MODULE** nombremódulo;
[sección de declaraciones]	**const** nombre = valor;	(declaración de variables y constantes)	<lista de importar> <declaración de constantes>
inicio	**type** nombre = tipo de datos;	**IMPLICIT**	<declaración de tipo>
S1	**var** nombre,	**CHARACTER**	<declaración de
S2	nombre,…:tipo;	**PARAMETER**	variables>
.	**procedure/function**	(declaración de arrays)	
.	[definición del	**DIMENSION**	
Sn	subprograma]	(declaración de funciones)	<declaraciones de procedimientos>
		COMMON	**BEGIN**
fin	**begin**	**EQUIVALENCE**	<sentencias>
	S1	**EXTERNAL**	
	S2	**INTRINSIC**	
	.	(inicialización de variables)	
	.	**DATA**	
	.	(definición de funciones de sentencia)	
	Sn	Instrucciones ejecutables	
	.	.	
	.	.	
	.	.	
	Sn	**END**	
	end		**END** nombremódulo

Tabla F.2. Conversión de pseudocódigos a programas.*(Cont.)*

Pseudocódigo	Pascal	FORTRAN	Modula-2
Declaración de variables			
var lista de nombres1: tipo 1 lista de nombres2: tipo 2	**var** lista nombres1: tipo1 lista nombres2: tipo2	*tipo* lista de variables *tipo* es **INTEGER** **REAL** **DOUBLE PRECISION** **LOGICAL** **COMPLEX** **CHARACTER** * n **(n, longitud de la cadena)**	**VAR** <lista de variables>: tipo *tipo* es **INTEGER** **REAL** **BOOLEAN** **CARDINAL** **BITSET** **CHAR**
Asignación			
V ← E	V: = E	V = E	V: = E
Lectura de datos			
leer lista de variables	**read** (lista de variables) **readln** (lista de variables)	**READ** (n, f) lista de variables n número unidad entrada *f etiqueta sentencia FORMAT* **READ** (*,*) lista de variables **READ** f, lista	Read (c) ReadString (s) ReadInt (i) ReadReal (x) c: tipo CHAR i = integer s = tipo cadena x = tipo real

Tabla F.2. Conversión de pseudocódigos a programas. *(Cont.)*

Pseudocódigo	Pascal	FORTRAN	Modula-2
Escritura de datos **escribir** lista de expresiones	**write** (lista de expresiones) **writeln** (lista de expresiones)	**WRITE** (n ,f) lista de expresiones **WRITE** (*,*) lista de expresiones **WRITE** f, lista de expresiones **PRINT** f, lista de expresiones	Write (c) WriteString (s) WriteInt (i, n) WriteReal (x, n) WriteLn
Comentarios	1. {mensaje} 2. (* mensaje *)	1. C esto es un comentario 2. * esto es un comentario *Nota*: C o * se escriben en la primera columna	(* comentarios *)

Selectivas
Alternativa simple
 	
si *condicion*
entonces
S1
S2
.
.
Sn
fin_si
	1. **if** *condicion*
	then begin
	S1
	S2
	.
	.
	Sn
	end
	2. **if** *condicion* **then**
	begin
	S1
	S2
	.
	.
	Sn
	end
	3. **if** *condicion* **then**
	si **else** S2
		1. **IF** *logico*
		IF (*condicion*) S
		IF (*condicion*) **GOTO** etiqueta
		
		2. **IF** de bloques
		IF *condicion* **THEN**
		S1
		S2
		.
		.
		Sn
		END IF
			IF *condicion* **THEN**
			S1
			S2
			.
			.
			Sn
			END

Tabla F.2. Conversión de pseudocódigos a programas *(Cont.)*

Pseudocódigo	Pascal	FORTRAN	Modula-2
Alternativa doble			
	1. **if** *condicion* **then**	**IF** *condicion* **THEN**	**IF** *condicion* **THEN**
	S11	S11	S11
si *condicion*	S12	S12	S12
entonces	.	.	.
S11	.	.	.
S12	S1n	S1n	S1n
sino	**else**	**ELSE**	**ELSE**
S21	**begin**	S21	S21
S22	S21	S22	S22
fin_si	S22	.	.
	.	.	.
	.	S2n	S2n
	S2n	**ENDIF**	**END**
	end		
	2. **if** *condicion* **then**		
	begin		
	S11		
	S12		
	.		
	.		
	S1n		
	end		
	else		
	begin		
	S12		
	S22		
	.		
	.		
	S2n		
	end		

Tabla F.2. Conversión de pseudocódigos a programas.*(Cont.)*

Pseudocódigo	Pascal	FORTRAN	Modula-2
repetitiva (mientras) **mientras** S1 S2 . . Sn **fin_mientras**	1. **while** *condicion* **do** **begin** S1 S2 . . Sn **end** 2. **while** *condicion* **do** S1		**WHILE** *condicion* **DO** S1 S2 . . Sn **END**
repetitiva (repetir) **repetir** S1 S2 . . Sn **fin_repetir**	**repeat** S1 S2 . . Sn **until** *condicion*		**REPEAT** S1 S2 . . Sn **UNTIL** *condicion*
repetitiva (desde para) **desde** V = Vi **hasta** Vf [**incremento** x] **hacer** ... **fin_desde**	1. **for** v: = vi **to** vf **do** S1 2. **for** v: = vi **downto** Vf **do** S1 3. **for** v: = vi **to** Vf **do** **begin** S1 S2 . . Sn **end** 4. **for** v: = vi **downto** Vf **do** **begin** S1 S2 . . Sn **end**	**DO** n i = m1, m2, m3 . . . n **CONTINUE** *n* etiqueta de línea m1 valor inicial m2 valor final m3 incremento	**FOR V:** = Vi **TO** Vf [**BY** x] **DO** S1 S2 . . Sn **end**

Notas: 1 y 3, incrementos positivos; 2 y 4, incrementos negativos

Tabla F.2. Conversión de pseudocódigos a programas. *(Cont.)*

Pseudocódigo	Pascal	FORTRAN	Modula-2
repetitiva iterar			
iterar			**LOOP**
S11		S11	S1
S12		S12	S2
.		.	.
.		.	.
Sn		S1n	
salir si *condicion*	**while not** *condicion* **do**		**UNTIL** *condicion*
S21	**begin**		**LOOP**
S22		S21	S11
.		S22	S12
.		.	.
S2n		.	.
fin_iterar		S2n	**EXIT**
		S11	S21
		S12	S22
		.	**END**
		.	
		end	

F.2. PRÁCTICAS DE PROGRAMACIÓN

Ejemplo F.1. Calcular el volumen de un cilindro círculo

Análisis

La fórmula del volumen del cilindro es $V = \pi \cdot R^2 \cdot H$
Variables de entrada: RADIO, ALTURA de tipo real
Variables de salida: VOLUMEN de tipo real
Constantes: π = PI = 3.141592

Algoritmo

Los pasos generales del algoritmo son:

```
inicio
  determinar los valores del radio R y altura H
  el volumen se obtiene con la expresion V = p * R² * H
  calcular y escribir el valor de V
fin
```

Pseudocódigo

```
algoritmo volumen
{declaracion de variables y constantes}
var RADIO, ALTURA, VOLUMEN = real
constante PI = 3.141592
inicio
  {lectura de datos}
  leer RADIO, ALTURA
  {proceso}
  VOLUMEN → PI * RADIO * RADIO * ALTURA
  {salida de datos}
  escribir VOLUMEN
fin
```

Codificación en **Visual BASIC**

```
100 'programa volumen
110 'calculo del volumen de un cilindro
120 '
130 'declaracion de variables
140 DEFSNG RADIO, ALTURA, VOLUMEN
150 'inicio de las sentencias ejecutables
160 'entrada de datos
```

```
170 PI = 3.141592
180 INPUT RADIO, ALTURA
190 VOLUMEN = PI * RADIO ^ 2 * ALTURA
200 'salida de datos
210 PRINT 'el volumen es ', VOLUMEN
220 END
```

Codificación en **FORTRAN**

```
      PROGRAM cilindro
C     Calcular el volumen de un cilindro
      INTEGER SALIDA
      DATA ENTRADA, SALIDA/5, 6/
      DATA PI /3.141592/
      READ (ENTRADA, 100) RADIO, ALTURA
      VOLUMEN = PI * RADIO * RADIO * ALTURA
      WRITE (SALIDA, 200) VOLUMEN
      STOP
100   FORMAT (2 F8.0)
200   FORMAT ('el volumen es ', F8.2)
        END
```

NOTA: Los dispositivos físicos del teclado y pantalla se asignan a los números de dispositivos 5 y 6 respectivamente.

Codificación en **Pascal**

```
program CILINDRO (INPUT, OUTPUT)
{calcular el volumen de un cilindro}
const
   PI = 3.141592
var
   RADIO, ALTURA, VOLUMEN: real
begin
   {lectura de datos}
   read (RADIO, ALTURA)
   VOLUMEN: = PI * SQR (RADIO) * ALTURA
   write ('el volumen es ', VOLUMEN = 8:2)
end {fin de CILINDROS}
```

Codificación en **MODULA-2**

```
MODULE cilindro
{calcular el volumen de un cilindro}
FROM EasyInOut IMPORT ReadReal, WriteReal, WriteString;
  CONST
    PI = 3.14.1592
```

```
VAR
  Radio, Altura, Volumen: REAL
BEGIN
  ReadReal (Radio); ReadReal (Altura)
Volumen := PI * Radio * Radio * Altura
  WriteString ('el volumen es '); WriteReal (Volumen, 8,2)
END cilindro
```

Ejemplo F.2. Calcular la media aritmética de una serie indeterminada de números positivos

Análisis

La media de una serie de números $X1, X2, ..., Xn$ responde a la fórmula

$$\text{MEDIA} = \frac{X1 + X2 + ... + Xn}{n}$$

Se necesitará un sumador que realice la suma de los números y un contador que cuente el total de números:

Variables de entrada X: real
Variables de salida MEDIA: real
Variables intermedias N: entera; SUMA: real

Consideraremos como valor final de archivo (FF) o valor centinela, el número –99.

Pseudocódigo

```
algoritmo MEDIA
{calculo de la media aritmetica de N numeros positivos leidos
del teclado}
{variables enteras = N; variables reales = X, MEDIA, SUMA}
inicio
  {inicializacion de variables}
  N ← 0
  SUMA ← 0
  {lectura de datos}
  leer X
  mientras X <> FF hacer  {FF, fin de archivo o valor centinela}
      N ← N + 1
      SUMA ← SUMA + X
      leer X
  fin_mientras
  MEDIA ← SUMA/N
  escribir 'el total de numeros es ', N
  escribir 'la media es = ', MEDIA
fin
```

BASIC

```
100 'Programa MEDIA
110 'calculo de la medida aritmetica de N numeros, leidos del teclado
120 'variables enteras = N; variables reales = X, MEDIA, SUMA
130 DEFIN N
140 DEFSNG X, MEDIA, SUMA
150 N = 0
160 SUMA = 0
17 INPUT X
180 WHILE X <> - 99
190    N = N + 1
200    SUMA = SUMA + X
210    INPUT X
220 WEND
230 MEDIA = SUMA/N
240 PRINT "el total de numeros es", N
250 PRINT "la media es = ", MEDIA
260 END
```

PASCAL

```
program media (input, output);
  {calculo de la media aritmetica de N numeros leidos del teclado}
var X, SUMA, MEDIA = real;
     N : integer;
begin
   N = 0;
   SUMA : = 0;
   read (X);
   while not eof do
     begin
        N: = N + 1;
        SUMA := SUMA + X;
        read (x)
     end;
        MEDIA:= SUMA/N;
        writeln ('el total de numeros es');
        writeln ('la media es ')
end.
```

Codificación en **FORTRAN**

```
      PROGRAM MEDIA
C     Calculo de la media aritmetica de N numeros leidos del teclado
      INTEGER N
```

```
      REAL X, SUMA, MEDIA
      N = 0
      SUMA = 0
100   READ (5, * , END = 200) X
      N = N + 1
      SUMA = SUMA + X
      GO TO 100
200   MEDIA = SUMA/N
      PRINT *, 'el total de numeros es', N
      PRINT *, 'la media es ', MEDIA
      END
```

Codificación en **MODULA-2**

```
MODULE Media
   FROM RealInOut  IMPORT ReadReal, Flag, WriteReal;
   FROM InOut  IMPORT WriteInt, WriteString, Writeln;
   (*Calculo de la media aritmetica de N numeros positivos
     leidos del teclado *)
   VAR X, SUMA, MEDIA: REAL; n: INTEGER;

BEGIN

   n:= 0
   SUMA : = 0
   WriteString ("Leer una serie de numeros'); Writeln;
    ReeadReal (X); Writeln;
   WHILE Flag DO
      n: = M + 1
      Suma: = Suma + X;
      ReadReal (X); Writeln
   END;
   Media: = Suma/FLOAT (n8);
   Writeln; WriteString ('el total de numero es'); WriteInt (n, 8);
   Writeln; WriteString ('la media es = '); WriteReal (media, 10);
END Media.
```

APÉNDICE G

GLOSARIO DE PROGRAMACIÓN

Abortar *(abort)*. Terminar la ejecución de un programa antes de consumar el proceso normal.

Acceso *(acces)*. Acción de acceder o acudir a un dispositivo de almacenamiento (unidad de cinta o disco) para realizar en él una operación de entrada/salida de datos.

Acceso aleatorio o directo *(random acces)*. Calificativo que se aplica al tipo de acceso a memoria en el que el tiempo de acceso a una información es independiente de la dirección de dicha unidad. Modo de acceso en el que se puede acceder a cualquier dato en cualquier zona.

Acceso secuencial *(sequential acces)*. Tipo de acceso en el que se requiere como condición previa al acceso a un elemento de información haber accedido a todos los elementos anteriores a él.

Actualizar (*update*). Operación de modificar, añadir o eliminar información de un archivo.

Algoritmo *(algoritm)*. Método paso a paso para solucionar un problema que termina en un número finito de pasos.

Análisis sintáctico *(syntax analysis, parsing)*. Proceso mediante el cual un programa traductor realiza la detección de los errores sintácticos de un programa.

Anidamiento *(nesting)*. Inserción de subprogramas, programas o grupos de instrucciones dentro de otras entidades para formar una estructura jerárquica.

ANSI *(American National Standard Institute)*. Organismo americano creado para la estandarización de las características de productos. Se subdivide en comités que tratan temas específicos. Uno de ellos se refiere a la estadarización de los lenguajes de programación.

Aproximación *(approximation)*. Estimación de un valor que se utiliza en lugar del valor real.

Apuntador. *Véase* **Puntero.** Término usual en Latinoamérica.

Árbol binario *(binary tree)*. Estructura enlazada en la que cada nodo tiene como máximo dos descendientes.

Archivo *(file)*. Conjunto de datos organizados y manipulados como una unidad completa. Un archivo se compone de registros y éstos, a su vez, de campos. *Véase* **Fichero.**

Argument *(argument)*. Valor dado a una función (parámetro de entrada) a fin de obtener el resultado.

Array *(array, vector, matriz)*. Tipo de dato estructurado con un número fijo de componentes del mismo tipo. En Latinoamérica, este término se traduce por *arreglo*.

Arreglo. *Véase* **Array**.

ASCII. Siglas de *American Standard Code for Information Interchange* (Código Normalizado Americano para el Intercambio de Información). Se utiliza con profusión en las computadoras actuales. Es un código de 7 bits que permite 128 caracteres diferentes, y existe un código ampliado de 8 bits que admite hasta 256 caracteres. Los programadores deben recurrir con mucha frecuencia al código ASCII.

Asignación *(assignment)*. Acto de almacenar un valor calculado o especificado en una posición de memoria.

Asignar (assign). Identificar un valor que pertenece a un conjunto de datos.

Biblioteca. *Véase* **Librería**.

Bifurcación *(branch)*. Cambio en el flujo de un programa tal que los pasos no se ejecutan en el orden secuencial en que se han escrito.

Binario *(binary)*. Término utilizado para describir objetos que obtienen dos valores, tal como dígitos binarios que pueden ser 0 o 1.

Bit *(bit)*. Dígito binario (0 o 1) utilizado para almacenar información en una computadora. Los bits se agrupan en *bytes* (octetos) y palabras.

Boolean *(lógico)*. Tipo de dato que toma dos posibles valores «verdadero» *(true)* y «falso» *(false)*.

Byte, octeto *(byte)*. Ocho bits. Unidad de información básica que procesan las computadoras.

Bucle, ciclo, lazo *(loop)*. Secuencia de instrucciones repetida continuamente hasta que se cumple una determinada condición de salida del bucle.

Bucle anidado *(nested loop)*. Tipo de bucle que puede situarse dentro de otros bucles.

Bucle infinito *(infinite loop)*. Un bucle que nunca cumple la condición de terminación y sólo puede terminarse por medios externos al programa.

Bucle vacío *(empty loop)*. Aquel bucle que no contiene sentencias en su interior.

Cadena *(string)*. Secuencia contigua de caracteres concatenados.

Campo *(field)*. Parte de un registro que contiene datos del mismo tipo; numérico o alfanumérico.

Carácter *(character)*. Es una letra, dígito, signo de puntuación u otro símbolo utilizado en la representación de información.

Carga *(load)*. En programación, introducción de datos en la memoria de la computadora desde un dispositivo de entrada (cinta, disco magnético).

Cargador *(loader)*. Programa que permite cargar en memoria RAM otro programa desde un dispositivo de entrada.

Clave *(key)*. Campo utilizado para especificar un registro en las operaciones de acceso.

COBOL. Contracción de «*Common Business Oriented Language*» o lenguaje orientado a aplicaciones de gestión. Es el lenguaje por excelencia del campo de la gestión.

Codificación *(coding)*. Acción de codificar una información.

Código *(code)*. En programación, forma de representar los datos y los programas de modo que puedan ser aceptados y tratados por la computadora.

Código fuente *(source code)*. Código de un programa que un intérprete/compilador traduce en un programa en lenguaje máquina. Programa escrito en un lenguaje de programación.

Código máquina *(machine code)*. Código de representación de la información interna en una computadora. Es el código que entiende directamente la máquina.

Código objeto *(object code)*. Salida resultante de la traducción de un programa redactado en código fuente (lenguaje de alto nivel). El código objeto puede ser código máquina puro, directamente cargable en memoria y ejecutable o puede adoptar la forma de código binario trasladable o reubicable que necesita de una fase de montaje o enlace para ser ejecutado.

Cola *(queue)*. Estructura dinámica de datos cuyos elementos se introducen por un extremo y se eliminan por el otro. Se denomina con frecuencia FIFO.

Código automático *(autocode)*. Denominación genérica dada a los precursores de los lenguajes modernos de programación de alto nivel. Hoy día el término está en desuso.

Coma fija *(fixed-point)*. Representación de números reales en que la posición de la coma es fija.

Coma flotante *(floating point)*. Representación de los números en la que la coma que separa la parte entera de la decimal no ocupa una posición fija.

Comentario *(comment)*. Un componente de programa de computadora utilizado para documentación y explicación de las partes del programa y que sean legibles para el usuario.

Compatibilidad *(compatibility)*. Dos computadoras son compatibles cuando un paquete de software se puede ejecutar indistintamente en una y en otra.

Compilación *(compilation)*. Proceso de convertir un programa escrito en un lenguaje de alto nivel en un lenguaje máquina.

Compilador *(compiler)*. Programa que traduce un programa fuente escrito en lenguaje de alto nivel en un programa en código máquina.

Concatenación *(concatenation)*. Una operación que conecta dos cadenas juntas para formar una cadena.

Condición *(condition)*. Expresión que se puede evaluar como verdadera o falsa.

Constante *(constant)*. Elemento que no varía su valor en el transcurso de un programa.

Contador *(counter)*. En programación, una variable que va incrementando o decrementando su valor de modo constante.

Datos *(data)*. Elemento de información procesado por una computadora.

Declaración *(declaration)*. En los lenguajes de programación, las variables han de ser declaradas utilizando reglas sintácticas preescritas, con objeto de informar al compilador acerca de las características detalladas que se supone que representan las variables.

Delimitador *(delimiter)*. Cadena de símbolos o caracteres, tales como (;), (,) o bien **begin** y **end**, que delimita o señala una parte de un programa o una expresión.

Depuración *(debugging)*. Puesta a punto de un programa para que pueda ser ejecutado sin ningún tipo de errores.

Diagrama de flujo *(flowchart)*. Representación gráfica para la definición, análisis o solución de un problema en la que los símbolos se utilizan para representar operaciones, datos, flujos, ejemplos, etc.

Dirección *(address)*. La dirección de una posición de memoria es un número utilizado como un medio de referirse a una posición. Las posiciones de memoria se numeran 0, 1, 2, 3,..., etc.

Documentación externa de un programa *(external documentation)*. Documentación que acompaña a un programa: análisis, organigramas, codificación, etc.

Documentación interna de un programa *(internal documentation)*. Conjunto de comentarios que se incluyen en un programa y que sirven para aumentar la legibilidad y el entendimiento de las diferentes secuencias de instrucciones.

Editor *(editor)*. Programa para crear o modificar un archivo de texto (programa fuente escrito en un lenguaje de alto nivel, textos, etc.).

Eficiencia *(efficiency)*. La eficiencia de un programa se mide por la cantidad de recursos que el programa consume. Mientras menos recursos consume, más eficiente es. El tiempo y el almacenamiento son los recursos que normalmente se consideran.

Ejecución *(run)*. Realización en forma secuencial de todas las instrucciones que componen un programa.

Elemento *(element)*. Posición específica de almacenamiento en un *array*.

Enlazar, montar *(link)*. Proceso en el que uno o más programas o módulos objeto se enlazan o encadenan con bibliotecas de subrutinas o procedimientos utilizados por el programa.

Ensamblador *(assembler)*. Programa que traduce un programa fuente escrito en lenguaje ensamblador y lo convierte en un programa escrito en código máquina entendible directamente por ella.

Entero *(integer)*. Tipo de datos que contiene números positivos o negativos sin parte decimal.

Entrada/salida *(input/output)*. En programación, las operaciones relativas a la transferencia de información entre la memoria y un periférico; por defecto —normalmente— el teclado y la pantalla.

Error de progamación *(bug)*. Errores que se producen en un programa achacables normalmente al programador.

Error en tiempo de compilación *(compile-time error)*. Un error, tal como un error de sintaxis, detectado durante la compilación de un programa.

Error lógico *(logic error)*. Un error normalmente procedente del diseño del algoritmo.

Estructura de control *(control structure)*. Una construcción de programación, tal como selección o bucles, que determina el orden en que se ejecutarán las sentencias.

Estructura de control indefinida *(indefinite control structure)*. Una estructura de control, tal como WHILE o REPEAT-UNTIL, donde el número de iteraciones del cuerpo del bucle puede variar dependiendo del valor de una expresión arbitraria booleana.

Estructura de datos *(data structure)*. Sistema de organización de datos que permiten un fácil almacenamiento, proceso y recuperación.

Expresión aritmética *(arithmetic expression)*. Una expresión de variables, constantes y operaciones aritméticas que pueden ser evaluadas como un único valor numérico.

Expresión booleana *(boolean expression)*. Expresión que, cuando se evalúa, produce un valor lógico o booleano.

Fichero, archivo *(file)*. Conjunto de registros afines tratados como una unidad de almacenamiento de datos; por ejemplo, archivo de direcciones, archivo de agenda, etc. *Véase* **Archivo**.

Fichero de acceso directo *(direct acces file)*. Un archivo en el que la recuperación o almacenamiento de los datos no depende del orden de los componentes o registros y se accede directamente a ellos.

Fichero secuencial *(sequential file)*. Archivos cuyos registros se encuentran ordenados y a los cuales sólo se puede acceder por orden secuencial.

Flujo de control *(flow of control)*. El orden en el que cada línea o parte de un programa se ejecuta. Normalmente el flujo de control es secuencial.

Formateado *(formatting)*. Definición de las características de aspecto formal de un documento o bien organización inicial de un disco magnético.

FORTRAN. Lenguaje científico por excelencia.

Función *(function)*. Subprograma que calcula y devuelve un solo valor.

Gestión de archivos *(file processing)*. Conjunto de funciones de un sistema operativo que realiza controladamente la interconexión entre los archivos y los soportes físicos de los mismos en un sistema informático. La gestión de archivos está también relacionada con el lenguaje de programación elegido.

Identificador *(identifier)*. Una palabra o secuencia de caracteres que forma el nombre de un programa objeto, tal como un nombre de programa, una variable, una constante, un tipo, una función o un procedimiento.

Indexación, indización *(indexing)*. Técnica de acceso a los datos en base a la utilización de un índice o de un registro de índices.

Indexar, indizar *(index)*. Asociar un índice a un dato para su acceso.

Indicador, interruptor, bandera *(flag)*. Una variable utilizada para indicar la ocurrencia de un evento o la presencia de una condición. En BASIC se realiza con una variable `SW` que toma dos valores, 0 y 1. En Pascal un interruptor se representa con una variable booleana.

Inicializar, iniciar *(initialize)*. Asignar un valor inicial a una variable antes de su uso.

Intérprete *(interpreter)*. Programa que traduce un programa escrito en un lenguaje de alto nivel (BASIC. Logo, etc.) a programa escrito en código máquina, realizando la traducción instrucción a instrucción y cada vez que se ejecuta el programa.

Instrucción *(instruction)*. Elemento básico constitutivo de los programas. Una instrucción es la formulación de una orden a la computadora y que se manifiesta en la expresión del conjunto de operaciones que debe ejecutar la computadora.

Juego de caracteres *(character set)*. Conjunto de caracteres entendibles por la máquina codificados de acuerdo a un determinado código; por ejemplo, el ASCII.

Lazo. *Véase* **Bucle**.

Lenguaje de alto nivel *(high-level language)*. Lenguaje que permite generar código máquina a partir de sentencias independientes de la máquina y representar los problemas de un usuario de modo simple y claro. Una sentencia en lenguaje evolucionado se traducirá con frecuencia en una serie de instrucciones máquina o de subprogramas en lenguajes máquina. Lenguajes de alto nivel son: Visual BASIC, FORTRAN, COBOL, Pascal, C, C++, Modula-2 y Ada.

Lenguaje de bajo nivel *(low-level language)*. Lenguaje de programación o código máquina en el que a cada instrucción simbólica le corresponde una en código máquina equivalente. Lenguaje de bajo nivel es el ensamblador. El lenguaje de bajo nivel depende de la máquina.

Lenguaje de programación *(programming language)*. Conjunto de vocabulario, símbolos y reglas utilizados para controlar las operaciones de una computadora.

Lenguaje máquina *(machine language)*. Lenguaje en el que las instrucciones se especifican en binario, de tal forma que sean cargables directamente en memoria y ejecutables. Es el lenguaje de más bajo nivel que permite utilizar una computadora y es el único lenguaje que sabe interpretar la unidad de control de la misma.

Librería, biblioteca *(library)*. Colección de programas y paquetes disponibles para uso común. Pueden ser propios del compilador o definidos por el usuario.

Lista doblemente enlazada o ligada *(doubly linked list)*. Una lista cuyos nodos utilizan dos punteros para permitir el movimiento hacia adelante y atrás de un nodo a otro perteneciente a la estructura lista.

Memoria principal *(main memory)*. Memoria utilizada por el sistema de computadora para almacenar instrucciones y datos a los que se accede directamente por la unidad central de proceso.

Mezcla, fusión *(merge)*. Proceso de combinar dos o más listas o archivos ordenados en una lista o archivo ordenado.

Modularidad *(modularity)*. Un programador *modular* es uno que se divide en unidades (tales como procedimientos) de tal modo que cada unidad tiene un medio bien definido de comunicarse con el resto del programa.

Montador *(linker)*. Programa de utilidad que combina todos los módulos de un programa compilador por separado mezclándolos en un módulo que será directamente ejecutable por la máquina.
Montar, enlazar *(link)*. Proceso en el que uno o más programas objetos se enlazan con bibliotecas de subrutinas o procedimientos utilizados por el programador.

Nodo *(node)*. El punto o vértice de un árbol; normalmente representa un dato.
Nombre de variable *(variable name)*. Identificador que se asocia con una variable.
Notación exponencial *(exponential notation)*. Método de expresar un número como un valor multiplicado por 10, elevado a la potencia indicada por el exponente.

Operador *(operator)*. Símbolo especial o palabra reservada que especifica una operación aritmética o lógica.
Operador boolean, lógico *(boolean operator)*. Operador que combina expresiones lógicas utilizando aritmética lógica. Operadores booleanos son, entre otros: AND, OR y NOT.
Operando *(operand)*. El valor utilizado cuando se evalúa un operador tal como la suma (+) o multiplicación (*).
Orden ascendente *(ascending order)*. Orden desde el nivel más bajo al nivel más alto.
Ordenación/mezcla *(sort/merge)*. Procesos de ordenación/clasificación y mezcla o fusión de listas o archivos de datos.
Ordenación por burbuja *(bubble sort)*. Algoritmo de ordenación multipaso que generalmente requiere más de un paso a través de la lista a fin de garantizar que la lista se clasifica en el orden deseado.

Palabra clave *(keyword)*. *Véase* **Palabra reservada**.
Palabra reservada *(reserved word)*. Palabras utilizadas por los lenguajes de programación para representar variables, sentencias, órdenes, etc.
Parámetro *(parameter)*. Un valor o variable que aparece en la lista de parámetros de un procedimiento o función, o en la lista de parámetros de una invocación de un procedimiento o función.
Parámetro formal *(formal parameter)*. Un identificador que aparece en la lista de parámetros de un procedimiento o función (subprograma).
Pila *(stack)*. Estructura dinámica de datos en la que los elementos se añaden y eliminan por un extremo.
Portabilidad, transportabilidad *(portability)*. Capacidad de transportar un programa desde un sistema de computadora a otra con mínimos cambios.
Precedencia, prioridad *(precedence)*. Jerarquía que determina el orden de operaciones en una expresión con más de un operador.
Procedimiento *(procedure)*. Subprograma con un nombre y parámetros opcionales que pueden ser llamados o invocados y ejecutados. Subprograma que se identifica por su nombre. Realiza unas operaciones bien definidas y ejecuta una tarea determinada.
Procesos por lotes *(batch processing)*. Método de ejecución de programas en el que éstos se ejecutan en el mismo orden en que están definidos.
Programa ejecutable *(executable program)*. Programa complicado y traducido a código máquina y que puede ser ejecutado directamente por la computadora.
Programa fuente *(source program)*. Programa escrito en un lenguaje de alto nivel que requiere la compilación o interpretación para su ejecución.
Programa objeto *(object program)*. Programa en lenguaje máquina resultante de la traducción del programa fuente.

Programación descendente *(top-dow programming)*. Una estrategia de diseño de programas que utiliza un refinamiento algorítmico. La parte «superior» es una definición abstracta del problema y la parte «inferior» es una solución muy detallada.
Programación estructurada *(structured programming)*. Método de construcción y diseño de programas en el que se hace incidencia en la legibilidad, modificabilidad y facilidad.
Pseudocódigo, seudocódigo *(pseudocode)*. Mezcla de instrucciones de computadora y lenguaje natural (español o inglés) utilizados para expresar un algoritmo.
Puesta a punto, depuración *(debugging)*. Proceso de búsqueda y eliminación de errores de un programa.

Recursión, recursividad *(recursión)*. Proceso de ejecución de procedimientos que pueden llamarse a sí mismos.
Refinamiento por pasos *(stepwise refinement)*. Proceso de romper un problema en pasos por refinamiento con trucos.
Registro *(record)*. Una estructura de datos formada por diferentes campos.

Sentencia *(sentence)*. Instrucción de un lenguaje de programación.
Sintaxis *(syntax)*. Reglas formales que definen las construcciones válidas de un lenguaje.
Subíndice *(subscript)*. Valor o índice utilizado para acceder a los componentes de un *array*.
Subprograma *(subprogram)*. Función o subrutina (procedimiento).
Subrutina *(subroutine)*. Subprograma que realiza una tarea determinada y que normalmente será repetitiva a lo largo de un programa.

Valor por defecto o **por omisión** *(default value, by default)*. Valor utilizado por la computadora cuando no se especifica ningún valor por el usuario o programador.
Variable *(variable)*. Un objeto en un programa que puede cambiar su valor durante su ejecución. Posición de memoria en el que se almacena la información y de la que se puede recuperar.
Variable global *(global variable)*. Una variable declarada en un subprograma o en un programa principal y, por consiguiente, accesible en cualquier subprograma dentro del ámbito de la variable.
Variable local *(local variable)*. Una variable declarada en un procedimiento (o función) conocida sólo dentro de ese procedimiento (o función).
Variable lógica, boolean *(boolean variable)*. Una variable que toma sólo dos posibles valores: *verdadero, falso*.

APÉNDICE **H**

CÓDIGOS ASCII

El código ASCII (American Standard Code for Information Interchange; código estándar americano para intercambio de información) es un código que traduce caracteres alfabéticos y caracteres numéricos, así como símbolos e instrucciones de control en un código binario de siete u ocho bits.

Tabla H.1. Código ASCCI de la computadora personal PC

Valor ASCII	Carácter	Valor ASCII	Carácter
0	Nulo	25	↓
1	☺	26	→
2	☻	27	←
3	♥	28	Cursor a la derecha
4	♦	29	Cursor a la izquierda
5	♣	30	Cursor arriba
6	♠	31	Cursor abajo
7	Sonido (pitido, *bip*)	32	Espacio
8	◘	33	!
9	Tabulación	34	"
10	Avance de línea	35	#
11	Cursor a inicio	36	$
12	Avance de página	37	%
13	Retorno de carro	38	&
14	♫	39	'
15	☼	40	(
16	►	41	)
17	◄	42	*
18	↕	43	+
19	!!	44	,
20	π	45	–
21	§	46	.
22	▬	47	/
23	↨	48	0
24	↑	49	1

Valor ASCII	Carácter	Valor ASCII	Carácter
50	2	99	c
51	3	100	d
52	4	101	e
53	5	102	f
54	6	103	g
55	7	104	h
56	8	105	i
57	9	106	j
58	:	107	k
59	;	108	l
60	<	109	m
61	=	110	n
62	>	111	o
63	?	112	p
64	@	113	q
65	A	114	r
66	B	115	s
67	C	116	t
68	D	117	u
69	E	118	v
70	F	119	w
71	G	120	x
72	H	121	y
73	I	122	z
74	J	123	{
75	K	124	|
76	L	125	}
77	M	126	~
78	N	127	⌂
79	O	128	Ç
80	P	129	ü
81	Q	130	é
82	R	131	â
83	S	132	ä
84	T	133	à
85	U	134	å
86	V	135	ç
87	W	136	ê
88	X	137	ë
89	Y	138	è
90	Z	139	Ï
91	[	140	Î
92	\	141	Ì
93	]	142	Ä
94	^	143	Å
95	_	144	É
96	`	145	æ
97	a	146	Æ
98	b	147	ô

Valor ASCII	Carácter	Valor ASCII	Carácter
148	ö	197	+
149	ò	198	╞
150	û	199	╟
151	ù	200	╚
152	ÿ	201	╔
153	Ö	202	╩
154	Ü	203	╦
155	¢	204	╠
156	£	205	═
157	¥	206	╬
158	Pt	207	╧
159	*f*	208	╨
160	á	209	╤
161	í	210	╥
162	ó	211	╙
163	ú	212	╘
164	ñ	213	╒
165	Ñ	214	╓
166	a	215	╫
167	o	216	╪
168	¿	217	┘
169	⌐	218	┌
170	¬	219	■
171	1/2	220	▄
172	1/4	221	▌
173	¡	222	▐
174	«	223	▀
175	»	224	α
176	▒	225	β
177	▒	226	Γ
178	▓	227	π
179	│	228	Σ
180	┤	229	σ
181	╡	230	µ
182	╢	231	τ
183	╖	232	Φ
184	╕	233	Θ
185	╣	234	Ω
186	║	235	δ
187	╗	236	∞
188	╝	237	∅
189	╜	238	ε
190	╛	239	∩
191	┐	240	≡
192	└	241	±
193	┴	242	≥
194	┬	243	≤
195	├	244	⌠
196	─	245	⌡

Valor ASCII	Carácter	Valor ASCII	Carácter
246	÷	251	√
247	≈	252	n
248	°	253	2
249	•	254	■
250	·	255	(blanco 'FF')

CÓDIGOS AMPLIADOS DE TECLAS

Los códigos ampliados de teclas se devuelven por esas teclas o combinaciones de teclas que no se pueden representar por los códigos ASCII listados en la Tabla H.1.

Tabla B.2. Códigos ampliados de teclas

Segundo código	Significado
3	NULL (carácter nulo)
15	Shift Tab (– < vv)
16-25	Alt-Q/W/E/R/T/Y/U/I/O/P
30-38	Alt-A/S/D/F/G/H/I/J/K/L
44-50	Alt-Z/X/C/V/B/N/M
59-68	Teclas F1-F10 (desactivados con teclas soft)
71	Home (*Inicio*)
72	Cursor arriba (↑)
73	PgUp (*RePág*)
75	Cursor a la izquierda (←)
77	Cursor a la derecha (→)
79	End (*Fin*)
80	Cursor abajo (↓)
81	PgDn (*AvPág*)
82	Ins
83	Del (*Supr*)
84-93	F11-F20 (Shift-F1 a Shift-F10)
94-103	F21-F30 (Ctrl-F1 hasta F10)
104-113	F31-F40 (Alt-F1 hasta F10)
114	Ctrl-PrtSc (*Ctrl-ImprPant*)
115	Ctrl-Flecha izquierda (Ctrl←)
116	Ctrl-Flecha derecha (Ctrl→)
117	Ctrl-End (*Ctrl-Fin*)
118	Ctrl-PgDn (*Ctrl-AvPág*)
119	Ctrl-Home (*Ctrl-Inicio*)
120-131	Alt-1/2/3/4/5/6/7/8/9/0/–/=
132	Ctrl-PgUp (*Ctrl-RePág*)
133	F11
134	F12
135	Shift-F11 (*Mayús-F11*)
136	Shift-F12 (*Mayús-F11*)

Segundo código	Significado
137	Ctrl-F11
138	Ctrl-F12
139	Alt-F11
140	Alt-F12

CÓDIGOS DE EXPLOTACIÓN DE TECLADO

Los códigos de exploración de teclado son los códigos devueltos de las teclas en el teclado IBM-PC, tal como se ven por Turbo Pascal.

Estas teclas son útiles cuando se trabaja a nivel de lenguaje ensamblador. Los códigos de exploración de la tabla se visualizan en valores hexadecimales (dígitos 0,1,2,..., 9, A,B,C,D,E,F).

Tabla B.2. Códigos de exploración del teclado

Tecla	Código de exploración en hexadecimal	Tecla	Código de exploración en hexadecimal
Esc	01	←/→	0F
¡1	02	Q	10
@ 2	03	W	11
# 3	04	E	12
$ 4	05	R	13
% 5	06	T	14
^ 6	07	Y	15
& 7	08	U	16
* 8	09	I	17
(9	0A	O	18
) 0	0B	P	19
—	0C	{ [	1A
+ =	0D	}]	1B
Retroceso (*Backspace*)	0E	Return	1C
Ctrl	1D	/ \	2B
A	1E	Z	2C
S	1F	X	2D
D	20	L	26
F	21	¨	27
G	22	“	28
H	23	~‘	29
J	24	←Shift (←*Mayús*)	2A
K	25	Barra espaciadora	39
F8	42	Caps Lock (*BloqMayús*)	3A
F9	43	F1	3B
F10	44	F2	3C
F11	D9	F3	3D
F12	DA	F4	3E
Scroll Lock (*BloqDespl*)	46	F5	3F

Tecla	Código de exploración en hexadecimal	Tecla	Código de exploración en hexadecimal
F6	40	>.	34
F7	41	?/	35
Signo menos	4A	→ Shift (→*Mayús*)	36
4 ←	4B	PrtSc* (*ImprPant*)	37
5	4C	Alt	38
6 →	4D	7 Home (*Inicio*)	47
+	4E	8 (↑)	48
1 End (Fin)	4F	9 PgUp (*Repág*)	49
C	2E	2 ↓	50
V	2F	3 PgDn (*AvPág*)	51
B	30	0 Ins	52
N	31	Del (*Supr*)	53
M	32	Num Lock (*BloqNum*)	45
<,	33		

BIBLIOGRAFÍA

TEORÍA DE PROGRAMACIÓN EN GENERAL

BRAUNSTEIN, Silvia L., y GIOIA, Alicia B.: *Introducción a la programación y a las estructuras de datos.* Eudeba, Buenos Aires, 1986.

CASTELLANI, X.: *Método general de análisis de una aplicación informática.* Masson, 1986.

CLAVEL, Biondi: *Introducción a la programación.* Tomo 1, *Algorítmica y lenguajes.* Tomo 2, *Estructura de datos.* Masson, 1985.

CLAVEL, Jorgesen: *Introducción a la programación.* Tomo 3, *Ejercicios corregidos.* Masson, 1986.

COLEMAN, D.: *Organización de datos y programación estructurada.* Gustavo Gili, 1986.

DAHL, O. J.; DIJGSTRA, y E. W. HOARE C. A. R.: *Structued Programming.* Academic Press, 1973.

DAHL; DIJKSTRA, y HOARE: *Structured Programming,* 1972.

DIJKSTRA, E. W.: *«Go to statement considered harmful».* Comm. Assoc. Comp. Mach., 11 (1968, pp. 147-148).

DIJKSTRA, E. W.: *«Notes on Structured Programming».* Structured Programming. Academic Press, Nueva York, 1972.

GHEZI, Carlo, y JAZAYER Meh: *Conceptos de lenguajes de programación.* Díaz de Santos, 1986.

GOLDSCHLAGER; LES, y LISTER, Andrew: *Introducción moderna a la ciencia de la computación.* Prentice-Hall, 1986.

HOARE, C. A. R.: *«Notes on Data Structuring».* Structured Programming. Academic Press, Nueva York, 1972.

HUGHE, Juan K.; MICHTOM, Glen C., y MICHTOM, Jay I.: *A Structured approach to programming.* 2.ª edición. Prentice-Hall, 1977.

JOYANES, Luis: *Metodología de la Programación,* McGraw-Hill, 1986; *Problemas de Metodología de la Programación,* McGraw-Hill, 1990.

JOYANES, Luis, RODRÍGUEZ, Luis, y FERNÁNDEZ, Matilde: *Fundamentos de programación. Libro de problemas.* McGraw-Hill, 1996.

KNUTH, D. E.: *Structured Programming with goto statements.* Comput. Serv. 6(1974), pp. 261-301.

LASALA; CRISTÓBAL, y LAPENA: *Técnicas de análisis y programación* para IBM PC (y compatibles). Universidad de Zaragoza, 1985.

MORALES PASCUAL, José Luis: *Programación de ordenadores,* tomos (1 a 6). ECC, 1986.

NAYLOR, Jeff: *Introduction to Programming.* Paradign Publishing Ltd., 1987.

RODRÍGUEZ ASENSIO: *Metodología de análisis y programación.* R. Asensio, 1985.

SABATINI, Domenico: *Introduzione alla programazione structurada.* Bufetti Editore, 1985.

VIRGO, Fernando: *Técnicas y elementos de programación.* Gustavo Gili, Barcelona, 1985.

WIRTH, Niklaus: *Introducción a la programación sistemática.* El Ateneo. Buenos Aires, 1984.

WOODHOUSE, David; JOHNSTONE, Greg, y MCDOUGAL, Anne: *Computer Science.* Wiley & Sons, 1982.

ALGORITMOS Y ESTRUCTURA DE DATOS

BRASSARD, G.; BRATLEY, P.: *Algorithmique.* Masson, 1987.

BOUSSARD, J. C., y MAHL, R.: *Programmation Avancee: Algorithmique et Structures de Donnss.* Eyrolles, 1983.

COURTIN, J., y KOWARSKI, I.: *Initiation a l'algorithmique et aux structures de donness.* Dunod, 1987.

DALE, y LILLY: *Pascal y estuctura de datos.* McGraw-Hill, 1986.

DIJKSTRA, E. W.: *A discipline of programming.* Prentice-Hall, 1976.

DIJKSTRA, E. W.: *Goto Statement Considered Harmful.* Communications of the ACM. Vol. 11, núm. 3, marzo 1968, 147-148, 538, 541.
HAREL, David: *Algorithmics.* Addison Wesley, 1987.
HELMAN, Paul, y VEROFF, Robert: *Intermediate Problem Solving and Data Structures.* Benjamín Cummings, 1986.
HOARE C. A. R., y DAHL, O.: *Structured Programming.* Academic Press, 1972.
HOROWITZ, Ellis, y SAHNI, Sartaj: *Fundamentals of computer algorithms.* Computer Science, Press, Inc., 1978.
KNUTH, E. E.: *The art of Computer Programming.* Vol. 1, *Fundamental Algorithms,* 1969. Vol. 2, *Sorting and Searching.*, 1972. Addison Wesley.
KNUTH, D. E.: *Structured programming with go-to statements.* ACM Computing Surveys. Vol. 6,4, 1974, pp. 161-301.
KRUSE, Robert L.: *Data Structures and Program Design.* 2.ª edición. PHI, 1987.
LEWIS, T. G., y SMITH, M. Z.: *Estructuras de datos.* Paraninfo, 1985.
LUCAS; REYRIN, y SCHOLL: *Algorítmica y representación de datos.* Vol. 1, *Secuencias, autómatas de estados finitos.* Masson, 1985.
MELHORN, N.: *Data Structures and Algorithms.* Vol. 1, *Sorting and Searching.* Springer-Verlag, 1984. Vol. 2, *Graph Algorithms and WP-Completeness.* Springer-Verlag. Berlín.
LIPSCHUTZ, Seymour: *Estructura de datos.* McGraw-Hill, 1986.
SCHOLL, P. C.: *Algorítmica y representación de árboles.* Vol. 2, *Recursividades y árboles.* Masson, 1986.
TREMBLAY, Jean-Paul, y BUNT, Richard B.: *Introducción a la ciencia de las computadoras.* McGraw-Hill, 1982.
WIRTH, Niklaus: *Algoritmos + estructuras de datos = Programas.* Ediciones Castillo.
— *Algoritmos y estructuras de datos.* Prentice-Hall, 1987.
— *Data Structured and Algorithms.* Scientific American, 1984.

ARCHIVOS (FICHEROS)

GROSSHAUS, Daniel: *File System.* Prentice-Hall, 1986.
JOYANES AGUILAR, Luis: *Introducción a la teoría de ficheros (archivos).* UPS, 1987.
LOOMIS, Mary E. S.: *Data Management and File Processing.* Prentice-Hall, 1983.
WALTER: *Introduction to Data Management and File Design.* Reston, 1986.

ALGORITMOS DE ORDENACIÓN Y BÚSQUEDA

GUIHUR, R: *Procedimientos de clasificación.* Masson, 1987.
KNUTH, D. E.: *The arth of Computer Programming.* Vol. 2, *Sorting and Searching.* Addison Wesley, 1973.
ROUX, M.: *Algorithmes de classification.* Masson, 1985.
SEDGEWICK, R.: *Quicksort.* These. Stanford University, 1975.
WIRTH, Niklaus: *Algoritmos + Estructuras de datos = programas.* Ediciones del Castillo.

LENGUAJES DE PROGRAMACIÓN

GHEZZI, Carlos, y JAZAYERI, MEHDI: *Conceptos de lenguajes de programación.* Díaz de Santos, 1986.
MARCOTTY; Michael, y LEDGAR, Henry: *The world of Programming Languages.* Springer-Verlag, 1987.
PRATT, Terrence W: *Lenguajes de programación.* Prentice-Hall, 1984.
SMEDEMA, C. H.; MEDENA, P., y BOASSON, M.: *Les langages de programmation.* Masson, 1986.

TENNENT, R. D.: *Principles of Programming Languages.* Prentice-Hall, 1980.
TERRY, Patrick, D.: *Programming language traslation.* International Computer Science. Addison Wesley, 1986.
TUKER: *Lenguajes de programación.* McGraw-Hill, 1986.
YOUNG, S. J.: *Real Time Languages.* Ellis Horwood-Publishers, 1982.

LENGUAJE ADA

ALABAN, M.: *Le langage Ada. Theorie et pratique,* Eyrolles, 1995.
BARNES, J. G. P.: *Programación en Ada,* Díaz de Santos, 1987.
FELDMAN, M. B., y KOFFMAN, E. B.: *Ada 95. Problem Solving and Program Design,* Addison-Wesley, 1996.
LUCAS, Ángel: *Ada. Metodología y Lenguaje.* Paraninfo, 1990.
SAVITCH, Walter J., y PETERSEN, C. G.: *Ada. An Introduction to the Art and Science of Programming. Benjamin Cummings,* 1992.

LENGUAJE BASIC

ETTLIN, y SOLBERG: *Manual de MBASIC.* McGraw-Hill, 1985.
GOTTFRIED: *Programación BASIC,* 3.ª edición. McGraw-Hill, 1986.
GRAHAM, y FIELD: *IBM/PC. Guía del IBM/PC.* McGraw-Hill, 1986.
HERGERT, Douglas: *Microsoft Quick BASIC.* Microsoft Press, 1987. *Using Turbo BASIC.* McGraw-Hill, 1987.
JOYANES AGUILAR, Luis: *Programación BASIC para microcomputadoras.* 3.ª edición. McGraw-Hill, 1992.
— *IBM PC. Manual de BASIC.* McGraw-Hill, 1986.
— *Introducción a la programación estructurada en BASIC (True BASIC).* UPS, 1987.
— *BASIC Avanzado. Tratamiento y diseño de archivos.* McGraw-Hill, 1987.
— *Apple II. Manual de BASIC.* McGraw-Hill, 1986.
— *ZX Spectrum. Guía del usuario.* McGraw-Hill, 1986
— *Programación en QuickBasic/QBasic,* 2.ª edición, McGraw-Hill, 1995.
JOYANES, Luis, y VILLAR, Luis: *QuickBASIC avanzado,* McGraw-Hill, 1993.
KURTZ, y KEMMENY: *Structured BASIC Programmig.* Wiley, 1987.

LENGUAJE C

DELANNOY, C.: *El libro del C. Primer lenguaje.* Eyrolles, 1995.
KERNIGHAM, B. M., y RITCHIE D. M.: *El lenguaje de programación C.* 2.ª edición. Prentice-Hall, 1991.
PLAUGER, P. J.: *The Standard C Library.* Prentice-Hall, 1992.
PLAUGER, P. J., y BRODIE, J.: *C estándar.* Anaya, 1990.
RADCLIFFE, Robert A.: *Encyclopedia C.* Sybex, 1991.

LENGUAJE C++

AHMERAL, Lendert: *C++ for Programmers.* Wiley, 1991.
ANDERSON, Arthur, y HEINZE, William: *C++ Programming and Fundamentals Concepts.* Prentice-Hall, 1992.
AT & T C++: *Reference Manual,* 1989.
ATKINSON & ATKINSON: *Using C/C++.* Quie, 1993.
BARKAKATI: *Object-Oriented Programming in C++.* Sams, 1991.

BERRY, John Tomas: *C++ Programmings.* Second Edition. The Waite Groups's. Sams, 1992.
BOOCH, Grady: *Object-Oriented Design with Applications.* Benjamming Cummings, 1991.
BUDD, Timothy: *An Introduction to Object-Oriented Programing.* Addison-Wesley, 1991.
BORLAND C++: *Programer's Guide, Library Reference, User's Guide, Tools Utilities Guide, Borland C++ 4.* Borland International, 1991, 1992, 1993.
CANTÚ, Marco, y TENDON, Steve: *Borland C++ 3.1. Object-Oriented Programming.* Bantam Books, 1992.
COX, Brad J., y NOVOBILSKI, Andrew: *Object-Oriented Programming. An Evolutionary Approach.* Addison-Wesley, 1991.
CHRISTIAN, Kaare: *The Microsoft Guide to C++ Programming.* Microsoft Press, 1992.
DEWHURST & STARK: *Programming in C++.* Prentice-Hall, 1989.
ECKEL, Bruce: *Aplique C++.* Osborne McGraw-Hill, 1991. *C++ Inside & Out.* McGraw-Hill, 1993.
EGE, Raimund K: *Programming in a Object-Oriented Environment. Academic Press, 1992.*
ELLIS & STROUTRUP: *The annotated C++ Reference Manual.* Addison-Wesley, 1990.
ENSTMINGER, Gary: *The Tao of Objects.* M. & T Books, 1991.
FAISON: *Borland C++4. Objects Oriented Programming.* Sams, 1994.
FLAMING, Bryan: *Turbo C++. A Self-Teaching Guide.* Wiley, 1991, *Turbo C++. Step-by-step.* Wiley, 1991.
GORLEN; ORLOW, y PLEXICO: *Conception et Programmation Orientées Object en C++.* Dunod, 1991.
HOLUB, Allen I: C+ C++. *Programming with Objects in C and C++.* McGraw-Hill, 1992.
JOYANES, Luis: *Turbo C++. Manual de bolsillo.* McGraw-Hill, 1992.
— *C++. Manual de Bolsillo.* McGraw-Hill, 1993.
— *Borland C++. Manual de bolsillo.* McGraw-Hill, 1993.
— *C++ a su alcance. Un enfoque orientado a objetos.* McGraw-Hill, 1994.
— *Microsoft C/C++. Manual de bolsillo.* McGraw-Hill, 1994.
— *Programación orientada a objetos.* McGraw-Hill, 1996.
— *Turbo C++. Iniciación y referencia.* McGraw-Hill, 1996.
— *Visual C++. Iniciación y referencia.* McGraw-Hill, 1996.
— *Curso de programación con Turbo++.* McGraw-Hill, 1996.
KERNIGHAM & RITCHIE: *El Lenguaje de Programación C.* Segunda edición. Prentice-Hall, 1991.
LAFORE, Robert: *Object-Oriented Programming in Turbo C++.* Waite Group Press, 1991.
LIPPMAN, Stanley B: *C++ Primer.* Primera y segunda edición. Addison-Wesley, 1989, 1991.
MCCORD: *Borland C++ 3.1. Programmer's Reference.* Que, 1992.
Microsoft Languages Library: *Microsoft C. Run-Time Library Reference.* Microsoft Press, 1990.
— *Run-Time Library Reference. Microsoft C/C++.* 1991.
— *Class Libraries User's.* 1991.
— Guide. *C++ Language Reference.* 1991.
PAPAS & MURRAY: *Microsoft C/C++ 7. The Complete Reference.* McGraw-Hill, 1992.
POHL, Ira: *C++ for C Programmers.* Benjamming/Cummings, Redwood City, CA, 1989.
PRATA, Stephen: *C++ Primer Plus.* Waite Group Press, 1991.
RANADE, Jay & ZAMIR, Saba: *C++ Primer for Programmers C.* McGraw-Hill, 1992.
SHAMMAS: *Borland C++4. Programmer Should Know,* Sams, 1994.
SCHILDT: *Aplique Turbo C++.* McGraw-Hill, 1991.
SKINNER, M. T: *The Advancec C++ Book.* Prentice-Hall, 1992. M & T Books, 1990.
SMITH, Michael A: *Object-Oriented software in C++.* Chapman & Hall, 1993.
STROUSTRUP, Bjarne: *The C++ Programming Language.* Second Edition. Addison-Wesley, 1991.
SWAN, Ton: *Learning C++.* Sams, 1991. *Mastering Borland C++.* Sams, 1992.
VOSS, Greg: *Object-Oriented Programming.* McGraw-Hill, 1991.
WAITE: *Turbo C++ Bible.* Sams, 1990.
WAITE & PRATA: *New C Primer Plus.* Second Edition. Sams, 1993.
WEISKAMP & FLAMING: *C++ The Complete Primer.* Academic Pres, 1991.
WIENNER & PINSON: *An Introduction to Object-Oriented Programming and C++.* Addison-Wesley, 1988.
WINBLAD; EDWARDS, y KING: *Object-Oriented software.* Addison-Wesley, 1990.

LENGUAJE COBOL

DE LA FUENTE, Ciro: *Lenguaje COBOL*, 10.ª edición, *El autor*, 1993.
GALÁN PASCUAL, C.: *Programación con Lenguaje COBOL*, Paraninfo, 1986
GARCÍA CERRO: *Programación COBOL., Algoritmos estructurados*, McGraw-Hill, 1994.
HURTADO MELERO, T.: *Lenguaje COBOL con programación estructurada,* Paraninfo, 1988.
— *Programación COBOL*, Paraninfo, 1988.
JARILLO, P., y JARILLO, M. L.: *COBOL*. Anaya, 1995.
MÁRQUEZ, T.: *Diseño de programas en COBOL con programación estructurada y pseudocódigo,* McGraw-Hill, 1995.
NEWCOMER, L.: *Programación avanzada en COBOL estructurado,* McGraw-Hill, 1989.
— *Programación en COBOL estructurado,* McGraw-Hill, 1988.
PHILIPPAKIS, G.: *COBOL estructurado*. 3.ª edición, McGraw-Hill, 1958.

LENGUAJE FORTRAN

DAVIS: *FORTRAN 77*. McGraw-Hill, 1986.
ETTER, D. M.: *Structured FORTRAN 77*. Benjamin Cummings, 1987.
HAMMOND; ROGER, y CRITTENDEN: *Introduction to FORTRAN 77 and the personal Computer.* McGraw-Hill, 1987.
JOYANES AGUILAR, Luis: *Introducción al FORTRAN 77*. UPS, 1988.
JOYANES LUIS, y ZAHONERO, Ignacio: *Programación en FORTRAN 77 y 90, Los autores,* Distribución Librería Paraninfo (Argüelles, Madrid), 1993.
KOFFMAN, y FRIEDMAN: *Problem Solving and Structured Programming in FORTRAN 77,* 3.ª edición. Addison Wesley, 1987.
POLLACK, Seymour V.: *Structures FORTRAN 77 Programming*. Boyd & Fraser Computer Science Series, 1982.

LENGUAJE MODULA-2

GREENFIELD, Stuart B.: *Invitation to Modula-2*. Pretocelli Books, Inc. 1985.
KAARE, Christian: *A guide to Modula-2*. Springer-Verlag 1986.
KELLY, y BOOTLE: *Modula-2*. Primer, 1987.
MOORE, y MCKAY: *Modula-2. Text and Reference*. Reston, 1987.
OGILVIE, John W. L.: *Modula-2 Programming*. McGraw-Hill, 1985.
TERRY, Patrick D.: *An Introduction Programming with Modula-2*. Addison Wesley, 1987.
WIRTH, Niklaus: *Programming in Modula-2*. Springer-Verlag, 1985.

LENGUAJE PASCAL

DELGADO, Alonso: *Sistema para programación en Pascal sobre microordenadores*. Gustavo Gili, 1986.
DALE, y LILLY: *Pascal y estructura de datos*. McGraw-Hill, 1986.
DALE, y ORSHALICK: *Pascal*. McGraw-Hill, 1986.
GARLAND STEPHEN, J.: *Introduction to Computer Science with applications in Pascal*. Addison Wesley, 1986.
GOTTFRIED, Byron: *Programación en Pascal*. McGraw-Hill, 1985.
GROGONO: *Programación en Pascal*. Fondo Educativo Interamericano. McGraw-Hill, 1984.
HOARE, y WIRTH: *An Axiomatic Definition of the Programming Language Pascal,* Acta Informática, 2, núm. 4, 1973, 335-355.

JAMSA, y NAMEROFF: *Turbo Pascal. Biblioteca de programas,* 1987.
JENSEN, K, y WIRTH, N.: *Pascal. Manual de l'utilisateur et report.* 3.ª edición. Eyrolles.
JENSEN, y WIRTH, Niklaus: *Pascal. Manual del usuario e informe.* El Ateneo, Buenos Aires, 1985.
JOYANES, Luis: *Programación en Turbo Pascal.* 2.ª edición.
— *Turbo Pascal 6.0 a su alcance.* McGraw-Hill.
— *Turbo Pascal 7. Manual de bolsillo.* McGraw-Hill 1995.
— *Turbo Pascal 6. Manual de bolsillo.* McGraw-Hill, 1993.
JOYANES Luis; ZAHONERO, Ignacio, y HERMOSO. Ángel: *Pascal y Turbo Pascal. Un enfoque práctico,* McGraw-Hill, 1995.
KONVALENA, y WILEMAN: *Programming with Pascal.* McGraw-Hill, 1987.
LLESTMA, Sanford, y NYHOFF, Larry: *Pascal.* Macmillan, 1984.
LINZ: *Programming Concept and Problem Solving.* Benjamin Cummings, 1983.
MILLER, Lawrence, H.: *Advanced Programing. Design and Structure using Pascal.* Addison Wesley, 1986.
TISSERANT, A.: *Pascal ISO/AFNOR.* Mason, 1986.

TERMINOLOGÍA INFORMÁTICA

OXFORD: *Diccionario Oxford de Informática.* Díaz de Santos, 2.ª edición, 1993.
ROSEMBERG: *Diccionary of Computers. Data Processing & Telecomunications.* Wiley, 1985.
SPILL, Charles J.: *Macmillan Dictionary of Microcomputing.* Macmillan, 1985.
VAQUERO, y JOYANES: *Informática. Glosario de términos.* McGaw-Hill, 1985.

ÍNDICE

abrir, 276
abs , 26
acumulador, 73, 79-80
Ada, 10, 703
algoritmo, 2, 84
  características, 5
  codificación, 4, 667
  concepto, 4
  declaración de variables, 84
  diseño, 4, 41, 68
  escritura, 43, 87-88
  refinamiento, 41, 43
  representación gráfica, 46
al-Khowârizmî, 2
ámbito, 178-181
análisis
  del problema, 40, 68
  funcional, 494-495
  oportunidad, 494
  orgánico, 494-495
ANSI, 46
apuntador, 386, 391. *Véase* puntero
árbol, 204
  altura, 441
  ascendiente, 441
  binario, 442, 447, 449-453
  bosque, 441, 447
  búsqueda en un, 457
  completo, 445
  concepto, 440
  de búsqueda, 455
  degenerado, 445
  descendiente, 441
  eliminación, 459
  en-orden, 454
  equilibrado, 445
  general, 441
  hoja, 441
  implementación con arrays, 461
  in-orden, 454
  nodo terminal, 441
  nodos, 440
  post-orden, 454
  pre-orden, 454
  profundidad, 441
  raíz, 441
  recorrido, 453
  subárbol, 440
archivos, 259, 261-262
  abrir, 276
  acceso directo, 267
  acceso secuencial, 266
  actualización, 270, 272
  altas, 279
  artículo, 260
  bajas, 279
  borrar, 275, 278
  campo, 260-261
  cerrar, 275, 278
  clasificación, 270, 273, 369
  concepto, 260
  consulta, 270-271
  creación, 270, 271, 275
  destrucción, 270, 171
  directos, 287, 296
  estallido, 270, 273

factor de bloqueo, 263
fusión, 270, 274, 361
gestión, 274, 278
mantenimiento, 278
modificaciones, 279
operaciones, 270
ordenados, 360
organización, 265
partición, 364
registro, 260, 261
reorganización, 270, 273
rotura, 270, 273
secuenciales, 280-287
secuencial indexado, 268, 296
texto, 287
array, 203
almacenamiento en memoria, 221-224
bidimensional, 215, 216
multidimensional, 215, 218-221
unidimensional, 205
arreglo, *véase* array, 203
ASCII, 23, 238, 695-700
asignación, 27-29

bandera, 133, 134
base de datos, 261
BASIC, 10, 25, 98, 703
bifurcación, 72
condicional, 72, 73
incondicional, 72
bloque, 262
borrar, 250
bucles, 73, 74, 117
anidado, 75, 142
cuerpo, 74
cruzados, 76
infinitos, 122
salida, 74
tipos, 76, 133-135
buffer, 264
búsqueda, 327
binaria, 327, 332-336
hash, 327, 337-343
lineal, 327
secuencial, 327-332

C, 10, 11, 98, 704
C++, 10, 11, 98
cadena, 15, 237, 240-244
asignación, 244
borrar, 249
búsqueda, 248
cambiar, 249-250
comparación, 240
concatenación, 244, 246-247
conversión a números, 251
desigualdad, 245
entrada/salida, 244
escribir, 244
igualdad, 245
insertar, 249
longitud, 240, 243-245
nula, 240
operaciones, 244
vacía, 240
carácter, 15, 242
case, 112
caso-de, 111
centinela, 123, 133
clave, 262, 267
clasificación, 306, 369
de archivos, 369
Clipper, 10
COBOL, 10, 11, 98, 705
código
ASCII, 238-240
EBCDIC, 240-241
máquina, 7
objeto, 11
cola, 387, 419
aprovechamiento de memoria, 426
doble cola, 428
implementación, 421, 424
representación, 420
comentarios, 484
compilación, 11
compilador, 11, 12
compilar, 12
computadora, 2, 3, 40
programa de, 4
concatenación, 244, 246-247
constante, 16, 73, 242
boolean, 16
lógica, 16
real, 16
contador, 73, 76-79, 134
conversión
de claves, 268
de tipos, 29
cos, 26

crear, 276
cuadrado, 27

datos, 2, 13
  booleanos, 15
  cadena, 15
  carácter, 13, 242
  enteros, 13
  entrada, 2, 69, 71
  enumerados, 13
  lógicos, 13, 15
  numéricos, 13
  reales, 13, 14
  salida, 2, 69
  subrango, 13
  tipos de datos, 13
desde, 128-138
diagrama
  de flujo, 42, 46, 48, 49, 50-58, 68, 105, 109
  N/S, 42, 56, 57, 68, 101, 105, 108
  símbolos, 49
Dijkstra, 99
diseño
  descendente, 41, 99, 165, 481, 484
  top-down, 41
dispositivos, 3
  entrada, 3
  entrada/salida, 3
  E/S, 3
  salida, 3
div, 18-20
«divide y vencerás», 41, 97
doble cola, 428
documentación, 68, 482, 489-494
do-while, 120
driver, 164

EBCDIC, 23, 28, 240-241
editor, 12
efectos laterales, 192-194
end, 58
end of file, 124
enlazador, 11
entrada, 5, 31, 40, 68
eof, 124, 267
equipo
  físico, 3
  lógico, 3
escribir, 31, 59, 70, 113
estructura, 73
  básica, 99
  decisión anidada, 138-142
  decisión múltiple, 111
  repetitiva, 73, 117, 138, 142-144
  secuencial, 73, 100
  selectiva, 73, 104
estructura de datos, 203
  clasificación, 204
  concepto, 204
  dinámica, 204, 385-386
  estática, 204
  lineales, 386-387
  no lineales, 387
explotación, 496
expresión, 17, 73
  aritmética, 18, 19
  booleana, 22
  lógica, 22
  precedencia, 20
  prioridad, 20
  relacional, 22

FA, 267
factor de bloqueo, 263
falso, 15
FDA, 267
FF, 267
fichero. *Véase* archivo, 259
FIFO, 419
fin, 48, 59, 70, 112
flowchart, 46
for, 128
FORTRAN, 10, 98
función, 26, 166-173
  declaración, 166, 167
  externa, 166
  interna, 26, 166
  invocación a, 168-169
fusión, 359, 361-364

GB, 260
GigaByte, 260
glosario de programación, 687
goto, 44, 144
grafo, 204, 466
  camino, 428
  completo, 469
  concepto, 466

lista de adyacencia, 472
matriz de adyacencia, 470
múltiple, 467
ponderado, 469
representación, 470
terminología, 467
vértice, 466

hacer, 111
hardware, 3, 7
hash, 268

identificador, 17, 73, 205
if-then, 104
if-then-else, 105
indentación, 44
información, 2
entrada, 31
salida, 31
inicio, 48, 59, 70, 112
insertar, 243
instrucción, 3, 7, 69, 73
aritmético-lógica, 7
asignación, 70
bifurcación, 70, 72
entrada/salida, 7
escritura, 70
fin, 70
inicio, 70
lectura, 70, 71
repetitiva, 7
salida, 72
selectiva, 7
tipos, 70
intercalación, 343-345
interruptor, 73, 81-83
ir_a, 144
iteración, 117, 118
iterar, 132

juego de caracteres, 238
ASCII, 23, 238, 695-700
EBCDIC, 23, 238, 240-241

KB, 260
K-Byte, 260
KiloByte, 260

leer, 31, 59, 70-71
lenguaje ADA, 10
bibliografía, 703
lenguaje algorítmico UPSAM, 53
archivos, 539
caracteres especiales, 541
elementos, 533-537
estructuras de control, 537
funciones, 541
operadores, 541
palabras reservadas, 541-544
procedimientos, 541
programación modular, 538
variables dinámicas, 541
lenguaje BASIC, 10, 25, 95
bibliografía, 703
lenguaje Borland/Turbo Pascal, 583
bibliografía, 704
constantes, 587
control de dispositivos, 601
directivas de compilación, 600
dispositivos, 585
elementos básicos, 583
funciones
definidas por el usuario, 598
estándar, 593
operadores, 589
Pascal estándar, 603
procedimientos, 593
sentencias, 591
tipos de datos, 585
variables, 587
ámbito, 593
localidad, 593
lenguaje C, 607
apuntadores, 627
archivo de cabecera, 609
bibliografía, 703
cadenas, 625
campos de bits, 627
comentarios, 610
definición de macros, 610
elementos básicos, 607
estructura, 625
de datos, 624
de un programa, 607
expresiones, 614
funciones, 623
de E/S, 616
printf, 616
scanf, 618

operadores, 614
palabras reservadas, 608
punteros, 627
sentencias de control, 618
tipos de datos, 610
uniones, 627
variables, 611
lenguaje C++, 631
apuntadores, 652
arrays, 657
bibliografía, 703, 704
cadenas, 660
cin, 642
constantes, 635
conversión de tipos, 635
cout, 642
declaración de variables, 636
elementos, 631
entradas y salidas, 642
enumeraciones, 658
estructuras, 658
flujos, 642
funciones, 661
operadores, 637
operador new, 656
operador delete, 656
punteros, 652
tipos de datos, 634
uniones, 658
lenguaje COBOL, 10, 11, 98
bibliografía, 705
lenguaje FORTRAN, 10, 98
bibliografía, 705
lenguaje Pascal, 545
bibliografía, 705
elementos, 545
entradas y salidas, 562
estructura de un programa, 548
for, 566
forward, 571
funciones, 567
definidas por el usuario, 567
intrínsecas, 571
lógicas, 573
numéricas, 571
ordinales, 572
transferencia, 572
goto, 565
operadores, 560
procedimientos, 567
definidos por el usuario, 567
intrínsecos, 573
programa fuente, 581
Read, 563
ReadLn, 563
repeat, 566
sentencias, 563
tabla de identificadores, 547
tipos compatibles, 559
tipos de datos, 550
while, 566
Write, 562
WriteLn, 562
lenguaje de programación, 3
alto nivel, 6, 9
bajo nivel, 6, 8
concepto, 6
ensamblador, 6, 8
máquina, 7
traductor de, 10
lenguajes de programación
bibliografía, 702-703
LIFO, 411
linker, 11
LISP, 10
lista, 204, 388
circular, 407-408
cola, 204
doblemente enlazada, 408-410
encadenada. *Véase* enlazada
enlazada, 390-393
procesamiento, 393-407

llamar_a, 173-174, 181

mantisa, 15
matriz, 203, 205
MB, 260
MegaByte, 260
memoria, 7
auxiliar, 3
central, 3
mezcla, 359, 361
directa, 369
natural, 372
secuencias equilibradas, 376
mientras, 45, 120, 131
diferencias con repeat, 126
mod, 18, 19, 20, 25
Modula-2, 2, 10

bibliografía, 705
módulo, 96, 97, 98
tamaño, 97
implementación, 98
vista de un, 99
montador, 11, 12

nemotécnicos, 8
no, 25

O, 25
opción, 111
operador, 18
aritméticos, 18, 19
booleano, 24
exponencial, 20
lógico, 22, 24
relación, 22
operando, 18
ordenación, 306, 307
burbuja, 308-314
externa, 359
inserción, 314-317
intercambio, 308
métodos, 307
ordenación rápida, 322-327
selección, 317-320
Shell, 320
ordenador, 2. *Véase* computadora

palabra reservada, 59, 73
para, 128-130
parámetro, 166
actual, 168, 174, 187
comparación, 185
comunicación con subprograma, 181
declaración, 181
dummy, 168
ficticio, 168
formal, 166, 168, 174-175
función como, 190
mudo, 168
paso de, 181-190
por dirección, 187
por referencia, 184, 187
por valor, 183, 187
procedimiento como, 190
procedimientos, 174
variable, 187
partición, 364
por contenido, 365
por secuencial, 368
Pascal, 2, 10, 11, 25
periférico, 3
pila, 194, 387, 411
aplicaciones, 417
implementación, 413
poner, 412
posición, 249
pop, 412
push, 412
quitar, 412
precedencia, 20
prioridad, 20, 25
procedimiento, 173-177
anidado, 179
comparación con funciones, 173
declaración, 174
parámetros, 174-175
procesador, 2, 3
programa, 3, 68, 73
codificación, 68
diseño, 486
documentación, 68
elementos básicos, 73
ensamblador, 8
escritura, 87
estructurado, 95
fuente, 8, 11, 12
lineal, 69
mantenimiento, 68
no lineal, 69
objeto, 8, 9, 11, 12
partes, 68
puesta a punto, 486
transportable, 9
programación, 3
estilo, 481
estructurada, 96, 99
glosario, 687
modular, 95, 96, 97, 485
proceso, 68
técnicas, 96
teorema, 100
PROLOG, 10
pseudocódigo, 42, 58, 101, 105
puntero, 386, 391

Quiksort, 322-327

raiz2, 26, 27, 30
read, 31, 70
recursión, 194
recursividad, 194
recursos abstractos, 99
redondeo, 26, 27
registro, 204, 262
  físico, 264
  lógico, 264
repeat, 58, 125-127
  diferencias con mientras, 126
repetir, 45, 125-127, 132
resolución de problemas, 40

salida, 5, 40, 68
salir_bucle, 132
segun_sea, 111
si_entonces, 104, 138-141
software, 3
soporte, 265
  direccionable, 265
  secuencial, 265
sort, 173
stack, 194, 411
stepwise, 41
stop, 58
string, 15
subalgoritmo, 163-164
subcadena, 241, 247
subcadena, 247
subprograma, 96, 163-165
  ámbito, 163
  comunicación entre, 181
  concepto, 164-166
  efectos laterales, 163, 192
  funciones, 163
  procedimientos, 163
  recursividad, 163
  subrutinas, 163
  variables globales, 163
  variables locales, 163
Subrutina, 173-177

tabla, 203, 205
tabla de decisión, 497
  ampliadas, 504, 503
  cerradas, 514-516
  comparación, 506-507
  concepto, 498
  construcción, 509-513
  conversión de TD en programas, 516-521
  definición, 48
  encadenamiento, 513
  limitadas, 504
  matriz de acciones, 498
  matriz de condiciones, 498
  mixtas, 504, 505
  propia, 500
  reglas de decisión, 499
  tipos, 507-508
terminología informática
  bibliografía, 706
tipos
  archivo, 204
  cadena, 204
  carácter, 16, 204
  compuestos, 204
  conjunto, 204
  entero, 204
  enumerativo, 204
  estándar, 204
  estáticos, 204
  estructurados, 204
  fichero, 204
  lógico, 204
  real, 204
  simples, 204
  subrango, 204
top-down, 99
traductor, 10
  compilador, 10
  intérprete, 10
transportabilidad, 9
trunc, 26, 27

UCP, 3, 8
Unidad Central de Proceso, 3
until, 58

variable, 17, 73
  cadena, 17
  carácter, 17, 242
  continuas, 386
  declaración, 84-86
  entera, 17

estáticas, 385
global, 178, 186, 192-194
local, 178
lógica, 17
nombre, 17, 483
real, 17
verdadero, 15
Visual BASIC, 10
vector, 203, 205, 208
acceso secuencial, 210
actualizar, 212
asignación, 209
escritura, 210
lectura, 210
operaciones con, 208-215
recorrido, 210

While, 58, 120
Wirth, 2
Write, 70

xor, 543, 589

y, 25